Statistics From the Ground Up

Data, Uncertainty, and the Numbers Behind Every Claim

From Averages to Machine Learning

November 2026

Preface

A service answers five requests in 112, 98, 105, 240, and 101 ms. The mean is 131.2 ms; the median (the middle value once sorted) is 105 ms. Without the 240, the other four have mean 104 ms and median 103 ms. One slow request pulled the mean up by 27 ms and moved the median by only 2 ms (Figure 1).

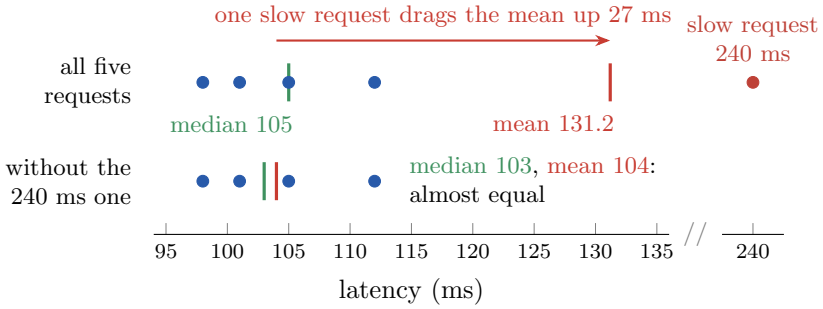


Figure 1: The five latencies from the opening example, one dot per request. Compare the two rows: removing the single 240 ms request moves the red mean line by 27 ms but the green median line by only 2 ms, because the median looks only at the middle value. The axis is broken between 136 and 235 ms, where no request falls, so the four fast dots stay far enough apart to count.

Which number should go on the dashboard? If tomorrow's mean is 125 ms, did the service get faster, or did it just draw a different five requests? How many requests would you need before a 5 ms change is real rather than luck? This book exists to answer questions of that kind, with numbers you can compute and reasons you can check.

Who this book is for

The reader we have in mind is an engineer, analyst, or student who uses statistics every week and wants to stop taking it on faith. You can read code and algebra. You may have computed a standard deviation or run a t-test, but you could not say why the standard deviation divides by $n-1$, or why averaging 400 measurements is only twice as precise as averaging 100.

The level is that of a solid first course: the content of Khan Academy's statistics track and AP Statistics, plus the statistics that machine learning (ML) texts assume you already know: *An Introduction to Statistical Learning* (ISLR), or the opening chapters of Bishop's *Pattern Recognition and Machine Learning* and Goodfellow, Bengio, and Courville's *Deep Learning*. Nothing beyond algebra

and a little calculus (sums, a derivative, the idea of an integral as area) is needed.

What you will be able to do

By the end you should be able to:

- read a latency dashboard and say what p50, p95, and p99 mean (the values that 50%, 95%, and 99% of requests stay under), and when the mean misleads;
- explain why the sample standard deviation (the typical distance of a value from the mean) divides by $n - 1$, and why the standard error of a mean (the typical distance of the mean itself from the truth) is $\sigma/\sqrt{n}$;
- build and interpret confidence intervals for a mean, a proportion, and a median, including a bootstrap interval for any statistic;
- run an A/B test, read its p-value correctly (how often data at least this extreme would turn up if there were no real difference), and choose its sample size in advance;
- recognise multiple-testing, peeking, and confounding before they produce a false claim;
- fit a linear or logistic regression, read every number in its output, and check its residuals;
- read notation such as $\mathbb{E}_{x\sim p}[\ell(\theta; x)]$ in a machine learning paper (the average loss ℓ of parameters θ over data x drawn from a distribution p) and know which statistical idea it encodes.

The road map

The book has sixteen chapters in five parts. Each part uses the tools of the one before it. Figure 2 shows the path.

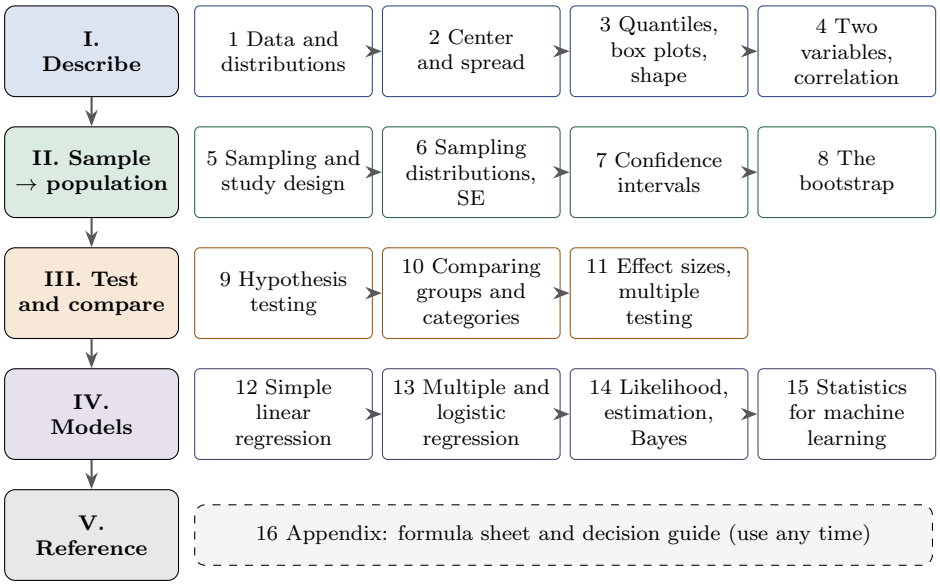


Figure 2: The sixteen chapters in five parts. Read top to bottom: you describe data, then learn how a sample speaks for a population, then use that to test claims, then build models and carry them into machine learning. Chapter 16 is a reference you can consult at any point.

Part I (Chapters 1–4) works with data you already have: histograms, means, medians, standard deviations, quantiles, and correlation. No probability is needed yet.

Part II (Chapters 5–8) asks what a sample says about the population it came from. This is where the standard error and the confidence interval appear, and where the bootstrap (resampling your own data many times to see how much a statistic wobbles) lets a computer do the algebra for you.

Part III (Chapters 9–11) turns intervals into decisions: p-values, t-tests, A/B tests, chi-square tests, effect sizes, and the ways testing goes wrong.

Part IV (Chapters 12–15) fits models. Regression first, then the general principles behind it (likelihood, Bayes), then the statistics inside machine learning: train/test splits, cross-validation, bias and variance, and loss functions as expectations.

Part V (Chapter 16) is a formula sheet and a decision guide for choosing a method. Keep it open while you read.

The five ideas that run the whole book

Almost every chapter is an application of five ideas. Figure 3 draws them.

1. **Variability.** Measurements differ. The standard deviation measures by how much (Chapter 2).
2. **Sampling distribution.** A statistic computed from a sample, such as the sample mean $\bar{x}$, would come out differently on another sample. Its distribution over all possible samples is the object inference is about (Chapter 6).
3. **SE $\propto 1/\sqrt{n}$.** If single values have standard deviation σ, the spread of $\bar{x}$ over repeated samples

of size n is $\sigma/\sqrt{n}$, called the standard error (SE). To halve the uncertainty you need four times the data (Chapter 6).

4. **Uncertainty intervals.** Report an estimate as estimate ± (about 2) × SE. The "95%" is a promise about the procedure: over many repeated samples, about 95% of the intervals built this way contain the true value (Chapters 7 and 8).

5. **Models are signal plus noise.** $y = f(x) + \varepsilon$: the outcome y is a systematic part $f(x)$ plus random noise ε. Regression, tests, and ML all estimate f while measuring how much the noise could have misled them (Chapters 12–15).

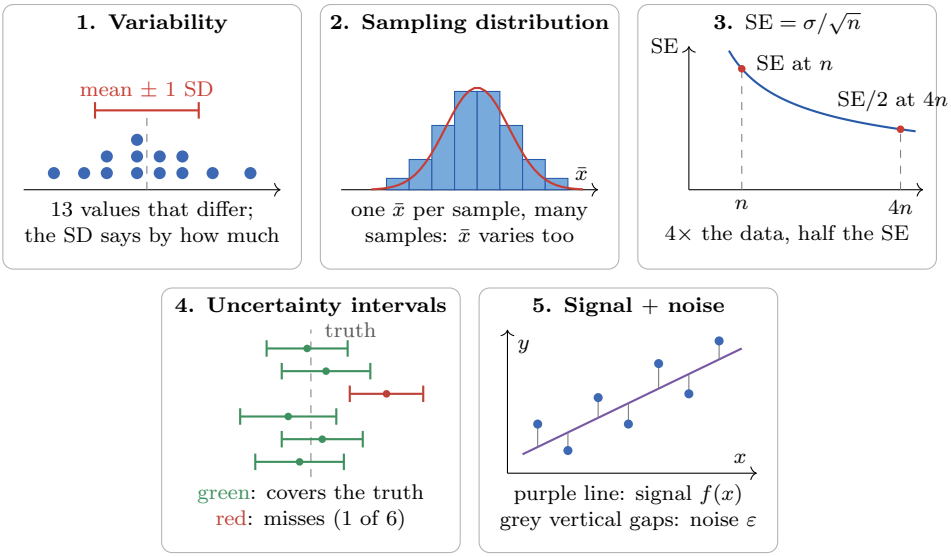


Figure 3: The five ideas. Data vary (1); so does any statistic computed from them, and its distribution over repeated samples is the sampling distribution (2); its spread shrinks like $1/\sqrt{n}$ (3); that spread sets the width of an interval that usually, but not always, covers the truth (4); and a model splits each observation into a systematic part and leftover noise (5).

Idea 3 decides how much data every experiment needs, so here are the numbers. Suppose single request latencies have standard deviation $\sigma = 40$ ms. The standard error of the mean of n requests is $40/\sqrt{n}$:

$$n = 25:\ 8 \text{ ms}, \qquad n = 100:\ 4 \text{ ms}, \qquad n = 400:\ 2 \text{ ms}, \qquad n = 1600:\ 1 \text{ ms}.$$

Each halving of the SE costs four times as many requests.

Pause

Pause and think. With $\sigma = 40$ ms, how many requests do you need for an SE of 0.5 ms? (Solve $40/\sqrt{n} = 0.5$: $\sqrt{n} = 80$, so $n = 6400$. Four times the 1 ms answer, as the rule predicts.)

Figure 4 plots the curve. A 5 ms shift in the mean is invisible against an SE of 8 ms but stands out clearly against an SE of 1 ms.

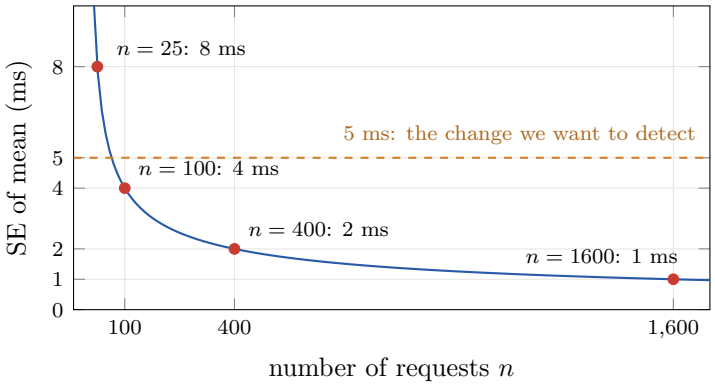


Figure 4: Standard error of a mean latency, $40/\sqrt{n}$, when $\sigma = 40$ ms. Where the blue curve is above the dashed line (fewer than 64 requests), the mean's own noise is bigger than the 5 ms change we hope to see; at 400 requests the SE is 2 ms, so a 5 ms change is 2.5 SEs. The curve flattens: going from 400 to 1600 requests buys only 1 ms more precision.

Relation to *Probability From the Ground Up*

This book has a sibling, *Probability From the Ground Up*. Probability starts from a known model and asks what data it will produce. Statistics runs the other way: it starts from data and asks what model produced it.

You do not need the probability book to read this one. What you need is a working sense of three facts about a random quantity X, and each is re-explained at the point where it is first used:

- the expectation $\mathbb{E}[X]$ is the long-run average value of X;
- the variance $\mathrm{Var}(X) = \mathbb{E}\left[(X - \mathbb{E}[X])^2\right]$ measures spread, and $\mathrm{SD}(X) = \sqrt{\mathrm{Var}(X)}$;
- for constants a, b, $\mathbb{E}[aX + b] = a\,\mathbb{E}[X] + b$ and $\mathrm{Var}(aX + b) = a^2\,\mathrm{Var}(X)$; for independent X, Y, $\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$.

The variance rules in the last item give $\mathrm{SE} = \sigma/\sqrt{n}$: the sum of n independent values has variance $n\sigma^2$, and dividing by n scales that by $1/n^2$, leaving σ^2/n (Chapter 6). When a proof needs more machinery, such as the Central Limit Theorem or maximum likelihood theory, we state the result, show it numerically, and point to the relevant chapter of the probability book (Chapters 8–9 for expectation and variance, 20–21 for the law of large numbers and the CLT, 30–32 for estimation and classical inference).

How the book is written

Each idea starts with a small dataset worked by hand, then the general formula, then the reason the formula is what it is. Figures carry much of the argument; read them as carefully as the text. Coloured boxes mark specific voices: *Intuition* for the picture, *Why it works* for the derivation, *Common trap* for the mistakes people actually make, *Connection* for links to machine learning, and *Pause* for a quick check to try before reading on. Short Python snippets (`numpy`, `scipy`) show how to do each computation in practice. Chapters 1–15 each end with a summary and exercises with

answers.

Do the exercises. Statistics is easy to nod along to and hard to apply, and the gap closes only by computing a few intervals and tests yourself.

Contents

Preface iii

I Describing Data 1

1 Data and Distributions 3

1.1 What statistics is for . . . 3

1.2 Units, variables, and types . . . 4

1.2.1 The shape of a data set . . . 4

1.2.2 Types of variables . . . 4

1.2.3 Populations and samples . . . 5

1.3 Categorical data: counts and bar charts . . . 6

1.3.1 Frequency tables . . . 6

1.3.2 Bar charts . . . 6

1.4 Numerical data: seeing every value . . . 7

1.4.1 Dot plots . . . 8

1.4.2 Stem-and-leaf plots . . . 8

1.5 Grouping numerical data: histograms . . . 9

1.5.1 From frequency table to histogram . . . 9

1.5.2 Discrete numerical data: coin flips . . . 10

1.5.3 Larger data: 200 latencies . . . 11

1.6 Bin width changes the picture . . . 11

1.7 Density histograms: area is proportion . . . 13

1.7.1 The problem with unequal bins . . . 13

1.8 Shapes of distributions . . . 15

1.8.1 A left-skewed example: exam scores . . . 16

1.8.2 A bimodal example: cache hits and misses . . . 17
1.9 Outliers . . . 18
1.10 The distribution is the object of study . . . 18
1.10.1 How to read any distribution . . . 19
1.11 Summary . . . 20
1.12 Exercises . . . 21

2 Center and Spread **23**
2.1 The mean . . . 23
2.1.1 The mean is the balance point . . . 23
2.2 The median, the mode, and the trimmed mean . . . 24
2.3 Mean versus median: skew and outliers . . . 25
2.3.1 Skew moves the mean toward the tail . . . 26
2.3.2 Robustness and the breakdown point . . . 27
2.4 Deviations, variance, and standard deviation . . . 28
2.4.1 A worked table . . . 28
2.5 Why divide by $n-1$? . . . 30
2.5.1 The sample mean hugs its own data . . . 30
2.5.2 Every possible sample, listed . . . 31
2.5.3 The derivation . . . 32
2.5.4 Simulation . . . 33
2.6 The key link: which center minimises which error? . . . 35
2.7 Other measures of spread: range, IQR, MAD . . . 36
2.8 Standardising: z-scores . . . 37
2.9 The 68–95–99.7 rule . . . 39
2.10 Chebyshev's inequality: the guarantee for any data . . . 40
2.11 Linear transformations of data . . . 41
2.12 The mean of grouped data . . . 42
2.13 Choosing a summary . . . 43
2.14 Summary . . . 44
2.15 Exercises . . . 44

3 Quantiles, Box Plots and Shape **47**

3.1 Percentiles and quantiles . . . 47
3.1.1 The trouble with “roughly” . . . 48
3.1.2 Why numpy has nine methods . . . 49
3.2 The five-number summary and the box plot . . . 50
3.2.1 Drawing the box plot . . . 50
3.2.2 Comparing groups side by side . . . 52
3.3 The empirical CDF . . . 53
3.4 The quantile function: the CDF read backwards . . . 54
3.5 Latency dashboards: p50, p90, p99 . . . 56
3.5.1 Why averages lie about tails . . . 56
3.5.2 Percentiles do not average . . . 58
3.6 Shape: skewness and kurtosis . . . 58
3.6.1 Kurtosis: how heavy are the tails? . . . 59
3.7 Taming right skew with a log transform . . . 60
3.8 QQ plots: comparing a sample with a model . . . 61
3.8.1 Building one by hand . . . 62
3.8.2 Reading the patterns . . . 63
3.9 Smooth density estimates . . . 64
3.9.1 Building a KDE from bumps . . . 64
3.9.2 Bandwidth is the whole game . . . 65
3.9.3 Violin plots . . . 66
3.10 Choosing a picture . . . 67
3.11 Summary . . . 67
3.12 Exercises . . . 68

4 Two Variables: Tables, Scatter Plots, and Correlation 71

4.1 Two categorical variables: contingency tables . . . 71
4.2 Two numerical variables: scatter plots . . . 73
4.3 Covariance: the quadrant picture . . . 74
4.4 Pearson correlation . . . 76
4.4.1 Properties of r . . . 77
4.5 Always plot: Anscombe’s quartet . . . 78
4.6 When r misses the relationship . . . 80

4.6.1 A perfect relationship with $r = 0$ . . . 80
4.6.2 Spearman rank correlation . . . 80
4.7 Correlation is not causation . . . 81
4.8 Simpson's paradox . . . 83
4.9 Ecological fallacy . . . 85
4.10 Restriction of range . . . 86
4.11 Preview: the regression line . . . 87
4.12 Summary . . . 88
4.13 Exercises . . . 89

II From Sample to Population **91**

5 Sampling and Study Design **93**
5.1 Populations, parameters, and statistics . . . 93
5.2 Simple random sampling . . . 94
5.2.1 Why randomness beats judgement . . . 95
5.3 Other sampling designs . . . 96
5.3.1 Stratified sampling . . . 96
5.3.2 Cluster sampling . . . 98
5.3.3 Systematic sampling . . . 99
5.3.4 Convenience sampling . . . 99
5.4 Bias: errors that do not average out . . . 100
5.4.1 Selection bias . . . 100
5.4.2 Nonresponse bias . . . 100
5.4.3 Survivorship bias . . . 101
5.4.4 Response bias . . . 102
5.5 Size does not cure bias: the Literary Digest poll . . . 102
5.6 Observational studies and experiments . . . 104
5.6.1 Confounding . . . 104
5.6.2 Randomised experiments . . . 106
5.6.3 Blocking . . . 107
5.6.4 Matched pairs . . . 107

5.6.5 Blinding and placebos . . . 108
5.7 A/B tests are randomised experiments . . . 108
5.8 Generalisation versus causation . . . 109
5.9 Selection in tech logs and data leakage . . . 110
5.9.1 Survivorship in latency logs . . . 110
5.9.2 Other selection traps in product data . . . 111
5.9.3 Data leakage . . . 111
5.10 Summary . . . 112
5.11 Exercises . . . 113

6 Sampling Distributions and Standard Error 115
6.1 Three tools: expectation and variance rules . . . 115
6.2 A statistic is a random variable . . . 116
6.2.1 A sampling distribution you can list by hand . . . 117
6.3 The sampling distribution of the mean, by simulation . . . 118
6.4 Deriving the centre and the spread . . . 119
6.4.1 Why $\sqrt{n}$, and what it costs you . . . 120
6.5 Standard deviation versus standard error . . . 121
6.6 The sampling distribution of a proportion . . . 122
6.7 The shape: Central Limit Theorem and Law of Large Numbers . . . 123
6.7.1 When is n "large enough"? . . . 124
6.7.2 The Law of Large Numbers . . . 125
6.8 Estimated standard error . . . 125
6.9 Standard error of a difference . . . 126
6.10 Statistics beyond the mean: the median . . . 127
6.10.1 Most statistics have no tidy formula . . . 129
6.11 Finite population correction . . . 130
6.12 Summary . . . 131
6.13 Exercises . . . 131

7 Confidence Intervals 133
7.1 From standard error to interval . . . 133
7.2 The z-interval from a pivot . . . 134

7.3 What "95% confidence" means . . . 135
7.3.1 Confidence level versus width . . . 137
7.4 When σ is unknown: the t-distribution . . . 138
7.4.1 Worked example . . . 140
7.5 Interval for a proportion . . . 141
7.5.1 Why Wald fails near 0 and 1 . . . 141
7.5.2 The Wilson interval . . . 142
7.6 Margin of error and sample-size planning . . . 144
7.6.1 What "±3%" in a poll means . . . 144
7.7 Comparing two groups . . . 145
7.7.1 Difference in means (Welch) . . . 146
7.7.2 Difference in proportions . . . 147
7.7.3 Paired data . . . 147
7.8 One-sided bounds . . . 149
7.9 Error bars: SD, SE, or CI? . . . 149
7.9.1 The overlapping-intervals fallacy . . . 150
7.10 Choosing an interval . . . 152
7.11 Summary . . . 152
7.12 Exercises . . . 153

8 The Bootstrap 155

8.1 The problem: statistics with no formula . . . 155
8.2 The plug-in principle . . . 156
8.3 Resampling with replacement, by hand . . . 158
8.4 The bootstrap distribution . . . 159
8.5 Bootstrap standard error . . . 161
8.6 The percentile interval . . . 162
8.7 The basic interval . . . 162
8.8 Better intervals: bootstrap-t and BCa . . . 163
8.8.1 Bootstrap-t . . . 163
8.8.2 BCa: bias-corrected and accelerated . . . 164
8.9 Worked examples: median and p90 latency . . . 164
8.10 Difference in medians between two groups . . . 165

8.11 An exact interval for the median from order statistics . . . 166
8.11.1 Any quantile . . . 169
8.12 How many resamples? Monte Carlo error . . . 169
8.13 How well do the intervals cover? . . . 170
8.14 When the bootstrap fails . . . 171
8.14.1 Tiny samples . . . 172
8.14.2 Extremes: the maximum . . . 172
8.14.3 Heavy tails . . . 172
8.14.4 Dependent data and the block bootstrap . . . 173
8.15 Bootstrap vs permutation: a preview . . . 174
8.16 Choosing a method . . . 175
8.17 Summary . . . 175
8.18 Exercises . . . 176

III Testing and Comparing 179

9 Hypothesis Testing 181
9.1 The skeptic's game . . . 181
9.1.1 Drawing a line in advance . . . 183
9.2 Absence of evidence is not evidence of absence . . . 185
9.3 A permutation test: the null world by shuffling . . . 186
9.4 The vocabulary, precisely . . . 188
9.4.1 One-sided and two-sided . . . 188
9.5 The z-test . . . 189
9.6 The one-sample t-test . . . 190
9.7 Two samples: pooled and Welch t-tests . . . 191
9.8 Paired data . . . 193
9.9 Errors, significance and power . . . 194
9.9.1 The general picture . . . 195
9.9.2 Sample size for an A/B test . . . 197
9.10 Tests and confidence intervals are two views of one thing . . . 198
9.11 What a p-value is not . . . 199

9.12 Assumptions and robustness of the t-test . . . 201
9.13 Summary . . . 204
9.14 Exercises . . . 204

10 Comparing Groups and Categorical Data 207

10.1 Two proportions: the A/B conversion test . . . 207
10.1.1 A worked example . . . 207
10.1.2 Pooled for the test, unpooled for the interval . . . 208
10.2 Chi-square goodness of fit . . . 210
10.2.1 Is this die fair? . . . 210
10.2.2 The chi-square family . . . 211
10.3 Chi-square test of independence . . . 212
10.3.1 Does conversion depend on device? . . . 212
10.4 Fisher's exact test . . . 214
10.5 One-way ANOVA: comparing several means . . . 215
10.5.1 Three server configurations . . . 215
10.5.2 Splitting the variation . . . 215
10.5.3 The F statistic . . . 216
10.5.4 The F distribution . . . 217
10.5.5 ANOVA with two groups is the t-test . . . 217
10.5.6 Post-hoc comparisons . . . 218
10.6 Nonparametric alternatives: tests on ranks . . . 219
10.6.1 Mann–Whitney / Wilcoxon rank-sum test . . . 219
10.6.2 Wilcoxon signed-rank test . . . 220
10.6.3 The sign test . . . 221
10.7 A/B testing in practice . . . 222
10.7.1 The randomisation unit . . . 222
10.7.2 Ratio metrics and the delta method . . . 223
10.7.3 Novelty and primacy effects . . . 223
10.7.4 Sample ratio mismatch . . . 224
10.7.5 CUPED: variance reduction with pre-experiment data . . . 224
10.8 Which test when . . . 226
10.9 Summary . . . 227

10.10Exercises . . . 227

11 Effect Sizes, Multiple Testing and Pitfalls 229

11.1 Statistical versus practical significance . . . 229
11.2 Effect sizes: saying how big . . . 231
11.2.1 Difference in means, with its interval . . . 231
11.2.2 Cohen's d: difference in units of spread . . . 232
11.2.3 Proportions: difference, lift, risk ratio, odds ratio . . . 233
11.3 Report intervals, not just p-values . . . 235
11.4 Multiple comparisons . . . 235
11.4.1 The jelly-bean problem . . . 235
11.4.2 Bonferroni: divide alpha by m . . . 236
11.4.3 Holm: a free upgrade . . . 237
11.4.4 Benjamini–Hochberg: controlling the false discovery rate . . . 237
11.5 p-hacking and the garden of forking paths . . . 240
11.5.1 Optional stopping: the peeking problem . . . 241
11.5.2 Sequential testing: peeking done right . . . 243
11.6 Publication bias and the winner's curse . . . 244
11.7 Regression to the mean . . . 245
11.8 Base rates: how many significant findings are true? . . . 247
11.9 Simpson's paradox, revisited . . . 248
11.10Survivorship bias . . . 249
11.11A checklist for reading a statistical claim . . . 250
11.12Summary . . . 252
11.13Exercises . . . 252

IV Models 255

12 Simple Linear Regression 257

12.1 The model . . . 257
12.2 Residuals and the least squares criterion . . . 258
12.3 Deriving the least squares line . . . 260
12.4 A worked example by hand . . . 262

12.5 Interpreting slope and intercept . . . 263
12.6 How good is the fit? R^2 . . . 264
12.7 Regression to the mean . . . 265
12.8 Residual plots and diagnostics . . . 268
12.9 Inference for the slope . . . 270
12.9.1 Estimating σ and the standard error . . . 272
12.10 Predicting: mean response vs. a new observation . . . 273
12.11 The assumptions: LINE . . . 275
12.12 Transformations and extrapolation . . . 275
12.13 Least squares is maximum likelihood . . . 277
12.14 Summary . . . 278
12.15 Exercises . . . 278

13 Multiple and Logistic Regression **281**
13.1 From a line to a plane . . . 281
13.2 The matrix form . . . 282
13.2.1 Stacking the equations . . . 282
13.2.2 The normal equations . . . 283
13.3 The geometric picture: least squares is a projection . . . 284
13.3.1 The hat matrix . . . 285
13.4 What a coefficient means: "holding the others fixed" . . . 286
13.5 Building richer models with the same machinery . . . 288
13.5.1 Categorical predictors and dummy variables . . . 288
13.5.2 Interactions . . . 289
13.5.3 Polynomial features and overfitting . . . 290
13.6 Multicollinearity . . . 292
13.7 Adjusted R^2 . . . 293
13.8 Uncertainty in the coefficients . . . 294
13.9 Reading a regression output table . . . 294
13.10 Logistic regression: predicting yes or no . . . 296
13.10.1 Why a straight line fails . . . 296
13.10.2 Odds, log-odds and the sigmoid . . . 297
13.10.3 Interpreting coefficients as odds ratios . . . 298

13.10.4 Fitting by maximum likelihood . . . 299
13.10.5 From probabilities to decisions . . . 301
13.11 Generalised linear models: the umbrella . . . 301
13.12 Summary . . . 302
13.13 Exercises . . . 303

14 Likelihood, Estimation and Bayes **305**

14.1 Estimators and how they miss . . . 305
14.1.1 An estimator is a recipe . . . 305
14.1.2 Bias, variance and mean squared error . . . 306
14.1.3 Unbiased is not the same as best . . . 307
14.1.4 Consistency . . . 308
14.1.5 Method of moments, quickly . . . 309
14.2 Likelihood . . . 309
14.2.1 Turning the probability around . . . 309
14.2.2 Logs make it easy . . . 310
14.2.3 MLE for a Bernoulli proportion . . . 310
14.2.4 MLE for a Normal: why it divides by n . . . 311
14.2.5 MLE for a Poisson rate . . . 311
14.3 Every loss function is a likelihood . . . 312
14.3.1 Least squares is Gaussian MLE . . . 312
14.3.2 Cross-entropy is Bernoulli MLE . . . 313
14.3.3 Absolute error is Laplace MLE . . . 313
14.3.4 The dictionary . . . 314
14.4 How sure is the MLE? . . . 315
14.4.1 Curvature is information . . . 315
14.4.2 Likelihood ratios . . . 316
14.5 The Bayesian view . . . 317
14.5.1 From likelihood to a distribution over θ . . . 317
14.5.2 Beta–Binomial: the worked case . . . 318
14.5.3 How much does the prior matter? . . . 319
14.5.4 Credible intervals versus confidence intervals . . . 320
14.6 MAP estimation is regularisation . . . 321

14.6.1 The posterior mode as a penalised MLE . . . 321
14.6.2 Gaussian prior gives ridge; Laplace prior gives lasso . . . 321
14.7 When frequentist versus Bayesian matters in practice . . . 323
14.8 Summary . . . 324
14.9 Exercises . . . 325

15 Statistics for Machine Learning **327**
15.1 A notation decoder . . . 327
15.1.1 True distribution versus empirical distribution . . . 328
15.2 Expected risk, empirical risk, and the generalisation gap . . . 329
15.2.1 A tiny regression, by hand . . . 329
15.2.2 Empirical risk minimisation . . . 330
15.3 Honest error estimates: splits and cross-validation . . . 331
15.3.1 Train, validation, test . . . 331
15.3.2 k-fold cross-validation . . . 332
15.3.3 Data leakage . . . 333
15.4 The bias–variance tradeoff for models . . . 333
15.4.1 Many fits, one picture . . . 334
15.5 Regularisation: ridge and lasso . . . 335
15.5.1 Shrinkage by hand . . . 336
15.5.2 Why lasso gives exact zeros: the geometry . . . 336
15.6 Evaluating classifiers . . . 337
15.6.1 The confusion matrix . . . 337
15.6.2 Scores, thresholds, and the ROC curve . . . 338
15.6.3 Precision–recall curves and base rates . . . 341
15.6.4 Calibration . . . 342
15.7 Uncertainty in ML metrics . . . 343
15.7.1 A confidence interval for accuracy . . . 343
15.7.2 A bootstrap interval for AUC . . . 344
15.7.3 Comparing two models on the same test set . . . 345
15.8 Stochastic gradients are Monte Carlo estimates . . . 346
15.8.1 By hand: four points . . . 346
15.9 Covariance matrices, the multivariate Normal, and PCA . . . 347

15.9.1 The covariance matrix . . . 348
15.9.2 The multivariate Normal . . . 348
15.9.3 PCA: the directions of greatest spread . . . 348
15.10 What you can now read . . . 349
15.11 Summary . . . 350
15.12 Exercises . . . 351

V Reference **353**

16 Appendix: Formula Sheet and Decision Guide **355**
16.1 Notation . . . 355
16.2 Descriptive statistics . . . 356
16.2.1 Two variables . . . 357
16.3 Expectation and variance rules . . . 358
16.3.1 Common distributions . . . 358
16.4 Standard errors . . . 359
16.5 Confidence interval recipes . . . 360
16.5.1 Order-statistic CI for the median . . . 361
16.6 Choosing a test . . . 362
16.6.1 Test statistics at a glance . . . 363
16.7 Critical values . . . 363
16.7.1 Standard normal . . . 363
16.7.2 Student's t (two-sided) . . . 364
16.7.3 Chi-square (upper tail) . . . 365
16.8 Sample size and power . . . 365
16.9 Python one-liners . . . 367
16.10 Glossary . . . 367

Part I

Describing Data

Chapter 1

Data and Distributions

Here are twenty response times, in milliseconds, from one API endpoint over one minute:

12	14	15	15	17	18	18	19	21	22
23	23	24	26	28	31	35	42	58	140

Is this endpoint fast? Is 140 a problem or just bad luck? Would tomorrow look the same? Staring at the list does not answer any of these. Twenty numbers are already too many to hold in your head, and production systems log millions per hour. This chapter is about the first move every statistician makes: turn a pile of numbers into a *picture of how the values are spread out*. That picture is called a *distribution*, and nearly everything in the rest of the book is a question about one.

1.1 What statistics is for

Statistics does three jobs. Each one asks something different of the same data.

- **Describe.** Summarize the data you have. "Of these 20 requests, 15 finished in under 30 ms." No guessing is involved; it is bookkeeping done well.
- **Infer.** Use the data you have to say something about data you do not have. "Across all of today's traffic, about 75% of requests finish in under 30 ms, give or take a few points." Now there is uncertainty, and we must quantify it.
- **Predict.** Say what the next observation will look like. "The next request will probably take between 12 and 60 ms." This is the job a machine learning model does.

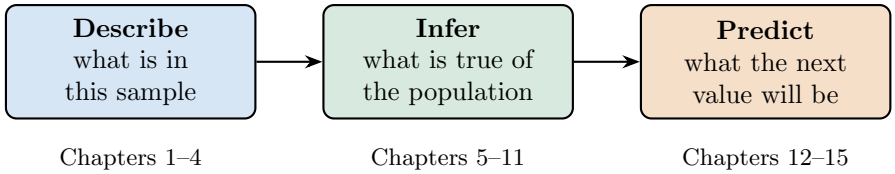


Figure 1.1: The three jobs of statistics. Each builds on the one before: you cannot infer anything about a population until you can describe a sample, and prediction is inference about a single future value.

This chapter and the next three are about describing. Description sounds like the easy part, but many mistakes in practice happen here: the wrong chart, the wrong summary, a misleading bin width, an outlier nobody looked at.

1.2 Units, variables, and types

1.2.1 The shape of a data set

A data set is a table. Each row is one **observational unit**: the thing you measured. Each column is a **variable**: one property you recorded about every unit. Here is a slice of a request log.

request_id	endpoint	status	latency_ms	retries	priority
1	/search	200	21.4	0	high
2	/search	200	18.9	0	medium
3	/cart	404	12.1	0	low
4	/checkout	200	57.6	1	high
5	/search	503	140.2	3	medium

Table 1.1: Five rows of a request log. Each row is one request (the unit). Each column is a variable.

Picking the unit is a real decision. If each row were a *user* instead of a request, the latency column would have to become something like "average latency of that user's requests," and the questions you could answer would change. Before any analysis, say out loud: one row is one _____.

1.2.2 Types of variables

The type of a variable decides which summaries and charts make sense. There are two big families.

Definition 1.1 (Categorical and numerical variables)**.** A **categorical** variable places each unit into one of a set of groups (`endpoint`, `status`). A **numerical** variable records a quantity, a number for which arithmetic means something (`latency_ms`, `retries`).

The test is simple: does averaging make sense? The average latency of requests 1 and 2 is 20.15 ms, a meaningful number. The "average status code" of a 200 and a 404 is 302, which is a real HTTP code (a redirect) that neither request returned. Status codes are labels written with digits. They are categorical.

Each family splits again.

- A categorical variable is **nominal** if its groups have no natural order (`endpoint`: `/cart` is not "more" than `/search`), and **ordinal** if they do (`priority`: low $<$ medium $<$ high). Ordinal means you can rank the groups but the gaps between them have no fixed size: "high minus medium" is not a number.
- A numerical variable is **discrete** if it takes separated values you can list, usually counts (`retries` is 0, 1, 2, ...; never 1.5), and **continuous** if it can take any value in an interval (`latency_ms` could be 21.4 or 21.41 or 21.4137, limited only by the clock's precision).

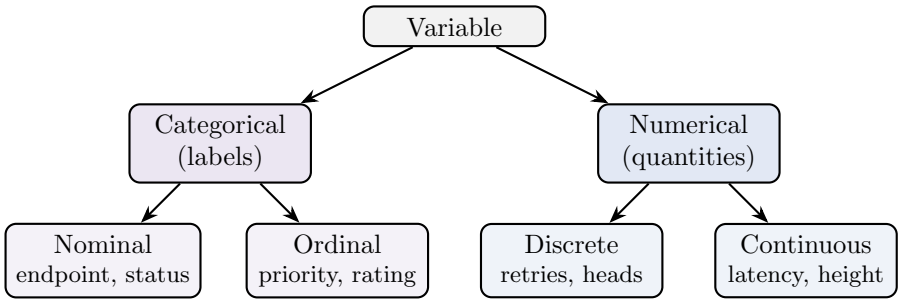


Figure 1.2: The four variable types. The first split asks "does arithmetic make sense?" The second asks "is there an order?" (categorical) or "can I list the possible values?" (numerical).

Common trap

Numbers in a column do not make a variable numerical. Zip codes, user IDs, status codes, and product SKUs are categorical. A plotting library will happily draw a histogram of zip codes; the library did what it was told, and the mistake is yours. Going the other way, a 1–5 star rating is ordinal: treating it as numerical (averaging stars) is common and often useful, but it assumes the step from 1 to 2 stars equals the step from 4 to 5, which nobody checked.

Pause

Classify each: (a) number of items in a shopping cart; (b) browser name; (c) time to first byte; (d) T-shirt size S/M/L/XL. *Answers:* discrete, nominal, continuous, ordinal.

1.2.3 Populations and samples

The twenty latencies above are not all the requests this endpoint will ever serve. They are a **sample**: the units we actually observed. The **population** is the full set of units we care about, such as every request this endpoint served today. A number that describes the population (the true fraction of today's requests under 30 ms) is a **parameter**. The same number computed from the sample is a **statistic**. Parameters are usually unknown; statistics are what we can compute.

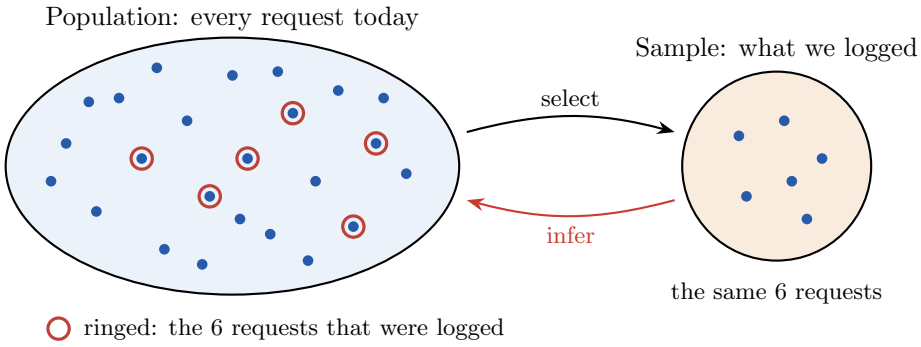


Figure 1.3: A sample is a subset of the population: the six ringed requests are the ones we logged, and they are all we get to see. Describing them is arithmetic. Going back along the red arrow, from those six to every request, is inference, and it is only as good as the way the six were selected.

How samples are drawn and why random selection matters is the subject of Chapter 5. For now, keep the vocabulary straight: in this chapter and the next three we only describe the data in hand.

1.3 Categorical data: counts and bar charts

1.3.1 Frequency tables

Fifty requests come back with these status codes: 41 returned 200, 4 returned 304, 3 returned 404, and 2 returned 500. A **frequency table** lists each category with its count (its *frequency*). The **relative frequency** of a category is its count divided by the total: the fraction of units that fell in that group.

status	frequency	relative frequency
200 OK	41	$41/50 = 0.82$
304 Not Modified	4	$4/50 = 0.08$
404 Not Found	3	$3/50 = 0.06$
500 Server Error	2	$2/50 = 0.04$
total	50	1.00

Table 1.2: Frequency table for 50 status codes. Relative frequencies always sum to 1.

Relative frequencies are what let you compare data sets of different sizes. “2 errors” means nothing until you know whether it was out of 50 requests or 50 million. “4% errors” is comparable across a quiet minute and a busy hour.

1.3.2 Bar charts

A **bar chart** draws one bar per category, with height equal to the count or the relative frequency. The bars are separated by gaps, because the categories are separate things; there is nothing “between” 304 and 404.

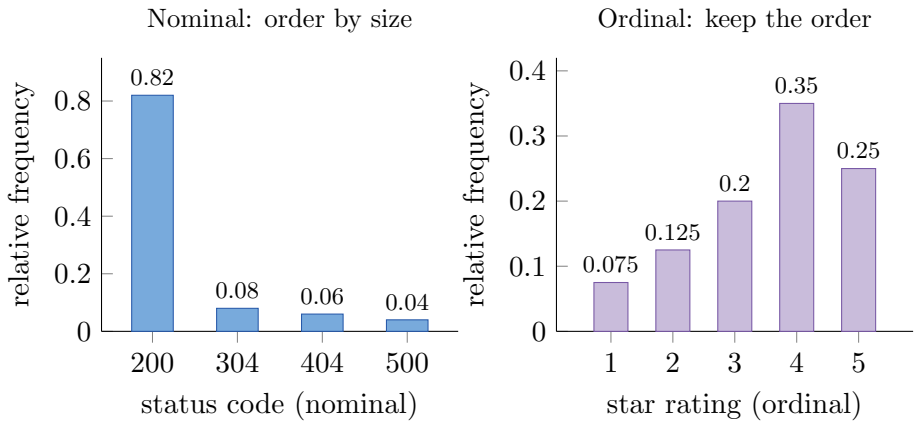


Figure 1.4: Bar charts for categorical data. Left: 50 status codes; with no natural order, sort bars by height so the big category is obvious. Right: 40 star ratings (counts 3, 5, 8, 14, 10); the order 1 to 5 carries information, so never re-sort an ordinal axis.

Common trap

A bar chart whose y-axis does not start at zero exaggerates differences. Bar *length* is what the eye compares. If the axis starts at 0.75, a 0.82 bar is drawn $0.07/0.03 \approx 2.3$ times as long as a 0.78 bar, though the two values differ by only 5% (Figure 1.5). For bar charts, start at zero.

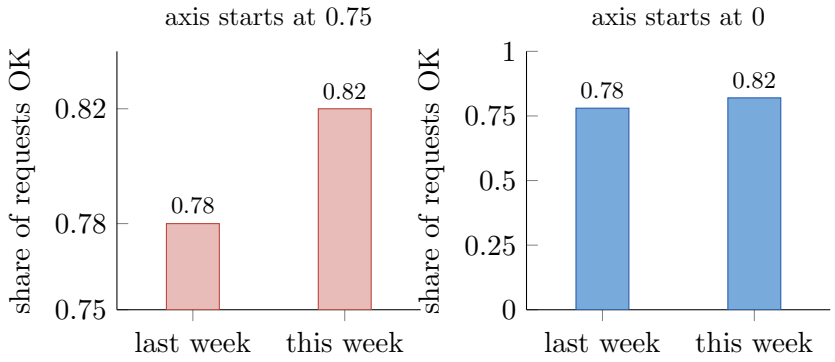


Figure 1.5: The same two numbers, 0.78 and 0.82, drawn twice. Left: with the axis cut at 0.75, this week's bar is 2.3 times as long as last week's and looks like a dramatic jump. Right: from zero, the bars are almost equal, which is the truth (a 5% change).

1.4 Numerical data: seeing every value

Return to the twenty latencies. With a small data set we can draw every value.

1.4.1 Dot plots

A **dot plot** puts a number line under the data and stacks one dot per observation above its value. Nothing is hidden or grouped.

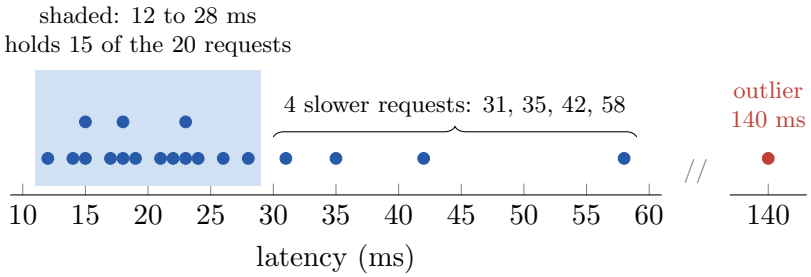


Figure 1.6: Dot plot of the 20 latencies: one dot per request, stacked when two requests have the same value (15, 18 and 23 ms each occur twice). The shaded band holds 15 of the 20 requests (12 to 28 ms), four more trail off to 58 ms, and one request sits alone at 140 ms. The axis is broken between 61 and 135 ms because no request falls there; drawn to scale, that empty stretch would squeeze the other 19 dots into an unreadable clump.

Three features are visible at once that were invisible in the list: where the bulk of the data sits (around 20 ms), that the values trail off further to the right than to the left, and that 140 is separated from everything else by a wide gap.

1.4.2 Stem-and-leaf plots

A **stem-and-leaf plot** is a dot plot made of digits. Split each number into a *stem* (all digits but the last) and a *leaf* (the last digit). For 23, the stem is 2 and the leaf is 3. Write each stem once, then list the leaves of all values with that stem.

```
 1 | 2 4 5 5 7 8 8 9
 2 | 1 2 3 3 4 6 8
 3 | 1 5
 4 | 2
 5 | 8
 6 |
 7 |
 8 |
 9 |
10 |
11 |
12 |
13 |
14 | 0
```

Key: `2 | 3` means 23 ms.

Turn the page sideways and the leaves form a histogram with bins of width 10. Unlike a histogram, you can read back every original value.

The run of empty stems from 6 to 13 is the gap before the outlier.

Figure 1.7: Stem-and-leaf plot of the 20 latencies. It keeps every digit and shows the shape at the same time, which makes it a good hand tool for small data sets.

Stem-and-leaf plots stop being practical beyond about a hundred values. Their real use is reading small data by hand, and they teach the move that the histogram makes: group nearby values together and count.

1.5 Grouping numerical data: histograms

1.5.1 From frequency table to histogram

For a numerical variable, a frequency table needs **bins**: intervals that split the number line into adjacent, non-overlapping pieces. Use bins of width 10 ms, closed on the left and open on the right, so $[10, 20)$ contains 10 but not 20. Count the latencies in each bin.

bin (ms)	values	frequency	relative frequency
$[10, 20)$	12 14 15 15 17 18 18 19	8	0.40
$[20, 30)$	21 22 23 23 24 26 28	7	0.35
$[30, 40)$	31 35	2	0.10
$[40, 50)$	42	1	0.05
$[50, 60)$	58	1	0.05
$[60, 140)$	—	0	0.00
$[140, 150)$	140	1	0.05
total		20	1.00

Table 1.3: Binned frequency table for the 20 latencies. The eight bins from 60 to 140 are empty and have been collapsed into one row.

A **histogram** draws this table. Each bin becomes a bar sitting on the number line, as wide as the bin, as tall as the count. Adjacent bars touch, because the bins tile the number line with no space between them.

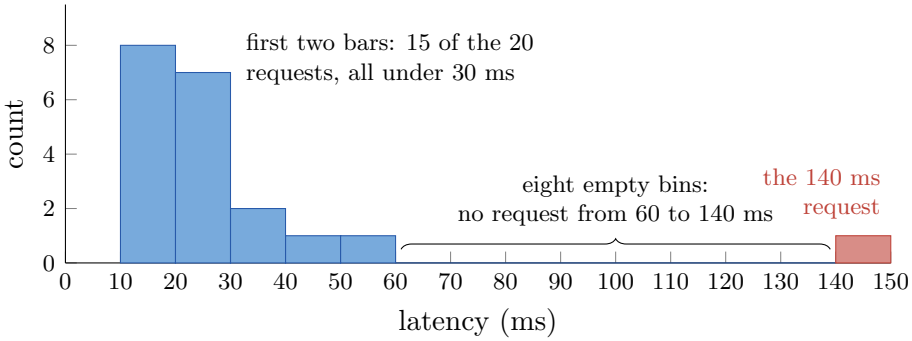


Figure 1.8: Histogram of the 20 latencies with bins of width 10: each bar counts the requests in its 10 ms bin. The bars touch because latency is continuous. Unlike the dot plot, this axis is not broken: here the run of eight empty bins is itself the information, showing how far the 140 ms request sits from the rest.

Intuition

Bar chart or histogram? Ask what the horizontal axis is. If it is a list of labels, you can shuffle them, and gaps between bars say "these are separate things": bar chart. If it is a number line, position matters and there are no gaps in the number line, so there are none between bars: histogram.

The same tables in Python, for the status codes and the 20 latencies:

```
from collections import Counter
import numpy as np
codes = [200]*41 + [304]*4 + [404]*3 + [500]*2
print(Counter(codes).most_common())      # categorical: counts
lat = np.array([12,14,15,15,17,18,18,19,21,22,23,23,24,26,28,31,35,42,58,140])
counts, edges = np.histogram(lat, bins=np.arange(10, 151, 10))
print(counts)                            # [8 7 2 1 1 0 0 0 0 0 0 0 0 1]
print(counts / len(lat))                 # relative frequencies
```

1.5.2 Discrete numerical data: coin flips

Discrete numerical data sits between bar charts and histograms: the axis is a number line, but only whole numbers can occur. Flip a coin 20 times:

H T H H T T H T H H H T T H T H H T H T

The outcome of one flip is categorical (heads or tails); its frequency table is 11 heads, 9 tails, relative frequencies 0.55 and 0.45. Now change the unit. Let one unit be *a batch of 10 flips*, and the variable be *the number of heads in the batch*. That variable is discrete numerical: it takes the values $0, 1, \ldots, 10$. Repeat for 100 batches and count.

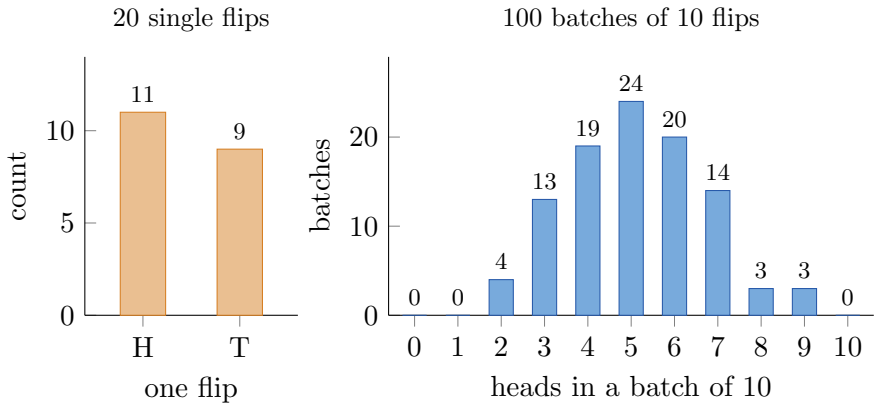


Figure 1.9: Left: the outcome of one flip is categorical. Right: the number of heads in 10 flips is discrete numerical. Bars sit on the integers, thin enough that you cannot mistake 4.5 for a possible value. The pile centers near 5 and falls off roughly symmetrically on both sides.

Exactly 5 heads happened in 24 of the 100 batches. Probability theory predicts $\binom{10}{5}/2^{10} = 252/1024 \approx 0.246$ (see *Probability From the Ground Up*, Ch. 7 on the binomial). The observed 0.24 is close but

not equal. That gap between a relative frequency and the probability behind it is the gap that inference is about.

1.5.3 Larger data: 200 latencies

Dot plots and stem-and-leaf plots break down with many values. Here are 200 latencies from the same kind of endpoint, ranging from 13 to 110 ms. Grouped into bins of width 25:

bin (ms)	$[0, 25)$	$[25, 50)$	$[50, 75)$	$[75, 100)$	$[100, 125)$
count	29	120	43	7	1
relative frequency	0.145	0.600	0.215	0.035	0.005

These counts are correct. Whether they are *useful* depends on the bins, and that is the next problem.

1.6 Bin width changes the picture

There is no single histogram of a data set. Every choice of bin width gives a different picture. Figure 1.10 shows the same 200 latencies three ways.

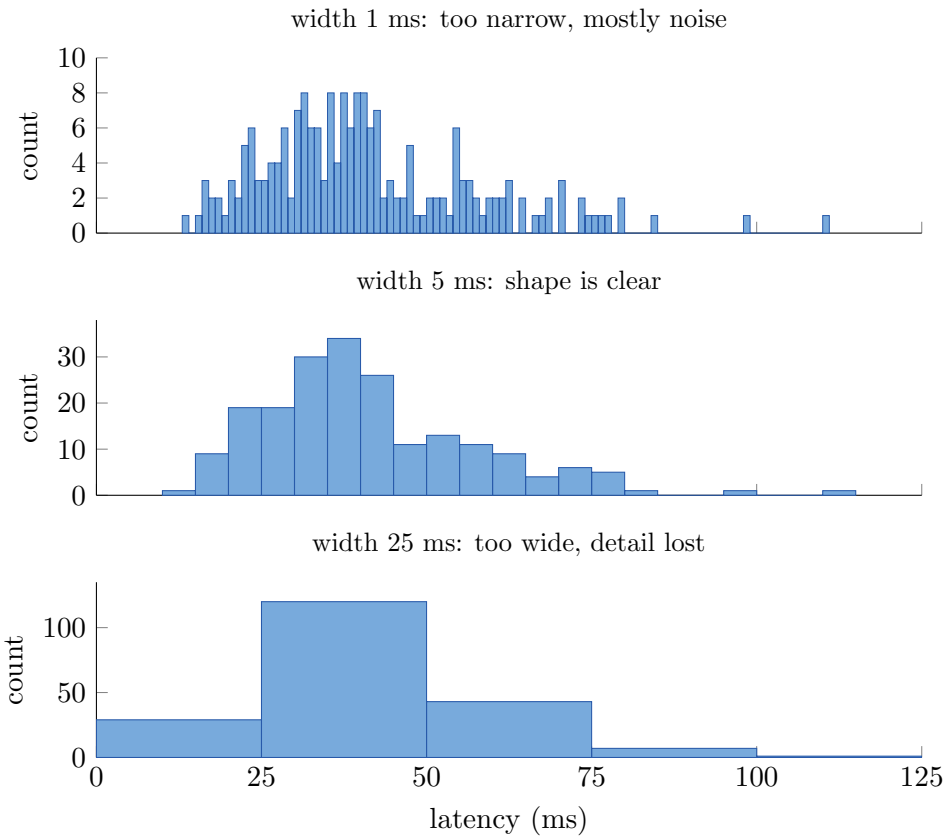


Figure 1.10: The same 200 latencies with three bin widths. Width 1 shows the random jitter of individual counts (note the meaningless spikes and holes). Width 25 squeezes the whole data set into five bars and hides the long right tail. Width 5 shows a single peak near 35 ms and a tail stretching to the right.

What goes wrong at each extreme:

- **Too narrow.** Each bin holds only a few values, so its count is dominated by luck. The bar at 31 ms has height 8, the bar at 34 ms has height 3. Nobody believes requests "prefer" 31 ms over 34 ms; another sample would put the spikes elsewhere.
- **Too wide.** Each bin averages over a big stretch of the number line. The $[25, 50)$ bar says 60% of requests fall there but cannot tell you whether they cluster near 30 or near 45. The tail beyond 75 ms, which is exactly what an on-call engineer cares about, shrinks to two stubby bars.

Why it works

Bin width trades two errors against each other. A narrow bin is *faithful to its location* but holds few points, so its height is noisy. A wide bin holds many points, so its height is stable, but it blurs everything inside it into one flat bar. With more data, each bin can be narrower before noise takes over. That is why rules of thumb for the number of bins grow with n: Sturges' rule uses $1 + \log_2 n$ bins, which for $n = 200$ gives about 8.6, and the Freedman–Diaconis rule sets the width to $2 \cdot \mathrm{IQR} \cdot n^{-1/3}$ (the IQR, a measure of spread, is defined in Chapter 2). For

the 200 latencies, which run from 13 to 110 ms, Sturges suggests a width of about $97/8.6 \approx 11$ ms and Freedman–Diaconis about 7 ms. Treat them as starting points and always try a few widths.

Common trap

Changing the bin width or shifting where the bins start can make a bump appear or vanish. If a histogram is evidence for some claim ("there are two kinds of users"), check that the feature survives at two or three other bin widths before you believe it. Figure 1.11 shows how little it takes.

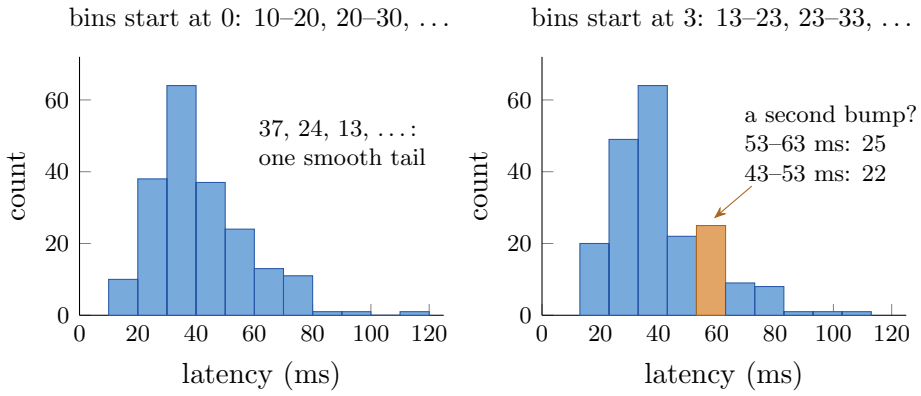


Figure 1.11: The same 200 latencies, bins of width 10 in both panels; only the starting edge moved, by 3 ms. On the right a dip and a second bump appear near 55 ms; on the left they do not exist. Neither histogram is wrong, which is why a feature must survive such changes before you trust it.

1.7 Density histograms: area is proportion

1.7.1 The problem with unequal bins

Sometimes bins must have different widths. Dashboards often use wide bins in the tail, where data is sparse, and narrow bins in the middle. Take the 200 latencies with edges $0, 20, 30, 40, 50, 60, 80, 120$. The counts are $10, 38, 64, 37, 24, 24, 3$. Plot those counts as bar heights and you get the left panel of Figure 1.12.

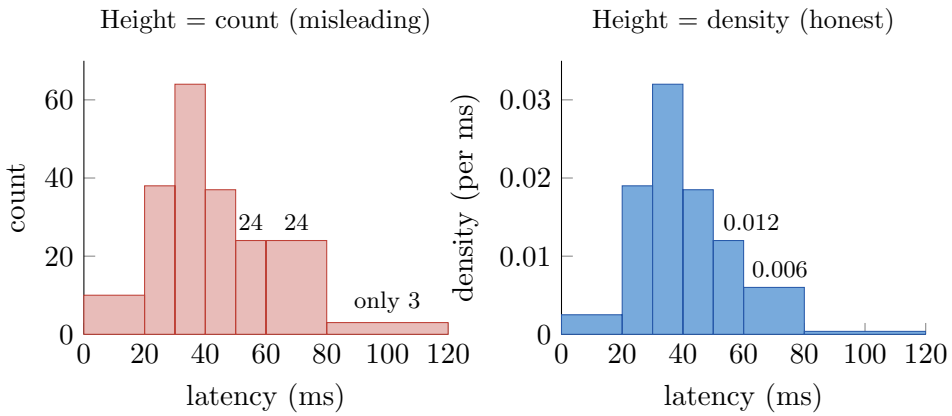


Figure 1.12: Same data, same unequal bins. Left: the $[60, 80)$ bar is as tall as the $[50, 60)$ bar because both hold 24 requests, but its 24 are spread over twice the width, so the data is half as dense there; and the wide $[80, 120)$ bar has a big visible area for only 3 requests. Right: dividing each count by n times the bin width halves the $[60, 80)$ bar (0.012 to 0.006) and flattens the tail, so area now matches the data.

The eye reads a bar's *area*, not its height. A bar twice as wide looks like twice as much data. The fix is to choose heights so that area tells the truth.

Definition 1.2 (Density histogram)**.** For a data set of n values and a bin of width w containing k values, the bar height is

$$\text{density} = \frac{k}{n\,w} = \frac{\text{relative frequency}}{\text{bin width}}.$$

The units are "fraction of the data per unit of the variable," here per millisecond.

For the bin $[60, 80)$: $k = 24$, $n = 200$, $w = 20$, so density $= 24/(200 \cdot 20) = 0.006$ per ms. For $[50, 60)$: $24/(200 \cdot 10) = 0.012$. Same count, half the density, as it should be.

Why it works

The area of a bar is height times width:

$$\frac{k}{n\,w} \cdot w = \frac{k}{n},$$

which is exactly the fraction of the data in that bin. Since every value lands in exactly one bin, the bar areas add up to $\sum k/n = n/n = 1$. Check it on our bins:

$$0.05 + 0.19 + 0.32 + 0.185 + 0.12 + 0.12 + 0.015 = 1.$$

So on a density histogram, "what fraction of requests took between a and b ms?" is answered by "the area of the bars between a and b," whatever the bin widths.

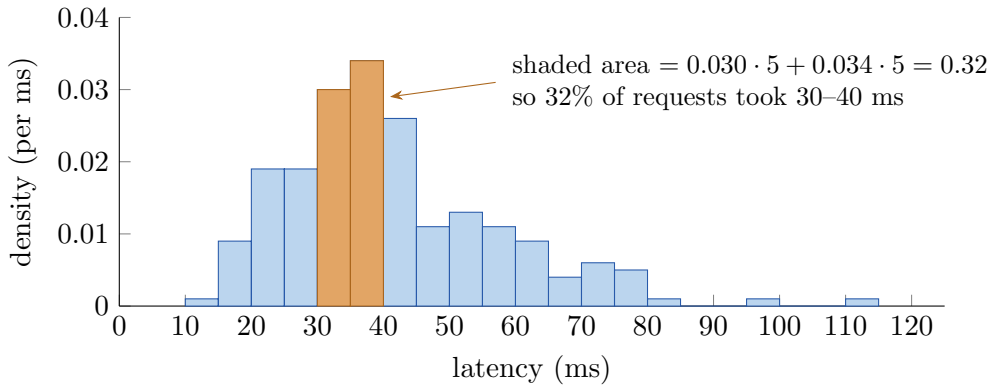


Figure 1.13: Density histogram of the 200 latencies, bins of width 5. The orange area equals the fraction of requests between 30 and 40 ms. Counting directly confirms it: 64 of the 200 latencies lie in $[30, 40)$, and $64/200 = 0.32$.

Connection

Area-equals-proportion is the idea that turns a histogram into a probability density. As the data grows and the bins shrink, the tops of the bars trace out a smooth curve $f(x)$, and "fraction of data between a and b" becomes $\int_a^b f(x)\,\mathrm{d}x$. That is precisely the definition of a probability density function (*Probability From the Ground Up*, Ch. 10). When an ML paper writes $x \sim p$, read it as "x is drawn from p": the p is this curve.

```
import numpy as np
lat = np.loadtxt("latencies.txt")          # the 200 values
edges = [0, 20, 30, 40, 50, 60, 80, 120]
counts, _ = np.histogram(lat, bins=edges)   # [10 38 64 37 24 24 3]
dens, _ = np.histogram(lat, bins=edges, density=True)
print(dens)                                 # 0.0025 0.019 0.032 ...
print((dens * np.diff(edges)).sum())        # 1.0: areas sum to one
```

Pause

A density histogram has a bar over $[40, 50)$ of height 0.0185 per ms, with $n = 200$. How many values are in that bin? *Answer:* area $= 0.0185 \times 10 = 0.185$, and $0.185 \times 200 = 37$.

1.8 Shapes of distributions

Once the bins are sensible, the first thing to read off a histogram is its overall shape. A handful of shapes cover most of what you will meet.

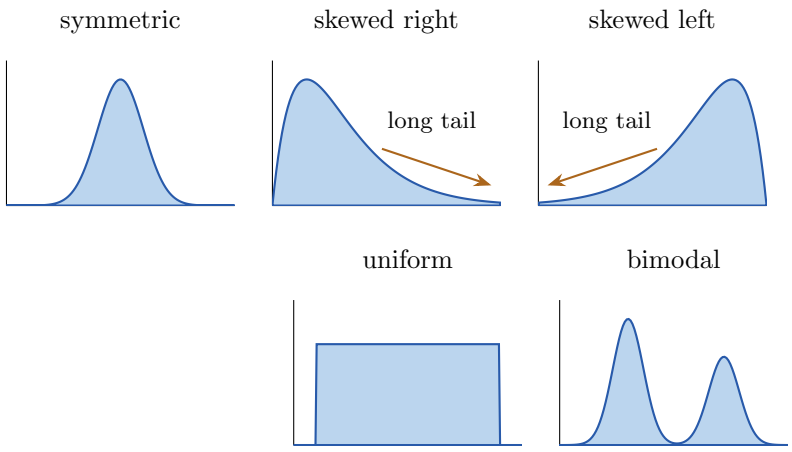


Figure 1.14: Five basic shapes, drawn as smoothed histogram outlines (horizontal axis: the value; height: how common it is). Skew is named for the direction of the long tail (orange arrows), not the peak: "skewed right" has its peak on the left and a tail reaching right.

- **Symmetric**: the left half mirrors the right half. Heights of adults of one sex, measurement errors, and the coin-flip counts in Figure 1.9 look like this.
- **Skewed right** (positively skewed): a long tail toward large values. Latencies, incomes, file sizes, house prices. Anything with a hard floor (latency cannot go below zero, or below the network round trip) but no hard ceiling tends to skew right.
- **Skewed left** (negatively skewed): a long tail toward small values. Exam scores on an easy test, age at death in a rich country. A hard ceiling with no floor nearby produces left skew.
- **Uniform**: roughly flat, every region equally common. The last digit of a timestamp in ms, or the face of a fair die over many rolls.
- **Bimodal**: two peaks. Almost always a sign that two different groups have been mixed into one column.

A peak is called a **mode**; "unimodal" means one peak and "bimodal" means two.

Common trap

"Skewed right" confuses people because the bulk of the data is on the *left*. Name the skew after the tail: if the long thin end points toward large values, the distribution is skewed right.

1.8.1 A left-skewed example: exam scores

Eighty students take an exam scored out of 100. Their scores, in bins of 10, are: $[30, 40)$: 1, $[40, 50)$: 2, $[50, 60)$: 2, $[60, 70)$: 5, $[70, 80)$: 17, $[80, 90)$: 31, $[90, 100]$: 22. (The top bin includes 100.)

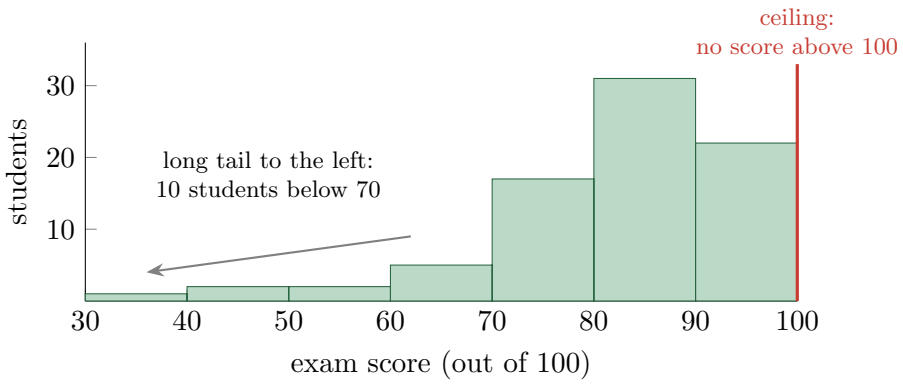


Figure 1.15: Exam scores of 80 students. The test was easy for most: the peak is in the 80s and nobody can score above 100. A few struggling students form a long tail toward low scores, making the distribution skewed left.

Here 53 of 80 students (66%) scored 80 or above, while the lowest bin reaches down into the 30s. Chapter 2 shows that a tail like this pulls the average below the middle value: in a left-skewed distribution the mean sits below the median.

1.8.2 A bimodal example: cache hits and misses

A service answers from a cache when it can (fast) and from a database when it cannot (slow). Here are 150 latencies, bins of width 5 ms.

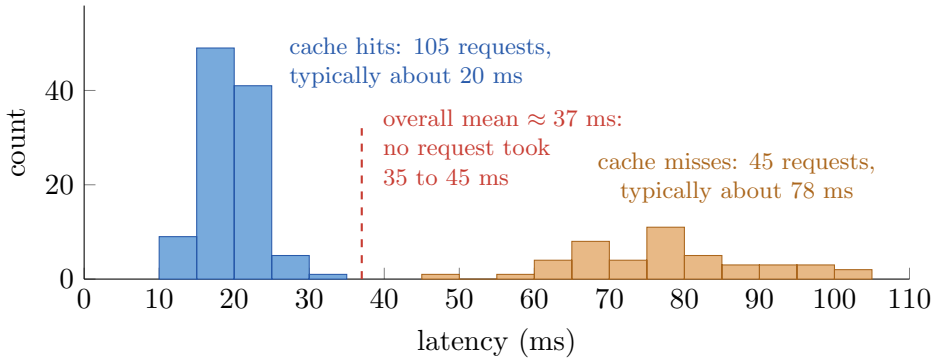


Figure 1.16: A bimodal latency distribution: two kinds of request (blue: answered from the cache; orange: answered from the database) share one column. The overall mean (dashed line) lands in the empty valley between the two peaks, a value that describes almost none of the actual requests.

Of the 150 requests, only 2 took between 30 and 55 ms, and none between 35 and 45. Yet the average of all 150 is about 37 ms. The arithmetic shows why: 105 hits (70%) average about 20 ms and 45 misses (30%) average about 78 ms, so the overall mean is roughly $0.7 \cdot 20 + 0.3 \cdot 78 \approx 37$, a weighted point between the two peaks. A single summary number cannot describe two groups. The right move is to split the data (by a cache-hit flag, if the log has one) and describe each group separately. A histogram is how you notice that you need to.

1.9 Outliers

An **outlier** is a value that sits far from the bulk of the data, separated from it by a gap. The 140 ms request in Figure 1.6 is one: the other 19 values are all under 60, and nothing lies between 58 and 140.

There is no universal cutoff for "far"; Chapter 3 gives a standard rule based on quartiles. What matters more is what you *do* with an outlier. It can be any of these:

- **A recording error.** A latency of −3 ms, a height of 17 m, a score of 850 on a 100-point test. Fix it if you can, drop it if you cannot, and write down that you did.
- **A different process.** The 140 ms request returned 503 and retried three times (Table 1.1). It is a real request, but it is a failure, not a slow success.
- **The thing you care about.** For a latency service-level objective (a promise such as "99% of requests finish within 100 ms"), the slow requests *are* the story. Deleting them would make the dashboard look better and the product worse.

Outliers can also come in clumps. Clients usually give up after a fixed timeout, and the log records the cap rather than the true time.

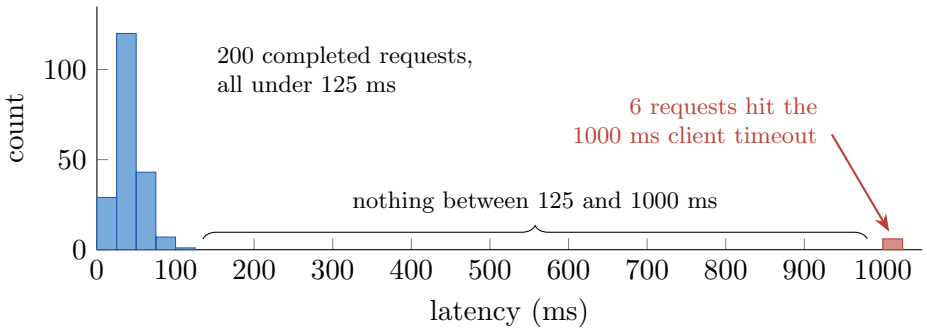


Figure 1.17: The 200 latencies plus 6 timeouts recorded as exactly 1000 ms, drawn to scale so the empty stretch shows how far the cap sits from real data. The red bar is not part of the tail; it is a pile-up at a hard cap, and the true times of those requests are unknown (they are at least 1000 ms). Averaging it in would treat "gave up" as a measurement.

Common trap

Never delete outliers just because they are inconvenient. Investigate first. Whatever you decide, report results both with and without them if the conclusion changes.

1.10 The distribution is the object of study

Every chart in this chapter answers the same question: *which values occur, and how often?* That pairing of values with their frequencies is the distribution.

Definition 1.3 (Distribution of a variable)**.** The **distribution** of a variable in a data set is the list of values it takes together with how often it takes each one (as counts, relative frequencies, or

densities). For a population, the distribution describes the whole population; for a sample, it is called the **empirical distribution**.

The empirical distribution is a rough copy of the population distribution, and the copy gets sharper as the sample grows. Figure 1.18 shows density histograms of latency samples of three sizes, all drawn from the same underlying process, with the population density drawn on top.

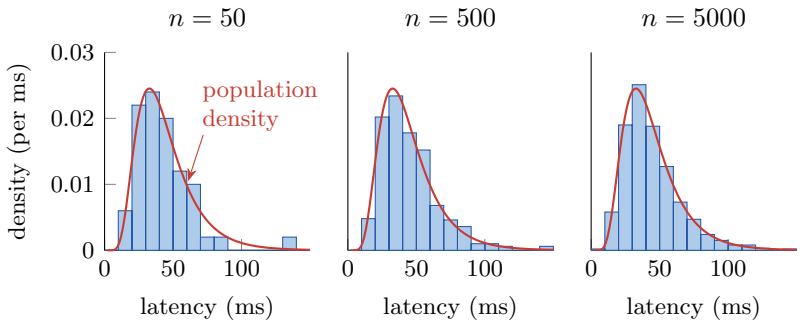


Figure 1.18: Density histograms of three samples from one latency process, bin width 10 ms, all on the same scale. The red curve is the population density. At $n = 50$ the bars are lumpy; at $n = 5000$ they hug the curve. The population distribution is the fixed target; each sample is a noisy picture of it.

What is really happening

This figure is a preview of the whole book. The red curve is what we want to know and never see directly. The bars are what we have. Describing the bars is Chapters 1–4. Saying how far the bars might be from the curve, for a given n, is inference: sampling distributions, standard errors, and confidence intervals (Chapters 6–8). Using a fitted curve to say what comes next is prediction (Chapters 12–15).

1.10.1 How to read any distribution

When you meet a new histogram, read it in a fixed order. Each step points to the chapter that develops it.

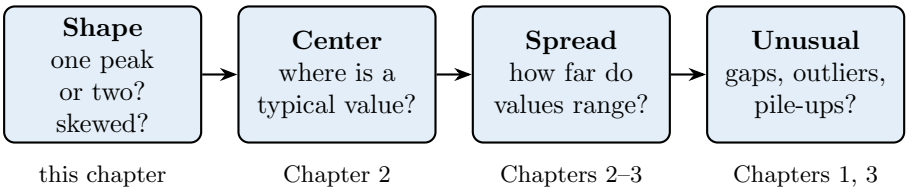


Figure 1.19: A fixed reading order for any distribution. Shape comes first because it decides which measures of center and spread are trustworthy: the mean works for the symmetric coin counts and fails for the bimodal cache latencies.

Apply it to the 200 latencies (Figure 1.10, middle panel). *Shape*: one peak, skewed right. *Center*: the peak bin is $[35, 40)$ and half the values are at or below 38 ms. *Spread*: most values lie between 15

and 80 ms. *Unusual*: isolated values at 98 and 110 ms worth a look. Those four sentences already tell an on-call engineer what to expect from this endpoint.

History

Karl Pearson, a British mathematician building the new field of mathematical statistics, introduced the word *histogram* in his lectures in 1891. He needed a name for the bar-picture he used to compare measured data (skull sizes, crab shell widths, death ages) with the smooth curves his theory predicted, and the name stuck. Eighty years later, John Tukey, a Princeton and Bell Labs statistician, complained that statisticians rushed to fit models before looking at their data. In the 1970s he invented quick pencil-and-paper displays for looking first, including the stem-and-leaf plot and the box plot (Chapter 3), and collected them in his 1977 book *Exploratory Data Analysis*. His point became standard practice: plot the distribution before you compute anything.

1.11 Summary

- Statistics describes data in hand, infers facts about a population, and predicts new values. Description comes first.
- A data set is a table: rows are observational units, columns are variables. Decide what one row is before analyzing.
- Variables are categorical (nominal or ordinal) or numerical (discrete or continuous). Digits do not make a variable numerical; ask whether arithmetic makes sense.
- A sample is the part of the population you observed. Parameters describe populations; statistics describe samples.
- Frequency tables count; relative frequencies (count/n) sum to 1 and compare across data sets of different sizes.
- Categorical data: bar charts with gaps, starting at zero; keep ordinal order. Numerical data: dot plots and stem-and-leaf for small n, histograms with touching bars for larger n.
- Bin width trades noise (too narrow) against blur (too wide). Try several.
- Density histogram: height $= k/(nw)$, so bar area is the fraction of data in the bin and total area is 1. Required for unequal bins; the bridge to probability densities.
- Shapes: symmetric, skewed right (long right tail), skewed left, uniform, bimodal. Bimodal usually means two groups mixed together.
- Outliers may be errors, a different process, or the main story. Investigate before removing.
- The distribution (which values, how often) is the object of study. Read it in order: shape, center, spread, unusual features.

Chapter 2 turns the second and third items of that reading order into numbers: the mean and median for center, and the standard deviation for spread, including why it divides by $n - 1$.

1.12 Exercises

1. For each variable, give its type (nominal, ordinal, discrete, continuous): (a) CPU temperature; (b) number of open pull requests; (c) country of a user; (d) severity level P0/P1/P2/P3; (e) HTTP method (GET, POST, ...).
2. A survey of 250 engineers asks their primary language: Python 110, Java 60, Go 45, C++ 35. Compute the relative frequencies and say which chart you would draw and how you would order its bars.
3. Make a stem-and-leaf plot of these page sizes in KB: 41, 47, 52, 38, 55, 49, 63, 44, 58, 36, 51, 47. Describe the shape in one sentence.
4. Using the binned table for the 20 latencies (Table 1.3), what fraction of requests took under 30 ms? Under 40 ms?
5. A density histogram of $n = 400$ values has a bar over $[2, 5)$ of height 0.15. How many values lie in $[2, 5)$?
6. Bins $[0, 10)$, $[10, 20)$, $[20, 50)$ hold 30, 50, 60 values out of $n = 140$. Compute the three densities. Which bin has the tallest bar on a density histogram? On a count histogram?
7. Predict the shape (symmetric, skewed right, skewed left, uniform, bimodal) of: (a) file sizes on a laptop; (b) the second-hand reading of a wall clock at random moments; (c) heights of a group containing both 6-year-olds and their parents; (d) age at retirement.
8. Explain why a histogram of 30 values with 30 bins and a histogram with 2 bins would both be poor summaries.
9. In the cache example (Figure 1.16), why is "average latency 37 ms" a misleading headline? What two numbers would you report instead?

Answers

1. (a) continuous; (b) discrete; (c) nominal; (d) ordinal; (e) nominal.
2. $110/250 = 0.44$, $60/250 = 0.24$, $45/250 = 0.18$, $35/250 = 0.14$ (sum 1). A bar chart with gaps, bars sorted by height, since language is nominal.
3. Stems 3 to 6: `3 | 6 8`, `4 | 1 4 7 7 9`, `5 | 1 2 5 8`, `6 | 3`. Roughly symmetric, centered in the high 40s, no outliers.
4. Under 30: $(8 + 7)/20 = 0.75$. Under 40: $(8 + 7 + 2)/20 = 0.85$.
5. Area $= 0.15 \times 3 = 0.45$, so $0.45 \times 400 = 180$ values.
6. $30/(140 \cdot 10) \approx 0.0214$, $50/(140 \cdot 10) \approx 0.0357$, $60/(140 \cdot 30) \approx 0.0143$. Tallest density bar: $[10, 20)$. Tallest count bar: $[20, 50)$, which misleads because that bin is three times as wide.
7. (a) skewed right (many small files, a few huge ones); (b) uniform; (c) bimodal; (d) roughly symmetric around a typical retirement age, often with a left tail from early retirees.
8. With 30 bins most bins hold 0 or 1 value, so the picture is noise. With 2 bins every feature (peaks, skew, outliers) is averaged away.

9. No request takes anywhere near 37 ms; the data are two groups. Report each group separately, e.g. typical cache-hit latency (about 20 ms) and typical cache-miss latency (about 78 ms), along with the hit rate (105 of 150, 70%).

Chapter 2

Center and Spread

A dashboard shows five request latencies from one server, in milliseconds:

$$20, \quad 22, \quad 25, \quad 28, \quad 35.$$

Two questions come up immediately. What is a *typical* latency? And how *different* are the requests from each other? The first question asks for a measure of **center**; the second asks for a measure of **spread**. Every summary in this chapter answers one of those two questions, and each answer has a precise sense in which it is the "right" one. Chapter 1 drew the whole shape of a data set as a histogram; this chapter squeezes that shape into two or three numbers you can put on a dashboard.

Notation: the data are $x_1, x_2, \ldots, x_n$. Sorted in increasing order they are written $x_{(1)} \le x_{(2)} \le \cdots \le x_{(n)}$. These are the *order statistics*: $x_{(1)}$ is the smallest value, $x_{(n)}$ the largest, and the bracket in the subscript means "position after sorting."

2.1 The mean

Definition 2.1 (Sample mean)**.** The **mean** (arithmetic average) of $x_1, \ldots, x_n$ is

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i.$$

For the latencies,

$$\bar{x} = \frac{20 + 22 + 25 + 28 + 35}{5} = \frac{130}{5} = 26 \text{ ms}.$$

No request actually took 26 ms. The mean need not be a data value; it is a summary, not a member.

2.1.1 The mean is the balance point

Put each observation as an equal weight on a weightless ruler. The ruler balances at exactly one place, and that place is $\bar{x}$. The reason is one line of algebra. Call $d_i = x_i - \bar{x}$ the **deviation** of observation i. Then

$$\sum_{i=1}^{n} d_i = \sum_{i=1}^{n} x_i - n\bar{x} = n\bar{x} - n\bar{x} = 0.$$

Deviations to the left exactly cancel deviations to the right. For the latencies the deviations are $-6, -4, -1, +2, +9$: the left side totals -11 and the right side totals $+11$. In physics terms the total *torque* (weight times signed distance from the pivot, the quantity that makes a seesaw tip) about $\bar{x}$ is zero.

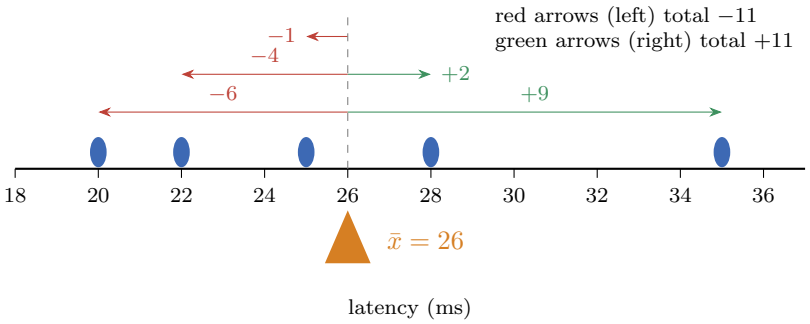


Figure 2.1: The five latencies as equal weights on a ruler, with the pivot (orange triangle) at the mean $\bar{x} = 26$. Each arrow is one signed deviation from the mean: the left ones total -11, the right ones $+11$, so they cancel and the ruler balances.

The balance picture explains the mean's main weakness. A weight far from the rest has a long lever arm. Move the 35 ms request to 180 ms and the balance point moves from 26 to $(20+22+25+28+180)/5 = 55$ ms, above four of the five values.

Pause

Pause and think: add a sixth request that took exactly 26 ms. Where does the ruler balance now? (It stays at 26. The new weight sits on the pivot, so it adds no torque.)

2.2 The median, the mode, and the trimmed mean

Definition 2.2 (Median)**.** Sort the data. If n is odd, the **median** is the middle value $x_{((n+1)/2)}$. If n is even, it is the average of the two middle values, $\frac{1}{2}\big(x_{(n/2)} + x_{(n/2+1)}\big)$.

The five latencies have median 25 (the third of five). Add a sixth request of 180 ms. Now $n = 6$, the middle pair is 25 and 28, and the median is 26.5. The mean jumps from 26 to $(130+180)/6 \approx 51.7$.

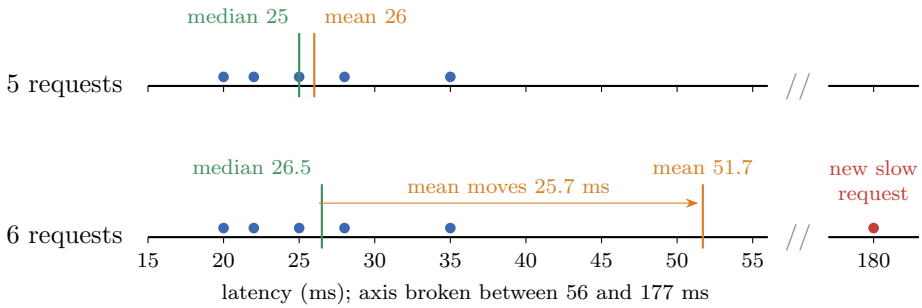


Figure 2.2: Top: the five latencies, where mean (26) and median (25) almost coincide. Bottom: add one slow request at 180 ms (red). The mean jumps to 51.7 ms, above five of the six values; the median only moves to 26.5. The axis is broken because nothing lies between 56 and 177 ms; drawn to scale, the five dots would merge into one blob.

The median ignores *how far* the extreme values are. It only uses their *rank*. That is why "p50 latency" (the 50th percentile: half of requests are faster) on a monitoring dashboard is the median: a single pathological request cannot distort it.

Definition 2.3 (Mode)**.** The **mode** is the most frequent value.

The mode is the only center that works for categorical data. In a log of HTTP status codes $200, 200, 404, 200, 500, 200, 404$ the mode is 200. A mean of status codes is meaningless. For continuous measurements every value is usually distinct, so the mode is read off a histogram as the tallest bar (the "modal class"), and a distribution with two clear peaks is called **bimodal**.

Definition 2.4 (Trimmed mean)**.** The **α-trimmed mean** sorts the data, deletes the $\lfloor \alpha n \rfloor$ smallest and $\lfloor \alpha n \rfloor$ largest values, and averages what remains.

The 0% trimmed mean is the ordinary mean. Trimming just under 50% from each end leaves only the middle and gives the median. A trimmed mean sits between the two: it uses the magnitudes of most of the data but discards the extremes. Olympic diving and figure-skating scores use exactly this rule.

2.3 Mean versus median: skew and outliers

Example 2.1 (Salaries at a small company)**.** Ten employees earn, in thousands of dollars per year,

$$40,\ 45,\ 48,\ 50,\ 52,\ 55,\ 58,\ 60,\ 65,\ 900.$$

The last salary is the founder's. The sum is $473 + 900 = 1373$, so

$$\bar{x} = 137.3, \qquad \text{median} = \frac{52 + 55}{2} = 53.5.$$

Nine of the ten people earn less than the mean. The 10% trimmed mean drops one value from each end (the 40 and the 900) and averages the remaining eight: $433/8 = 54.125$. Without the founder the mean would be $473/9 \approx 52.6$ and the median 52.

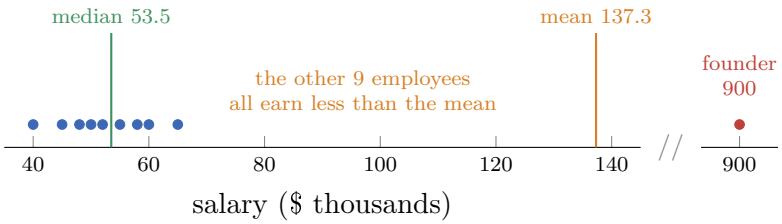


Figure 2.3: Salaries at a ten-person company, one dot per person. The founder's 900 pulls the mean to 137.3, to the right of all nine other employees; the median, 53.5, sits in the middle of them. The axis is broken between 145 and 890 so the nine dots stay separate.

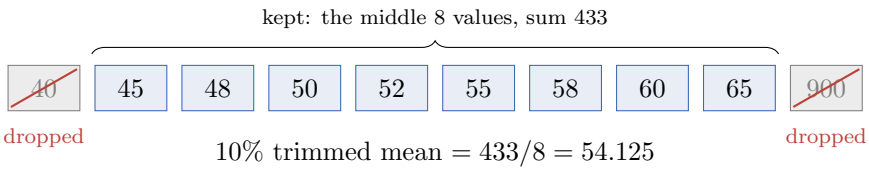


Figure 2.4: The 10% trimmed mean of the ten salaries, step by step. Sort, cross out $\lfloor 0.1 \times 10 \rfloor = 1$ value at each end (the 40 and the founder's 900), and average the eight that remain. The result, 54.1, lands next to the median (53.5), far from the mean (137.3).

Which number is "correct"? Both, for different questions. If the company wants to know its total payroll, the mean answers it: total $= n\bar{x} = 1373$. If a job candidate wants to know what a typical employee earns, the median answers it. The mean is the right summary for *totals*; the median is the right summary for *a typical individual*.

2.3.1 Skew moves the mean toward the tail

A distribution is **right-skewed** (positively skewed) if it has a long tail to the right: incomes, house prices, latencies, file sizes, city populations. The tail values pull the mean right while the median stays with the bulk. The typical ordering is

$$\text{mode} < \text{median} < \text{mean} \qquad \text{(right skew)},$$

reversed for left skew (exam scores with a ceiling, age at death in rich countries). For a symmetric distribution all three coincide.

Pause

Pause and think: a news story reports that the mean house price in a city is $620,000 and the median is $450,000. Which way is the distribution skewed, and roughly what share of houses sell for less than $620,000? (Right-skewed; more than half, since the median is below the mean.)

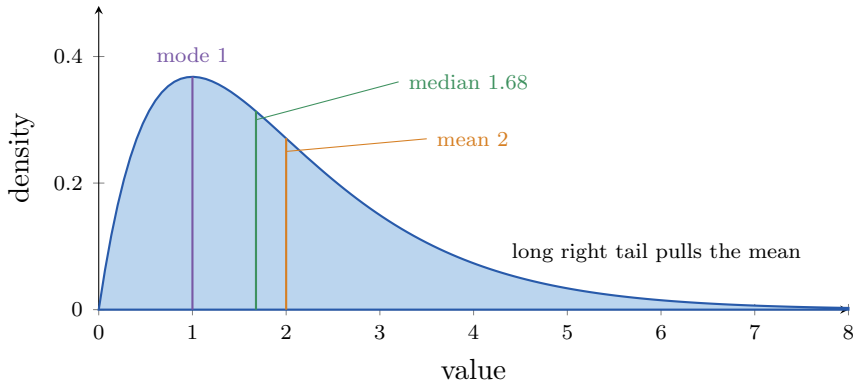


Figure 2.5: A right-skewed density curve, $f(x) = xe^{-x}$ (area under the curve between two values is the share of data between them). The peak (mode) is at 1, half the area lies left of 1.68 (median), and the balance point (mean) is at 2. The tail drags the mean furthest.

Common trap

"The average household income is \$X" does not say which center was used. If it is a mean, then because incomes are right-skewed, well over half of households earn less than \$X. When a distribution is skewed, ask which center was used. Government statistics on income and house prices usually report the median for this reason.

2.3.2 Robustness and the breakdown point

How much bad data can a statistic absorb before it becomes useless? Keep the nine ordinary salaries and let the founder's salary v grow. The median stays at 53.5 for every $v \geq 55$. The mean is $(473 + v)/10$, which grows without bound.

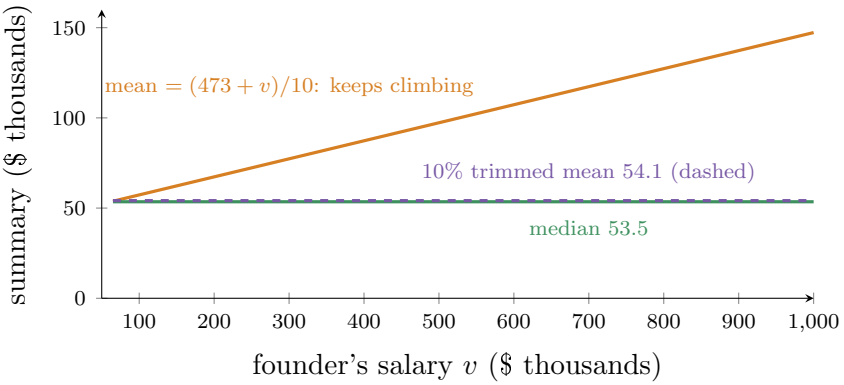


Figure 2.6: Let the founder's salary v grow. The mean rises along a straight line and can be made as large as you like. The median and the trimmed mean stay flat, almost on top of each other: no single value can move them far.

Definition 2.5 (Breakdown point)**.** The **breakdown point** of a statistic is the smallest fraction of the data that, if replaced by arbitrary values, can make the statistic arbitrarily large.

- **Mean:** one bad value out of n suffices, so the breakdown point is $1/n$, which tends to 0.
- **Median:** you must corrupt half the data to drag the middle value away. Breakdown point $\approx 50\%$, the maximum possible.
- **α-trimmed mean:** the trimming absorbs up to αn bad values at each end. Breakdown point $\approx \alpha$.

A statistic with a high breakdown point is called **robust**. Robustness has a price: the median throws away the magnitudes of most observations, so for clean, symmetric, bell-shaped data it is noisier than the mean: from sample to sample its standard deviation is about 25% larger (Chapter 6 shows this by simulation). Choose the mean when the data are clean and you care about totals; choose the median when outliers or heavy tails are plausible.

2.4 Deviations, variance, and standard deviation

Two servers can share a mean latency of 26 ms while one is steady and the other erratic (Figure 2.7). Spread measures how far observations typically sit from the center.

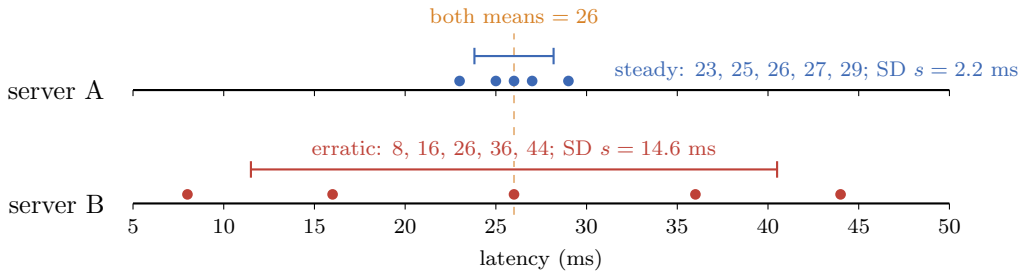


Figure 2.7: Two servers, five requests each, with the same mean latency of 26 ms. The mean cannot tell them apart; a measure of spread can. Bars show mean ± 1 SD, defined below: about 2 ms for server A and about 15 ms for server B.

The natural starting point is the deviations $d_i = x_i - \bar{x}$. Their average is useless: it is always 0 (Section 2.1). Positive and negative deviations cancel. Two fixes remove the signs: take absolute values, or square. Squaring gives the variance.

Definition 2.6 (Sample variance and standard deviation)**.**

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2, \qquad s = \sqrt{s^2}.$$

s^2 is the **sample variance**; s is the **sample standard deviation** (SD). The divisor $n-1$ is explained in Section 2.5.

2.4.1 A worked table

Build s for the five latencies column by column.

i	x_i (ms)	$d_i = x_i - 26$	d_i^2
1	20	-6	36
2	22	-4	16
3	25	-1	1
4	28	$+2$	4
5	35	$+9$	81
sum	130	0	138

1. Mean: $\bar{x} = 130/5 = 26$.
2. Deviations: subtract 26. Check that they sum to 0.
3. Square each deviation and add: $\sum d_i^2 = 138$ (the **sum of squares**).
4. Divide by $n - 1 = 4$: $s^2 = 138/4 = 34.5$ ms^2.
5. Take the square root: $s = \sqrt{34.5} \approx 5.87$ ms.

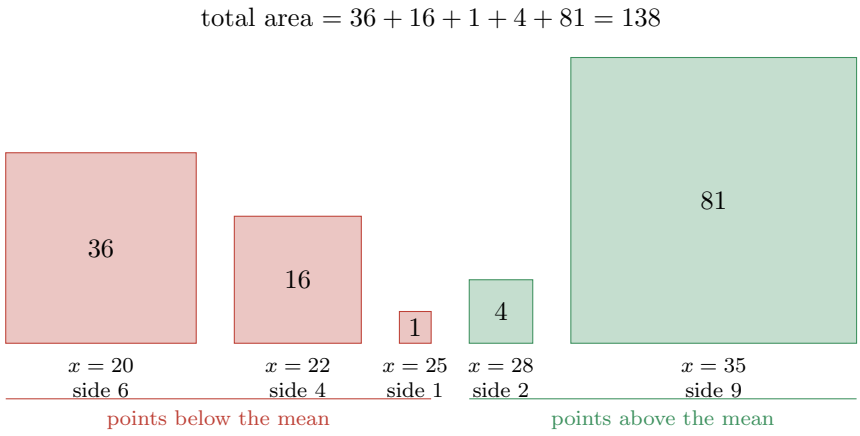


Figure 2.8: Each squared deviation drawn as a square whose side is that point's distance from the mean $\bar{x} = 26$ (red: points below the mean, green: above), all to the same scale. The variance is the total area, 138, divided by $n - 1 = 4$. The 35 ms point alone supplies $81/138 \approx 59\%$ of it.

The squares make one fact obvious: squaring weights large deviations heavily. A point twice as far from the mean contributes four times as much. So the variance and SD are *not* robust: their breakdown point is $1/n$, like the mean. The salary data have $s \approx 268$ thousand with the founder and $s \approx 7.8$ thousand without.

Intuition

Variance is in squared units (ms^2, dollars^2), which no one can picture. The square root returns to the original units. Read $s \approx 5.9$ ms as "a typical request is roughly 6 ms from the average." It is not exactly the average distance (that would be the mean absolute deviation, $22/5 = 4.4$ ms here); squaring then rooting gives extra weight to the far points.

A useful shortcut for hand computation expands the square:

$$\sum (x_i - \bar{x})^2 = \sum x_i^2 - n\bar{x}^2.$$

For the latencies, $\sum x_i^2 = 400 + 484 + 625 + 784 + 1225 = 3518$ and $n\bar{x}^2 = 5 \cdot 676 = 3380$, giving 138 again. On a computer, avoid this form: when $\bar{x}$ is large compared with s, the two big numbers nearly cancel and floating-point error destroys the answer.

2.5 Why divide by $n - 1$?

Dividing by n would seem natural: the variance is "the average squared deviation." Indeed the **population variance** of a finite population of N values with mean μ is

$$\sigma^2 = \frac{1}{N}\sum_{j=1}^{N}(x_j - \mu)^2.$$

Here the **population** is the whole collection we care about (every request the server will ever handle), μ is its mean, and σ^2 is its average squared distance from μ. A **sample** is the handful of values we actually see. The goal is to guess σ^2 from the sample.

The catch: we do not know μ. We measure distances from $\bar{x}$ instead, and $\bar{x}$ was computed from the same points we are measuring. It sits in the middle of *these* points, wherever they happen to fall, so they always look a little closer to their center than they really are to the true center. Take the most extreme case, two points. Their mean sits exactly halfway between them, so each is half the gap away from it. The true mean μ is usually somewhere else, and then at least one of the two points is more than half the gap from μ. Distances from $\bar{x}$ come out too small, every time, in the same direction. The rest of this section measures exactly how much too small, and shows that the cure is to divide by $n - 1$.

2.5.1 The sample mean hugs its own data

Define the sum of squares about an arbitrary point c:

$$S(c) = \sum_{i=1}^{n}(x_i - c)^2.$$

Write $x_i - c = (x_i - \bar{x}) + (\bar{x} - c)$ and expand:

$$S(c) = \sum(x_i - \bar{x})^2 + 2(\bar{x} - c)\underbrace{\sum(x_i - \bar{x})}_{=0} + n(\bar{x} - c)^2 = S(\bar{x}) + n(\bar{x} - c)^2.$$

So $S(c)$ is a parabola in c with minimum exactly at $c = \bar{x}$. Whatever the true mean μ is, $S(\mu) \geq S(\bar{x})$, with a gap of $n(\bar{x} - \mu)^2$.

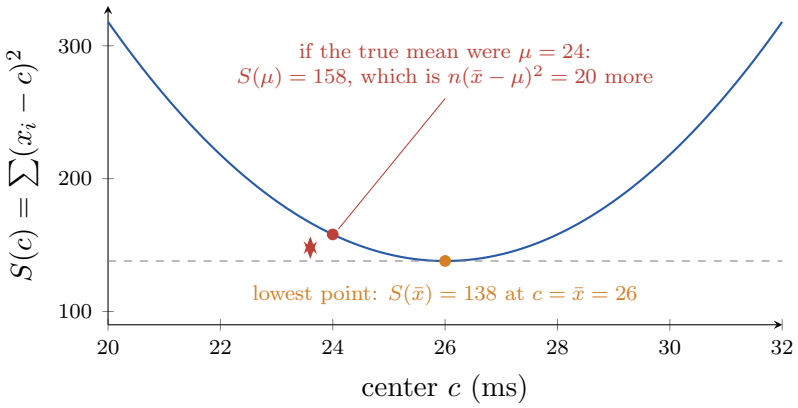


Figure 2.9: For the latency data, $S(c) = 138 + 5(26 - c)^2$, a parabola. Its lowest point (orange) is at the sample mean. The sum of squares about the true mean (red, if $\mu = 24$) is higher, so measuring from $\bar{x}$ instead of μ can only make the sum smaller, never larger.

The identity has a plain reading. The squared distances to the true center equal the squared distances to the sample's own center, *plus* n times the squared distance between the two centers. That extra term is exactly what $\bar{x}$ hides.

2.5.2 Every possible sample, listed

Before any algebra, here is the bias in a case small enough to check by hand. A population has three equally common values, 0, 3 and 6. Its mean is $\mu = 3$ and its variance is

$$\sigma^2 = \tfrac{1}{3}\big((0-3)^2 + (3-3)^2 + (6-3)^2\big) = \tfrac{1}{3}(9 + 0 + 9) = 6.$$

Draw a sample of $n = 2$ values, each drawn independently (so the same value can come up twice). There are $3 \times 3 = 9$ equally likely ordered samples. For each one, compute the sum of squares about the true $\mu = 3$ and about the sample's own mean.

sample	$\bar{x}$	$\sum(x_i - \mu)^2$	$\sum(x_i - \bar{x})^2$	gap $2(\bar{x} - \mu)^2$
0, 0	0	18	0	18
0, 3	1.5	9	4.5	4.5
0, 6	3	18	18	0
3, 0	1.5	9	4.5	4.5
3, 3	3	0	0	0
3, 6	4.5	9	4.5	4.5
6, 0	3	18	18	0
6, 3	4.5	9	4.5	4.5
6, 6	6	18	0	18
average	3	$12 = n\sigma^2$	$6 = (n-1)\sigma^2$	$6 = \sigma^2$

Read the bottom row. Measured from the true center, the sum of squares averages $12 = 2\sigma^2$, so dividing by $n = 2$ would be exactly right. Measured from $\bar{x}$, it averages only 6. In no row is it larger

than the true-center column, and in six of the nine rows it is smaller. Dividing by $n = 2$ gives an average estimate of 3, half the truth. Dividing by $n - 1 = 1$ gives 6, exactly the truth.

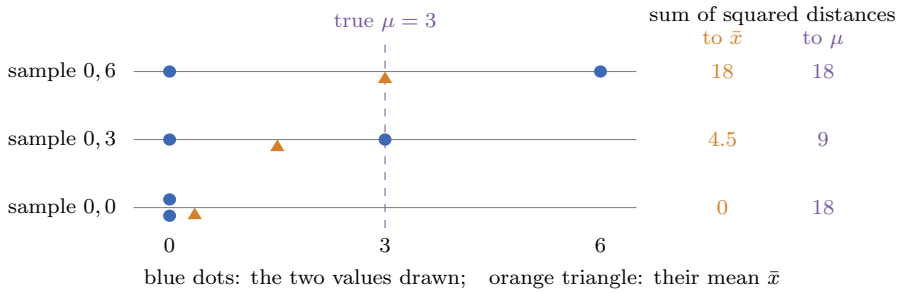


Figure 2.10: Three of the nine samples from the population $\{0, 3, 6\}$. When the sample straddles μ (top), $\bar{x} = \mu$ and nothing is lost. When it lands to one side, $\bar{x}$ follows it and the spread looks smaller than it is: 4.5 instead of 9, and in the bottom row (both values 0; the triangle is drawn just to the right) 0 instead of 18.

2.5.3 The derivation

Now do the same bookkeeping for any population and any n. Three words first. An **estimator** is a recipe that turns a sample into a guess for an unknown population number, such as "divide the sum of squares by n." Because the sample is random, the guess is random too. Its **expectation**, written $\mathbb{E}[\cdot]$, is the long-run average of the guess over many repeated samples, exactly the "average" row of the table above. The **variance** of a random quantity, $\mathrm{Var}(\cdot)$, is the expectation of its squared distance from its own expectation.

Treat the data as random: $X_1, \ldots, X_n$ independent, each with mean μ and variance σ^2 (capital letters mark values not yet drawn). Apply the identity with $c = \mu$ and rearrange:

$$\sum_{i=1}^{n}(X_i - \bar{X})^2 = \sum_{i=1}^{n}(X_i - \mu)^2 - n(\bar{X} - \mu)^2.$$

Take expectations of each term. (Expectation of a sum is the sum of expectations.)

- $\mathbb{E}\left[(X_i - \mu)^2\right] = \sigma^2$ by definition of variance, so the first sum has expectation $n\sigma^2$.
- $\mathbb{E}\left[(\bar{X} - \mu)^2\right] = \mathrm{Var}(\bar{X})$, because $\bar{X}$ has expectation μ. For independent values, variances add, so $\mathrm{Var}(X_1 + \cdots + X_n) = n\sigma^2$. Dividing a quantity by n divides its variance by n^2, so $\mathrm{Var}(\bar{X}) = n\sigma^2/n^2 = \sigma^2/n$. (Chapter 6 builds this result carefully; Probability From the Ground Up proves that variances of independent values add.) So the second term has expectation $n \cdot \sigma^2/n = \sigma^2$.

Therefore

$$\mathbb{E}\left[\sum_{i=1}^{n}(X_i - \bar{X})^2\right] = n\sigma^2 - \sigma^2 = (n-1)\sigma^2.$$

In the table: $n\sigma^2 = 12$, minus $\sigma^2 = 6$, leaves $(n-1)\sigma^2 = 6$. Dividing the sum of squares by n gives an estimator whose average is $\frac{n-1}{n}\sigma^2$, too small. Dividing by $n - 1$ gives

$$\mathbb{E}[s^2] = \sigma^2.$$

An estimator whose expectation equals the target is called **unbiased**; the difference (expectation minus target) is its **bias**. The divide-by-n estimator has bias $-\sigma^2/n$. The correction from n to $n-1$ is **Bessel's correction**.

History

In the early 1800s astronomers combined many slightly disagreeing telescope readings of a star's position and needed to report how precise the result was. Carl Friedrich Gauss, in an 1823 treatise on combining observations, showed that the sum of squared *residuals* (each reading minus the fitted value) comes out too small, and that dividing by the number of readings *minus the number of quantities fitted from them* removes the shortfall. For a single fitted mean that divisor is $n-1$. The name honours his contemporary Friedrich Bessel, an astronomer who studied observation errors. The same rule, "subtract one per fitted parameter," gives the $n-2$ in simple regression (Chapter 12).

Why it works

Two ways to see why exactly one σ^2 goes missing.
The wander of the center. $\bar{X}$ misses μ by a random amount whose typical square is σ^2/n. Every one of the n points is measured from the wrong center, and the identity says the shortfall is $n(\bar{X}-\mu)^2$. On average that is $n \times \sigma^2/n = \sigma^2$: one data point's worth of variance.
Degrees of freedom. The n deviations $X_i - \bar{X}$ must sum to zero, so once you know any $n-1$ of them the last one is forced. With $n=2$ the two deviations are always $+a$ and $-a$: one free number, not two. The sum of squares contains only $n-1$ free pieces of information, called **degrees of freedom**, and we average over those. Each quantity estimated from the data before measuring spread uses up one.

2.5.4 Simulation

Draw $n=3$ values from a Normal population with $\sigma^2=4$, compute both the divide-by-n estimate and s^2, and repeat. Figure 2.11 tracks the running average of each estimator. The $n-1$ version settles near 4; the divide-by-n version settles near $\frac{2}{3}\cdot 4 \approx 2.67$.

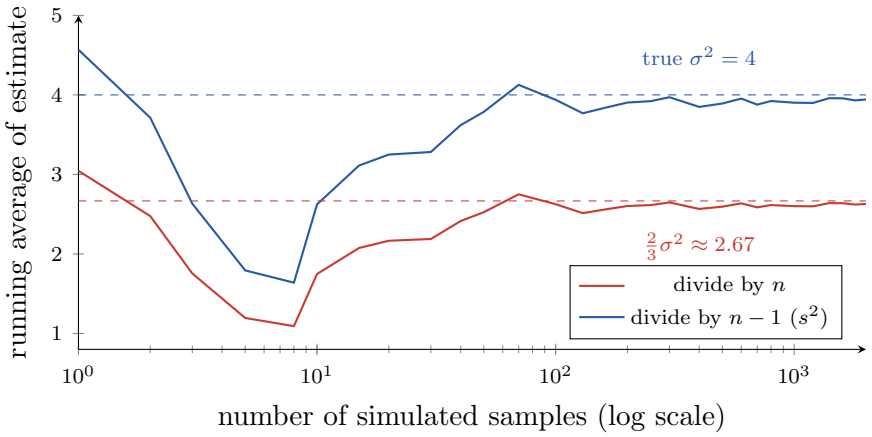


Figure 2.11: 2000 simulated samples of size $n = 3$ from $\mathcal{N}(10, 4)$. Individual estimates are noisy, but their long-run averages separate cleanly: s^2 averages to the true variance, while dividing by n averages to $\frac{n-1}{n}\sigma^2$.

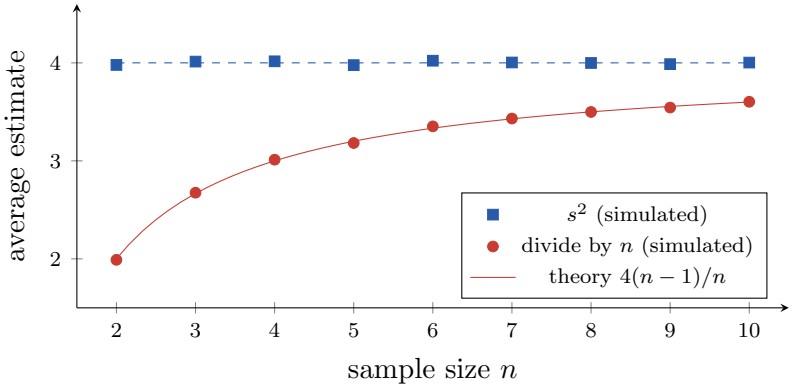


Figure 2.12: Average of 50,000 estimates at each sample size. The divide-by-n estimator follows the theoretical curve $\sigma^2(n-1)/n$: half the true variance at $n = 2$, still 10% low at $n = 10$. The bias vanishes as n grows, which is why the choice matters most for small samples.

Common trap

Software disagrees on the default. NumPy's `np.var` and `np.std` divide by n unless you pass `ddof=1`; pandas' `.var()` and `.std()` divide by $n-1$. The same data give different answers. (`ddof` stands for "delta degrees of freedom": the divisor is $n -$ `ddof`.) Also, s itself is *not* unbiased for σ: the square root bends downward, so averaging and then rooting gives more than rooting and then averaging, and $\mathbb{E}[s] < \sqrt{\mathbb{E}[s^2]} = \sigma$. For Normal data s runs about 20% low on average at $n = 2$, 6% at $n = 5$, and 3% at $n = 10$. Beyond small samples this is usually ignored.

When do you divide by n? Two cases. If the data *are* the whole population (every employee of the company, every request in the log you care about), there is nothing larger to estimate, and the divide-by-n formula is the population variance. And if the true mean μ is known from outside the data, measure deviations from μ, not $\bar{x}$; then $\frac{1}{n}\sum(x_i - \mu)^2$ is unbiased (the first column of the table).

When the data are a sample used to learn about something larger and μ is estimated by $\bar{x}$, divide by $n-1$.

Pause

Pause and think: what does s^2 give for a sample of size $n = 1$? (Zero over zero: undefined.) Why is that the honest answer? (One value says nothing about spread; dividing by n would claim the spread is exactly 0.)

2.6 The key link: which center minimises which error?

The identity $S(c) = S(\bar{x}) + n(\bar{x} - c)^2$ says more than Bessel's correction. It says:

Theorem 2.1 (Mean minimises squared error)**.** *For any data,* $\bar{x} = \arg\min_c \sum_{i=1}^{n} (x_i - c)^2$.

The median has the same property for a different loss.

Theorem 2.2 (Median minimises absolute error)**.** *For any data, a median minimises*

$$A(c) = \sum_{i=1}^{n} |x_i - c|.$$

Why it works

Move c a small step δ to the right. Every point to the right of c gets δ closer; every point to the left gets δ farther. So $A(c)$ changes by $\delta \times (\#\text{left} - \#\text{right})$. While more points lie to the right, moving right helps. The improvement stops exactly when the counts balance, that is, at the median. Only the *count* of points on each side matters, never their distances, which is the same reason the median is robust.

Take the data $1, 2, 3, 4, 10$ (mean 4, median 3). Here $\sum x_i = 20$ and $\sum x_i^2 = 130$, so

$$S(c) = 5c^2 - 40c + 130, \qquad S(4) = 50.$$

The absolute loss is piecewise linear: $A(0) = 20$, $A(1) = 15$, $A(2) = 12$, $A(3) = 11$, $A(4) = 12$, $A(10) = 30$.

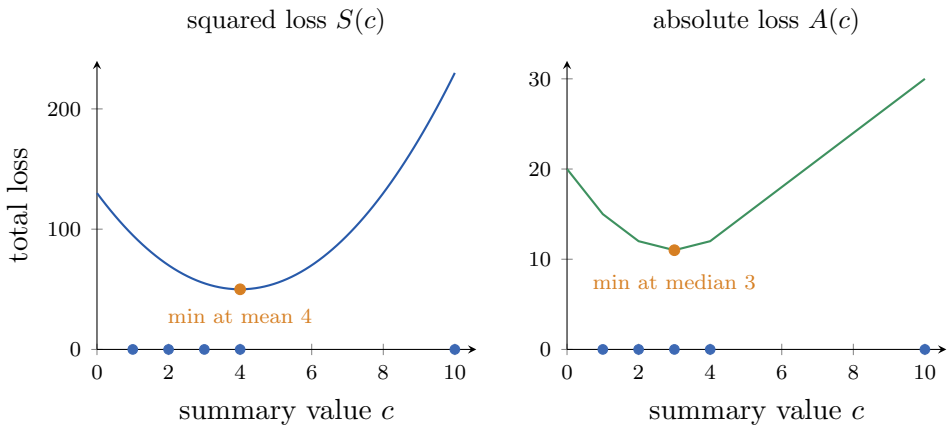


Figure 2.13: Total loss if every value in the data $1, 2, 3, 4, 10$ (blue dots on the c axis) is summarised by the single number c. Left: squared loss is a smooth parabola whose lowest point is the mean, 4; the outlier at 10 pulls it right. Right: absolute loss is a broken line with a corner at each data value, lowest at the median, 3.

Connection

This is the most important idea in the chapter for machine learning. Choosing a loss function chooses which "center" your model predicts.

- A regression model trained with **mean squared error** (MSE, L_2 loss) learns to predict the conditional *mean* of the target: the average label among inputs like the one in front of it. Linear regression (Chapter 12) is this idea with c replaced by a line.
- A model trained with **mean absolute error** (MAE, L_1 loss) learns the conditional *median*, and is less sensitive to outlying labels.
- **Quantile (pinball) loss** generalises L_1 to predict any quantile, such as p90 latency (Chapter 3 defines quantiles).
- **Huber loss** is quadratic near zero and linear far away: mean-like for small errors, median-like for outliers.

Picking the prediction that makes the average loss over the training data as small as possible is called *empirical risk minimisation*, and $\bar{x}$ is its simplest instance: a model that predicts one constant for everyone. Chapter 15 returns to it.

2.7 Other measures of spread: range, IQR, MAD

The SD inherits the mean's sensitivity to outliers. Three alternatives trade efficiency for robustness or simplicity. Take nine page-load times in ms, one of which hit a cold cache:

$$12,\ 14,\ 15,\ 15,\ 17,\ 18,\ 20,\ 23,\ 90.$$

Mean ≈ 24.9, median 17, SD $s \approx 24.6$. Without the 90 the SD is about 3.5: one point inflated it sevenfold.

Definition 2.7 (Range)**.** range $= x_{(n)} - x_{(1)}$.

Here $90 - 12 = 78$. The range uses only the two most extreme values, so it has the worst possible robustness and it grows with n (more data, more chances for an extreme). It is useful for sanity checks ("are all values physically possible?"), not as a summary.

Definition 2.8 (Interquartile range)**.** IQR $= Q_3 - Q_1$, where Q_1 and Q_3 are the first and third quartiles: the values below which 25% and 75% of the data fall.

With nine sorted values the 25% point is the 3rd value and the 75% point is the 7th (the default convention in NumPy and R; Chapter 3 compares conventions). So $Q_1 = 15$, $Q_3 = 20$, and IQR $= 5$. The IQR is the width of the middle half of the data. Its breakdown point is 25%.

Definition 2.9 (Median absolute deviation)**.** MAD $=$ median $\big(|x_i - \text{median}(x)|\big)$: the median distance from the median.

The absolute distances from 17 are $5, 3, 2, 2, 0, 1, 3, 6, 73$. Sorted: $0, 1, 2, 2, \mathbf{3}, 3, 5, 6, 73$. The MAD is 3. It is a median of a median, with breakdown point 50%.

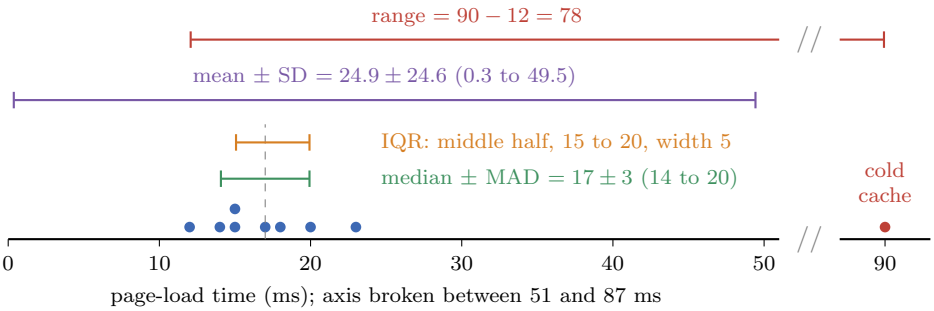


Figure 2.14: Four spread measures for the nine page-load times (one dot per page load; 15 occurs twice). The range and the SD band are stretched by the single 90 ms value: the SD band even reaches down to 0.3 ms, where no data lie. The IQR and MAD bars cover where the bulk of the data actually sits.

Remark 2.1. For Normal data the MAD is about $0.674\,\sigma$ and the IQR about $1.35\,\sigma$. So $1.4826 \times$ MAD and IQR $/1.35$ are robust estimates of σ. Here $1.4826 \times 3 \approx 4.4$, close to the SD of the eight clean values (3.5) and nowhere near 24.6. Robust outlier detectors use this: flag a point if it is more than about $3 \times 1.4826 \times$ MAD from the median.

```
import numpy as np
from scipy import stats
x = np.array([12, 14, 15, 15, 17, 18, 20, 23, 90])
x.mean(), np.median(x)              # 24.89, 17.0
stats.trim_mean(x, 0.1)             # drops 0 from each end here (floor(0.9)=0)
x.std(ddof=1)                       # 24.64  (ddof=1 -> divide by n-1)
q1, q3 = np.percentile(x, [25, 75]) # 15.0, 20.0
stats.median_abs_deviation(x)       # 3.0
```

2.8 Standardising: z-scores

Alice scores 82 on an exam where the class mean is 70 and the SD is 8. Bob scores 90 on a different exam with mean 80 and SD 5. Who did better relative to their class? Raw points cannot answer

this: the exams have different centers and spreads. Measure each score in units of its own SD from its own mean.

Definition 2.10 (z-score)**.** The **z-score** of x in a data set with mean $\bar{x}$ and SD s is

$$z = \frac{x - \bar{x}}{s}.$$

It says how many standard deviations x lies above ($z > 0$) or below ($z < 0$) the mean.

$$z_{\text{Alice}} = \frac{82 - 70}{8} = 1.5, \qquad z_{\text{Bob}} = \frac{90 - 80}{5} = 2.0.$$

Bob is further above his class.

Pause

Pause and think: a third student scores 61 on Exam A. What is her z-score, and what does its sign tell you? ($z = (61 - 70)/8 \approx -1.1$: about one SD below the mean.)

Standardising a whole data set (replacing every x_i by its z_i) produces values with mean 0 and SD 1, whatever the original units; Section 2.11 shows why.

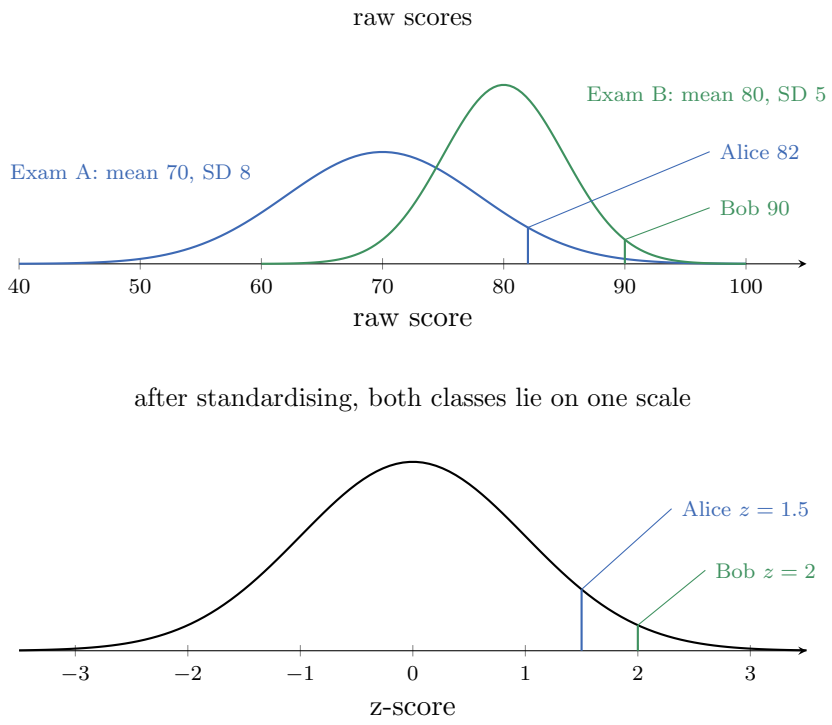


Figure 2.15: Top: two exams with different means and spreads. Bob's 90 and Alice's 82 cannot be compared directly. Bottom: in SD units from each class mean, Bob ($z = 2$) is further into the upper tail than Alice ($z = 1.5$).

Connection

Feature standardisation in ML (`StandardScaler` in scikit-learn) is exactly this: subtract each feature's mean, divide by its SD. Features measured in metres and in milligrams end up on the same scale, so gradient descent converges faster and size penalties on coefficients treat all features equally. Fit the mean and SD on the training set only, then apply the same numbers to test data.

2.9 The 68–95–99.7 rule

What does "$z = 2$" mean in terms of how unusual a value is? It depends on the shape. The most common shape is the **Normal** (or Gaussian) curve: a symmetric bell, highest at the mean and falling off quickly on both sides. Heights, measurement errors, and averages of many small effects tend to look like this (Chapter 6 explains why; Probability From the Ground Up derives it). For data whose histogram is roughly bell-shaped, the answer is fixed:

- about 68% of values lie within 1 SD of the mean,
- about 95% lie within 2 SD,
- about 99.7% lie within 3 SD.

The exact Normal values are 68.27%, 95.45%, and 99.73%.

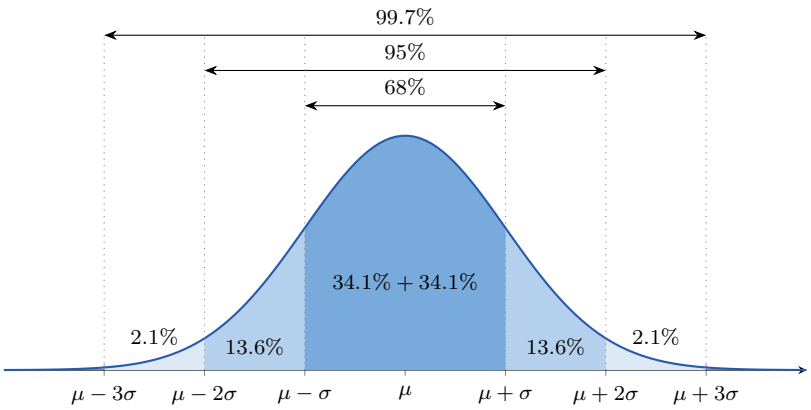


Figure 2.16: The 68–95–99.7 rule for a Normal distribution, with the exact share in each band. Darker bands are closer to the mean. Each tail beyond 2σ holds about 2.3%; the cut-off that leaves exactly 2.5% in each tail is 1.96σ, the "1.96" in 95% confidence intervals (Chapter 7).

Example 2.2. Adult heights in a population are roughly Normal with mean 170 cm and SD 7 cm. Then about 68% of adults are between 163 and 177 cm, about 95% between 156 and 184 cm, and a height of 191 cm ($z = 3$) is rarer than 0.15% (one tail of the 0.3% outside ± 3 SD).

Common trap

The rule is for bell-shaped data only. For skewed data such as latencies it can be badly wrong: a "3-sigma" alert threshold on a heavy-tailed metric fires far more often than 0.3% of the time. Plot the histogram before you use the rule.

2.10 Chebyshev's inequality: the guarantee for any data

If you do not know the shape, is there anything you can say? Yes, and it needs only the mean and SD.

Theorem 2.3 (Chebyshev's inequality)**.** *For any data set (or any distribution with finite variance) and any $k > 1$, the fraction of values more than k standard deviations from the mean is at most $1/k^2$. Equivalently, at least $1 - 1/k^2$ lie within k SDs.*

Why it works

Let p be the fraction of the n values with $|x_i - \mu| \geq k\sigma$ (using the divide-by-n σ of the data). Each such value contributes at least $(k\sigma)^2$ to the sum of squares, and the others contribute at least 0. So

$$n\sigma^2 = \sum_i (x_i - \mu)^2 \;\geq\; (pn)(k\sigma)^2 \quad\Longrightarrow\quad p \leq \frac{1}{k^2}.$$

The variance is a budget of squared distance. Far-away points are expensive, so only a few can afford to be far.

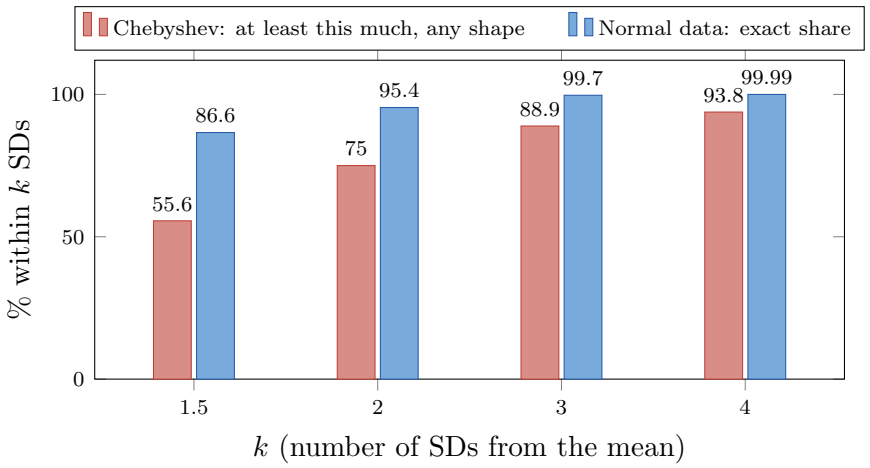


Figure 2.17: Share of data within k SDs of the mean. Red: the least that Chebyshev guarantees for data of any shape. Blue: the actual share for Normal data. The guarantee is always met, but conservatively: at $k = 2$ it promises at least 75%, while Normal data have 95%.

For the nine page-load times, $\bar{x} \approx 24.9$ and the population-style SD is $\sigma \approx 23.2$, so the point 90 is at $z \approx 2.8$. Chebyshev says at most $1/2.8^2 \approx 13\%$ of points can be that far out; one of nine (11%) is. Chebyshev is tight only for contrived distributions, but it never fails. That is the property you want

when you know nothing about the shape. It also gives a one-line proof of the law of large numbers (Chapter 6), the fact that a sample mean settles down to the population mean as n grows.

2.11 Linear transformations of data

Unit conversions, currency conversions, and standardisation all have the form $y_i = a + bx_i$. How do the summaries change?

Theorem 2.4 (Shift and scale)**.** *If $y_i = a + b\,x_i$ for all i, then*

$$\bar{y} = a + b\,\bar{x}, \qquad s_y^2 = b^2 s_x^2, \qquad s_y = |b|\,s_x.$$

The median transforms like the mean, and the IQR, MAD, and range like the SD.

Why it works

Averaging is linear: $\frac{1}{n}\sum(a + bx_i) = a + b\bar{x}$. For the spread, look at the deviations: $y_i - \bar{y} = (a+bx_i) - (a+b\bar{x}) = b(x_i - \bar{x})$. The shift a cancels completely, because adding a constant moves every point and the center together. The scale b multiplies every deviation, so it multiplies the squared deviations by b^2 and the SD by $|b|$.

Example 2.3 (Celsius to Fahrenheit)**.** Five daily highs in °C: $18, 20, 21, 23, 28$. Mean 22°C, SD $s \approx 3.81$°C. With $F = 32 + 1.8\,C$ the readings become $64.4, 68, 69.8, 73.4, 82.4$. Without recomputing,

$$\bar{F} = 32 + 1.8 \times 22 = 71.6°\text{F}, \qquad s_F = 1.8 \times 3.81 \approx 6.85°\text{F}.$$

The 32 does not touch the SD: a difference of temperatures has no offset.

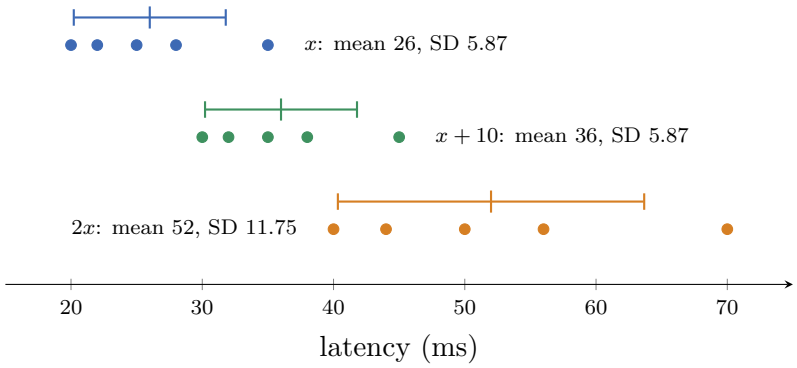


Figure 2.18: The five latencies (top), the same plus 10 ms (middle), and doubled (bottom). Each bar spans mean ±1 SD, with a tick at the mean. Adding 10 slides everything right and leaves the bar's width unchanged; doubling doubles both the mean and the width.

Standardisation is the special case $a = -\bar{x}/s$, $b = 1/s$: $z_i = (x_i - \bar{x})/s$. The theorem gives $\bar{z} = -\bar{x}/s + \bar{x}/s = 0$ and $s_z = s/s = 1$.

Common trap

The shift-and-scale rule is for *linear* maps only. For a nonlinear transformation such as log, the mean of the logs is not the log of the mean. (The median does commute with any increasing function: the median of $\log x$ is log of the median of x, because ranks are preserved.)

2.12 The mean of grouped data

Dashboards and reports often give only a frequency table, not the raw values. Fifty API response times were binned:

bin (ms)	midpoint m_j	count f_j	$f_j m_j$
0–100	50	12	600
100–200	150	25	3750
200–300	250	8	2000
300–500	400	5	2000
total		50	8350

Pretend every value in a bin sits at the bin's midpoint. Then

$$\bar{x} \approx \frac{\sum_j f_j m_j}{\sum_j f_j} = \frac{8350}{50} = 167 \text{ ms}.$$

This is a **weighted mean**: each midpoint is weighted by how many observations it represents. The same idea gives a grouped variance,

$$s^2 \approx \frac{1}{n-1}\sum_j f_j (m_j - \bar{x})^2 = \frac{498{,}050}{49} \approx 10{,}164, \qquad s \approx 101 \text{ ms}.$$

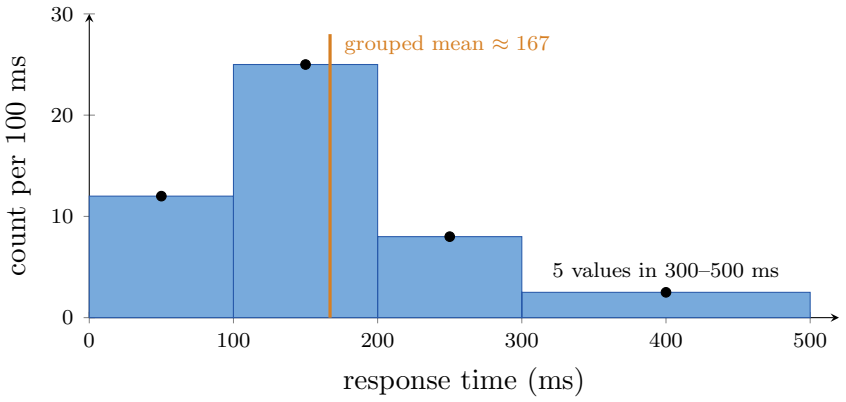


Figure 2.19: Histogram of the grouped response times. The last bin is twice as wide, so its height is the count per 100 ms ($5/2 = 2.5$) to keep area proportional to count. Dots mark the bin midpoints used to approximate the mean.

The approximation is exact if values are spread symmetrically within each bin, and off otherwise. In the long right tail of skewed data the histogram falls across each bin, so values in a bin cluster near its left edge and the midpoint rule tends to overestimate. The grouped median can be read off by finding the bin containing the 25th and 26th values: the 100–200 bin (values 13 through 37).

Connection

The weighted mean $\sum_j w_j m_j$ with $w_j = f_j/n$ is an expectation in disguise: the w_j are probabilities that sum to 1, and the formula is $\mathbb{E}[X] = \sum_x x\,p(x)$. When an ML paper writes $\mathbb{E}_{x\sim p}[f(x)]$, read it as "the p-weighted average of f." The sample mean is the special case where each of the n data points gets weight $1/n$.

2.13 Choosing a summary

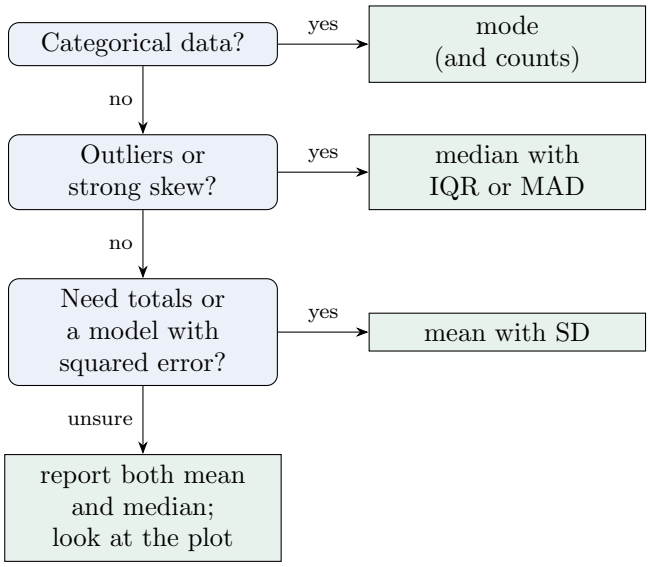


Figure 2.20: A rough guide for picking a center and spread. The pairing matters: the SD measures spread around the mean, and the IQR or MAD measure spread around the median.

When in doubt, compute both the mean and the median. If they are close, the data are roughly symmetric and either will do. If they differ a lot, that difference is itself the finding: the distribution is skewed or has outliers, and a single number will mislead someone. Chapter 3 goes beyond a single center and spread: quantiles such as p90 and p99, box plots that draw the IQR, and measures of shape.

2.14 Summary

measure	formula	minimises / meaning	breakdown
mean $\bar{x}$	$\frac{1}{n}\sum x_i$	$\sum(x_i - c)^2$	$1/n$
median	middle sorted value	$\sum\lvert x_i - c\rvert$	50%
α-trimmed mean	mean of middle	between the two	α
mode	most frequent	works for categories	–
variance s^2	$\frac{1}{n-1}\sum(x_i - \bar{x})^2$	unbiased for σ^2	$1/n$
SD s	$\sqrt{s^2}$	typical distance, original units	$1/n$
IQR	$Q_3 - Q_1$	width of middle half	25%
MAD	$\text{median}\lvert x_i - \text{median}(x)\rvert$	robust spread	50%
range	$x_{(n)} - x_{(1)}$	sanity check only	$1/n$

- Deviations from the mean sum to zero: the mean is the balance point.
- Skew pulls the mean toward the long tail; for right skew, mode $<$ median $<$ mean.
- $\mathbb{E}\left[\sum(X_i - \bar{X})^2\right] = (n-1)\sigma^2$, because $\bar{X}$ is fitted to the data. Hence divide by $n-1$.
- Squared loss leads to the mean; absolute loss leads to the median. Your choice of ML loss function chooses which one your model predicts.
- $z = (x - \bar{x})/s$ measures distance in SD units. Bell-shaped data: 68/95/99.7% within 1/2/3 SDs. Any data: at least $1 - 1/k^2$ within k SDs.
- $y = a + bx$ gives $\bar{y} = a + b\bar{x}$ and $s_y = |b|s_x$.
- Grouped mean: $\sum f_j m_j / \sum f_j$, a weighted average of midpoints.

2.15 Exercises

1. For the data $3, 5, 7, 7, 8$, compute the mean, median, mode, sample variance, and SD by hand using a deviations table.
2. A data set has $n = 10$ values. What is the smallest number of values you must replace to make the median arbitrarily large? What about the mean?
3. Prove that $\sum_{i=1}^{n}(x_i - \bar{x}) = 0$ for any data.
4. A city's daily highs over a month have mean 15°C and SD 4°C. Give the mean, SD, and variance in °F ($F = 32 + 1.8C$).
5. An IQ-style test has mean 100 and SD 15, and scores are roughly Normal. What is the z-score of 130? Using the 68–95–99.7 rule, roughly what share of people score above 130?
6. Without knowing the shape of a distribution, what is the minimum fraction of values within 2.5 SDs of the mean?

7. With $n = 2$ observations X_1, X_2 (independent, variance σ^2), show that $s^2 = (X_1 - X_2)^2/2$, and verify directly that $\mathbb{E}[s^2] = \sigma^2$.
8. Estimate the mean from the grouped data: bin 0–10 has 4 values, 10–20 has 10 values, 20–40 has 6 values.
9. For the data $1, 2, 3, 4$, which values of c minimise $\sum|x_i - c|$? Which value minimises $\sum(x_i - c)^2$?
10. Compute the MAD of $2, 4, 4, 5, 6, 9, 30$. Why is it so much smaller than the SD?

Answers

1. Mean $30/5 = 6$; median 7; mode 7. Deviations $-3, -1, 1, 1, 2$, squares $9, 1, 1, 1, 4$, sum 16. $s^2 = 16/4 = 4$, $s = 2$.
2. Five (half). With 5 huge values they occupy sorted positions 6–10, so the 6th value, and hence the median, is huge; with only 4 the 5th and 6th values are untouched. For the mean, one value suffices.
3. $\sum(x_i - \bar{x}) = \sum x_i - n\bar{x} = n\bar{x} - n\bar{x} = 0$.
4. Mean $32 + 1.8 \times 15 = 59°$F; SD $1.8 \times 4 = 7.2°$F; variance $7.2^2 = 51.84$ $(°\text{F})^2$.
5. $z = (130 - 100)/15 = 2$. About 5% lie beyond ± 2 SD, half of that above: roughly 2.5% (exact Normal value 2.3%).
6. $1 - 1/2.5^2 = 1 - 0.16 = 0.84$, so at least 84%.
7. $\bar{X} = (X_1 + X_2)/2$, so $X_1 - \bar{X} = (X_1 - X_2)/2 = -(X_2 - \bar{X})$ and the sum of squares is $2 \cdot (X_1 - X_2)^2/4$; dividing by $n - 1 = 1$ gives $(X_1 - X_2)^2/2$. Since $X_1 - X_2$ has mean 0 and variance $2\sigma^2$, $\mathbb{E}[(X_1 - X_2)^2] = 2\sigma^2$ and $\mathbb{E}[s^2] = \sigma^2$.
8. Midpoints $5, 15, 30$: $(4 \cdot 5 + 10 \cdot 15 + 6 \cdot 30)/20 = 350/20 = 17.5$.
9. Every c in $[2, 3]$ minimises the absolute loss (two points on each side, so the slope is 0 there); the squared loss is minimised only at $\bar{x} = 2.5$.
10. Median 5; absolute deviations $3, 1, 1, 0, 1, 4, 25$; sorted $0, 1, 1, 1, 3, 4, 25$; MAD = 1. The SD squares the deviation of the outlier 30, so that one point dominates it (here $s \approx 9.7$); the MAD only uses the middle deviation.

Chapter 3

Quantiles, Box Plots and Shape

Chapter 2 summarised a data set with two numbers: a centre (mean or median) and a spread (standard deviation or IQR). Two numbers cannot tell you that a service is fine for most users and terrible for one in a hundred. This chapter adds the tools that can: quantiles, which locate any position in the data; box plots and ECDFs, which draw those positions; and a vocabulary for shape (skew, heavy tails, several bumps) with the plots that reveal it.

Here is the running data set. A service handled twelve requests, with these latencies in milliseconds, already sorted:

$$12,\ 14,\ 15,\ 16,\ 17,\ 18,\ 20,\ 22,\ 25,\ 31,\ 48,\ 120.$$

The mean is $358/12 \approx 29.8$ ms and the median is $(18 + 20)/2 = 19$ ms. Only three of the twelve requests were slower than the mean. One slow request (120 ms) dragged the mean far above what a typical request experienced.

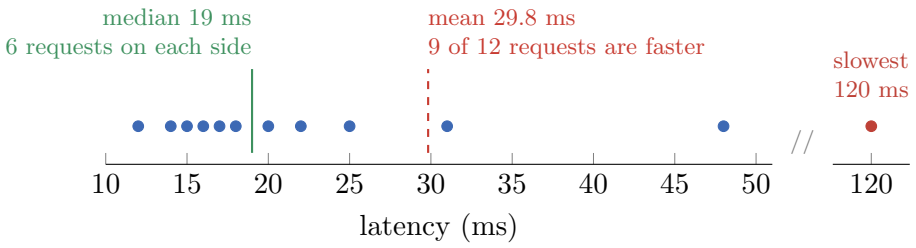


Figure 3.1: The twelve latencies, one dot per request. The axis is broken between 51 and 115 ms (no request falls there) so that the eleven faster dots stay separate. Nine requests sit left of the mean: the single 120 ms request pulls the mean right, while the median stays with the bulk of the data.

3.1 Percentiles and quantiles

The median splits the data in half. Nothing stops us from splitting at other points.

Definition 3.1 (Quantile, percentile)**.** For $0 < p < 1$, a **p-quantile** of a data set is a number q such that roughly a fraction p of the data lies at or below q and a fraction $1 - p$ lies above it. The **$100p$-th percentile** is the same thing on a 0–100 scale: the 0.9-quantile is the 90th percentile, written p90.

Picture walking along the sorted data from left to right with a counter that tracks the fraction of points you have passed. The p-quantile is where the counter reads p. The median is the 0.5-quantile. The **quartiles** are the 0.25-, 0.5-, and 0.75-quantiles, called Q_1, Q_2, Q_3: they cut the data into four equal-sized groups.

3.1.1 The trouble with "roughly"

With twelve points, what is p90? Ten percent of twelve is 1.2 points. No data value has exactly 1.2 points above it. The word "roughly" in the definition is hiding a real choice, and different software makes it differently.

The most common rule, used by default in numpy, R, Excel's `PERCENTILE.INC` and pandas, places the sorted values at evenly spaced positions and draws straight lines between them.

Definition 3.2 (Linear-interpolation quantile)**.** Sort the data as $x_{(1)} \le x_{(2)} \le \cdots \le x_{(n)}$, where $x_{(i)}$ is the i-th smallest value (the **i-th order statistic**). Put $x_{(i)}$ at position $(i-1)/(n-1)$ on a 0-to-1 scale, so the minimum sits at 0 and the maximum at 1. Compute

$$h = (n-1)p + 1, \qquad k = \lfloor h \rfloor, \qquad f = h - k.$$

Then

$$Q(p) = x_{(k)} + f\left(x_{(k+1)} - x_{(k)}\right).$$

In words: h is a fractional rank. If $h = 10.9$, go to the 10th value and then 90% of the way to the 11th.

Example 3.1 (Quartiles and p90 of the twelve latencies)**.** Here $n = 12$, so $h = 11p + 1$.

- Q_1: $h = 11(0.25) + 1 = 3.75$. Go to $x_{(3)} = 15$, then 75% of the way to $x_{(4)} = 16$: $Q_1 = 15 + 0.75(1) = 15.75$.
- Median: $h = 6.5$. Halfway from $x_{(6)} = 18$ to $x_{(7)} = 20$: 19.
- Q_3: $h = 9.25$. From $x_{(9)} = 25$, a quarter of the way to $x_{(10)} = 31$: $Q_3 = 25 + 0.25(6) = 26.5$.
- p90: $h = 10.9$. From $x_{(10)} = 31$, 90% of the way to $x_{(11)} = 48$: $31 + 0.9(17) = 46.3$.

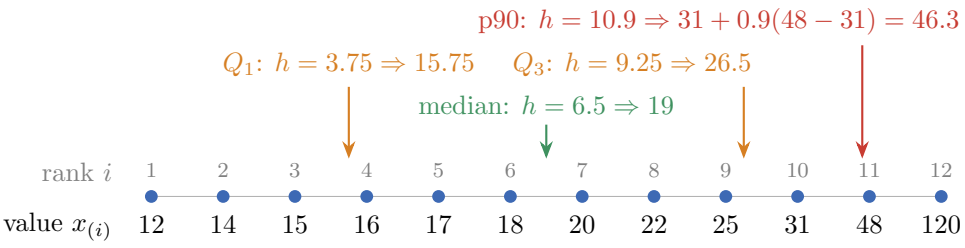


Figure 3.2: Linear interpolation treats rank as a continuous scale. Each arrow marks a fractional rank h; the quantile lies the same fraction of the way between the two neighbouring values. For p90, $h = 10.9$ sits 90% of the way from rank 10 (31 ms) to rank 11 (48 ms).

3.1.2 Why numpy has nine methods

The linear rule is one choice. Others put $x_{(i)}$ at position $i/(n+1)$, or at $(i-0.5)/n$, or refuse to interpolate and return an actual data value. Hyndman and Fan (1996) catalogued nine such rules, and `numpy.quantile` exposes all of them through its `method` argument. Table 3.1 shows what several of them return for our twelve latencies.

numpy `method`	position of $x_{(i)}$	R type	Q_1	median	Q_3	p90
`inverted_cdf`	no interpolation	1	15	18	25	48
`lower`	round rank down	–	15	18	25	31
`nearest`	round rank to nearest	–	16	20	25	48
`hazen`	$(i-0.5)/n$	5	15.5	19	28	69.6
`weibull`	$i/(n+1)$	6	15.25	19	29.5	98.4
`linear` (default)	$(i-1)/(n-1)$	7	15.75	19	26.5	46.3

Table 3.1: Six quantile rules applied to the same twelve numbers. They nearly agree in the middle and disagree wildly at p90, where the data are sparse.

The `weibull` p90 is more than double the `linear` p90. Check it by hand: with positions $i/(n+1)$ the fractional rank is $h = (n+1)p = 13(0.9) = 11.7$, so $Q = x_{(11)} + 0.7\,(x_{(12)} - x_{(11)}) = 48 + 0.7(72) = 98.4$. The rule reaches further into the gap between 48 and 120 because it reserves some probability for values beyond the largest observation. Figure 3.3 draws the two rules side by side.

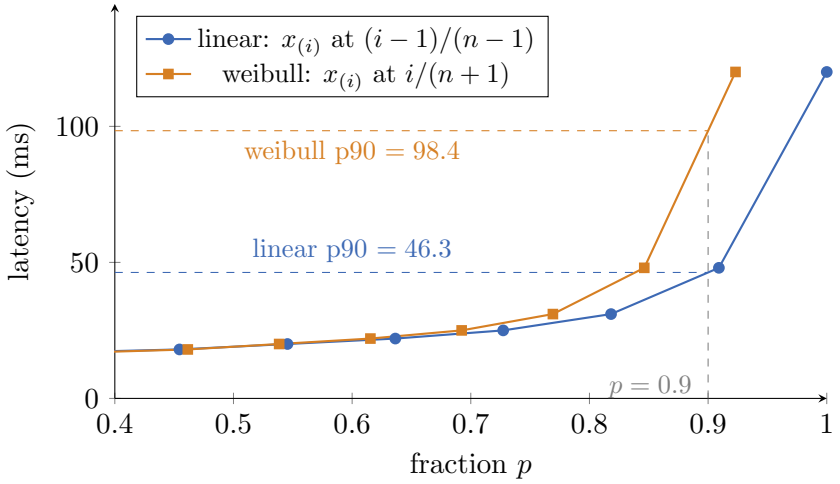


Figure 3.3: Two quantile rules for the twelve latencies (upper half shown). Each rule pins the sorted values to different fractions p and joins them with straight lines. Weibull pins 120 ms at $p = 12/13 \approx 0.92$ rather than at 1, so its steep 48-to-120 segment is already underway at $p = 0.9$: the two p90 values differ by a factor of two.

None of these rules is wrong. They answer slightly different questions ("which actual data value?", "where, on average, does the i-th smallest of n draws sit?") and they all converge to the same answer as n grows. The gap matters when n is small or p is extreme. A p99 computed from 50 requests rests on the one or two largest values, and the method choice can swing it by a factor of two.

Common trap

Two teams compute "p90 latency" from the same log and get different answers. Before debugging the pipeline, check which quantile method each tool uses. Spreadsheets, databases and monitoring systems do not all default to numpy's `linear`.

```
import numpy as np
x = np.array([12,14,15,16,17,18,20,22,25,31,48,120])
np.quantile(x, [0.25, 0.5, 0.75, 0.9])          # 15.75 19 26.5 46.3
np.quantile(x, 0.9, method="weibull")           # 98.4
np.percentile(x, 90)                            # same as quantile(x, 0.9)
```

Pause

Without computing: as $n \to \infty$ with the same underlying population, do the `linear` and `weibull` p90 values move closer together or further apart? (Closer. The positions $(i-1)/(n-1)$ and $i/(n+1)$ differ by at most $1/(n+1)$.)

3.2 The five-number summary and the box plot

Five quantiles give a compact portrait of a data set:

$$\text{min}, \quad Q_1, \quad \text{median}, \quad Q_3, \quad \text{max}.$$

For the latencies (linear method): 12, 15.75, 19, 26.5, 120. The middle half of the requests took between 15.75 and 26.5 ms. The **interquartile range** is the width of that middle half:

$$\text{IQR} = Q_3 - Q_1 = 26.5 - 15.75 = 10.75 \text{ ms}.$$

The five numbers already tell a story about shape. The distance from the median up to Q_3 (7.5 ms) is more than double the distance down to Q_1 (3.25 ms), and the max is 93.5 ms above Q_3 while the min is only 3.75 ms below Q_1. The data stretch to the right.

3.2.1 Drawing the box plot

A **box plot** turns the five-number summary into a picture, with one extra rule for flagging unusual points.

1. Draw a box from Q_1 to Q_3. Its length is the IQR.
2. Draw a line across the box at the median.
3. Compute the **fences**, invisible boundaries 1.5 IQRs beyond the box:

$$\text{lower fence} = Q_1 - 1.5 \text{ IQR}, \qquad \text{upper fence} = Q_3 + 1.5 \text{ IQR}.$$

4. Extend a **whisker** from each end of the box to the most extreme data point that is still inside the fences.

5. Plot every point beyond a fence individually. These are flagged as **outliers**: points far enough from the middle half to deserve a look.

For the latencies: $1.5 \times 10.75 = 16.125$, so the lower fence is $15.75 - 16.125 = -0.375$ and the upper fence is $26.5 + 16.125 = 42.625$. Nothing falls below the lower fence, so the lower whisker reaches the minimum, 12. The largest value inside the upper fence is 31, so the upper whisker stops there. The values 48 and 120 are plotted as outliers.

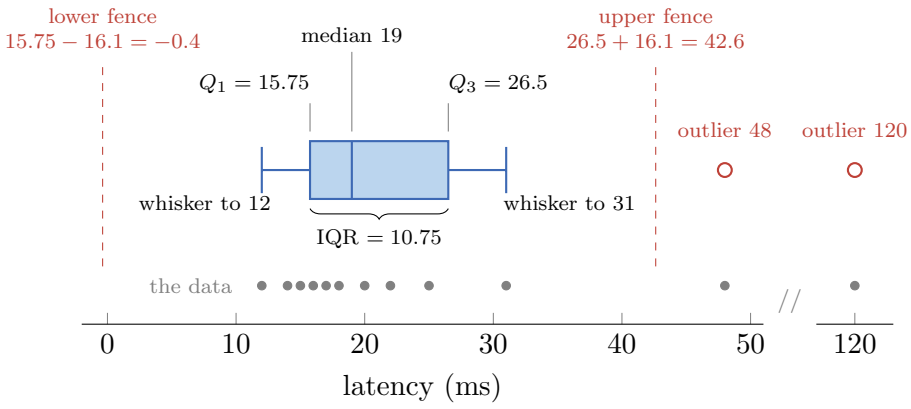


Figure 3.4: Anatomy of a box plot for the twelve latencies, with the raw data as grey dots underneath. The box spans the middle half (15.75 to 26.5 ms); the whiskers stop at the last data points inside the dashed fences (12 and 31 ms); 48 and 120 lie beyond the upper fence and are drawn individually. The axis is broken between 51 and 115 ms, where there are no data.

Why it works

Why 1.5? John Tukey chose it so that clean bell-shaped data would almost never trigger the rule. For a Normal distribution with standard deviation σ, the quartiles sit 0.674σ from the centre, so IQR $= 1.349\sigma$ and the upper fence is at $0.674\sigma + 1.5(1.349\sigma) \approx 2.70\sigma$. The chance of a Normal value beyond either fence is about 0.7%. A flagged point in bell-shaped data is therefore genuinely rare: roughly 7 per 1000 points.

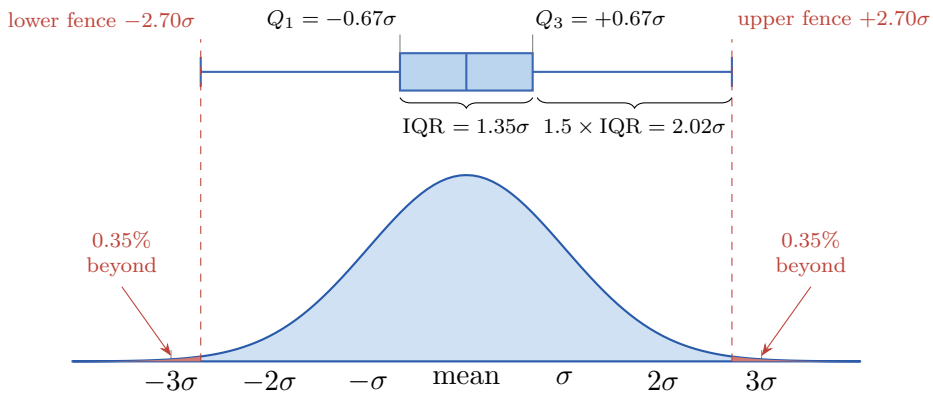


Figure 3.5: Tukey's rule applied to a whole Normal population. The box spans $\pm 0.67\sigma$, and the fences sit 1.5 IQRs further out, at $\pm 2.70\sigma$. Only the red slivers beyond the fences would be flagged: 0.35% on each side, 0.7% in total.

Common trap

"Outlier" in a box plot means "beyond the fences", not "error" or "delete me". For skewed data the rule fires constantly. Exponential data (a common model for waiting times) have $Q_1 = \ln(4/3)$ and $Q_3 = \ln 4$ in units of the mean, so the upper fence is $\ln 4 + 1.5 \ln 3 \approx 3.03$ and about 4.8% of all points lie beyond it. A service with 10,000 requests would show roughly 480 "outliers" that are simply its normal tail.

Two more conventions you will meet. Some textbooks compute Q_1 and Q_3 as the medians of the lower and upper halves of the data (Tukey's *hinges*). For our twelve points the halves are $\{12, \ldots, 18\}$ and $\{20, \ldots, 120\}$, giving 15.5 and 28: slightly different from 15.75 and 26.5, for the reason in the previous section. And some plotting libraries draw whiskers to the min and max with no outlier rule. Read the documentation before reading the whiskers.

3.2.2 Comparing groups side by side

The real power of the box plot is comparison. Put several box plots on one axis and your eye compares medians, spreads and tails at once. Below, 60 requests were sampled from each of three regions.

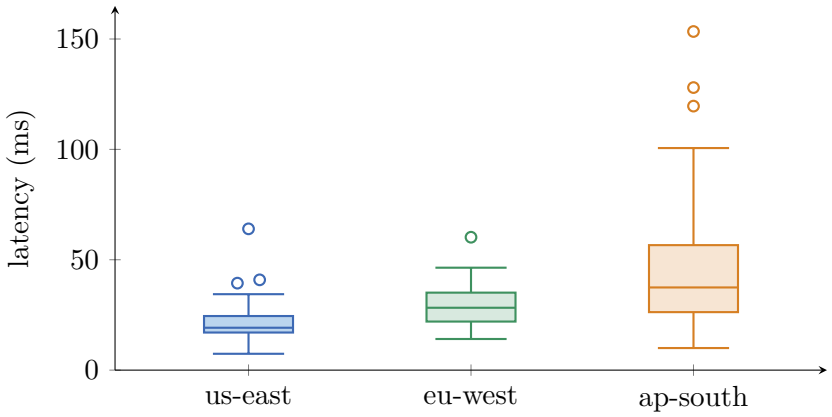


Figure 3.6: Latency by region, 60 requests each. ap-south has the highest median and by far the widest box: its typical request is slower and much less predictable. Every flagged point is on the high side, a sign of right skew.

Read it from the boxes outward. Medians: about 19, 28 and 37 ms. IQRs: 7.5, 13.1 and 30.4 ms. The ap-south box is four times as tall as us-east's, which matters more to users than the 18 ms difference in medians. Each region has a few flagged points, all on the high side. A table of means (21.4, 29.1, 45.6) would have hidden almost all of this.

3.3 The empirical CDF

A box plot keeps five numbers. The **empirical cumulative distribution function** (ECDF) keeps all of them.

Definition 3.3 (Empirical CDF)**.** For data $x_1, \dots, x_n$, the ECDF is the function

$$\hat{F}(t) = \frac{\#\{i : x_i \le t\}}{n},$$

the fraction of data points at or below t.

Picture sliding a vertical line from left to right across a dot plot. Each time it passes a data point, the ECDF jumps up by $1/n$. Between data points it is flat. It starts at 0 far to the left and ends at 1 far to the right. With our twelve latencies each jump is $1/12 \approx 0.083$. For example $\hat{F}(20) = 7/12 \approx 0.58$ because seven requests took 20 ms or less.

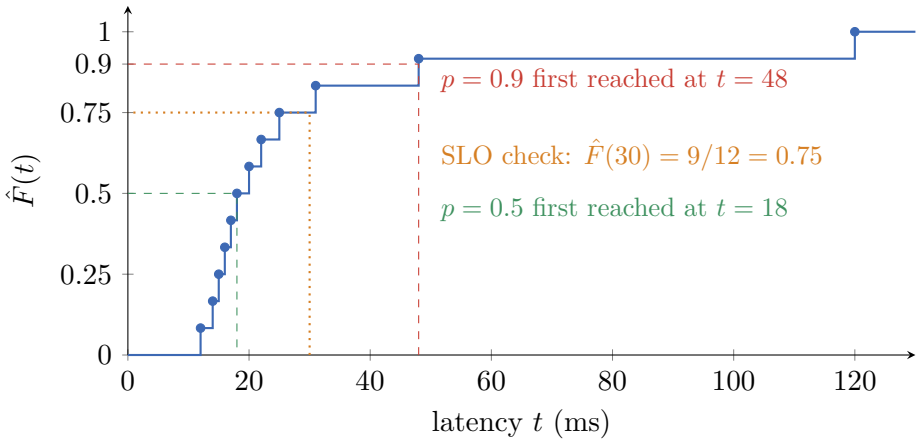


Figure 3.7: ECDF of the twelve latencies; each dot is one request, each step 1/12 high. Read a quantile by going across from p on the vertical axis until you hit the staircase, then down (red, green); read a "fraction under a threshold" by going up from t, then across (orange). The long flat step from 48 to 120 is the gap in the data that made the quantile methods disagree.

To read a quantile off the ECDF, go horizontally from height p until you meet the staircase, then drop to the horizontal axis. At $p = 0.9$ the staircase first reaches 0.9 at $t = 48$ (where it jumps from 0.83 to 0.92). That is the `inverted_cdf` answer in Table 3.1: the "no interpolation" rule is exactly "invert the ECDF". The interpolating rules replace the staircase with a sloped line through it, which is why they produce values between data points.

The ECDF has three advantages over a histogram. It needs no bin width. Every data point is visible. And quantiles, medians and "fraction below a threshold" are all read directly. A service-level objective (SLO), a promise such as "most requests finish under 30 ms", becomes the question "what fraction finished under 30 ms?", and the answer is $\hat{F}(30) = 9/12 = 0.75$, one glance.

Pause

Two ECDFs are drawn on one plot. Curve A lies everywhere to the left of curve B. Which data set has the smaller median? (A. At height 0.5, A is reached first.)

3.4 The quantile function: the CDF read backwards

The same picture works for a whole population, not just a sample. Think of a random variable X as the latency of the next request, before it happens. Its **cumulative distribution function** (CDF) is $F(x) = \mathbb{P}(X \le x)$: the probability that the request finishes in x ms or less. It is the population version of the ECDF: a curve rising from 0 to 1, smooth instead of a staircase. The ECDF of a large sample hugs it closely. (*Probability From the Ground Up* develops CDFs in full.)

Definition 3.4 (Quantile function)**.** The **quantile function** of X is

$$Q(p) = \min\{x : F(x) \ge p\}, \qquad 0 < p < 1,$$

the smallest value at which the CDF reaches p. When F is continuous and strictly increasing, Q is the inverse function F^{-1}.

The CDF answers "given a value, what fraction lies below it?" The quantile function answers the reverse: "given a fraction, what value has that fraction below it?" Their graphs are mirror images across the diagonal $y = x$.

Example 3.2 (Exponential latencies)**.** Suppose latency is exponential with mean 20 ms, so $F(x) = 1 - e^{-x/20}$ for $x \geq 0$. Solve $F(x) = p$ for x:

$$1 - e^{-x/20} = p \implies Q(p) = -20 \ln(1 - p).$$

The median is $Q(0.5) = 20 \ln 2 \approx 13.9$ ms, well below the mean of 20. The p99 is $Q(0.99) = 20 \ln 100 \approx 92.1$ ms, and the p99.9 is $20 \ln 1000 \approx 138.2$ ms. Each extra "nine" adds the same $20 \ln 10 \approx 46$ ms.

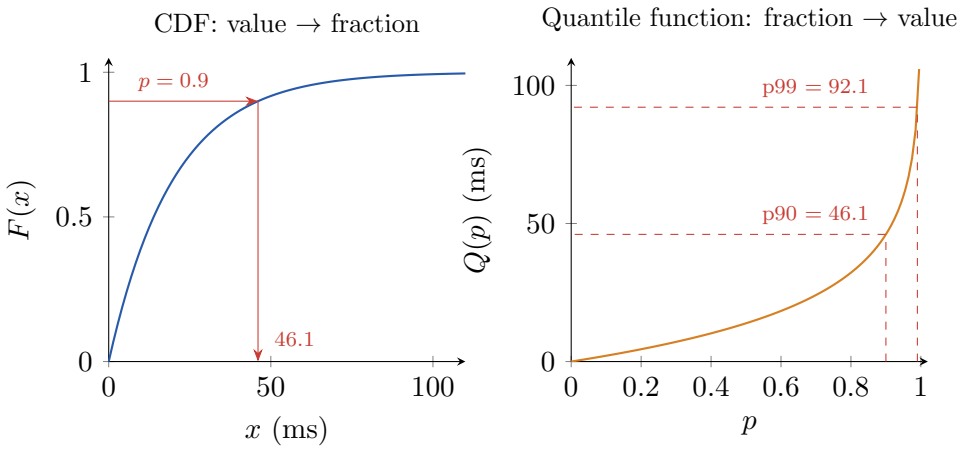


Figure 3.8: The exponential CDF with mean 20 ms (left) and its quantile function (right). Same curve, axes swapped. The quantile function shoots up as $p \to 1$: the tail quantiles are where the large values live.

Quantiles have one property the mean lacks: they pass straight through any increasing transformation. If g is increasing (such as log or $\sqrt{\ }$), then

$$Q_{g(X)}(p) = g\big(Q_X(p)\big),$$

because g preserves the order of the data, and quantiles only depend on order. The median of log-latency is the log of the median latency. The mean has no such rule: the mean of the logs is not the log of the mean.

Connection

The quantile function is also how computers draw random numbers from a given distribution. If U is uniform on $(0, 1)$, then $Q(U)$ has CDF F. This is **inverse transform sampling**: `-20*np.log(1-np.random.rand())` draws an exponential latency with mean 20. The proof is one line, $\mathbb{P}(Q(U) \leq x) = \mathbb{P}(U \leq F(x)) = F(x)$; see *Probability From the Ground Up* for details. The same idea powers quantile regression and the "pinball loss" used to train models that predict p90 instead of the mean.

3.5 Latency dashboards: p50, p90, p99

Production monitoring rarely shows a mean latency. It shows a few percentiles, usually p50, p90 and p99, sometimes p99.9. Here is why.

Take 200 requests from a typical web service. Figure 3.9 shows their histogram with four summary numbers marked.

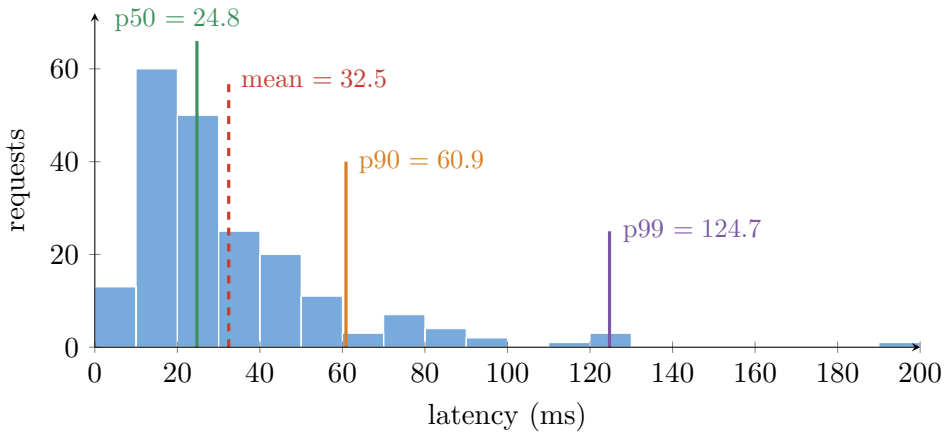


Figure 3.9: 200 request latencies. The mean (32.5 ms) is not a latency anyone typically gets: about two thirds of requests are faster. The p99 (124.7 ms) is five times the median and is invisible in both the mean and the median.

Each percentile answers a concrete question. p50: what does a typical request feel like? p90: what do the unlucky 1-in-10 get? p99: what is the experience of the slowest 1%? The mean answers none of these. It mixes the typical and the extreme into one number that describes neither.

3.5.1 Why averages lie about tails

Figure 3.10 shows a 30-minute dashboard, one point per minute, each summarising 600 requests. Between minutes 12 and 17, 2% of requests hit a slow code path that takes about 400 ms.

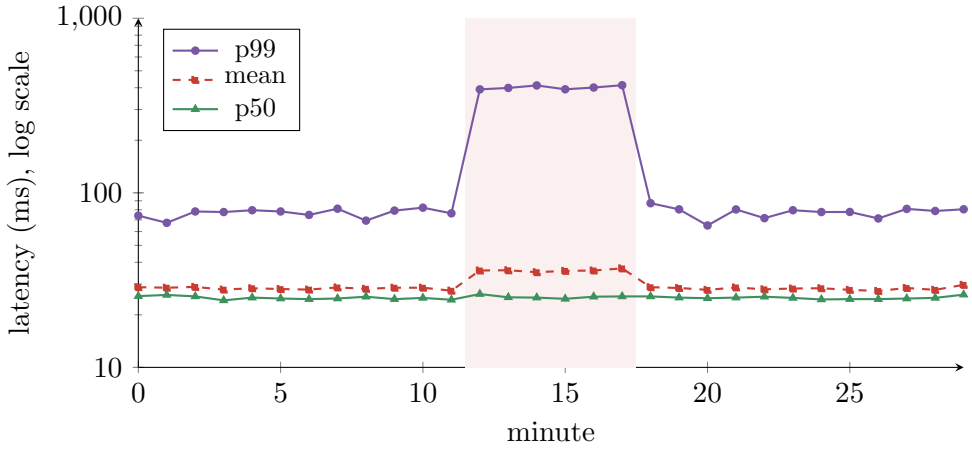


Figure 3.10: A slow path hits 2% of requests during the shaded minutes. The median does not move. The mean rises from about 28 to 36 ms, a change easy to dismiss as noise. The p99 jumps from about 77 to 400 ms, unmistakable.

The arithmetic explains the picture. Moving 2% of requests from about 28 ms to about 400 ms raises the mean by roughly $0.02 \times (400 - 28) \approx 7.4$ ms. The median cannot move at all, since 98% of requests are unchanged. The p99 sits right inside the slow 2%, so it jumps to about 400 ms.

Tails matter more than their size suggests because users make many requests. If loading one page fans out to 100 backend calls and the page waits for all of them, the chance that at least one call is slower than the backend's p99 is

$$1 - 0.99^{100} \approx 0.63.$$

Most page loads see a p99 event. The backend's tail becomes the user's typical experience. Figure 3.11 shows how fast this grows with the number of calls.

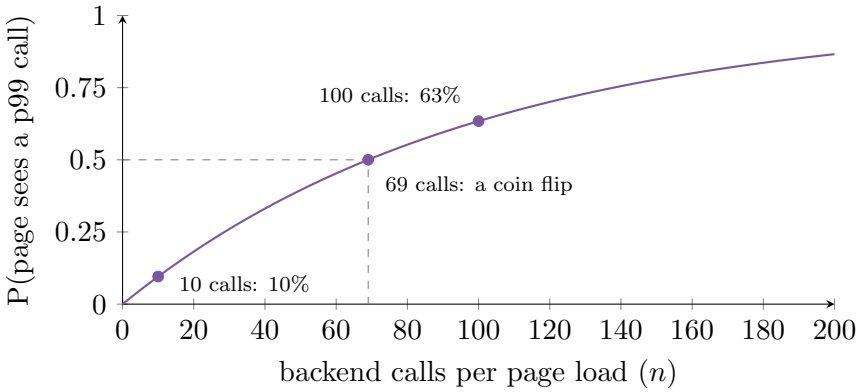


Figure 3.11: Chance that a page load waiting on n independent backend calls hits at least one call slower than the backend's p99, $1 - 0.99^n$. A 1-in-100 event per call becomes the usual case once a page makes more than about 70 calls.

Common trap

A green mean is not evidence that users are happy. A mean can hide a slow 1% entirely and a slow 5% largely. If you only have room for one latency number, use a high percentile, and always pair it with the request count it was computed from.

3.5.2 Percentiles do not average

A fleet of servers each reports its own p99. It is tempting to average them. Try the same move on medians with tiny numbers. Server A served requests of $1, 2, 3$ ms (median 2). Server B served $10, 11, 12, 13, 14$ ms (median 12). The average of the medians is 7. But the fleet's eight requests $1, 2, 3, 10, 11, 12, 13, 14$ have median $(10 + 11)/2 = 10.5$. The answer depends on how many requests each server handled and how their distributions overlap, and the two medians alone do not carry that information.

At scale the error is just as real. Suppose server A handles 9,000 fast requests (p99 = 52.0 ms) and server B handles 1,000 slow ones (p99 = 286.8 ms). The average of the two p99s is 169.4 ms. The true fleet p99, computed from all 10,000 requests, is 218.1 ms. And 3.5% of all requests exceed 169.4 ms, so the "averaged p99" is really about a p96.5.

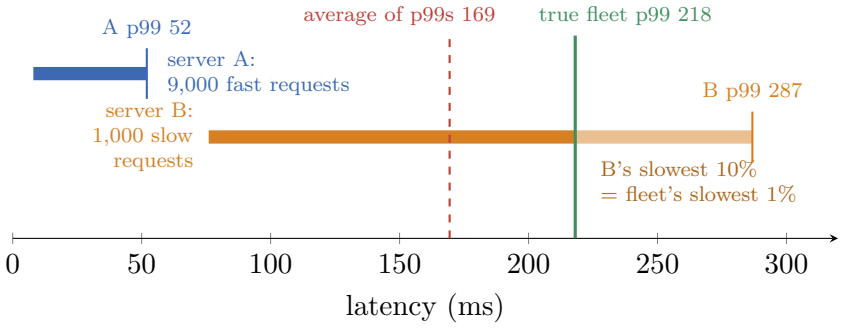


Figure 3.12: Each bar spans roughly a server's p1 to p99. The fleet's slowest 1% (100 of 10,000 requests) all come from B's slowest 10% (light orange), so the fleet p99 is B's p90, about 218 ms. Averaging the two p99s gives 169 ms, a value 3.5% of requests exceed.

The fix is to merge data, not summaries. Either combine the raw latencies, or have every server report a *histogram* (counts in fixed latency buckets) or a mergeable sketch such as a t-digest or HDR histogram. Bucket counts add across servers and across time; the combined percentile is then read off the merged counts. The same rule applies over time: the p99 for an hour is not the average of sixty per-minute p99s.

3.6 Shape: skewness and kurtosis

The latency histograms all lean the same way: a big pile on the left and a long tail trailing right. That lean has a name and a number.

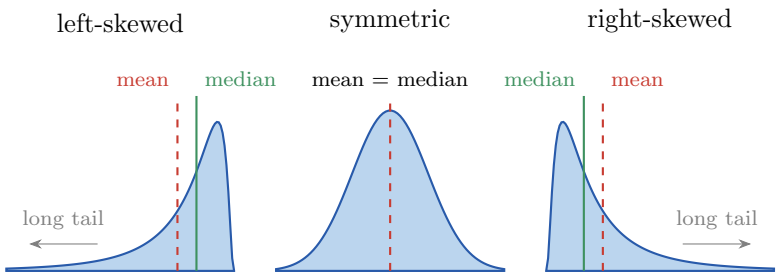


Figure 3.13: Skew is named after the direction of the long tail (grey arrows). In the right-skewed curve the few large values in the tail pull the mean (dashed red) to the right of the median (solid green); in the left-skewed curve, to the left. In a symmetric curve the two coincide.

A distribution is **right-skewed** (positively skewed) when its long tail points right, toward large values. Latencies, incomes, house prices, file sizes and city populations are right-skewed: they are bounded below (by zero) but not above. Exam scores on an easy test are **left-skewed**: most students near the top, a tail of low scores.

Definition 3.5 (Sample skewness)**.** With $\bar{x}$ the mean, let $m_k = \frac{1}{n}\sum_i (x_i - \bar{x})^k$ be the average k-th power of the deviations. The sample **skewness** is

$$g_1 = \frac{m_3}{m_2^{3/2}}.$$

Why the cube? Cubing keeps the sign of each deviation, so points above the mean push g_1 up and points below push it down. Cubing also exaggerates big deviations, so a long tail on one side wins. Dividing by $m_2^{3/2}$ (the cube of a standard deviation) makes the number unit-free: skewness in milliseconds and in seconds is the same. Symmetric data give $g_1 \approx 0$.

For the twelve latencies, $g_1 \approx 2.52$: strongly right-skewed. The single value 120 ms supplies all of it. Its cubed deviation, $(120 - 29.83)^3 \approx 733{,}000$, is larger than the whole sum of cubes ($\approx 718{,}000$). The nine fast requests contribute negative cubes totalling about $-20{,}900$, more than cancelling the $+6{,}000$ from 31 and 48.

Intuition

Skewness is a measure of tail imbalance, and it is fragile. One extreme value can dominate it, as it did here. For small samples treat g_1 as a description of the data in hand, not a stable property of the population. A box plot (whisker lengths, median position in the box) shows the same imbalance more robustly.

3.6.1 Kurtosis: how heavy are the tails?

Skewness compares the two tails. **Kurtosis** measures how heavy both tails are compared with a Normal distribution of the same standard deviation:

$$\text{excess kurtosis} = \frac{m_4}{m_2^2} - 3.$$

The fourth power amplifies large deviations even more than the cube, and ignores their sign. The "-3" is there because a Normal distribution has $m_4/m_2^2 = 3$, so excess kurtosis is 0 for a Normal, positive for heavier tails, negative for lighter ones.

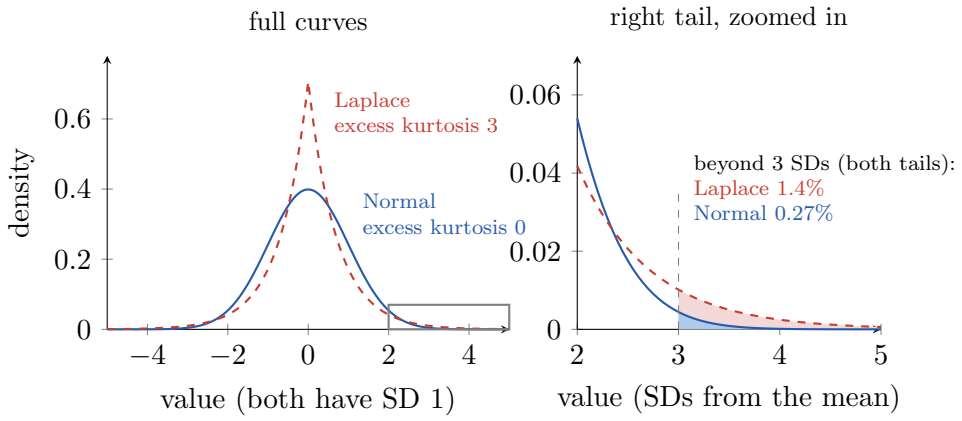


Figure 3.14: Two distributions with the same mean and standard deviation. Left: the Laplace has more mass near the centre *and* in the tails, with less in the shoulders. Right: the grey-boxed tail region magnified; beyond 3 SDs the Laplace curve (red shading) sits well above the Normal (blue), so extreme values are about five times as likely. That tail excess is what kurtosis measures.

Reference values: uniform -1.2, Normal 0, Laplace 3, and 6 for Student's t with 5 degrees of freedom (a heavy-tailed cousin of the Normal that Chapter 7 uses). Our twelve latencies have excess kurtosis 5.1, again almost entirely from the value 120 (it supplies over 99% of $\sum(x_i - \bar{x})^4$).

Common trap

Kurtosis is often described as "peakedness". That is misleading. Distributions with flat tops can have high kurtosis and pointed ones can have low kurtosis. What kurtosis detects is how often extreme values occur. For latency data it is a warning that p99 and p99.9 will be far from the median, and that the SD (Chapter 2) is being pushed around by a handful of points.

3.7 Taming right skew with a log transform

Right-skewed data are hard to look at. In Figure 3.9 most of the 200 requests crowd into the first few bins and the tail is a sparse scatter. Taking logarithms fixes this. A log scale turns ratios into distances: 10 to 100 ms is the same step as 100 to 1000 ms. Latency often builds up through multiplicative effects (a cache miss doubles the time, a slow disk triples it), and logs turn those multiplications into additions.

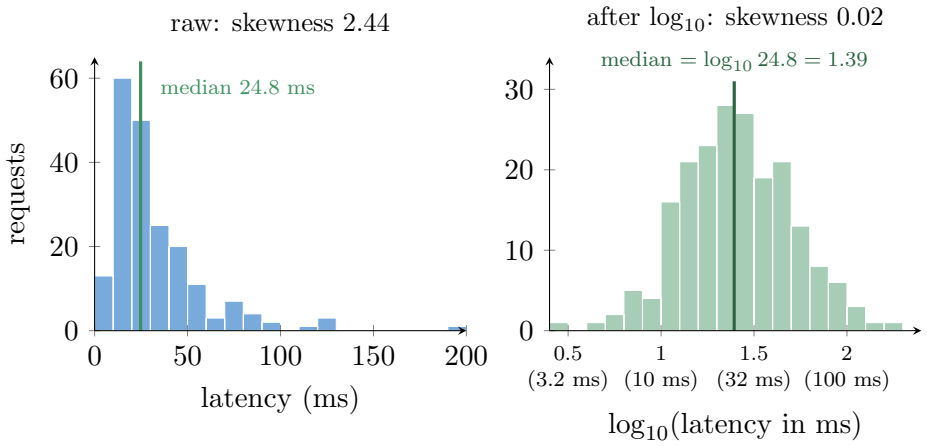


Figure 3.15: The same 200 latencies before and after taking $\log_{10}$. The long right tail becomes a roughly symmetric bell, and the median moves with the data ($\log_{10} 24.8 = 1.39$). Equal steps on the log axis are equal ratios: each tick is $\sqrt{10} \approx 3.2$ times the one before (3.2, 10, 32, 100 ms).

What the transform buys:

- **Visibility.** The bulk and the tail fit on one readable axis.
- **Symmetry.** Skewness drops from 2.44 to 0.02, so methods that work best on symmetric, bell-shaped data (many in Chapters 7–13) behave better.
- **Interpretable effects.** A difference of 0.1 in $\log_{10}$ latency is a factor of $10^{0.1} \approx 1.26$, a 26% slowdown, whatever the baseline.

Back-transforming needs care. Quantiles pass straight through: the median of $\log_{10}$ latency is $\log_{10}$ of the median, so $10^{\text{median of logs}}$ recovers 24.8 ms exactly. The mean of the logs, transformed back, gives the **geometric mean**, here 25.5 ms. It is close to the median, and well below the ordinary mean of 32.5 ms. You cannot recover the ordinary mean from the mean of the logs.

Common trap

Logs need positive numbers. Zero latencies, zero counts or negative profits break $\log x$. A common patch is $\log(x + 1)$; it works for counts but the "+1" is an arbitrary choice that matters when values are small. Also, the log of the twelve latencies still has skewness 1.53: a transform cannot hide a genuine outlier such as 120 ms in a sample of 12.

3.8 QQ plots: comparing a sample with a model

Is this sample consistent with a Normal distribution? The histogram is a blurry way to answer, because it depends on bin width and our eye is bad at judging curves. A **quantile–quantile plot** (QQ plot) turns the question into "are these points on a straight line?", which the eye answers well.

3.8.1 Building one by hand

Eight adult heights in cm, sorted: $158, 162, 165, 167, 170, 171, 175, 184$. Mean 169, SD 8.07.

The idea: if the data were a perfect Normal sample, the i-th smallest of n values should sit near the population quantile at fraction $(i - 0.5)/n$. (The -0.5 puts the i-th point in the middle of its $1/n$ slice. With $n = 8$ the lowest point gets fraction $1/16$, the middle of the bottom eighth. Using i/n instead would send the top point to fraction 1, whose Normal quantile is infinite.) So:

1. Sort the data: $x_{(1)} \le \cdots \le x_{(n)}$.
2. Compute plotting positions $p_i = (i - 0.5)/n$.
3. Convert each to a standard Normal quantile $z_i = \Phi^{-1}(p_i)$, where Φ is the standard Normal CDF and Φ^{-1} its quantile function.
4. Plot the points $(z_i,\, x_{(i)})$.

i	1	2	3	4	5	6	7	8
$x_{(i)}$	158	162	165	167	170	171	175	184
p_i	0.0625	0.1875	0.3125	0.4375	0.5625	0.6875	0.8125	0.9375
z_i	-1.534	-0.887	-0.489	-0.157	0.157	0.489	0.887	1.534

Table 3.2: QQ plot ingredients for eight heights. The z_i values are symmetric because the plotting positions are.

If the data are Normal with mean μ and SD σ, their quantiles are $\mu + \sigma z$, so the points should fall near the line $x = \mu + \sigma z$. Using the sample values, that reference line is $x = 169 + 8.07z$. At $z = 1.534$ the line predicts $169 + 8.07(1.534) \approx 181.4$; the tallest person is 184, a bit above.

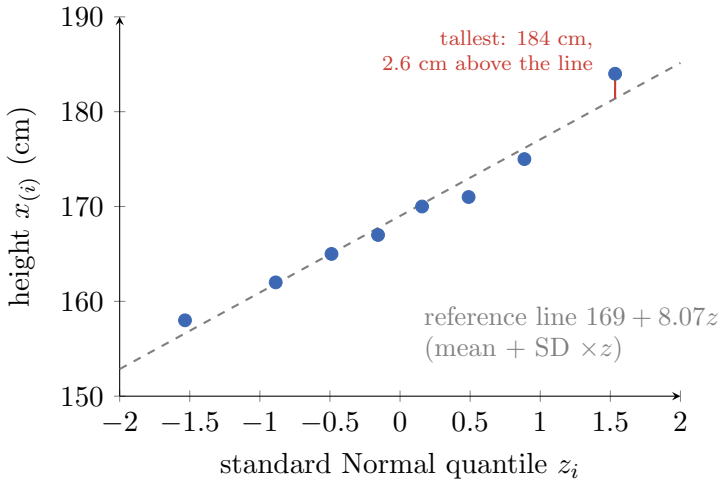


Figure 3.16: QQ plot of eight heights against the Normal. The points hug the reference line, so a Normal model is reasonable. With $n = 8$ wobbles of a few cm are expected.

The slope of the line is the SD and its height at $z = 0$ is the mean. That is why the QQ plot compares

shape only: a Normal sample with any mean and SD gives a straight line, and only departures from straightness signal a different shape.

3.8.2 Reading the patterns

Departures from the line are easiest to read at the ends, where the tails live. Figure 3.17 shows the idealised pattern for four shapes, each standardised to mean 0 and SD 1 and plotted with $n = 40$ points.

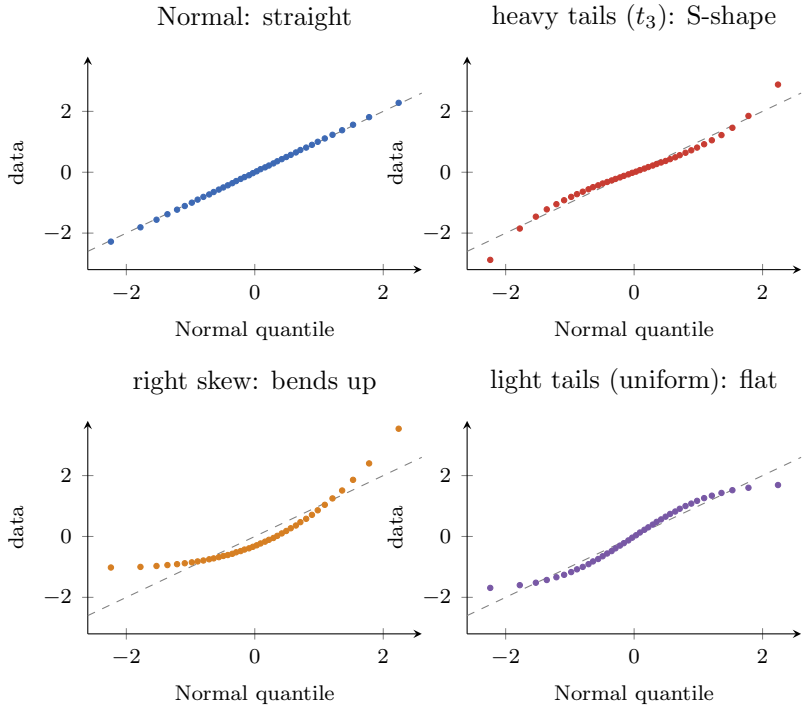


Figure 3.17: Four QQ plot signatures against the Normal; the dashed line is where a perfect Normal sample would fall. Look at the two ends: heavy tails pull them away from the line, light tails pull them in, right skew lifts both. Points are exact quantiles, so the patterns are noise-free; real samples wobble around them, most at the ends.

What you see	What it means
Points on the line	Shape matches the Normal
Low end below, high end above the line (S)	Heavier tails than Normal: more extreme values
Low end above, high end below the line	Lighter tails than Normal: values bounded
Curve bending upward (convex)	Right skew: long upper tail
Curve bending downward (concave)	Left skew: long lower tail
One or two points far off, rest on the line	Isolated outliers

Table 3.3: Reading a Normal QQ plot.

The logic behind every row is the same. The vertical position of a point is the data quantile; the line

marks where a Normal quantile would be. In a heavy-tailed sample the extreme quantiles are more extreme than Normal ones, so the ends of the plot pull away from the line, downward at the left and upward at the right. In a right-skewed sample the low quantiles are bunched (above the line) and the high quantiles spread out (above the line again, increasingly), giving a curve that bends up.

```
import numpy as np
from scipy import stats
x = np.sort(np.array([158,162,165,167,170,171,175,184]))
p = (np.arange(1, len(x) + 1) - 0.5) / len(x)
z = stats.norm.ppf(p)                # -1.534 ... 1.534
# plot (z, x); or let scipy do it all:
stats.probplot(x, dist="norm")       # uses slightly different p_i
```

Connection

QQ plots are not only for the Normal. Replace Φ^{-1} with any distribution's quantile function to test that model; plot log-latency against the Normal to check a log-normal model. Once regression arrives (Chapter 12), a Normal QQ plot of the residuals is the standard check of the Normal-errors assumption.

Pause

A QQ plot of request latencies against the Normal curves sharply upward at the right end. You take logs and redraw; now the points lie on a line. What distribution do the raw latencies follow, approximately? (Log-normal: their logarithm is Normal.)

3.9 Smooth density estimates

A histogram's shape depends on where you put the bins. Shift the bin edges by half a bin and a bump can appear or vanish. A **kernel density estimate** (KDE) replaces the boxes with smooth bumps and removes the bin-edge arbitrariness.

3.9.1 Building a KDE from bumps

Place a small bell curve, called a **kernel**, centred on each data point. Every bell has the same width h, called the **bandwidth**, and area $1/n$. Add the bells up. Where data points crowd together their bells pile up into a hill; where data are sparse, the sum is low.

Definition 3.6 (Kernel density estimate)**.** With a kernel K (a symmetric density, usually the standard Normal φ) and bandwidth $h > 0$,

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right).$$

The factor $1/h$ inside and outside keeps each bump's area at $1/n$ as you stretch it, so the total area is 1: $\hat{f}_h$ is a genuine density. At a point x, the estimate counts nearby data, with closer points counting more.

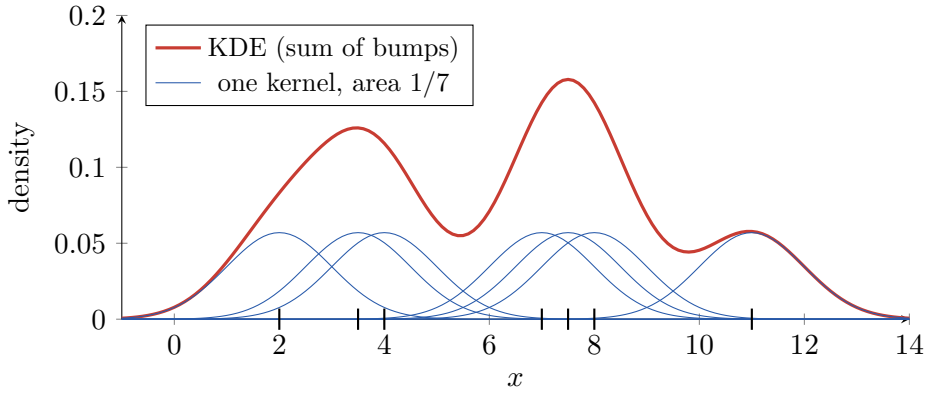


Figure 3.18: A KDE of seven data points (tick marks on the axis) with Gaussian kernels of bandwidth $h = 1$. Each thin bump has area 1/7; the thick curve is their sum. The three points at 7, 7.5 and 8 overlap most, so they build the tallest hill (height 0.158 at $x = 7.5$); the lone point at 11 makes only a single bump's height.

At $x = 7$ in Figure 3.18, the three bumps at 7, 7.5 and 8 contribute most; the estimate there is $\hat{f}(7) = \frac{1}{7}\sum_i \varphi(7 - x_i) \approx 0.143$.

3.9.2 Bandwidth is the whole game

The kernel shape barely matters. The bandwidth matters enormously. Small h gives narrow bumps and a spiky curve that follows every quirk of the sample. Large h gives wide bumps that blur real features away. Figure 3.19 shows 38 reaction times drawn from two groups (a fast majority near 30 ms and a slower group near 50 ms).

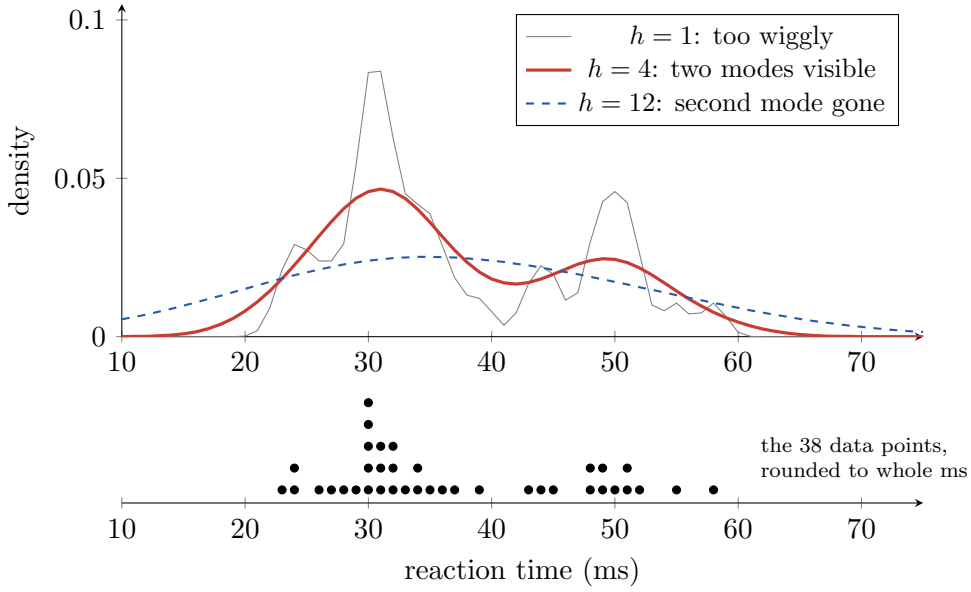


Figure 3.19: Three KDEs of the same 38 reaction times (dot plot below). $h = 1$ invents bumps that are just sampling noise. $h = 12$ smooths away the slower group. $h = 4$, a little below Silverman's rule of thumb (5.1), shows the two groups.

This is the first appearance of a trade-off that runs through all of statistics and machine learning. A small bandwidth has low *bias* (it can follow any shape) but high *variance* (a new sample would give a very different curve). A large bandwidth is stable but systematically wrong. Software picks h with a rule of thumb; the most common, **Silverman's rule**, is

$$h = 1.06\, s\, n^{-1/5},$$

where s is the sample SD. For these 38 points ($s \approx 9.9$) it gives $h \approx 1.06 \times 9.9 \times 38^{-1/5} \approx 5.1$. The rule is tuned for roughly Normal data and tends to oversmooth data with several modes, so always try a smaller and a larger value and see which features survive.

```
from scipy import stats
kde = stats.gaussian_kde(x)              # Scott's rule, close to Silverman's
kde_narrow = stats.gaussian_kde(x, bw_method=0.1)   # factor times SD of x
grid = np.linspace(10, 75, 200)
density = kde(grid)                      # evaluate the curve on a grid
```

Common trap

A KDE puts probability where no data can be. Gaussian bumps around latencies near zero spill into negative milliseconds; bumps around exam scores of 100 spill past 100. For bounded data, either estimate on a transformed scale (log for positive data) or read the curve near the boundary with suspicion.

3.9.3 Violin plots

A **violin plot** is a box plot with a KDE wrapped around it: the density is drawn vertically and mirrored, so wide parts of the violin are where data are dense. It keeps the box plot's compactness for comparing groups and adds the shape information the box plot throws away.

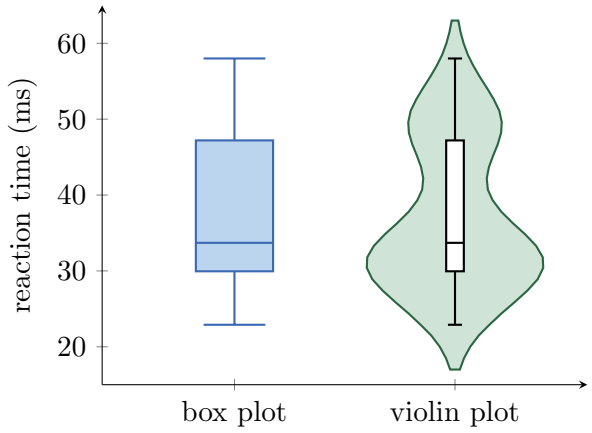


Figure 3.20: The 38 reaction times as a box plot and as a violin plot. The box plot shows a tall upper half but cannot show that the data form two separate groups. The violin's outline is the $h = 4$ KDE of Figure 3.19, mirrored; its two bulges, at about 30 and 50 ms, make the two groups obvious.

History

John Tukey (1915–2000), a statistician at Princeton and Bell Labs, spent the 1960s and 1970s arguing that statisticians jumped to formal models before looking at their data. His problem was practical: with pencil and paper, how do you summarise a batch of numbers in a way that a few wild values cannot distort? His answer, in the 1977 book *Exploratory Data Analysis*, used medians and quartiles (which ignore how extreme the extremes are) instead of means and SDs, and drew them as the box-and-whisker plot, with the 1.5 IQR fences to flag points worth a second look. It unlocked fast visual comparison of many groups at once, and it is still the default summary plot in every statistics package. Violin plots (Hintze and Nelson, 1998) and the ECDF-based tools in this chapter are refinements of the same idea: look at the whole distribution before summarising it.

3.10 Choosing a picture

Each display in this chapter keeps different information.

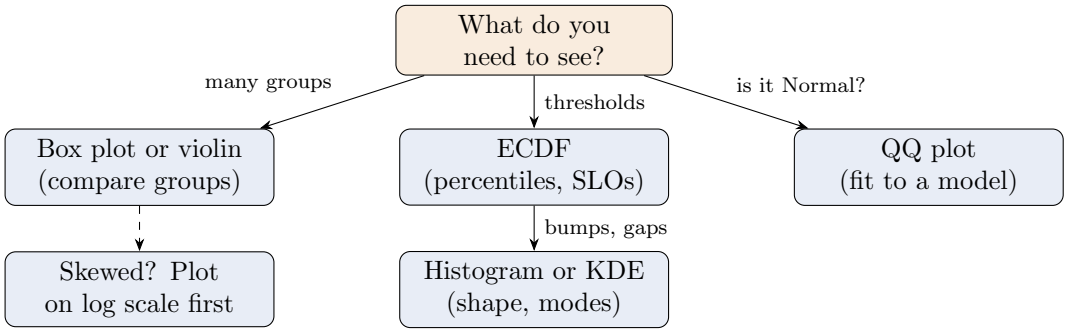


Figure 3.21: A rough guide to choosing a display. In practice look at two: one that shows the whole shape (ECDF, KDE) and one that summarises (box plot, a few percentiles).

3.11 Summary

- The p-quantile has a fraction p of the data below it. With finite data the definition needs an interpolation rule; numpy offers nine. They agree in the middle and can disagree badly in sparse tails.
- Five-number summary: min, Q_1, median, Q_3, max. IQR $= Q_3 - Q_1$.
- Box plot: box from Q_1 to Q_3, median line, whiskers to the last points within 1.5 IQR of the box, outliers plotted individually. Roughly 0.7% of Normal data and 4.8% of exponential data fall outside the fences.
- ECDF $\hat{F}(t) =$ fraction of data $\leq t$. Quantiles are read by going across and down. The quantile function is the inverse of the CDF.
- Quantiles commute with increasing transforms: $Q_{g(X)}(p) = g(Q_X(p))$. Means do not.

- For latency, report p50/p90/p99 with counts. The mean hides tails. Percentiles do not average across servers or time windows; merge histograms or raw data instead.
- Skewness $g_1 = m_3/m_2^{3/2}$ measures tail imbalance; excess kurtosis $m_4/m_2^2 - 3$ measures tail weight relative to the Normal. Both are sensitive to single points.
- Logs turn right-skewed, multiplicative data into roughly symmetric data. The back-transformed mean of logs is the geometric mean.
- QQ plot: sorted data against $\Phi^{-1}((i-0.5)/n)$. Straight means Normal-shaped; S-shape means heavy tails; upward bend means right skew.
- KDE: a sum of bumps of width h. Bandwidth controls the bias–variance trade-off. Violin plots wrap a KDE around a box plot.

Chapter 4 moves from one variable to two: scatter plots, covariance and correlation.

3.12 Exercises

1. For the data $3, 7, 8, 12, 20$, compute Q_1, the median and Q_3 with numpy's default (linear) method. Then compute the fences and decide whether any point is an outlier.
2. For the same data, compute Q_1 with the `weibull` rule (position $i/(n+1)$).
3. Using the twelve latencies of this chapter, find $\hat{F}(25)$ and $\hat{F}(40)$.
4. Latency is exponential with mean 50 ms. Find the median and the p99.
5. A page makes 20 independent backend calls and waits for all of them. What is the probability that at least one is slower than the backend's p99?
6. Server A served requests of $10, 20, 30$ ms; server B served one request of 100 ms. Compare the average of the two medians with the median of all four requests.
7. The median latency of a service is 25 ms. What is the median of $\log_{10}$ latency? Can you say what the mean of $\log_{10}$ latency is?
8. A Normal QQ plot shows points above the line at the left end and below the line at the right end. What does that say about the tails? What shape would a curve that bends downward (concave) indicate?
9. Out of 1000 values drawn from a Normal distribution, about how many would a box plot flag as outliers?
10. Describe $\hat{f}_h$ in the limits $h \to 0$ and $h \to \infty$.

Answers

1. $n = 5$, $h = 4p + 1$. Q_1: $h = 2$, so $Q_1 = 7$. Median 8. Q_3: $h = 4$, so $Q_3 = 12$. IQR $= 5$, fences $7 - 7.5 = -0.5$ and $12 + 7.5 = 19.5$. The value 20 is an outlier.
2. $h = (n+1)p = 6(0.25) = 1.5$, so $Q_1 = 3 + 0.5(7 - 3) = 5$.
3. $\hat{F}(25) = 9/12 = 0.75$; $\hat{F}(40) = 10/12 \approx 0.83$.

4. Median $= 50 \ln 2 \approx 34.7$ ms; p99 $= 50 \ln 100 \approx 230.3$ ms.

5. $1 - 0.99^{20} \approx 0.18$.

6. Medians 20 and 100 average to 60. The four requests $10, 20, 30, 100$ have median 25.

7. $\log_{10} 25 \approx 1.40$, because quantiles pass through increasing transforms. The mean of the logs cannot be found from the median alone (it is $\log_{10}$ of the geometric mean).

8. Light tails (the extremes are less extreme than Normal ones), as in uniform data. A concave curve indicates left skew.

9. About 0.7% of 1000, so roughly 7.

10. As $h \to 0$ the bumps become spikes at the data points (the curve is zero almost everywhere and infinitely tall at each x_i). As $h \to \infty$ the bumps become one very wide, flat curve that forgets the data's shape.

Chapter 4

Two Variables: Tables, Scatter Plots, and Correlation

Chapters 1–3 looked at one variable at a time: its center, its spread, its quantiles, its shape. Real questions almost always involve two. Does latency grow with payload size? Do customers who contact support by phone get answers faster? Does one treatment work better than another? This chapter builds the tools for describing how two variables move together, and then spends a long time on the ways those tools mislead.

There are two cases. If both variables are categories (device type, yes/no), we count pairs in a table. If both are numbers (size, latency), we draw a scatter plot and summarize it with the correlation coefficient.

4.1 Two categorical variables: contingency tables

A support team logs 600 tickets. For each ticket it records the channel the customer used (chat, email, phone) and how long resolution took (same day, 1–3 days, longer). Counting every combination gives a **contingency table**: a grid with one row per category of the first variable, one column per category of the second, and a count in each cell.

	Same day	1–3 days	Longer	Row total
Chat	120	60	20	200
Email	90	150	60	300
Phone	70	20	10	100
Column total	280	230	90	600

The row and column totals are the **marginal distributions**: the distribution of one variable alone, ignoring the other. The name comes from where they sit, in the margins of the table. Overall, $280/600 \approx 46.7\%$ of tickets were resolved the same day.

Raw counts are hard to compare because the rows have different sizes. Email has 90 same-day tickets and phone has 70, but email had three times as many tickets. The fix is to divide each cell by its

row total. The result is a **conditional distribution**: the distribution of resolution time *given* a particular channel.

	Same day	1–3 days	Longer
Chat	$120/200 = 60\%$	$60/200 = 30\%$	$20/200 = 10\%$
Email	$90/300 = 30\%$	$150/300 = 50\%$	$60/300 = 20\%$
Phone	$70/100 = 70\%$	$20/100 = 20\%$	$10/100 = 10\%$
All	$280/600 \approx 46.7\%$	$230/600 \approx 38.3\%$	$90/600 = 15\%$

Now the story is visible. Phone resolves 70% of tickets the same day; email only 30%. A **segmented bar chart** draws each row of this table as a bar of height 100%, cut into pieces whose lengths are the conditional proportions.

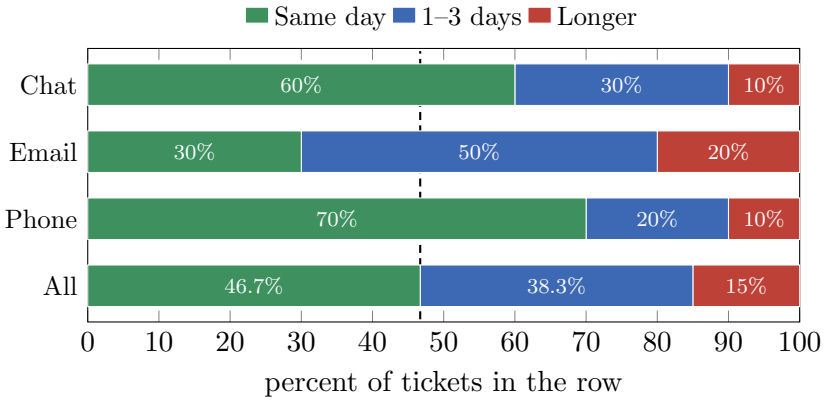


Figure 4.1: Segmented bar chart of resolution time given channel. The dashed line marks the overall same-day share, 46.7%. If channel and resolution time were unrelated, every green segment would end at that line. Instead email's stops well short (30%) and phone's runs well past it (70%).

Definition 4.1 (Association for categorical variables)**.** Two categorical variables are **associated** in a data set if the conditional distribution of one differs across the categories of the other. They show **no association** if every row has the same conditional distribution, which is then equal to the marginal distribution.

No association has a concrete numerical meaning. If channel told us nothing about resolution time, then 46.7% of email's 300 tickets would be same-day, that is

$$300 \times \frac{280}{600} = 140 \text{ tickets.}$$

In general the count we would expect in row i, column j under no association is

$$\text{expected}_{ij} = \frac{(\text{row } i \text{ total}) \times (\text{column } j \text{ total})}{n}.$$

Email actually has 90 same-day tickets, 50 fewer than expected. Chapter 10 turns the gap between observed and expected counts into the chi-square test, which asks whether gaps this large could come from chance alone.

Common trap

Conditioning in the other direction answers a different question. Dividing by *column* totals gives "of the same-day tickets, what fraction came by chat?": $120/280 \approx 42.9\%$. That is not the same as "of chat tickets, what fraction were same-day?": $120/200 = 60\%$. In symbols, $P(\text{chat} \mid \text{same day}) \neq P(\text{same day} \mid \text{chat})$. Mixing these two up is the root of the base-rate fallacy (Probability From the Ground Up covers it with Bayes' rule). Always say which variable you conditioned on.

```
import pandas as pd
df = pd.DataFrame({"channel": ["Chat"]*200 + ["Email"]*300 + ["Phone"]*100,
                   "res": ["Same"]*120 + ["1-3"]*60 + ["Long"]*20
                        + ["Same"]*90 + ["1-3"]*150 + ["Long"]*60
                        + ["Same"]*70 + ["1-3"]*20 + ["Long"]*10})
pd.crosstab(df.channel, df.res, margins=True)          # counts
pd.crosstab(df.channel, df.res, normalize="index")     # row proportions
```

4.2 Two numerical variables: scatter plots

When both variables are numbers we plot each observation as a point (x_i, y_i). This is a **scatter plot**. By convention the variable we think of as the input, or the one we could control, goes on the horizontal axis and is called the **explanatory variable**; the one we want to understand goes on the vertical axis and is called the **response**.

Read a scatter plot in four passes:

- **Direction.** Do points drift up to the right (positive association) or down to the right (negative association)?
- **Form.** Do they follow a straight line, a curve, or separate clusters?
- **Strength.** How tightly do they hug that form?
- **Outliers.** Are there points far from the overall pattern?

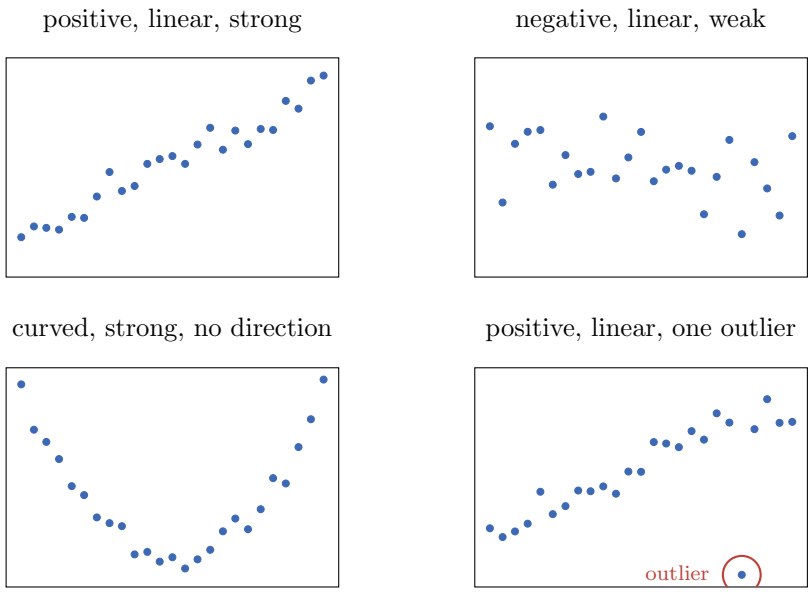


Figure 4.2: Four scatter plots, each described by direction, form, strength, and outliers. The curved panel has a strong relationship but no overall up or down direction; a single number measuring "upness" will miss it. In the last panel one point (circled) sits far below the line the other 24 follow.

Pause

Before reading on: in the bottom-right panel, would removing the circled point make the relationship look stronger or weaker? (Stronger. The point sits far below the line the other 24 points follow.)

4.3 Covariance: the quadrant picture

We want one number that says "these points go up together". Start with six API requests, recording payload size x in KB and response latency y in ms.

i	x_i (KB)	y_i (ms)	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$
1	1	12	-3	-5	15
2	2	19	-2	2	-4
3	3	13	-1	-4	4
4	5	20	1	3	3
5	6	22	2	5	10
6	7	16	3	-1	-3
sum	24	102	0	0	25

The means are $\bar{x} = 24/6 = 4$ KB and $\bar{y} = 102/6 = 17$ ms. Draw a vertical line at $\bar{x}$ and a horizontal line at $\bar{y}$. They cut the plane into four quadrants centered on the point of means $(\bar{x}, \bar{y})$.

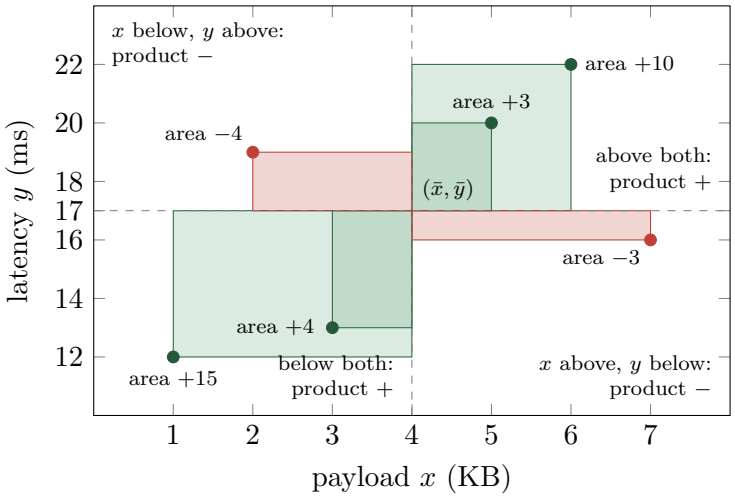


Figure 4.3: Each point spans a rectangle with the point of means $(\bar{x}, \bar{y}) = (4, 17)$; the label next to each point is the rectangle's signed area $(x_i - \bar{x})(y_i - \bar{y})$. Green rectangles (upper right, lower left) count as positive, red ones as negative. Green wins, $(15+4+3+10)-(4+3) = 25$, so the association is positive.

A point in the upper right is above average in both variables, so both deviations are positive and the product is positive. A point in the lower left is below average in both, and the product of two negatives is positive again. Points in the other two quadrants are above average in one variable and below in the other, and contribute negative products. Adding all the products asks: on balance, do points fall in the agreeing quadrants or the disagreeing ones?

Definition 4.2 (Sample covariance)**.** For paired data $(x_1, y_1), \ldots, (x_n, y_n)$ the **sample covariance** is

$$s_{xy} = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}).$$

It is the average signed rectangle area, with the same $n - 1$ divisor as the sample variance from Chapter 2.

For the requests, $s_{xy} = 25/5 = 5$ KB·ms. Notice that s_{xx}, the covariance of x with itself, is exactly the sample variance s_x^2: every rectangle becomes a square.

With more points the picture is the same. Figure 4.4 colours 30 points by the sign of their product.

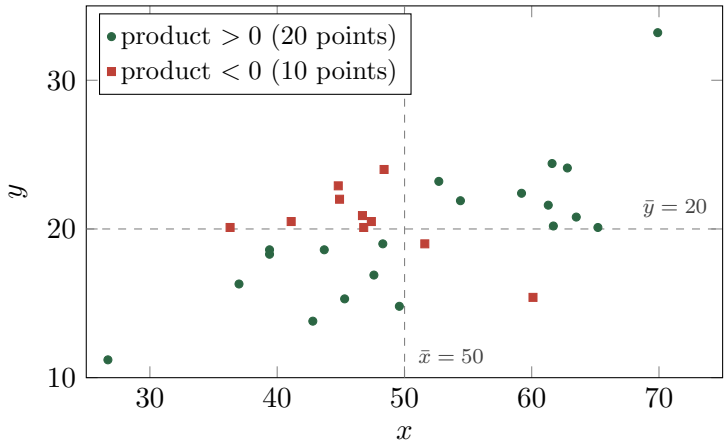


Figure 4.4: Thirty points split by the dashed lines at $\bar{x}$ and $\bar{y}$. Twenty (green circles) fall in the agreeing quadrants and ten (red squares) in the disagreeing ones. Most red squares hug the line $y = \bar{y}$, so their negative rectangles are thin: the sum of products, and the covariance, is clearly positive.

Covariance has a flaw: its size depends on units. Measure payload in bytes instead of KB and every $x_i - \bar{x}$ grows by a factor of 1000, so s_{xy} jumps from 5 to 5000. The relationship did not change; only the ruler did. A covariance of 5000 tells you nothing until you know the scales of both variables.

4.4 Pearson correlation

The cure for the units problem is the same trick as in Chapter 2: measure each deviation in standard deviations. Recall that the **z-score** of x_i is $z_{x,i} = (x_i - \bar{x})/s_x$, the number of standard deviations x_i sits above its mean. A z-score has no units.

Definition 4.3 (Pearson correlation)**.** The **(Pearson) correlation coefficient** of paired data is the average product of z-scores,

$$r = \frac{1}{n-1}\sum_{i=1}^{n} z_{x,i}\, z_{y,i} = \frac{1}{n-1}\sum_{i=1}^{n} \frac{x_i - \bar{x}}{s_x} \cdot \frac{y_i - \bar{y}}{s_y} = \frac{s_{xy}}{s_x s_y}.$$

So r is covariance computed after rescaling each axis so that one standard deviation has length 1. The quadrant picture is unchanged: same quadrants, same signs, only the rectangles are resized.

Example 4.1 (Correlation by hand)**.** For the six requests, $\sum(x_i - \bar{x})^2 = 9 + 4 + 1 + 1 + 4 + 9 = 28$ and $\sum(y_i - \bar{y})^2 = 25 + 4 + 16 + 9 + 25 + 1 = 80$. So

$$s_x = \sqrt{28/5} \approx 2.366 \text{ KB}, \qquad s_y = \sqrt{80/5} = 4 \text{ ms}.$$

Then

$$r = \frac{s_{xy}}{s_x s_y} = \frac{5}{2.366 \times 4} \approx 0.528.$$

The z-score route gives the same thing. The z-scores are

$$z_x : -1.268,\ -0.845,\ -0.423,\ 0.423,\ 0.845,\ 1.268,$$
$$z_y : -1.25,\ 0.5,\ -1,\ 0.75,\ 1.25,\ -0.25.$$

The six products are

$$1.585,\ -0.423,\ 0.423,\ 0.317,\ 1.056,\ -0.317,$$

which sum to 2.641. Dividing by $n-1=5$ gives $r \approx 0.528$. In bytes and seconds instead of KB and ms, r is still 0.528.

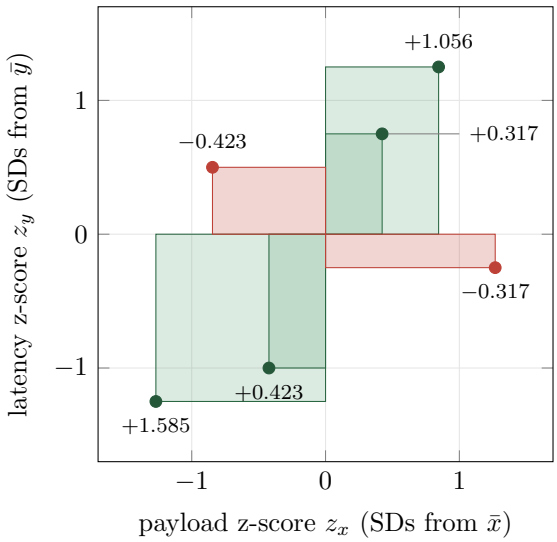


Figure 4.5: Figure 4.3 redrawn with each axis measured in standard deviations, so the point of means sits at $(0,0)$ and each grid square is one SD by one SD. Every point stays in its quadrant with its colour; only the rectangles are resized, and the number by each point is now its product of z-scores. The areas add to $3.381 - 0.740 = 2.641$, and r is that total divided by $n-1=5$: 0.528.

```
import numpy as np
x = np.array([1, 2, 3, 5, 6, 7]); y = np.array([12, 19, 13, 20, 22, 16])
np.cov(x, y)[0, 1]        # 5.0   (uses n-1)
np.corrcoef(x, y)[0, 1]   # 0.528
```

4.4.1 Properties of r

Theorem 4.1 (Properties of the correlation coefficient)**.** *For any paired data with $s_x > 0$ and $s_y > 0$:*

1. $-1 \le r \le 1$.
2. $r = 1$ *exactly when all points lie on a line with positive slope, and* $r = -1$ *exactly when they lie on a line with negative slope.*
3. r *has no units. Replacing x by $a + bx$ with $b > 0$ (a change of units) leaves r unchanged; with $b < 0$ it flips the sign of r.*
4. r *is symmetric: the correlation of x with y equals that of y with x.*

Why it works

Why can r never pass ± 1? Z-scores satisfy $\sum z_{x,i}^2 = n-1$, because $\sum (x_i - \bar{x})^2 / s_x^2 = (n-1)s_x^2/s_x^2$. The same holds for y. A sum of squares cannot be negative, so

$$0 \le \sum_{i=1}^{n} (z_{x,i} \pm z_{y,i})^2 = \sum z_{x,i}^2 \pm 2 \sum z_{x,i} z_{y,i} + \sum z_{y,i}^2 = (n-1) \pm 2(n-1)r + (n-1).$$

Dividing by $2(n-1)$ gives $0 \le 1 \pm r$, so $-1 \le r \le 1$. Equality $r = 1$ needs $\sum (z_{x,i} - z_{y,i})^2 = 0$, so $z_{y,i} = z_{x,i}$ for every point: y is an exact increasing linear function of x. That is property 2. For property 3, the change $x \mapsto a + bx$ turns the mean into $a + b\bar{x}$, so each deviation $x_i - \bar{x}$ is multiplied by b and s_x by $|b|$; the ratio $(x_i - \bar{x})/s_x$ is multiplied by $b/|b| = \pm 1$.

Figure 4.6 calibrates the eye. Each panel has 40 points with both variables standardized.

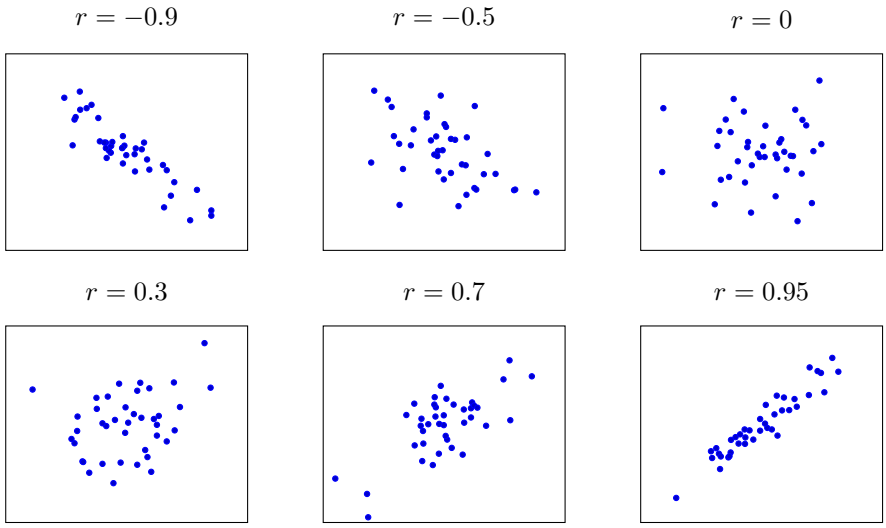


Figure 4.6: Scatter plots with correlations from -0.9 to 0.95, every panel drawn on the same standardized scale so only the correlation changes. Moderate values look weaker than you might expect: at $r = 0.3$ the cloud is almost round, and only near ± 0.9 does it look like a line.

Common trap

r measures *linear* association only, and only its strength, not its slope. A cloud with $r = 0.95$ can have a slope of 0.001 or of 1000; the slope depends on units and r does not. "Strong correlation" does not mean "big effect".

4.5 Always plot: Anscombe's quartet

In 1973 the statistician Francis Anscombe published four small data sets, each with 11 points, built to make one point. Here they are in full.

I		II		III		IV	
x	y	x	y	x	y	x	y
10	8.04	10	9.14	10	7.46	8	6.58
8	6.95	8	8.14	8	6.77	8	5.76
13	7.58	13	8.74	13	12.74	8	7.71
9	8.81	9	8.77	9	7.11	8	8.84
11	8.33	11	9.26	11	7.81	8	8.47
14	9.96	14	8.10	14	8.84	8	7.04
6	7.24	6	6.13	6	6.08	8	5.25
4	4.26	4	3.10	4	5.39	19	12.50
12	10.84	12	9.13	12	8.15	8	5.56
7	4.82	7	7.26	7	6.42	8	7.91
5	5.68	5	4.74	5	5.73	8	6.89

All four have $\bar{x} = 9$, $s_x^2 = 11$, $\bar{y} \approx 7.50$, $s_y^2 \approx 4.12$–4.13, $r \approx 0.816$, and the same best-fit line $\hat{y} \approx 3.00 + 0.500x$ (Chapter 12 explains how that line is found). A table of summary statistics cannot tell them apart. A picture can.

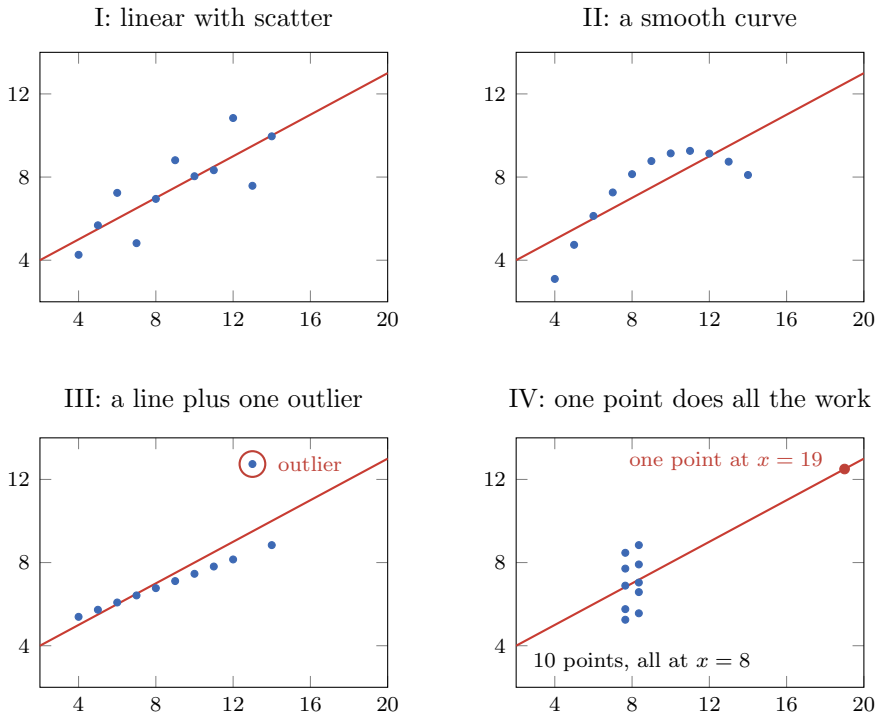


Figure 4.7: Anscombe's quartet. Same means, variances, correlation (0.816) and fitted line (red). Only panel I is described well by "linear with $r = 0.82$". II is a smooth curve, III is a perfect line ruined by one point, and IV has no relationship at all apart from one point. (In IV the ten points at $x = 8$ are nudged alternately left and right so that all ten can be seen.)

Each panel teaches a separate lesson. Panel II: r only measures fit to a line, and a curve that fits perfectly gets 0.82. Panel III: one outlier dragged r down from 1 to 0.82. Panel IV: one **high-leverage** point, a point whose x value sits far from all the others, manufactured a correlation out of nothing; remove it and x has no variation left at all.

4.6 When r misses the relationship

4.6.1 A perfect relationship with $r = 0$

Take $x = -3, -2, -1, 0, 1, 2, 3$ and $y = x^2$, so $y = 9, 4, 1, 0, 1, 4, 9$. Here y is completely determined by x. Yet $\bar{x} = 0$, $\bar{y} = 28/7 = 4$, and the products $(x_i - \bar{x})(y_i - \bar{y}) = x_i(y_i - 4)$ are

$$-15,\ 0,\ 3,\ 0,\ -3,\ 0,\ 15,$$

which sum to 0. So $r = 0$.

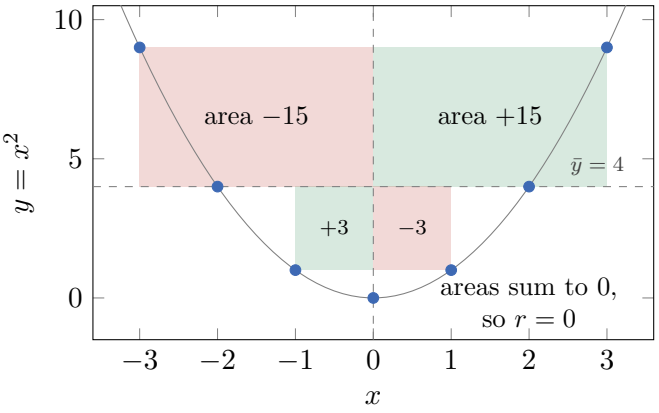


Figure 4.8: Points exactly on a parabola, with each point's rectangle to the point of means $(0, 4)$. The three points on the dashed lines, $(-2, 4)$, $(0, 0)$ and $(2, 4)$, have zero area. Each rectangle on the right cancels its mirror image on the left, so the covariance and r are exactly 0. Zero correlation means no *linear* trend, not no relationship.

The lesson runs in one direction. If x and y are independent (knowing one tells you nothing about the other), then in a large sample r is near 0. The converse fails: $r = 0$ does not imply independence.

4.6.2 Spearman rank correlation

Sometimes the relationship is curved but still always increasing. Suppose a service is tested at load levels $x = 1, 2, \ldots, 8$ and the queue length is $y = 2, 4, 8, \ldots, 256$: it doubles with every step. Bigger x always means bigger y, yet $r \approx 0.850$, because the points bend away from any line.

A relationship where y always goes up when x goes up (or always down) is called **monotone**. To measure monotone association, replace each value by its **rank**, its position when the values are sorted from smallest (rank 1) to largest (rank n), and compute Pearson's r on the ranks.

Definition 4.4 (Spearman rank correlation)**.** The **Spearman correlation** r_s is the Pearson correlation of the ranks of the x values with the ranks of the y values. Tied values each get the average of the ranks they occupy. When there are no ties,

$$r_s = 1 - \frac{6 \sum_{i=1}^{n} d_i^2}{n(n^2 - 1)},$$

where d_i is the difference between the rank of x_i and the rank of y_i.

For the doubling queue, the ranks of y are $1, 2, \ldots, 8$, identical to the ranks of x. Every $d_i = 0$ and $r_s = 1$.

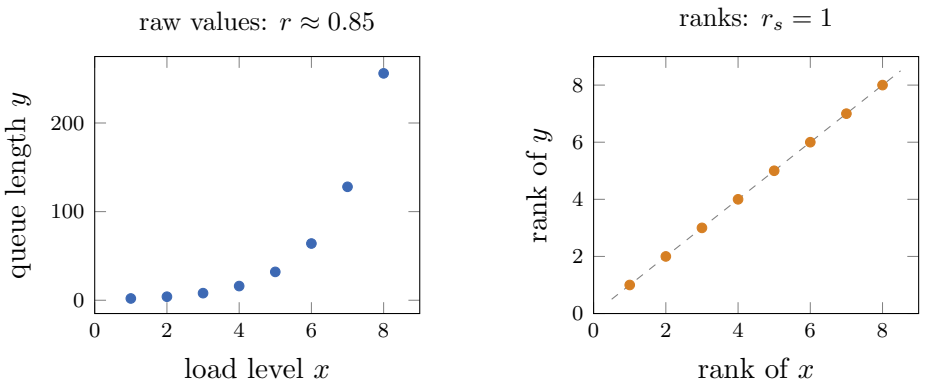


Figure 4.9: Ranking straightens any monotone relationship. The doubling curve on the left becomes a perfect line on the right, so Spearman's r_s is 1 while Pearson's r is only 0.85.

Example 4.2 (Spearman by hand)**.** For the six requests the x values are already in order, with ranks $1, 2, 3, 4, 5, 6$. The latencies $12, 19, 13, 20, 22, 16$ have ranks $1, 4, 2, 5, 6, 3$. The differences are $d = 0, -2, 1, -1, -1, 3$, so $\sum d_i^2 = 0 + 4 + 1 + 1 + 1 + 9 = 16$ and

$$r_s = 1 - \frac{6 \times 16}{6 \times 35} = 1 - \frac{96}{210} \approx 0.543.$$

This is close to Pearson's 0.528, as expected for data without strong curvature.

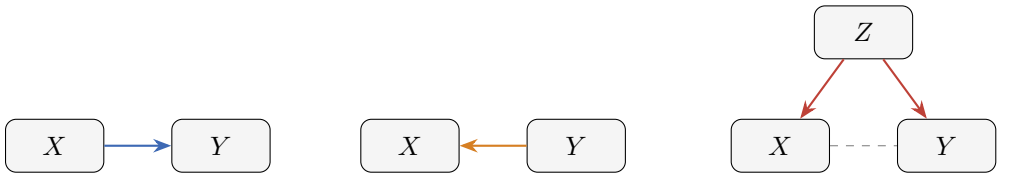
Ranks also tame outliers. A rank can be at most n, however extreme the value, so a single wild point cannot dominate. In the outlier panel of Figure 4.2, Pearson's r is 0.729 with the outlier and 0.960 without; Spearman's r_s is 0.801 with it. The same logic made the median more robust than the mean in Chapter 2.

```
from scipy import stats
x = [1, 2, 3, 4, 5, 6, 7, 8]; y = [2**k for k in x]
stats.pearsonr(x, y).statistic    # 0.850
stats.spearmanr(x, y).statistic   # 1.0
```

4.7 Correlation is not causation

Across the days of a year, a city's ice cream sales and its number of drownings are positively correlated. Ice cream does not cause drowning. Hot weather drives both: more people buy ice cream and more people swim. Temperature is a **confounder**, a third variable that influences both the explanatory variable and the response and so creates an association between them even when neither affects the other.

A **causal diagram** draws each variable as a box and each direct causal influence as an arrow. It is a picture of the story we believe generated the data. An observed correlation between X and Y is consistent with several different diagrams.



(a) X causes Y
e.g. payload → latency

(b) Y causes X
e.g. crashes → more RAM

(c) Z confounds
e.g. heat → ice cream, drownings

Figure 4.10: Three causal stories that all produce a correlation between X and Y. Arrows are causes; the examples underneath come from this chapter. Data on X and Y alone cannot tell the three apart. In (c) the dashed line is an association with no arrow behind it: X and Y move together only because Z pushes both.

Concretely:

- **Reverse causation.** Servers with more RAM have more crashes in a fleet. Perhaps RAM causes crashes, but more likely the team upgraded the RAM on servers that were already crashing.

- **Confounding.** Users who enable dark mode have higher retention. Dark mode is buried in settings, so the users who find it are engaged power users; engagement drives both.

- **Chance.** Search enough pairs of variables and some will correlate strongly by luck. Chapter 11 treats this as the multiple testing problem.

Intuition

The way to break the confounder's grip is to set X yourself, at random. If a coin flip decides who gets dark mode, then engagement cannot influence who has it, so arrow $Z \to X$ is cut. Any remaining difference in retention must flow along $X \to Y$, up to chance. This is why randomized experiments (A/B tests) are the gold standard for causal claims. Chapter 5 develops the design of studies and experiments.

History

Francis Galton, a Victorian scientist and cousin of Charles Darwin, spent the 1870s and 1880s measuring parents and children: heights, arm lengths, sweet-pea seed sizes. He was stuck on how to put a single number on "how strongly does a child's height follow a parent's". In 1888 he proposed measuring both variables in units of their own spread, which is the z-score idea, and called the resulting slope the "co-relation". Karl Pearson turned this into the formula $r = s_{xy}/(s_x s_y)$ in 1896, which is why it carries his name. Eight years later the psychologist Charles Spearman introduced the rank version to study intelligence test scores, where only the ordering seemed meaningful. Pearson's colleague George Udny Yule noticed as early as 1903 that combining groups could reverse an association, the effect now named after Edward Simpson, who wrote it up in 1951. Galton's original goal, predicting one variable from another, became regression (Chapter 12).

4.8 Simpson's paradox

Confounding can do more than create an association. It can flip one. A 1986 study in the *British Medical Journal* compared two treatments for kidney stones: open surgery (A) and a less invasive procedure through a small puncture (B). A treatment counted as a success if, three months later, the stone was gone or only tiny fragments remained.

	Small stones	Large stones	All patients
Treatment A (open surgery)	$81/87 = 93\%$	$192/263 = 73\%$	$273/350 = 78\%$
Treatment B (puncture)	$234/270 = 87\%$	$55/80 = 69\%$	$289/350 = 83\%$

Read the table one column at a time. For small stones, A wins: 93% versus 87%. For large stones, A wins: 73% versus 69%. For all patients together, B wins: 83% versus 78%. Every patient has either a small or a large stone, so it seems impossible that A is better in both groups and worse overall. It is not impossible, and the arithmetic is right.

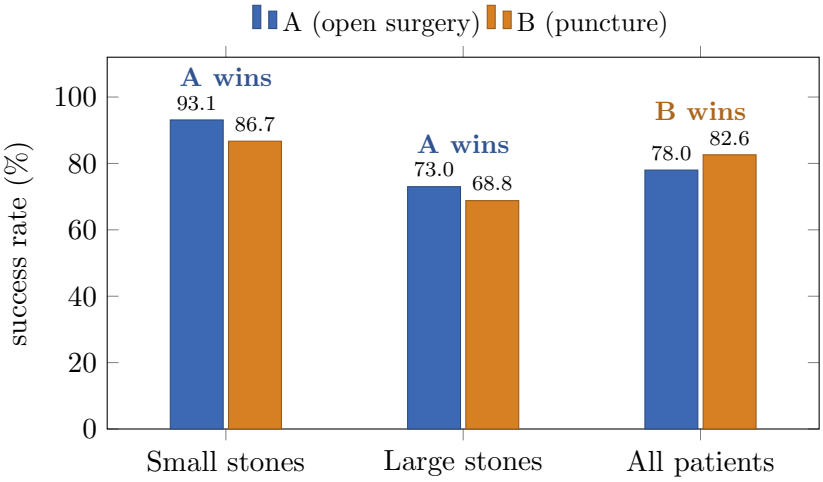


Figure 4.11: Simpson's paradox in the kidney-stone data (bars start at 0). Compare the two bars in each group: A is taller within small stones and within large stones, yet B is taller when the groups are combined.

The resolution is in who got which treatment. Doctors gave the invasive surgery to the hard cases: 263 of A's 350 patients (75%) had large stones, but only 80 of B's 350 (23%) did. Large stones are harder to treat by any method. So A's overall rate is mostly a large-stone rate, and B's is mostly a small-stone rate. Each overall rate is a weighted average of the two group rates:

$$\text{A overall} = \tfrac{87}{350}(93.1\%) + \tfrac{263}{350}(73.0\%) \approx 78.0\%,$$
$$\text{B overall} = \tfrac{270}{350}(86.7\%) + \tfrac{80}{350}(68.8\%) \approx 82.6\%.$$

The weights differ, and the weights carry the stone-size effect.

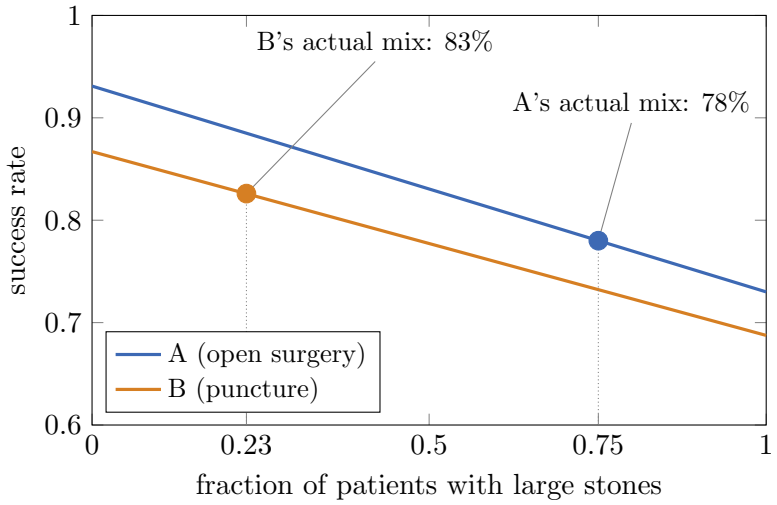


Figure 4.12: Each line shows a treatment's overall success rate if its patients had a given fraction of large stones: the left end is its small-stone rate, the right end its large-stone rate. A's line is above B's everywhere, so A is better at every fixed mix. The dots mark the actual mixes: A treated mostly large stones (75%), B mostly small (23% large), and that pushes A's dot below B's.

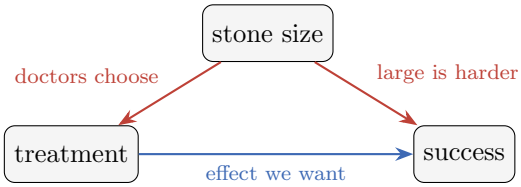


Figure 4.13: Causal diagram for the kidney-stone study. Stone size affects both which treatment a patient gets and whether it succeeds, so it confounds the comparison. Comparing within stone size removes its influence.

Common trap

There is no rule "always trust the split table". Which table answers your question depends on the causal story. Here stone size is measured *before* treatment and affects it, so you compare within stone size, and A is the better treatment. If the splitting variable were something the treatment itself caused (say, blood pressure measured after surgery), splitting would block part of the treatment's effect and the combined table would be the right one. The numbers alone cannot tell you; the diagram can.

The most famous case involves university admissions. In fall 1973 the University of California, Berkeley admitted about 44% of its 8442 male graduate applicants and about 35% of its 4321 female applicants. A 1975 analysis by Bickel, Hammel and O'Connell looked department by department and found that in most departments women were admitted at a similar or higher rate than men. Women had applied disproportionately to departments that admitted few applicants of either sex. The department chosen played the role of stone size.

Pause

Check your understanding: could A have won in both groups and *also* overall? (Yes, if both treatments had been given to similar mixes of patients. The reversal needs the groups to differ in both size mix and baseline difficulty.)

4.9 Ecological fallacy

A correlation computed from group averages (countries, states, schools, server racks) is called an **ecological correlation**. The **ecological fallacy** is reading it as if it described individuals.

Averaging removes the scatter within each group. What is left is usually a much tighter pattern than the individuals show, and sometimes a pattern in a different direction.

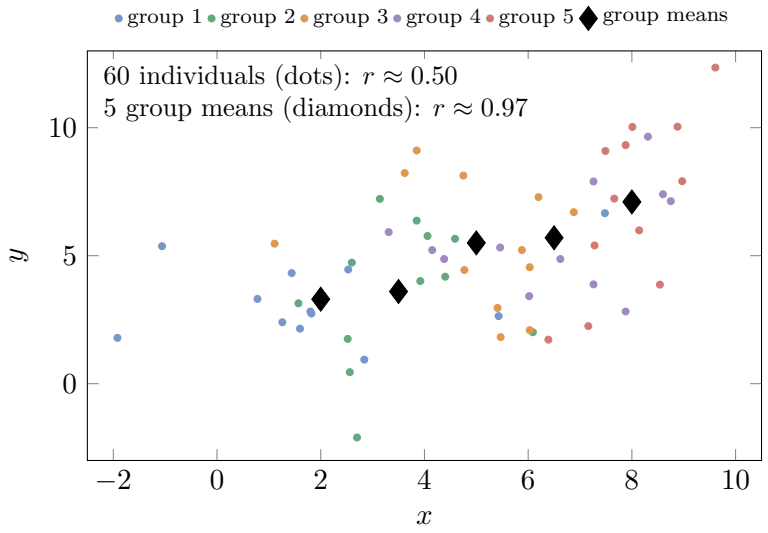


Figure 4.14: Sixty individuals in five groups (colours) and the five group means (black diamonds). The diamonds lie almost on a line ($r \approx 0.97$), but the individuals around them scatter widely ($r \approx 0.50$). A correlation of averages overstates how well x predicts y for any one person.

The classic real example is W. S. Robinson's 1950 study of the 1930 US census. Across the 48 states, the percentage of residents born abroad correlated *negatively* with the percentage who were illiterate: $r \approx -0.53$. Read naively, immigrants were more literate. Across individuals the correlation was *positive*, about 0.12: immigrants were somewhat more likely to be illiterate. The state-level pattern arose because immigrants tended to settle in states whose native-born population had high literacy. Figure 4.15 shows how such a reversal can happen.

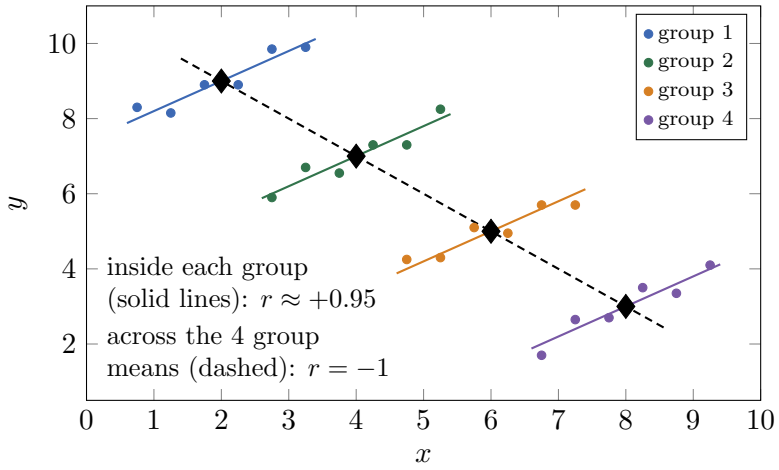


Figure 4.15: An ecological reversal, built to mirror Robinson's census data. Inside every group, y rises with x (solid lines). But the groups with larger x sit lower, so the group means (diamonds) fall along the dashed line. The group-level correlation has the opposite sign from the one that holds for every individual within a group.

Common trap

"Countries that eat more chocolate win more Nobel prizes" is a statement about countries. It says nothing about whether a chocolate-eating person is more likely to win one. Before using a correlation, ask what one data point is: a person, a request, a server, a city, a week.

4.10 Restriction of range

A university admits students using an entrance exam score and later finds that, among admitted students, the exam barely correlates with first-year grades. Is the exam useless? Not necessarily.

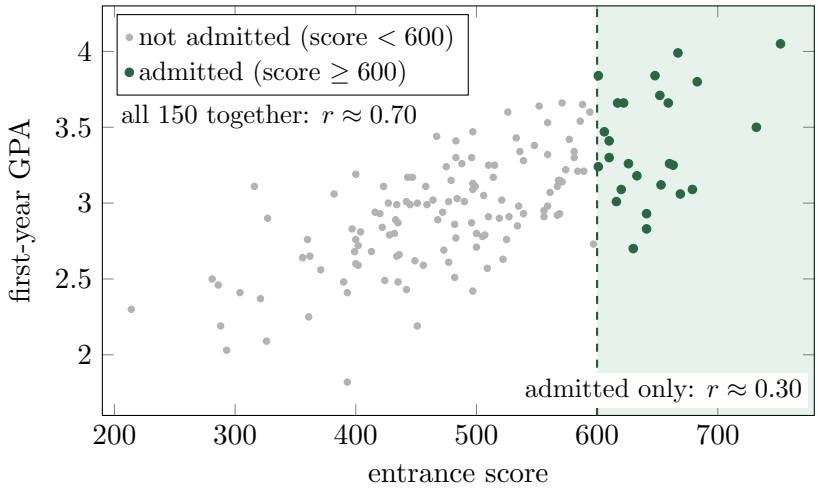


Figure 4.16: Restriction of range. Over all 150 applicants (grey and green dots together) the score and GPA correlate at 0.70. Keep only those with a score of at least 600 and the correlation falls to about 0.30. The relationship did not change; the range of x shrank.

Correlation compares the trend in y to the scatter around it. If you keep only a narrow slice of x values, the trend across the slice is small, but the scatter of y around the trend is as large as ever. So r drops. The same thing happens when you study only the employees who passed the hiring screen, only the requests that did not time out, or only the patients healthy enough to enter a trial. Correlations from truncated samples cannot be compared directly to correlations from full ones.

Pause

The reverse also holds. If you sample only the very small and very large payloads and skip the middle, would r go up or down? (Up. A wider spread in x makes the same trend stand out more against the same scatter.)

4.11 Preview: the regression line

Correlation is symmetric and unitless. Often we want something asymmetric with units: a rule that predicts y from x. The natural candidate is a line through the point of means $(\bar{x}, \bar{y})$. Its slope turns out to be

$$b = r \, \frac{s_y}{s_x}.$$

In words: when x rises by one standard deviation, the predicted y rises by r standard deviations. For the six requests,

$$b = 0.528 \times \frac{4}{2.366} \approx 0.893 \text{ ms per KB},$$

which is also $s_{xy}/s_x^2 = 5/5.6 = 25/28$. The intercept is $\bar{y} - b\bar{x} = 17 - 0.893 \times 4 \approx 13.43$, so the line is $\hat{y} \approx 13.43 + 0.893x$.

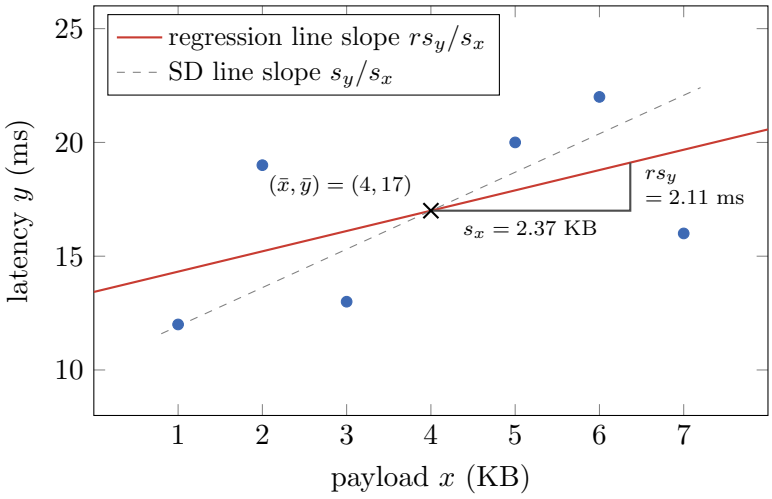


Figure 4.17: The regression line (red) passes through $(\bar{x}, \bar{y})$ (black cross). The step drawn from the cross shows its slope: one standard deviation to the right in x (2.37 KB), it rises only r standard deviations in y ($0.528 \times 4 = 2.11$ ms), so it is flatter than the dashed line that rises a full s_y per s_x. Chapter 12 shows that this line minimizes the sum of squared vertical errors.

Because $|r| \leq 1$, the prediction for y is never more standard deviations from its mean than x is from its mean, and strictly fewer unless the points lie on a line. Tall parents tend to have children who are tall, but less extreme. This is *regression to the mean*, the effect Galton was studying, and it is pure arithmetic of the slope formula, not a force of nature.

Connection

In machine learning, the correlation matrix of features is a standard first look at a data set, and features with $|r|$ near 1 are flagged as redundant (Chapter 13 calls this collinearity). Two cautions from this chapter carry over directly: near-zero r does not mean a feature is useless to a nonlinear model such as a tree or a neural network, and a high r between a feature and the label does not mean the feature causes the label, so it may stop working when the world shifts. In probability language, r estimates $\mathrm{Corr}(X, Y) = \mathrm{Cov}(X, Y)/(\mathrm{SD}(X)\,\mathrm{SD}(Y))$, where $\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}\,X)(Y - \mathbb{E}\,Y)]$ (Probability From the Ground Up treats these for random variables).

4.12 Summary

- **Two categorical variables**: make a contingency table, divide by row totals to get conditional distributions, and compare them with a segmented bar chart. No association means every row matches the marginal.
- **Two numerical variables**: draw a scatter plot; describe direction, form, strength and outliers.
- **Covariance** $s_{xy} = \frac{1}{n-1}\sum(x_i - \bar{x})(y_i - \bar{y})$ is the average signed rectangle area around $(\bar{x}, \bar{y})$. Its size depends on units.
- **Pearson's** $r = \frac{1}{n-1}\sum z_{x,i} z_{y,i} = s_{xy}/(s_x s_y)$: unitless, symmetric, between -1 and 1, equal to ± 1 only for a perfect line.

- r measures **linear** association only. Anscombe's quartet: same r, four different pictures. A parabola can have $r = 0$. Always plot.
- **Spearman's** r_s is Pearson's r on ranks; it measures monotone association and resists outliers.
- **Correlation is not causation**: reverse causation, confounders, and chance all create correlations. Randomization breaks confounding.
- **Simpson's paradox**: a confounder can reverse an association when groups are combined. The causal diagram decides which table to trust.
- **Ecological fallacy**: correlations of group averages are usually stronger than, and can even oppose, individual-level correlations.
- **Restriction of range** shrinks r toward 0.
- **Regression slope** $b = r\, s_y/s_x$, line through $(\bar{x}, \bar{y})$ (Chapter 12).

Every trap in this chapter came from how the data were gathered: who chose the treatment, which groups were averaged, which applicants were admitted. Chapter 5 takes that question head on: how sampling and study design decide what a data set can tell you.

4.13 Exercises

1. A/B test results by device. Desktop: 400 users, 48 converted. Mobile: 600 users, 42 converted. (a) Give the conversion rate for each device and overall. (b) Under no association between device and conversion, how many desktop conversions would you expect?
2. For the points $(1, 2), (2, 1), (3, 4), (4, 3), (5, 5)$ compute $\bar{x}$, $\bar{y}$, s_{xy}, s_x, s_y and r.
3. Temperatures x in degrees Celsius have correlation $r = 0.6$ with ice cream sales y. What is the correlation if temperature is converted to Fahrenheit, $x' = 32 + 1.8x$? What if x is replaced by $-x$?
4. Compute Pearson's r and Spearman's r_s for $(1, 1), (2, 2), (3, 3), (4, 4), (5, 20)$. Which better describes "y goes up whenever x goes up"?
5. Give data with five points on the curve $y = |x|$ for which $r = 0$ exactly. Explain using the quadrant picture.
6. A study finds that cities with more police officers per capita have more crime. Draw two different causal diagrams consistent with this.
7. Hospital P has a 90% survival rate and hospital Q has 85%. Describe how Q could still be the better hospital for every type of patient. What additional variable would you ask for?
8. An engineering team computes $r = 0.1$ between interview score and job performance, using only people they hired. Why might the interview still be a good predictor among all applicants?
9. Prove that if every $y_i = a + bx_i$ with $b < 0$, then $r = -1$, by writing $z_{y,i}$ in terms of $z_{x,i}$.
10. For the six requests in Section 4.3, use the regression line to predict the latency for a 4 KB and a 10 KB payload. Which prediction do you trust less, and why?

Answers

1. (a) Desktop $48/400 = 12\%$, mobile $42/600 = 7\%$, overall $90/1000 = 9\%$. (b) $400 \times 90/1000 = 36$.

2. $\bar{x} = \bar{y} = 3$. Deviations x: $-2, -1, 0, 1, 2$; y: $-1, -2, 1, 0, 2$. Products $2, 2, 0, 0, 4$, sum 8, so $s_{xy} = 8/4 = 2$. $s_x = s_y = \sqrt{10/4} \approx 1.581$. $r = 2/2.5 = 0.8$.

3. Still 0.6 (positive rescaling). With $-x$ it becomes -0.6.

4. With $\bar{x} = 3$ and $\bar{y} = 6$, the products of deviations sum to 40, $\sum(x_i - \bar{x})^2 = 10$ and $\sum(y_i - \bar{y})^2 = 250$, so $r = 40/\sqrt{2500} = 0.8$. The ranks of y are $1, 2, 3, 4, 5$, so $r_s = 1$. Spearman describes the monotone pattern; Pearson is pulled around by the point $(5, 20)$.

5. $x = -2, -1, 0, 1, 2$, $y = 2, 1, 0, 1, 2$. $\bar{y} = 1.2$; the products $x_i(y_i - 1.2)$ are $-1.6, 0.2, 0, -0.2, 1.6$, summing to 0. Mirror-image points sit in opposite-sign quadrants and cancel.

6. (i) Crime $\rightarrow$ more police (cities hire in response: reverse causation). (ii) City size or density $\rightarrow$ both police and crime (confounding).

7. If Q treats a much larger share of high-risk patients, each risk group can have higher survival at Q while Q's overall rate is lower (Simpson's paradox). Ask for patient severity and compare within severity groups.

8. Restriction of range: hires all scored above a cutoff, so the range of scores is narrow and r is shrunk toward 0.

9. $\bar{y} = a + b\bar{x}$ and $s_y = |b| s_x = -b s_x$, so $z_{y,i} = b(x_i - \bar{x})/(-b s_x) = -z_{x,i}$. Then $r = \frac{1}{n-1}\sum(-z_{x,i}^2) = -\frac{n-1}{n-1} = -1$.

10. $\hat{y}(4) \approx 13.43 + 3.57 = 17.0$ ms (the mean, since $\bar{x} = 4$); $\hat{y}(10) \approx 13.43 + 8.93 = 22.4$ ms. The 10 KB prediction is less trustworthy: it lies outside the observed range 1–7 KB (extrapolation), and $r \approx 0.53$ from six points is a weak basis.

Part II

From Sample to Population

Chapter 5

Sampling and Study Design

Chapters 1–4 described the data in front of us. This chapter asks where the data came from. Every later formula (standard errors, confidence intervals, p-values, regression coefficients) assumes something about how the numbers were collected. If that assumption is false, the formula still produces a number, and the number is wrong. No amount of arithmetic repairs a bad sample or a confounded comparison.

Two questions organise the chapter:

- **Can I generalise?** Does what I saw in my sample hold for the larger group I care about? That depends on *how units were selected*.
- **Can I claim cause?** Did the treatment produce the difference, or did something else? That depends on *how treatments were assigned*.

5.1 Populations, parameters, and statistics

Suppose you run a fleet of 6 cache servers and want the average p50 latency (each server's median request time) across the fleet. Their true p50 latencies (ms) are

$$10, \quad 14, \quad 18, \quad 22, \quad 26, \quad 48.$$

The fleet average is $(10+14+18+22+26+48)/6 = 138/6 = 23$ ms. If you can measure all six, you are done. Usually you cannot: the fleet has 60,000 machines, or measuring one is expensive, or the "fleet" is every user who will ever visit your site. So you measure a few and reason about the rest.

Definition 5.1 (Population, sample, parameter, statistic)**.** The **population** is the full set of units you want to learn about. A **sample** is the subset you actually observe. A **parameter** is a number describing the population (fixed, usually unknown). A **statistic** is a number computed from the sample (known, but it changes from sample to sample).

The rule of thumb for notation: *Greek letters for parameters, Latin letters for statistics.* Hats also mark estimates: $\hat{p}$ is "our estimate of p".

Quantity	Parameter (population)	Statistic (sample)	Example
Mean	μ	$\bar{x}$	mean latency
Standard deviation	σ	s	spread of latency
Variance	σ^2	s^2	
Proportion	p	$\hat{p}$	conversion rate
Regression slope	β (or β_1)	b (or $\hat{\beta}_1$)	price per m^2
Correlation	ρ	r	height vs. weight
Size	N	n	fleet vs. sample size

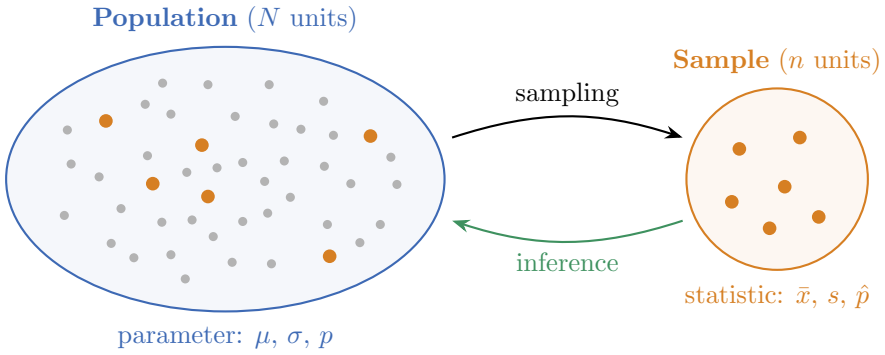


Figure 5.1: Sampling goes from population to sample; inference goes back. The statistic is computed; the parameter is what we actually want. How the arrow labelled "sampling" was carried out decides whether the return arrow is justified.

Common trap

The "population" is defined by the question, not by the data you happen to have. If your logs contain only users who logged in this week, the population for a question about "all customers" is larger than your logs. Write down the target population before you look at the sample.

5.2 Simple random sampling

Definition 5.2 (Simple random sample)**.** A **simple random sample** (SRS) of size n from a population of size N is a sample chosen so that every subset of n units has the same chance of being the one selected.

Back to the six servers. Take an SRS of size $n = 2$. There are $\binom{6}{2} = 15$ possible pairs, each with probability $1/15$. Here are all 15 sample means:

$$12,\ 14,\ 16,\ 16,\ 18,\ 18,\ 20,\ 20,\ 22,\ 24,\ 29,\ 31,\ 33,\ 35,\ 37.$$

(For example, the pair $\{10, 14\}$ gives 12, and $\{26, 48\}$ gives 37.) Their average is $345/15 = 23$, exactly the population mean μ. Individual samples miss, some by a lot, but the misses in each direction

cancel on average: the high misses (up to +14) are fewer but larger than the low ones (down to −11).

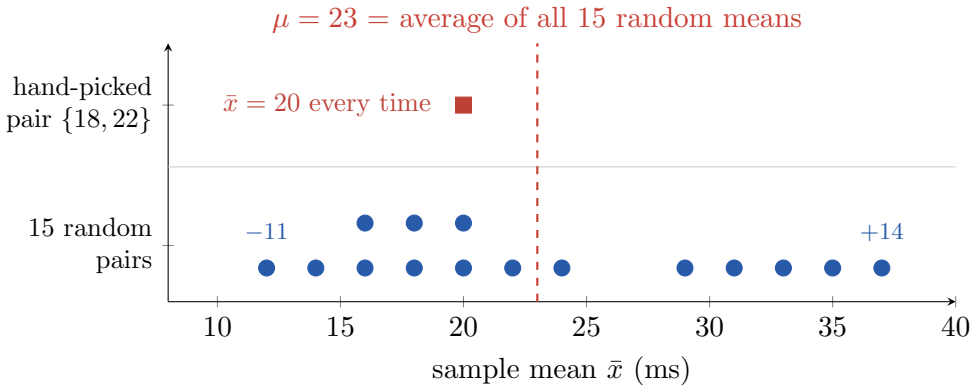


Figure 5.2: Bottom row: all 15 equally likely SRS means for $n = 2$ from the fleet $\{10, 14, 18, 22, 26, 48\}$, one dot per pair (stacked dots are two pairs with the same mean). Misses run from −11 to +14 ms but balance exactly at the true mean 23. Top row: a hand-picked "representative" pair that skips the odd server lands at 20, and lands there every time it is chosen.

Definition 5.3 (Unbiased)**.** A statistic is **unbiased** for a parameter if its average over all possible samples equals the parameter. Its **bias** is Bias = $\mathbb{E}$[statistic] − parameter, where $\mathbb{E}[\cdot]$ (the expected value) means exactly that average over all possible samples, each weighted by its probability.

Pause

Pause and think: replace the 48 ms server by one at 100 ms. Is the average of the 15 sample means still equal to the new μ? (Yes. Each server still appears in exactly 5 of the 15 pairs, so the grand average is still the plain average of the six servers, now $190/6 \approx 31.7$ ms.)

Under simple random sampling, $\bar{x}$ is unbiased for μ. The calculation above is the whole proof in miniature: each unit appears in the same number of samples, so each unit gets equal weight in the grand average. Chapter 6 studies the *spread* of the 15 values (the standard error); here the point is the *centre*.

5.2.1 Why randomness beats judgement

The intuitive alternative to random selection is to pick "representative" units by hand. An engineer asked for two typical servers would likely choose 18 and 22 and skip the odd one at 48. That gives $\bar{x} = 20$, and it gives 20 *every time* the engineer is asked. The error is −3 ms, and it never averages out.

Why it works

Human judgement has preferences: it avoids outliers, favours units that are convenient, familiar, or look "normal". Those preferences are correlated with the thing being measured, so they produce a bias that repeats on every sample. A random mechanism has no preferences. It can still produce a bad sample, but bad in an unpredictable direction, and the size of that

chance error is something probability theory can quantify (Chapter 6). Randomness converts an unknown systematic error into a known random one.

In practice an SRS needs a **sampling frame**: a list of every unit, from which a random number generator picks. For servers, users, or transactions this list usually exists in a database.

```
import numpy as np
rng = np.random.default_rng(seed=42)       # fixed seed: reproducible
user_ids = np.arange(1_000_000)            # the sampling frame
srs = rng.choice(user_ids, size=500, replace=False)
# SQL equivalent (approx.): SELECT * FROM users ORDER BY random() LIMIT 500;
```

5.3 Other sampling designs

Simple random sampling is the reference design. Three variations trade simplicity for precision or cost; one shortcut abandons randomness altogether.

5.3.1 Stratified sampling

A product has 1000 users: 800 on the free tier and 200 paid. Free users average 3 sessions per week (SD 2); paid users average 15 (SD 5). The population mean is

$$\mu = \frac{800 \cdot 3 + 200 \cdot 15}{1000} = \frac{2400 + 3000}{1000} = 5.4.$$

An SRS of 100 users might, by bad luck, contain 10 paid users or 30. Since paid users are so different, that luck swings $\bar{x}$ a lot. Instead, split the population into the two tiers (**strata**) and sample each separately: 80 free and 20 paid, in proportion to their sizes. Then combine:

$$\bar{x}_{\text{strat}} = 0.8\,\bar{x}_{\text{free}} + 0.2\,\bar{x}_{\text{paid}}.$$

Definition 5.4 (Stratified random sample)**.** Divide the population into non-overlapping groups (**strata**) that differ from each other. Take an SRS within each stratum. Combine stratum means with weights $W_h = N_h/N$ equal to each stratum's share of the population.

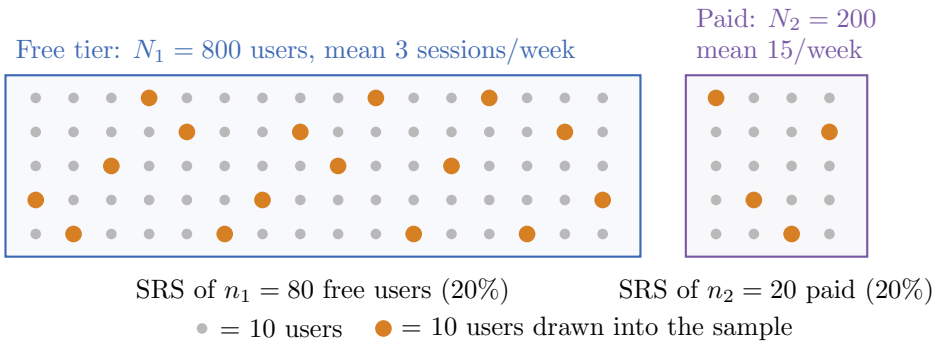


Figure 5.3: Proportional stratified sampling: the same 20% fraction is drawn at random inside each tier (orange dots: 16 of 80 on the left, 4 of 20 on the right). The paid tier is guaranteed exactly its fair share of the sample, so luck in the mix of tiers, which matters because the tiers average 3 and 15 sessions, is removed.

How much does this help? Chapter 6 develops standard errors properly; here is the preview, ignoring the small correction for sampling without replacement. For an SRS the standard error of $\bar{x}$ is $\sigma/\sqrt{n}$, where σ^2 is the variance of the whole population. That variance has two parts:

$$\begin{aligned} \text{within tiers:} \quad & 0.8 \cdot 2^2 + 0.2 \cdot 5^2 = 3.2 + 5.0 = 8.2, \\ \text{between tiers:} \quad & 0.8\,(3 - 5.4)^2 + 0.2\,(15 - 5.4)^2 = 4.608 + 18.432 = 23.04, \\ \text{total:} \quad & \sigma^2 = 8.2 + 23.04 = 31.24. \end{aligned}$$

So $\text{SE}_{\text{SRS}} = \sqrt{31.24/100} \approx 0.559$. Stratification removes the between-tier part entirely, because the tier mix is fixed by design:

$$\text{SE}_{\text{strat}} = \sqrt{0.8^2 \cdot \frac{2^2}{80} + 0.2^2 \cdot \frac{5^2}{20}} = \sqrt{0.032 + 0.05} = \sqrt{0.082} \approx 0.286.$$

The same 100 users give roughly half the standard error. To get that from an SRS you would need about $31.24/0.082 \approx 381$ users.

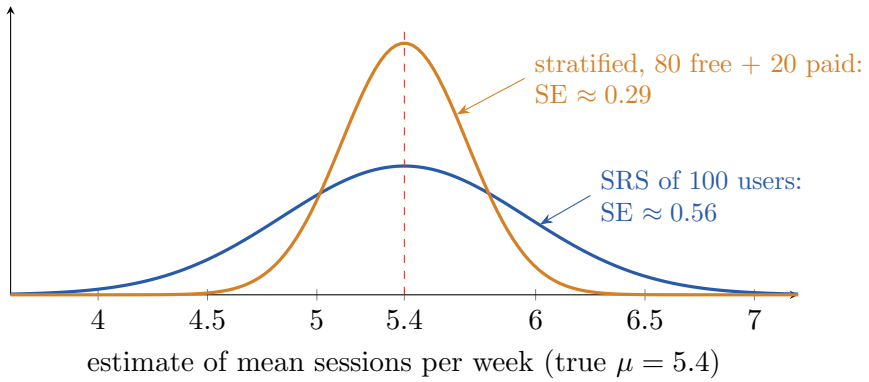


Figure 5.4: Where the estimate lands over many repeated samples of 100 users (normal approximation, Chapter 6). Both designs are centred on the true 5.4; the stratified curve is about half as wide, because the luck of how many paid users you happen to draw has been designed out.

Pause

Pause and think: if free and paid users had the same mean (5.4 each) but the same SDs as before, would stratifying still help? (Barely. The between-tier part would be 0, so σ^2 would equal the within-tier 8.2 and SRS would already achieve SE ≈ 0.286.)

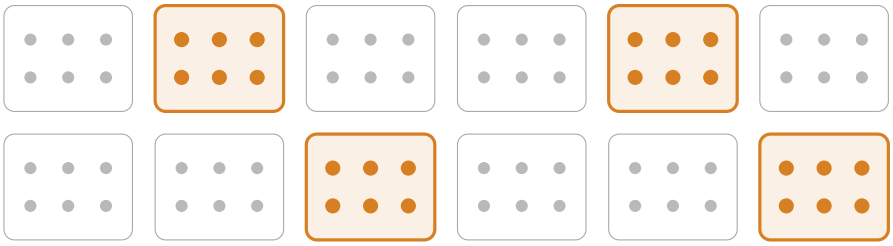
Intuition

Stratify on variables that (a) you know for every unit in advance and (b) strongly predict the outcome. Platform, country, plan tier, and device class are typical. The more different the strata are from each other, the larger the gain. Stratifying on something unrelated to the outcome costs nothing but gains nothing.

5.3.2 Cluster sampling

To survey on-call engineers about tooling, you could build a list of all 3000 engineers and pick 120 at random, which would mean interviewing in 40 offices. Instead, pick 4 teams of 30 at random and interview everyone on them. The teams are **clusters**.

Definition 5.5 (Cluster sample)**.** Divide the population into groups (**clusters**), take an SRS of clusters, and measure every unit inside the chosen clusters (or a random subsample of them).



each box is one team (dots: its engineers); 4 teams picked at random (orange)
everyone on a picked team is interviewed; nobody on the other teams is

Figure 5.5: Cluster sampling picks whole groups, here 4 teams, as in the on-call survey. It is cheap when travel or setup per group dominates the cost, but engineers on one team tend to resemble each other, so n clustered units carry less information than n independent ones.

Common trap

Strata and clusters are opposites. Strata should be *internally similar and different from each other*; you sample *from every* stratum. Clusters are ideally *internally diverse, each a mini-population*; you sample *some* clusters. Analysing a cluster sample as if it were an SRS understates the standard error, often badly. Users within the same company, requests from the same session, and pixels from the same image are all clusters.

5.3.3 Systematic sampling

To audit 10,000 log lines, pick a random start between 1 and 50 and then take every 50th line. This is a **systematic sample**. It is simple and spreads the sample evenly over the list. It behaves like an SRS when the list order is unrelated to the outcome.

It fails when the list has a period that lines up with the step. Traffic has a daily cycle. Sampling every 24th hour hits the same clock time every day.

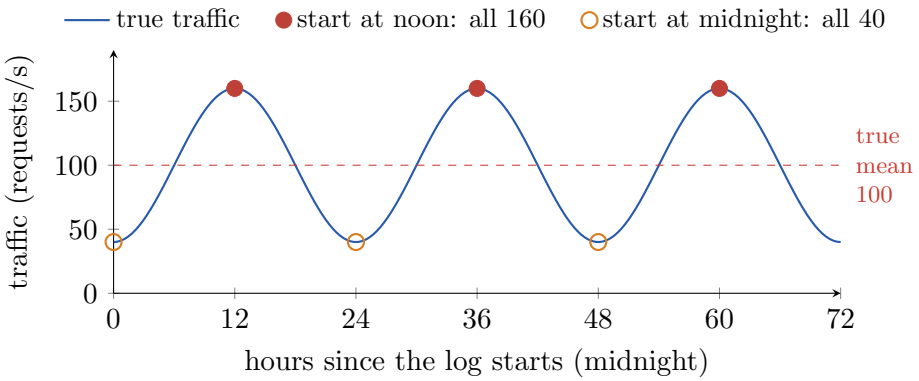


Figure 5.6: Systematic sampling with a step (24 h) equal to the traffic cycle. A noon start puts every sample at the daily peak (estimate 160); a midnight start puts every sample at the trough (estimate 40). A random start does not rescue it: whichever start you draw, all points share one phase, so the estimate never averages toward 100 within one sample.

5.3.4 Convenience sampling

A **convenience sample** uses whoever is easiest to reach: the users who answered an in-app pop-up, colleagues down the hall, the first 1000 rows of a table sorted by signup date. There is no random mechanism, so there is no way to compute its error. Convenience samples are useful for finding bugs and generating hypotheses. They are not a basis for estimating population numbers.

Design	How units are chosen	Use when
SRS	every subset of size n equally likely	you have a full list; default
Stratified	SRS inside each stratum	known groups differ on the outcome
Cluster	SRS of groups, all units inside	listing or reaching units is costly
Systematic	every kth unit, random start	list order unrelated to outcome
Convenience	whoever is easy	exploration only

5.4 Bias: errors that do not average out

A statistic can be wrong in two ways. **Chance error** (variance) changes from sample to sample and shrinks as n grows. **Bias** is a systematic offset that comes from the method and stays the same no matter how many units you collect.

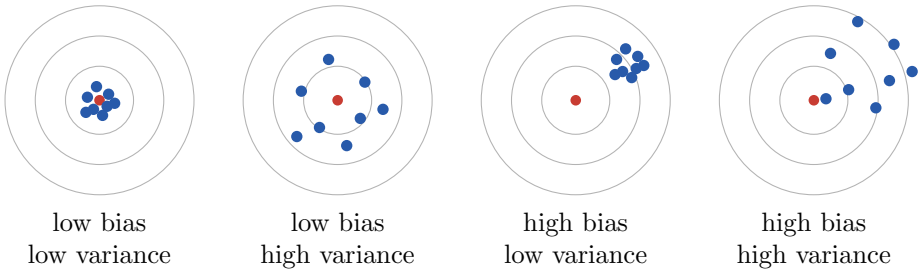


Figure 5.7: Bias versus variance. Red centre: the parameter. Blue dots: estimates from repeated samples. Bias is where the cloud is centred; variance is how wide it is. A larger sample shrinks the cloud but does not move its centre.

Most bias in practice comes from one of four sources.

5.4.1 Selection bias

Selection bias occurs when the way units enter the sample is related to the outcome. Surveying gym members about exercise habits, measuring app satisfaction among users who still have the app installed, estimating commute times from people who answered the survey at the office at 9 a.m.: in each case the selection process prefers certain values of the thing being measured.

5.4.2 Nonresponse bias

You email a satisfaction survey to 2000 customers. 800 reply (40%), and 640 of those are satisfied, so $\hat{p} = 640/800 = 80\%$. What can you say about all 2000?

The 1200 non-responders are unknown. In the worst case none of them are satisfied: $640/2000 = 32\%$. In the best case all are: $(640 + 1200)/2000 = 92\%$. If non-responders are satisfied at 50% (plausible: unhappy customers often ignore surveys), the truth is $(640 + 600)/2000 = 62\%$, which is 18 points below the headline.

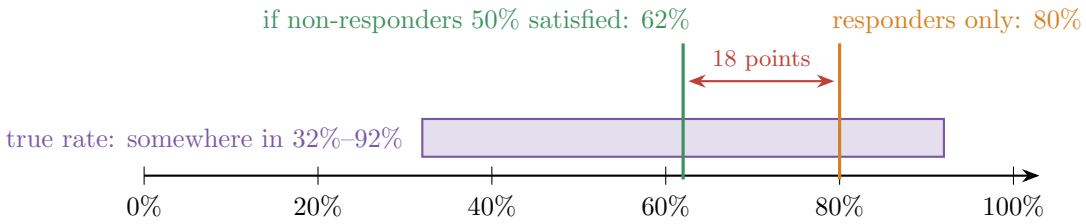


Figure 5.8: With a 40% response rate, the data alone pin the true satisfaction rate only to the purple 60-point-wide range (all 1200 non-responders unhappy at the left end, all happy at the right). The headline 80% sits near the top of it. A larger n of responders does not narrow this range; only a higher response rate does.

Common trap

Reporting $\hat{p}$ among responders as if it were the population value silently assumes non-responders look exactly like responders. That is an assumption about data you did not collect. Report the response rate next to every survey estimate, and treat low rates ($< 50\%$) with suspicion.

5.4.3 Survivorship bias

Survivorship bias is selection bias where the filter is "still around to be measured": units that failed, died, or dropped out vanish from the data, and the survivors look better than the whole group did.

History

In 1943 the US military was losing bombers over Europe and wanted to add armour. Armour is heavy, so it could only go where it mattered most. Officers examined bombers returning from missions and recorded where they had been hit. The holes clustered on the wings, fuselage, and tail; the engines had few. The natural conclusion was to armour where the holes were.
Abraham Wald, a Hungarian-born mathematician working at the Statistical Research Group at Columbia University (a wartime team of statisticians solving military problems), pointed out the selection: these were the planes that *came back*. Bullets do not prefer wings to engines. Planes hit in the engines mostly did not return, so they were missing from the data. The clean areas on survivors marked where hits were fatal, and armour belonged there. Wald went further and worked out how to estimate the vulnerability of each area from the survivors alone, an early example of correcting an estimate for data that are missing for a known reason.

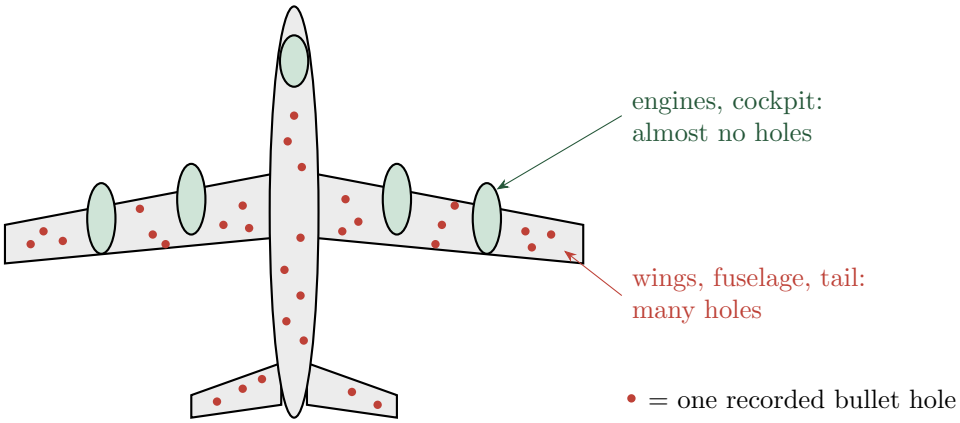


Figure 5.9: Hits recorded on bombers that returned (schematic). The data are conditioned on survival. Areas that look safe on survivors are the ones where a hit was most likely to bring the plane down, so the missing planes carry the important information.

Survivorship bias is everywhere in tech: studying only startups that succeeded, only code that is still in the repository, only users who did not churn, only requests that completed. Section 5.9 returns to the last one.

5.4.4 Response bias

Response bias is error in the answers rather than in who answers. Causes include leading wording ("Don't you agree the new design is cleaner?"), social desirability (people under-report drinking and over-report voting), recall error ("How many hours did you use the app last month?"), and the order in which options appear. Logged behaviour is usually more reliable than self-report for the same question. Wording effects can be large: changing "forbid" to "not allow" in otherwise identical questions has shifted survey answers by more than 20 percentage points.

5.5 Size does not cure bias: the Literary Digest poll

In 1936 the magazine *Literary Digest* mailed about 10 million straw-poll ballots (unofficial mock ballots asking "whom will you vote for?") for the US presidential election between Franklin Roosevelt and Alf Landon. About 2.4 million came back. The Digest predicted Landon would win with 57% to Roosevelt's 43%. Roosevelt won with about 61% of the vote, one of the largest landslides in US history.

George Gallup, with a sample of about 50,000 chosen to mirror the electorate, predicted a Roosevelt win with 56%. He also predicted, in advance, that the Digest would show Roosevelt near 44%.

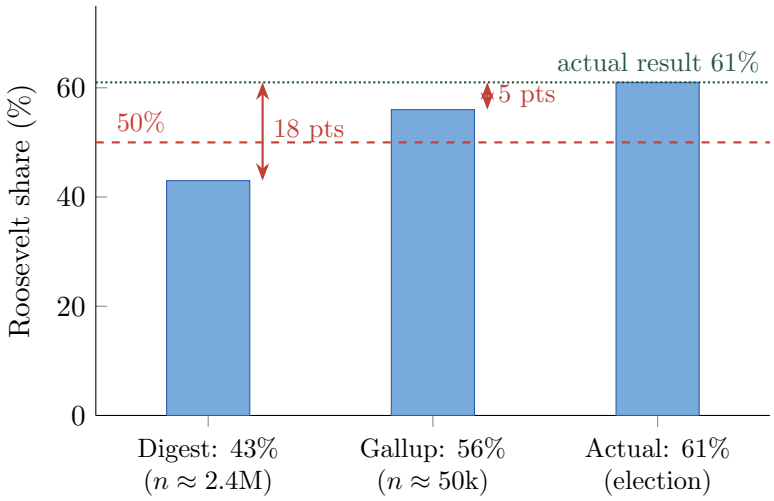


Figure 5.10: The largest poll was the worst by far. Red arrows: each poll's miss against the actual result (dotted line); dashed line: the 50% needed to win. The Digest was off by about 18 percentage points and called the wrong winner; Gallup, with a sample 48 times smaller, was off by about 5 points.

What went wrong? The Digest built its mailing list from its own subscribers, telephone directories, and car registrations. In 1936, during the Depression, phones and cars were markers of wealth, and wealthier voters leaned Republican (selection bias). Then only about a quarter of recipients replied, and Landon supporters were more motivated to reply (nonresponse bias). Both biases pushed the same way.

How large was the Digest's *chance* error? For a proportion near 0.43 and $n = 2.4$ million, the standard error (Chapter 6) is

$$\sqrt{\frac{0.43 \times 0.57}{2{,}400{,}000}} \approx 0.00032,$$

about 0.03 percentage points. The chance error was negligible. The entire 18-point miss was bias.

Pause

Pause and think: if the Digest had mailed 100 million ballots to the same kinds of lists, would its prediction have improved? (No. The SE would drop to about 0.005 points, and the estimate would sit even more firmly near 43%.)

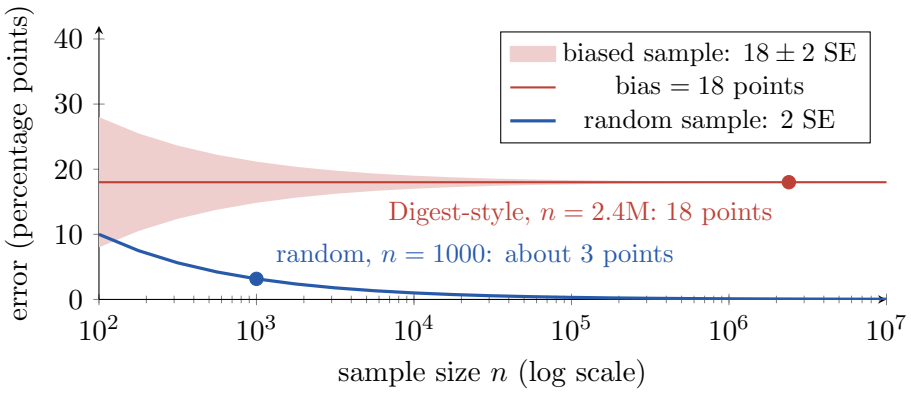


Figure 5.11: Typical error versus sample size for a proportion near 50%. A random sample's error ($2\ \text{SE} = 100/\sqrt{n}$ points) falls toward zero. A sample with 18 points of bias converges, with growing confidence, to the wrong answer. A random sample of 1000 beats a biased sample of any size.

What is really happening

Increasing n shrinks variance, never bias. A huge biased sample is worse than useless because its tiny standard error makes the wrong answer look precise. Big data from logs or scraped sources is often exactly this: millions of rows, collected by a process nobody randomised.

Connection

The total error of an estimate combines both pieces. The **mean squared error** (the average squared distance between estimate and parameter over repeated samples) splits as $\text{MSE} = \text{Bias}^2 + \text{Var}$. Chapter 14 proves this decomposition; Chapter 15 meets it again as the bias–variance trade-off in machine learning. The Digest had $\text{MSE} \approx 18^2 + 0.03^2$, all bias.

5.6 Observational studies and experiments

Sampling decides whom you measure. A second design choice decides whether you can claim cause and effect.

Definition 5.6 (Observational study, experiment)**.** In an **observational study** the researcher records the treatment each unit happens to receive (a feature used, a drug taken, a habit) without influencing it. In an **experiment** the researcher *assigns* the treatment.

5.6.1 Confounding

Your analytics show that users of a new "saved searches" feature have 85% 30-day retention, compared with 45% for non-users. Does the feature cause a 40-point lift?

Split users by engagement level, measured *before* the feature launched:

	Uses feature		Does not use	
Group	users	retained	users	retained
Heavy users	900	810 (90%)	100	90 (90%)
Light users	100	40 (40%)	900	360 (40%)
All	1000	850 (85%)	1000	450 (45%)

Within each engagement group, feature users and non-users retain at exactly the same rate. The 40-point gap comes entirely from *who* adopts the feature: 90% of adopters are heavy users, who retain well anyway.

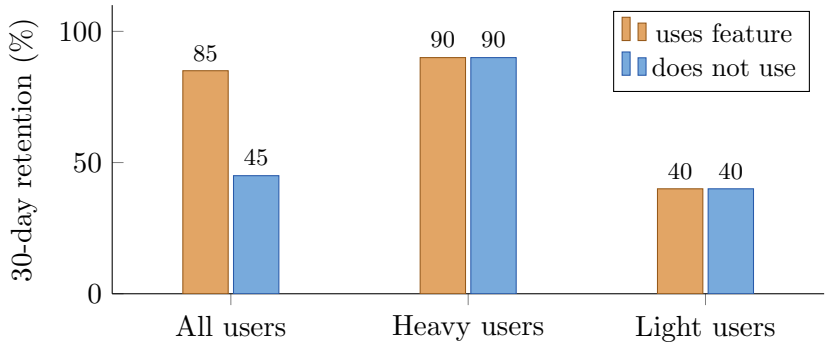


Figure 5.12: A 40-point gap overall, zero gap within each engagement level. Engagement drives both feature adoption and retention, so comparing adopters with non-adopters mixes the feature's effect with the engagement effect.

Definition 5.7 (Confounder)**.** A **confounder** (a hidden third factor that pushes on both sides of a comparison) is a variable that influences both the treatment and the outcome, creating an association between them that is not caused by the treatment.

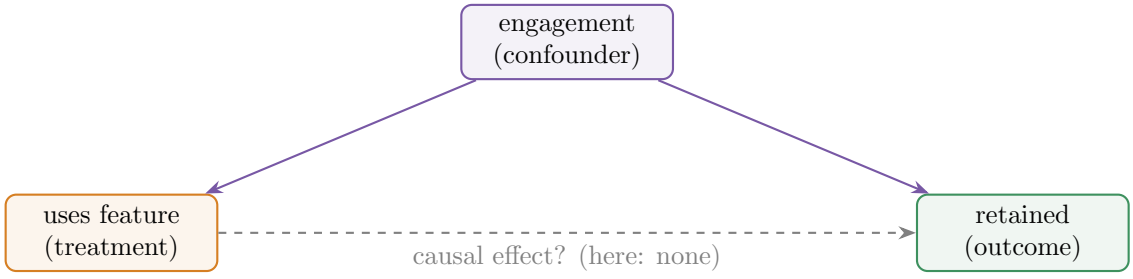


Figure 5.13: The confounding triangle. Arrows mean "influences". The observed association between treatment and outcome flows through the back path via engagement, whether or not the direct arrow exists.

Pause

Pause and think: in the table, what fraction of heavy users adopted the feature, and what fraction of light users? (900/1000 = 90% and 100/1000 = 10%.) That imbalance, combined

with heavy users retaining better, is the whole 40-point gap.

Splitting by engagement fixed this example because we knew the confounder and had measured it. In general you cannot list every confounder. Heavy users differ from light users in a hundred ways: job, device, motivation, how they found the product. Adjusting for the ones you thought of leaves the ones you did not. Chapter 13 shows how multiple regression adjusts for measured confounders and why it cannot fix unmeasured ones.

Common trap

"Correlation is not causation" is too weak a slogan. The useful version: an observational difference between groups equals the treatment effect *plus* the pre-existing difference between the kinds of units that chose each treatment. You see only the sum.

5.6.2 Randomised experiments

The fix is to take the choice away from the units. Flip a coin for each user: heads gets the feature, tails does not. Now heavy and light users, motivated and unmotivated, iOS and Android, all land in each group in roughly equal proportions. Any confounder, measured or not, is balanced *on average*.

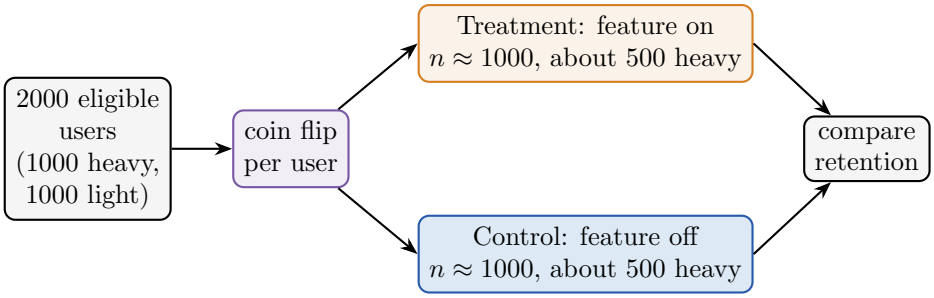


Figure 5.14: A randomised controlled experiment on the users of the confounding example. The coin, not the user, decides who gets the feature, so heavy users split roughly evenly between the arms instead of 900 to 100. The two groups then differ systematically only in the treatment, and a difference in outcomes beyond chance is caused by it.

Why it works

Before treatment, the two random groups are two random samples from the same pool. So their average engagement, age, device mix, and every other pre-treatment trait differ only by chance, and chance differences shrink as the groups grow. After treatment, if outcomes differ by more than chance can explain (Chapter 9 makes "more than chance" precise), the only systematic difference left is the treatment itself.

The essential ingredients of a good experiment:

- **Control group.** A comparison group that does not get the treatment. Without it, a before/after change could be seasonality, a marketing push, or a bug fix shipped the same week.

- **Randomisation.** Assignment by a random mechanism, never by judgement, convenience, or the units' own choice.
- **Replication.** Enough units that chance differences between groups are small relative to the effect you want to detect.

5.6.3 Blocking

Randomisation balances groups on average, but a particular assignment can be lopsided. If iOS users convert at 8% and Android users at 3%, a random split that happens to put 55% of iOS users in the treatment group adds noise. **Blocking** is stratification applied to experiments: group units by a known, important trait, then randomise separately within each block.

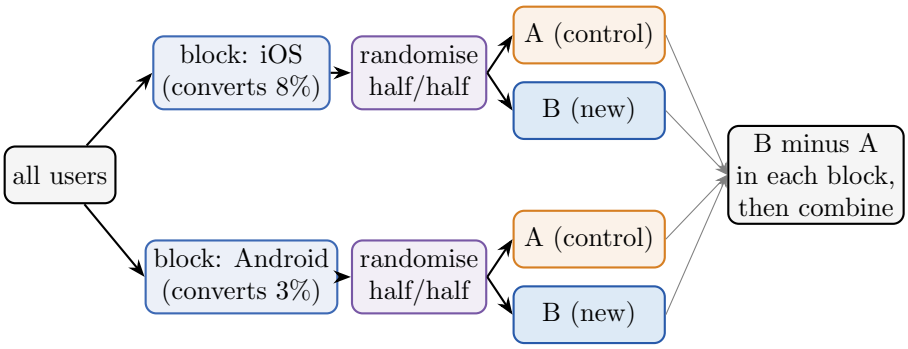


Figure 5.15: A randomised block design. Platform is fixed first, then each block is split at random, half to A and half to B, so both arms contain exactly the same iOS/Android mix. The 8% versus 3% platform difference cancels out of each within-block comparison instead of inflating its noise.

The slogan: *block what you can, randomise what you cannot.* Blocking handles the few big known factors; randomisation handles everything else.

5.6.4 Matched pairs

The extreme form of blocking is a block of size two. To test a new kernel setting, run each of 6 servers once with the old setting and once with the new, in random order. Measured mean latencies (ms):

Server	1	2	3	4	5	6
Old	120	95	210	150	80	175
New	112	90	198	146	75	163
Difference	8	5	12	4	5	12

The old and new means are 138.3 and 130.7 ms; the SDs across servers are about 49 and 47 ms. Compared as two separate groups, a 7.7 ms gap is buried in 48 ms of server-to-server spread. But every server improved. The differences have mean $46/6 \approx 7.67$ ms and SD about 3.6 ms. Pairing removes the server-to-server variation from the comparison.

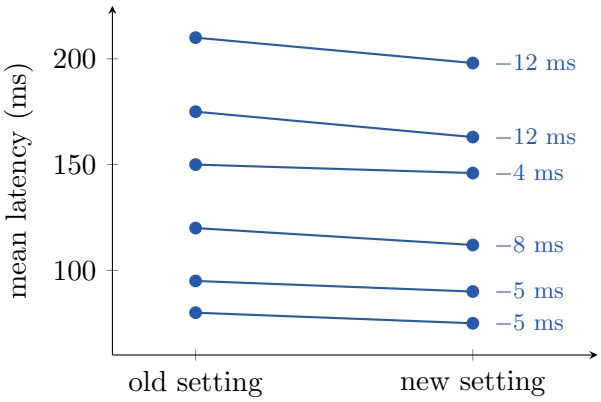


Figure 5.16: Matched pairs: each line is one server, labelled with its change. Levels differ hugely between servers (75 to 210 ms), but every line slopes down, by 4 to 12 ms. Analysing the differences uses that consistency; comparing the two columns as unrelated groups throws it away.

Matched pairs also covers before/after measurements on the same unit (with a randomised order or a control to rule out time trends), and twin studies. Chapter 9 gives the paired t-test that analyses such data.

5.6.5 Blinding and placebos

People who know they are treated behave differently. A patient told they are getting a new painkiller may report less pain whether or not the pill does anything: the **placebo effect**. A doctor who knows which patients got the drug may rate them more favourably.

- A **placebo** is a fake treatment (a sugar pill, a sham procedure) given to the control group so both groups have the same experience apart from the active ingredient.
- **Single-blind**: subjects do not know their group. **Double-blind**: neither subjects nor the people measuring outcomes know.

In software the analogue is usually automatic: users rarely know they are in an experiment. Blinding matters for the *analysts*. Deciding which metric is "primary", when to stop, and which users to exclude, *after* seeing which arm is winning, reintroduces judgement into a design built to exclude it (Chapter 11).

5.7 A/B tests are randomised experiments

An A/B test is the experiment of Figure 5.14 at web scale. Each user is assigned to variant A (control) or B (treatment) by a deterministic function of their user ID that behaves like a coin flip. For example, A has 10,000 users with 520 conversions (5.2%) and B has 10,000 users with 580 (5.8%). Because assignment was random, the 0.6-point difference is either caused by B or is chance; Chapters 9 and 10 decide which.

```
import hashlib
def variant(user_id: str, experiment: str = "checkout_v2") -> str:
```

```
    h = hashlib.sha256(f"{experiment}:{user_id}".encode()).hexdigest()
    return "B" if int(h, 16) % 100 < 50 else "A"    # 50/50 split
# Deterministic per user (same user, same variant on every visit),
# but unrelated to anything about the user: effectively random.
```

Common trap

Common ways A/B tests stop being randomised experiments:

- **Analysing only users who saw the change.** If B's new button is below the fold (off the first screen, visible only after scrolling), "users who saw it" are the users who scrolled, a self-selected group. Compare everyone *assigned* to each arm, whether or not they saw the change. This is called **intention-to-treat** analysis: you analyse the groups the coin created, so the comparison stays randomised.
- **Sample ratio mismatch.** You configured 50/50 but got 10,000 vs 9,700. Something (a crash, a redirect, a bot filter) removed users from one arm non-randomly. Investigate before reading any metric.
- **Comparing to last week instead of a concurrent control.** That is a before/after observational study.

5.8 Generalisation versus causation

The two kinds of randomness answer different questions:

- **Random sampling** (who is in the study) licenses *generalisation* from the sample to the population.
- **Random assignment** (who gets which treatment) licenses *causal* conclusions about the treatment.

They are independent. A study can have both, either, or neither.

	random assignment: **yes**	random assignment: **no**
random sampling: **yes**	**Causal + generalisable** A/B test on all traffic; trial on a random sample of customers	**Generalisable, not causal** random survey of users; opinion poll
random sampling: **no**	**Causal for these units only** clinical trial on volunteers; A/B test on beta users	**Neither** "users of feature X retain more" from whatever logs exist

Figure 5.17: What each design licenses. Rows: were units randomly sampled from the target population? Columns: were treatments randomly assigned? Most medical trials sit bottom-left (causal, but only for people like the volunteers); most dashboards sit bottom-right.

Intuition

An A/B test randomises assignment, so it is causal. Whether it generalises depends on who was in it. A test run during the holiday week, or only in one country, or only on the beta channel, tells you the effect for that group. Extending it to everyone is an assumption, not a conclusion.

Pause

Pause and think: a company emails a random 1% of all customers a discount, and compares their purchases with the other 99%. Which cell is it in? (Top-left. Every customer is in the study, so the "sample" is the whole population, and the discount was assigned at random. The result is causal and applies to all current customers.)

5.9 Selection in tech logs and data leakage

Logs feel like a census: every request, every user, nothing sampled. They are not. Logs are produced by code, and code decides what gets written. That decision is a selection mechanism, and it is rarely random.

5.9.1 Survivorship in latency logs

A service handles 1000 requests. Clients give up at 2000 ms; the server only writes a latency record when it sends a response, so the 40 requests that timed out leave no record. The 960 logged latencies look like this:

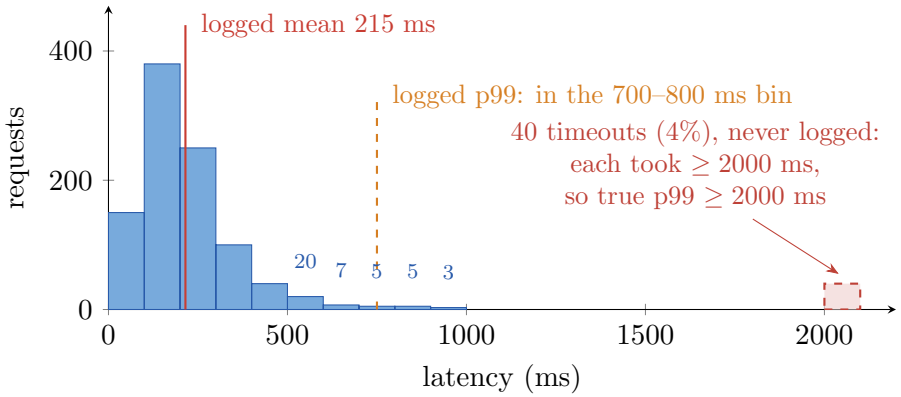


Figure 5.18: The logged latency histogram (blue, small numbers: counts in the thin tail bins) is conditioned on the request completing. The slowest 4% of requests (red, drawn at the 2000 ms timeout) are missing. Every summary computed from the log (mean, p95, p99) is biased low, and the tail percentiles are the most wrong: the logged p99 is under a second, the true one at least two.

Using bin midpoints, the logged mean is

$$\frac{50 \cdot 150 + 150 \cdot 380 + 250 \cdot 250 + \cdots + 950 \cdot 3}{960} = \frac{206{,}400}{960} = 215 \text{ ms}.$$

Each timed-out request took at least 2000 ms, so the true mean is at least

$$\frac{206{,}400 + 40 \cdot 2000}{1000} = \frac{286{,}400}{1000} \approx 286 \text{ ms}.$$

The logged p99 is the 951st of 960 sorted values, which falls in the 700–800 ms bin. The true p99 is the 990th of 1000; since 40 requests (4%) timed out, it is at least 2000 ms. The dashboard says p99 is under a second; users experience the opposite.

Common trap

Before trusting any metric from logs, ask: *which events fail to produce a log line?* Crashes, timeouts, users who bounce before analytics loads, clients with ad blockers, and requests dropped by a load balancer are typical. Log the attempt as well as the outcome, so failures are counted.

5.9.2 Other selection traps in product data

- **Active-user filters.** "Average session length among daily active users" rises when a change drives casual users away, even though no one's sessions got longer. The denominator changed.
- **Opt-in telemetry.** Users who enable diagnostics skew toward power users and developers.
- **Feedback loops.** A recommender only logs clicks on items it chose to show. Training the next model on those logs teaches it about the old model's choices, not about user taste in general.

5.9.3 Data leakage

Data leakage is the machine-learning form of the same error: the training data contain information that will not be available, or will look different, when the model is used. Two common forms:

- **Target leakage.** A churn model uses "number of support tickets in the last 30 days" computed over a window that ends *after* the cancellation. The feature partly records the outcome. Accuracy looks excellent offline and collapses in production.
- **Selection leakage.** A fraud model is trained only on transactions that a previous rule system flagged for review, because only those have labels. The model learns the rule system's blind spots as "safe".

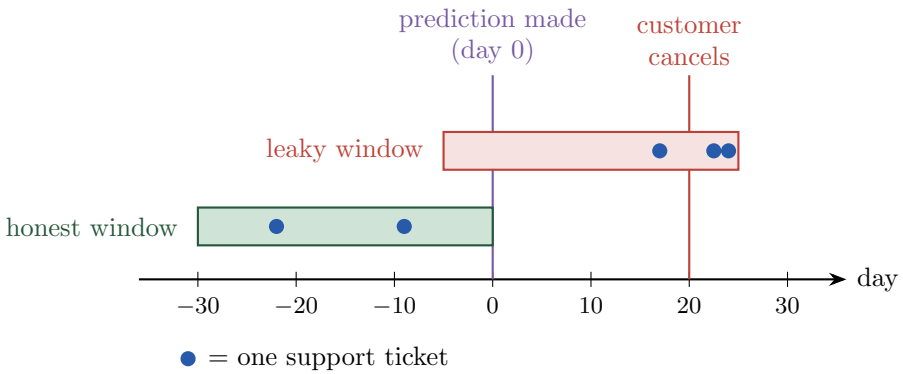


Figure 5.19: Target leakage in the churn model. The honest feature counts tickets in the 30 days before the prediction is made (green: 2 tickets). The leaky window ends five days after the cancellation (red), so it counts the tickets filed around the cancellation (3). Offline the feature looks prophetic; in production those tickets do not exist yet when the model must decide.

Connection

Chapter 15 treats train/test splits and cross-validation. Every one of those procedures assumes the test data are a random sample from the same process the model will face. The sampling ideas of this chapter are exactly what that assumption means, and leakage is what happens when it fails.

Work it yourself

Simulate nonresponse bias. Each of 100,000 customers is satisfied with probability 0.6. Satisfied customers respond with probability 0.5, unsatisfied ones with probability 0.2. Compute $\hat{p}$ among responders. Then raise n to 10^7: does $\hat{p}$ approach 0.6?
Answer. Responders: $0.6 \cdot 0.5 = 0.30$ satisfied, $0.4 \cdot 0.2 = 0.08$ unsatisfied, so $\hat{p} \to 0.30/0.38 \approx 0.789$. More customers make $\hat{p}$ settle more tightly around 0.789, not 0.6.

```
import numpy as np
rng = np.random.default_rng(0)
n = 100_000
satisfied = rng.random(n) < 0.6
responds = rng.random(n) < np.where(satisfied, 0.5, 0.2)
print(satisfied.mean(), satisfied[responds].mean())   # ~0.600, ~0.789
```

5.10 Summary

- A **parameter** (μ, σ, p, β) describes the population; a **statistic** $(\bar{x}, s, \hat{p}, b)$ is computed from a sample and estimates it.
- A **simple random sample** gives every subset the same chance; under it $\bar{x}$ is unbiased for μ. Random selection beats expert judgement because its errors are unsystematic and measurable.
- **Stratified** sampling (SRS within known groups) improves precision; **cluster** sampling (whole groups) cuts cost but loses precision; **systematic** sampling fails on periodic lists; **convenience**

samples support no inference.

- **Bias** does not shrink with n. Main sources: selection, nonresponse, survivorship, response bias. The 1936 Literary Digest poll (2.4 million responses, 18 points wrong) lost to Gallup's far smaller, better-designed sample.
- **Observational** comparisons are vulnerable to **confounding**. **Randomised experiments** with a control group balance all confounders on average, measured or not.
- **Blocking** and **matched pairs** remove known sources of variation; **blinding** and **placebos** remove expectation effects.
- Random **sampling** $\Rightarrow$ generalisation. Random **assignment** $\Rightarrow$ causation. An A/B test is causal for the users it included.
- Logs are selected data. Ask what fails to get logged; check for leakage before trusting an offline model metric.

Random sampling makes $\bar{x}$ right on average. Chapter 6 asks the next question: how far from μ is one particular $\bar{x}$ likely to be? The answer, the standard error, is what turns a random sample into a margin of error.

5.11 Exercises

1. Name the sampling design. (a) A school district picks 10 of its 60 schools at random and surveys every teacher in them. (b) A bank samples 1% of accounts at random within each of its five regions. (c) A QA engineer checks every 200th build artifact, starting from a random one of the first 200. (d) A product manager posts a poll in the company chat.
2. For each number, say whether it is a parameter or a statistic and give its symbol. (a) Of 400 randomly chosen sessions, 7.5% ended in a crash. (b) The mean age of all 12,340 employees on the HR roster is 34.1. (c) In a sample of 50 houses, the fitted price increase per extra bedroom is \$18,000.
3. A survey is sent to 500 users; 150 respond and 120 of them say "yes". Give the range of possible values for the proportion of all 500 who would say "yes". What would it be if non-responders said "yes" at half the responders' rate?
4. A city has three districts making up 50%, 30%, and 20% of households. A stratified sample gives mean monthly electricity use of 400, 600, and 1000 kWh respectively. Estimate the city-wide mean. Why would the unweighted average of the three stratum means be wrong?
5. An observational study finds that people who carry lighters have higher lung cancer rates. Name the likely confounder and draw the confounding triangle. What experiment would settle whether lighters cause cancer, and why is it unnecessary?
6. A service's client timeout is 5 s, and timed-out requests are not logged. The logged mean latency is 300 ms, and 2% of all requests time out. Give a lower bound for the true mean latency.
7. A metric is sampled once per week, every Monday at 09:00, for a year. What can go wrong? Propose a better design with the same number of measurements.

8. Place each study in the 2×2 grid of Figure 5.17. (a) 5000 customers chosen at random from all customers are each randomly shown one of two pricing pages. (b) Employees who volunteered are randomly split between two keyboard layouts. (c) A random sample of households reports how much TV their children watch and their school grades.

9. For the Literary Digest poll, compute the standard error of $\hat{p}$ with $n = 2.4$ million and $\hat{p} = 0.43$. Explain why that number says nothing about how accurate the poll was.

10. You want to test whether a new code-review tool reduces review time. Ten teams volunteer. Teams differ greatly in typical review time. Describe a matched-pairs or blocked design and say what it gains over randomly giving the tool to 5 teams.

Answers

1. (a) Cluster. (b) Stratified. (c) Systematic. (d) Convenience (voluntary response).

2. (a) Statistic, $\hat{p} = 0.075$. (b) Parameter, $\mu = 34.1$ (the whole population is measured). (c) Statistic, $b = 18{,}000$ dollars per bedroom.

3. Minimum $120/500 = 24\%$; maximum $(120 + 350)/500 = 94\%$. Responders say yes at 80%; at half that, 40% of 350 is 140, giving $(120 + 140)/500 = 52\%$.

4. $0.5 \cdot 400 + 0.3 \cdot 600 + 0.2 \cdot 1000 = 200 + 180 + 200 = 580$ kWh. The unweighted average $(400 + 600 + 1000)/3 \approx 667$ kWh over-weights the small, high-usage district.

5. Smoking causes both carrying a lighter and lung cancer. The experiment would randomly assign people to carry lighters; randomisation would balance smoking across groups. It is unnecessary because splitting by smoking status already answers the question: among smokers, and separately among non-smokers, carrying a lighter is not associated with cancer.

6. True mean $\geq 0.98 \cdot 300 + 0.02 \cdot 5000 = 294 + 100 = 394$ ms.

7. Weekly and daily cycles mean Monday 09:00 may be systematically unusual (e.g. a weekly peak), so the estimate is biased. Better: 52 times chosen at random over the year, or one random time within each week (stratified by week).

8. (a) Random sampling and random assignment: causal and generalisable to all customers. (b) Random assignment only: causal for volunteers. (c) Random sampling only: generalisable association, not causal.

9. $\sqrt{0.43 \cdot 0.57/2{,}400{,}000} \approx 0.00032$, i.e. 0.03 percentage points. The SE measures chance error only; the poll's error was 18 points of bias, which the SE cannot see.

10. Pair teams with similar typical review times (5 pairs) and flip a coin within each pair for who gets the tool; or measure each team with and without the tool in a random order. Comparing within pairs removes the large team-to-team differences, so a real effect is visible with far fewer teams. Note the result generalises only to teams like the volunteers.

Chapter 6

Sampling Distributions and Standard Error

Chapter 5 showed how to draw a sample that is random, so that it is representative on average. Random has a price: a different random sample gives a different answer. You measured the mean latency of 64 requests and got 48.7 ms. Run the same measurement tomorrow on 64 different requests and you will get a different number. How different? That question has a precise answer: the **standard error**, the typical size of the gap between your estimate and the truth. Every confidence interval and test later in the book is built from it.

We need three facts about expectation and variance, one change of viewpoint (a statistic is a random variable), and one simulation. Chapters 7 to 11 rest on what follows.

6.1 Three tools: expectation and variance rules

A **random variable** X is a number whose value is decided by chance: the roll of a die, the latency of the next request, the height of a randomly chosen person. Its **expectation** $\mathbb{E}[X]$ is the long-run average value, and its **variance** $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$ is the long-run average squared distance from that centre. The **standard deviation** $\mathrm{SD}(X) = \sqrt{\mathrm{Var}(X)}$ puts that spread back in the original units. Two random variables are **independent** if knowing the value of one tells you nothing about the other, like two separate dice. The proofs of the rules below are in *Probability From the Ground Up*; here we use them as tools.

Example 6.1 (One fair die)**.** $X \in \{1, \dots, 6\}$, each with probability $1/6$.

$$\mathbb{E}[X] = \frac{1+2+3+4+5+6}{6} = 3.5, \qquad \mathrm{Var}(X) = \frac{(2.5^2 + 1.5^2 + 0.5^2) \cdot 2}{6} = \frac{17.5}{6} = \frac{35}{12} \approx 2.917.$$

Theorem 6.1 (The three rules)**.** *For random variables X, Y and constants a, b:*

1. ***Expectation is linear:*** $\mathbb{E}[aX + bY] = a\,\mathbb{E}[X] + b\,\mathbb{E}[Y]$. *This holds* always*, independent or not.*
2. ***Scaling squares the variance:*** $\mathrm{Var}(aX + b) = a^2\,\mathrm{Var}(X)$*, so* $\mathrm{SD}(aX + b) = |a|\,\mathrm{SD}(X)$*. Adding a constant shifts but does not spread.*

3. ***Variances of independent variables add:*** *if* X *and* Y *are independent,* $\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$ *and also* $\mathrm{Var}(X-Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$.

Tiny checks. Converting latency from ms to seconds is $Y = X/1000$. If $\mathrm{SD}(X) = 30$ ms then $\mathrm{SD}(Y) = 0.03$ s and $\mathrm{Var}(Y) = 900/10^6 = 0.0009$ s^2. Adding a fixed 5 ms network overhead changes the mean by 5 and the SD by nothing.

Now roll two independent dice X_1, X_2. Rule 3 gives $\mathrm{Var}(X_1 + X_2) = 35/12 + 35/12 = 35/6 \approx 5.833$. Compare that with doubling a single roll, $2X_1$. Rule 2 gives $\mathrm{Var}(2X_1) = 4 \cdot 35/12 = 35/3 \approx 11.667$. Same mean (7), twice the variance. Figure 6.1 shows why: two independent dice partly cancel each other's extremes; one die doubled never cancels.

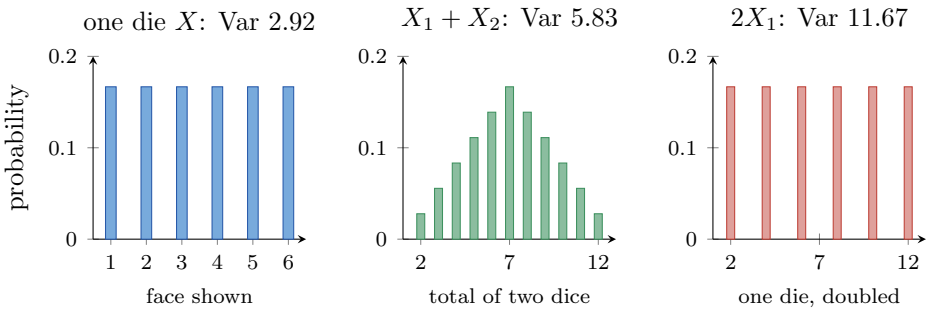


Figure 6.1: Compare the middle and right panels: both range from 2 to 12 with mean 7. The total of two independent dice (green) piles up near 7 because a high roll is often offset by a low one; a 2 or a 12 needs both dice extreme. Doubling one die (red) keeps every extreme as likely as the centre, so its variance is twice as large.

Common trap

Variances add; standard deviations do not. For two independent dice, $\mathrm{SD}(X_1 + X_2) = \sqrt{5.833} = 2.415$, not $1.708 + 1.708 = 3.416$. And for *dependent* variables even variances do not simply add: a covariance term appears (Chapter 4). Every SE formula in this chapter assumes independent observations.

6.2 A statistic is a random variable

A **statistic** is any number computed from a sample: the mean, the median, the proportion of conversions, the p99 latency, the slope of a regression line. Once you have your data, a statistic is just a number, $\bar{x} = 48.7$. Before you collect the data, the sample $X_1, \ldots, X_n$ is random, so the statistic computed from it is random too. We write it with capital letters, $\bar{X}$, to mark that.

Definition 6.1 (Sampling distribution)**.** The **sampling distribution** of a statistic is its probability distribution over all the random samples of size n that could have been drawn. The **standard error** of the statistic is the standard deviation of its sampling distribution.

Figure 6.2 shows the thought experiment: draw a sample, compute the statistic, put the sample back, repeat many times, and histogram the results.

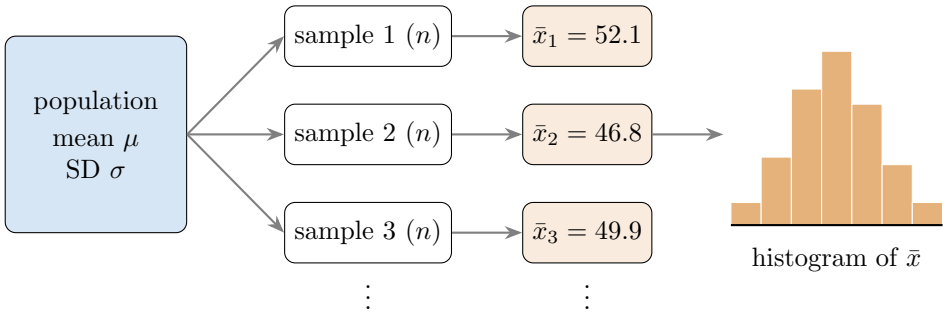


Figure 6.2: The sampling distribution is the histogram you would get from repeating the whole study many times. In practice you run it once; theory (or simulation) tells you what the histogram would look like.

6.2.1 A sampling distribution you can list by hand

Take a population of three numbers, $\{2, 4, 9\}$. Its mean is $\mu = 5$ and its variance (dividing by the population size 3) is

$$\sigma^2 = \frac{(2-5)^2 + (4-5)^2 + (9-5)^2}{3} = \frac{9+1+16}{3} = \frac{26}{3} \approx 8.667.$$

Draw a sample of $n = 2$ *with replacement*, so the two draws are independent. There are $3 \times 3 = 9$ equally likely ordered samples:

sample	(2,2)	(2,4)	(4,2)	(2,9)	(9,2)	(4,4)	(4,9)	(9,4)	(9,9)
$\bar{x}$	2	3	3	5.5	5.5	4	6.5	6.5	9

The average of the nine means is $45/9 = 5 = \mu$. Their variance is

$$\frac{(-3)^2 + 2(-2)^2 + (-1)^2 + 2(0.5)^2 + 2(1.5)^2 + 4^2}{9} = \frac{9+8+1+0.5+4.5+16}{9} = \frac{39}{9} \approx 4.333 = \frac{\sigma^2}{2}.$$

The mean of $\bar{X}$ equals μ and its variance is half the population variance. Section 6.4 shows this is no coincidence.

Pause

Pause and think: if you drew samples of size $n = 3$ from $\{2, 4, 9\}$ instead, what would you guess the variance of $\bar{X}$ to be? (Answer at the end of Section 6.4.)

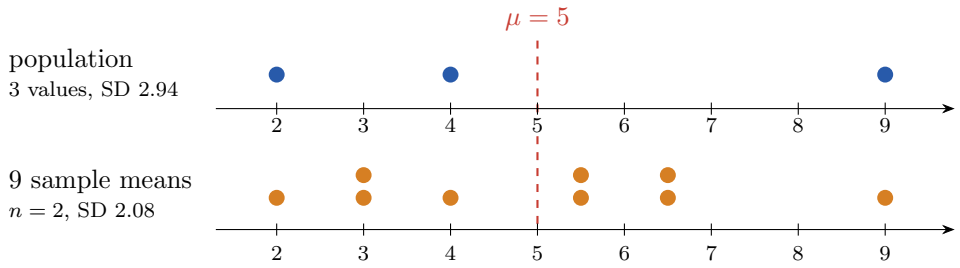


Figure 6.3: Top: the three population values. Bottom: the means of all nine equally likely samples of two draws; a stacked pair is two samples with the same mean, such as (2,4) and (4,2). The means balance at the same centre, $\mu = 5$, but crowd towards it: SD 2.08 instead of 2.94 (variance 4.33 instead of 8.67).

6.3 The sampling distribution of the mean, by simulation

Real populations are too large to list. We simulate instead. Our population is request latency, modelled as a fixed 20 ms floor plus an exponential delay with mean 30 ms. This population is strongly right-skewed: a long tail of slow requests stretches to the right. Its **skewness**, a unitless measure of that lopsidedness (0 for a symmetric shape, positive for a long right tail; Chapter 3), is 2. It has

$$\mu = 50 \text{ ms}, \qquad \sigma = 30 \text{ ms}, \qquad \text{median} = 20 + 30 \ln 2 \approx 40.8 \text{ ms}.$$

For each $n \in \{1, 4, 16, 64\}$ we drew 10,000 samples of size n, computed each sample's mean, and histogrammed the 10,000 means (`numpy` seed 6).

```
import numpy as np
rng = np.random.default_rng(6)
for n in [1, 4, 16, 64]:
    samples = 20 + rng.exponential(30, size=(10_000, n))
    means = samples.mean(axis=1)          # one mean per simulated study
    print(n, means.mean().round(2), means.std(ddof=1).round(2))
```

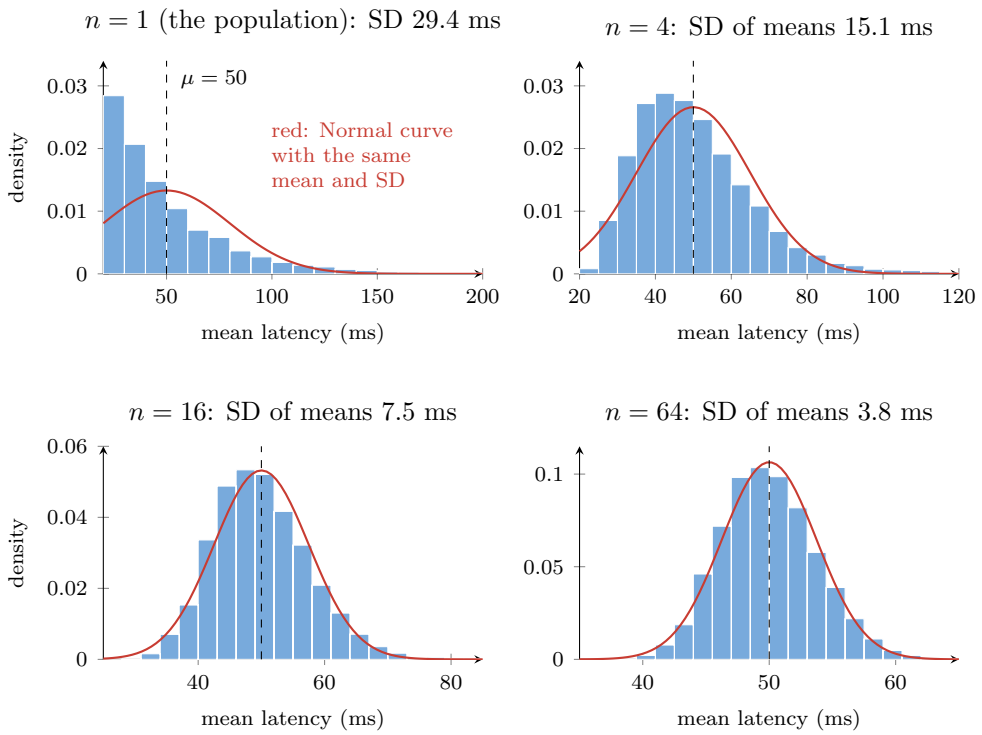


Figure 6.4: Histograms of 10,000 simulated sample means from the skewed latency population, for four sample sizes; each panel title gives the SD of its means. Watch two things: the SD halves each time n is quadrupled (note the shrinking x-ranges), and the lopsided shape fades until the red Normal curve $\mathcal{N}(50, 30^2/n)$ fits the bars. Every panel stays centred on $\mu = 50$ (dashed).

The simulation output:

n	mean of the $\bar{x}$'s	SD of the $\bar{x}$'s	$30/\sqrt{n}$	skewness of the $\bar{x}$'s
1	49.56	29.38	30.00	1.98
4	50.12	15.08	15.00	1.04
16	50.02	7.47	7.50	0.48
64	49.98	3.76	3.75	0.29

Three patterns stand out. The centre stays at $\mu = 50$. The spread follows $30/\sqrt{n}$ almost exactly: quadrupling n halves it. The shape becomes more symmetric; the skewness falls roughly like $2/\sqrt{n}$ (theory gives $2, 1, 0.5, 0.25$). The next three sections explain each pattern.

6.4 Deriving the centre and the spread

Let $X_1, \ldots, X_n \overset{\text{iid}}{\sim}$ some population with mean μ and variance σ^2. "iid" means independent and identically distributed: each draw comes from the same population and does not influence the others. The sample mean is $\bar{X} = \frac{1}{n}(X_1 + \cdots + X_n)$.

Theorem 6.2 (Mean and variance of the sample mean)**.**

$$\mathbb{E}[\bar{X}] = \mu, \qquad \mathrm{Var}(\bar{X}) = \frac{\sigma^2}{n}, \qquad \mathrm{SE}(\bar{X}) = \mathrm{SD}(\bar{X}) = \frac{\sigma}{\sqrt{n}}.$$

Why it works

Centre. By linearity (rule 1),

$$\mathbb{E}[\bar{X}] = \frac{1}{n}\big(\mathbb{E}[X_1] + \cdots + \mathbb{E}[X_n]\big) = \frac{1}{n} \cdot n\mu = \mu.$$

No independence was needed. $\bar{X}$ is *unbiased*: on average it hits the target.
Spread. Variances of independent variables add (rule 3), so the *sum* has variance

$$\mathrm{Var}(X_1 + \cdots + X_n) = n\sigma^2.$$

Dividing by n multiplies variance by $1/n^2$ (rule 2):

$$\mathrm{Var}(\bar{X}) = \frac{1}{n^2} \cdot n\sigma^2 = \frac{\sigma^2}{n}.$$

The sum's variance grows like n, but the division shrinks it by n^2. The net effect is $1/n$ on the variance, hence $1/\sqrt{n}$ on the SD.

Check with the tiny population: $\sigma^2/n = (26/3)/2 = 13/3 \approx 4.333$. Check with the simulation: $30/\sqrt{16} = 7.5$, observed 7.47. (The pause question: with $n = 3$ the variance would be $(26/3)/3 = 26/9 \approx 2.889$.)

6.4.1 Why $\sqrt{n}$, and what it costs you

The standard error falls like $1/\sqrt{n}$, not $1/n$. To halve the error you need four times the data. To cut it by 10 you need 100 times the data.

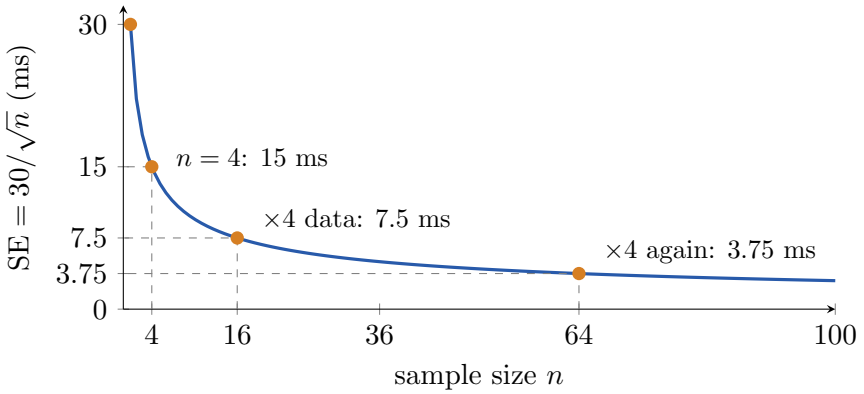


Figure 6.5: Standard error of the mean against n for $\sigma = 30$ ms. Each quadrupling of the sample size halves the error. The curve flattens fast: the first few observations buy a lot of precision, later ones buy little.

Intuition

Why not $1/n$? Errors in independent observations point in random directions. Adding n of them is like a random walk: the total drifts about $\sqrt{n}$ steps from zero, not n steps, because many errors cancel. Dividing that $\sqrt{n}$ drift by n leaves $1/\sqrt{n}$.

Pause

Pause and think: an A/B dashboard shows a conversion-rate SE of 0.4 points after one week. Roughly what will it show after four weeks of the same traffic? (Half: 0.2 points, since four times the data halves the SE.)

Example 6.2 (Planning a sample size)**.** A benchmark of 25 runs gives SE $= \sigma/5$. The team wants the SE cut to a third. They need $n = 25 \times 3^2 = 225$ runs. Generally, $n_{\text{new}} = n_{\text{old}} \cdot (\text{SE}_{\text{old}} / \text{SE}_{\text{new}})^2$.

6.5 Standard deviation versus standard error

These two numbers sound alike and answer different questions.

- **SD** (σ, estimated by s): how much *individual observations* vary. A property of the population. Does not shrink with n.
- **SE** ($\sigma/\sqrt{n}$): how much the *statistic* would vary across repeated studies. A property of your estimation procedure. Shrinks with n.

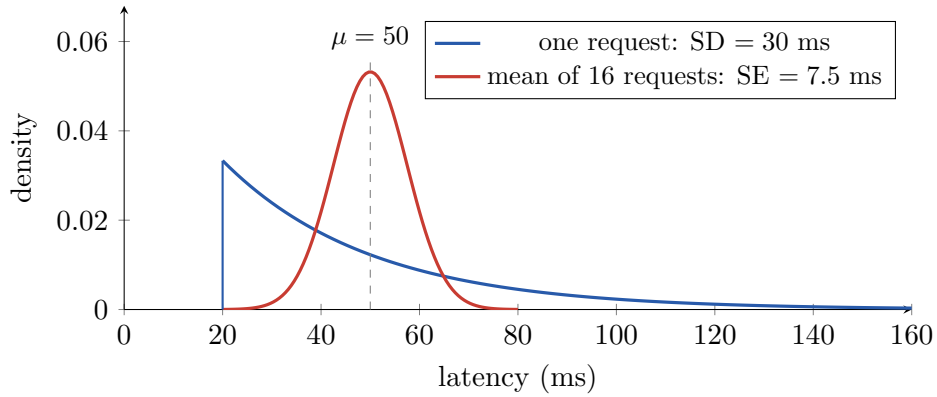


Figure 6.6: The population of single latencies (blue) and the sampling distribution of the mean of 16 of them (red, Normal approximation). Collecting more data does not change the blue curve. It only squeezes the red one.

Common trap

Error bars of ±SE presented as if they described individuals. A paper reports "mean recovery time 12.0 ± 0.4 days (SE), $n = 400$." The SD is $0.4\sqrt{400} = 8$ days. Individual patients vary by about 8 days, not 0.4 (Figure 6.7). Always check which one a ± means. Rule of thumb: if you want to predict *one new observation*, use SD; if you want to know *how well*

you pinned down the mean, use SE.
The reverse mistake also happens: reporting SD bars on a comparison of two means makes a solid difference look like noise.

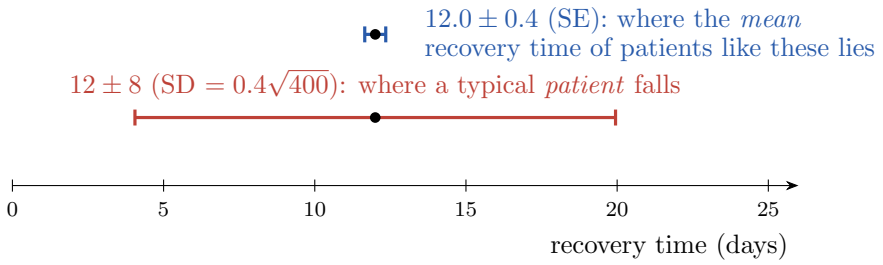


Figure 6.7: The recovery-time report drawn to scale. The SE bar (blue) is so short it barely shows: with $n = 400$ the mean is pinned down to within half a day. The SD bar (red) is 20 times wider: individual patients routinely recover anywhere from about 4 to 20 days. Reading the blue bar as if it described patients is the trap.

6.6 The sampling distribution of a proportion

A proportion is a mean in disguise. Code each visitor as $X_i = 1$ if they convert and 0 otherwise. Then the sample proportion is

$$\hat{p} = \frac{\text{number of 1s}}{n} = \frac{X_1 + \cdots + X_n}{n} = \bar{X}.$$

Each X_i is Bernoulli with $\mathbb{E}[X_i] = p$ and

$$\mathrm{Var}(X_i) = \mathbb{E}[X_i^2] - p^2 = p - p^2 = p(1-p),$$

using $X_i^2 = X_i$ for a 0/1 variable. Theorem 6.2 then gives everything for free.

Theorem 6.3 (Sample proportion)**.** *For n independent 0/1 observations with success probability p,*

$$\mathbb{E}[\hat{p}] = p, \qquad \mathrm{SE}(\hat{p}) = \sqrt{\frac{p(1-p)}{n}}.$$

Example 6.3 (Conversion rate)**.** A landing page converts at $p = 0.10$. With $n = 400$ visitors, $\mathrm{SE}(\hat{p}) = \sqrt{0.1 \cdot 0.9/400} = \sqrt{0.000225} = 0.015$, i.e. 1.5 percentage points. Observed rates of 8.5% or 11.5% are one SE from the truth and entirely routine.

The numerator $p(1-p)$ is largest at $p = 0.5$, where it equals 0.25 (Figure 6.8). So $\mathrm{SE}(\hat{p}) \le 0.5/\sqrt{n}$ for every p. A poll of $n = 1000$ has SE $\le 0.5/\sqrt{1000} = 0.0158$, and twice that, 0.032, is the "±3 points" margin printed under news polls (Chapter 7 explains the factor of about 2).

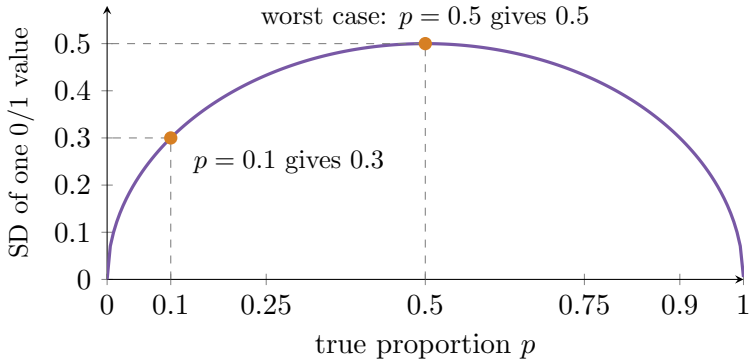


Figure 6.8: The per-observation SD of a 0/1 variable, $\sqrt{p(1-p)}$. It peaks at $p = 0.5$ and is symmetric. Divide by $\sqrt{n}$ to get $\mathrm{SE}(\hat{p})$. Using 0.5 gives a conservative SE when p is unknown.

When p is near 0 or 1 the sampling distribution of $\hat{p}$ is lopsided for small n, because $\hat{p}$ cannot go below 0. Figure 6.9 shows $p = 0.05$. The usual rule is that the Normal approximation is fine when both $np \geq 10$ and $n(1-p) \geq 10$.

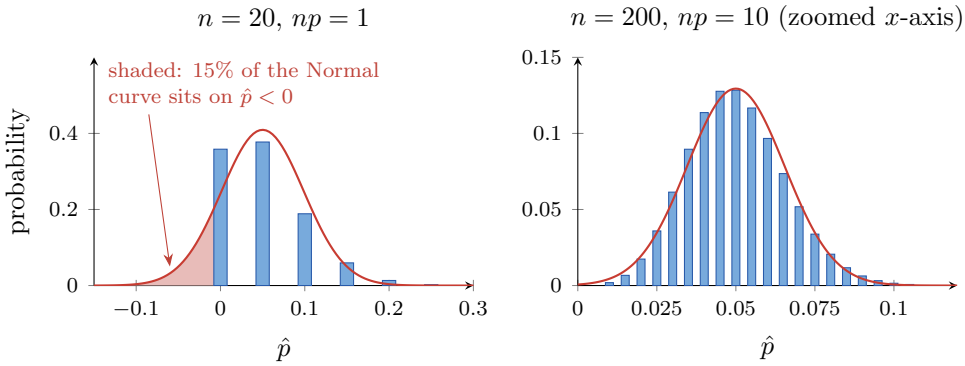


Figure 6.9: Exact (binomial) sampling distributions of $\hat{p}$ when $p = 0.05$ (blue bars, one per possible count of conversions), with the Normal approximation in red. At $n = 20$ the bars are lopsided and the Normal curve puts 15% of its probability on impossible negative proportions (shaded). At $n = 200$, where $np = 10$, the bars are nearly symmetric and the curve fits them. The right panel's x-axis is zoomed so that each bar is visible.

6.7 The shape: Central Limit Theorem and Law of Large Numbers

Theorem 6.2 gives the centre and spread of $\bar{X}$ for any population. The shape comes from the Central Limit Theorem (CLT).

Theorem 6.4 (Central Limit Theorem)**.** *If $X_1, \ldots, X_n \overset{iid}{\sim}$ a population with mean μ and finite variance σ^2, then for large n*

$$\bar{X} \overset{approx}{\sim} \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right), \qquad \textit{equivalently} \qquad Z = \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \overset{approx}{\sim} \mathcal{N}(0,1).$$

The proof is in *Probability From the Ground Up*. What matters here is what it buys: once you know

μ and SE, you know the whole sampling distribution, so you know, for example, that $\bar{X}$ lands within 2 SE of μ about 95% of the time. Chapter 7 turns this sentence into confidence intervals.

History

Abraham de Moivre, a French mathematician working in London, was stuck in 1733 on a gambler's computation: the probability of, say, between 3400 and 3600 heads in 6000 coin tosses needs a sum of thousands of enormous binomial coefficients. He found that the binomial probabilities trace out a bell-shaped curve, the curve we now call the Normal density, so the sum could be replaced by an area under it. Pierre-Simon Laplace extended the result around 1810 from coin tosses to sums of almost any independent quantities, and used it to put error bars on astronomical averages. That generalisation is the Central Limit Theorem. It is the reason one bell curve and one number, the standard error, are enough to describe the uncertainty of almost any average.

6.7.1 When is n "large enough"?

There is no universal number. "$n \geq 30$" is folklore that works for mildly skewed data and fails for heavily skewed data. The governing quantity is the population's skewness γ. The skewness of $\bar{X}$ is exactly

$$\gamma_{\bar{X}} = \frac{\gamma}{\sqrt{n}}.$$

(Our latency population has $\gamma = 2$; the simulated skewness of the means was $1.98, 1.04, 0.48, 0.29$ against theory $2, 1, 0.5, 0.25$.) A skewness below about 0.3 is usually close enough to Normal for intervals and tests, which needs $n \geq (\gamma/0.3)^2$: at least 45 for $\gamma = 2$, but about 1100 for $\gamma = 10$.

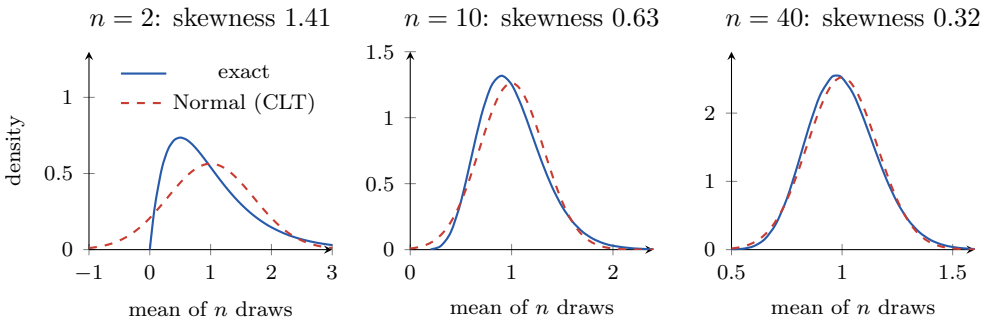


Figure 6.10: Exact density of the mean of n exponential observations with mean 1 (blue; skewness $2/\sqrt{n}$, given in each title) against the CLT's Normal approximation (red, dashed). At $n = 2$ the Normal curve even puts mass below zero, where no mean can fall. At $n = 10$ the exact peak still sits left of the Normal peak. At $n = 40$, with skewness near the 0.3 threshold, the two nearly coincide.

Common trap

The CLT needs a finite variance. For extremely heavy-tailed data (some file sizes, some incomes; mathematically, Pareto or "power-law" tails, where the chance of exceeding x falls only like $1/x^{\alpha}$ with $\alpha < 2$) the sample mean converges painfully slowly or not at all, and a few huge values dominate every sample. For such data report medians or quantiles (Chapter

3), or use the bootstrap (Chapter 8) with care.

6.7.2 The Law of Large Numbers

The CLT describes the *fluctuations* of $\bar{X}$. The Law of Large Numbers (LLN) is the cruder statement that those fluctuations vanish:

$$\bar{X}_n \to \mu \quad \text{as } n \to \infty.$$

It follows directly from $\mathrm{Var}(\bar{X}) = \sigma^2/n \to 0$: a quantity whose spread around μ shrinks to zero must end up at μ. (The formal step is Chebyshev's inequality, which bounds the chance of landing k SDs away by $1/k^2$; see *Probability From the Ground Up*.) Figure 6.11 shows a single running average of latency draws settling down inside the funnel $\mu \pm 2\sigma/\sqrt{n}$.

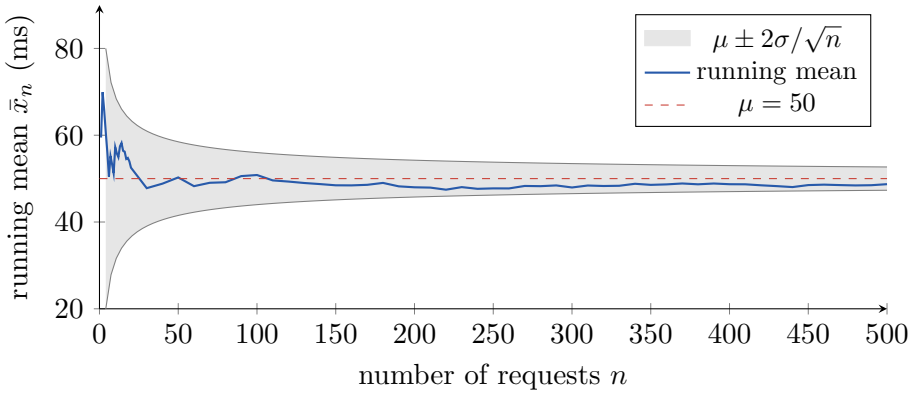


Figure 6.11: One stream of 500 simulated latencies. The running mean wanders early (it is 69.9 ms after two requests) and then settles near $\mu = 50$, ending at 48.7 ms. The shaded funnel narrows like $1/\sqrt{n}$: the LLN says the mean converges, and the SE says how fast.

Common trap

The LLN does not say that luck "evens out" by compensation. After a run of slow requests, the next request is no more likely to be fast. The early excess is not cancelled; it is *diluted* by the growing denominator.

6.8 Estimated standard error

The formula $\sigma/\sqrt{n}$ contains σ, the population SD, which you do not know. Replace it with the sample SD s (Chapter 2, with the $n-1$ divisor):

$$\widehat{\mathrm{SE}}(\bar{X}) = \frac{s}{\sqrt{n}}, \qquad \widehat{\mathrm{SE}}(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}.$$

In practice "the SE" almost always means this estimated version.

Example 6.4 (Eight latencies)**.** Measured latencies (ms): $42, 35, 61, 48, 120, 39, 55, 44$. The sum is 444, so $\bar{x} = 55.5$. The squared deviations sum to 5254, so $s = \sqrt{5254/7} = \sqrt{750.57} \approx 27.40$ ms and

$$\widehat{\mathrm{SE}} = \frac{27.40}{\sqrt{8}} = \frac{27.40}{2.828} \approx 9.69 \text{ ms}.$$

Report: mean latency 55.5 ms, SE 9.7 ms. With only 8 skewed observations, the Normal shape for $\bar{X}$ is doubtful and the plug-in s is itself noisy; Chapter 7 handles the second issue with the t distribution.

Example 6.5 (Estimated SE of a proportion)**.** 48 of 400 visitors converted: $\hat{p} = 0.12$ and $\widehat{\mathrm{SE}} = \sqrt{0.12 \cdot 0.88/400} = \sqrt{0.000264} \approx 0.0162$.

```python
import numpy as np
from scipy import stats
x = np.array([42, 35, 61, 48, 120, 39, 55, 44])
print(x.std(ddof=1) / np.sqrt(len(x)))   # 9.686
print(stats.sem(x))                       # same: scipy uses ddof=1
```

6.9 Standard error of a difference

A/B tests, before/after comparisons and treatment/control studies all estimate a *difference* $\bar{X}_A - \bar{X}_B$. If the two groups are independent, rule 3 (variances add, even for a difference) gives

$$\mathrm{Var}(\bar{X}_A - \bar{X}_B) = \mathrm{Var}(\bar{X}_A) + \mathrm{Var}(\bar{X}_B), \qquad \mathrm{SE}(\bar{X}_A - \bar{X}_B) = \sqrt{\mathrm{SE}_A^2 + \mathrm{SE}_B^2} = \sqrt{\frac{\sigma_A^2}{n_A} + \frac{\sigma_B^2}{n_B}}.$$

Example 6.6 (A/B latency test)**.** Variant A: $n_A = 100$, $s_A = 30$ ms, so $\mathrm{SE}_A = 3.00$. Variant B: $n_B = 150$, $s_B = 40$ ms, so $\mathrm{SE}_B = 40/\sqrt{150} \approx 3.27$.

$$\mathrm{SE}(\bar{x}_A - \bar{x}_B) = \sqrt{\frac{900}{100} + \frac{1600}{150}} = \sqrt{9 + 10.67} = \sqrt{19.67} \approx 4.43 \text{ ms}.$$

Adding the SEs would give 6.27 ms, an overstatement of 41%.

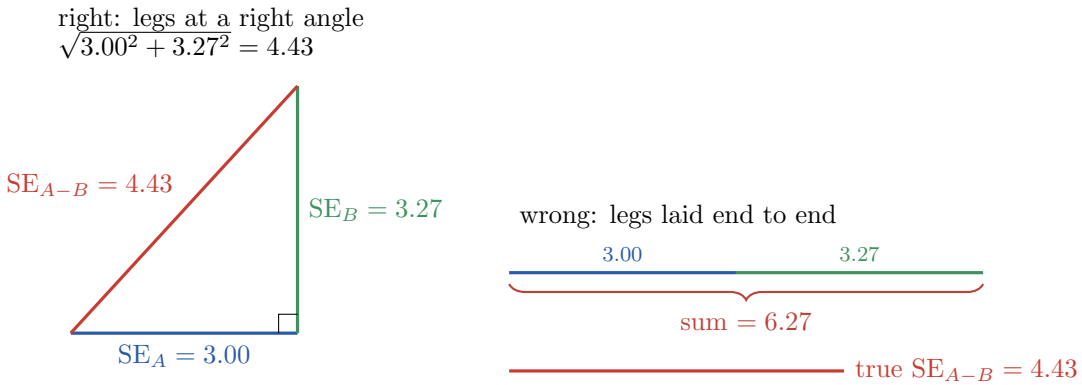


Figure 6.12: The A/B example drawn to scale. Standard errors of independent estimates combine like the legs of a right triangle (left): the SE of the difference is the hypotenuse, 4.43 ms. Laying them end to end (right) gives 6.27 ms, 41% too large; the hypotenuse is always shorter than the sum of the legs.

Why it works

Why does the difference not have a *smaller* variance? Subtracting $\bar{X}_B$ means multiplying it by $a = -1$, and rule 2 says that multiplies its variance by $a^2 = +1$. Noise in B pushes the difference around just as much whichever sign it carries. Independence then makes the two variances add.

Common trap

The formula needs independent groups. In a paired design (the same 50 servers measured before and after a patch) compute the 50 differences first and use $s_{\text{diff}}/\sqrt{50}$. Treating paired data as independent usually *overstates* the SE because it ignores the positive correlation between the two measurements on the same server (Chapter 9).

6.10 Statistics beyond the mean: the median

Every statistic has a sampling distribution and an SE, not only the mean. The median is the natural next case. We simulated 10,000 samples of $n = 25$ heights from $\mathcal{N}(170, 10^2)$ and recorded both the mean and the median of each.

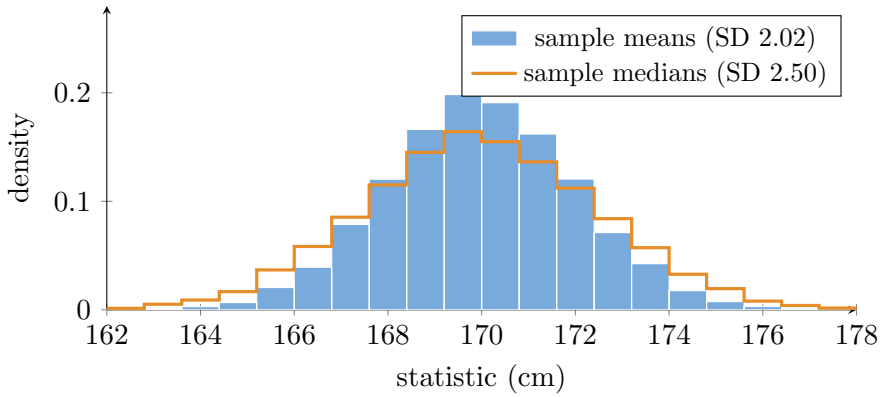


Figure 6.13: Sampling distributions of the mean and median for $n = 25$ draws from $\mathcal{N}(170, 10^2)$, 10,000 simulated samples. Both are centred on the true centre, 170 cm. The orange outline (medians) is lower in the middle and spills further into both tails than the blue bars (means): for Normal data the median is the noisier estimate, SD 2.50 against 2.02.

The mean's SD matches $10/\sqrt{25} = 2$. The median's SD, 2.50, matches the large-sample formula below: $1.2533 \times 2 = 2.51$.

Theorem 6.5 (Large-sample SE of the median)**.** *If the population has density f and median m with $f(m) > 0$, then for large n the sample median is approximately Normal with mean m and*

$$\text{SE}(\textit{median}) \approx \frac{1}{2f(m)\sqrt{n}}.$$

For a Normal population $f(m) = 1/(\sigma\sqrt{2\pi})$, which gives

$$\text{SE}(\textit{median}) \approx \frac{\sigma\sqrt{2\pi}}{2\sqrt{n}} = \sqrt{\frac{\pi}{2}}\,\frac{\sigma}{\sqrt{n}} \approx 1.2533\,\frac{\sigma}{\sqrt{n}}.$$

Why it works

The median is the point with half the sample below it. Count how many observations fall below the true median m. That count is binomial with $p = 1/2$, so the *fraction* below m has SE $\sqrt{0.25/n} = 0.5/\sqrt{n}$. This wobble lives on the vertical (probability) axis of the CDF. To turn it into a horizontal wobble in the median, divide by the slope of the CDF at m, which is the density $f(m)$. Result: $0.5/(f(m)\sqrt{n}) = 1/(2f(m)\sqrt{n})$. A steep CDF (lots of data near the median) pins the median down; a flat one lets it slide (Figure 6.14).

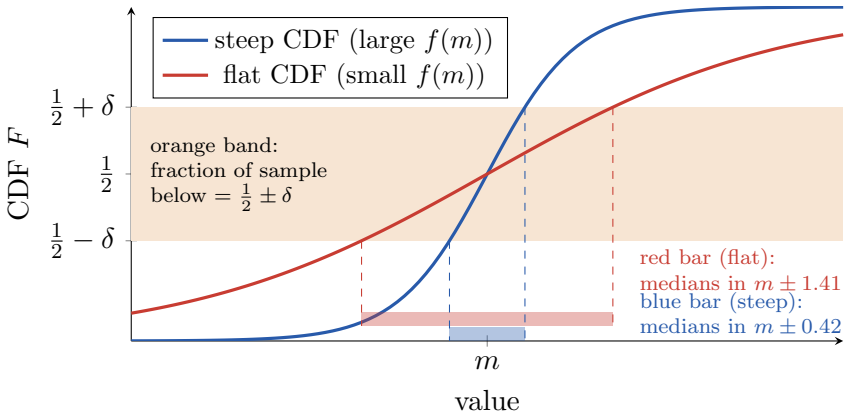


Figure 6.14: Why SE(median) $\propto 1/f(m)$. The fraction of the sample below any point wobbles by about $\delta = 0.5/\sqrt{n}$ (orange band). Follow the band across to each CDF and down to the value axis: through the steep CDF it pins the median to a narrow range (blue bar); through the flat CDF the same band gives a range more than three times wider (red bar). The CDF's slope at m is the density $f(m)$.

Example 6.7 (How much data does the median cost?)**.** Normal data, $\sigma = 20$, $n = 400$: $\text{SE}(\bar{X}) = 20/20 = 1.00$, while SE(median) ≈ 1.2533. Since SE scales as $1/\sqrt{n}$, the median needs $1.2533^2 = \pi/2 \approx 1.571$ times as much data to match the mean: about 628 observations instead of 400.

That is the price for Normal data. For skewed or outlier-prone data the comparison can flip. In our latency population, $m \approx 40.8$ ms and $f(m) = \frac{1}{30}e^{-\ln 2} = 1/60$, so SE(median) $\approx 60/(2\sqrt{n}) = 30/\sqrt{n}$, exactly the same as the mean's $\sigma/\sqrt{n}$. With a single wild outlier (120 ms in the eight latencies above) the median barely moves while the mean and s both jump (Figure 6.15). Robustness is often worth the efficiency cost.

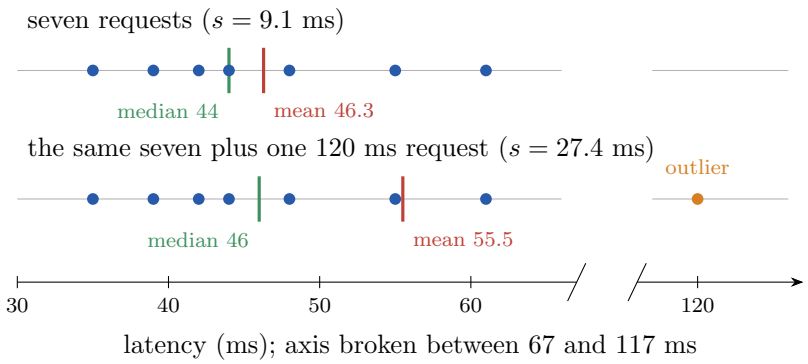


Figure 6.15: The eight latencies from the example above, with and without the one slow request. Adding the 120 ms value drags the mean (red) 9.2 ms to the right and triples s, but moves the median (green) only 2 ms. The axis is broken so the outlier fits while the seven typical values stay separately visible.

Common trap

The median formula needs $f(m)$, which you do not know and which is hard to estimate from a small sample. For discrete or heavily tied data (ratings 1 to 5, latencies rounded to 10 ms) the formula can be badly off. This is exactly the situation the bootstrap was invented for.

6.10.1 Most statistics have no tidy formula

The mean, a proportion, and a difference of means have exact, simple SEs. The median has only an approximate one that depends on an unknown density. For the p99 latency, a trimmed mean, a correlation, a ratio of two means, or the AUC of a classifier (the area under its ROC curve, a ranking-quality score between 0.5 and 1), a usable closed form is rare or messy. Chapter 8 gives a single recipe that works for nearly all of them: approximate the sampling distribution by resampling your own data (the bootstrap).

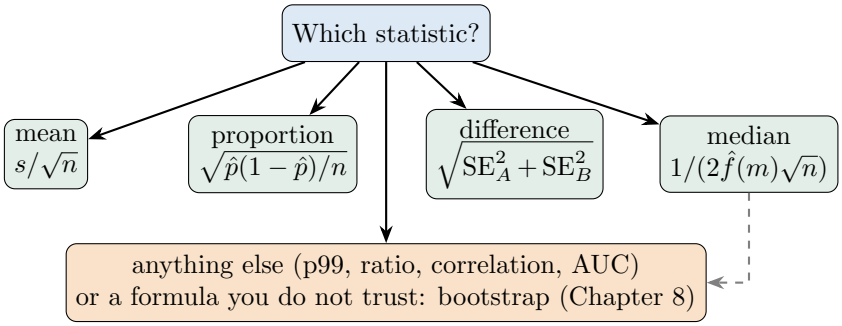


Figure 6.16: Choosing a standard error. Closed forms exist for a handful of statistics; everything else, and any case where the formula's assumptions look shaky, goes to the bootstrap.

6.11 Finite population correction

All formulas so far assume independent draws: sampling with replacement, or from a population so large that it makes no difference. When you sample *without* replacement a large fraction n/N of a finite population of size N, each draw removes a unit and the sample covers more of the population than independent draws would. The SE shrinks by the **finite population correction**:

$$\mathrm{SE}(\bar{X}) = \frac{\sigma}{\sqrt{n}}\sqrt{\frac{N-n}{N-1}} \approx \frac{\sigma}{\sqrt{n}}\sqrt{1-\frac{n}{N}}.$$

Example 6.8 (Surveying a company)**.** A company has $N = 2000$ employees; you survey $n = 500$ chosen at random without replacement. The correction is $\sqrt{1500/1999} \approx 0.866$, so the SE is 13% smaller than the textbook formula. Survey all 2000 and the correction is 0: there is no sampling error left, only measurement error.

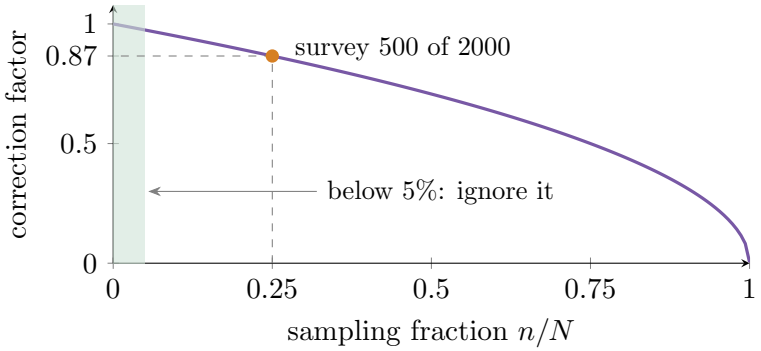


Figure 6.17: The finite population correction $\sqrt{1-n/N}$. It stays above 0.97 while you sample less than 5% of the population (green), which covers almost all practical cases. It only matters when you sample a large share.

Connection

In machine learning, training usually computes the gradient (the direction that most reduces the loss) on a minibatch, a small random subset of the training examples. The minibatch gradient is an average of per-example gradients, so it is a sample mean. Theorem 6.2 is why gradient noise has SD proportional to $1/\sqrt{B}$ for batch size B, and why quadrupling the batch only halves the noise. The same $\sigma/\sqrt{n}$ governs the error bars on a test-set accuracy: a classifier scored on 1000 examples at 90% accuracy has SE $= \sqrt{0.9 \cdot 0.1/1000} \approx 0.0095$, so a 0.5-point "improvement" between two models is well inside the noise.

6.12 Summary

Statistic	Standard error	Notes
mean $\bar{X}$	$\sigma/\sqrt{n}$, estimated by $s/\sqrt{n}$	exact for iid data
proportion $\hat{p}$	$\sqrt{p(1-p)/n}$	at most $0.5/\sqrt{n}$
difference $\bar{X}_A - \bar{X}_B$	$\sqrt{\mathrm{SE}_A^2 + \mathrm{SE}_B^2}$	independent groups only
median	$1/(2f(m)\sqrt{n})$; Normal: $1.2533\,\sigma/\sqrt{n}$	large n
other (p99, ratios, ...)	no simple formula	bootstrap, Chapter 8

- $\mathbb{E}$ is linear; $\mathrm{Var}(aX + b) = a^2\,\mathrm{Var}(X)$; variances of independent variables add (for sums and differences). SDs do not add.
- A statistic is a random variable. Its distribution over repeated samples is the sampling distribution; its SD is the standard error.
- $\mathbb{E}[\bar{X}] = \mu$ and $\mathrm{Var}(\bar{X}) = \sigma^2/n$. Four times the data halves the SE.
- SD describes individuals and does not shrink with n. SE describes an estimate and does.
- CLT: $\bar{X}$ is approximately $\mathcal{N}(\mu, \sigma^2/n)$ for large n. Skewness of $\bar{X}$ is $\gamma/\sqrt{n}$, so skewed data need larger n. For proportions, want $np \geq 10$ and $n(1-p) \geq 10$.
- LLN: $\bar{X} \to \mu$. Early deviations are diluted, not compensated.
- Sampling more than about 5% of a finite population without replacement: multiply the SE by $\sqrt{1 - n/N}$.

Chapter 7 takes the next step: an estimate plus or minus about two standard errors becomes a confidence interval, with a precise statement of how often such intervals catch the truth.

6.13 Exercises

1. $\mathrm{Var}(X) = 4$. Find $\mathrm{Var}(3X + 2)$ and $\mathrm{SD}(3X + 2)$. If Y is independent of X with $\mathrm{Var}(Y) = 5$, find $\mathrm{Var}(X - Y)$.
2. A population has $\sigma = 12$. What is the SE of the mean for $n = 36$? What n gives an SE of 1?
3. A feature is used by 20% of users. What is $\mathrm{SE}(\hat{p})$ from a random sample of 100 users? Without knowing p, how large must n be to guarantee $\mathrm{SE}(\hat{p}) \leq 0.01$?
4. Two independent estimates have $\mathrm{SE}_A = 0.8$ and $\mathrm{SE}_B = 0.6$. What is the SE of their difference? What answer does the common mistake give?
5. From the data $4, 8, 6, 5, 7$ compute $\bar{x}$, s and the estimated SE of the mean.
6. A report says "mean session length 6.2 ± 0.1 min (SE), $n = 2500$." Estimate the SD of session length. Would a session of 9 minutes be unusual?
7. For Normal data with $\sigma = 20$ and $n = 400$, find the SE of the mean and of the median. How many observations would the median need to match the mean's SE?

8. A population has skewness 3. Using $\gamma/\sqrt{n} \le 0.3$ as the criterion, how large must n be for the CLT approximation to be adequate?

9. You survey $n = 100$ of the $N = 500$ members of a club, without replacement. The population SD is 10. Compute the SE of the mean with and without the finite population correction.

10. Explain in two sentences why $\mathrm{Var}(X_1 + X_2) \neq \mathrm{Var}(2X_1)$ for two independent dice, though both have mean 7.

Answers

1. $\mathrm{Var}(3X + 2) = 9 \cdot 4 = 36$, SD $= 6$. $\mathrm{Var}(X - Y) = 4 + 5 = 9$.
2. $12/\sqrt{36} = 2$. Need $12/\sqrt{n} = 1$, so $n = 144$.
3. $\sqrt{0.2 \cdot 0.8/100} = 0.04$. Worst case $0.5/\sqrt{n} \le 0.01$ gives $n \ge 2500$.
4. $\sqrt{0.64 + 0.36} = 1.0$. The mistake (adding SEs) gives 1.4.
5. $\bar{x} = 6$; deviations $-2, 2, 0, -1, 1$, squares sum to 10; $s = \sqrt{10/4} \approx 1.581$; $\widehat{\mathrm{SE}} = 1.581/\sqrt{5} \approx 0.707$.
6. SD $\approx 0.1\sqrt{2500} = 5$ min. A 9-minute session is only $(9 - 6.2)/5 \approx 0.56$ SDs above the mean: perfectly ordinary. (It would be wildly unusual only if you wrongly used the SE.)
7. $\mathrm{SE}(\bar{X}) = 1$, SE(median) ≈ 1.2533. Need $400 \cdot \pi/2 \approx 628$ observations.
8. $3/\sqrt{n} \le 0.3$ gives $\sqrt{n} \ge 10$, so $n \ge 100$.
9. Without: $10/\sqrt{100} = 1$. With: $1 \cdot \sqrt{400/499} \approx 0.895$.
10. Independent dice often offset each other, so the sum's variance is $35/12 + 35/12 = 35/6$. Doubling one die copies its deviation exactly, so the variance is $2^2 \cdot 35/12 = 35/3$, twice as large.

Chapter 7

Confidence Intervals

A dashboard says the mean page-load time this week is 212 ms. Is the true mean 212.0? Almost certainly not. It is 212 plus or minus something, and the size of that something decides whether last week's 205 ms is a real regression or noise. Chapter 6 gave us the raw ingredient: the standard error SE, the typical distance between an estimate and the truth. This chapter turns SE into a range with a stated success rate.

7.1 From standard error to interval

Take $n = 36$ page loads. Their mean is $\bar{x} = 212$ ms. Suppose years of monitoring tell us the per-request standard deviation is $\sigma = 30$ ms. Then

$$\mathrm{SE}(\bar{X}) = \frac{\sigma}{\sqrt{n}} = \frac{30}{\sqrt{36}} = \frac{30}{6} = 5 \text{ ms}.$$

By the Central Limit Theorem (Chapter 6), $\bar{X}$ is approximately Normal, centred on the true mean μ with standard deviation 5 ms. A Normal variable lands within 1.96 standard deviations of its centre 95% of the time. So 95% of the time,

$$|\bar{X} - \mu| \le 1.96 \times 5 = 9.8 \text{ ms}.$$

If $\bar{X}$ is within 9.8 ms of μ, then μ is within 9.8 ms of $\bar{X}$. That gives the interval

$$212 \pm 9.8 = [202.2,\ 221.8] \text{ ms}.$$

Last week's 205 ms is inside it. This week's data are consistent with no change at all.

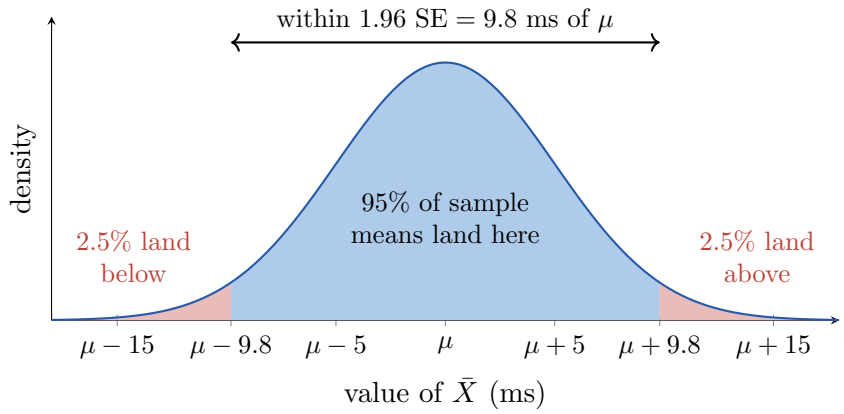


Figure 7.1: Where the sample mean $\bar{X}$ can land when $n = 36$ and $\sigma = 30$ (SE $= 5$ ms). The blue middle holds 95% of the area: 95% of samples give an $\bar{x}$ within 9.8 ms of the true μ. Read backwards, "$\bar{x}$ is within 9.8 ms of μ" is the confidence interval $\bar{x} \pm 9.8$.

Every interval in this chapter has the same skeleton.

Definition 7.1 (Confidence interval, general form)**.** A *confidence interval* for a parameter θ at confidence level $1 - \alpha$ is a range computed from the data,

$$\text{estimate} \ \pm \ (\text{critical value}) \times \text{SE}(\text{estimate}),$$

built so that, before the data are drawn, the random interval contains θ with probability $1 - \alpha$. The number α is the miss rate we accept: $\alpha = 0.05$ gives a 95% interval. The product (critical value) $\times$ SE is the *margin of error*.

Three ingredients change from case to case. The *estimate* ($\bar{x}$, $\hat{p}$, a difference of means). Its *standard error* (always derived from the sampling distribution). The *critical value* (1.96 for a 95% Normal interval, a slightly larger t value when σ is estimated).

7.2 The z-interval from a pivot

The informal argument above can be made exact. Assume $X_1, \dots, X_n \overset{\text{iid}}{\sim} \mathcal{N}(\mu, \sigma^2)$ with σ known (or n large enough for the CLT). Define

$$Z = \frac{\bar{X} - \mu}{\sigma / \sqrt{n}}.$$

Z depends on the unknown μ, yet its distribution, $\mathcal{N}(0, 1)$, does not depend on μ or anything else unknown. A quantity with this property is called a *pivot*. Because its distribution is fully known, we can write an exact probability statement about it:

$$\mathbb{P}\left(-1.96 \le \frac{\bar{X} - \mu}{\sigma / \sqrt{n}} \le 1.96 \right) = 0.95.$$

Now rearrange the inequalities to isolate μ. Multiply through by $\sigma / \sqrt{n}$:

$$-1.96 \frac{\sigma}{\sqrt{n}} \le \bar{X} - \mu \le 1.96 \frac{\sigma}{\sqrt{n}}.$$

Subtract $\bar{X}$ and multiply by -1 (which flips both inequalities):

$$\bar{X} - 1.96\frac{\sigma}{\sqrt{n}} \le \mu \le \bar{X} + 1.96\frac{\sigma}{\sqrt{n}}.$$

The event has not changed, only how it is written. So its probability is still 0.95.

Theorem 7.1 (z-interval for a mean)**.** *If $\bar{X}$ is Normal (exactly, or approximately by the CLT) and σ is known, then*

$$\bar{X} \pm z_{\alpha/2}\,\frac{\sigma}{\sqrt{n}}$$

contains μ with probability $1-\alpha$, where $z_{\alpha/2}$ is the point with upper-tail area $\alpha/2$ under $\mathcal{N}(0,1)$: $z_{0.05} = 1.645$ (90%), $z_{0.025} = 1.960$ (95%), $z_{0.005} = 2.576$ (99%).

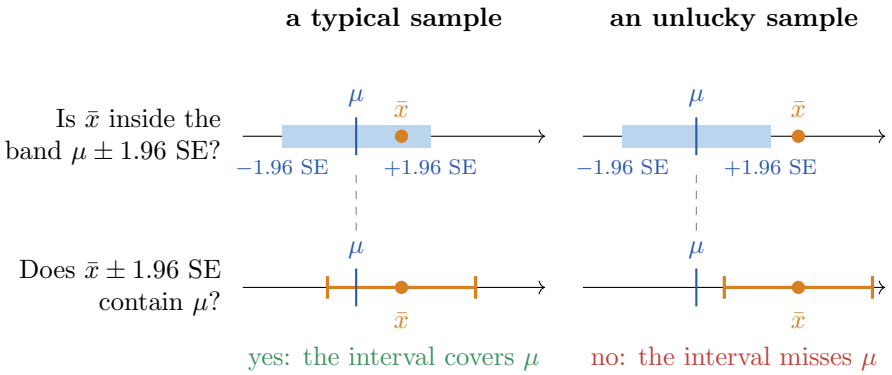


Figure 7.2: The pivot argument in one picture. Top row: where $\bar{x}$ lands relative to the blue band around μ. Bottom row: the orange interval built around that same $\bar{x}$. When $\bar{x}$ is inside the band (left), the interval reaches μ; when $\bar{x}$ falls outside (right), the interval misses. The two questions always have the same answer, so they have the same probability, 0.95.

Why it works

Why 1.96? Because $\mathbb{P}(|Z| \le 1.96) = 0.95$ for a standard Normal. The critical value is not a convention; it is whatever number traps the chosen fraction of the pivot's distribution. A 99% interval needs to trap 99%, so it reaches out to 2.576. Nothing else about the construction changes.

Pause

Pause and think: with the same 36 page loads, what is the 99% interval? (Answer: $212 \pm 2.576 \times 5 = 212 \pm 12.9 = [199.1,\ 224.9]$ ms. Wider, because it must succeed more often.)

7.3 What "95% confidence" means

The probability statement above is about $\bar{X}$, a random variable, before sampling. Once we have computed $[202.2, 221.8]$, nothing is random any more: μ is a fixed number and it either is or is not in that range. The 95% describes the *procedure*, not the particular interval.

The cleanest way to see this is to run the procedure many times. We simulated a population with $\mu = 200$ ms and $\sigma = 30$ ms, drew 25 independent samples of $n = 36$, and built the 95% z-interval $\bar{x} \pm 9.8$ from each.

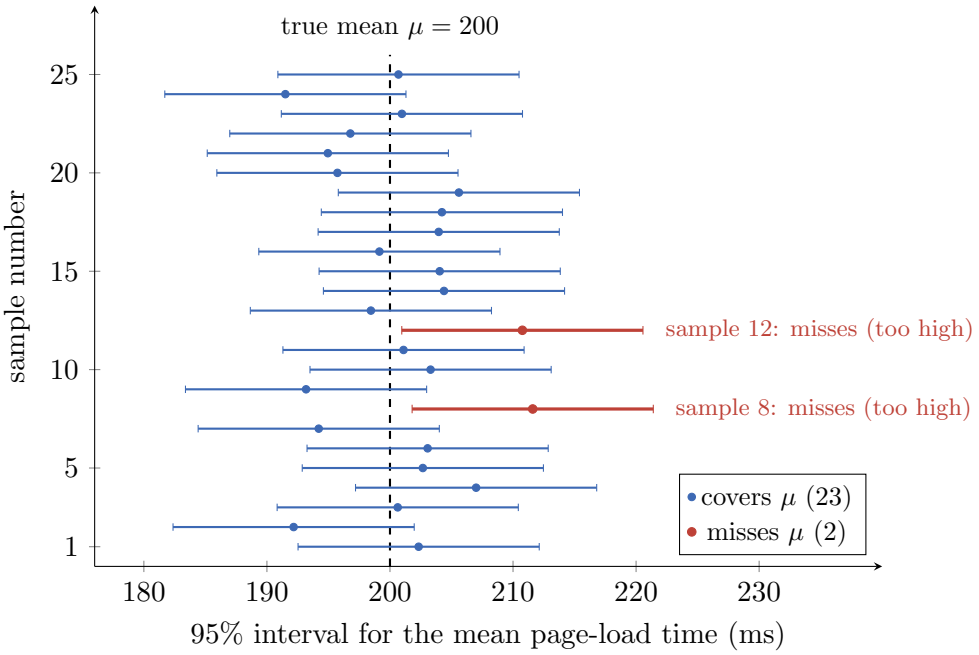


Figure 7.3: Twenty-five 95% intervals from 25 simulated samples ($n = 36$, $\sigma = 30$, true $\mu = 200$). Each interval has the same width (19.6 ms); only its centre moves. Samples 8 and 12 drew unusually slow requests and their intervals (red) miss μ. Over the long run, 5% of intervals miss; here 2 of 25 did (8%), which is within ordinary chance.

Definition 7.2 (Coverage)**.** The *coverage probability* of an interval procedure is the probability, over repeated samples, that the interval contains the true parameter. A procedure is a valid 95% interval if its coverage is (at least approximately) 0.95 for every possible value of the parameter.

Intuition

Think of a ring toss. The peg (μ) is fixed. Each sample throws a ring of fixed width at it. A 95% procedure is a thrower who rings the peg 95% of the time. After a single throw you do not get to see the peg; you only know the thrower's success rate.

Common trap

Statements that are **wrong** for the interval $[202.2, 221.8]$:

- "There is a 95% probability that μ lies between 202.2 and 221.8." In frequentist statistics μ is not random; the probability is 0 or 1. (A Bayesian *credible* interval does make this statement; see Chapter 14.)
- "95% of page loads take between 202.2 and 221.8 ms." The interval is about the *mean*. Individual loads vary with $\sigma = 30$; a range covering 95% of loads would be roughly 212 ± 59.

- "If we repeat the study, 95% of new sample means will fall in $[202.2, 221.8]$." New means scatter around μ, not around this $\bar{x}$. That fraction is lower, about 83% on average.
- "The interval contains all plausible values with certainty." One in twenty intervals misses, and you never know which one.

The right statement: "We used a method that captures the true mean in 95% of samples; applied to our sample, it gives $[202.2, 221.8]$."

Pause

Pause and think: in Figure 7.3, if we had drawn 1000 samples instead of 25, about how many intervals would be red? (About 50. The count of misses is itself Binomial$(1000, 0.05)$, so anywhere from roughly 37 to 64 is typical.)

7.3.1 Confidence level versus width

Higher confidence is not free. With the same data, raising the level raises the critical value, so the interval widens. Using $\bar{x} = 212$ and SE $= 5$:

Level	$z_{\alpha/2}$	Margin of error	Interval (ms)
80%	1.282	6.41	[205.59, 218.41]
90%	1.645	8.22	[203.78, 220.22]
95%	1.960	9.80	[202.20, 221.80]
99%	2.576	12.88	[199.12, 224.88]
99.9%	3.291	16.45	[195.55, 228.45]

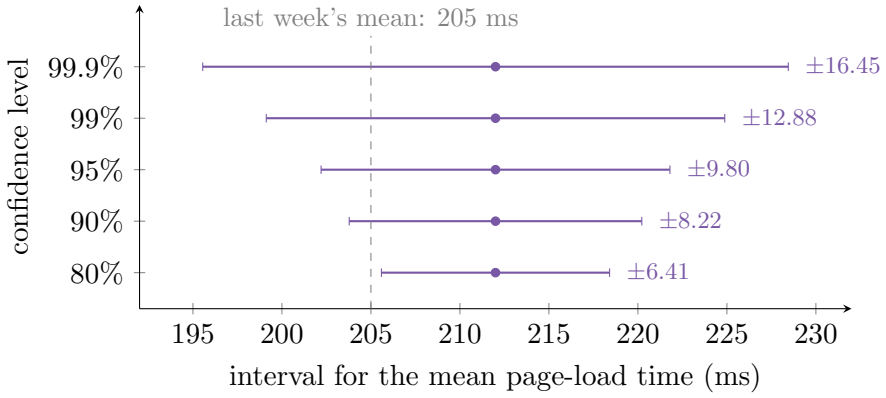


Figure 7.4: The same data ($\bar{x} = 212$, SE $= 5$ ms) at five confidence levels; the number at the right of each bar is its margin of error. Going from 95% to 99.9% widens the interval by 68%. Look at last week's 205 ms: only the 80% interval excludes it, so whether the change "looks real" depends on the level you chose. The only ways to get a narrow interval with high confidence are more data or less noisy data.

A 100% confidence interval exists: $(-\infty, \infty)$. It is useless. Every choice of level is a trade between

being right more often and saying something specific. 95% is a convention, not a law; 90% is common in engineering dashboards and 99% where errors are expensive.

7.4 When σ is unknown: the t-distribution

In practice we almost never know σ. The natural fix is to plug in the sample standard deviation s (Chapter 2):

$$T = \frac{\bar{X} - \mu}{s/\sqrt{n}}.$$

This is no longer standard Normal. The denominator is now random too. With a small sample, s is a noisy estimate of σ; when s happens to come out low, T gets inflated. Those occasional inflations give T heavier tails than Z.

How much does this matter? If we ignore it and use $\bar{x} \pm 1.96\, s/\sqrt{n}$ on Normal data, the true coverage is:

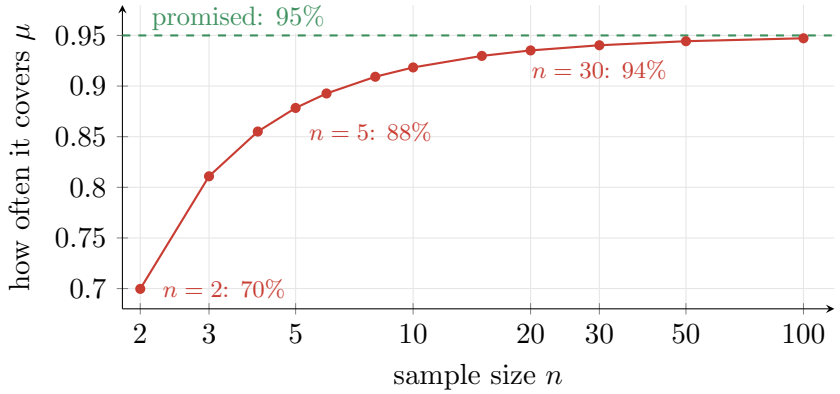


Figure 7.5: How often the naive interval $\bar{x} \pm 1.96\, s/\sqrt{n}$ actually covers μ for Normal data (exact calculation), against the 95% it promises (green line). At $n = 5$ it covers only 88% of the time; at $n = 2$, 70%. The shortfall is caused entirely by the noise in s.

History

In 1908 William Sealy Gosset, a chemist at the Guinness brewery in Dublin, had to judge barley and hop quality from tiny batches: often four or five measurements. The practice of the day was to plug s into the Normal formula as if it were σ, and with $n = 4$ that gave badly overconfident intervals, exactly the shortfall in Figure 7.5. Gosset worked out (partly by hand simulation with numbers drawn from shuffled cards) the distribution of $(\bar{X} - \mu)/(s/\sqrt{n})$. Guinness did not allow staff to publish under their own names, so he signed the paper "Student". The result made honest inference from small samples possible, and his ratio is still called Student's t.

Theorem 7.2 (Student's t). *If* $X_1, \ldots, X_n \overset{iid}{\sim} \mathcal{N}(\mu, \sigma^2)$, *then*

$$T = \frac{\bar{X} - \mu}{s/\sqrt{n}} \sim t_{n-1},$$

the t-distribution with $n-1$ degrees of freedom. It is symmetric around 0, bell-shaped, has heavier tails than $\mathcal{N}(0,1)$, and approaches $\mathcal{N}(0,1)$ as $n \to \infty$.

The *degrees of freedom* (df) is the single number that picks out which member of the t family you have: small df means a squat curve with fat tails, large df means nearly Normal. Here df $= n-1$ for the same reason s^2 divides by $n-1$: the n deviations $x_i - \bar{x}$ sum to zero, so only $n-1$ of them carry independent information about spread (Chapter 2). The proof that T has exactly this distribution uses the independence of $\bar{X}$ and s^2 for Normal data; see Probability From the Ground Up for the derivation.

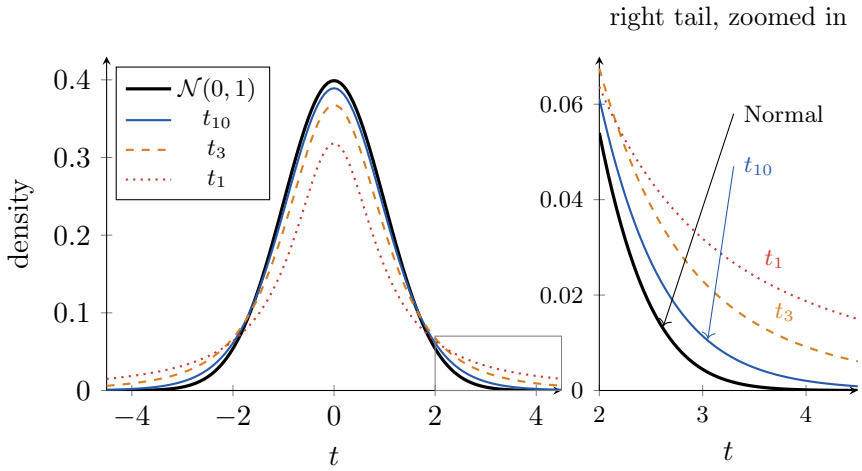


Figure 7.6: The t-distribution for 1, 3 and 10 degrees of freedom against the standard Normal. Left: fewer degrees of freedom means a lower peak. Right: the grey box from the left panel, enlarged, where the difference that matters lives: the Normal tail (black) dies out by $t = 3.5$ while t_3 and t_1 keep real probability out there. At $t = 3$, the two-sided tail area is 0.27% for the Normal but 5.8% for t_3. By t_{30} the curves are nearly indistinguishable.

Fatter tails mean we must reach further out to trap 95%. The *critical value* (the cut-off on the horizontal axis that leaves the chosen tail area outside) becomes $t_{\alpha/2,\,n-1}$ in place of $z_{\alpha/2}$:

df	90% ($t_{0.05}$)	95% ($t_{0.025}$)	99% ($t_{0.005}$)
1	6.314	12.706	63.657
2	2.920	4.303	9.925
3	2.353	3.182	5.841
4	2.132	2.776	4.604
5	2.015	2.571	4.032
6	1.943	2.447	3.707
7	1.895	2.365	3.499
8	1.860	2.306	3.355
9	1.833	2.262	3.250
10	1.812	2.228	3.169
15	1.753	2.131	2.947
20	1.725	2.086	2.845
30	1.697	2.042	2.750
50	1.676	2.009	2.678
100	1.660	1.984	2.626
∞	1.645	1.960	2.576

The last row is the Normal. The table makes the practical point: below about $n = 30$ the difference matters; above it, t and z agree to within a few percent.

Theorem 7.3 (t-interval for a mean)**.** *For a sample of size n with mean $\bar{x}$ and standard deviation s,*

$$\bar{x} \pm t_{\alpha/2,\,n-1}\,\frac{s}{\sqrt{n}}$$

has coverage exactly $1-\alpha$ for Normal data, and approximately $1-\alpha$ for other data when n is moderate and the data are not heavily skewed.

7.4.1 Worked example

Walking question

Eight latency measurements (ms) from a staging server:

$$182,\ 195,\ 210,\ 177,\ 224,\ 201,\ 189,\ 214.$$

Step 1: mean. The sum is 1592, so $\bar{x} = 1592/8 = 199$ ms.
Step 2: standard deviation. Deviations from 199: $-17, -4, 11, -22, 25, 2, -10, 15$. Squares: $289, 16, 121, 484, 625, 4, 100, 225$, summing to 1864. So

$$s^2 = \frac{1864}{8-1} = 266.29, \qquad s = 16.32 \text{ ms}.$$

Step 3: standard error. SE $= 16.32/\sqrt{8} = 16.32/2.828 = 5.77$ ms.
Step 4: critical value. df $= 7$, so $t_{0.025,7} = 2.365$ from the table.
Step 5: interval. Margin $= 2.365 \times 5.77 = 13.64$ ms, giving

$$199 \pm 13.64 = [185.4,\ 212.6] \text{ ms}.$$

Using 1.96 instead would give $[187.7, 210.3]$, an interval 17% too narrow, with real coverage about 91% instead of 95%.

```python
import numpy as np
from scipy import stats
x = np.array([182, 195, 210, 177, 224, 201, 189, 214])
n, m, s = len(x), x.mean(), x.std(ddof=1)     # ddof=1 -> divide by n-1
t = stats.t.ppf(0.975, df=n-1)                 # 2.365
print(m - t*s/np.sqrt(n), m + t*s/np.sqrt(n))  # 185.36 212.64
# one call: stats.t.interval(0.95, n-1, loc=m, scale=s/np.sqrt(n))
```

Common trap

The t-interval assumes the *sample mean* is roughly Normal, not the data. For small n with strong skew (latencies often have a long right tail), the t-interval can still under-cover. Look at the data first (Chapter 3). For skewed data, consider a log transform, an interval for the median, or the bootstrap (Chapter 8).

7.5 Interval for a proportion

A landing page gets $n = 400$ visitors, of whom $x = 46$ sign up. The sample proportion is $\hat{p} = 46/400 = 0.115$. Each visitor is a Bernoulli trial, so (Chapter 6)

$$\mathrm{SE}(\hat{p}) = \sqrt{\frac{p(1-p)}{n}}.$$

We do not know p, so the simplest move is to plug in $\hat{p}$.

Definition 7.3 (Wald interval)**.**

$$\hat{p} \pm z_{\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}.$$

For the landing page: $\mathrm{SE} = \sqrt{0.115 \times 0.885/400} = 0.01595$, margin $= 1.96 \times 0.01595 = 0.0313$, interval $[0.084,\ 0.146]$. Somewhere between 8.4% and 14.6% of visitors sign up.

7.5.1 Why Wald fails near 0 and 1

Now a smaller test: 25 runs of a flaky integration test, 0 failures. Then $\hat{p} = 0$, $\mathrm{SE} = \sqrt{0 \times 1/25} = 0$, and the Wald interval is $[0, 0]$. It claims certainty that the failure rate is exactly zero after 25 runs. With 2 failures out of 25 it gives $0.08 \pm 0.106 = [-0.026,\ 0.186]$, which includes impossible negative rates.

Two things go wrong at once:

1. **The SE is estimated badly.** When $\hat{p}$ is near 0, the plug-in $\hat{p}(1-\hat{p})$ is small, so the interval is too narrow exactly when $\hat{p}$ happens to have come out low.

2. **The Normal shape is wrong.** A Binomial with small np is strongly right-skewed and cannot go below zero. A symmetric interval cannot match it.

We can compute the Wald interval's true coverage exactly, without simulation. For fixed n and p, sum the Binomial probabilities of every count x whose interval contains p. Figure 7.7 does this for $n = 30$ and p from 0.01 to 0.99.

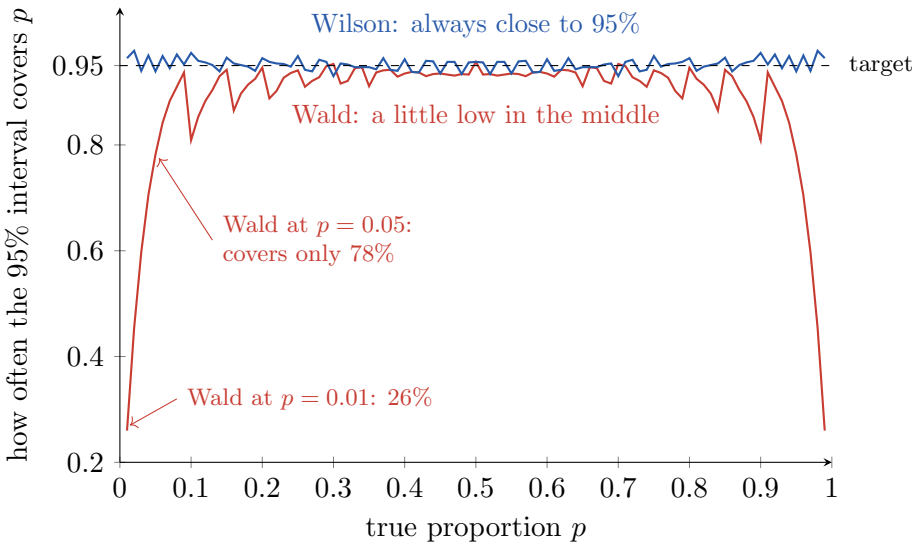


Figure 7.7: Exact coverage of intervals that promise 95% for a proportion, $n = 30$, at every true p from 0.01 to 0.99 (dashed line: the 95% target). Wald (red) collapses near the edges: at $p = 0.05$ it covers only 78% of the time, at $p = 0.01$ only 26%. Wilson (blue) stays between 93% and 98% everywhere. Both curves are jagged because x can only take 31 values.

7.5.2 The Wilson interval

The fix is to stop plugging $\hat{p}$ into the standard error. Ask instead: which values of p are close enough to $\hat{p}$, measured in units of *their own* standard error?

Definition 7.4 (Wilson score interval)**.** The set of p satisfying $|\hat{p} - p| \le z\sqrt{p(1-p)/n}$ is the interval

$$\frac{\hat{p} + \dfrac{z^2}{2n}}{1 + \dfrac{z^2}{n}} \;\pm\; \frac{z}{1 + \dfrac{z^2}{n}} \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}},$$

where $z = z_{\alpha/2}$.

Why it works

Square both sides of $|\hat{p} - p| \le z\sqrt{p(1-p)/n}$:

$$(\hat{p} - p)^2 \le \frac{z^2}{n}\, p(1-p).$$

This is a quadratic inequality in p. Its two roots are the endpoints above. The pivot $(\hat{p} - p)/\sqrt{p(1-p)/n}$ uses the *true* p in the SE, so it is much closer to $\mathcal{N}(0,1)$ than the Wald version. Two consequences: the centre is pulled from $\hat{p}$ toward $1/2$, and the interval can never

leave $[0, 1]$.

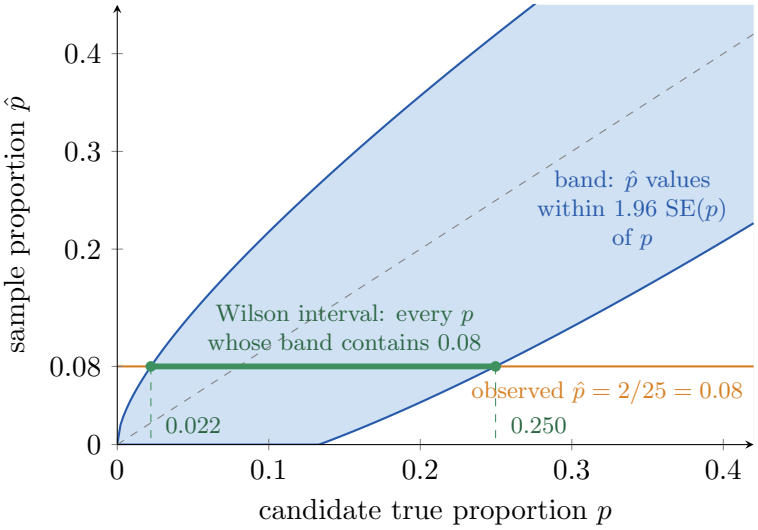


Figure 7.8: How the Wilson interval is found, for 2 failures in 25 runs. For each candidate true rate p (horizontal axis), the blue band shows the sample proportions within 1.96 of *that p's own* standard error $\sqrt{p(1-p)/25}$. Walk along the orange line at the observed $\hat{p} = 0.08$: the band contains it only for p between 0.022 and 0.250, and that range is the 95% Wilson interval. The band is narrow near $p = 0$, so the interval stretches further right than left of 0.08.

For the flaky test, $x = 0$, $n = 25$: with $\hat{p} = 0$ the formula simplifies to $[0,\ (z^2/n)/(1 + z^2/n)] = [0,\ 0.1537/1.1537] = [0,\ 0.133]$. Zero failures in 25 runs is consistent with a failure rate up to 13%.

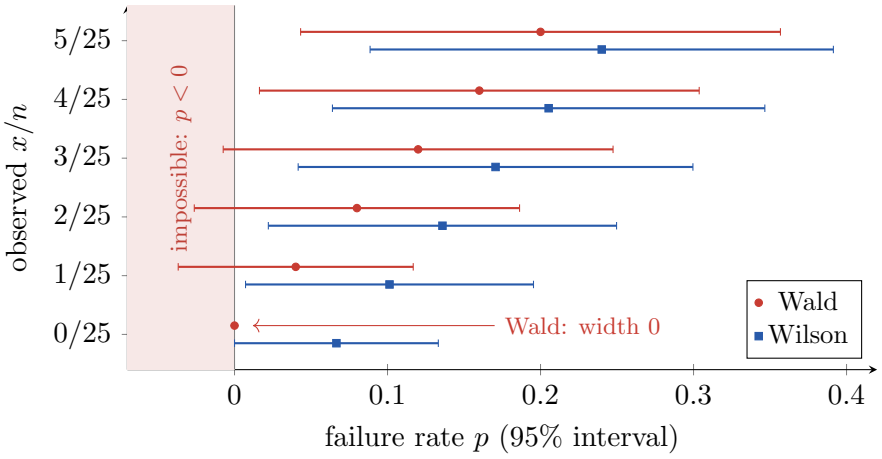


Figure 7.9: Wald (red, circle at $\hat{p}$) and Wilson (blue, square at its centre) 95% intervals for x successes out of $n = 25$. Wald is a single point at $x = 0$ and spills below zero for $x = 1, 2, 3$. Wilson is asymmetric, shifted toward $1/2$, and stays inside $[0, 1]$.

```
from statsmodels.stats.proportion import proportion_confint
proportion_confint(46, 400, alpha=0.05, method="normal")  # Wald  (0.084, 0.146)
```

```
proportion_confint(46, 400, alpha=0.05, method="wilson")   # Wilson(0.087, 0.150)
proportion_confint(0, 25, alpha=0.05, method="wilson")     # (0.0, 0.133)
```

Common trap

The textbook rule "Wald is fine if $n\hat{p} \geq 10$ and $n(1-\hat{p}) \geq 10$" is a minimum, not a guarantee. Even at $p = 0.5$ with $n = 30$, Wald's coverage oscillates between 93% and 96%. Wilson costs nothing extra. Use it by default, especially for error rates, click-through rates and anything measured in fractions of a percent.

7.6 Margin of error and sample-size planning

Turn the interval around. Before collecting data, decide how precise the answer must be, then solve for n. For a mean, the margin of error is $E = z\,\sigma/\sqrt{n}$. Solving for n:

$$n = \left(\frac{z\,\sigma}{E}\right)^2,$$

rounded *up*. You need a guess for σ: from a pilot study, past data, or a conservative upper bound.

Example 7.1. We want the mean page-load time to within ± 5 ms at 95% confidence, and historical data say $\sigma \approx 30$ ms. Then $n = (1.96 \times 30/5)^2 = 11.76^2 = 138.3$, so 139 requests. To halve the margin to ± 2.5 ms we need $(1.96 \times 30/2.5)^2 = 553.2$, so 554 requests: four times as many.

Intuition

Precision costs quadratically. Because the margin shrinks like $1/\sqrt{n}$, each extra decimal place of precision (a factor of 10) costs 100 times the data.

7.6.1 What "±3%" in a poll means

For a proportion, $E = z\sqrt{p(1-p)/n}$. The product $p(1-p)$ is largest at $p = 1/2$, where it equals $1/4$. Planning with $p = 1/2$ is therefore conservative:

$$E_{\max} = z\sqrt{\frac{1}{4n}} = \frac{z}{2\sqrt{n}}, \qquad n = \left(\frac{z}{2E}\right)^2.$$

For $E = 0.03$ at 95%: $n = (1.96/0.06)^2 = 1067.1$, so 1068 respondents. A poll of 1000 people has $E_{\max} = 1.96/(2\sqrt{1000}) = 0.031$. That is where the ubiquitous "margin of error ±3 percentage points" comes from.

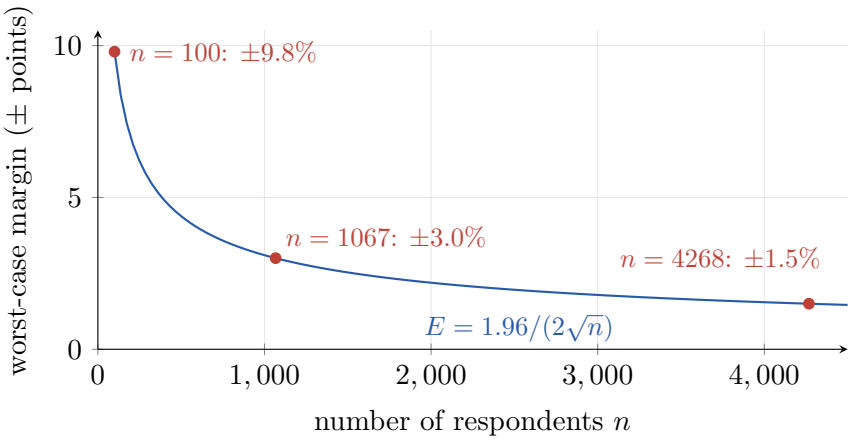


Figure 7.10: Worst-case 95% margin of error for a proportion versus sample size. The curve flattens fast: going from 100 to 1067 respondents cuts the margin from 9.8 to 3 points, but halving it again to 1.5 points needs four times as many, about 4300.

Common trap

The poll margin covers *only* random sampling error. It says nothing about who refused to answer, leading question wording, or people lying (Chapter 5). Those biases do not shrink with n and are often larger than 3 points. Also: in a two-way race the margin on the *gap* between the candidates is roughly twice the margin on each share, because the shares move in opposite directions: every respondent who switches adds to one and subtracts from the other.

The population size does not appear in any of these formulas. A random sample of 1000 gives the same precision for a city of 50,000 as for a country of 300 million, as long as the population is much larger than the sample.

7.7 Comparing two groups

Most practical questions are comparisons: is the new build faster, does the new button convert better? The estimate is a difference, and we need the SE of a difference.

Why it works

For independent estimates A and B, $\mathrm{Var}(A - B) = \mathrm{Var}(A) + \mathrm{Var}(B)$. Variances add even when you subtract, because a minus sign flips direction but not spread. So

$$\mathrm{SE}(A - B) = \sqrt{\mathrm{SE}(A)^2 + \mathrm{SE}(B)^2}.$$

Standard errors combine like the sides of a right triangle, not by plain addition.

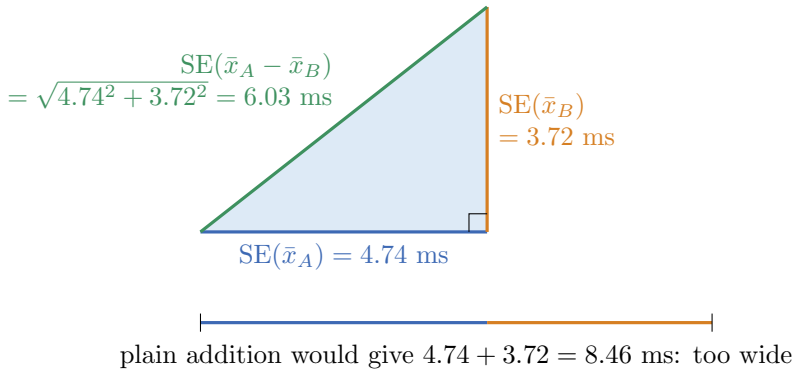


Figure 7.11: Standard errors of independent estimates add like the legs of a right triangle. With the build-comparison numbers below (SEs of 4.74 and 3.72 ms), the SE of the difference is the hypotenuse, 6.03 ms, noticeably shorter than the 8.46 ms you would get by stacking the two SEs end to end.

7.7.1 Difference in means (Welch)

Walking question

Old build: $n_A = 40$ requests, $\bar{x}_A = 212$ ms, $s_A = 30$ ms. New build: $n_B = 35$ requests, $\bar{x}_B = 198$ ms, $s_B = 22$ ms.
Estimate. $\bar{x}_A - \bar{x}_B = 14$ ms.
SE. $s_A^2/n_A = 900/40 = 22.50$ and $s_B^2/n_B = 484/35 = 13.83$, so SE $= \sqrt{22.50 + 13.83} = \sqrt{36.33} = 6.03$ ms.
Degrees of freedom (Welch–Satterthwaite):

$$\nu = \frac{\left(\frac{s_A^2}{n_A} + \frac{s_B^2}{n_B}\right)^2}{\frac{(s_A^2/n_A)^2}{n_A - 1} + \frac{(s_B^2/n_B)^2}{n_B - 1}} = \frac{36.33^2}{\frac{22.50^2}{39} + \frac{13.83^2}{34}} = \frac{1319.9}{12.98 + 5.62} = 70.9.$$

Interval. $t_{0.025,\,70.9} = 1.994$, so $14 \pm 1.994 \times 6.03 = 14 \pm 12.0 = [2.0,\ 26.0]$ ms.
The new build is faster by somewhere between 2 and 26 ms. The interval excludes zero, but only just: the data are compatible with a negligible speedup or a large one.

Definition 7.5 (Welch interval)**.**

$$(\bar{x}_A - \bar{x}_B) \pm t_{\alpha/2,\,\nu}\sqrt{\frac{s_A^2}{n_A} + \frac{s_B^2}{n_B}},$$

with ν from the Welch–Satterthwaite formula above. It does not assume the two groups have equal variances.

The ν formula is an approximation: it picks the t distribution whose degrees of freedom best match how noisy the estimated SE is. You do not need to compute it by hand; software does. It always lies between $\min(n_A, n_B) - 1$ and $n_A + n_B - 2$, and using $\min(n_A, n_B) - 1$ is a safe hand shortcut.

Older texts use a "pooled" interval that assumes equal variances. Welch loses almost nothing when variances are equal and is far safer when they are not, so use Welch by default.

```
from scipy import stats
res = stats.ttest_ind(a, b, equal_var=False)    # Welch
print(res.confidence_interval(0.95))            # CI for mean(a) - mean(b)
```

7.7.2 Difference in proportions

An A/B test shows the old checkout page to 2400 users (120 buy, $\hat{p}_A = 0.050$) and the new page to 2400 users (156 buy, $\hat{p}_B = 0.065$). The difference is $\hat{p}_B - \hat{p}_A = 0.015$, or 1.5 percentage points.

$$\text{SE} = \sqrt{\frac{0.05 \times 0.95}{2400} + \frac{0.065 \times 0.935}{2400}} = \sqrt{0.00001979 + 0.00002532} = 0.00672.$$

The 95% interval is $0.015 \pm 1.96 \times 0.00672 = 0.015 \pm 0.0132 = [0.0018,\ 0.0282]$. The new page lifts conversion by between 0.2 and 2.8 points. In relative terms, that is anywhere from a 4% to a 56% improvement on the 5% baseline: the direction is fairly clear, the size is not.

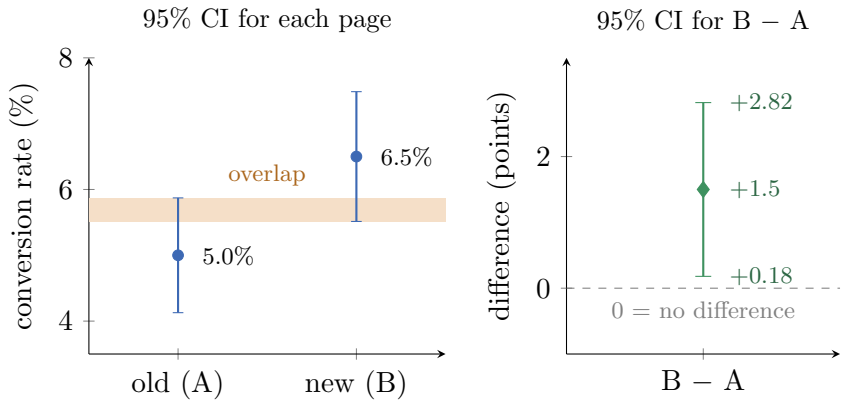


Figure 7.12: The A/B checkout test with 95% intervals. Left: each page's conversion rate; the two intervals overlap (shaded strip). Right: the interval for the difference, $[0.18, 2.82]$ percentage points, which sits entirely above zero. The question "is B better?" is answered by the right panel, not by squinting at the left (Section 7.9 explains why overlap misleads).

7.7.3 Paired data

Eight servers had their p50 latency measured before and after a kernel upgrade (ms):

Server	1	2	3	4	5	6	7	8
Before	241	198	225	260	210	233	247	219
After	228	195	214	249	207	220	238	213
Difference d	13	3	11	11	3	13	9	6

Pause

Pause and think before reading on: should we compare the mean of the "before" row with the mean of the "after" row as two separate groups?

The two columns are not independent samples. Server 4 is slow both times; server 2 is fast both times. The before and after columns have correlation 0.99. The right analysis works on the differences, which turns the problem into a one-sample t-interval:

$$\bar{d} = 8.625, \quad s_d = 4.14, \quad \text{SE} = 4.14/\sqrt{8} = 1.463, \quad t_{0.025,7} = 2.365,$$

$$8.625 \pm 2.365 \times 1.463 = 8.625 \pm 3.46 = [5.17,\ 12.08] \text{ ms}.$$

If we wrongly treat the columns as independent groups (Welch), the SE becomes 9.44 ms and the interval is $[-11.7,\ 28.9]$ ms, which includes zero and is six times wider. The server-to-server variation (SD about 20 ms) swamps the upgrade effect (about 9 ms). Pairing subtracts it out.

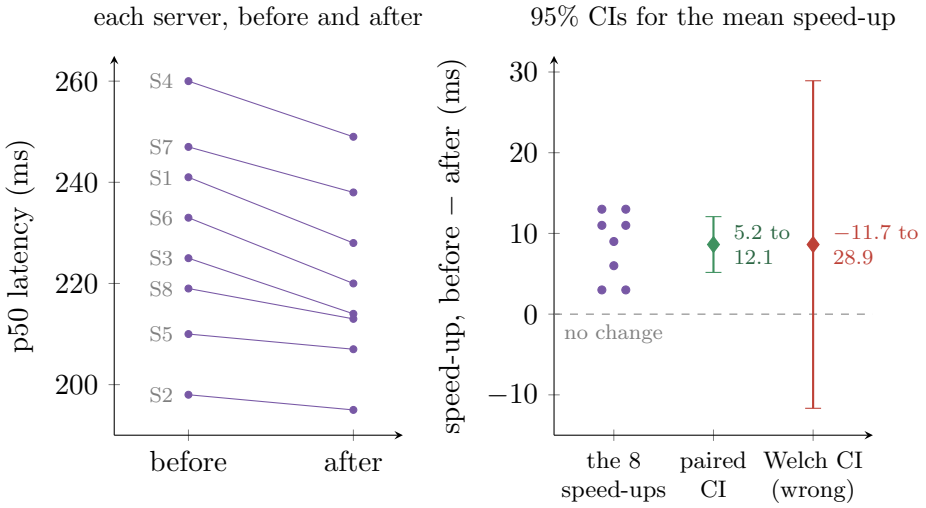


Figure 7.13: The paired latency data. Left: one line per server (S1 to S8). The spread *between* servers (about 60 ms top to bottom) is large, yet every line slopes down. Right: the eight before-minus-after differences, all positive, and two 95% intervals for their mean. The paired interval works on the differences alone and sits well above zero; the Welch interval treats the columns as unrelated groups, so the server-to-server spread inflates it six times and it straddles zero.

Common trap

Whenever each observation in group A has a natural partner in group B (same user, same server, same patient, same day), use the paired interval. Using a two-sample interval on paired data does not give a wrong centre, but it throws away the pairing and can make a clear effect look like noise.

7.8 One-sided bounds

Sometimes only one direction matters. An SLO says mean latency must be below 220 ms; we want an *upper confidence bound*: a number we are 95% confident the mean does not exceed. Put the whole $\alpha = 5\%$ in one tail:

$$\text{upper bound} = \bar{x} + t_{\alpha,\,n-1}\frac{s}{\sqrt{n}}.$$

For the eight staging latencies ($\bar{x} = 199$, SE $= 5.77$, df $= 7$): $t_{0.05,7} = 1.895$, so the bound is $199 + 1.895 \times 5.77 = 209.9$ ms. The SLO of 220 ms is met with 95% confidence.

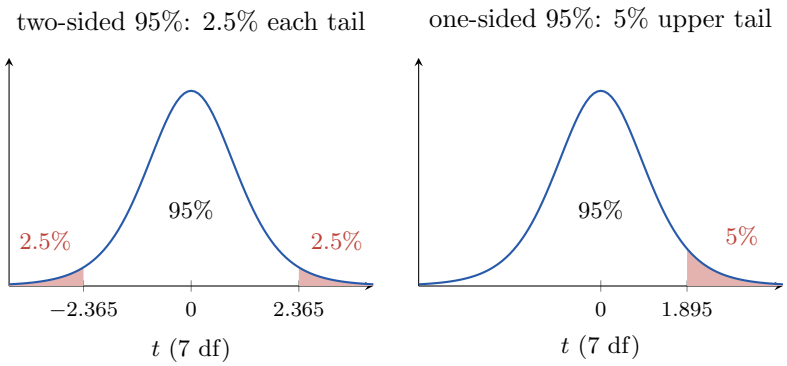


Figure 7.14: The t_7 density (eight staging latencies), with the miss rate shaded red. A two-sided 95% interval splits the 5% miss rate between both tails and reaches out to 2.365. A one-sided bound puts all 5% in one tail, so its critical value is smaller (1.895) and the bound is tighter on the side you care about.

Common trap

Decide on a one-sided bound *before* looking at the data. Picking the side after seeing which way the estimate went is a way of quietly doubling your error rate. The one-sided 95% upper bound equals the upper end of a two-sided 90% interval, which shows exactly what you are giving up.

7.9 Error bars: SD, SE, or CI?

A bar chart with whiskers is meaningless until you know what the whiskers are. Three common choices look alike and mean very different things. Take two groups of $n = 10$ task-completion times (seconds):

Group	mean	SD	SE $= \text{SD}/\sqrt{10}$	95% CI half-width ($t_{0.025,9} = 2.262$)
A	50.0	9.79	3.09	7.00
B	62.0	8.88	2.81	6.35

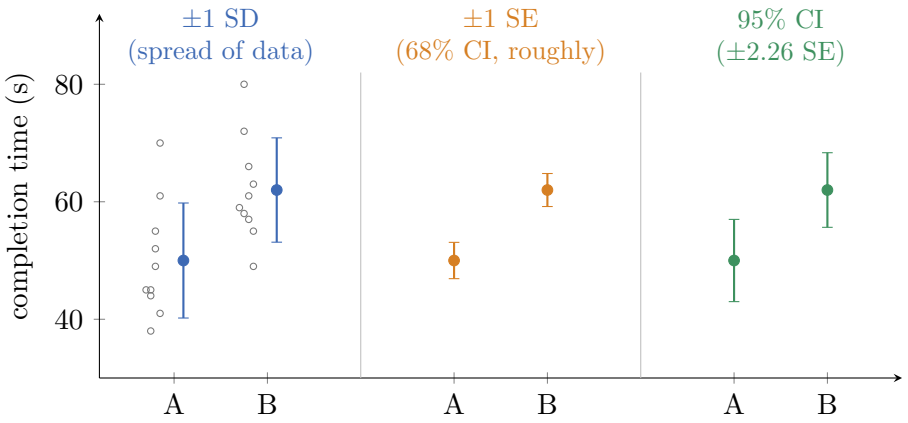


Figure 7.15: The same two groups with three kinds of error bars. SD bars (with the raw points in grey) describe individuals and do not shrink with n. SE bars are the shortest and look the most impressive, but they are only about a 68% interval. 95% CI bars are the ones that support inference about the means.

- **SD bars** answer "how variable are individuals?" They are descriptive. Use them when the reader cares about the spread of outcomes (how long will *my* task take?).
- **SE bars** answer "how precisely is each mean known?" in units of one SE. They are about $1/\sqrt{n}$ as long as SD bars, so they look tight. A ±1 SE bar covers the true mean only about 68% of the time.
- **95% CI bars** answer the same question at a stated level. They are what most readers think they are seeing.

Common trap

Always label error bars in the caption: "mean ± SD", "mean ± SE", or "mean with 95% CI". An unlabelled error bar is uninterpretable, and switching SD to SE is a common way (deliberate or not) to make noisy results look solid.

7.9.1 The overlapping-intervals fallacy

A tempting shortcut: "the two 95% intervals overlap, so the difference is not significant." It is wrong.

Two services have mean latencies 200 ms and 207 ms, each with SE = 2 ms. Their 95% intervals are [196.1, 203.9] and [203.1, 210.9]. They overlap. But the difference is 7 ms with

$$\text{SE(diff)} = \sqrt{2^2 + 2^2} = 2.83, \qquad 7 \pm 1.96 \times 2.83 = [1.5,\ 12.5],$$

which excludes zero.

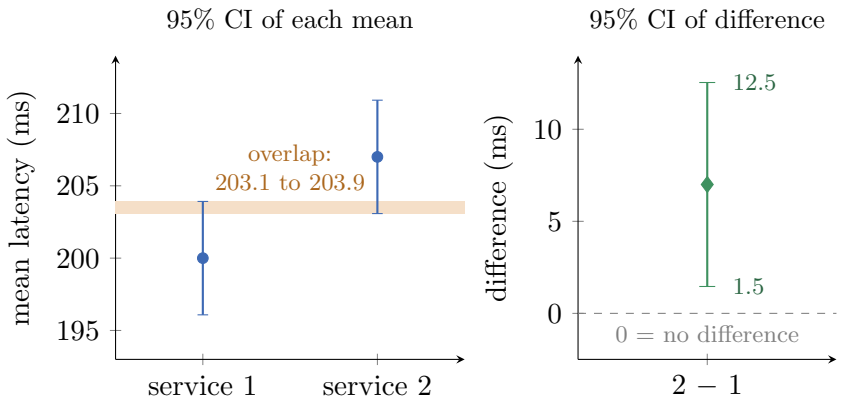


Figure 7.16: Overlapping individual intervals (left, shaded strip) can coexist with a difference interval that excludes zero (right): the services differ by somewhere between 1.5 and 12.5 ms. The SE of a difference is $\sqrt{2} \approx 1.41$ times one SE, not 2 times, so judging by overlap is too conservative.

Why it works

Non-overlap of two 95% intervals requires the gap to exceed $1.96(\mathrm{SE}_1 + \mathrm{SE}_2)$. Significance of the difference requires the gap to exceed $1.96\sqrt{\mathrm{SE}_1^2 + \mathrm{SE}_2^2}$, which is smaller (the hypotenuse is shorter than the two legs combined). With equal SEs the ratio is $\sqrt{2}$. So:

- intervals do not overlap $\Rightarrow$ the difference is significant (with equal SEs, at about the 0.6% level, not 5%);
- intervals overlap $\Rightarrow$ you cannot conclude anything. Compute the interval for the difference.

For paired data, overlap is even less informative, because the individual intervals ignore the pairing entirely.

7.10 Choosing an interval

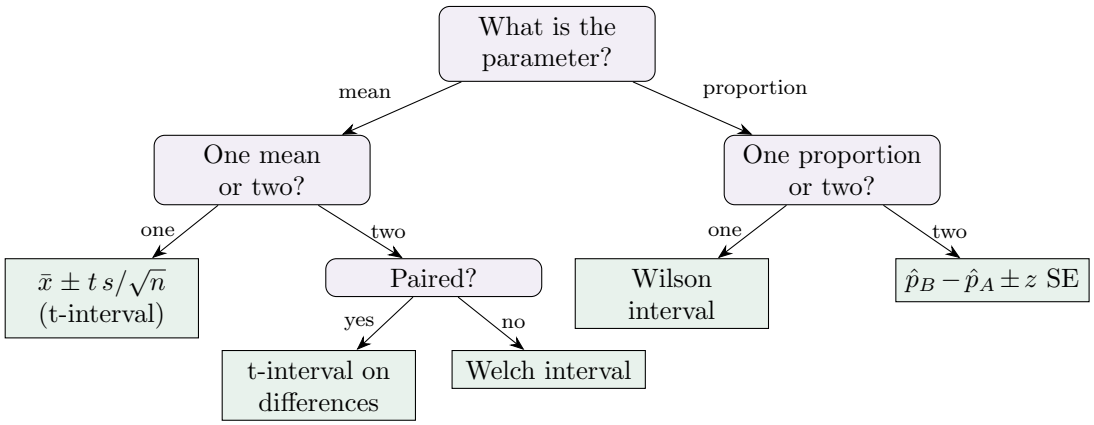


Figure 7.17: Choosing among the intervals of this chapter: start at the top, answer each question about your data, and stop at a green box. For medians, percentiles such as p99, and other statistics without a simple SE formula, use the bootstrap (Chapter 8).

Connection

Machine-learning papers report "accuracy 91.2 ± 0.4". Check what the $\pm$ is: often it is the SD across 5 random seeds, which measures training instability, not the uncertainty in the true accuracy. A test-set accuracy is a proportion, so a Wilson interval on (correct, test-set size) gives the sampling uncertainty from the finite test set: with 2000 test examples and 91.2% accuracy, the 95% margin is about ± 1.2 points, three times the quoted ± 0.4. Chapter 15 returns to this.

7.11 Summary

Parameter	Interval	Notes
Mean, σ known	$\bar{x} \pm z_{\alpha/2}\, \sigma/\sqrt{n}$	rare in practice
Mean	$\bar{x} \pm t_{\alpha/2,n-1}\, s/\sqrt{n}$	default for means
Proportion	Wilson	avoid Wald near 0 or 1
Two means	$\Delta\bar{x} \pm t_{\alpha/2,\nu}\sqrt{s_A^2/n_A + s_B^2/n_B}$	Welch ν
Two proportions	$\Delta\hat{p} \pm z_{\alpha/2}\sqrt{\sum \hat{p}(1-\hat{p})/n}$	A/B tests
Paired	$\bar{d} \pm t_{\alpha/2,n-1}\, s_d/\sqrt{n}$	use differences
One-sided upper	$\bar{x} + t_{\alpha,n-1}\, s/\sqrt{n}$	all of α in one tail

- Every interval is estimate $\pm$ critical value $\times$ SE.
- "95% confidence" is the long-run hit rate of the method. A computed interval either contains μ or does not.

- Use t instead of z whenever s replaces σ; it matters for $n \lesssim 30$.
- Higher confidence means wider intervals; precision improves only like $\sqrt{n}$. Plan with $n = (z\sigma/E)^2$ or $n = (z/2E)^2$ for proportions.
- SEs of independent estimates combine like the sides of a right triangle: $\sqrt{\mathrm{SE}_A^2 + \mathrm{SE}_B^2}$. Pair when you can.
- Label error bars. Overlapping CIs do not imply "no difference".

Every interval here needed a formula for the SE. For a median, a p99, or a ratio of two means there is no tidy formula. Chapter 8 gets the SE, and the interval, by resampling the data itself: the bootstrap.

7.12 Exercises

Exercise 7.1. A sample of $n = 64$ has $\bar{x} = 50$ and the population SD is known to be $\sigma = 8$. Give the 95% z-interval for μ.

Exercise 7.2. An analyst quadruples the sample size. By what factor does the width of the 95% interval change? What about if she instead raises the level from 95% to 99% at the original n?

Exercise 7.3. Six battery lifetimes (hours): $4.1, 3.8, 5.0, 4.6, 4.4, 3.9$. Compute a 95% t-interval for the mean lifetime.

Exercise 7.4. In a survey of 200 users, 30 report a bug. Compute the 95% Wald interval for the proportion of users who hit the bug.

Exercise 7.5. A deployment pipeline ran 50 times with no failures. What does the Wald interval say? What is the 95% Wilson upper bound on the failure rate?

Exercise 7.6. You want to estimate mean session length to within ± 3 minutes with 99% confidence; a pilot suggests $\sigma \approx 15$ minutes. How many sessions do you need?

Exercise 7.7. A poll of $n = 1067$ reports $\pm 3\%$. Roughly how many respondents are needed for $\pm 1.5\%$?

Exercise 7.8. True or false: (a) A 95% CI of $[4.1, 4.8]$ means $\mathbb{P}(4.1 \le \mu \le 4.8) = 0.95$. (b) If we built 95% intervals for 100 independent studies, we would expect about 5 to miss their true parameter. (c) A 99% interval from the same data is always wider than the 95% interval.

Exercise 7.9. Two group means each have a 95% CI of half-width 4, and the means differ by 6. The intervals overlap. Assuming independent groups with equal SEs, is the difference significant at the 5% level?

Answers

1. $\mathrm{SE} = 8/8 = 1$; $50 \pm 1.96 = [48.04,\ 51.96]$.

2. Width scales as $1/\sqrt{n}$, so it halves. Going to 99% multiplies the width by $2.576/1.960 = 1.31$.

3. $\bar{x} = 4.30$, $s = 0.456$, $\mathrm{SE} = 0.186$, $t_{0.025,5} = 2.571$; $4.30 \pm 0.479 = [3.82,\ 4.78]$ hours.

4. $\hat{p} = 0.15$, SE $= \sqrt{0.15 \times 0.85/200} = 0.0252$; $0.15 \pm 0.049 = [0.101,\ 0.199]$.

5. Wald gives $[0, 0]$, which is absurd. Wilson with $x = 0$: upper $= (z^2/n)/(1 + z^2/n) = 0.0768/1.0768 = 0.071$. The failure rate could plausibly be as high as 7%.

6. $n = (2.576 \times 15/3)^2 = 12.88^2 = 165.9$, so 166 sessions.

7. Halving the margin needs four times the sample: about $4 \times 1067 \approx 4270$.

8. (a) False: μ is fixed; 95% is the method's hit rate. (b) True. (c) True: $z_{0.005} > z_{0.025}$ (and likewise for t) with the same SE.

9. Each SE is $4/1.96 = 2.04$; SE(diff) $= 2.04\sqrt{2} = 2.89$; the difference interval is $6 \pm 1.96 \times 2.89 = 6 \pm 5.66 = [0.34,\ 11.66]$, which excludes 0. Yes, significant, despite the overlap.

Chapter 8

The Bootstrap

Chapter 7 built confidence intervals for a mean and a proportion. Each one rested on a formula for the standard error: $s/\sqrt{n}$ for a mean, $\sqrt{\hat{p}(1-\hat{p})/n}$ for a proportion. Those formulas came from Chapter 6, where we worked out by algebra how much the sample mean wobbles from sample to sample.

Now your dashboard shows a median latency of 41 ms, a p90 of 71 ms, and a week-over-week drop in the median of 17 ms. Your manager asks: "plus or minus what?" There is no $s/\sqrt{n}$ for a median. There is no tidy formula for the standard error of a p90, of a ratio of two means, or of a correlation coefficient under a skewed population. The algebra that worked for the mean either gets ugly or needs things you do not know, like the height of the population density at the median.

The bootstrap replaces that algebra with computation. It answers "how much would this number wobble if I could re-run the sampling?" by re-running the sampling on the data itself. This chapter shows how it works, when to trust it, when an exact method beats it, and where it breaks.

8.1 The problem: statistics with no formula

Here are 40 request latencies (ms) from one service, sorted:

12	18	21	23	24	24	24	24	25	27
29	31	31	32	33	36	36	37	39	40
42	43	48	50	50	51	52	53	55	58
59	63	64	68	68	70	79	79	83	145

The mean is 46.15, the median is $(40 + 42)/2 = 41$, and the sample SD is 24.48. Figure 8.1 shows the shape: a hump in the 20s to 50s and a long right tail ending at one 145 ms request.

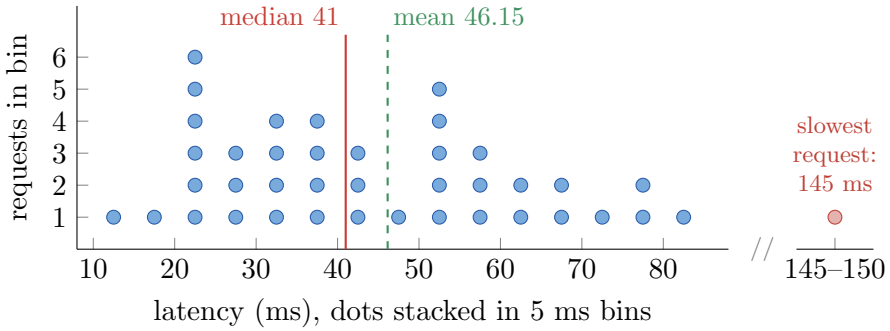


Figure 8.1: The 40 latencies, one dot per request, stacked within 5 ms bins. Most requests sit between 20 and 60 ms; one request at 145 ms makes a long right tail, and that tail pulls the mean (green, 46.15) above the median (red, 41). The axis is broken between 88 and 143 ms, where there are no requests. We want an interval for the population median, and later for the p90, and no textbook formula gives one directly.

For the mean we already know the answer: $\text{SE} = 24.48/\sqrt{40} = 3.87$, and Chapter 7 gives a t-interval. For the median there is a large-sample formula,

$$\text{SE}(\text{median}) \approx \frac{1}{2f(m)\sqrt{n}},$$

where $f(m)$ is the height of the population density at the true median m. A *density* is the smooth curve whose area over an interval gives the probability of landing there (Chapter 1). We do not know f, and estimating its height from 40 points is shaky. For a p90, a ratio of two means, or a correlation, the formulas get worse. We need a method that works for *any* statistic you can compute.

Pause

Pause and think: what single object would let you compute the standard error of *any* statistic by brute force? (Answer: the population itself. If you had it, you could draw thousands of fresh samples and watch the statistic vary. The bootstrap's trick is a stand-in for that population.)

8.2 The plug-in principle

Every standard error is a property of the *population* F (the full distribution of latencies you would see if you logged forever) and the sample size n. If we knew F, we would simulate: draw n values from F, compute the median, repeat 10,000 times, and take the SD of the 10,000 medians. That SD *is* the standard error. This is the sampling distribution of Chapter 6, run by computer.

We do not know F. The plug-in principle says: use your best estimate of F in its place, and run the same simulation.

Definition 8.1 (Empirical distribution)**.** The **empirical distribution** $\hat{F}_n$ of data $x_1, \ldots, x_n$ puts probability $1/n$ on each data point. Picture: a bag containing your n values written on slips of paper. Its cumulative distribution function, the **ECDF**, is

$$\hat{F}_n(t) = \frac{\#\{i : x_i \le t\}}{n},$$

the fraction of data at or below t. It is a staircase that rises by $1/n$ at each data point.

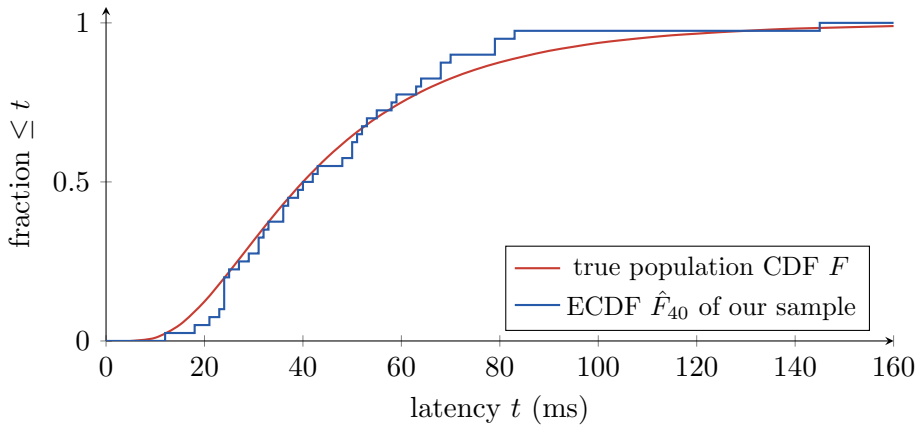


Figure 8.2: The staircase ECDF of the 40 latencies hugs the true CDF (these data were simulated from a known log-normal population so we can see both). The bootstrap uses the staircase as if it were the population.

Figure 8.2 shows why this is reasonable. The law of large numbers (Probability From the Ground Up, Ch. 20) says each staircase height $\hat{F}_n(t)$ converges to $F(t)$, and a stronger result (the Glivenko–Cantelli theorem) says the whole staircase converges uniformly. With 40 points the staircase is already close.

Intuition

The plug-in principle is something you already do. You estimate the population mean by the sample mean, which is the mean of $\hat{F}_n$. You estimate the population median by the sample median, the median of $\hat{F}_n$. The bootstrap goes one step further: it estimates the *sampling variability* of a statistic under F by its sampling variability under $\hat{F}_n$.

Figure 8.3 lays out the parallel.

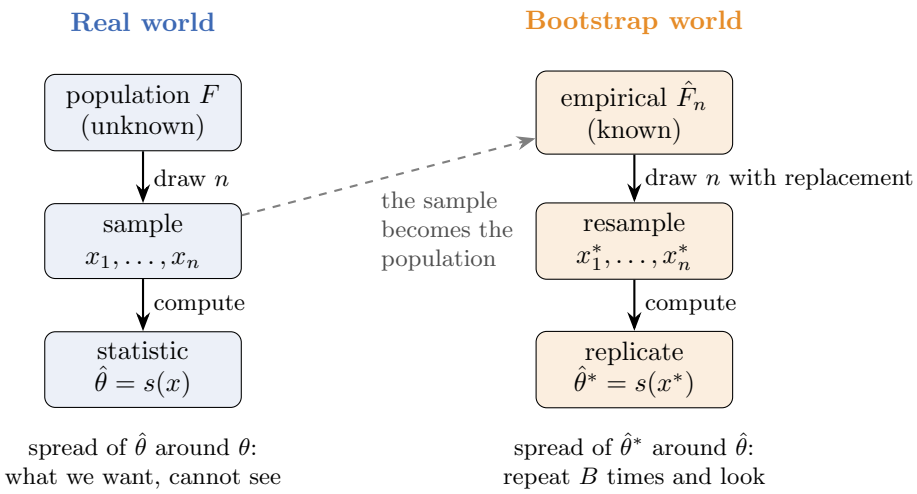


Figure 8.3: The bootstrap copies the real-world sampling process inside a world we fully control. The sample plays the role of the population; resamples play the role of samples.

8.3 Resampling with replacement, by hand

Drawing from $\hat{F}_n$ means: pick one of the n data points uniformly at random, write it down, put it back, and repeat n times. The result is a **bootstrap resample**: a list of n values, each one copied from the data, where some originals appear twice or more and some not at all.

Take a tiny dataset of $n = 8$ latencies:

$$12,\ 15,\ 18,\ 21,\ 24,\ 30,\ 45,\ 110.$$

The sample median is $(21 + 24)/2 = 22.5$. Figure 8.4 shows five random resamples, each sorted so the median (average of positions 4 and 5) is easy to read.

data	12	15	18	21	24	30	45	110	median = 22.5
resample 1	12	15	21	24	45	45	110	110	median = 34.5
resample 2	15	18	18	18	21	21	45	45	median = 19.5
resample 3	12	12	24	24	30	45	45	45	median = 27
resample 4	12	12	18	18	21	21	45	45	median = 19.5
resample 5	15	15	18	21	21	24	110	110	median = 21

Figure 8.4: Five bootstrap resamples of the 8-point dataset (sorted; the red frame marks the two middle values, whose average is the median; the outlier 110 is shaded pink). The outlier 110 appears twice in resamples 1 and 5 and not at all in 2, 3, 4. The resample medians, $34.5, 19.5, 27, 19.5, 21$, spread around the original 22.5, and that spread is what the bootstrap measures.

Three facts are visible already.

- Each resample has exactly $n = 8$ values. Same size as the data, because we are imitating a sample of size n.

- Duplicates are the whole point. Sampling *without* replacement would give back the same 8 numbers in a new order and the same median every time.

- The chance that a particular data point is missing from a resample is $(1 - 1/n)^n$. For $n = 8$ that is $(7/8)^8 = 0.344$; as n grows it tends to $1/e \approx 0.368$. So a typical resample uses about 63% of the distinct original points.

The five medians have SD 6.49. With five resamples that is a crude estimate, but it is already a standard error for the median of 8 latencies, computed with no formula. Run $B = 2000$ resamples instead of 5 and the estimate stabilises.

Common trap

Resample the *units* that were sampled independently. For a correlation between x and y, each resample draws whole (x_i, y_i) pairs. Resampling the x's and the y's separately destroys the pairing: every replicate is a correlation between unrelated numbers, so the replicates cluster around zero whatever the real correlation is.

8.4 The bootstrap distribution

Back to the 40 latencies. Draw $B = 2000$ resamples of size 40, compute the median of each, and plot the 2000 medians.

```
import numpy as np
lat = np.array([48,21,63,70,12,18,43,33,40,24,68,64,42,79,53,24,50,23,68,39,
                36,27,83,36,31,32,55,50,51,52,145,31,29,25,58,79,37,24,24,59])
rng = np.random.default_rng(0)
boot = rng.choice(lat, size=(2000, 40), replace=True)  # 2000 resamples
med_star = np.median(boot, axis=1)                      # 2000 medians
print(med_star.std(ddof=1))                   # bootstrap SE   -> 5.28
print(np.quantile(med_star, [0.025, 0.975]))  # percentile CI  -> [33. 52.]
```

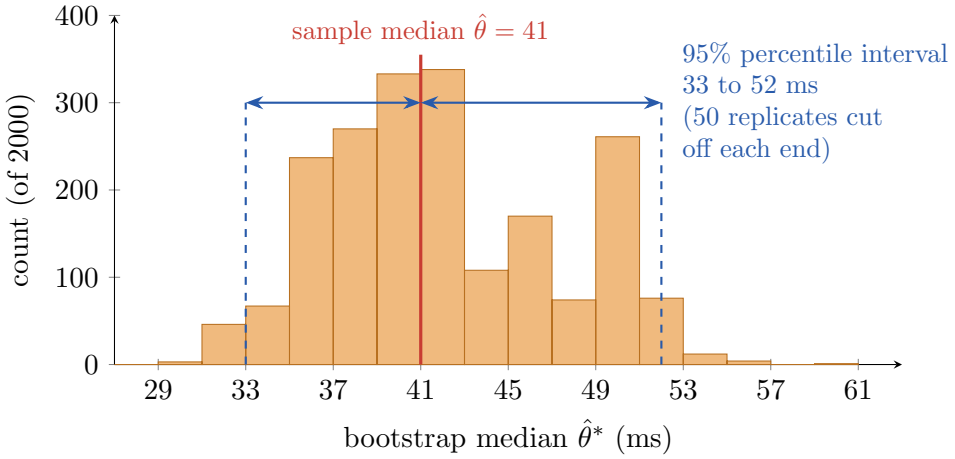


Figure 8.5: Bootstrap distribution of the median: one bar per 2 ms, counting how many of the 2000 resample medians fell there. The dashed blue lines cut off the lowest and highest 2.5% of replicates; what is left, 33 to 52 ms, is the percentile interval. The comb-like lumps are real: a median of 40 values is an average of two data values, so it takes only a few distinct values.

Definition 8.2 (Bootstrap distribution)**.** The **bootstrap distribution** of a statistic is the distribution of $\hat{\theta}^* = s(x^*)$ when x^* is a resample from the data. In practice we see it through B replicates $\hat{\theta}^*_1, \ldots, \hat{\theta}^*_B$. It is our estimate of the sampling distribution of $\hat{\theta}$.

The bootstrap distribution estimates the *shape and spread* of the sampling distribution. It does not estimate where the truth is. Its centre sits near $\hat{\theta}$, not near θ: the mean of the 2000 medians is 41.84, close to the sample median 41. Nothing in resampling can move you toward a population value you never observed. Figure 8.6 puts the bootstrap histogram next to the true sampling distribution, which we can compute here only because the data were simulated.

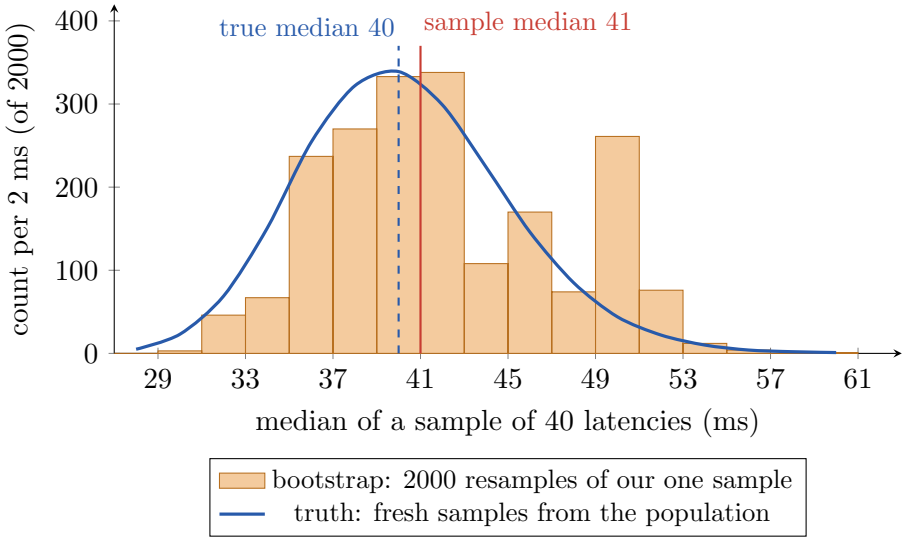


Figure 8.6: The bootstrap histogram (orange) is centred on our sample's median, 41, not on the true median, 40. What it gets roughly right is the width: SD 5.3 ms against the true 4.7 ms. That width, not the centre, is what the bootstrap is used for.

What is really happening

The bootstrap distribution measures how much the statistic changes when the sample is perturbed in the way random sampling perturbs it. Picture shaking the data: some points double up, some vanish. A statistic that barely moves under this shaking (the median of 10,000 values) has a small SE. A statistic that jumps around (the max of 40 values) has a large or meaningless one.

8.5 Bootstrap standard error

Definition 8.3 (Bootstrap standard error)**.**

$$\widehat{\mathrm{SE}}_{\mathrm{boot}} = \sqrt{\frac{1}{B-1}\sum_{b=1}^{B}\left(\hat{\theta}_b^* - \bar{\theta}^*\right)^2}, \qquad \bar{\theta}^* = \frac{1}{B}\sum_{b=1}^{B}\hat{\theta}_b^*.$$

It is the ordinary sample SD (Chapter 2) of the B replicates.

For the latency median, $\widehat{\mathrm{SE}}_{\mathrm{boot}} = 5.28$ ms. The mean's formula SE is 3.87 ms. Is the median the noisier statistic here? Because these data were simulated, we know the population and can find the true SEs by brute force (draw 200,000 fresh samples of 40): 4.7 ms for the median and 5.0 ms for the mean. Both one-sample estimates are off. The bootstrap overstates the median's SE because the values near the middle of this sample happen to be spread out $(39, 40, 42, 43, 48)$, so reshuffles move the midpoint by several ms. The formula understates the mean's SE because 40 draws rarely include the population's rare very slow requests, so s comes out too small. An estimated SE is itself a noisy number computed from one sample.

Why it works

Why should the SD of replicates estimate the SE? The SE is a fixed function of the population and n: write it $\mathrm{SE}(F, n)$. The bootstrap SE is the same function evaluated at the empirical distribution, $\mathrm{SE}(\hat{F}_n, n)$, which is the plug-in principle again. As $\hat{F}_n \to F$, the two agree for any statistic that depends smoothly on F. The B replicates are only a Monte Carlo approximation to $\mathrm{SE}(\hat{F}_n, n)$, and Section 8.12 shows how big B must be.

A quick sanity check uses the mean. One draw from $\hat{F}_n$ is a random data point, so its variance is $\hat{\sigma}^2 = \frac{1}{n}\sum_i (x_i - \bar{x})^2$, the SD computed with divisor n instead of $n-1$. A resample mean averages n independent such draws, so by Chapter 6 its SD is $\hat{\sigma}/\sqrt{n}$. With unlimited resamples the bootstrap SE of the mean is exactly this: $24.17/\sqrt{40} = 3.82$, against the formula's 3.87. (Our 2000 resamples give 3.86; the gap from 3.82 is Monte Carlo noise, Section 8.12.)

A normal-theory interval $\hat{\theta} \pm 1.96\,\widehat{\mathrm{SE}}_{\mathrm{boot}}$ is the simplest use: $41 \pm 1.96(5.28) = [30.7, 51.3]$. It assumes the sampling distribution is bell-shaped and centred at θ. The bootstrap can do better by using the shape it sees.

8.6 The percentile interval

Definition 8.4 (Percentile interval)**.** The 95% **bootstrap percentile interval** is

$$\left[\hat{\theta}^*_{(0.025)},\ \hat{\theta}^*_{(0.975)}\right],$$

the 2.5th and 97.5th percentiles of the B replicates. Picture: sort the 2000 replicates and chop off 50 from each end.

For the median latency this is $[33, 52]$ ms. The interval is not symmetric around 41: it reaches 8 ms below and 11 ms above, because the bootstrap distribution has more room on the right, which mirrors the right-skewed data.

Why it works

Why should chopping the tails of the bootstrap histogram give a confidence interval? Suppose some increasing transformation g (a log, a square root) makes things symmetric: $g(\hat{\theta})$ is normal around $g(\theta)$ with some SD τ, and in the bootstrap world $g(\hat{\theta}^*)$ is normal around $g(\hat{\theta})$ with the same τ. Follow three steps.

1. On the g scale the ordinary interval is $g(\hat{\theta}) \pm 1.96\tau$, and it covers $g(\theta)$ 95% of the time.
2. The bootstrap replicates $g(\hat{\theta}^*)$ are normal with centre $g(\hat{\theta})$ and SD τ, so their 2.5th and 97.5th percentiles are exactly $g(\hat{\theta}) - 1.96\tau$ and $g(\hat{\theta}) + 1.96\tau$.
3. Percentiles pass through increasing functions unchanged: the 2.5th percentile of $g(\hat{\theta}^*)$ is g of the 2.5th percentile of $\hat{\theta}^*$. Mapping the interval back with g^{-1} therefore gives $[\hat{\theta}^*_{(0.025)}, \hat{\theta}^*_{(0.975)}]$.

So the percentile interval is the right interval whenever such a g exists, and you never need to know g. That is why it handles skewness reasonably well and why it respects natural ranges: the interval for a proportion never leaves $[0, 1]$.

8.7 The basic interval

There is a second way to read the same histogram. Suppose the error $\hat{\theta} - \theta$ in the real world behaves like the error $\hat{\theta}^* - \hat{\theta}$ in the bootstrap world. The bootstrap errors run from $33 - 41 = -8$ to $52 - 41 = +11$ (95% of the time). So

$$-8 \le \hat{\theta} - \theta \le 11 \quad \Longleftrightarrow \quad \hat{\theta} - 11 \le \theta \le \hat{\theta} + 8.$$

Definition 8.5 (Basic interval)**.**

$$\left[2\hat{\theta} - \hat{\theta}^*_{(0.975)},\ 2\hat{\theta} - \hat{\theta}^*_{(0.025)}\right].$$

For the median latency: $[82 - 52,\ 82 - 33] = [30, 49]$. The basic interval is the percentile interval reflected through $\hat{\theta}$ (Figure 8.7).

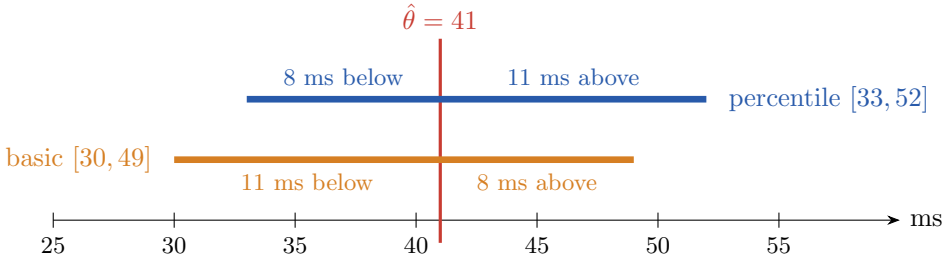


Figure 8.7: The basic interval is the percentile interval flipped around $\hat\theta$: the long arm on the right becomes a long arm on the left. Which one is right depends on whether the skew in the bootstrap distribution is a property of the statistic's sampling distribution (percentile) or an error to be corrected (basic).

The two readings disagree whenever the bootstrap distribution is skewed. The basic interval is right if the bootstrap world reproduces the real-world *error* faithfully, shape and all. The percentile interval is right if a hidden symmetrising transformation exists. For the median of a skewed distribution, the simulation in Section 8.13 shows the percentile interval covering much better. In practice the basic interval is seldom the best choice; it is worth knowing because it is the logic behind the next two methods.

8.8 Better intervals: bootstrap-t and BCa

Percentile and basic intervals are **first-order accurate**: the chance that the truth falls beyond one particular endpoint misses its target 2.5% by an error that shrinks like $1/\sqrt{n}$. Two refinements shrink that error to order $1/n$ for smooth statistics like means, ratios and correlations.

8.8.1 Bootstrap-t

The t-interval of Chapter 7 works because $(\bar X - \mu)/\,\mathrm{SE}$ has a distribution (Student's t) that does not depend on the unknowns. A quantity like that, whose distribution is free of unknown parameters, is called a **pivot**. The bootstrap-t builds a pivot for any statistic and lets the bootstrap find its distribution instead of looking it up in a t table:

$$t_b^* = \frac{\hat\theta_b^* - \hat\theta}{\widehat{\mathrm{SE}}_b^*}, \qquad \text{interval} = \left[\hat\theta - t^*_{(0.975)}\,\widehat{\mathrm{SE}},\ \hat\theta - t^*_{(0.025)}\,\widehat{\mathrm{SE}}\right].$$

Here $\widehat{\mathrm{SE}}_b^*$ is the standard error computed *inside* resample b. For a mean that is just $s_b^*/\sqrt{n}$. For a statistic with no formula, it requires a second, nested bootstrap inside each resample: $B = 1000$ outer times 200 inner means 200,000 medians. Studentizing (dividing by an estimated SE) removes the dependence of spread on θ, which is where the extra accuracy comes from.

Prefer bootstrap-t when the statistic has a stable SE estimate: means, trimmed means, differences of means, regression slopes. Avoid it for the median and other quantiles, whose resample SEs are themselves erratic; the ratio then explodes when $\widehat{\mathrm{SE}}_b^*$ is tiny.

8.8.2 BCa: bias-corrected and accelerated

BCa keeps the percentile idea but picks different percentiles. Two adjustments are estimated from the data.

- **Bias correction** z_0: if more than half the replicates fall below $\hat{\theta}$, the bootstrap distribution is shifted relative to $\hat{\theta}$. Then $z_0 = \Phi^{-1}(\text{fraction of } \hat{\theta}_b^* < \hat{\theta})$, where Φ^{-1} turns a fraction into a standard normal quantile.
- **Acceleration** a: measures how fast the SE changes as θ changes, estimated from the jackknife (recompute the statistic leaving out one point at a time; a is proportional to the skewness of those n values).

The interval uses replicate percentiles α_1, α_2 instead of $0.025, 0.975$:

$$\alpha_{1,2} = \Phi\left(z_0 + \frac{z_0 + z_\alpha}{1 - a\,(z_0 + z_\alpha)}\right), \qquad z_\alpha = \mp 1.96.$$

With $z_0 = a = 0$ this collapses to the percentile interval.

```
from scipy import stats
res = stats.bootstrap((lat,), np.median, n_resamples=2000,
                      method='BCa', random_state=0)
print(res.confidence_interval)      # -> (32.7, 51.5)
p90 = lambda a, axis: np.quantile(a, 0.9, axis=axis)
res = stats.bootstrap((lat,), p90, n_resamples=2000,
                      method='BCa', random_state=0)
print(res.confidence_interval)      # -> (63.5, 89.2)
```

Common trap

BCa's acceleration comes from the jackknife, and the jackknife is unreliable for the median: leaving out one of 40 points moves the median to one of only two values. For the median, BCa is close to the percentile interval and gains little. Its real advantage is for smooth statistics (means of skewed data, ratios, correlations, variances).

8.9 Worked examples: median and p90 latency

Walking question

Question. From the 40 latencies, give a 95% interval for the population median and for the population p90.
Median. $\hat{\theta} = 41$. With $B = 2000$ (seed 0): $\widehat{\text{SE}}_{\text{boot}} = 5.28$; percentile $[33, 52]$; basic $[30, 49]$; BCa $[32.7, 51.5]$. Report: "median 41 ms, 95% CI 33 to 52 ms."
p90. The sample p90 (`numpy`'s default linear interpolation, Chapter 3) is 70.9. From the same 2000 resamples: $\widehat{\text{SE}}_{\text{boot}} = 8.35$; percentile $[59.4, 83.0]$; basic $[141.8 - 83.0,\ 141.8 - 59.4] = [58.8, 82.4]$; BCa $[63.5, 89.2]$.
The p90 intervals disagree more than the median intervals, and the BCa interval is shifted right. That disagreement is a warning sign: the tail of 40 points is thin, and Figure 8.8 shows

why.

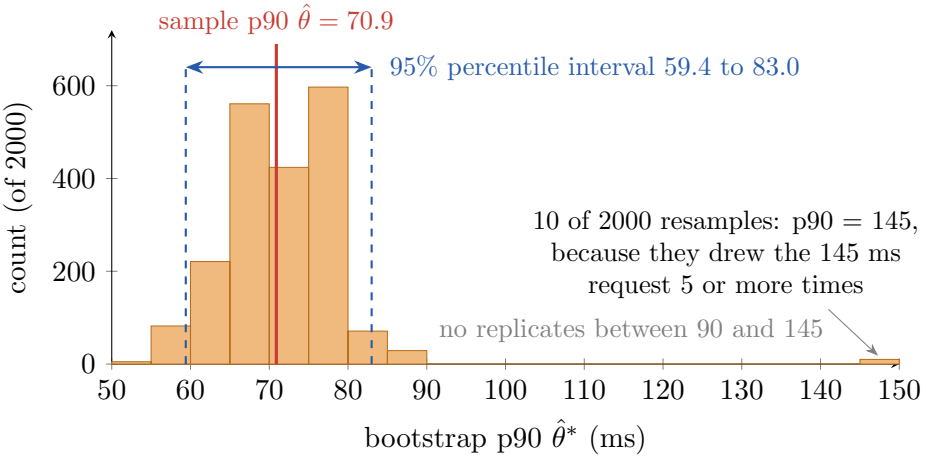


Figure 8.8: Bootstrap distribution of the p90 (bars 5 ms wide). Almost every replicate is built from the handful of data values above 63 ms, so the histogram is lumpy, has a gap from 90 to 145 ms, and stops dead at the largest observation. The bootstrap cannot imagine a latency above 145 ms.

The p90 replicates are built from the top 5 or so data points. A resample can only rearrange those points; it cannot produce the 100 ms request that the population will occasionally send. The further into the tail you go, the less the bootstrap knows. For p99 from 40 points it knows essentially nothing.

8.10 Difference in medians between two groups

After a deploy, 35 latencies from the new version have median 24 ms, against 41 ms for the 40 old ones (Figure 8.9).

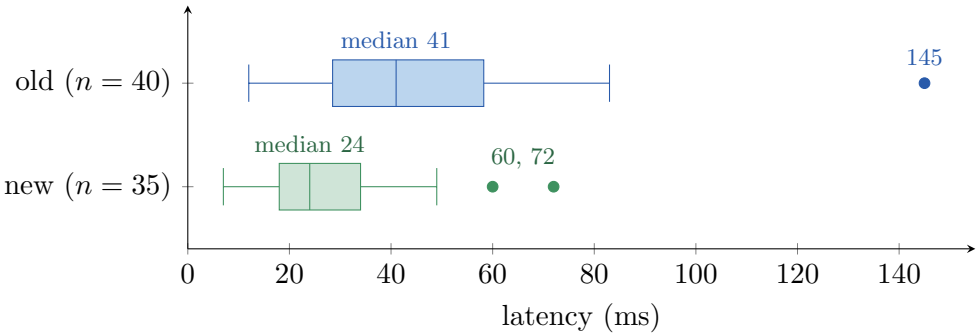


Figure 8.9: Old and new latencies. Each box spans the middle half of a group, the line inside is its median, whiskers reach the furthest values within 1.5 box lengths, and dots are the slower requests beyond. The new median sits 17 ms lower. Is that gap bigger than resampling noise?

The two samples were drawn independently, so the bootstrap imitates that:

1. Resample 40 values with replacement from the old group; take the median.
2. Resample 35 values with replacement from the new group; take the median.
3. Record the difference (old minus new). Repeat B times.

```
new = np.array([32,38,27,19,24,18,33,72,24,22,43,40,34,18,31,49,14,24,
                10,15,11,28,15,38,35,29,7,23,31,34,13,24,18,20,60])
rng = np.random.default_rng(1)
m_old = np.median(rng.choice(lat, (2000, 40)), axis=1)
m_new = np.median(rng.choice(new, (2000, 35)), axis=1)
d = m_old - m_new                       # observed difference: 41 - 24 = 17
print(d.std(ddof=1), np.quantile(d, [0.025, 0.975]))   # 5.96 [ 5. 27.5]
```

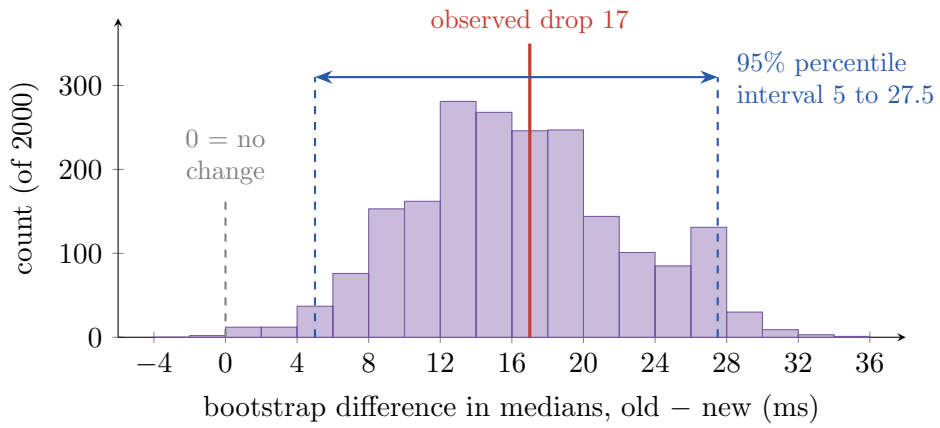


Figure 8.10: Bootstrap distribution of the drop in median latency. The 95% percentile interval, 5 to 27.5 ms (between the dashed blue lines), does not reach 0 (dashed gray): only 7 of the 2000 replicates (0.35%) are at or below zero.

Report: "median latency dropped by 17 ms (95% CI 5 to 27.5 ms)." The basic interval is $[34 - 27.5,\ 34 - 5] = [6.5, 29]$; both exclude zero. The interval is wide because the two medians wobble on their own: in these resamples the old median has SE 5.22 and the new one 2.91. SEs of independent estimates combine by squares, $\sqrt{5.22^2 + 2.91^2} = 5.98$, matching the 5.96 above. The bootstrap does this combination for you, for any statistic.

Common trap

Do not pool the two groups and resample from the pool to build a CI. Pooling pretends both groups come from one population, which is the no-difference hypothesis. That is a valid move for a *test* (Section 8.15) but it gives the wrong spread for an interval around the observed difference.

8.11 An exact interval for the median from order statistics

For medians and other quantiles there is something better than the bootstrap: an interval whose coverage is guaranteed for *any* continuous population and any n, with no resampling at all. It needs

only one fact.

Intuition

For a continuous population, each observation lands below the true median m with probability exactly $1/2$. That is the definition of the median. So the number of observations below m behaves like the number of heads in n fair coin flips.

Write $x_{(1)} \le x_{(2)} \le \cdots \le x_{(n)}$ for the sorted data. These are the **order statistics**; $x_{(j)}$ is the j-th smallest value. Let K be the number of observations below m. Then $K \sim \text{Binomial}(n, 1/2)$ (Probability From the Ground Up, Ch. 7).

Now ask when the interval $[x_{(j)}, x_{(n+1-j)}]$ misses m.

- The whole interval sits above m ($m < x_{(j)}$) exactly when fewer than j observations are below m: $K \le j-1$.
- The whole interval sits below m ($m > x_{(n+1-j)}$) exactly when fewer than j observations are above m: $n - K \le j-1$.

By the symmetry of Binomial$(n, 1/2)$ both events have the same probability, and they cannot happen together. So:

Theorem 8.1 (Distribution-free CI for the median)**.** *If $X_1, \ldots, X_n$ are iid from a continuous distribution with median m, then*

$$\mathbb{P}\big(X_{(j)} \le m \le X_{(n+1-j)}\big) = 1 - 2\,\mathbb{P}(K \le j-1), \qquad K \sim \text{Binomial}(n, \tfrac{1}{2}).$$

Choose the largest j that keeps the coverage at least 95%. For $n = 20$ the table below lists the binomial probabilities.

k	$\mathbb{P}(K = k)$	$\mathbb{P}(K \le k)$	interval $[x_{(k+1)}, x_{(20-k)}]$	coverage $1 - 2\,\mathbb{P}(K \le k)$
2	0.0002	0.0002	$[x_{(3)}, x_{(18)}]$	0.9996
3	0.0011	0.0013	$[x_{(4)}, x_{(17)}]$	0.9974
4	0.0046	0.0059	$[x_{(5)}, x_{(16)}]$	0.9882
5	0.0148	0.0207	$[x_{(6)}, x_{(15)}]$	**0.9586**
6	0.0370	0.0577	$[x_{(7)}, x_{(14)}]$	0.8847
7	0.0739	0.1316	$[x_{(8)}, x_{(13)}]$	0.7368
8	0.1201	0.2517	$[x_{(9)}, x_{(12)}]$	0.4966

The answer for $n = 20$ is $[x_{(6)}, x_{(15)}]$, with exact coverage 95.86%. Figure 8.11 shows where the number comes from.

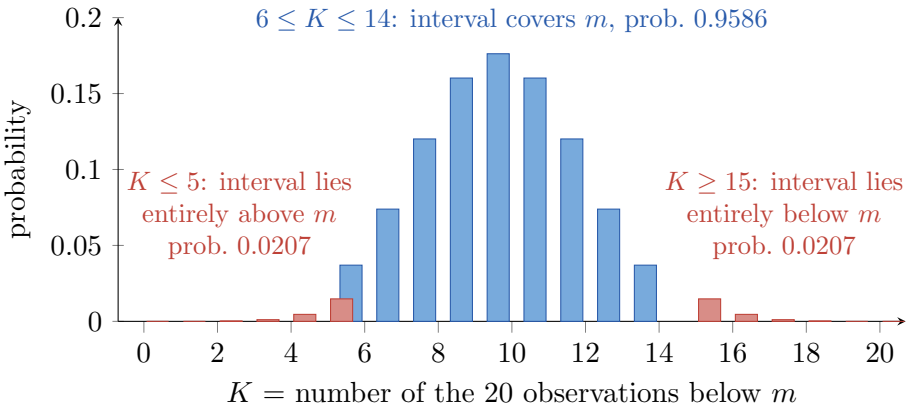


Figure 8.11: Binomial$(20, 1/2)$. The interval $[x_{(6)}, x_{(15)}]$ misses the median only when 5 or fewer, or 15 or more, observations fall below it (red). Total miss probability $2 \times 0.0207 = 0.0414$.

Walking question

Take the first 20 of our latencies. Sorted:

12	18	21	23	24	**24**	33	39	40	42
43	48	50	53	**63**	64	68	68	70	79

The sample median is $(42 + 43)/2 = 42.5$. The 95.86% interval is $[x_{(6)}, x_{(15)}] = [24, 63]$ ms. No resampling, no assumption about the shape of the latency distribution, and the coverage is exact rather than approximate.

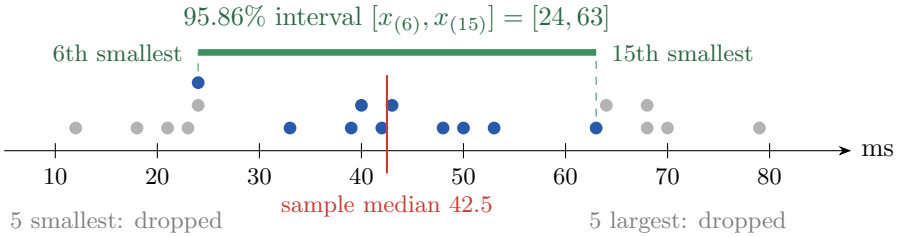


Figure 8.12: The order-statistic interval counts in from each end of the sorted 20 latencies: drop the 5 smallest and the 5 largest (gray) and keep the range of the middle 10 (blue). Dots are stacked where two values are equal or within 1 ms of each other, so every one of the 20 can be counted.

For all 40 latencies the same recipe with Binomial$(40, 1/2)$ gives $[x_{(14)}, x_{(27)}] = [32, 52]$ with coverage 96.15%. Compare the bootstrap percentile interval $[33, 52]$: nearly the same. That is no accident. A resample median is at most $x_{(j)}$ roughly when at least half of the 40 draws land on the j smallest data values. Each draw does so with probability $j/40$, so that count is Binomial$(40, j/40)$. The bootstrap's lower endpoint is where this event has probability 2.5%. With the normal approximation, that is where half the draws, 20, sits 1.96 SDs above the mean j:

$$20 \approx j + 1.96\sqrt{40 \cdot \tfrac{1}{2} \cdot \tfrac{1}{2}} \quad \Longrightarrow \quad j \approx 20 - 6.2 = 13.8.$$

The exact method chose the largest j with $\mathbb{P}(K \le j - 1) \le 0.025$ for $K \sim$ Binomial$(40, 1/2)$: again a cut-off about $1.96 \times 3.16 = 6.2$ below the centre 20, which the exact table turns into $j = 14$. Both

methods are counting coin flips with probability near 1/2, so for the median they land on nearly the same order statistics. The exact method gets there with a guarantee; the bootstrap gets there by simulation.

The guarantee assumed a continuous population. If the population is discrete (latencies rounded to whole ms, with ties), each observation is at or below m with probability at least 1/2 and at or above m with probability at least 1/2. Both miss probabilities can only shrink, so the closed interval $[x_{(j)}, x_{(n+1-j)}]$ covers at least as often as the table says.

8.11.1 Any quantile

The argument works for any quantile q_p (the value with fraction p of the population below it). Now each observation falls below q_p with probability p, so $K \sim \text{Binomial}(n, p)$, and

$$\mathbb{P}\big(X_{(j)} \le q_p < X_{(k)}\big) = \mathbb{P}(j \le K \le k-1), \qquad K \sim \text{Binomial}(n, p).$$

For a rough first guess, use the normal approximation to the binomial: $j \approx np - 1.96\sqrt{np(1-p)}$ and $k \approx np + 1.96\sqrt{np(1-p)} + 1$, then adjust with exact binomial probabilities.

Walking question

p90 from $n = 40$: $np = 36$, $\sqrt{np(1-p)} = 1.90$, so the guess is $j \approx 32.3$, $k \approx 40.7$. The upper index runs past n. Exact search: $[x_{(32)}, x_{(40)}] = [63, 145]$ has coverage $\mathbb{P}(32 \le K \le 39) = 0.970$. The upper end is the sample maximum.

That interval is honest about what 40 points can tell you about a p90: the upper limit is "somewhere up to the largest thing we saw." The bootstrap percentile interval $[59.4, 83.0]$ looked tighter only because the bootstrap cannot see beyond the data. For an upper bound on p99 the arithmetic is stark: the sample max exceeds $q_{0.99}$ with probability $1 - 0.99^n$, which first reaches 95% at $n = 299$. With fewer than 299 requests, no order statistic gives a distribution-free one-sided 95% upper bound on the p99.

Pause

Pause and think: with $n = 5$ observations, can any pair of order statistics give a 95% distribution-free interval for the median? (Answer: no. The widest choice is $[x_{(1)}, x_{(5)}]$, which misses when all five land on one side of m: probability $2 \times (1/2)^5 = 1/16$, so coverage is only 93.75%. With $n = 6$ the range $[x_{(1)}, x_{(6)}]$ reaches $1 - 2/64 = 96.9\%$. Six is the smallest sample that supports a 95% median interval at all.)

8.12 How many resamples? Monte Carlo error

There are two sources of randomness in a bootstrap answer.

1. **Sampling error**: the data are one random sample. This is what the interval is supposed to describe. More resamples cannot reduce it; only more data can.
2. **Monte Carlo error**: we used B random resamples instead of all n^n possible ones. Re-running with a different seed gives a slightly different answer. This shrinks like $1/\sqrt{B}$.

To see the second kind, repeat the whole 40-latency bootstrap 300 times with different seeds for each B and look at how much the reported SE moves (Figure 8.13).

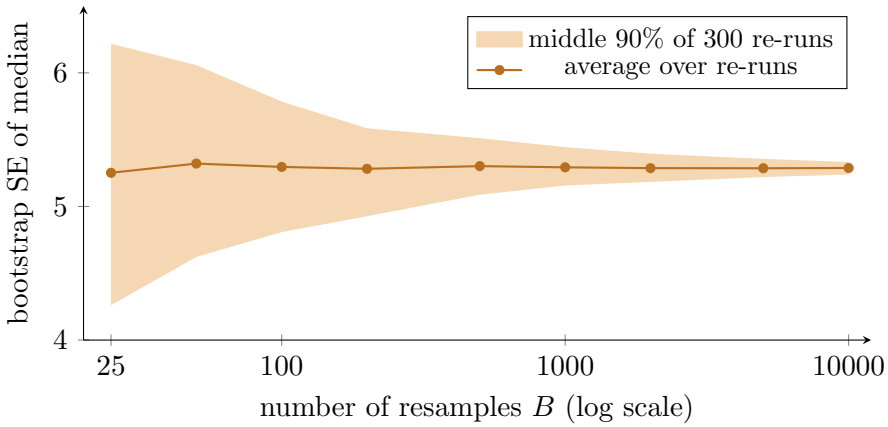


Figure 8.13: Monte Carlo error in the bootstrap SE. With $B = 25$ one run might report anything from 4.3 to 6.2; with $B = 1000$ the band is 5.16 to 5.44; with $B = 10{,}000$ it is 5.24 to 5.33. The centre does not move: more resamples sharpen the estimate of $\mathrm{SE}(\hat{F}_n, n)$ but never change it.

For a roughly bell-shaped bootstrap distribution, the Monte Carlo SD of the bootstrap SE is about $\mathrm{SE}/\sqrt{2B}$: at $B = 1000$, about 2.2% of the SE. Here we measured 0.09 out of 5.29, about 1.7%. Percentile endpoints need more resamples because they depend on the few replicates in each tail: at $B = 1000$ the lower 2.5% endpoint had Monte Carlo SD 0.37 ms; at $B = 10{,}000$, 0.10 ms.

goal	resamples B
bootstrap SE, rough	200
bootstrap SE, reported to 2 digits	1000
percentile or BCa 95% interval	2000–10,000
99% interval or small p-value	10,000+

Computers are fast and the latency median of 10,000 resamples takes a fraction of a second. Pick B big enough that re-running with a new seed does not change the digits you report. Then fix and report the seed.

8.13 How well do the intervals cover?

A 95% interval method should capture the truth in 95% of repeated samples (Chapter 7). We can check this by simulation because we know the population that generated our latencies: log-normal with median exactly 40 ms. For each sample size n, draw 4000 samples, build each interval (with $B = 1000$ for the bootstrap ones), and count how often it contains 40.

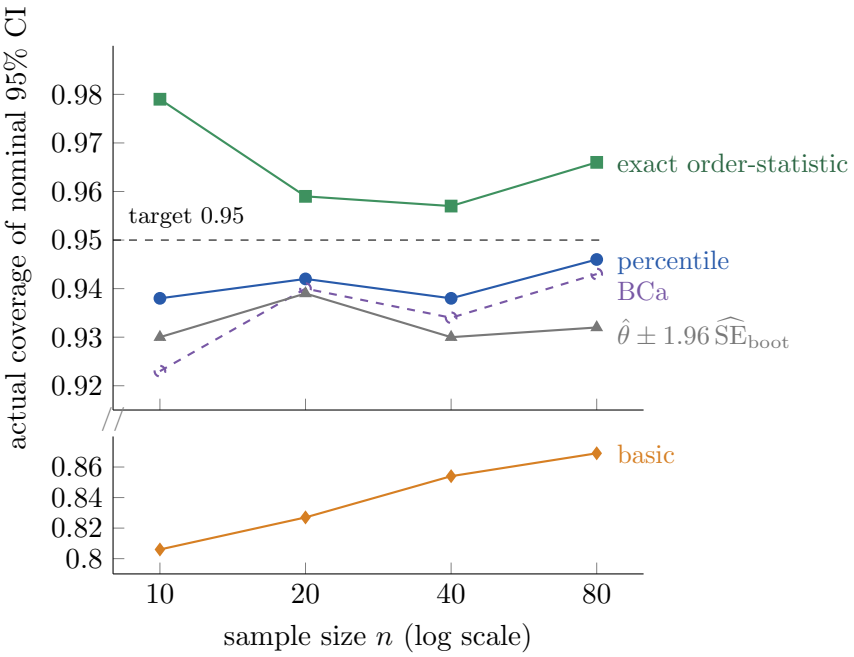


Figure 8.14: Coverage of five 95% intervals for the median of a log-normal population (4000 simulated samples per point; Monte Carlo SD about 0.0035). The vertical axis is broken between 0.88 and 0.915 so the four upper lines can be told apart. The exact interval (green) never drops below 95%; its zig-zag comes from the binomial steps (Section 8.11). Percentile and BCa run 1 to 2 points low. The basic interval (bottom panel) under-covers badly for the median.

Four lessons from Figure 8.14:

- For a median or quantile, use the exact order-statistic interval. It is guaranteed, and it is at least as cheap as the bootstrap.
- The bootstrap percentile interval is a good general default, but for the median at these sample sizes expect it to run 1 to 2 points short of nominal (about 94% instead of 95%).
- BCa buys nothing for the median (the jackknife issue above). For smooth statistics it is usually the best bootstrap interval.
- The basic interval is not a safe default. Its mirror-image logic fails when the bootstrap distribution is lumpy and skewed.

8.14 When the bootstrap fails

The bootstrap needs two things: $\hat{F}_n$ must resemble F in the ways that matter for the statistic, and the statistic must respond smoothly to small changes in the data. Each failure below breaks one of them.

8.14.1 Tiny samples

With $n = 8$ there are only $\binom{15}{8} = 6435$ distinct resamples (as multisets, ignoring order) and the median takes a few dozen values. $\hat{F}_n$ is a bag of 8 slips; it cannot represent the population's spread or tail. Below $n \approx 15$ treat any bootstrap interval as rough, and prefer an exact method when one exists.

8.14.2 Extremes: the maximum

Resample the 40 latencies and take the max. The largest value, 145, is in the resample unless all 40 draws miss it:

$$\mathbb{P}(\max^* = 145) = 1 - (39/40)^{40} = 0.637.$$

Figure 8.15 shows the result: almost two-thirds of the bootstrap distribution is a single spike at the observed max.

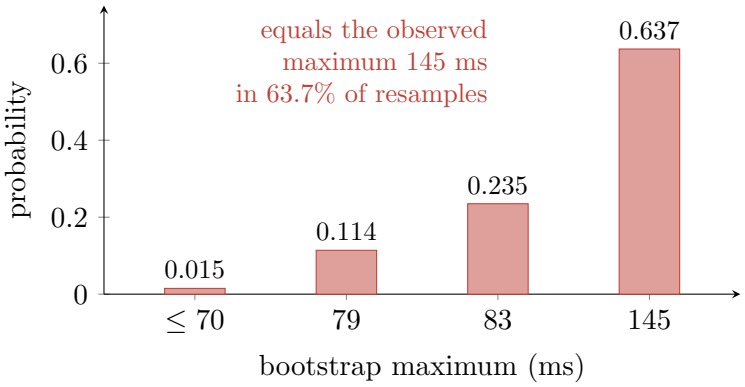


Figure 8.15: Exact bootstrap distribution of the maximum of 40 latencies. The replicate can never exceed the observed maximum, and 63.7% of the time it equals it. Any interval built from this distribution sits at or below 145, even though the population maximum is larger.

The problem is structural. If data come from Uniform$(0, \theta)$ and you want θ, every bootstrap replicate is at most $x_{(n)} < \theta$, so the percentile interval has coverage exactly zero. For the max, min, and very high quantiles, $\hat{F}_n$ is wrong in exactly the region the statistic looks at.

8.14.3 Heavy tails

A **heavy-tailed** distribution is one where extreme values are common enough that the variance is huge or infinite. The Pareto family with tail index α is the standard example; smaller α means heavier tails, and the variance is infinite for $\alpha \leq 2$. Bootstrap percentile intervals for the *mean* of $n = 50$ Pareto values (2000 simulations, $B = 1000$):

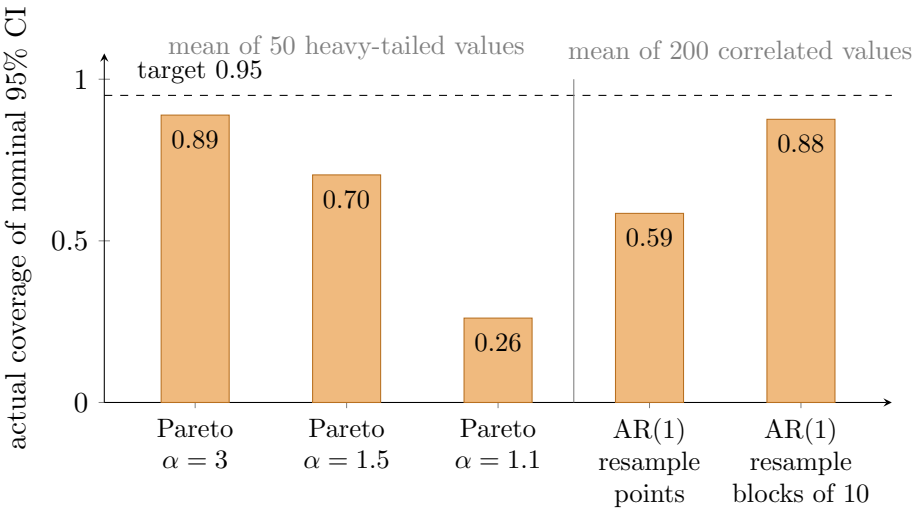


Figure 8.16: Where the percentile bootstrap breaks: every bar falls short of the dashed 95% target. Left three bars: CI for the mean of 50 Pareto values; the heavier the tail (smaller α), the worse the coverage. Right two: CI for the mean of 200 autocorrelated values; resampling blocks of 10 instead of single points recovers most of the lost coverage. Each bar is 1000–2000 simulated datasets.

With $\alpha = 1.1$ coverage collapses to 26%. The sample rarely contains the giant values that dominate the population mean, so $\hat{F}_n$ has a much lighter tail than F and the bootstrap confidently reports a small spread. For heavy-tailed metrics (revenue per user, file sizes, p99.9 latency) prefer the median or a trimmed mean, or a log transform, over bootstrapping the raw mean.

8.14.4 Dependent data and the block bootstrap

Resampling individual points assumes they were drawn independently. A latency time series is not: a slow minute is usually followed by another slow minute. For a series where each value is 0.7 times the previous one plus noise (an AR(1) process, "autoregressive of order 1"), $n = 200$, the ordinary bootstrap CI for the mean covered only 58.5% of the time. Shuffling points destroys the runs of high and low values, so it underestimates how much the mean of a correlated series wanders.

The **moving block bootstrap** resamples contiguous chunks instead (Figure 8.17): cut the series into all overlapping blocks of length ℓ, draw n/ℓ blocks with replacement, and glue them together. Correlation inside each block survives. With $\ell = 10$, coverage rose to 87.6%, still short of 95% but far closer. Longer blocks keep more dependence but leave fewer distinct blocks; a common rule of thumb is ℓ around $n^{1/3}$, tuned up for strongly correlated series.

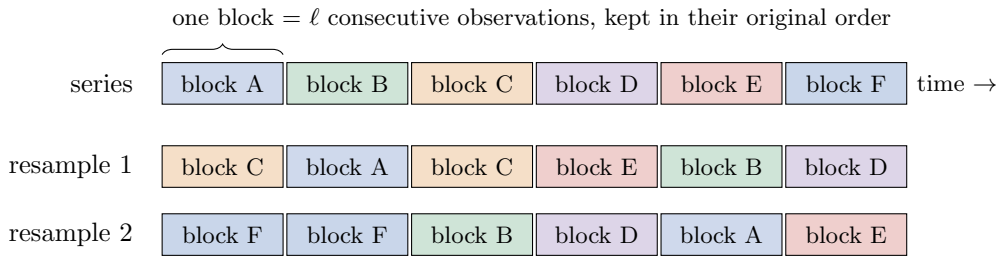


Figure 8.17: Block bootstrap: the unit of resampling is a run of consecutive observations, so short-range dependence is kept inside each block. (Shown with non-overlapping blocks for clarity; the moving block version allows every starting point.)

The same logic applies to clustered data. If you have 50 users each with 20 sessions, resample *users*, not sessions. The unit you resample must be the unit that was sampled independently.

8.15 Bootstrap vs permutation: a preview

Section 8.10 asked "how big is the drop, give or take?" Chapter 9 asks a different question: "could a drop this big appear if the deploy changed nothing?" Both are answered by resampling, but differently (Figure 8.18).

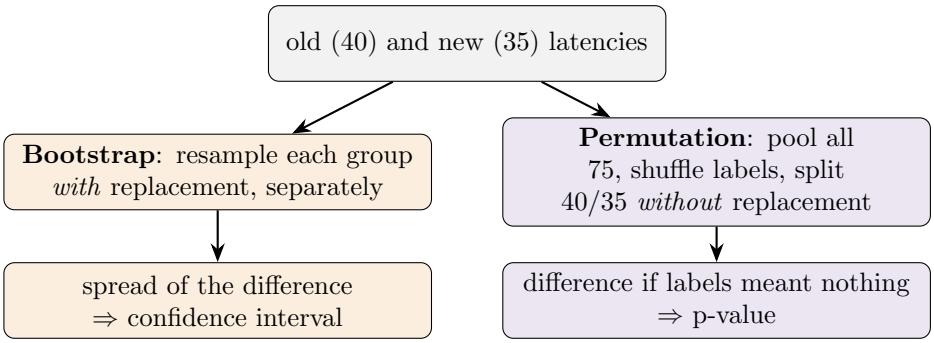


Figure 8.18: Two resampling tools with different jobs. The bootstrap keeps the groups apart and measures uncertainty around the observed effect. The permutation test enforces "no difference" by shuffling labels and asks how unusual the observed effect is under that assumption.

Connection

The bootstrap shows up throughout machine learning. *Bagging* (bootstrap aggregating) trains one model per bootstrap resample and averages them; random forests are bagged decision trees. The $\approx 37\%$ of points left out of each resample (the $1/e$ from Section 8.3) form the "out-of-bag" set, a free validation set. Evaluation code that reports a model's accuracy "$\pm$" something often bootstraps the test set: resample test examples, recompute accuracy, take percentiles. Chapter 15 returns to this.

History

Bradley Efron, a statistician at Stanford, published "Bootstrap methods: another look at the jackknife" in 1979. The *jackknife* (Maurice Quenouille, 1949; John Tukey, 1958) estimated bias and standard errors by recomputing a statistic with one observation left out at a time, but it gave nonsense for the median and offered no way to get a full sampling distribution. Efron saw that the jackknife was a crude approximation to a cleaner idea: treat the sample as the population and simulate. The name comes from the phrase "pulling yourself up by your own bootstraps," since the data are used to assess themselves. The idea only became practical as computers got cheap; it unlocked standard errors and intervals for medians, correlations, and complex estimators that had no workable formula, and Efron's later BCa method (1987) made the intervals accurate enough for routine use.

8.16 Choosing a method

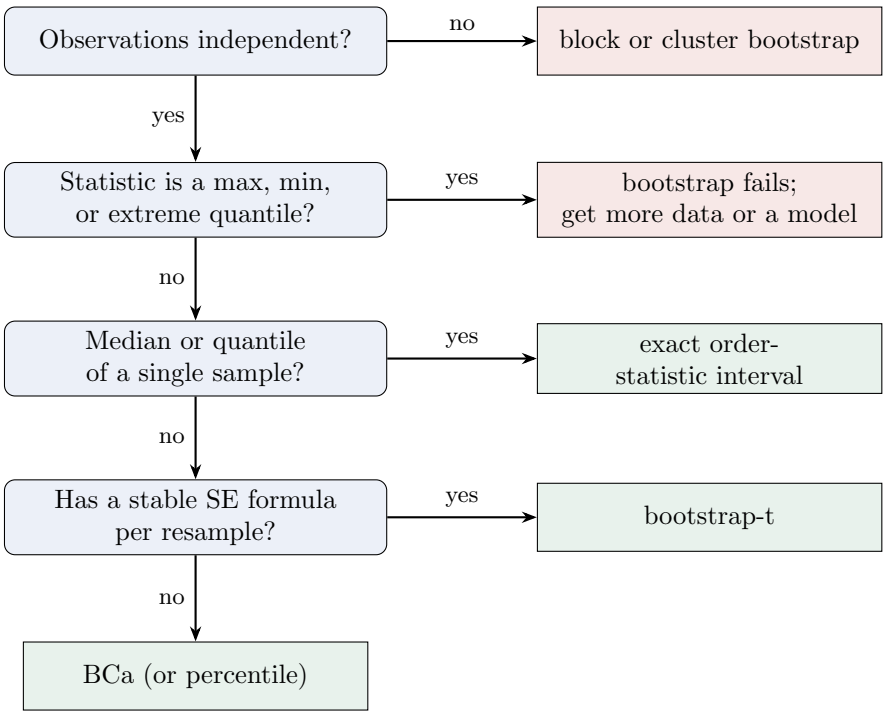


Figure 8.19: A decision path for a 95% interval when no closed-form formula applies. With n below about 15, read any bootstrap answer as rough.

8.17 Summary

- **Plug-in principle**: the empirical distribution $\hat{F}_n$ (each data point with weight $1/n$) stands in for the unknown population.
- **Resample**: draw n values with replacement; compute the statistic; repeat B times. The replicates form the bootstrap distribution, centred near $\hat{\theta}$, with roughly the spread and shape

of the sampling distribution.

- **Bootstrap SE** = SD of the replicates. For the mean it reproduces $\hat{\sigma}/\sqrt{n}$.
- **Intervals**: percentile $[\hat{\theta}^*_{(.025)}, \hat{\theta}^*_{(.975)}]$ (good default); basic $[2\hat{\theta} - \hat{\theta}^*_{(.975)}, 2\hat{\theta} - \hat{\theta}^*_{(.025)}]$ (often worse); bootstrap-t (studentized, for statistics with a stable SE); BCa (bias and skew corrected, best for smooth statistics).
- **Two groups**: resample each group separately, record the difference.
- **Exact median CI**: $[x_{(j)}, x_{(n+1-j)}]$ with coverage $1 - 2\,\mathbb{P}(K \le j-1)$, $K \sim \text{Binomial}(n, 1/2)$; for $n = 20$ use $j = 6$ (95.86%). For quantile q_p use $K \sim \text{Binomial}(n, p)$.
- **Choose** B so the reported digits do not change with the seed: 1000 for an SE, 2000–10,000 for intervals.
- **Failures**: tiny n, maxima and extreme quantiles, heavy tails, dependent data (use blocks or clusters).
- **Bootstrap vs permutation**: bootstrap gives intervals around an effect; permutation gives p-values under "no effect" (Chapter 9).

Chapter 9 turns the question around: instead of "what range of values is plausible?" it asks "is this one value, zero, plausible?" and builds the machinery of hypothesis tests.

8.18 Exercises

1. The data are $\{3, 5, 9\}$. How many distinct resamples are there, counting a resample as a multiset? What is the probability that a resample's median is 9?
2. For $n = 10$, what is the probability that a given observation is absent from a resample? Compare with the limit as $n \to \infty$.
3. Use Binomial$(10, 1/2)$ to find the order statistics that give a distribution-free CI for the median with coverage at least 95%, and state the exact coverage.
4. A statistic has $\hat{\theta} = 10$ and bootstrap percentiles $\hat{\theta}^*_{(0.025)} = 8$, $\hat{\theta}^*_{(0.975)} = 15$. Write down the percentile and the basic intervals. Which side of $\hat{\theta}$ is longer in each?
5. With $B = 200$ resamples you get a bootstrap SE of 5.0. Roughly how much would that number change if you re-ran with a new seed? How large must B be to make the Monte Carlo SD about 0.05?
6. Data are iid Uniform$(0, \theta)$, $n = 20$. What is the probability that a bootstrap resample's maximum equals the sample maximum? What is the coverage of the percentile interval for θ?
7. What is the smallest n for which the sample maximum is a one-sided 95% upper confidence bound for the p95?
8. You have 2000 user sessions from 100 users, and want a CI for mean session length. What do you resample, and why?
9. To bootstrap the correlation between height and weight from 50 people, a colleague resamples 50 heights and, independently, 50 weights. What will the bootstrap distribution look like, and what is the fix?

Answers

1. $\binom{3+3-1}{3} = 10$ multisets. The median is 9 when at least two of three draws are 9: $3(1/3)^2(2/3) + (1/3)^3 = 7/27 \approx 0.259$.
2. $(0.9)^{10} = 0.349$; the limit is $1/e = 0.368$.
3. $\mathbb{P}(K \le 1) = 11/1024$, so $[x_{(2)}, x_{(9)}]$ has coverage $1 - 22/1024 = 0.979$. ($[x_{(3)}, x_{(8)}]$ gives only 0.891.)
4. Percentile $[8, 15]$, longer on the right (5 vs 2). Basic $[20 - 15, 20 - 8] = [5, 12]$, longer on the left.
5. Monte Carlo SD $\approx 5.0/\sqrt{400} = 0.25$. Need $5/\sqrt{2B} = 0.05$, so $2B = 10{,}000$, $B = 5000$.
6. $1 - 0.95^{20} = 0.642$. Coverage is 0: every replicate is at most $x_{(n)} < \theta$, so the interval lies entirely below θ.
7. Need $1 - 0.95^n \ge 0.95$, i.e. $0.95^n \le 0.05$: $n \ge \ln 0.05 / \ln 0.95 = 58.4$, so $n = 59$.
8. Resample users (all of a user's sessions together). Sessions from one user are correlated; resampling sessions treats 2000 dependent values as independent and gives an interval that is too narrow.
9. Pairing is destroyed, so replicates are correlations of unrelated variables, centred near 0 with spread about $1/\sqrt{50}$. Resample whole (height, weight) pairs.

Part III

Testing and Comparing

Chapter 9

Hypothesis Testing

Chapters 7 and 8 built intervals: ranges of parameter values that are compatible with the data. This chapter asks a sharper, yes-or-no style question. A friend says her new checkout button is faster, or that she can predict coin flips, or that a drug lowers blood pressure. The data show *some* effect. Could plain luck have produced it? A hypothesis test answers that by playing a game with a skeptic.

9.1 The skeptic's game

Here is the puzzle. Your friend claims she can predict coin flips. You flip a fair coin 20 times; before each flip she calls heads or tails. She gets 16 of the 20 calls right.

Is she gifted? You cannot answer that directly, because "gifted" is vague. Gifted at the level of 60%? 75%? 99%? Each would predict a different number of hits, and you have no way to pick among them. But there is one world you can describe exactly: the world where she has no gift at all and each call is a fair 50/50 guess. Call it the **null world**, the boring world where nothing is going on.

The skeptic's move is to say: "Let us assume she is guessing, and see how surprising 16 would be." In the null world, the number of correct calls is a count of heads in 20 fair flips. You can simulate it. Imagine 1000 people with no gift at all, each making 20 blind guesses against 20 coin flips and writing down how many they got right. Figure 9.1 is the histogram of those 1000 scores (a real simulation with a fixed seed).

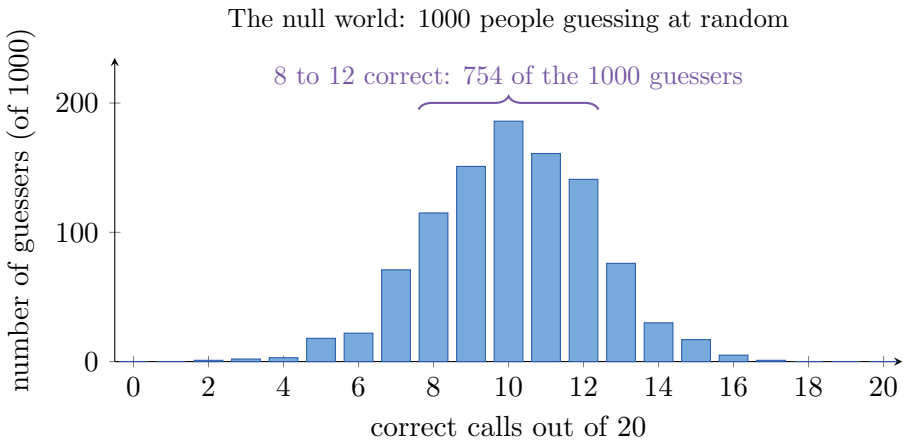


Figure 9.1: What "nothing is going on" looks like. Three quarters of the random guessers score between 8 and 12 out of 20, and scores far from 10 are rare. This picture is the *null distribution*; it was computed without knowing anything about your friend.

Now drop her score onto the picture. In Figure 9.2 the bars at 16 or more are shaded red. Only $5 + 1 = 6$ of the 1000 random guessers did as well as she did.

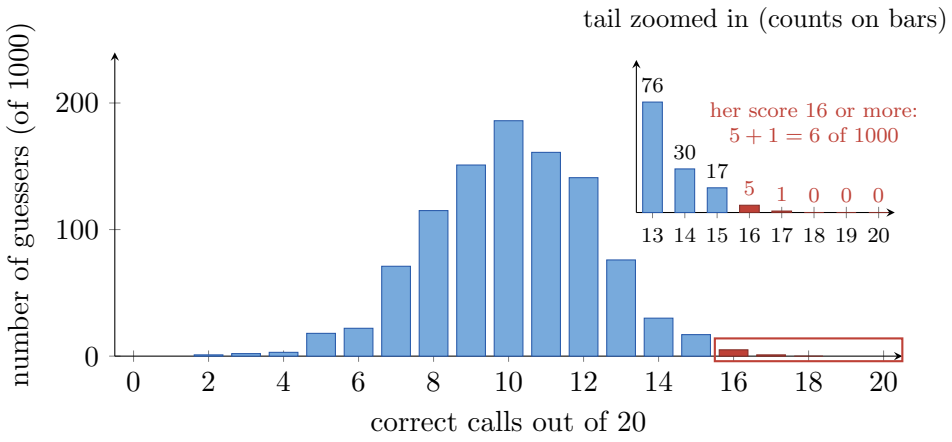


Figure 9.2: The observed result dropped into the null world. In the full histogram the tail at 16 or more (red box) is too thin to see, so the inset zooms in on it: only 6 of the 1000 simulated guessers did as well as your friend. That fraction, about 0.006, measures how surprised a skeptic should be.

That fraction is the **p-value**: the share of the null world that produces a result at least as extreme as the one you saw. The simulation gives 0.006. The exact binomial calculation (Probability From the Ground Up, Ch. 7) gives

$$\mathbb{P}(X \geq 16) = \sum_{k=16}^{20} \binom{20}{k} 0.5^{20} = \frac{6196}{1048576} \approx 0.0059.$$

Now the skeptic has two choices. Either your friend really is a random guesser and a 1-in-170 fluke just happened, or the null world is the wrong description of her. Most people find the second option more believable. That is the whole logic of a hypothesis test.

Intuition

A hypothesis test is a proof by contradiction that is allowed to fail with small probability. In a mathematical proof you assume a statement, derive something impossible, and conclude the statement is false. Here you assume "nothing is going on", derive that the data you saw would be *very unlikely*, and conclude that "nothing is going on" is hard to believe. "Impossible" becomes "improbable".

Why assume the boring hypothesis and not the exciting one? Because the boring one is the only one specific enough to compute with. Try drawing the exciting world, "she has a gift", and you get stuck at once: to draw a histogram you must pick a hit rate θ (theta), her true long-run fraction of correct calls. Figure 9.3 draws three of the infinitely many choices.

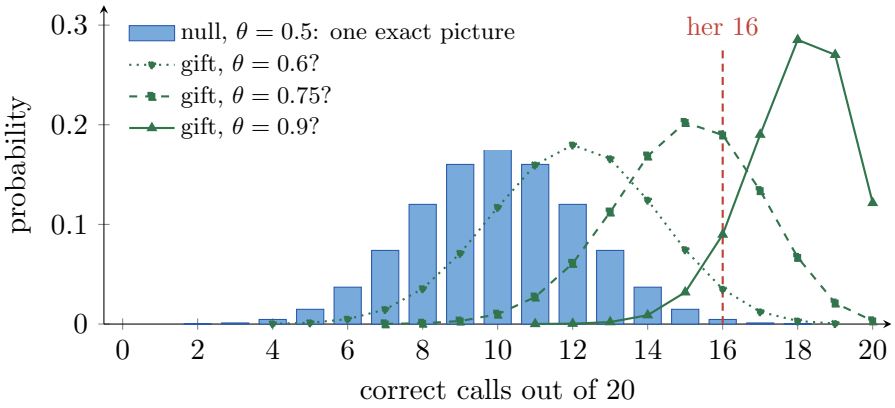


Figure 9.3: The null hypothesis is one world; "she has a gift" is a whole family of worlds. Blue bars: guessing ($\theta = 0.5$). Green curves: three of the infinitely many gifted worlds, each a different prediction. Only the blue picture is pinned down well enough to measure surprise against.

"No gift" pins θ to exactly 0.5 and produces one histogram. "Some gift" means $\theta > 0.5$, which is every green curve at once and no histogram in particular. The test needs one concrete world in which to measure surprise, and only the null hypothesis supplies one. That is the asymmetry at the heart of testing: you can argue *against* the null world by showing your data look out of place in it, but there is no single alternative world to argue *for*.

A courtroom works the same way. The defendant is presumed innocent (the null). The prosecution must show the evidence would be very unlikely if the defendant were innocent. A verdict of "not guilty" does not mean "proven innocent"; it means the evidence was not strong enough. Hold on to that last point. It is the subject of Section 9.2.

9.1.1 Drawing a line in advance

Someone has to decide how small is small enough. The convention is to fix a threshold α (alpha), the **significance level**, before looking at the data. α is the fraction of the null world you are willing to call "surprising". With $\alpha = 0.05$ and a one-sided test you reject the null when your friend's score lands in the top 5% of the null world.

For 20 coin calls the counts are whole numbers, so the cut-off cannot hit 5% exactly. $\mathbb{P}(X \geq 14) =$

0.058 is too big, and $\mathbb{P}(X \geq 15) = 0.021$ fits. So the rule is: *reject "no gift" if she gets* 15 *or more right.* Figure 9.4 shows this rejection region with exact probabilities.

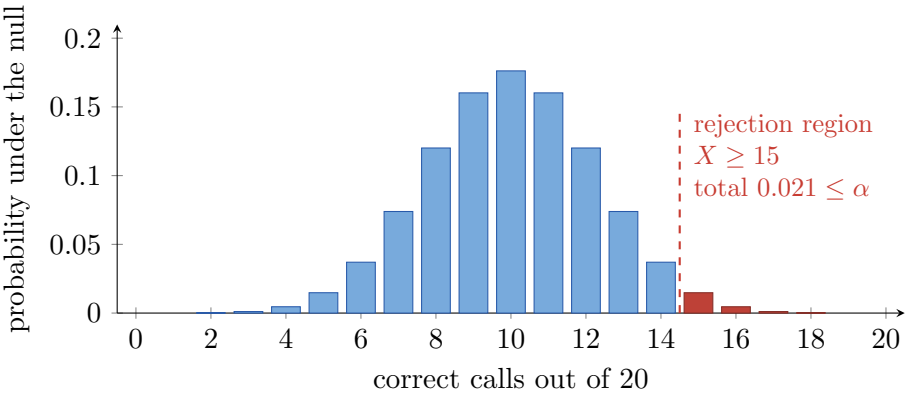


Figure 9.4: Fixing $\alpha = 0.05$ before the experiment turns the null world into a decision rule. Results in the red region lead to rejecting the null. If the null is true, this happens with probability 0.021, never more than α.

The whole procedure fits in one storyboard (Figure 9.5). Every test in this chapter follows panels 1–5; only the statistic and the null world change. Panel 6 is the step the null world cannot do alone: to ask how often the test would catch a *real* gift, you must pick one gifted world and overlay it. That overlay is called power, and Section 9.9 develops it.

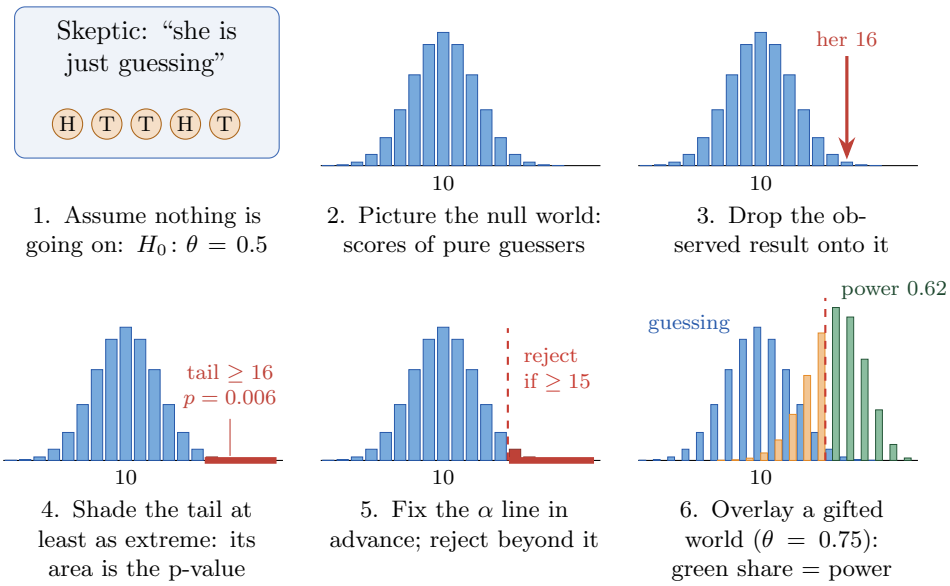


Figure 9.5: The logic of every hypothesis test as a storyboard. Panels 1–5 use only the null world: assume it, draw it, place the data, measure the tail, compare with a line drawn in advance. Panel 6 overlays one gifted world (orange and green bars, next to the blue guessing bars) to see how often a real effect would land past the line; the green share is the power.

Pause

Pause and think: if your friend had scored 13 of 20, what would the skeptic conclude? (Look at Figure 9.4: 13 sits in the blue bulk. $\mathbb{P}(X \geq 13) \approx 0.13$, so the data are unsurprising under the null and you do not reject. You also have not shown she lacks a gift.)

9.2 Absence of evidence is not evidence of absence

Suppose a second friend makes 10 calls and gets 6 right, a 60% hit rate. A third friend makes 1000 calls and gets 600 right, also 60%. Same rate. Very different verdicts.

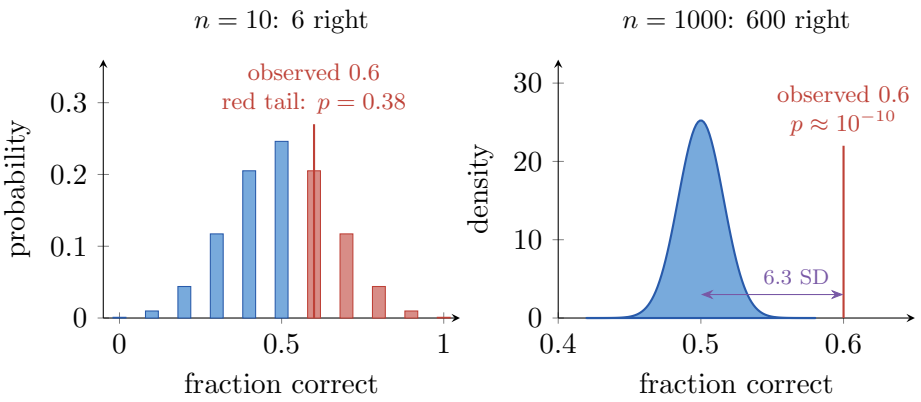


Figure 9.6: Same observed hit rate (60%), two sample sizes; each panel is the null world for that many calls. With 10 calls (left) the null world is wide and 0.6 sits inside it: 38% of pure guessers do at least as well. With 1000 calls (right) the null world is narrow and 0.6 is 6.3 standard deviations outside it. Failing to reject on the left says "too little data", not "no gift".

On the left of Figure 9.6, the null world for $n = 10$ is so wide that 6 of 10 is utterly ordinary: $\mathbb{P}(X \geq 6) = 0.377$. On the right, the null world for $n = 1000$ has standard deviation $\sqrt{0.25/1000} \approx 0.016$ on the fraction scale, and 0.6 sits more than six standard deviations out.

The second friend might have exactly the same skill as the third. Ten calls cannot tell a 60% predictor from a 50% guesser. To see this, run the test not only for the hit rate 0.5 but for every candidate hit rate θ_0, and keep the ones the data fail to reject (two-sided at $\alpha = 0.05$, exact binomial probabilities, 2.5% allowed in each tail). Figure 9.7 shows the survivors.

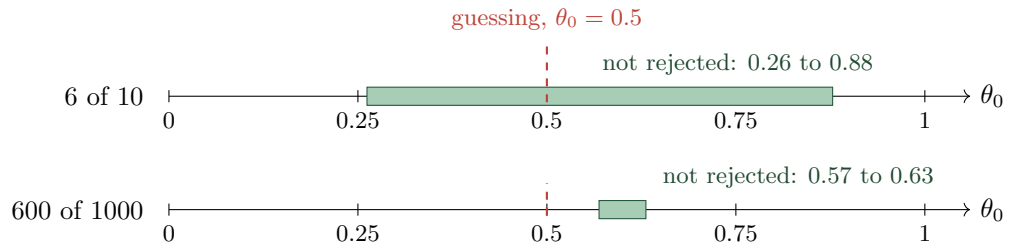


Figure 9.7: Which hit rates survive the test? With 6 of 10 (top), every value from 0.26 to 0.88 survives, including 0.5 and also a strong gift. Failing to reject 0.5 cannot single it out. With 600 of 1000 (bottom) the survivors shrink to a narrow band that excludes 0.5.

With 6 of 10, guessing survives, but so does a 60% gift and so does an 85% gift. The data have not picked out 0.5; they have failed to rule out a wide range of worlds, and 0.5 is one of them. Declaring "$\theta = 0.5$ is accepted" would be as arbitrary as declaring "$\theta = 0.85$ is accepted". That is why a test can *reject* the null but never *accept* it. When you fail to reject, the honest report is "the data are compatible with the null", and also with everything else in the green band. (The green band is a confidence interval; Section 9.10 shows that this is no coincidence.) Section 9.9 turns "too little data" into a number called power.

Common trap

"We found no significant difference, so the two versions perform the same." This is the most common misreading of a test. A large p-value from a small experiment is an absence of evidence. To claim two things are *equal*, look at the confidence interval for the difference (Chapter 7): if it is narrow and sits inside a range you consider negligible, then you have shown equivalence.

9.3 A permutation test: the null world by shuffling

Coin flips have a known null world. Most real questions do not. Here is a technique that builds the null world out of the data itself.

An engineer rewrites the rendering path of a checkout page. She loads the page 12 times: 6 with the old code, 6 with the new, in random order. Render times in ms:

old	212	198	225	240	205	231	mean 218.5
new	190	184	207	195	178	201	mean 192.5

The new code is 26.0 ms faster on average. Could the labels "old" and "new" be irrelevant, with the 26 ms gap an accident of which loads happened to land in which group?

The null hypothesis here is: *the code version makes no difference to render time.* If that is true, each of the 12 numbers would have come out the same no matter which label it got. The labels are then arbitrary tags, and any other way of assigning 6 "old" and 6 "new" tags was equally likely. So shuffle the labels and recompute the difference (Figure 9.8).

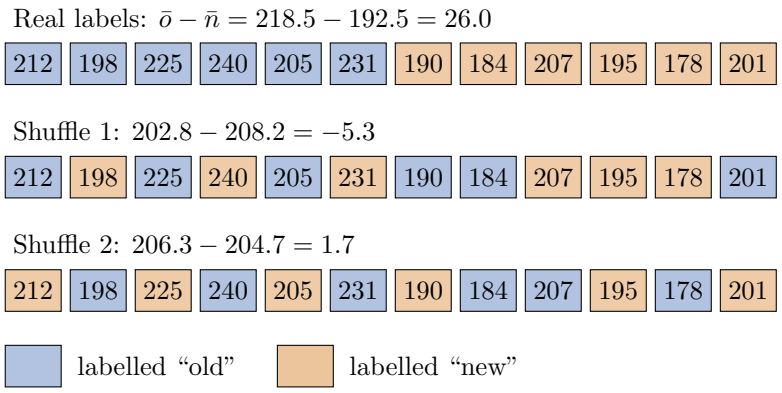


Figure 9.8: Shuffling labels. The twelve measurements stay put; only the colours (labels) move. Under the null the labels carry no information, so each shuffle is a draw from the null world. Shuffled differences are typically a few ms, far from 26.

There are $\binom{12}{6} = 924$ ways to choose which 6 of the 12 values get the "old" label. With so few, we can compute the difference for every one of them. Figure 9.9 is the histogram of all 924.

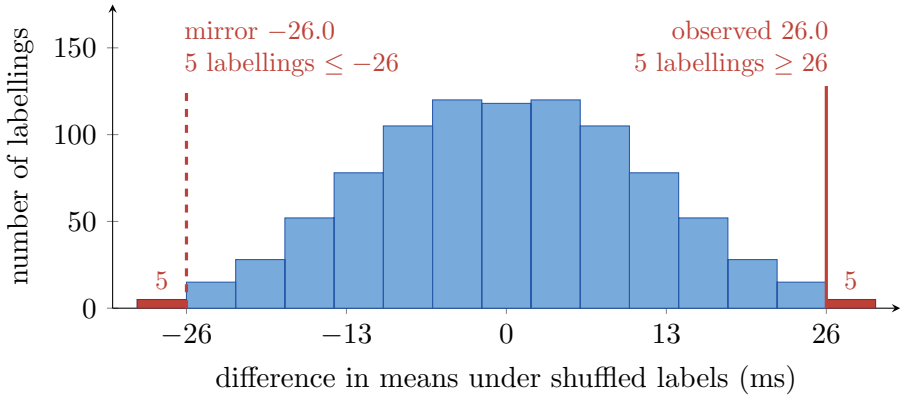


Figure 9.9: The exact null distribution of the difference in means, from all 924 relabellings (bins 4 ms wide, edges at ±26). The real difference, 26.0 ms, sits in the far right tail: the red bar holds the only 5 labellings with a difference of 26 or more, and its mirror holds 5 more with −26 or less. The histogram is symmetric because swapping every label flips the sign.

Count the tail. Exactly 5 of the 924 labellings give a difference of 26.0 or more, and 5 more give −26.0 or less. So:

$$p_{\text{one-sided}} = \frac{5}{924} \approx 0.0054, \qquad p_{\text{two-sided}} = \frac{10}{924} \approx 0.011.$$

If the code change did nothing, a gap this large would show up in about 1 random labelling in 90. That is strong evidence the new code is faster.

Notice what we did not need: no Normal distribution, no formula for a standard error, no assumption about the shape of render times. We only needed one fact: under the null, the labels are exchangeable (any relabelling is as likely as any other). That fact holds because the engineer randomized which loads used which code (Chapter 5).

```
1 import numpy as np
```

```
old = np.array([212,198,225,240,205,231]); new = np.array([190,184,207,195,178,201])
pooled = np.concatenate([old, new]); obs = old.mean() - new.mean()
rng = np.random.default_rng(0); diffs = []
for _ in range(100_000):                    # random shuffles instead of all 924
    p = rng.permutation(pooled)
    diffs.append(p[:6].mean() - p[6:].mean())
print(np.mean(np.abs(diffs) >= obs))        # 0.011, two-sided p-value
```

With larger groups, enumerating every labelling is impossible ($\binom{100}{50} \approx 10^{29}$), so you shuffle a large random sample of labellings instead, as in the code. The answer converges to the exact one.

9.4 The vocabulary, precisely

Every test has the same parts. Here they are, each tied to the examples above.

Definition 9.1 (Hypotheses)**.** The **null hypothesis** H_0 is a statement specific enough to determine the distribution of the data (or at least of a chosen statistic). The **alternative hypothesis** H_1 is what you conclude if you reject H_0.

For the coin: $H_0 : \theta = 0.5$ versus $H_1 : \theta > 0.5$, where θ is her true hit rate. For the render times: H_0: the code makes no difference; in mean-based tests this becomes $H_0 : \mu_{\text{old}} = \mu_{\text{new}}$, with $H_1 : \mu_{\text{old}} \neq \mu_{\text{new}}$.

Definition 9.2 (Test statistic and null distribution)**.** A **test statistic** T is a number computed from the data that tends to be larger (or further from zero) when H_1 is true. Its distribution assuming H_0 is the **null distribution**.

The count of correct calls and the difference in group means are test statistics. Figures 9.1 and 9.9 are null distributions.

Definition 9.3 (p-value)**.** Let t_{obs} be the observed value of T. The **p-value** is

$$p = \mathbb{P}_{H_0}\big(T \text{ is at least as extreme as } t_{\text{obs}}\big),$$

the probability, computed in the null world, of a result at least as extreme as the one observed. "Extreme" means "in the direction of H_1".

The notation $\mathbb{P}_{H_0}$ reads "probability computed as if H_0 were true". It is a reminder that the p-value is a statement about the null world, not about the real one.

9.4.1 One-sided and two-sided

"Extreme" depends on H_1. If only an increase counts as evidence (the coin friend claims to be *better* than chance), extreme means large, and the p-value is the area of the right tail. If either direction counts (the new code might be slower or faster), extreme means far from the null value on either side, and the p-value adds both tails.

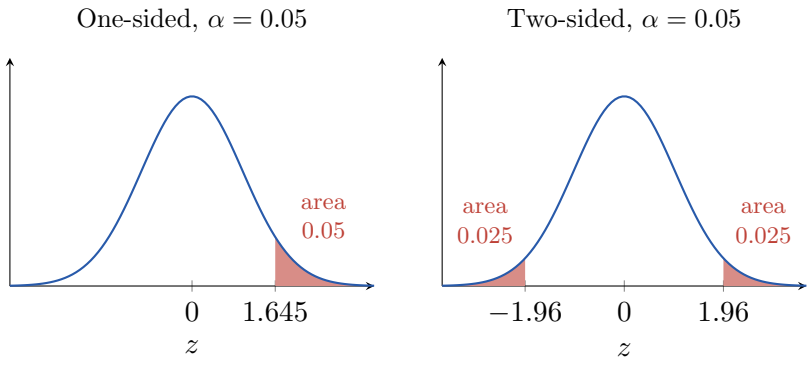


Figure 9.10: Rejection regions (red) for a statistic that is $\mathcal{N}(0,1)$ under H_0; both panels shade a total area of 0.05. A one-sided test spends all of α in one tail, so its cut-off (1.645) is closer to zero. A two-sided test splits α between both tails, so each cut-off moves out to ± 1.96.

Common trap

Choose the sides *before* seeing the data. Running a two-sided test, getting $p = 0.08$, then switching to one-sided to get $p = 0.04$ doubles your real false-alarm rate. The default in practice is two-sided. Use one-sided only when an effect in the other direction would lead to exactly the same action as no effect.

9.5 The z-test

The permutation test built the null world by brute force. When the statistic is a sample mean, Chapter 6 already told us what the null world looks like: by the Central Limit Theorem, $\bar{X}$ is approximately Normal with mean μ and standard error $\sigma/\sqrt{n}$. That gives a null world from a formula.

Example 9.1 (API latency)**.** Historically, an API's response time has mean 200 ms and standard deviation $\sigma = 40$ ms. After a dependency upgrade, a random sample of $n = 64$ requests has mean $\bar{x} = 211$ ms. Did the mean change?

Hypotheses. $H_0 : \mu = 200$ versus $H_1 : \mu \neq 200$.

Null world. If H_0 holds, $\bar{X} \approx \mathcal{N}(200, \mathrm{SE}^2)$ with $\mathrm{SE} = 40/\sqrt{64} = 5$ ms.

Statistic. Measure the observed mean in units of standard error:

$$z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}} = \frac{211 - 200}{5} = 2.2.$$

p-value. Under H_0, $Z \sim \mathcal{N}(0,1)$, so

$$p = \mathbb{P}(|Z| \geq 2.2) = 2\big(1 - \Phi(2.2)\big) = 2(0.0139) = 0.028,$$

where Φ is the standard Normal CDF (the area to the left of a point under the bell curve). Since $0.028 < 0.05$, reject H_0 at the 5% level: the mean latency appears to have risen.

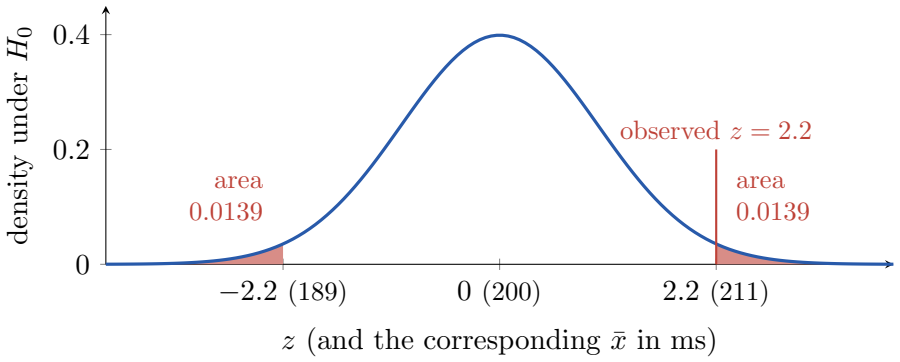


Figure 9.11: The two-sided p-value for $z = 2.2$ is the total red area, 0.028. The left tail counts because a drop to 189 ms would have been equally surprising under H_0: $\mu = 200$.

The z-test needs σ known. That is rare: here it came from a long history of the same service. Usually σ must be estimated from the same sample, which changes the null world slightly. That is the t-test.

9.6 The one-sample t-test

Replace the unknown σ by the sample standard deviation s (Chapter 2):

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}.$$

This looks like z, but its null distribution is wider. The denominator s is itself random: in some samples it comes out too small, which inflates t. The extra spread is captured by the t **distribution with $n-1$ degrees of freedom**, written t_{n-1}. "Degrees of freedom" is the $n-1$ from the variance formula: the number of independent deviations left after estimating the mean.

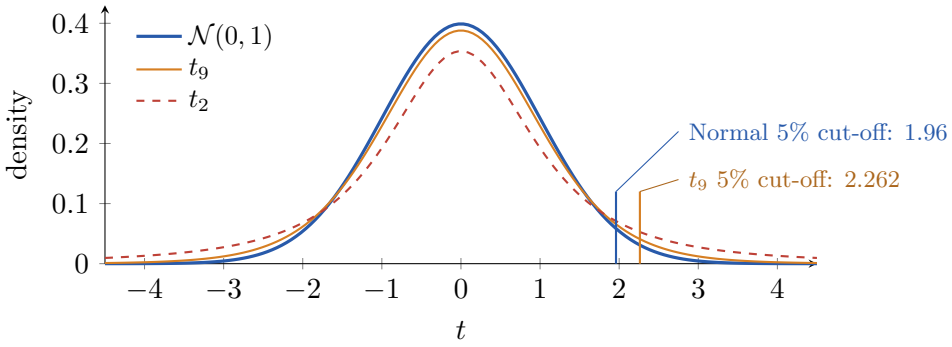


Figure 9.12: The t distribution is a bell with heavier tails. With few degrees of freedom (small samples) extreme values are more common, so the two-sided cut-off for $\alpha = 0.05$ (vertical lines) is further out: 2.262 for t_9 versus 1.96 for the Normal. As n grows, t_{n-1} approaches $\mathcal{N}(0,1)$.

Why it works

If the data are $\mathcal{N}(\mu, \sigma^2)$, then $(\bar{X} - \mu)/(\sigma/\sqrt{n})$ is exactly $\mathcal{N}(0,1)$, $(n-1)s^2/\sigma^2$ has a chi-square distribution with $n-1$ degrees of freedom, and the two are independent. A standard Normal divided by the square root of an independent (chi-square / its degrees of freedom) is, by definition, t_{n-1}. The σ cancels, which is why the test needs no knowledge of it. (Derivation: see Probability From the Ground Up, Ch. 32.)

Example 9.2 (Does the service meet its claim?)**.** A vendor claims mean response time 120 ms. You time $n = 10$ requests:

$$128,\ 135,\ 119,\ 142,\ 126,\ 131,\ 124,\ 138,\ 117,\ 133.$$

Summaries. The sum is 1293, so $\bar{x} = 129.3$. The squared deviations from 129.3 sum to 584.1, so $s^2 = 584.1/9 = 64.9$ and $s = 8.056$. The standard error is $s/\sqrt{10} = 2.548$.

Statistic. $H_0 : \mu = 120$ versus $H_1 : \mu \neq 120$:

$$t = \frac{129.3 - 120}{2.548} = 3.65, \qquad \mathrm{df} = 9.$$

p-value. $p = \mathbb{P}(|T_9| \geq 3.65) = 0.0053$. The cut-off for $\alpha = 0.05$ is $t_{9,\,0.975} = 2.262$, and 3.65 is well past it. Reject: the true mean is very likely above 120 ms.

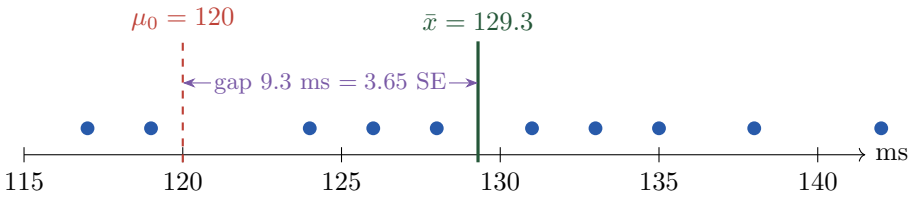


Figure 9.13: The ten response times (one dot per request), the claimed mean and the sample mean. The gap of 9.3 ms is 3.65 standard errors: large relative to how much $\bar{x}$ wobbles from sample to sample, even though it is small relative to the spread of individual requests.

9.7 Two samples: pooled and Welch t-tests

Return to the render times. Now test $H_0 : \mu_{\text{old}} = \mu_{\text{new}}$ using a formula instead of shuffles. The statistic is again "difference over its standard error":

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\mathrm{SE}(\bar{x}_1 - \bar{x}_2)}.$$

The two versions of the test differ only in how they estimate that standard error.

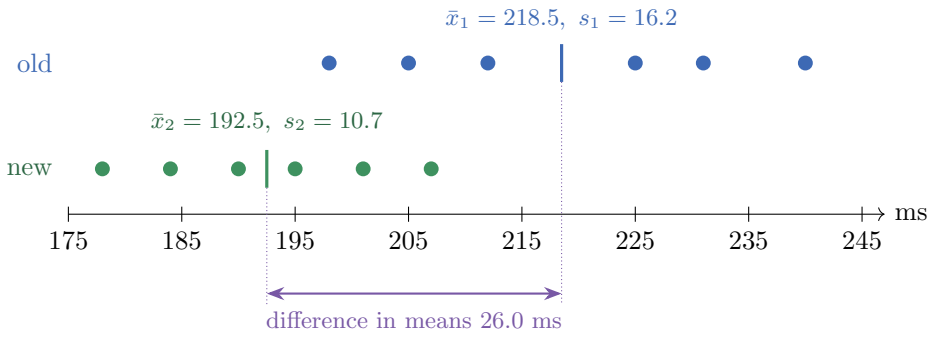


Figure 9.14: Render times by version (one dot per page load), with group means as vertical ticks. The groups overlap a little, and the old group is more spread out. The t-test asks whether the 26 ms gap between the means (purple arrow) is large compared with this within-group noise.

Pooled (Student) t-test. Assume both groups share one variance σ^2. Estimate it by combining the two sample variances, weighted by their degrees of freedom:

$$s_p^2 = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}, \qquad \mathrm{SE} = s_p\sqrt{\frac{1}{n_1} + \frac{1}{n_2}}, \qquad \mathrm{df} = n_1 + n_2 - 2.$$

For the render times, $s_1^2 = 261.1$ and $s_2^2 = 115.5$:

$$s_p^2 = \frac{5(261.1) + 5(115.5)}{10} = 188.3, \quad \mathrm{SE} = 13.72\sqrt{\tfrac{1}{6} + \tfrac{1}{6}} = 7.92, \quad t = \frac{26.0}{7.92} = 3.28.$$

With 10 degrees of freedom, $p = 0.0083$.

Welch t-test. Drop the equal-variance assumption. Each mean has its own variance s_i^2/n_i, and variances of independent quantities add:

$$\mathrm{SE} = \sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}, \qquad \mathrm{df} \approx \frac{\left(s_1^2/n_1 + s_2^2/n_2\right)^2}{\dfrac{(s_1^2/n_1)^2}{n_1 - 1} + \dfrac{(s_2^2/n_2)^2}{n_2 - 1}}.$$

The odd-looking df formula (the Welch–Satterthwaite approximation) picks the t distribution whose spread best matches the actual null distribution. It always lies between $\min(n_1, n_2) - 1$ and $n_1 + n_2 - 2$. Here $n_1 = n_2$, so the SE is again $\sqrt{261.1/6 + 115.5/6} = 7.92$ and $t = 3.28$, but $\mathrm{df} = 8.70$, so $p = 0.0099$.

Method	assumes	statistic	two-sided p
Permutation (exact)	exchangeable labels	26.0 ms	0.0108
Pooled t-test	Normal, equal variances	$t = 3.28$, df $= 10$	0.0083
Welch t-test	Normal	$t = 3.28$, df $= 8.70$	0.0099

Three different routes to the null world, three nearly identical answers. The permutation test is a useful sanity check on the formulas, and the formulas explain why the permutation histogram in Figure 9.9 looks like a bell.

Common trap

Use Welch by default. When variances really are equal it loses almost nothing. When they are not, and the group sizes differ, the pooled test can be badly wrong (Section 9.12 shows a case where its false-alarm rate is 30% instead of 5%). Pre-testing for equal variances and then choosing is worse than just using Welch.

9.8 Paired data

A build engineer measures a test suite's runtime (seconds) on 8 different CI machines, once with the old compiler and once with the new one:

machine	1	2	3	4	5	6	7	8
old a_i	52.1	61.4	47.8	70.2	55.6	66.3	49.9	58.7
new b_i	50.8	59.9	47.1	68.1	54.2	65.0	48.3	57.9
$d_i = a_i - b_i$	1.3	1.5	0.7	2.1	1.4	1.3	1.6	0.8

Pretend for a moment these were 16 unrelated runs and apply Welch. The means are 57.75 and 56.41, a gap of 1.34 s. But the machines differ wildly (from 47 s to 70 s), so $s_a = 7.92$, $s_b = 7.69$, SE $= 3.90$, $t = 0.34$, and $p = 0.74$. No evidence of anything.

Now look at the differences. Every single machine got faster, by between 0.7 and 2.1 seconds. Treat the 8 differences as one sample and run a one-sample t-test of $H_0\colon \mu_d = 0$:

$$\bar{d} = 1.3375, \quad s_d = 0.4438, \quad \text{SE} = \frac{0.4438}{\sqrt{8}} = 0.1569, \quad t = \frac{1.3375}{0.1569} = 8.52, \quad \text{df} = 7.$$

$p = 0.00006$. Same data, opposite conclusion. The paired analysis is the correct one, because the design paired each old run with a new run on the same machine.

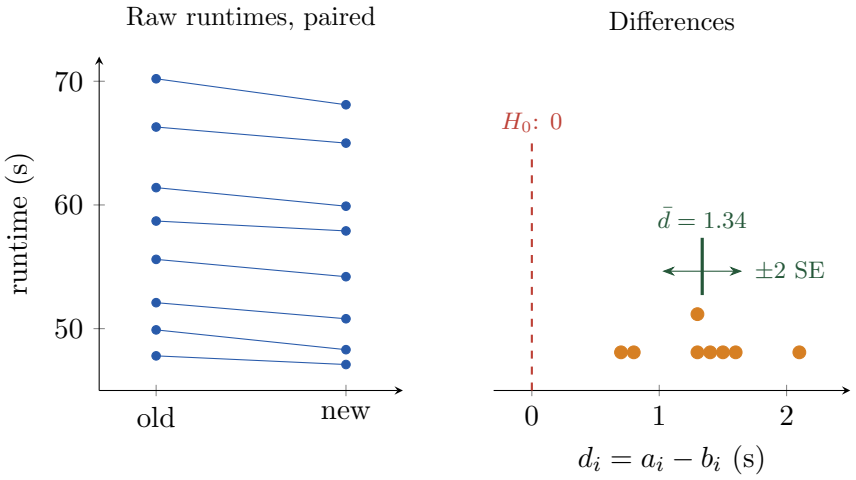


Figure 9.15: Left: each line is one machine. Machine-to-machine differences (the vertical spread) are huge, but every line slopes down. Right: the differences remove the machine effect and cluster tightly around 1.34 s, far from 0.

Why it works

For any two random quantities,

$$\mathrm{Var}(A - B) = \mathrm{Var}(A) + \mathrm{Var}(B) - 2\,\mathrm{Cov}(A, B).$$

An unpaired analysis implicitly sets the covariance to zero. But a slow machine is slow under both compilers, so A and B are strongly positively correlated (here $r = 0.9988$). The covariance term then cancels almost all the variance: $7.92^2 + 7.69^2 - 2(0.9988)(7.92)(7.69) \approx 0.197 = 0.4438^2$. Pairing subtracts the machine effect before it can drown the compiler effect. This is why before/after studies, matched pairs and "same user, both variants" designs are so much more sensitive than comparing two separate groups.

Pause

Pause and think: an A/B test assigns each user to only one variant. Paired or two sample? (Two sample: no user appears in both groups, so there is nothing to pair.)

9.9 Errors, significance and power

A test ends in a decision, and a decision can be wrong in two ways.

	H_0 true (nothing going on)	H_0 false (real effect)
Reject H_0	**Type I error** (false alarm), prob. α	correct, prob. $1 - \beta =$ **power**
Do not reject	correct, prob. $1 - \alpha$	**Type II error** (miss), prob. β

α is the false-alarm rate you chose. β (beta) is the probability of missing a real effect, and **power** $= 1 - \beta$ is the probability of detecting it. Unlike α, power is not a single number: it depends on how big the real effect is.

To compute power you need the picture the null hypothesis alone cannot give: an *alternative world*, as in panel 6 of Figure 9.5. Suppose the coin friend truly is right 75% of the time. Her score is then Binomial$(20, 0.75)$. Overlay that world on the null world and keep the same rejection rule, $X \geq 15$ (Figure 9.16).

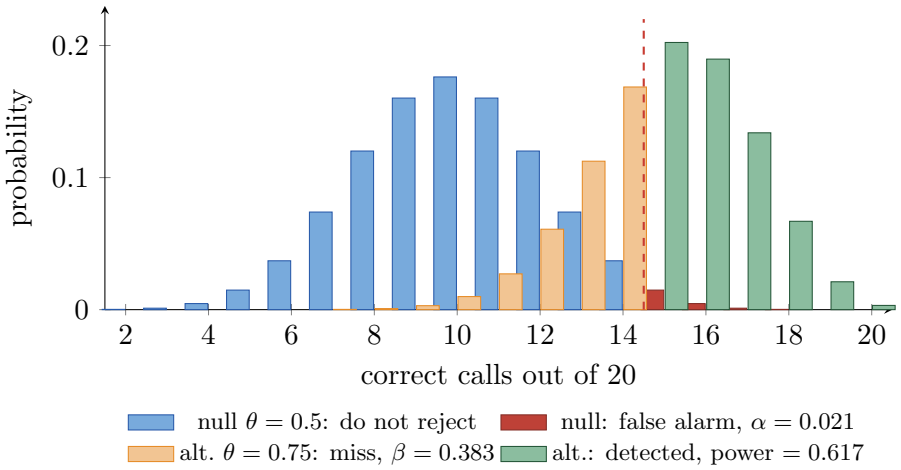


Figure 9.16: Null world (left bars of each pair) and an alternative world where her true hit rate is 0.75 (right bars). The dashed line is the rejection threshold. Red is the false-alarm probability; green is the power. Even a genuinely skilled friend fails this 20-flip test 38% of the time.

9.9.1 The general picture

For a z-type statistic the two worlds are two bells. Under H_0 the statistic is $\mathcal{N}(0, 1)$. Under an alternative with true effect δ it is $\mathcal{N}(\delta/\,\mathrm{SE}, 1)$: the same bell, shifted by the effect measured in standard errors. Figure 9.17 shows a shift of 2.8.

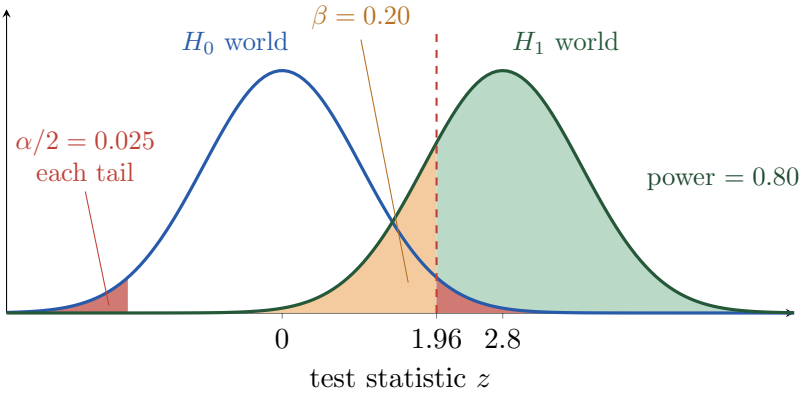


Figure 9.17: Two overlapping sampling distributions. Blue: the statistic if H_0 is true. Green: the statistic if the true effect is 2.8 standard errors. Red tails: α. Orange: β, the part of the H_1 world that falls short of the cut-off. Green fill: power. Moving the cut-off right lowers α but raises β.

Reading the picture gives the power formula. Ignoring the tiny left tail,

$$\text{power} \approx \mathbb{P}\left(Z > z_{\alpha/2} - \frac{\delta}{\mathrm{SE}}\right) = \Phi\left(\frac{\delta}{\mathrm{SE}} - z_{\alpha/2}\right),$$

where $z_{\alpha/2}$ is the cut-off (1.96 for $\alpha = 0.05$). With $\delta/\,\mathrm{SE} = 2.8$: $\Phi(2.8 - 1.96) = \Phi(0.84) = 0.80$. Three levers raise power:

- a larger effect δ (you rarely control this);

- a smaller SE, which means more data (SE $\propto 1/\sqrt{n}$), less noise, or a better design such as pairing;
- a larger α (more false alarms, the trade-off in the figure).

For comparing two groups of n each with common SD σ, SE $= \sigma\sqrt{2/n}$. Write the effect as $d = \delta/\sigma$, the difference in units of SD ("Cohen's d", a scale-free effect size: 0.2 is small, 0.5 medium, 0.8 large; see Chapter 11). Then $\delta/\,\mathrm{SE} = d\sqrt{n/2}$ and

$$\text{power} \approx \Phi\left(d\sqrt{n/2} - 1.96\right).$$

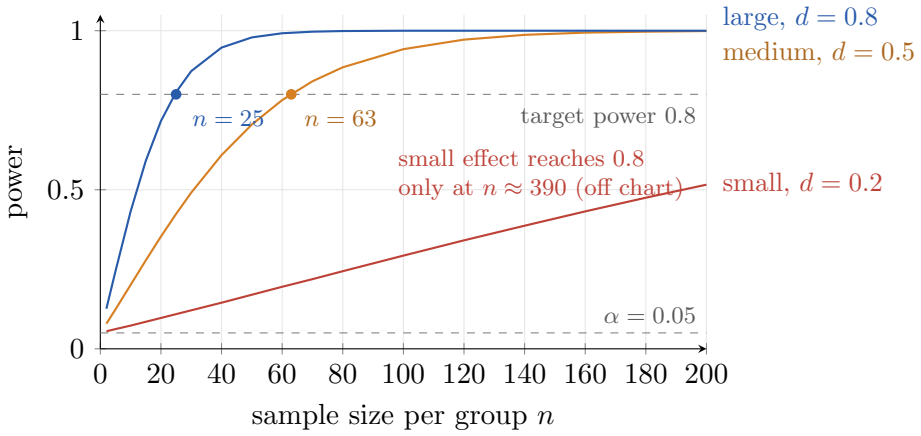


Figure 9.18: Power of a two-sided, two-sample test at $\alpha = 0.05$ as the sample size grows (Normal approximation). The dots mark where each curve reaches 80% power: about 25 per group for a large effect, 63 for a medium one, and about 390 for a small one. As n shrinks towards 0, power falls towards α: you would reject only by chance.

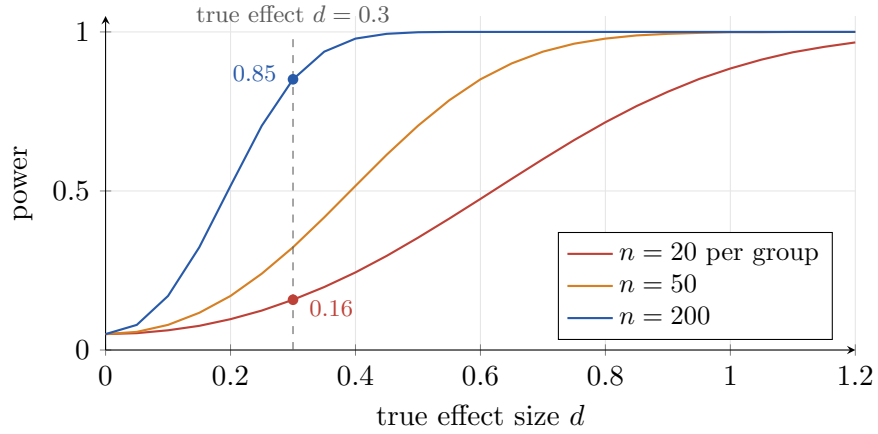


Figure 9.19: Power curves: the probability of rejecting H_0 as a function of the true effect. At $d = 0$ (the null is true) every curve equals $\alpha = 0.05$. Bigger samples give steeper curves. Follow the dashed line at $d = 0.3$: $n = 200$ per group detects it with 85% power, while $n = 20$ detects it only 16% of the time.

9.9.2 Sample size for an A/B test

Power calculations are done *before* the experiment, to decide how much data to collect. Here is the standard calculation.

Example 9.3 (How many users?)**.** A checkout page converts 10% of visitors. A redesign might lift that to 11%. How many users per variant do you need to detect that lift with $\alpha = 0.05$ (two-sided) and 80% power?

Step 1: the SE. With n users per arm, the observed conversion rates $\hat{p}_1, \hat{p}_2$ have variances $p(1-p)/n$ (Chapter 6), so

$$\mathrm{SE}(\hat{p}_2 - \hat{p}_1) = \sqrt{\frac{p_1(1-p_1) + p_2(1-p_2)}{n}}.$$

Step 2: the requirement. From Figure 9.17, power $1-\beta$ requires the alternative bell to sit $z_{\alpha/2} + z_\beta$ standard errors from zero, where z_β is the point with area β above it (0.8416 for $\beta = 0.2$). So $\delta / \mathrm{SE} = z_{\alpha/2} + z_\beta$. Solving for n:

$$n = \frac{(z_{\alpha/2} + z_\beta)^2 \left[p_1(1-p_1) + p_2(1-p_2)\right]}{\delta^2}.$$

Step 3: the numbers. $z_{\alpha/2} + z_\beta = 1.960 + 0.842 = 2.802$, whose square is 7.849. Next, $p_1(1-p_1) + p_2(1-p_2) = 0.0900 + 0.0979 = 0.1879$, and $\delta^2 = 0.01^2 = 0.0001$. So

$$n = \frac{7.849 \times 0.1879}{0.0001} \approx 14{,}748 \text{ users per arm},$$

about 29,500 in total.

Two features of the formula drive every A/B planning conversation. First, $n \propto 1/\delta^2$: detecting a lift half as big (0.5 points) needs about four times the users, 57,761 per arm (a little under four times 14,748, because $p_2(1-p_2)$ also shrinks slightly). Second, the variance term $p(1-p)$ is largest near 50%, so low-conversion metrics are cheaper per user but their absolute lifts are tiny. Figure 9.20 shows how steep the first effect is.

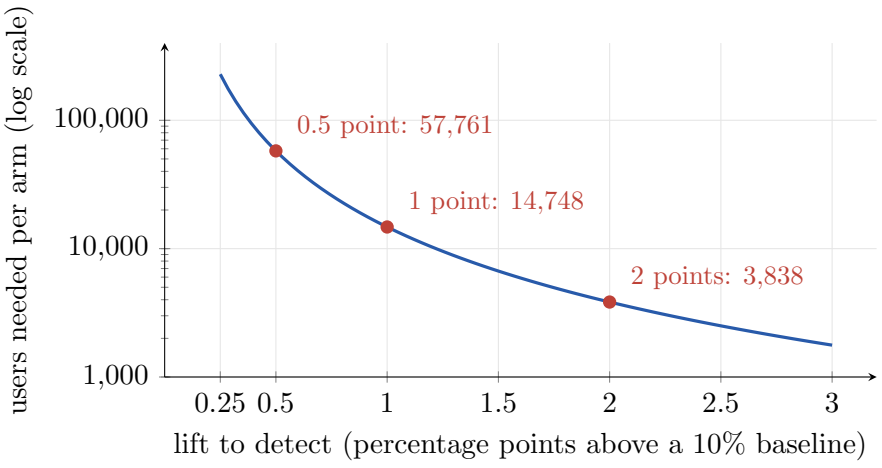


Figure 9.20: Users per arm for $\alpha = 0.05$ (two-sided) and 80% power, baseline conversion 10%. Halving the lift you want to detect roughly quadruples the users needed; the vertical axis is logarithmic, so each grid line is ten times the one below.

```python
from scipy.stats import norm
p1, p2, alpha, power = 0.10, 0.11, 0.05, 0.80
z = norm.ppf(1 - alpha/2) + norm.ppf(power)            # 1.960 + 0.842
n = z**2 * (p1*(1-p1) + p2*(1-p2)) / (p2 - p1)**2
print(round(n))                                        # 14748 per arm
```

Common trap

An underpowered study is not a cheap study; it is a wasted one. With 20% power, four out of five real effects come back "not significant", and the ones that do reach significance tend to overstate the effect (only lucky, inflated estimates clear the bar). Always run the power calculation first.

History

In the 1920s R. A. Fisher, working on crop experiments at Rothamsted in England, needed a way to say whether a fertilizer's apparent benefit could be due to chance variation between field plots. He introduced the p-value as a graded measure of surprise under a null hypothesis, and popularized 0.05 as a convenient bar. His famous "lady tasting tea" example (1935), in which a woman claims she can tell whether milk was poured before the tea, is the first permutation test: count the ways the cups could be labelled by pure guessing. In 1933 Jerzy Neyman and Egon Pearson reframed testing as a decision rule. They added the alternative hypothesis, the two error types, and power, which made it possible to design an experiment with a known chance of finding an effect. Modern practice is a blend of the two views, which explains some of its confusions.

9.10 Tests and confidence intervals are two views of one thing

Take the response-time data again: $\bar{x} = 129.3$, SE $= 2.548$, df $= 9$. The 95% confidence interval from Chapter 7 is

$$129.3 \pm 2.262 \times 2.548 = 129.3 \pm 5.76 = [123.54,\ 135.06].$$

Now test $H_0 : \mu = \mu_0$ for several values of μ_0 at $\alpha = 0.05$:

μ_0	t	p	decision
120	3.65	0.005	reject
125	1.69	0.126	do not reject
130	−0.27	0.790	do not reject
137	−3.02	0.014	reject

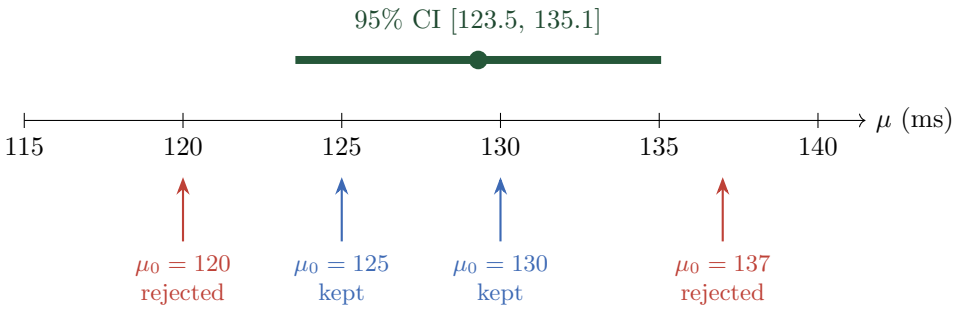


Figure 9.21: Duality between tests and intervals. The green bar is the 95% CI; the arrows are four candidate null values. The two inside it (125 and 130) are kept at $\alpha = 0.05$; the two outside it (120 and 137) are rejected. The CI is exactly the set of null values the test would not reject.

Theorem 9.1 (Duality)**.** *A two-sided test at level α rejects $H_0 \colon \mu = \mu_0$ exactly when μ_0 lies outside the $100(1-\alpha)\%$ confidence interval built from the same statistic.*

Why it works

The test does not reject when $|\bar{x} - \mu_0| / \mathrm{SE} \le t^*$, where t^* is the cut-off. Rearranged, that is $\bar{x} - t^* \,\mathrm{SE} \le \mu_0 \le \bar{x} + t^* \,\mathrm{SE}$, which is the confidence interval. Same inequality, solved for a different variable.

This is why the interval is usually the better report. A p-value answers one question ("is $\mu = 120$ plausible?"); the interval answers it for every μ_0 at once, and also shows how big the effect is and how precisely it is known.

9.11 What a p-value is not

Common trap

A p-value is $\mathbb{P}(\text{data this extreme} \mid H_0)$. It is *not*:

1. **The probability that H_0 is true.** $p = 0.03$ does not mean "3% chance of no effect". That would be $\mathbb{P}(H_0 \mid \text{data})$, a different conditional probability that needs a prior (Chapter 14).
2. **The probability the result is a fluke.** Same mistake in different words.
3. **One minus the probability that H_1 is true.** $p = 0.01$ does not make the alternative 99% likely.
4. **A measure of effect size.** A trivial 0.1 ms speedup gives a tiny p-value with enough data (Figure 9.6, right). Report the effect and its CI.
5. **Evidence for H_0 when large.** $p = 0.6$ means "compatible with H_0", not "H_0 shown" (Section 9.2).
6. **A bright line.** $p = 0.049$ and $p = 0.051$ are essentially the same evidence.
7. **The probability of replicating.** A study with $p = 0.05$ has only about a 50% chance

that an exact repeat also gives $p < 0.05$, if the true effect equals the observed one.

Item 7 surprises most people, and a picture shows why. Suppose the original study landed exactly on the line, $z = 1.96$ ($p = 0.05$), and the true effect really is that size. An exact repeat then produces a z that wobbles around 1.96 with standard deviation 1 (Figure 9.22). Half of that bell lies below the cut-off.

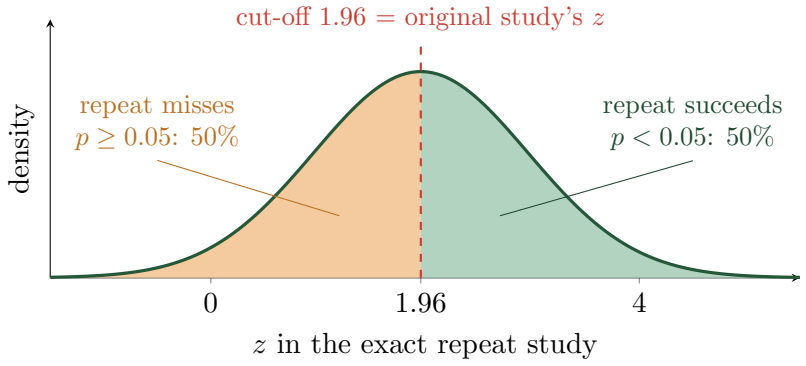


Figure 9.22: Where the z of an exact repeat lands if the original study had $p = 0.05$ and the true effect equals the observed one. The bell is centred on the cut-off, so the repeat is a coin flip: half the time (orange) it comes back "not significant".

The cleanest way to understand what a p-value *is* comes from asking how it behaves when the null is true. Simulate 2000 studies of $n = 20$ Normal observations where $H_0 : \mu = 0$ holds exactly, run a t-test in each, and plot the p-values. Then do the same where the true mean is 0.5σ.

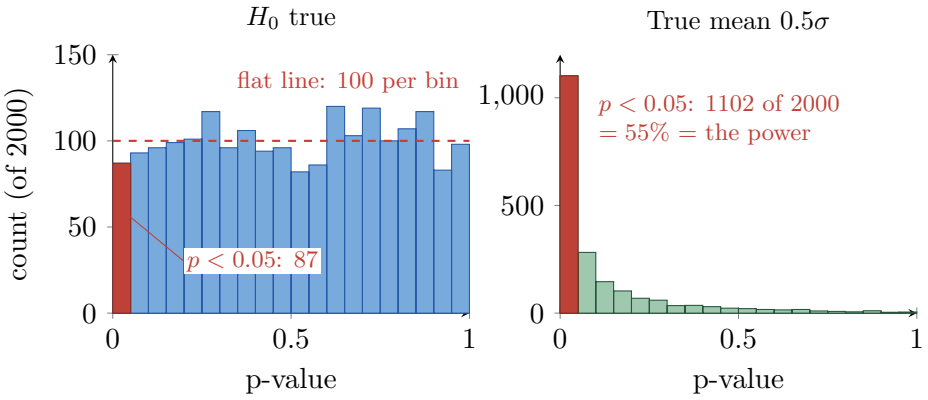


Figure 9.23: Same test, two worlds; the red bar counts p-values below 0.05. Left: when H_0 is true, p-values are spread evenly over $[0, 1]$ (about 100 per bin of width 0.05; dashed line), so in theory exactly the fraction α fall below α; here 87 of 2000 did. Right: when there is a real effect, p-values pile up near 0; the fraction below 0.05 (55% here) is the power.

Why it works

Take a continuous statistic T with null CDF F, and a right-tailed p-value $p = 1 - F(T)$. Under H_0, $F(T)$ is uniform on $[0, 1]$ (the probability integral transform: a variable plugged into its own CDF is uniform). So for any u,

$$\mathbb{P}_{H_0}(p \le u) = \mathbb{P}\big(F(T) \ge 1 - u\big) = u.$$

This is what makes α meaningful: "reject when $p \le 0.05$" raises false alarms in exactly 5% of true-null studies. It also shows why p-values from true nulls are pure noise. A null study is as likely to give $p = 0.04$ as $p = 0.94$.

This uniformity is the root of the multiple-testing problem in Chapter 11: run 20 tests of true nulls and you expect one $p < 0.05$ by construction.

9.12 Assumptions and robustness of the t-test

Every t-test assumes:

1. **Independence.** Observations do not influence each other (within and, for two-sample tests, between groups). This is the assumption that matters most and cannot be fixed by a bigger sample. Repeated measurements of the same user, or requests sharing a cache, violate it and make SE too small.

2. **Approximate Normality of the mean.** Exact for Normal data; otherwise supplied by the CLT once n is large enough. How large depends on skewness.

3. **Equal variances**, for the pooled test only. Welch drops this.

How robust is the test to non-Normal data? Simulate it. Draw 40,000 samples from a distribution whose true mean equals μ_0, run a two-sided one-sample t-test at $\alpha = 0.05$, and record how often it falsely rejects. A perfect test rejects 5% of the time.

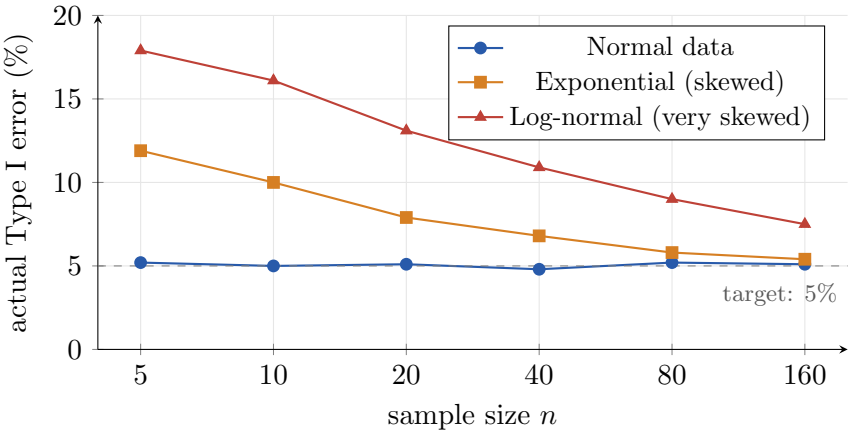


Figure 9.24: False-alarm rate of a nominal 5% one-sample t-test, from simulation (the dashed line is what it should be). For Normal data it is right at every n. For skewed data it is too high at small n and slowly converges to 5% as the CLT takes hold. Strong skew (latency-like data) needs well over 100 observations.

The lesson for latency data, which are typically right-skewed with a long tail: a t-test on 10 requests can be misleading, while a t-test on thousands is fine for the *mean*. If the median or p99 is the real question, use the bootstrap (Chapter 8) or a permutation test on that statistic instead.

Unequal variances combined with unequal group sizes are a sharper danger for the pooled test. Again by simulation (H_0 true, 40,000 repetitions, $\alpha = 0.05$):

group sizes	SDs	pooled rejects	Welch rejects
25, 25	1, 1	5.0%	5.0%
25, 25	4, 1	5.6%	5.2%
10, 40	4, 1	29.8%	5.1%
40, 10	4, 1	0.1%	5.1%

When the small group has the large variance, pooling underestimates the SE (the big, quiet group dominates s_p) and the test rejects six times too often. When the large group has the large variance, the test becomes uselessly conservative. Welch stays at 5% throughout.

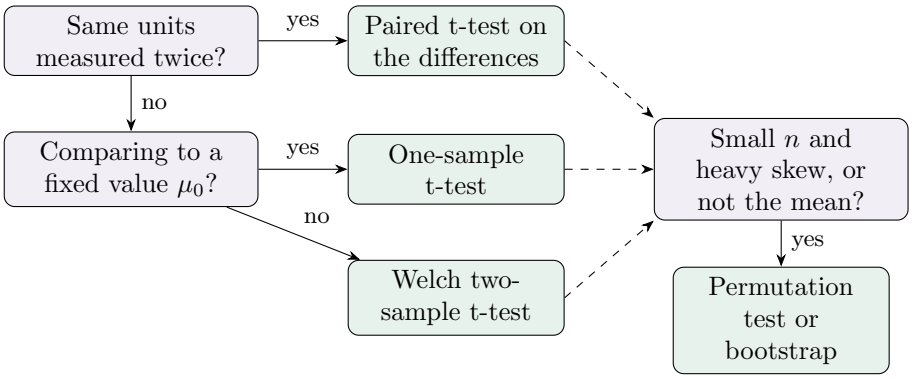


Figure 9.25: Choosing a test for a mean. Start top left. Whichever t-test you land on, check the robustness question on the right; if the answer is yes, switch to a resampling method.

```python
from scipy import stats
stats.ttest_1samp(x, 120)                    # t=3.65, p=0.0053
stats.ttest_ind(old, new)                    # pooled: t=3.28, p=0.0083
stats.ttest_ind(old, new, equal_var=False)   # Welch:  t=3.28, p=0.0099
stats.ttest_rel(a, b)                        # paired: t=8.52, p=0.00006
```

Connection

Comparing two ML models on the same test set is a paired problem: both models see the same examples, so compare per-example losses (or per-fold scores) with a paired test or a paired bootstrap. Treating the two sets of scores as independent throws away the pairing and can hide a real improvement, exactly as in Section 9.8.

Chapter 10 extends these ideas from means to proportions, several groups and categorical tables; Chapter 11 deals with effect sizes and what happens when many tests are run at once.

9.13 Summary

Idea	In one line
Null hypothesis	The "nothing is going on" world; chosen because it is specific enough to compute a distribution.
Null distribution	What the test statistic looks like in that world (by simulation, shuffling, or formula).
p-value	Fraction of the null world at least as extreme as the observed result.
α	False-alarm rate fixed in advance; reject when $p \le \alpha$.
Fail to reject	Absence of evidence, not evidence of absence.
Permutation test	Shuffle labels to build the null world; needs only randomization.
z-test / t-test	$(\text{estimate} - \text{null value})/\,\text{SE}$, compared to $\mathcal{N}(0,1)$ or t_{df}.
Two-sample	Welch by default: $\text{SE} = \sqrt{s_1^2/n_1 + s_2^2/n_2}$.
Paired	One-sample test on differences; removes between-unit variance.
Power	$1 - \beta \approx \Phi(\delta/\,\text{SE} - z_{\alpha/2})$; grows with effect size, n, α.
A/B sample size	$n = (z_{\alpha/2} + z_\beta)^2[p_1q_1 + p_2q_2]/\delta^2$ per arm.
Duality	Reject μ_0 at level $\alpha \iff \mu_0$ outside the $(1-\alpha)$ CI.
Under H_0	p-values are uniform on $[0,1]$.
Robustness	Independence is essential; skew needs large n; unequal variances need Welch.

9.14 Exercises

Exercise 9.1. IQ-style scores have $\sigma = 15$. A sample of $n = 36$ has $\bar{x} = 104.5$. Test $H_0\colon \mu = 100$ two-sided and one-sided ($H_1\colon \mu > 100$). What do you conclude at $\alpha = 0.05$ in each case?

Exercise 9.2. Group A $= \{3, 5, 7\}$, group B $= \{1, 2, 4\}$. List the $\binom{6}{3}$ relabellings and find the exact one-sided permutation p-value for $\bar{a} - \bar{b}$. Why can this test never give $p < 0.05$?

Exercise 9.3. Five users rate an app before and after a redesign. The differences (after − before) are $2, 4, 1, 3, 5$. Run a paired t-test of $H_0\colon \mu_d = 0$ at $\alpha = 0.05$.

Exercise 9.4. You run 20 independent tests at $\alpha = 0.05$ and every null is true. What is the probability of at least one "significant" result?

Exercise 9.5. In the A/B example (10% → 11%, 14,748 per arm), by what factor does n change if (a) you want 90% power instead of 80%; (b) you want to detect a lift of 2 points instead of 1 (ignore the small change in $p(1-p)$)?

Exercise 9.6. A 95% CI for a mean difference is $(2.1, 5.3)$. At $\alpha = 0.05$, do you reject $H_0\colon \delta = 0$? $H_0\colon \delta = 5$? Can you tell from this interval alone whether $H_0\colon \delta = 0$ is rejected at $\alpha = 0.01$?

Exercise 9.7. A report says: "$p = 0.02$, so there is a 98% chance the new feature increases engagement." Rewrite the sentence correctly.

Exercise 9.8. In the robustness table, the pooled test rejects 29.8% of the time with group sizes $10, 40$ and SDs $4, 1$. Compute s_p and the pooled SE using the true SDs, compare with the Welch SE, and explain the inflation.

Answers

1. SE $= 15/6 = 2.5$, $z = 1.8$. Two-sided $p = 0.072$: do not reject. One-sided $p = 0.036$: reject. This is why sides must be fixed in advance.

2. Over the 20 labellings, the sum of group A takes the values 6, 7, 8, 8, 9, 9, 10, 10, 10, 11, 11, 12, 12, 12, 13, 13, 14, 14, 15, 16. The observed sum is 15; sums ≥ 15 occur 2 times, so $p = 2/20 = 0.10$. The smallest possible p-value is $1/20 = 0.05$, reached only by the single most extreme labelling.

3. $\bar{d} = 3$, $s_d = \sqrt{2.5} = 1.581$, SE $= 0.707$, $t = 4.24$ on 4 df. The cut-off is 2.776, and $p = 0.013$: reject.

4. $1 - 0.95^{20} = 0.64$.

5. (a) $(1.960 + 1.282)^2/(1.960 + 0.842)^2 = 10.51/7.85 \approx 1.34$, so about 34% more users. (b) $n \propto 1/\delta^2$, so a quarter as many: about 3,700 per arm.

6. Reject $\delta = 0$ (0 is outside). Do not reject $\delta = 5$ (inside). For $\alpha = 0.01$ you would need the wider 99% interval, which this one does not give; you cannot tell for sure.

7. "If the feature had no effect, data showing an increase this large or larger would occur about 2% of the time." The probability that the feature works requires a prior and Bayes' rule (Chapter 14).

8. $s_p^2 = (9 \cdot 16 + 39 \cdot 1)/48 = 3.81$, $s_p = 1.95$, pooled SE $= 1.95\sqrt{1/10 + 1/40} = 0.690$. Welch SE $= \sqrt{16/10 + 1/40} = 1.275$. The pooled SE is about half the true one, so $|t|$ is about twice as large as it should be and the test rejects far too often.

Chapter 10

Comparing Groups and Categorical Data

Chapter 9 built the machinery of a test: a null hypothesis, a test statistic, its null distribution, and a p-value. It applied that machinery to means with the z- and t-tests. Real questions come in more shapes. Did the new checkout page convert better? Is this die fair? Does conversion depend on device? Do three server configurations have the same mean latency? What if the data are skewed and $n = 6$? This chapter gives one tool for each shape and ends with a flowchart for choosing among them.

Every test here follows the same recipe:

1. Compute what you would *expect* to see if nothing were going on.
2. Measure how far the data sit from that expectation, in units of noise.
3. Compare the distance to its null distribution to get a p-value.

10.1 Two proportions: the A/B conversion test

10.1.1 A worked example

An online shop shows its old checkout page (control, A) to 10,000 randomly chosen users and a new page (treatment, B) to another 10,000.

	Users	Conversions	Rate
A (control)	10,000	1,000	$\hat{p}_A = 0.100$
B (treatment)	10,000	1,100	$\hat{p}_B = 0.110$

The observed lift is $\hat{p}_B - \hat{p}_A = 0.010$, one percentage point. Is that a real effect, or could two identical pages produce a gap this big by chance?

Each user converts or not, so each arm is a sample of Bernoulli trials. A sample proportion is a sample mean of 0s and 1s, so Chapter 6 gives its standard error: $\mathrm{SE}(\hat{p}) = \sqrt{p(1-p)/n}$. The two

arms are independent, so variances add:

$$\operatorname{Var}(\hat{p}_B - \hat{p}_A) = \frac{p_A(1-p_A)}{n_A} + \frac{p_B(1-p_B)}{n_B}.$$

Under the null hypothesis $H_0\colon p_A = p_B = p$, both arms share one rate. The best estimate of that common rate pools all users:

$$\hat{p} = \frac{1000 + 1100}{10000 + 10000} = \frac{2100}{20000} = 0.105.$$

Plug it into the variance formula. The subscript 0 marks a standard error computed as if H_0 were true:

$$\mathrm{SE}_0 = \sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_A} + \frac{1}{n_B}\right)} = \sqrt{0.105 \times 0.895 \times \frac{2}{10000}} = \sqrt{1.8795 \times 10^{-5}} \approx 0.004335.$$

The test statistic counts how many null standard errors the observed gap is:

$$z = \frac{\hat{p}_B - \hat{p}_A}{\mathrm{SE}_0} = \frac{0.010}{0.004335} \approx 2.31.$$

By the CLT the statistic is close to $\mathcal{N}(0,1)$ under H_0. The two-sided p-value is $2\,\mathbb{P}(Z > 2.31) \approx 0.021$. At $\alpha = 0.05$ we reject H_0.

Pause

Keep the same rates (10% and 11%) but use only 1000 users per arm. Does z grow or shrink, and by how much? (Answer: the SE grows by $\sqrt{10}$, so z falls to about 0.73 and $p \approx 0.47$. Same lift, no evidence.)

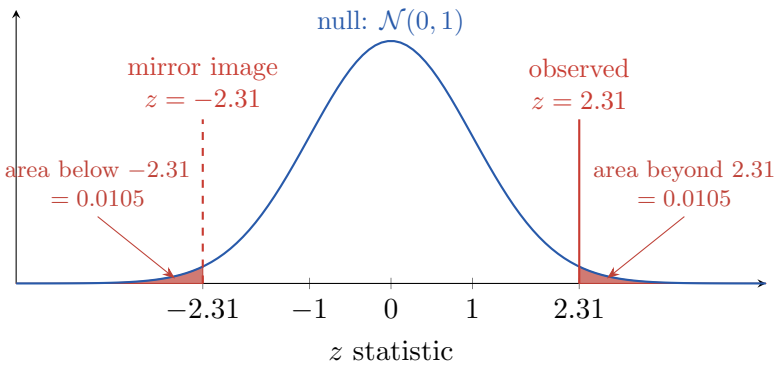


Figure 10.1: Null distribution of the two-proportion z statistic, the curve z would follow if the two pages converted equally. The red slivers are the p-value: the observed $z = 2.31$ leaves 0.0105 in the upper tail, and the equally extreme lower tail adds another 0.0105, so $p = 0.021$. A gap this large would be rare if the pages were identical.

10.1.2 Pooled for the test, unpooled for the interval

The test assumes H_0 is true, so it uses the pooled $\hat{p}$. A confidence interval for the lift makes no such assumption, so it uses each arm's own rate:

$$\mathrm{SE} = \sqrt{\frac{0.10 \times 0.90}{10000} + \frac{0.11 \times 0.89}{10000}} = \sqrt{9.00 \times 10^{-6} + 9.79 \times 10^{-6}} \approx 0.004335.$$

The 95% interval is $0.010 \pm 1.96 \times 0.004335 = [0.0015,\ 0.0185]$. The lift is somewhere between 0.15 and 1.85 percentage points. The interval excludes 0, which agrees with the test. Here the two standard errors match to four digits because the rates are close; they differ when the rates are far apart.

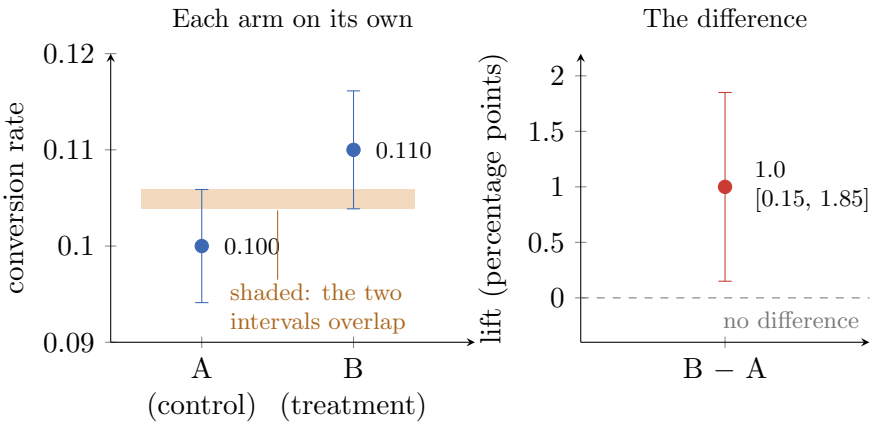


Figure 10.2: Left: each arm's conversion rate with its own 95% interval ($\pm 1.96\sqrt{\hat{p}(1-\hat{p})/n}$); the intervals overlap (shaded band). Right: the 95% interval for the *difference* stays entirely above zero. Overlapping individual intervals do not imply a non-significant difference; judge the difference by its own interval.

Common trap

Do not compare two groups by checking whether their separate confidence intervals overlap. The SE of a difference is $\sqrt{\mathrm{SE}_A^2 + \mathrm{SE}_B^2}$, which is smaller than $\mathrm{SE}_A + \mathrm{SE}_B$. Two intervals can overlap while the interval for the difference excludes zero, as in Figure 10.2. Always build the interval for the difference itself.

Why it works

Why is the z approximation good here? Each arm's count of conversions is a sum of 10,000 independent Bernoulli variables, so the CLT applies to each proportion. A common rule of thumb: you want at least about 10 successes and 10 failures in each arm. With 1000 and 9000 we are far past that. With 4 conversions in 40 users you are not, and Fisher's exact test (Section 10.4) is the safer choice.

```
import numpy as np
from scipy import stats
xA, nA, xB, nB = 1000, 10000, 1100, 10000
p = (xA + xB) / (nA + nB)                    # pooled rate 0.105
se0 = np.sqrt(p*(1-p)*(1/nA + 1/nB))
z = (xB/nB - xA/nA) / se0                    # 2.31
print(z, 2*stats.norm.sf(abs(z)))            # p = 0.021
```

10.2 Chi-square goodness of fit

10.2.1 Is this die fair?

You roll a die 120 times. A fair die should give each face $120/6 = 20$ times on average. You observe:

Face	1	2	3	4	5	6	Total
Observed O	15	22	18	25	16	24	120
Expected E	20	20	20	20	20	20	120
$O - E$	-5	2	-2	5	-4	4	0
$(O-E)^2/E$	1.25	0.20	0.20	1.25	0.80	0.80	4.50

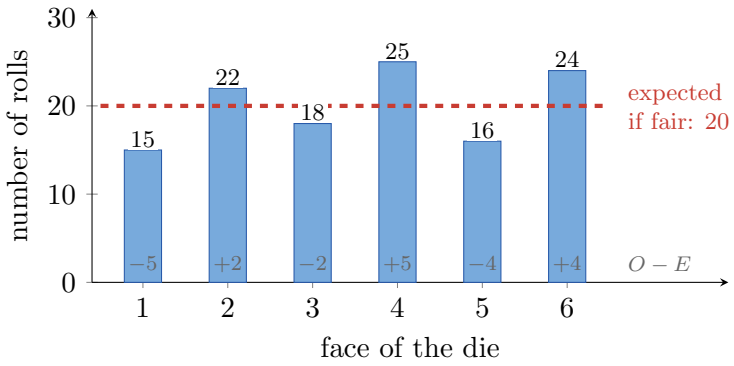


Figure 10.3: Observed die counts (numbers on bars) against the fair expectation of 20 per face (dashed). The grey numbers inside the bars are the misses $O - E$: no face is off by more than 5. The chi-square statistic decides whether misses of this size are more than chance.

Definition 10.1 (Pearson chi-square statistic)**.** For k categories with observed counts O_i and expected counts E_i under H_0,

$$X^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i}.$$

For a goodness-of-fit test with fully specified probabilities, X^2 is approximately χ^2_{k-1} under H_0.

The subscript $k - 1$ is the *degrees of freedom*: the number of counts that are free to vary once the total is fixed. The chi-square distribution χ^2_m is the distribution of a sum of m squared independent standard normal variables, so it describes "total squared surprise" from m free pieces.

For the die, $X^2 = 4.50$ with $k - 1 = 5$ degrees of freedom. The 95th percentile of χ^2_5 is 11.07, and $\mathbb{P}(\chi^2_5 > 4.5) \approx 0.48$. We have no evidence against fairness. Wobbles of this size happen about half the time with a fair die.

Why it works

Why divide by E_i? A count's natural wobble grows with its size: a cell expecting 1000 can easily miss by 30, a cell expecting 10 cannot. Each count O_i has variance $E_i(1 - p_i)$, close to

E_i, so $(O_i - E_i)/\sqrt{E_i}$ is roughly a standardised deviation and X^2 adds up squared standardised deviations. A sum of m independent squared standard normals is the definition of χ^2_m. Why $k-1$ and not k? The counts must add to n, so once five deviations are known the sixth is fixed: $\sum(O_i - E_i) = 0$. The deviations are not independent; that one constraint (together with the $1 - p_i$ factors left out of the denominators) removes exactly one degree of freedom. (See Probability From the Ground Up for the chi-square distribution itself.)

10.2.2 The chi-square family

The chi-square distribution with m degrees of freedom has mean m and variance $2m$. Its shape depends only on m. Small m gives a right-skewed curve piled near zero; large m looks more and more normal.

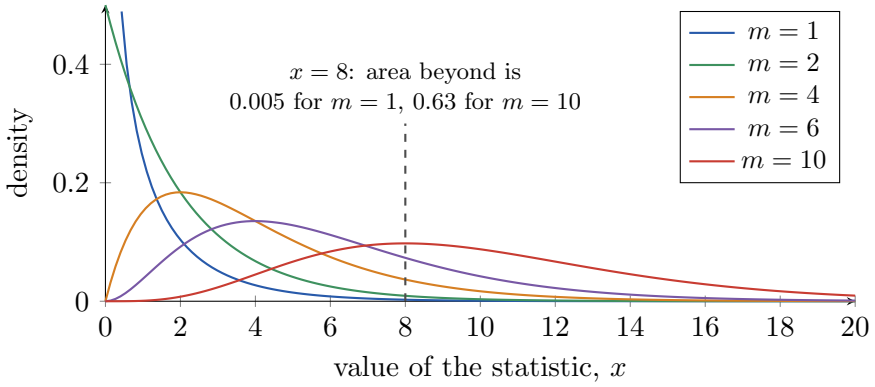


Figure 10.4: Chi-square densities for several degrees of freedom m. Each has mean m (and, for $m \geq 2$, its peak at $m-2$), so the curves slide right as m grows. The dashed line shows the consequence: the same $X^2 = 8$ is far in the tail for $m = 1$ and ordinary for $m = 10$, so always read X^2 against its own degrees of freedom.

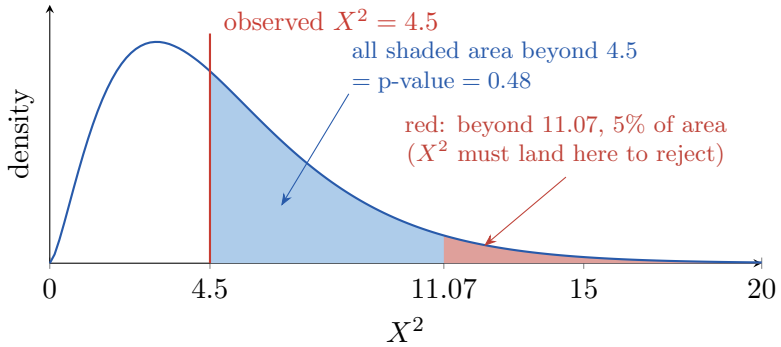


Figure 10.5: The χ^2_5 null distribution for the die. The shaded area to the right of the observed 4.5 (blue plus red) is the p-value, 0.48. To reject at 5% the statistic would have to land in the red tail beyond 11.07; ours is nowhere near it.

Common trap

Chi-square tests work on *counts*, never on percentages or rates. If you feed in $12.5\%, 18.3\%, \ldots$ instead of the counts, X^2 changes with the units and the p-value is meaningless. Also check that every expected count is at least about 5. If not, merge sparse categories or use an exact test.

History

Karl Pearson, working in London in 1900, wanted to test whether data fitted a proposed curve: did a set of dice throws, or measured traits, match the frequencies a theory predicted? Earlier methods judged fit by eye. Pearson introduced $X^2 = \sum(O - E)^2/E$ and worked out its large-sample distribution, the chi-square distribution. He got the degrees of freedom wrong for tables whose expected counts are estimated from the data; Ronald Fisher corrected this in 1922, giving the $(r - 1)(c - 1)$ rule. Fisher went on, at the Rothamsted agricultural station in the 1920s, to compare crop yields across many fertiliser treatments at once. He split total variation into between-plot and within-plot pieces, which became the analysis of variance; the F distribution is named in his honour. Together these gave the first general tools for comparing more than two groups.

10.3 Chi-square test of independence

10.3.1 Does conversion depend on device?

A site records 1000 visits by device and whether the visit converted.

	Converted	Not converted	Row total
Desktop	60	340	400
Mobile	50	450	500
Tablet	10	90	100
Column total	120	880	1000

Desktop converts at $60/400 = 15\%$; mobile and tablet at 10%. The null hypothesis is that device and conversion are *independent*: every device has the same conversion rate, which must then be the overall $120/1000 = 12\%$.

Expected counts. If independent, desktop should have 12% of its 400 visits converting: $0.12 \times 400 = 48$. In general,

$$E_{ij} = \frac{(\text{row } i \text{ total}) \times (\text{column } j \text{ total})}{\text{grand total}} = \frac{R_i\, C_j}{N},$$

where R_i is the row total, C_j the column total and N the grand total. For desktop and converted: $400 \times 120/1000 = 48$.

Pause

Why must the expected table have the same row and column totals as the observed table? (Because $\sum_j E_{ij} = R_i \sum_j C_j/N = R_i$. Independence only reshuffles counts inside the margins.)

The full expected table and each cell's contribution $(O-E)^2/E$:

	E: Conv.	E: Not	$\frac{(O-E)^2}{E}$: Conv.	$\frac{(O-E)^2}{E}$: Not
Desktop	48	352	$144/48 = 3.000$	$144/352 = 0.409$
Mobile	60	440	$100/60 = 1.667$	$100/440 = 0.227$
Tablet	12	88	$4/12 = 0.333$	$4/88 = 0.045$

Adding the six contributions gives $X^2 \approx 5.68$.

Degrees of freedom. For an $r \times c$ table, $\text{df} = (r-1)(c-1)$. Here $(3-1)(2-1) = 2$. The reason: once the row and column totals are fixed, you can fill only $(r-1)(c-1)$ cells freely; the rest are forced by the totals.

Conclusion. The 95th percentile of χ^2_2 is 5.99, and $\mathbb{P}(\chi^2_2 > 5.68) \approx 0.058$. The data hint that desktop converts better, but at $\alpha = 0.05$ the evidence falls just short. The cell contributions show where the signal is: desktop-converted contributes 3.0 of the 5.68.

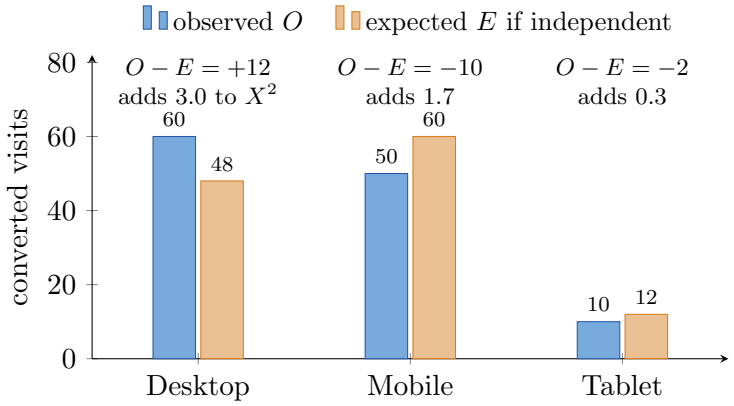


Figure 10.6: Observed versus expected conversions per device. Desktop has 12 more conversions than independence predicts and mobile has 10 fewer. Desktop's cell contributes most to X^2: its gap is the largest and its expected count (48) is smaller than mobile's (60), so $12^2/48 = 3.0$ beats $10^2/60 = 1.7$. (The "not converted" cells add the remaining 0.68.)

Connection

A 2×2 table is the A/B test in disguise. Put the checkout data in a table (rows A and B; columns converted, not converted) and run the chi-square test without continuity correction: $X^2 = 5.32$. That equals $z^2 = 2.3066^2 = 5.32$, and the p-values are identical (0.021). The two-proportion z-test and the 2×2 chi-square test are the same test. The z form has one advantage: it tells you the *direction* and gives a confidence interval for the lift.

Common trap

Independence tests detect association, not causation. If desktop users are mostly office workers buying on lunch break, device may be a proxy for intent. Only randomisation (Chapter 5) licenses a causal reading. The checkout A/B test is causal; the device table is not.

10.4 Fisher's exact test

When counts are small, the chi-square approximation breaks down. Fisher's exact test computes the p-value directly from the hypergeometric distribution, with no large-sample approximation.

A pilot gives a new onboarding flow to 10 users and the old one to 10. Results:

	Retained	Churned	Total
New flow	7	3	10
Old flow	2	8	10
Total	9	11	20

The idea. Fix the margins: 9 retained users among 20, 10 of whom got the new flow. If the flow has no effect, which 10 users received it is a random draw, and the number of retained users in the new-flow group X follows a hypergeometric distribution: draw 10 from an urn of 20 containing 9 "retained" balls. Then

$$\mathbb{P}(X = k) = \frac{\binom{9}{k}\binom{11}{10-k}}{\binom{20}{10}}.$$

Observing $X = 7$ or more has probability $0.0322 + 0.0027 + 0.0001 = 0.035$. The distribution is symmetric here, so the two-sided p-value is about 0.070.

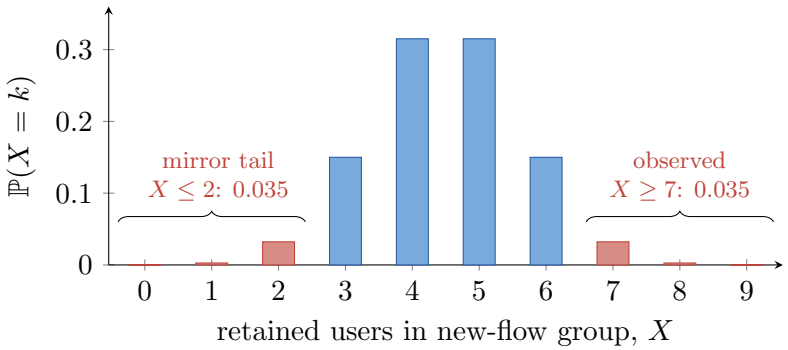


Figure 10.7: Null distribution of X, the retained count in the new-flow group, when the flow has no effect (margins 9/11 and 10/10). The red bars are outcomes at least as extreme as the observed 7: $X \ge 7$ has total probability 0.035, and the mirror-image tail $X \le 2$ adds another 0.035, giving a two-sided p-value of 0.070. (The bars at 0, 1, 8 and 9 are there but tiny: 0.0001 and 0.0027.)

The chi-square test on the same table gives $X^2 = 5.05$ and $p = 0.025$. That is too optimistic: two of the expected counts are 4.5, below the rule of thumb. With small counts, trust Fisher. With large counts the two agree and the chi-square test is faster.

10.5 One-way ANOVA: comparing several means

10.5.1 Three server configurations

You measure p50 latency (ms) on 5 runs each for three server configurations:

Config	Runs					Mean
A	12	14	11	13	15	13
B	16	18	15	17	19	17
C	13	15	14	12	16	14

The grand mean of all 15 runs is $220/15 \approx 14.667$. Are the three population means equal? You could run three pairwise t-tests, but each has a 5% false alarm rate, so the chance of at least one false alarm is about $1 - 0.95^3 \approx 14\%$ (Chapter 11 treats this properly). ANOVA asks one question: H_0: $\mu_A = \mu_B = \mu_C$. ANOVA stands for *analysis of variance*: it decides whether means differ by comparing two kinds of spread, the spread of the group means and the spread inside groups.

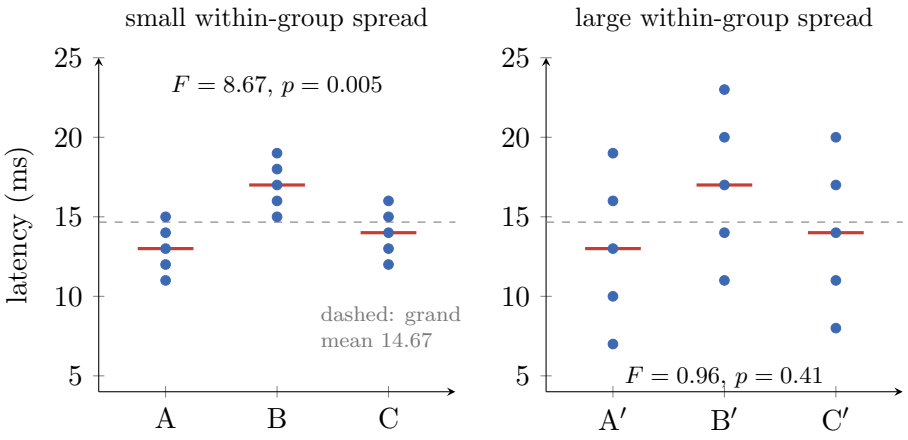


Figure 10.8: Five runs per configuration (blue dots); red bars mark each group's mean (13, 17, 14) and the dashed line the grand mean, identical in both panels. On the left the groups barely overlap, so the gaps between means stand out ($F = 8.67$). On the right each point is three times farther from its group mean, and the same gaps are lost in the noise ($F = 0.96$). ANOVA's F measures exactly this ratio of between-group to within-group spread.

10.5.2 Splitting the variation

Take any single observation x_{ij} (run j in group i). Its distance from the grand mean $\bar{x}$ splits into two pieces:

$$\underbrace{x_{ij} - \bar{x}}_{\text{total}} = \underbrace{(\bar{x}_i - \bar{x})}_{\text{between: group mean vs grand mean}} + \underbrace{(x_{ij} - \bar{x}_i)}_{\text{within: point vs its group mean}} .$$

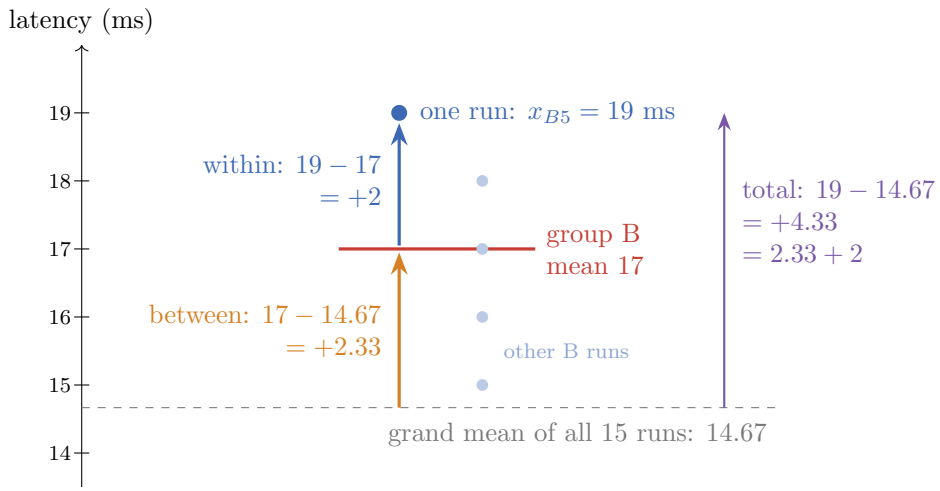


Figure 10.9: One run from group B, split into two pieces. Its total distance above the grand mean (purple, +4.33) is the between-group piece (orange, how far group B's mean sits above the grand mean) plus the within-group piece (blue, how far the run sits above its own group mean). ANOVA does this for every run.

Square both sides and sum over all observations. The cross term vanishes, because within each group the deviations $x_{ij} - \bar{x}_i$ sum to zero. What remains is the ANOVA identity:

$$\underbrace{\sum_{i,j}(x_{ij} - \bar{x})^2}_{\text{SST}} = \underbrace{\sum_{i} n_i(\bar{x}_i - \bar{x})^2}_{\text{SSB}} + \underbrace{\sum_{i,j}(x_{ij} - \bar{x}_i)^2}_{\text{SSW}}.$$

For the latency data:

$$\text{SSB} = 5\left[(13 - 14.667)^2 + (17 - 14.667)^2 + (14 - 14.667)^2\right] = 5(2.778 + 5.444 + 0.444) = 43.33,$$

$$\text{SSW} = \underbrace{(1+1+4+0+4)}_{A} + \underbrace{(1+1+4+0+4)}_{B} + \underbrace{(1+1+0+4+4)}_{C} = 30,$$

$$\text{SST} = 43.33 + 30 = 73.33.$$

10.5.3 The F statistic

Sums of squares grow with the number of terms, so divide each by its degrees of freedom to get a *mean square*. With k groups and N observations in total:

$$\text{MSB} = \frac{\text{SSB}}{k-1}, \qquad \text{MSW} = \frac{\text{SSW}}{N-k}, \qquad F = \frac{\text{MSB}}{\text{MSW}}.$$

Here MSB $= 43.33/2 = 21.67$ and MSW $= 30/12 = 2.5$, so $F = 21.67/2.5 = 8.67$.

Source	SS	df	MS	F
Between groups	43.33	2	21.67	8.67
Within groups	30.00	12	2.50	
Total	73.33	14		

Why it works

Why is this ratio the right statistic? MSW is a pooled sample variance: it estimates the noise variance σ^2 whether or not H_0 holds, since it only looks inside groups. MSB estimates σ^2 too *if* H_0 is true: the group means are then sample means of size n from one population, so they vary with variance σ^2/n, and multiplying their spread by n recovers σ^2. If the true means differ, MSB picks up that extra spread and grows. So $F \approx 1$ under H_0 and $F \gg 1$ when means differ. The degrees of freedom follow the usual rule: k group means minus 1 for the grand mean, and N observations minus k estimated group means.

10.5.4 The F distribution

If the groups are independent, normal, and share one variance, then under H_0 the statistic follows an F distribution with $(k-1, N-k)$ degrees of freedom. Formally, F_{d_1,d_2} is the distribution of $(U_1/d_1)/(U_2/d_2)$ for independent $U_1 \sim \chi^2_{d_1}$ and $U_2 \sim \chi^2_{d_2}$: a ratio of two variance estimates.

For our data, $F_{2,12}$ has 95th percentile 3.89, and $\mathbb{P}(F_{2,12} > 8.67) \approx 0.0047$. We reject equal means.

Pause

What value of F do you expect if the null hypothesis is true? Could F ever be below 1? (About 1, since both mean squares estimate σ^2. Yes: if group means happen to be closer together than noise alone predicts, MSB $<$ MSW.)

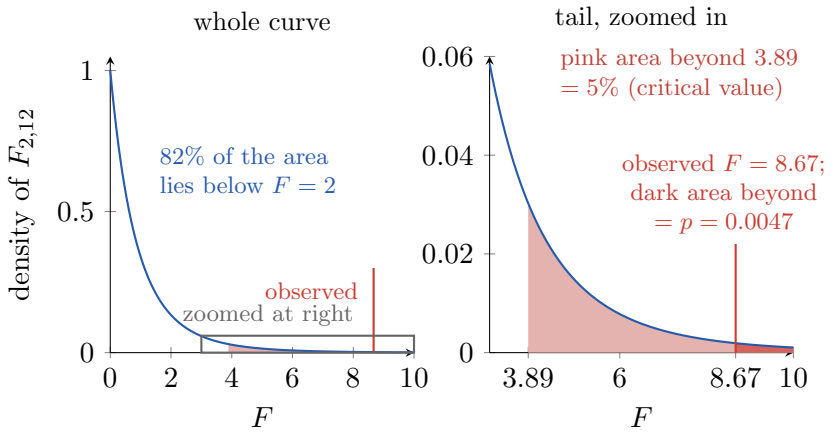


Figure 10.10: The $F_{2,12}$ null distribution for the server data. Left: the whole curve is right-skewed and piled up at small values (its mean is $12/10 = 1.2$), since under H_0 both mean squares estimate the same σ^2. Right: the boxed tail, enlarged. Only 5% of the area lies beyond 3.89; the observed 8.67 is far beyond that, with just 0.0047 of the area to its right.

10.5.5 ANOVA with two groups is the t-test

Run ANOVA on groups A and C only. The means are 13 and 14; each group has within sum of squares 10. Then SSB $= 5(0.5^2) + 5(0.5^2) = 2.5$ with 1 df, and MSW $= 20/8 = 2.5$, so $F = 1$. The

pooled two-sample t statistic (Chapter 9) is

$$t = \frac{14 - 13}{\sqrt{2.5\,(1/5 + 1/5)}} = \frac{1}{1} = 1.$$

And $t^2 = 1 = F$. For A versus B, $t = 4$ and $F = 16$, both with $p = 0.0039$. This holds in general: with $k = 2$, $F = t^2$ and $F_{1,d}$ is the distribution of the square of a t_d variable. One-way ANOVA is the pooled t-test extended to many groups.

Common trap

ANOVA assumes equal variances across groups. If one group is much noisier (a common situation with latencies), use Welch's ANOVA, the many-group cousin of Welch's t-test, or a rank-based test (Section 10.6). Moderate non-normality is fine with a few dozen points per group, by the CLT acting on the group means.

10.5.6 Post-hoc comparisons

A significant F says "not all means are equal". It does not say which ones differ. *Post-hoc* tests (Latin for "after this": tests run after the overall F test) compare pairs while keeping the overall false alarm rate at α.

- **Tukey's HSD** ("honestly significant difference") compares every pair, using a critical value from the distribution of the *largest* gap among all the group means, so the number of pairs is built in. For the latency data: B vs A, difference 4, $p = 0.005$; B vs C, difference 3, $p = 0.028$; C vs A, difference 1, $p = 0.59$. Config B is slower than both others; A and C are indistinguishable.
- **Bonferroni** runs ordinary pairwise tests at level α/m for m comparisons. It is simple and conservative.

Chapter 11 explains multiple-comparison corrections in depth.

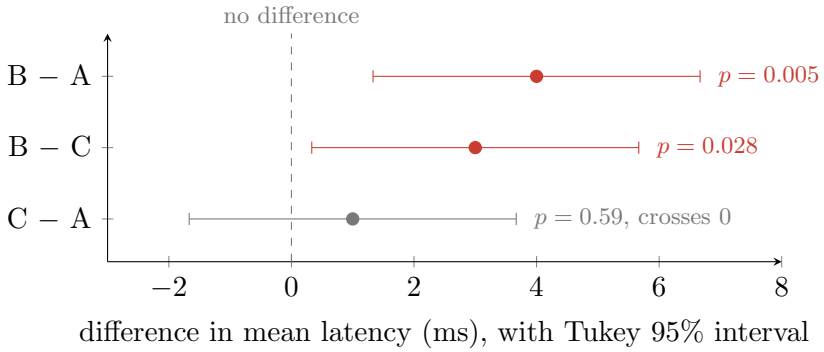


Figure 10.11: Tukey's pairwise comparisons for the three server configurations. Each dot is a difference of group means and each bar its 95% interval, widened (±2.67 ms) so that all three hold at once. The two intervals involving B (red) stay right of zero: B is slower than A and C. The C − A interval (grey) straddles zero, so A and C cannot be told apart.

```
from scipy import stats
A = [12,14,11,13,15]; B = [16,18,15,17,19]; C = [13,15,14,12,16]
print(stats.f_oneway(A, B, C))      # F = 8.67, p = 0.0047
print(stats.tukey_hsd(A, B, C))     # pairwise CIs and p-values
print(stats.kruskal(A, B, C))       # rank-based alternative
```

10.6 Nonparametric alternatives: tests on ranks

The t-test and ANOVA compare means and lean on normality for small samples. Latencies, incomes and session lengths are skewed and have outliers. Rank tests replace each value by its position in the sorted data. A 10-second outlier becomes "rank 11", no more influential than a value just above the rest.

10.6.1 Mann–Whitney / Wilcoxon rank-sum test

Page load times (ms) for two CDNs:

$$\text{A: } 12,\ 15,\ 19,\ 22,\ 30 \qquad \text{B: } 18,\ 25,\ 28,\ 35,\ 40,\ 41.$$

Pool the 11 values, sort, and rank them 1 to 11:

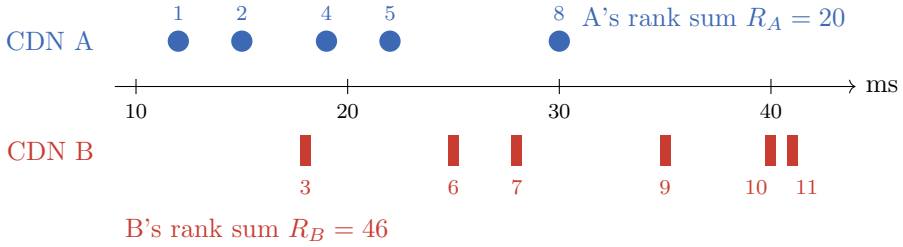


Figure 10.12: Each load time placed on one axis (CDN A above, B below); the small number next to each mark is its rank among all 11 values, 1 = fastest. A's ranks $1, 2, 4, 5, 8$ cluster at the fast end, so A tends to be faster. The rank-sum test asks whether such clustering is surprising if both CDNs had the same distribution.

A's rank sum is $R_A = 1 + 2 + 4 + 5 + 8 = 20$. The Mann–Whitney statistic subtracts the smallest possible rank sum, $n_A(n_A + 1)/2 = 15$:

$$U_A = R_A - \frac{n_A(n_A + 1)}{2} = 20 - 15 = 5.$$

U_A has a direct meaning: it counts the pairs (one from A, one from B) in which the A value is larger. Check: 19 and 22 each beat 18; 30 beats 18, 25, 28. That is $1 + 1 + 3 = 5$ of the $5 \times 6 = 30$ pairs. So an A load beats a B load only $5/30 \approx 17\%$ of the time.

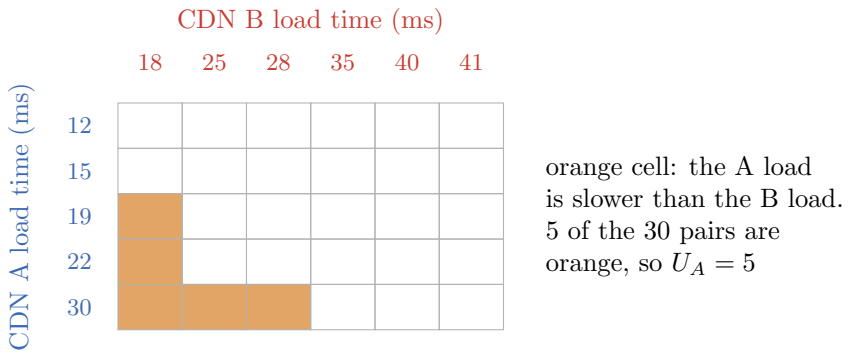


Figure 10.13: Every pairing of one A load with one B load. A cell is orange when the A value is the larger (slower) one. Only 5 of the 30 cells are orange, all in the bottom-left corner: this count is exactly U_A, and $5/30 \approx 0.17$ estimates $\mathbb{P}(X_A > X_B)$. If the CDNs were alike we would expect about half the cells, 15, to be orange.

Under H_0 (same distribution), every assignment of the 11 ranks to the two groups is equally likely. That gives an exact null distribution for U with mean $n_A n_B/2 = 15$. The exact two-sided p-value is 0.082. The normal approximation, with $\mathrm{SD}(U) = \sqrt{n_A n_B (n_A + n_B + 1)/12} = \sqrt{30} = 5.48$, gives $z = (5 - 15)/5.48 = -1.83$ and $p = 0.068$. With $n = 5$ and 6 we lack evidence at 5%, though the direction is suggestive.

Intuition

The Mann–Whitney test estimates $\mathbb{P}(X_A > X_B)$, the chance that a random A value exceeds a random B value. If both groups come from the same distribution this probability is 1/2, and the test asks whether the data are far from that. It is not a test of medians unless the two distributions have the same shape. That probability is often what a product manager actually wants: "how often is the new page faster?"

10.6.2 Wilcoxon signed-rank test

For *paired* data use differences. Eight servers have latency measured before and after a kernel patch:

Server	1	2	3	4	5	6	7	8
Before	120	135	128	150	142	138	160	131
After	112	130	129	138	135	131	149	127
d = Before − After	8	5	−1	12	7	7	11	4
Rank of $\lvert d \rvert$	6	3	1	8	4.5	4.5	7	2

Rank the absolute differences (ties share the average rank, so the two 7s get 4.5 each). Then add the ranks of the positive and negative differences separately:

$$W_+ = 6 + 3 + 8 + 4.5 + 4.5 + 7 + 2 = 35, \qquad W_- = 1.$$

The ranks total $1 + 2 + \cdots + 8 = 36$. Under H_0 (differences symmetric around zero), each rank is equally likely to carry either sign, giving $2^8 = 256$ equally likely sign patterns. Only 2 patterns have

$W_- \le 1$ (all positive, or only rank 1 negative). The two-sided p-value is $2 \times 2/256 = 0.0156$. The patch reduced latency.

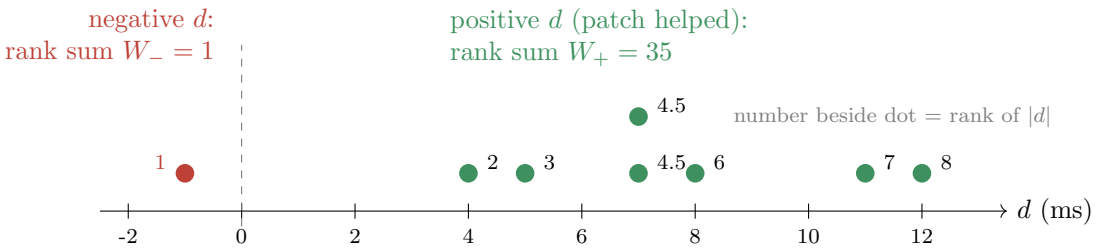


Figure 10.14: The eight paired differences (before minus after) on one axis, with the rank of $|d|$ beside each point (two servers tie at $d = 7$, rank 4.5 each, drawn stacked). Only one difference is negative, and it has the smallest magnitude (rank 1). If the patch did nothing, positive and negative signs would be scattered at random across the ranks.

10.6.3 The sign test

The crudest paired test throws away magnitudes and keeps only signs. If the patch did nothing, each difference is positive with probability 1/2, so the number of positive differences is Binomial$(8, 1/2)$. We saw 7 positives. The two-sided p-value is

$$2\,\mathbb{P}(\mathrm{Bin}(8, \tfrac{1}{2}) \ge 7) = 2 \cdot \frac{\binom{8}{7} + \binom{8}{8}}{256} = 2 \cdot \frac{9}{256} \approx 0.070.$$

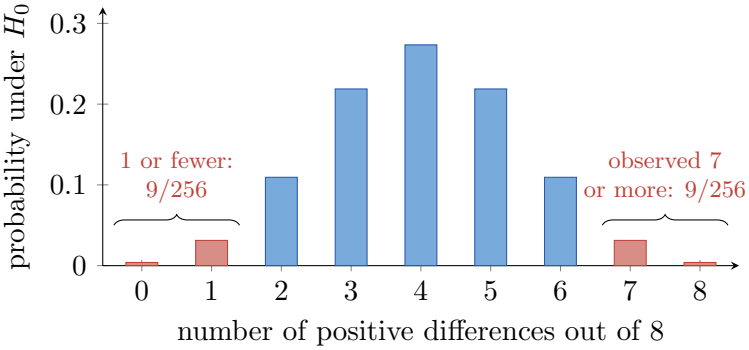


Figure 10.15: The Binomial$(8, 1/2)$ null distribution of the sign test: how many of the 8 differences would be positive if the patch did nothing. Outcomes at least as extreme as the observed 7 positives (red, both tails) have total probability $18/256 \approx 0.070$, not small enough to reject at 5%.

What is really happening

Three tests on the same eight differences: paired t-test $p = 0.0026$; signed-rank $p = 0.016$; sign test $p = 0.070$. Each discards more information than the one before. The t-test uses the values, the signed-rank test uses only their order, and the sign test uses only their signs. Less information means less power. The trade is robustness: an outlier of +500 ms would wreck the t-test and leave the other two untouched. Use the most informative test whose assumptions

you believe.

The rank-based cousin of one-way ANOVA is the **Kruskal–Wallis** test. It ranks all N values together and runs the ANOVA idea on the ranks: its statistic H measures how far each group's average rank sits from the overall average rank $(N+1)/2$, and under H_0 it is approximately χ^2_{k-1}. On the three server configurations it gives $H = 8.26$, $p = 0.016$, compared with ANOVA's $p = 0.0047$.

```
from scipy import stats
a = [12,15,19,22,30]; b = [18,25,28,35,40,41]
print(stats.mannwhitneyu(a, b, method='exact'))   # U = 5, p = 0.082
before = [120,135,128,150,142,138,160,131]
after  = [112,130,129,138,135,131,149,127]
print(stats.wilcoxon(before, after))              # p = 0.0156
print(stats.binomtest(7, 8))                      # sign test, p = 0.070
```

10.7 A/B testing in practice

The two-proportion test of Section 10.1 is the arithmetic of an A/B test. Getting a trustworthy answer takes more care than the arithmetic.

10.7.1 The randomisation unit

Every test so far assumes independent observations. In an A/B test, the independent unit is whatever you randomised. If you assign by *user*, the same user stays in one arm for every visit, and users are the independent units. If you assign by *pageview*, one user can see both versions.

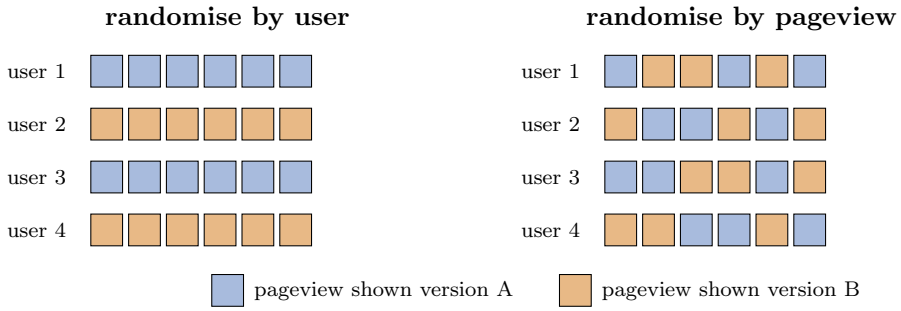


Figure 10.16: Each square is one pageview; each row is one user's six visits. Left: each user sees one version on every visit, so users are independent units. Right: versions are mixed within a user, so one user's experience of B can affect their behaviour on A, and their pageviews are not independent.

Common trap

Analyse at the level you randomised. If you randomise users but compute the standard error as if every pageview were independent, you pretend to have far more data than you do. Pageviews from one user are correlated: a heavy user contributes 200 similar pageviews. The SE comes out too small and false positives multiply.

10.7.2 Ratio metrics and the delta method

Many metrics are ratios of two per-user sums: clicks per session, revenue per order, errors per request. Let user i have Y_i clicks and X_i sessions. The metric is

$$R = \frac{\sum_i Y_i}{\sum_i X_i} = \frac{\bar{Y}}{\bar{X}}.$$

Both numerator and denominator are random, and they are correlated (users with more sessions click more). The *delta method* linearises the ratio around the true means μ_Y, μ_X with a first-order Taylor expansion, $r = \mu_Y/\mu_X$:

$$\frac{\bar{Y}}{\bar{X}} - r \approx \frac{1}{\mu_X}\left[(\bar{Y} - \mu_Y) - r(\bar{X} - \mu_X)\right].$$

The right side is an average of the per-user quantities $(Y_i - rX_i)/\mu_X$, so its variance is

$$\mathrm{Var}(R) \approx \frac{1}{n\,\mu_X^2}\left[\mathrm{Var}(Y) - 2r\,\mathrm{Cov}(X,Y) + r^2\,\mathrm{Var}(X)\right].$$

Numbers. Suppose $n = 1000$ users per arm, $\mu_X = 4$ sessions, $\mu_Y = 2$ clicks, so $r = 0.5$. Take $\mathrm{SD}(Y) = 3$, $\mathrm{SD}(X) = 5$, correlation 0.8, so $\mathrm{Cov}(X,Y) = 0.8 \times 3 \times 5 = 12$. Then

$$\mathrm{Var}(R) \approx \frac{9 - 2(0.5)(12) + 0.25(25)}{1000 \times 16} = \frac{3.25}{16000}, \qquad \mathrm{SE}(R) \approx 0.0143.$$

The naive approach treats the 4000 sessions as independent Bernoulli trials with rate 0.5: $\mathrm{SE} = \sqrt{0.25/4000} \approx 0.0079$. The naive SE is too small by a factor of 1.8. A test using it would declare far too many false wins.

Intuition

The delta method says: a smooth function of averages behaves like an average of a linearised per-unit quantity. Compute that per-user quantity, take its variance, divide by n. It is the same $\sigma/\sqrt{n}$ logic as Chapter 6, applied after a Taylor step. When in doubt, the bootstrap over users (Chapter 8) gives the same answer with no algebra.

10.7.3 Novelty and primacy effects

Users react to change itself. A new button colour gets curious clicks in week one and fades to its true effect later (novelty). Or users dislike any change at first and warm up (primacy). Either way, the first days mislead.

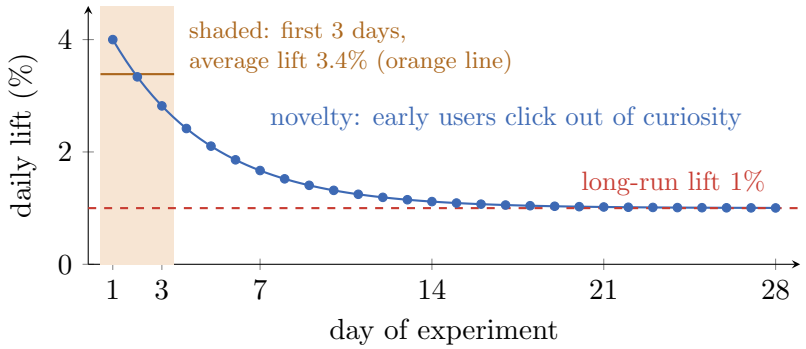


Figure 10.17: An illustrative novelty effect (one dot per day). Daily lift starts at 4% and decays to its long-run value of 1% (dashed). A test stopped after the three shaded days would report an average lift of about 3.4%, more than three times the true value.

Defences: run for at least one or two full weekly cycles; plot the daily lift and check that it has stabilised; compare new users (who never saw the old version) with returning users.

10.7.4 Sample ratio mismatch

An experiment designed as a 50/50 split ends with 50,600 users in A and 49,400 in B. That looks close. Is it? Run a chi-square goodness-of-fit test against the designed split, expected 50,000 each:

$$X^2 = \frac{600^2}{50000} + \frac{(-600)^2}{50000} = 7.2 + 7.2 = 14.4, \qquad \text{df} = 1, \qquad p \approx 0.00015.$$

A split this uneven happens about once in 7000 experiments by chance. This is a **sample ratio mismatch** (SRM). It means something in the pipeline is dropping or duplicating users in one arm: a redirect that loses slow clients, a bot filter that reacts to the new page, a crash in B that prevents logging.

Common trap

If the SRM check fails, do not interpret the metrics. The users who disappeared were not removed at random, so the arms are no longer comparable and the randomisation that justified a causal claim is broken. Find the bug first. Run the SRM check on every experiment before looking at any metric; it costs one line of code.

Pause

Suppose the same 1.2% imbalance (50.6% vs 49.4%) appears in an experiment with only 1000 users. Is that an SRM? (No. $X^2 = 6^2/500 \times 2 = 0.144$, $p \approx 0.70$. SRM tests become sharper as n grows, which is why large experiments catch small pipeline bugs.)

10.7.5 CUPED: variance reduction with pre-experiment data

Much of the variation in a user's spending this month is explained by their spending last month. That variation is noise for the experiment, since it existed before the treatment. CUPED (Controlled-experiment Using Pre-Experiment Data) removes it.

Let Y be the experiment metric and X the same metric in the weeks before the experiment. Define the adjusted metric

$$Y^{\text{adj}} = Y - \theta\,(X - \bar{X}), \qquad \theta = \frac{\operatorname{Cov}(X, Y)}{\operatorname{Var}(X)}.$$

Since X was measured before randomisation, its mean is the same in both arms in expectation, so the adjustment does not bias the treatment effect. But it shrinks the variance:

$$\operatorname{Var}(Y^{\text{adj}}) = \operatorname{Var}(Y)\,(1 - \rho^2),$$

where ρ is the correlation between X and Y. The choice of θ is the regression slope of Y on X (Chapter 12), which is exactly the value that minimises this variance.

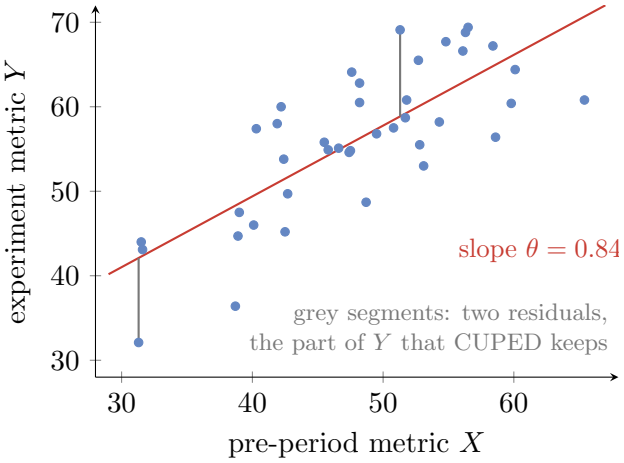


Figure 10.18: Forty users' pre-period and experiment-period values ($\rho = 0.77$). CUPED keeps only each user's vertical distance from the red line (two shown in grey). Those residuals vary much less than Y itself: in this sample the variance drops from 79.0 to 32.4, a factor of $0.41 = 1 - 0.77^2$.

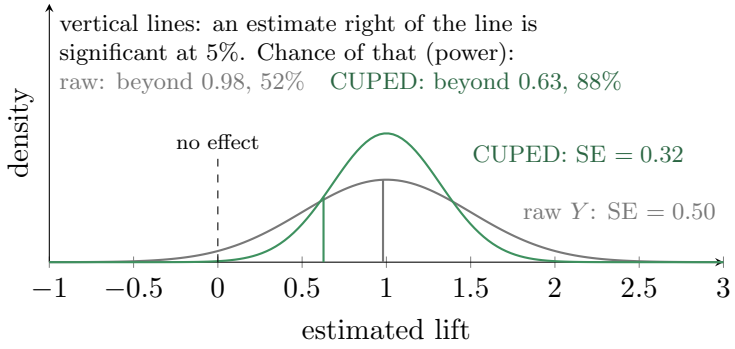


Figure 10.19: Sampling distributions of the estimated lift (true lift 1) without and with CUPED, for $\rho = 0.77$. The SE shrinks by a factor $\sqrt{0.41} \approx 0.64$, so the bar for significance (1.96 SE) drops from 0.98 to 0.63 and the same experiment detects the true lift 88% of the time instead of 52%.

Why it works

Why $1-\rho^2$? $\mathrm{Var}(Y-\theta X) = \mathrm{Var}(Y) - 2\theta\,\mathrm{Cov}(X,Y) + \theta^2\,\mathrm{Var}(X)$. This is a quadratic in θ, minimised at $\theta = \mathrm{Cov}(X,Y)/\,\mathrm{Var}(X)$. Plugging that in gives $\mathrm{Var}(Y) - \mathrm{Cov}(X,Y)^2/\,\mathrm{Var}(X) = \mathrm{Var}(Y)(1-\rho^2)$. Since required sample size scales with variance, $\rho = 0.7$ cuts the users needed by half ($1 - 0.49 = 0.51$). CUPED is regression adjustment; it is the "control for a pre-treatment covariate" idea of Chapter 13 applied to experiments.

10.8 Which test when

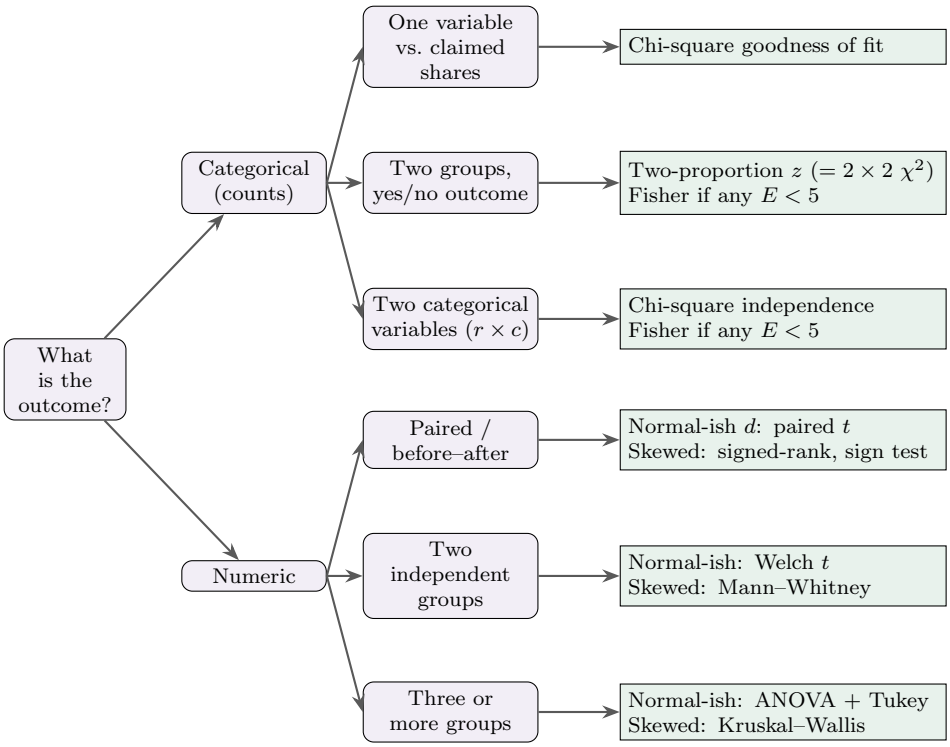


Figure 10.20: Choosing a test. First ask what kind of outcome you have, then how the data are structured, then whether normal-theory assumptions are plausible. In every A/B setting, also check the randomisation unit and run the SRM test first.

Common trap

Running several tests and reporting the one with the smallest p-value is p-hacking. Choose the test before seeing the data, from the structure of the design, using a chart like Figure 10.20. Chapter 11 covers why this matters and how badly it inflates false positives.

A p-value from any of these tests answers "is there a difference?" It does not answer "how big, and does it matter?" Chapter 11 turns to effect sizes, to what happens when you run many tests at once, and to the common ways a correct test still produces a misleading conclusion.

10.9 Summary

Test	Statistic	Null distribution
Two proportions	$z = (\hat{p}_B - \hat{p}_A)/\,\mathrm{SE}_0$, pooled $\hat{p}$	$\mathcal{N}(0,1)$
Goodness of fit	$X^2 = \sum (O-E)^2/E$	χ^2_{k-1}
Independence	same, $E_{ij} = R_i C_j / N$	$\chi^2_{(r-1)(c-1)}$
Fisher exact	count in one cell, margins fixed	hypergeometric
One-way ANOVA	$F = \mathrm{MSB}/\mathrm{MSW}$	$F_{k-1,\,N-k}$
Mann–Whitney	U = pairs with A > B	exact / normal
Signed-rank	W_+ = sum of positive ranks	exact / normal
Sign test	number of positive differences	Binomial$(n, 1/2)$

- Every test compares observed data with what H_0 predicts, scaled by noise.
- Pooled SE for the test of two proportions, unpooled SE for the interval. The 2×2 chi-square statistic equals z^2.
- Chi-square tests need counts and expected counts of at least about 5; otherwise use Fisher's exact test.
- ANOVA splits $\mathrm{SST} = \mathrm{SSB} + \mathrm{SSW}$; F compares between-group to within-group variance. With two groups, $F = t^2$. A significant F needs post-hoc tests to say which groups differ.
- Rank tests trade some power for robustness to skew and outliers.
- In A/B tests: analyse at the randomisation unit, use the delta method (or bootstrap) for ratio metrics, watch for novelty, check SRM before anything else, and use CUPED to cut variance by $1 - \rho^2$.

10.10 Exercises

1. A sign-up page converts 200 of 2000 visitors in A and 250 of 2000 in B. Compute the pooled rate, the null SE, z, and the two-sided p-value.
2. A coin lands heads 60 times in 100 flips. Compute the chi-square goodness-of-fit statistic against a fair coin and its p-value. Compare with the one-proportion z statistic.
3. A 2×2 table has row totals 30 and 70, column totals 40 and 60, grand total 100. Find all four expected counts under independence. How many degrees of freedom does the test have?
4. An ANOVA with $k = 4$ groups and $N = 24$ observations has SSB = 24 and SSW = 60. Complete the ANOVA table. Is the result significant at 5%? (The 95th percentile of $F_{3,20}$ is 3.10.)
5. A two-group ANOVA gives $F = 6.25$. What is $|t|$ for the pooled two-sample t-test on the same data?

6. Samples A: $3, 8, 10$ and B: $5, 12, 14, 20$. Compute the rank sum of A and U_A. What fraction of (A, B) pairs have A larger?
7. In 10 paired comparisons, 9 differences are positive. Compute the two-sided sign test p-value.
8. A 50/50 experiment ends with 10,300 users in A and 9,700 in B. Run the SRM check. What should you do next?
9. Pre-period and experiment metrics have correlation $\rho = 0.5$. By what factor does CUPED reduce the variance? Repeat for $\rho = 0.9$.
10. Why should you not run three pairwise t-tests at $\alpha = 0.05$ to compare three groups? Estimate the chance of at least one false positive if the tests were independent.

Answers

1. $\hat{p} = 450/4000 = 0.1125$; $\text{SE}_0 = \sqrt{0.1125 \times 0.8875 \times 2/2000} \approx 0.00999$; $z = 0.025/0.00999 \approx 2.50$; $p \approx 0.012$.
2. $X^2 = 10^2/50 + 10^2/50 = 4$, df $= 1$, $p \approx 0.046$. $z = (0.6 - 0.5)/\sqrt{0.25/100} = 2$ and $z^2 = 4$.
3. $E = 30 \cdot 40/100 = 12$, $30 \cdot 60/100 = 18$, $70 \cdot 40/100 = 28$, $70 \cdot 60/100 = 42$. df $= 1$.
4. MSB $= 24/3 = 8$, MSW $= 60/20 = 3$, $F = 2.67 < 3.10$: not significant ($p \approx 0.076$).
5. $|t| = \sqrt{6.25} = 2.5$.
6. Pooled order: 3, 5, 8, 10, 12, 14, 20. A has ranks 1, 3, 4, so $R_A = 8$ and $U_A = 8 - 6 = 2$. A is larger in $2/12 \approx 17\%$ of pairs ($8 > 5$ and $10 > 5$).
7. $2(\binom{10}{9} + \binom{10}{10})/1024 = 22/1024 \approx 0.021$.
8. $X^2 = 300^2/10000 \times 2 = 18$, $p \approx 0.00002$. SRM detected: do not read the metrics; find the logging or assignment bug first.
9. $1 - 0.25 = 0.75$ (25% less variance); $1 - 0.81 = 0.19$ (81% less, about 5 times fewer users needed).
10. The familywise false positive rate inflates: $1 - 0.95^3 \approx 0.14$. Use ANOVA, then a post-hoc procedure such as Tukey's HSD.

Chapter 11

Effect Sizes, Multiple Testing and Pitfalls

Chapters 9 and 10 built the machinery: a test statistic, a p-value, a decision. This chapter is about what goes wrong when that machinery meets real people with real deadlines. Every tool here has the same aim. You start with a claim that some result is "significant" and work out whether you should actually believe it, and whether it matters.

11.1 Statistical versus practical significance

Start with an A/B test on a checkout page. Each arm gets one million visitors.

Arm	Visitors	Purchases	Conversion
Control A	1,000,000	100,000	10.00%
Treatment B	1,000,000	101,000	10.10%

The two-proportion z-test from Chapter 10 uses the pooled rate $\hat{p} = 0.1005$:

$$\text{SE} = \sqrt{\hat{p}(1-\hat{p})\left(\tfrac{1}{n_A} + \tfrac{1}{n_B}\right)} = \sqrt{0.1005 \cdot 0.8995 \cdot 2 \times 10^{-6}} \approx 0.000425,$$

$$z = \frac{0.1010 - 0.1000}{0.000425} \approx 2.35, \qquad p \approx 0.019.$$

The result is "significant at the 5% level". The 95% confidence interval for the difference is 0.10 ± 0.08 percentage points, roughly [0.02, 0.18]. The new page lifts conversion by about one tenth of a percentage point. If the redesign cost three engineer-months and slows the page, nobody should ship it.

The p-value answered one narrow question: is the difference exactly zero? With a million users per arm, almost nothing is exactly zero, so almost everything is "significant". Here the p-value tells you far more about how big the sample was than about how big the effect is.

Definition 11.1 (Statistical and practical significance)**.** A result is **statistically significant** if its p-value falls below the chosen level α: the data are hard to explain by chance alone under the null. It is **practically significant** if the size of the effect is large enough to change a decision. The two are separate questions, and sample size drives the first much more than the second.

Figure 11.1 holds the true effect fixed at +0.1 percentage points and lets n grow. The p-value you would typically see slides from 0.94 to below 10^{-6}. Nothing about the product changed.

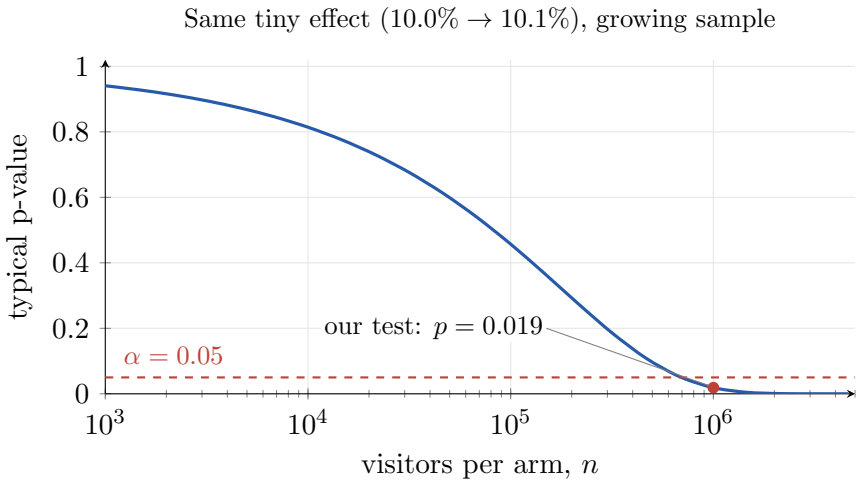


Figure 11.1: The p-value computed at the expected z for a fixed lift of 0.1 percentage points. Grow n and the curve dives under 0.05 while the effect stays exactly as small. A small p-value tells you the effect is probably not zero. It does not tell you the effect is large.

The fix is to decide in advance what size of effect would matter. Call it the **smallest effect of interest**: the lift below which you would not act, whatever the p-value. Then read the confidence interval against that threshold. Figure 11.2 shows the four cases you will meet.

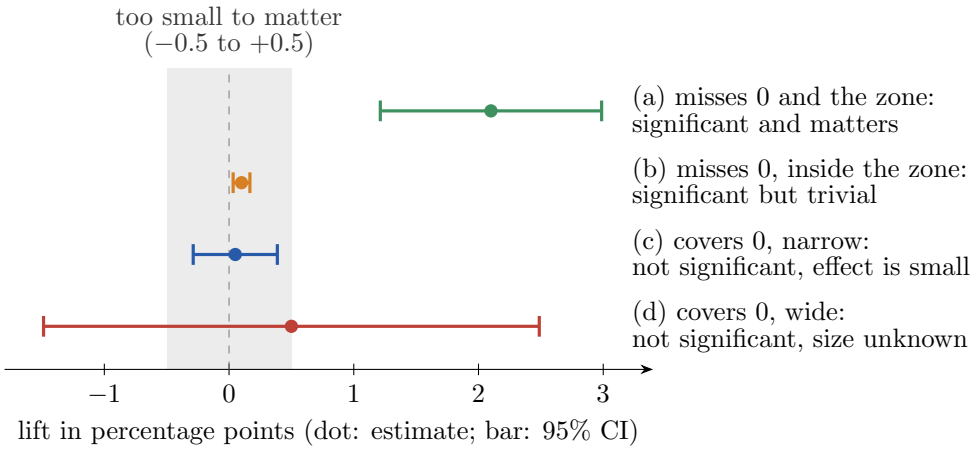


Figure 11.2: Four 95% confidence intervals for a lift, read against a shaded zone of effects too small to act on (±0.5 points). (a) excludes zero and the zone: ship it. (b) excludes zero but lies inside the zone: it is the million-user test above, [0.02, 0.18]. (c) includes zero yet is narrow, so you have learned the effect is small. (d) includes zero and is wide, so you have learned almost nothing. The p-value alone cannot tell (b) from (a) or (c) from (d).

Common trap

"Not significant" does not mean "no effect". Interval (d) in Figure 11.2 is consistent with a loss of 1.5 points and with a gain of 2.5. The experiment was too small to say. Absence of evidence is evidence of absence only when the interval is narrow, as in (c).

Pause

Pause and think. A drug trial with $n = 40$ reports $p = 0.30$. A colleague says "so the drug does nothing". What single extra number would you ask for? (The confidence interval for the effect. If it runs from strongly harmful to strongly helpful, the trial was uninformative.)

11.2 Effect sizes: saying how big

An **effect size** is a number that measures how large a difference or association is, in units someone can act on. The p-value mixes effect size and sample size into one number. An effect size keeps them apart. Which one to report depends on the data type.

11.2.1 Difference in means, with its interval

A backend team rewrites a service. They sample 50 request latencies before and 50 after.

	n	mean (ms)	SD (ms)
Old service	50	212	40
New service	50	196	36

The raw effect size is the difference in means, $\Delta = 212 - 196 = 16$ ms. Its standard error (Chapter 9, Welch form) is

$$\mathrm{SE}(\Delta) = \sqrt{\frac{40^2}{50} + \frac{36^2}{50}} = \sqrt{32 + 25.92} \approx 7.61 \text{ ms}.$$

Welch's degrees of freedom come out at about 96.9, so the critical value is $t^* \approx 1.985$ and the 95% interval is

$$16 \pm 1.985 \times 7.61 = 16 \pm 15.1 \quad \Longrightarrow \quad [0.9,\ 31.1] \text{ ms}.$$

The t statistic is $16/7.61 \approx 2.10$, giving $p \approx 0.038$.

Read the interval, not the p-value. The rewrite saves somewhere between about 1 ms and 31 ms per request. If your latency budget only cares about savings over 20 ms, this experiment has not settled the question. That judgement needs the interval and is invisible in "$p = 0.038$".

Intuition

The raw difference is the best effect size whenever the unit means something to the reader: milliseconds, dollars, kilograms, percentage points. Use a standardized measure (next) only when units differ across studies or have no natural meaning, such as scores on a 7-point survey scale.

11.2.2 Cohen's d: difference in units of spread

Definition 11.2 (Cohen's d)**.**

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s_p}, \qquad s_p = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}.$$

Here s_p is the **pooled standard deviation**: a weighted average of the two groups' spreads, a single typical distance of an individual from their own group's mean. So d says how many such typical distances separate the two group means.

For the latency data,

$$s_p = \sqrt{\frac{49 \cdot 1600 + 49 \cdot 1296}{98}} = \sqrt{1448} \approx 38.05, \qquad d = \frac{16}{38.05} \approx 0.42.$$

The means are less than half a standard deviation apart, so the two latency distributions overlap heavily: a random request to the new service is faster than a random request to the old one only about 62% of the time (the formula is in the caption of Figure 11.3).

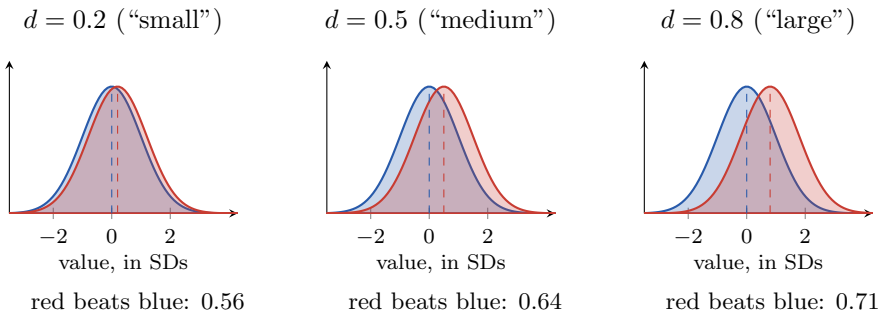


Figure 11.3: Blue: one group; red: a second group whose mean (dashed line) sits d standard deviations higher. Even a "large" effect leaves the curves mostly overlapping. The number under each panel is the chance a random member of the red group beats a random member of the blue group, $\Phi(d/\sqrt{2})$, where Φ is the standard Normal CDF (area to the left of a point). With no effect it would be 0.5.

Cohen's rough labels, 0.2 small, 0.5 medium, 0.8 large, came from behavioural science in the 1960s. They are a last resort. A d of 0.2 on mortality is enormous; a d of 0.8 on a survey of favourite colours may be meaningless. Translate back to real units whenever you can.

11.2.3 Proportions: difference, lift, risk ratio, odds ratio

A second A/B test, this one smaller:

Arm	Visitors	Signed up	Rate
Control	5,000	200	$p_A = 4.0\%$
Treatment	5,000	240	$p_B = 4.8\%$

There are four common ways to say how big this is. They describe the same table.

Measure	Formula	Value	Reads as
Absolute difference	$p_B - p_A$	0.8 points	8 more signups per 1000 visitors
Relative lift	$(p_B - p_A)/p_A$	+20%	signups up by a fifth
Risk ratio (RR)	p_B/p_A	1.20	treatment rate is 1.2× control
Odds ratio (OR)	$\dfrac{p_B/(1-p_B)}{p_A/(1-p_A)}$	1.21	odds of signing up 1.21×

The **odds** of an event are $p/(1-p)$: the chance it happens divided by the chance it does not. A 4% rate is odds of $0.04/0.96 = 1/24$, "1 to 24". Odds matter for two reasons. Logistic regression (Chapter 13) speaks in odds ratios. And in a case-control study, where researchers recruit people *because* they have the disease and compare them with people who do not, the rates are fixed by the design, so the OR is the only one of these measures the data can estimate.

Each measure has a 95% interval. For the difference,

$$\text{SE} = \sqrt{\frac{0.04 \cdot 0.96}{5000} + \frac{0.048 \cdot 0.952}{5000}} \approx 0.00410, \qquad 0.8 \pm 1.96 \times 0.41 = [-0.004,\ 1.60] \text{ points.}$$

Ratios are skewed (they cannot go below 0 but can go to infinity), so their intervals are built on the log scale, where the sampling distribution is close to Normal, and exponentiated back. The standard errors below are the textbook large-sample formulas; take them on trust (they come from the delta method, a first-order Taylor approximation of log around the true rate):

$$\text{SE}(\log \text{RR}) = \sqrt{\tfrac{1}{240} - \tfrac{1}{5000} + \tfrac{1}{200} - \tfrac{1}{5000}} \approx 0.0936, \qquad \text{RR} \in 1.20\, e^{\pm 1.96 \times 0.0936} = [1.00,\ 1.44].$$

$$\text{SE}(\log \text{OR}) = \sqrt{\tfrac{1}{200} + \tfrac{1}{4800} + \tfrac{1}{240} + \tfrac{1}{4760}} \approx 0.0979, \qquad \text{OR} \in 1.21\, e^{\pm 1.96 \times 0.0979} = [1.00,\ 1.47].$$

The pooled z-test gives $z \approx 1.95$, $p \approx 0.051$: just "not significant". Reporting only that would bury the real message. The data fit anything from no change to a 44% relative gain. A bigger test is warranted, not a shrug.

Common trap

Relative lift without the baseline misleads. "Doubles your risk" can mean 1 in a million to 2 in a million. Always give the absolute numbers next to a ratio. Headlines quote ratios because they are larger.

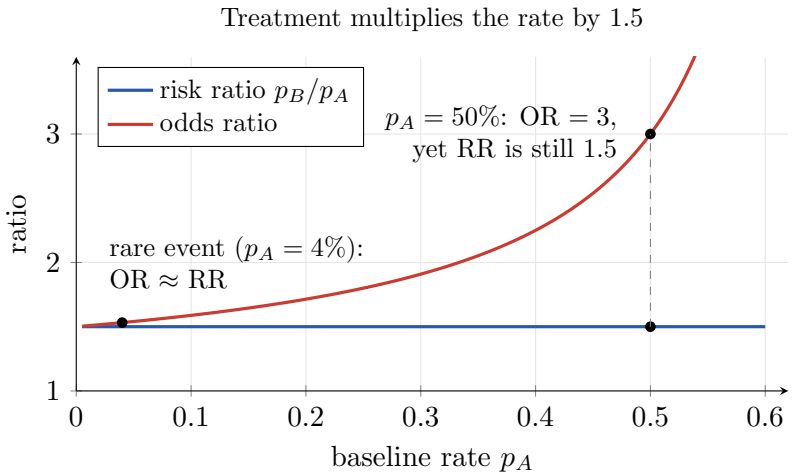


Figure 11.4: Hold the risk ratio at 1.5 and vary the baseline rate. For rare events the odds ratio and risk ratio agree. Once the baseline passes about 10% the odds ratio runs away, and reading an OR of 3 as "three times as likely" is wrong: at a 50% baseline the treatment makes the event only 1.5 times as likely.

```
import numpy as np
a, na, b, nb = 200, 5000, 240, 5000
pa, pb = a/na, b/nb
rr = pb/pa;  se_rr = np.sqrt(1/b - 1/nb + 1/a - 1/na)
orr = (b/(nb-b)) / (a/(na-a))
se_or = np.sqrt(1/a + 1/(na-a) + 1/b + 1/(nb-b))
```

```
print(rr, rr*np.exp([-1.96, 1.96]*se_rr))    # 1.20 [1.00 1.44]
print(orr, orr*np.exp([-1.96, 1.96]*se_or))  # 1.21 [1.00 1.47]
```

11.3 Report intervals, not just p-values

A 95% interval carries the same decision as a test at $\alpha = 0.05$: the test rejects H_0: $\Delta = 0$ exactly when 0 lies outside the interval (the duality of Chapter 9). The interval also tells you the size of the effect and how precisely it was measured. The p-value throws both away.

Figure 11.5 shows two studies with the same p-value. One is a small startup test, the other a large platform test. A reader who sees only "$p = 0.04$" cannot tell them apart.

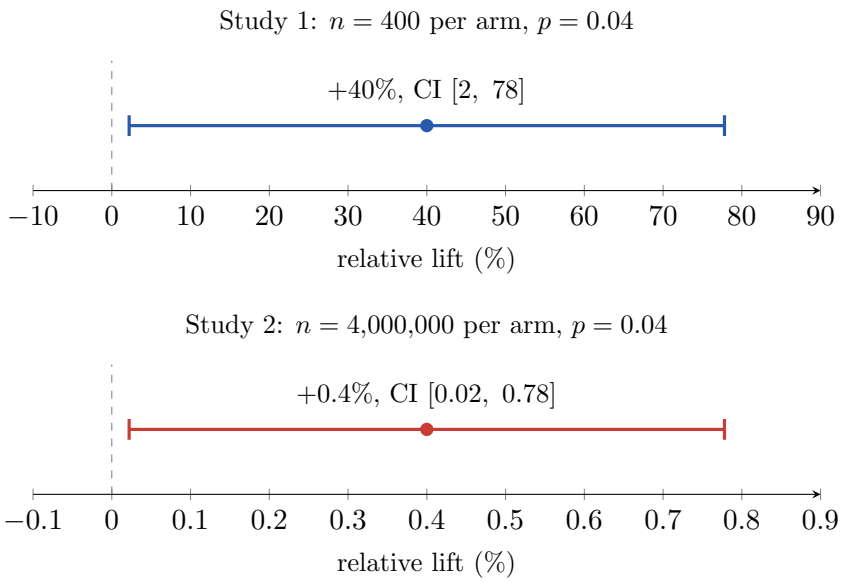


Figure 11.5: Two experiments with identical p-values. Note the axes: the second plot is zoomed in a hundredfold, because 10,000 times the data gives $\sqrt{10{,}000} = 100$ times the precision. Study 1 found a possibly huge effect measured imprecisely; study 2 found a tiny effect measured precisely. Only the intervals reveal this.

A good report of any comparison has four parts: the estimate in natural units, its confidence interval, the sample size, and (optionally) the p-value. In prose: "The new service reduced mean latency by 16 ms (95% CI 1 to 31 ms, $n = 50$ per group, $p = 0.04$)."

11.4 Multiple comparisons

11.4.1 The jelly-bean problem

Your A/B dashboard tracks 20 metrics. Suppose the change truly affects none of them. Each metric gets its own test at $\alpha = 0.05$. How likely is at least one red "significant" cell?

Each test has a 95% chance of correctly staying quiet. If the tests are independent, all 20 stay quiet with probability $0.95^{20} \approx 0.36$. So

$$P(\text{at least one false positive}) = 1 - (1-\alpha)^m = 1 - 0.95^{20} \approx 0.64.$$

Two times in three, a dashboard of 20 metrics on a change that does nothing will show something "significant".

Definition 11.3 (Family-wise error rate)**.** With m tests, the **family-wise error rate** (FWER) is the probability of making at least one false rejection among the whole family of tests. For m independent true nulls each tested at level α,

$$\text{FWER} = 1 - (1-\alpha)^m.$$

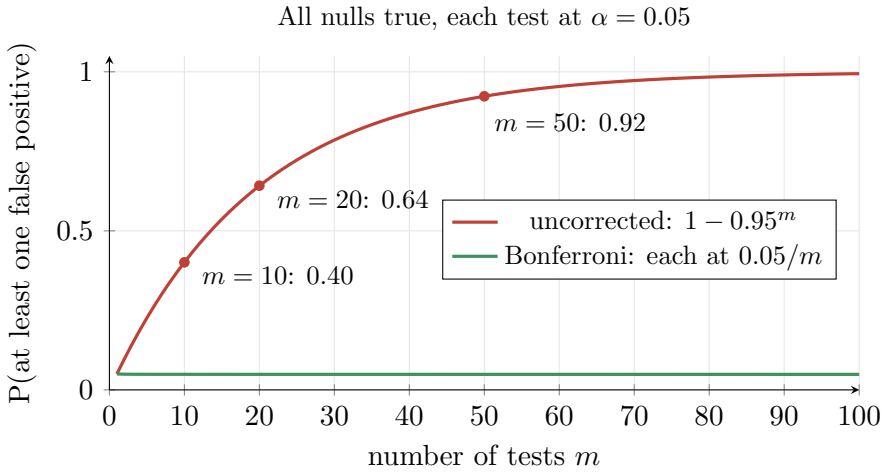


Figure 11.6: The chance of at least one false alarm climbs fast with the number of tests: past 50% at $m = 14$. Dividing α by m (Bonferroni) pins it just under 0.05 for every m.

11.4.2 Bonferroni: divide alpha by m

Theorem 11.1 (Bonferroni correction)**.** *Test each of m hypotheses at level α/m. Then* FWER $\le \alpha$*, whatever the dependence between the tests.*

Why it works

Let E_i be the event "test i falsely rejects". For a true null, $P(E_i) \le \alpha/m$. The probability of a union is at most the sum of the probabilities (the union bound: overlapping events are counted at least once in the sum, so the sum can only overcount), so

$$P(E_1 \cup \cdots \cup E_m) \le \sum_{i=1}^{m} P(E_i) \le m \cdot \frac{\alpha}{m} = \alpha.$$

No independence was used. That is why Bonferroni is safe everywhere, and also why it is conservative: when tests are positively correlated the union is much smaller than the sum.

The price is power. Testing 20 metrics at 0.0025 each makes a real but modest effect much harder to detect. Two refinements recover power.

11.4.3 Holm: a free upgrade

Holm's step-down method controls FWER at α under the same conditions as Bonferroni and never rejects less.

1. Sort the p-values: $p_{(1)} \le p_{(2)} \le \cdots \le p_{(m)}$.
2. Compare $p_{(1)}$ with α/m, $p_{(2)}$ with $\alpha/(m-1)$, $p_{(3)}$ with $\alpha/(m-2)$, and so on.
3. Stop at the first p-value that fails. Reject everything before it.

The idea: once you have rejected the smallest p-value, at most $m-1$ true nulls remain, so the next test only needs to guard against $m-1$ chances of error.

11.4.4 Benjamini–Hochberg: controlling the false discovery rate

Sometimes one false alarm among many findings is acceptable. A biologist screening 20,000 genes wants a list of candidates for follow-up, and can live with a few duds if most of the list is real. The right target is then not "no false positives at all" but "a small fraction of false positives among what I report".

Definition 11.4 (False discovery rate)**.** Among the hypotheses you reject (your "discoveries"), the **false discovery proportion** is the fraction that are actually true nulls. The **false discovery rate** (FDR) is its expected value:

$$\mathrm{FDR} = \mathbb{E}\left[\frac{\#\text{false rejections}}{\max(\#\text{rejections}, 1)}\right].$$

Theorem 11.2 (Benjamini–Hochberg procedure)**.** *Sort the p-values $p_{(1)} \le \cdots \le p_{(m)}$. Find the largest k with*

$$p_{(k)} \le \frac{k}{m}\, q.$$

Reject the hypotheses with the k smallest p-values. Here q is the false discovery rate you are willing to accept, often 0.05 or 0.10. If the tests are independent (or positively dependent), then $\mathrm{FDR} \le q$.

Walking question

An experiment reports ten metrics. The p-values, already sorted:

$$0.001,\ 0.004,\ 0.006,\ 0.021,\ 0.024,\ 0.060,\ 0.074,\ 0.205,\ 0.212,\ 0.416.$$

Which metrics count as moved, at $\alpha = q = 0.05$?

k	$p_{(k)}$	Bonferroni $0.05/10$	Holm $0.05/(11-k)$	BH $0.05\,k/10$	BH ok?
1	0.001	0.005 ✓	0.00500 ✓	0.005	✓
2	0.004	0.005 ✓	0.00556 ✓	0.010	✓
3	0.006	0.005	0.00625 ✓	0.015	✓
4	0.021	0.005	0.00714 stop	0.020	×
5	0.024	0.005	0.00833	0.025	✓
6	0.060	0.005	0.01000	0.030	×
7	0.074	0.005	0.01250	0.035	×
8	0.205	0.005	0.01667	0.040	×
9	0.212	0.005	0.02500	0.045	×
10	0.416	0.005	0.05000	0.050	×

Uncorrected: five p-values are below 0.05, so five "wins".
Bonferroni: only 0.001 and 0.004 are below 0.005. Two rejections.
Holm: $0.001 \le 0.005$, $0.004 \le 0.00556$, $0.006 \le 0.00625$, then $0.021 > 0.00714$ stops the procedure. Three rejections.
BH: the largest k with $p_{(k)} \le 0.005k$ is $k = 5$ ($0.024 \le 0.025$). Reject the five smallest, *including* $k = 4$, even though 0.021 missed its own threshold of 0.020. BH is a *step-up* rule: it looks for the last crossing, not the first failure.
So with ten metrics: two survive the strictest guarantee (Bonferroni), three survive Holm, and five survive FDR control. Matching the uncorrected count is luck of this data set; usually BH keeps fewer than the uncorrected count.

Figure 11.7 draws the same procedure. Plot the sorted p-values against their rank and draw the line $y = qk/m$ through the origin. Find the last point on or under the line. Everything to its left is a discovery.

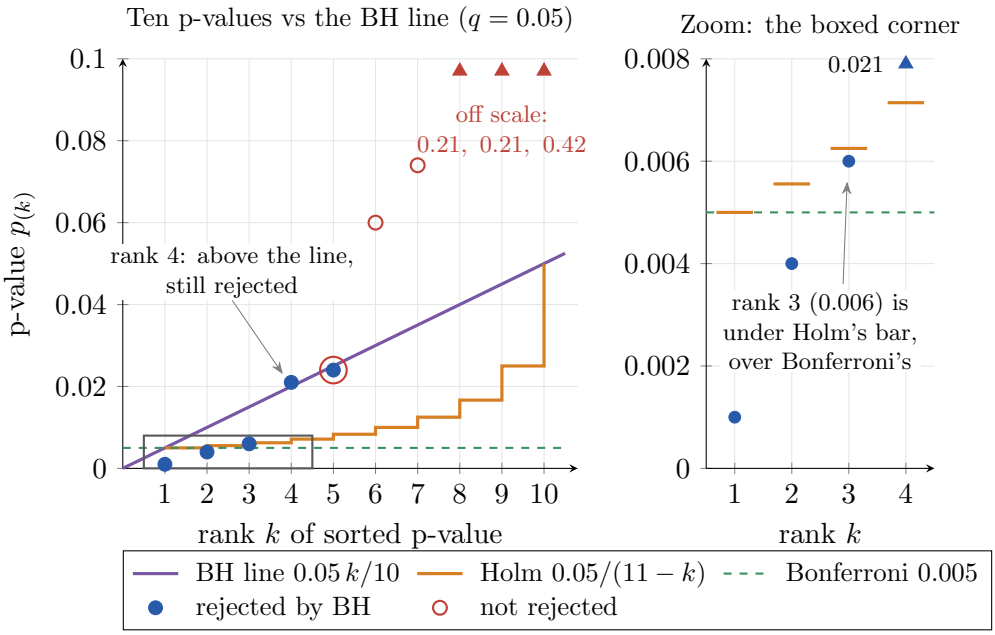


Figure 11.7: BH in one picture. Left: the last p-value on or under the purple line is rank 5 (circled), so ranks 1–5 are all discoveries, including rank 4, which sits just above the line. Holm's orange staircase and Bonferroni's green line hug the bottom. Right: zooming into the boxed corner shows why they admit only three and two points: ranks 1–3 sit under their Holm bars, but only ranks 1–2 are under Bonferroni's line, and rank 4 (0.021) clears neither.

Why it works

Why does the line qk/m control the false discovery fraction? Suppose m_0 of the nulls are true. True-null p-values are uniform on $[0, 1]$ (Chapter 9), so about $m_0 t$ of them land below any cutoff t. If BH ends up rejecting k hypotheses, its cutoff is $t = qk/m$, so the expected number of false rejections is about $m_0 \cdot qk/m$. Divide by the k rejections:

$$\text{false fraction} \approx \frac{m_0\, qk/m}{k} = \frac{m_0}{m}\, q \le q.$$

The threshold grows with k because more discoveries can absorb more false ones while keeping the same fraction. Making this heuristic exact takes a page of careful conditioning; Benjamini and Hochberg's 1995 paper does it.

History

In the early 1990s Yoav Benjamini and Yosef Hochberg, statisticians at Tel Aviv University, were stuck on a practical complaint. Researchers running dozens of comparisons found the Bonferroni correction so strict that real effects vanished, so many simply stopped correcting. Benjamini and Hochberg changed the target. Instead of the family-wise error rate (the chance of even one false positive) they proposed controlling the false discovery rate (the expected share of false positives among reported findings). Their 1995 paper in the *Journal of the Royal Statistical Society* was initially hard to publish. A few years later microarrays let biologists test thousands of genes at once, where Bonferroni is hopeless, and the BH procedure became

the default. It is now one of the most cited statistics papers ever written.

Method	Controls	Use when
Bonferroni	FWER, any dependence	few tests; any false positive is costly
Holm	FWER, any dependence	same as Bonferroni; always at least as powerful
Benjamini–Hochberg	FDR	many tests; you report a list and accept a small share of duds

```python
from statsmodels.stats.multitest import multipletests
p = [0.001,0.004,0.006,0.021,0.024,0.060,0.074,0.205,0.212,0.416]
for method in ["bonferroni", "holm", "fdr_bh"]:
    reject, p_adj, _, _ = multipletests(p, alpha=0.05, method=method)
    print(method, reject.sum())   # 2, 3, 5
```

Pause

Pause and think. A team adds a 21st metric whose p-value is 0.9. Can that change which of the original metrics BH rejects? (Yes. m becomes 21, so every threshold qk/m shrinks, and BH now keeps only three discoveries instead of five. Adding junk tests makes discovery harder, which is one more reason to pick metrics before the experiment.)

11.5 p-hacking and the garden of forking paths

Multiple testing is easy to see when the 20 tests are in a table. It is much harder to see when the "tests" are choices an analyst makes along the way, each of which seems reasonable on its own.

p-hacking means trying analyses until one gives $p < 0.05$, then reporting that one. It is rarely deliberate fraud. More often it is a sequence of defensible decisions, each made after peeking at the data: drop that weird outlier; revenue looks noisy, try orders instead; maybe the effect is only in new users. The statistician Andrew Gelman called this the **garden of forking paths**: the analyst only walks one path, but the path was chosen by the data, so the p-value no longer means what it claims.

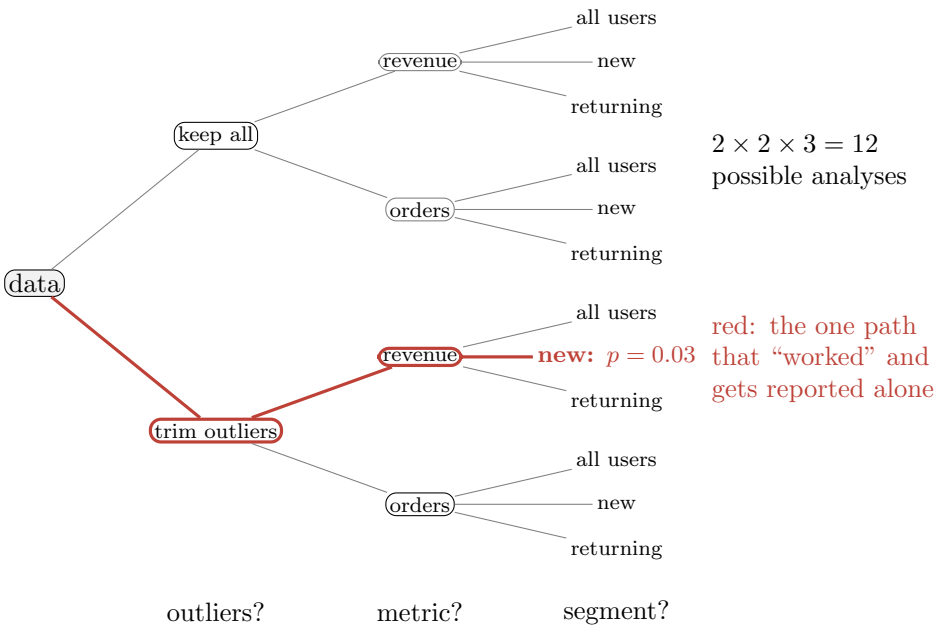


Figure 11.8: Three innocent-looking choices (outliers, metric, segment) produce 12 different analyses of the same A/B test. If the analyst follows the red path because it is the one that reached $p < 0.05$, and reports only that, the effective number of tests is 12, not 1.

If the 12 analyses were independent, the chance that at least one gives $p < 0.05$ under a true null would be $1 - 0.95^{12} \approx 0.46$. They are not independent, since they reuse the same users, so the true rate is lower. A simulation answers the question. Generate 2000 fake A/B tests with 2000 users per arm and *no* real difference, run all 12 analyses on each, and count how often the best one is "significant". Depending on how strongly the analyses overlap, the answer comes out at roughly 25 to 35%, five or more times the advertised 5%, even though each single analysis taken alone fires about 5% of the time.

The cure is to fix the analysis before seeing the outcome. A **pre-registration** is a dated, written plan that names the primary metric, the population, the exclusion rules and the test. Anything else you then look at is labelled exploratory: interesting, but a hypothesis for the next experiment, not a conclusion.

11.5.1 Optional stopping: the peeking problem

The most common fork in industry is time. A product manager checks the A/B dashboard every morning and stops the test the first day it shows $p < 0.05$.

Here is what that does. Take an A/A test, where both arms are identical so every "win" is false. Plan for 1000 observations. At each peek compute the z statistic on all data so far and stop if $|z| > 1.96$. Figure 11.9 follows one such test, peeking every 10 observations.

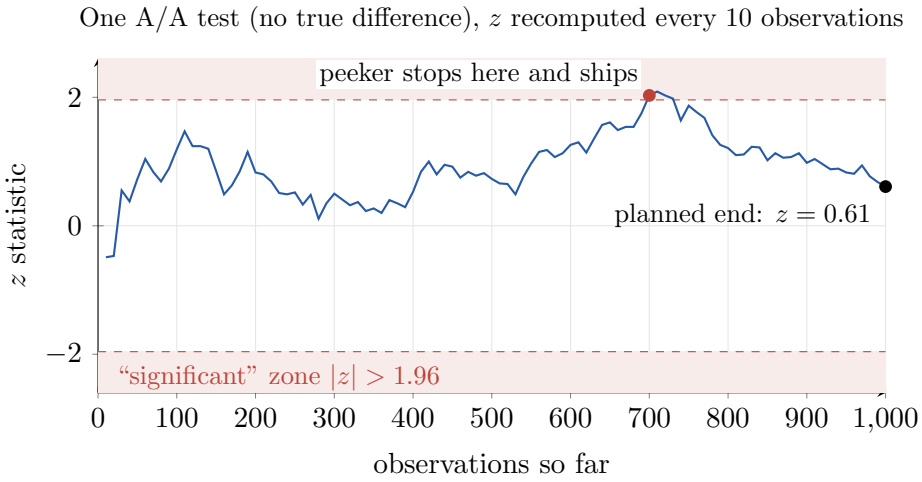


Figure 11.9: A real path from the simulation below. The z statistic wanders like a random walk. It pokes into the red zone at $n = 700$, and a daily peeker declares victory. Had the test run to its planned end it would have shown nothing.

Each single look has a 5% false-positive rate. But you get many looks at a wandering path, and you stop at whichever one is extreme. Simulating 20,000 A/A tests with equally spaced peeks shows how bad it gets. Figure 11.10 shows the share of A/A tests that were ever declared significant.

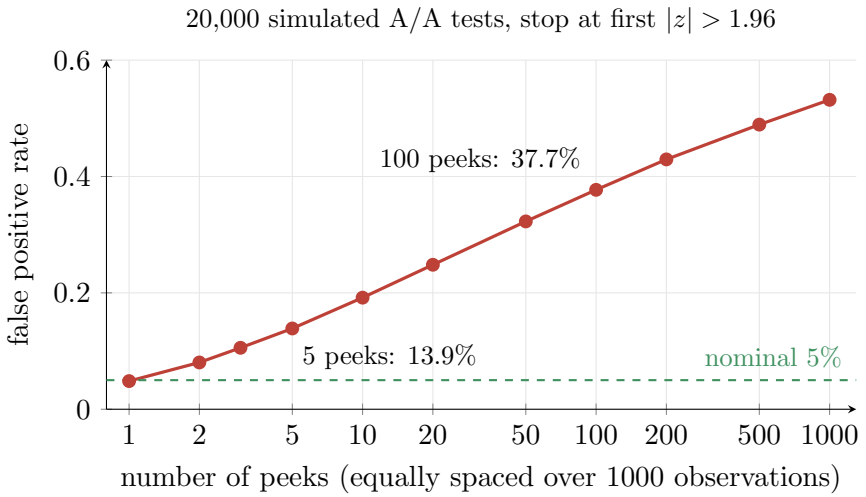


Figure 11.10: False positive rate of "stop when significant" as a function of how often you look. Five looks nearly triple the error rate. Checking after every observation pushes it past 50%. With unlimited patience it would reach 100%: the z statistic of a never-ending A/A test eventually crosses any fixed threshold (the law of the iterated logarithm).

```
import numpy as np
rng = np.random.default_rng(11)
R, N = 20000, 1000
S = np.cumsum(rng.standard_normal((R, N)), axis=1)  # A/A: true effect 0
for k in [1, 5, 20, 100]:
    looks = np.arange(1, k+1) * N // k
```

```
    z = S[:, looks-1] / np.sqrt(looks)
    print(k, (np.abs(z) > 1.96).any(axis=1).mean())  # .048 .139 .248 .377
```

11.5.2 Sequential testing: peeking done right

Peeking is not wrong in itself. Stopping a clearly harmful test early is good practice. The problem is using a 1.96 threshold at every look. A **sequential test** plans the looks in advance and uses stricter thresholds so that the total false-positive rate across all looks stays at α.

- **Pocock boundary**: the same stricter threshold at every look. For 5 equally spaced looks it is $|z| > 2.41$. In the same 20,000 simulated A/A tests, this gives a 4.9% false positive rate.
- **O'Brien–Fleming boundary**: very strict early, close to 1.96 at the end. For 5 looks the thresholds are 4.56, 3.23, 2.63, 2.28, 2.04. Early stops need overwhelming evidence, and the final look costs almost nothing. Simulated rate: 4.9%.
- **Always-valid methods** (mixture sequential probability ratio tests, "anytime p-values") let you look after every observation. Many commercial A/B platforms use them.

Figure 11.11 draws the three rules for five planned looks. At each look you stop and declare a winner only if $|z|$ is above the rule's line.

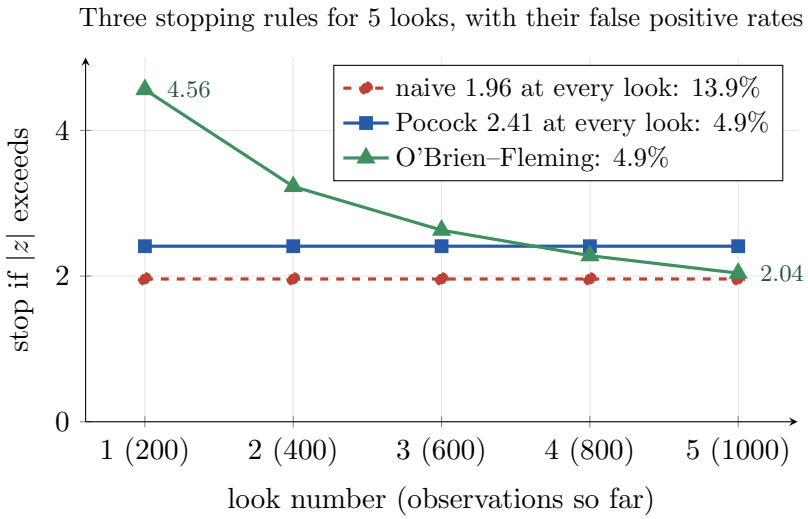


Figure 11.11: Stopping boundaries for five equally spaced looks at an A/A test of 1000 observations. The naive rule uses 1.96 every time and fires falsely 13.9% of the time. Pocock raises the bar evenly; O'Brien–Fleming makes early looks nearly impossible to pass and the last look almost as easy as a single test. Both bring the overall rate (from the same 20,000 simulated tests) back to about 5%.

The general idea is called **alpha spending**: you have a budget of 0.05 false-positive probability, and each look spends part of it. You can look as often as you like, provided the thresholds were set to respect the budget before the data arrived.

11.6 Publication bias and the winner's curse

Now zoom out from one analyst to a whole field. Journals, conference talks and launch reviews favour significant results. Studies with $p > 0.05$ end up in a drawer. This is **publication bias**, and it distorts the published record even if every individual study is honest.

Take a concrete case. A feature truly lifts a metric by 1 unit. Experiments are underpowered: each estimate has standard error 1, so the estimate $\hat{\theta} \sim \mathcal{N}(1, 1)$. A result gets published (or launched) only if it is significant in the positive direction, $\hat{\theta} > 1.96$.

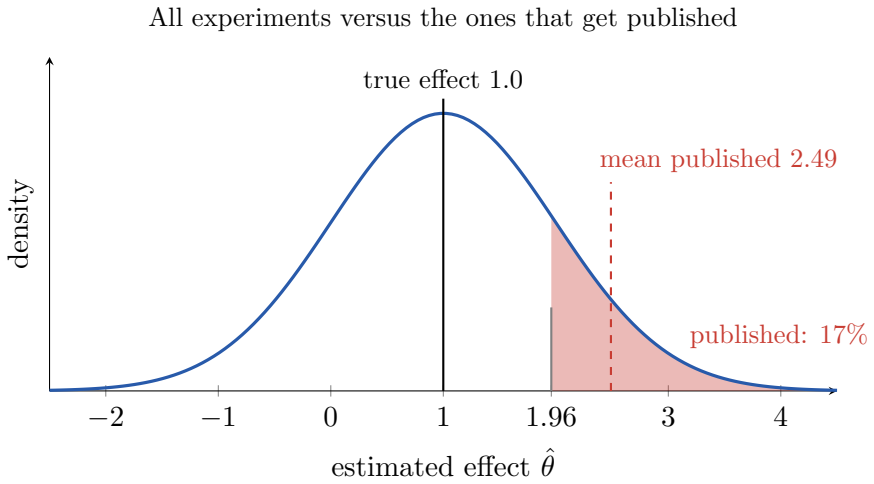


Figure 11.12: Every experiment's estimate is drawn from the blue curve, centred on the truth. Only the red tail, beyond the significance bar at 1.96 (grey line), gets published. The published estimates average 2.49, two and a half times the true effect, and every one of them overstates the truth by at least 0.96.

The share that clears the bar is the power: $P(\hat{\theta} > 1.96) = P(Z > 0.96) \approx 0.17$. The average published estimate is the mean of a Normal tail, which has a closed form:

$$\mathbb{E}[\hat{\theta} \mid \hat{\theta} > 1.96] = 1 + \frac{\varphi(0.96)}{1 - \Phi(0.96)} = 1 + \frac{0.2516}{0.1685} \approx 2.49.$$

Here φ is the standard Normal density (the height of the bell curve) and Φ its CDF (the area to the left of a point).

This is the **winner's curse**: an estimate selected *because* it was large is biased upward. It bites at every scale. Among 20 launched features, the ones whose A/B tests looked best will usually disappoint when measured again. Lower power makes the curse stronger, because only wild overestimates can clear the bar. Figure 11.13 repeats the calculation above for every level of power.

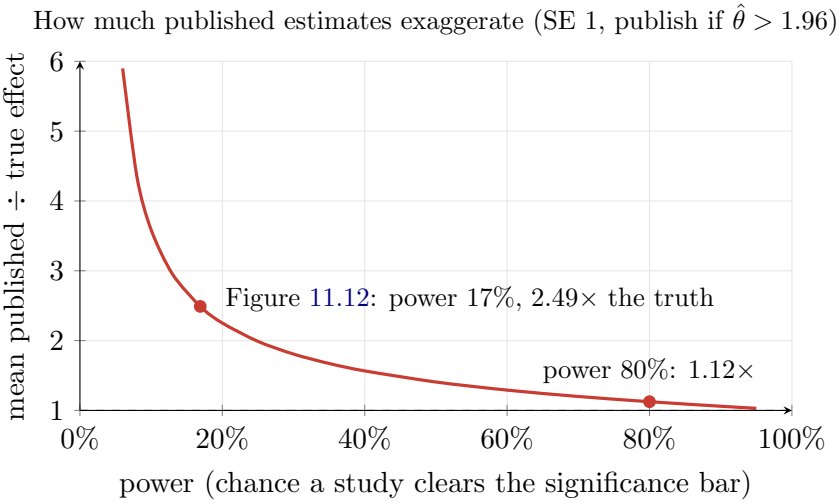


Figure 11.13: The average published estimate divided by the true effect, when only significant results are published. A well-powered study (80%) overstates by about 12%; at 17% power, as in Figure 11.12, published estimates are two and a half times the truth, and at 6% almost six times.

Common trap

If a follow-up study finds a smaller effect than the original, the usual reason is not that the effect "decayed". The original estimate was inflated by selection. Plan follow-up sample sizes using a shrunken estimate, not the headline number.

11.7 Regression to the mean

A school gives 120 students two equally hard tests a week apart. The 15 best scorers on test 1 (scores of 81 or more) averaged 84.9. On test 2 the same students averaged 76.9. The 16 worst on test 1 (58 or less) averaged 54.9 and then 60.7. A tutoring programme for the bottom group would look like a success, and praise for the top group like a disaster. Neither did anything.

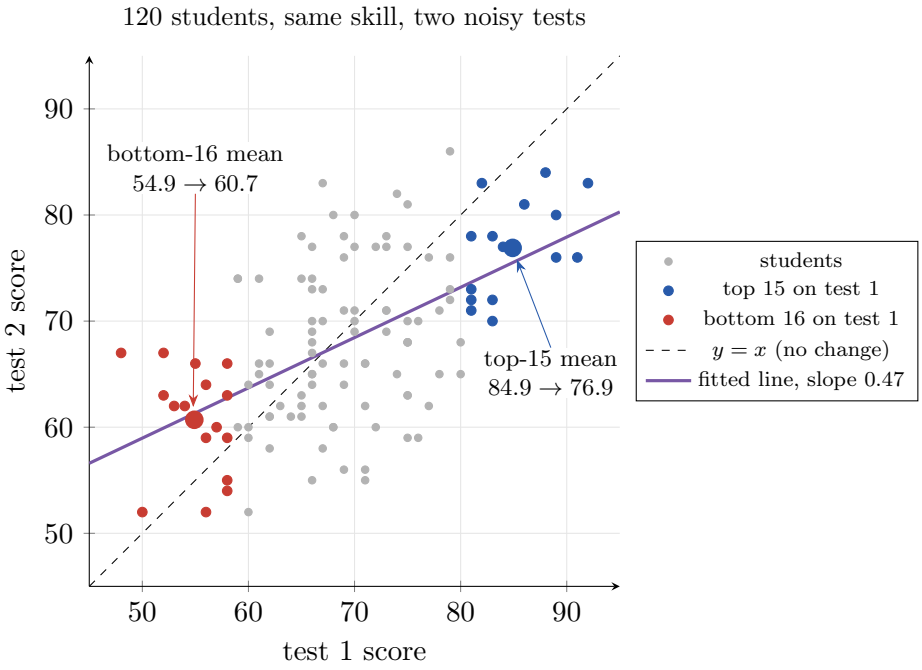


Figure 11.14: Simulated scores: each student has a fixed skill, and each test adds independent noise. Blue points (top on test 1) sit mostly *below* the diagonal, red points (bottom on test 1) mostly *above*. The fitted line is flatter than $y = x$. Extremes on the first measurement are partly luck, and luck does not repeat. The two large dots are the group means: the top group falls 8 points and the bottom group rises 6, with no intervention at all.

Why it works

Write each score as skill plus noise: $X_1 = S + \varepsilon_1$, $X_2 = S + \varepsilon_2$, with the noises independent of each other and of skill. A student who scored very high on test 1 probably has high skill *and* a lucky ε_1. On test 2 the skill carries over and the luck is redrawn with mean zero. So the expected second score is pulled back toward the average. In the simulation, skill had SD 8 and each noise SD 6, so the theoretical slope is

$$\frac{\mathrm{Var}(S)}{\mathrm{Var}(S) + \mathrm{Var}(\varepsilon)} = \frac{64}{64 + 36} = 0.64.$$

A slope below 1 is exactly "regression toward the mean". (This sample of 120 happens to give 0.47, on the low side: a slope fitted to 120 noisy points still wobbles by about ± 0.07 from sample to sample.) Chapter 12 shows that the least-squares slope always equals $r\, s_y / s_x$, and with equal spreads that is just the correlation $r < 1$.

Regression to the mean fools people whenever units are *selected because they were extreme*: the worst-performing stores get a new manager, patients enrol in a trial when symptoms peak, the rookie of the year has a "sophomore slump". The defence is a control group selected the same way. Compare treated extreme units with untreated extreme units, and the regression effect cancels.

11.8 Base rates: how many significant findings are true?

A p-value below 0.05 does *not* mean a 95% chance the effect is real. The chance depends on how many of the hypotheses you test were real to begin with. Take 1000 ideas tested by a product team. Suppose 10% truly work, the tests have 80% power, and $\alpha = 0.05$.

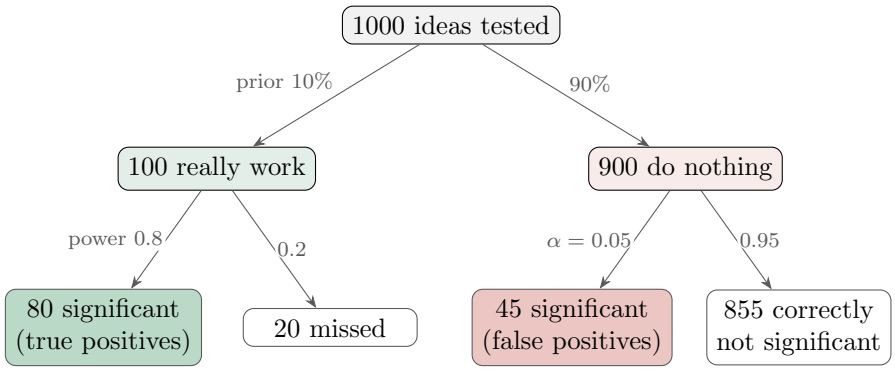


Figure 11.15: Of the $80 + 45 = 125$ significant results, only 80 are real. The chance that a significant finding is true is $80/125 = 64\%$, not 95%. The 45 false positives come from the large pool of ideas that do nothing.

Definition 11.5 (Positive predictive value)**.** The **positive predictive value** (PPV) is the probability that a significant result reflects a real effect. With prior share π of real effects, power $1 - \beta$ and level α, Bayes' rule gives

$$\text{PPV} = \frac{(1 - \beta)\,\pi}{(1 - \beta)\,\pi + \alpha\,(1 - \pi)}.$$

With $\pi = 0.1$ and power 0.8 this is $0.08/(0.08 + 0.045) = 0.64$. Cut the power to 0.2, common in small studies, and it drops to $0.02/(0.02 + 0.045) \approx 0.31$: most significant findings are false. This is the arithmetic behind John Ioannidis's 2005 paper "Why most published research findings are false". It is the same base-rate effect that makes a positive test for a rare disease usually a false alarm (see Probability From the Ground Up, the chapter on Bayes' rule).

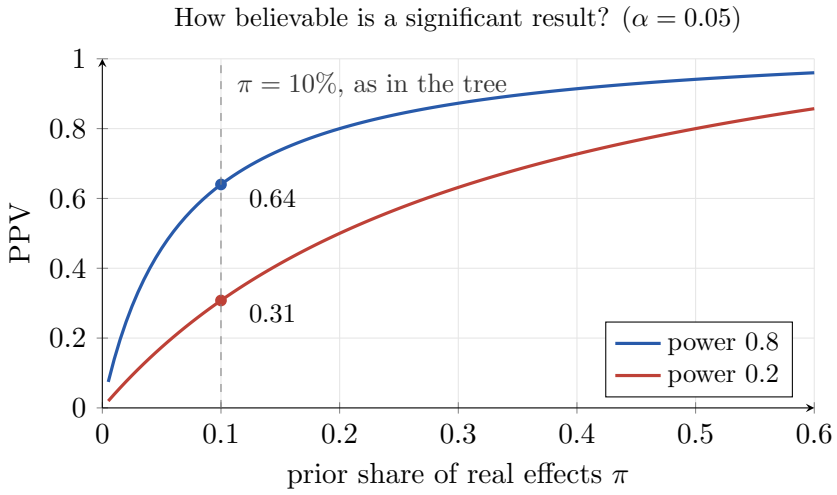


Figure 11.16: The share of significant results that are real, as a function of how plausible the tested ideas were to start with. The dots are the tree's example ($\pi = 10\%$) at two levels of power. Long-shot hypotheses tested with low power produce mostly false positives, however small the p-values look.

Connection

In machine learning the same logic applies to leaderboards. If hundreds of model variants are tried against one test set and the best is reported, its score has a winner's curse and a multiple-testing problem at once. Held-out test sets that are touched once, and fresh evaluation data for the final claim, are the field's version of pre-registration (Chapter 15).

11.9 Simpson's paradox, revisited

Chapter 4 met Simpson's paradox as a curiosity about correlation. It is also a pitfall for effect sizes. The classic data are a 1986 comparison of two kidney stone treatments.

	Small stones	Large stones	All patients
Treatment A	81/87 = 93%	192/263 = 73%	273/350 = 78%
Treatment B	234/270 = 87%	55/80 = 69%	289/350 = 83%

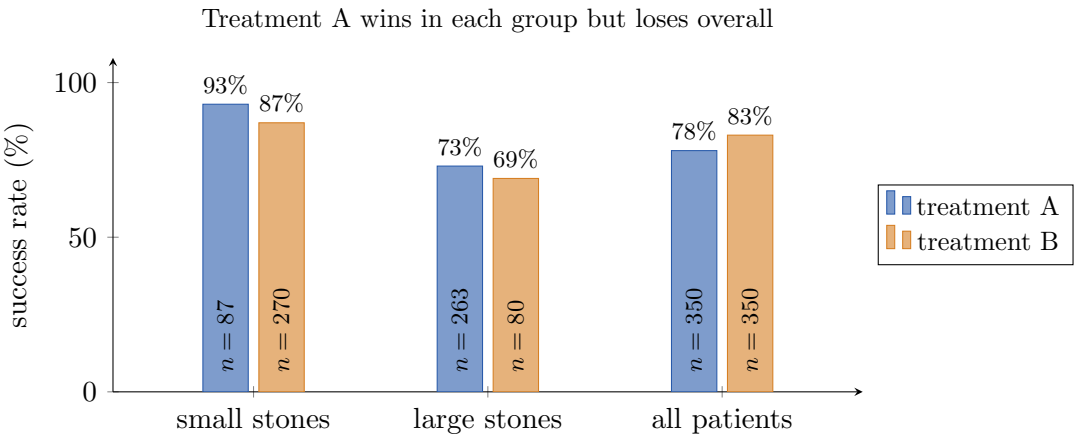


Figure 11.17: A beats B for small stones and for large stones, yet B looks better overall. Read the patient counts inside the bars: doctors gave A mostly to the hard large-stone cases (263 of 350) and B mostly to easy small ones (270 of 350), so A's overall rate is dragged down by case mix, not by the treatment.

A pooled effect size averages over a mix of groups. If the mix differs between the arms, the pooled comparison measures the mix as much as the treatment. Randomization (Chapter 5) makes the mix equal in expectation, which is the main reason randomized experiments are trusted. In observational data, compare within strata of the confounder, or adjust for it with regression (Chapter 13).

Common trap

Splitting the data is not automatically right either. If the splitting variable is *caused by* the treatment (say, "patients who finished the course"), the within-group comparison is the biased one. Which table to trust is a question about causes, not arithmetic.

11.10 Survivorship bias

During the Second World War, analysts studied bombers returning from missions and mapped where they had bullet holes, planning to armour those spots. The statistician Abraham Wald pointed out that the sample contained only planes that came back. Holes in the engines were rare among survivors because planes hit in the engines did not return. The armour belonged where the holes were *not*.

Survivorship bias is what you get when the data only include units that passed some filter linked to the outcome. Mutual funds are the textbook case. Figure 11.18 simulates 20 funds whose yearly returns are pure noise: mean 0%, SD 15%. A fund is closed and dropped from the database when its value falls below 80% of the start.

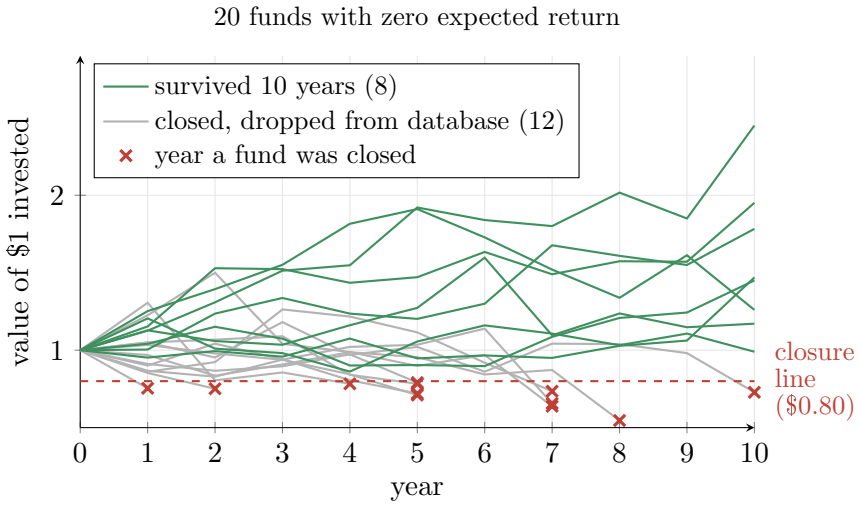


Figure 11.18: Every fund here has zero expected return; their paths are pure luck. The 12 grey funds fall below the closure line and vanish from the database (red crosses). The 8 green survivors average +4.2% a year. A "ten-year track record" study of today's funds sees only the green lines and concludes that fund managers beat the market.

Survivorship bias shows up as: startup advice drawn from successful founders; "old buildings were built better" (the badly built ones fell down); churn analyses that look only at current customers; and model training sets built from loans that were approved, so the model never sees the applicants who were turned away. The question to ask is always: *what filter decided which units I get to see, and could that filter be related to the outcome?*

11.11 A checklist for reading a statistical claim

Every pitfall in this chapter maps to one question. Run through them in order when you read a paper, a dashboard or a launch memo.

1. How big? Is there an effect size in real units, with a confidence interval? Is it bigger than the smallest effect that matters?

2. Compared with what? What is the baseline? Are ratios given with absolute numbers?

3. How many tries? How many metrics, subgroups, models or analyses were looked at? Was there a correction or a pre-registered primary metric?

4. When did they stop? Fixed sample size, a proper sequential design, or "until it was significant"?

5. Why am I seeing this? Was it selected for being striking (publication bias, winner's curse, top performers)? Has it been replicated?

6. Selected for being extreme? Could a before/after change be regression to the mean? Is there a control group?

7. How plausible beforehand? Low prior plus low power means most significant results are false.

8. Who is missing? Randomized, or could confounding (Simpson) or survivorship explain it?

Figure 11.19: Eight questions, one for each pitfall in this chapter. A claim that survives all eight deserves to be taken seriously.

Walking question

Headline: "Our new onboarding flow boosts 30-day retention by 35% ($p = 0.03$)." Running the checklist: (1) 35% relative lift on what base? From 2% to 2.7% is 0.7 points. Ask for the interval. (2) Which baseline cohort? (3) The memo shows 14 retention and engagement metrics; retention was the one that came out significant. With 14 tests the chance of at least one false positive is about 51%. (4) The test ran "until results stabilised": that is optional stopping. (5) This is the one result of several experiments that made it into the memo. Verdict: interesting, not established. Rerun with retention as the pre-registered primary metric and a fixed sample size.

11.12 Summary

Idea	Key fact
Practical significance	Large n makes tiny effects significant. Judge the CI against the smallest effect that matters.
Effect sizes	Difference in natural units with CI first; Cohen's d for unitless scales; for rates give absolute difference and RR/OR together.
Report CIs	An interval contains the test result plus size and precision.
Multiple testing	FWER $= 1-(1-\alpha)^m$. Bonferroni (α/m) and Holm control FWER; BH controls FDR via $p_{(k)} \le qk/m$.
Forking paths	Data-dependent analysis choices are hidden multiple tests. Pre-register.
Optional stopping	Stopping at first $p < 0.05$ inflates false positives (14% at 5 peeks, 38% at 100). Use sequential boundaries.
Winner's curse	Estimates selected for significance are inflated, badly so at low power.
Regression to the mean	Units selected for being extreme drift back. Needs a control group.
PPV	$\text{PPV} = \dfrac{(1-\beta)\pi}{(1-\beta)\pi + \alpha(1-\pi)}$; low prior and low power make most hits false.
Simpson, survivorship	Pooled comparisons and filtered samples can reverse or invent effects.

Chapter 12 moves from comparing groups to modelling a relationship: the regression line, whose slope below one you have already seen in the regression-to-the-mean picture.

11.13 Exercises

1. An A/B test with $n = 200{,}000$ per arm finds conversion 5.00% versus 5.05%. Compute the 95% CI for the difference (unpooled SE). Is it significant? Would you ship if the smallest effect of interest is 0.2 points?

2. Two groups of 30 have means 82 and 76 and SDs 10 and 14. Compute Cohen's d with the pooled SD.

3. A drug raises the rate of a side effect from 30% to 45%. Give the absolute difference, the risk ratio and the odds ratio. Which number would a headline writer prefer?

4. How many independent tests at $\alpha = 0.05$ make the chance of at least one false positive exceed 90%?

5. Apply Bonferroni, Holm and BH ($q = 0.05$) to the six p-values 0.002, 0.009, 0.012, 0.030, 0.041, 0.380. How many does each reject?

6. In the peeking simulation, the false positive rate with 5 peeks was about 14%. Explain in two sentences why it is not $1 - 0.95^5 \approx 23\%$.

7. A field tests hypotheses of which 5% are true, with power 0.5 and $\alpha = 0.05$. What fraction of significant findings are true?

8. A city installs speed cameras at the 10 junctions with the most crashes last year. Next year crashes there fall by 30%. Give a reason to doubt that the cameras caused all of it, and describe a better comparison.

9. An article reports that the average customer who has stayed 5 years spends more per month than new customers, and concludes that customers spend more the longer they stay. Name the bias and explain.

Answers

1. Difference 0.05 points; SE $= \sqrt{0.05 \cdot 0.95/2 \times 10^5 + 0.0505 \cdot 0.9495/2 \times 10^5} \approx 0.000691$, so CI $\approx 0.05 \pm 0.135 = [-0.085,\ 0.185]$ points. Not significant ($z \approx 0.72$). The whole interval lies below 0.2, so even the most optimistic plausible lift is too small: do not ship. This is case (c), informative.

2. $s_p = \sqrt{(29 \cdot 100 + 29 \cdot 196)/58} = \sqrt{148} \approx 12.17$, so $d = 6/12.17 \approx 0.49$.

3. Difference 15 points; RR = 1.5; OR $= (0.45/0.55)/(0.30/0.70) \approx 1.91$. The headline writer prefers the OR ("nearly doubles the odds"), which is easily misread as "nearly twice as likely".

4. $1 - 0.95^m > 0.9$ means $m > \ln 0.1 / \ln 0.95 \approx 44.9$, so $m = 45$.

5. Bonferroni ($0.05/6 \approx 0.00833$): rejects 0.002 only, so 1. Holm: $0.002 \le 0.00833$, $0.009 \le 0.01$, $0.012 \le 0.0125$, then $0.030 > 0.01667$ stops, so 3. BH thresholds are 0.00833, 0.0167, 0.025, 0.0333, 0.0417, 0.05; the last p-value under its threshold is $0.041 \le 0.0417$ at $k = 5$, so 5 rejected.

6. The five looks share data: the z at look 3 contains all the observations of looks 1 and 2, so the tests are strongly positively correlated. Correlated tests overlap in when they fire, so the union is smaller than the independent formula predicts.

7. PPV $= 0.5 \cdot 0.05/(0.5 \cdot 0.05 + 0.05 \cdot 0.95) = 0.025/0.0725 \approx 0.34$.

8. Junctions were selected for an extreme year, so crashes would regress toward their long-run mean even with no cameras. Compare with similar high-crash junctions that did not get cameras (ideally chosen at random from the high-crash list), or use several years of history before selection.

9. Survivorship bias. Low-spending customers are more likely to churn, so the 5-year group is filtered toward high spenders. Individual customers may not change their spending at all.

Part IV

Models

Chapter 12

Simple Linear Regression

Here are six file uploads. For each one we record the file size x (in MB) and the upload time y (in seconds).

upload i	1	2	3	4	5	6
size x_i (MB)	1	2	3	4	5	6
time y_i (s)	3	5	4	7	8	9

Bigger files take longer, but not perfectly: the 3 MB file was faster than the 2 MB file. Chapters 9 to 11 compared groups: version A against version B, one mean against another. Here x is not a group label but a number that can take any value, and we want to know how y changes as x slides. Chapter 4 measured the strength of such a relationship with the correlation r, a number between -1 and 1 that says how tightly the points hug a line. This chapter goes further. We fit a line, use it to predict, say how much of the variation it explains, and put error bars on its slope. That is **simple linear regression**: one predictor x, one response y.

12.1 The model

Definition 12.1 (Simple linear regression model)**.** The response is a straight-line function of x plus noise:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \qquad i = 1, \ldots, n,$$

where the errors ε_i are independent, have mean 0, and have the same variance σ^2. Often we also assume $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$. The unknown parameters are the **intercept** β_0, the **slope** β_1, and the **noise level** σ.

The model says: at each fixed x, the response has a distribution. Its mean sits on the line $\beta_0 + \beta_1 x$, and its spread is σ no matter where you are. Figure 12.1 draws this picture. The line is the *average* upload time for a file of a given size. Individual uploads scatter around it because of network jitter, server load, and everything else we did not measure.

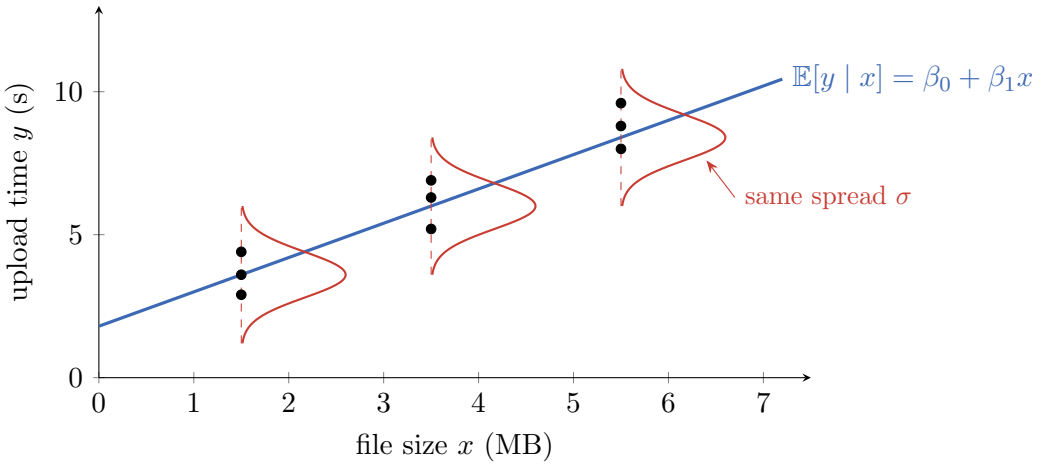


Figure 12.1: The regression model. At every x the response is a random variable centred on the line, with the same spread σ (red bells). Observed points (black) are single draws from these bells.

Intuition

Read β_1 as "how much the *average* y moves when x goes up by one unit." Read σ as "how far a typical point lands from that average." Regression estimates both. A small slope with small σ can be a clearer signal than a large slope buried in large σ.

We never see β_0, β_1, σ. We see n points. Our job is to compute estimates b_0, b_1, s from the data and then say how far off they might be. (Many texts, and Chapters 13–15, write $\hat{\beta}_0, \hat{\beta}_1$ for b_0, b_1; the hat means "estimated from data".) The fitted line is written

$$\hat{y} = b_0 + b_1 x.$$

The hat means "predicted value." For the i-th point, $\hat{y}_i = b_0 + b_1 x_i$.

12.2 Residuals and the least squares criterion

Definition 12.2 (Residual)**.** The **residual** of point i is the observed value minus the predicted value:

$$e_i = y_i - \hat{y}_i = y_i - (b_0 + b_1 x_i).$$

Geometrically it is the signed *vertical* distance from the point to the line.

A positive residual means the point lies above the line (the model under-predicted). Negative means below. Figure 12.2 shows the residuals of our six uploads for the line we are about to derive.

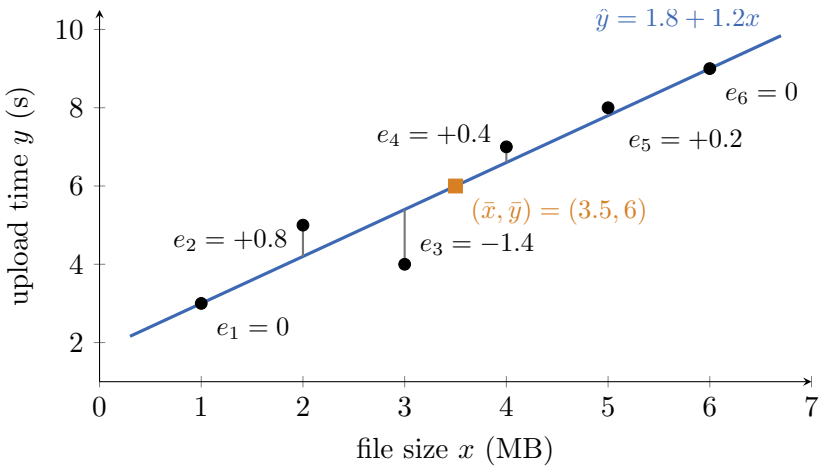


Figure 12.2: Each residual is the vertical gap from a point up or down to the fitted line, labelled with its signed size. Points 1 and 6 sit exactly on the line ($e = 0$); the others miss by at most 1.4 s. The orange square is the centroid $(\bar{x}, \bar{y})$; the least squares line always passes through it.

Common trap

Residuals are *vertical*, not perpendicular. We are predicting y from x, so the error that matters is the error in y at the given x. Minimising perpendicular distances gives a different line (total least squares, or the first principal component), which answers a different question.

Which line is "best"? We want residuals small as a whole. Their plain sum is useless: big positive and big negative residuals cancel. The standard choice is to square them and add.

Definition 12.3 (Least squares)**.** The **least squares** line is the choice of (b_0, b_1) that minimises the sum of squared errors

$$\mathrm{SSE}(b_0, b_1) = \sum_{i=1}^{n} \big(y_i - b_0 - b_1 x_i\big)^2.$$

Squaring has a literal picture. Each squared residual is the area of a square whose side is the residual. Least squares finds the line with the smallest total square area (Figure 12.3).

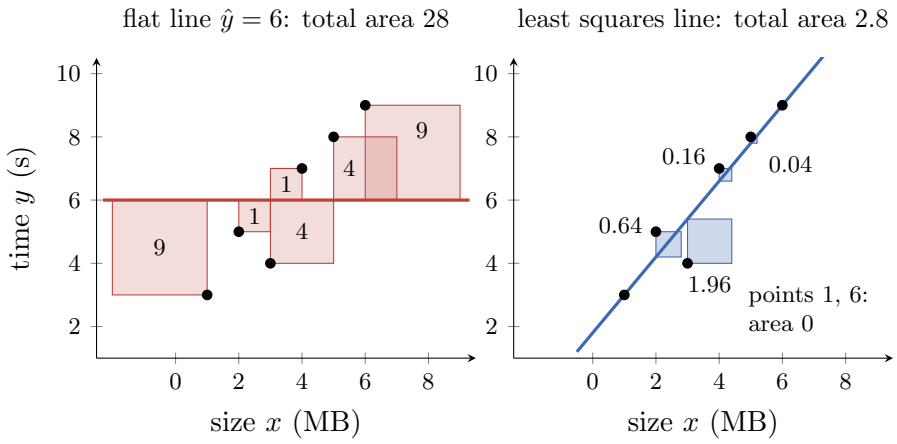


Figure 12.3: Each squared residual drawn as an actual square, with its area written on it. Left: the flat line at $\bar{y} = 6$ leaves big squares (areas $9+1+4+1+4+9 = 28$; some overlap because the points are close together). Right: the least squares line leaves only four small squares, total area 2.8. No other line does better.

Why it works

Why squares and not absolute values? Three reasons. (1) Squares make SSE a smooth bowl in (b_0, b_1), so calculus gives a closed-form answer. (2) The answer is built from means, variances and covariances, the same quantities as Chapters 2 and 4. (3) If the noise is Gaussian, least squares is exactly maximum likelihood (Section 12.13). Minimising absolute residuals instead (least absolute deviations) is more robust to outliers, just as the median is more robust than the mean, but has no closed form.

12.3 Deriving the least squares line

We need some shorthand. For data $(x_1, y_1), \dots, (x_n, y_n)$ define

$$S_{xx} = \sum_{i=1}^{n} (x_i - \bar{x})^2, \qquad S_{yy} = \sum_{i=1}^{n} (y_i - \bar{y})^2, \qquad S_{xy} = \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}).$$

These are the un-normalised versions of the variances and covariance: $s_x^2 = S_{xx}/(n-1)$, $s_y^2 = S_{yy}/(n-1)$, and the sample covariance is $S_{xy}/(n-1)$. The correlation from Chapter 4 is $r = S_{xy}/\sqrt{S_{xx}S_{yy}}$.

Theorem 12.1 (Least squares estimates)**.** *If the x_i are not all equal,* $\mathrm{SSE}(b_0, b_1)$ *has a unique minimiser*

$$b_1 = \frac{S_{xy}}{S_{xx}} = r\,\frac{s_y}{s_x}, \qquad b_0 = \bar{y} - b_1\,\bar{x}.$$

Proof. SSE is a quadratic in b_0 and b_1 that opens upward, so its minimum is where both partial derivatives vanish.

Intercept. Differentiate with respect to b_0:

$$\frac{\partial\,\mathrm{SSE}}{\partial b_0} = -2\sum_{i=1}^{n}(y_i - b_0 - b_1 x_i) = 0 \quad \Longrightarrow \quad \sum y_i = n b_0 + b_1 \sum x_i.$$

Divide by n: $\bar{y} = b_0 + b_1\bar{x}$. So $b_0 = \bar{y} - b_1\bar{x}$.

Slope. Differentiate with respect to b_1:

$$\frac{\partial \,\mathrm{SSE}}{\partial b_1} = -2\sum_{i=1}^{n} x_i(y_i - b_0 - b_1 x_i) = 0.$$

Substitute $b_0 = \bar{y} - b_1\bar{x}$, so that $y_i - b_0 - b_1 x_i = (y_i - \bar{y}) - b_1(x_i - \bar{x})$:

$$\sum_{i=1}^{n} x_i(y_i - \bar{y}) = b_1 \sum_{i=1}^{n} x_i(x_i - \bar{x}).$$

Because $\sum(y_i - \bar{y}) = 0$ and $\sum(x_i - \bar{x}) = 0$, we may replace the leading x_i by $x_i - \bar{x}$ on both sides without changing either sum. That gives $S_{xy} = b_1 S_{xx}$, so $b_1 = S_{xy}/S_{xx}$. We need $S_{xx} > 0$, which holds exactly when the x_i are not all equal.

The r form. Since $s_y/s_x = \sqrt{S_{yy}/S_{xx}}$,

$$r\,\frac{s_y}{s_x} = \frac{S_{xy}}{\sqrt{S_{xx}S_{yy}}} \cdot \frac{\sqrt{S_{yy}}}{\sqrt{S_{xx}}} = \frac{S_{xy}}{S_{xx}} = b_1.$$

□

The two equations set to zero are called the **normal equations**. Written in terms of residuals they say

$$\sum_{i=1}^{n} e_i = 0, \qquad \sum_{i=1}^{n} x_i e_i = 0.$$

Residuals of a least squares fit always sum to zero and are uncorrelated with x. We will use both facts repeatedly.

Pause

Pause and think: if every y_i is shifted up by 10, what happens to b_1 and b_0? (Answer: S_{xy} is unchanged, so b_1 is unchanged; $\bar{y}$ rises by 10, so b_0 rises by 10. The line just moves up.)

Intuition

The intercept equation $\bar{y} = b_0 + b_1\bar{x}$ says the point $(\bar{x}, \bar{y})$ lies on the fitted line. The least squares line always passes through the **centroid** of the data. So fitting reduces to one question: through the centroid, which way should the line tilt? The slope formula $b_1 = r\, s_y/s_x$ answers it: tilt by r standard deviations of y per standard deviation of x.

Because the line pivots on the centroid, SSE as a function of slope alone is $\sum\big((y_i - \bar{y}) - b(x_i - \bar{x})\big)^2 = S_{yy} - 2bS_{xy} + b^2 S_{xx}$, a parabola with vertex at $b = S_{xy}/S_{xx}$. Figure 12.4 plots it for the upload data.

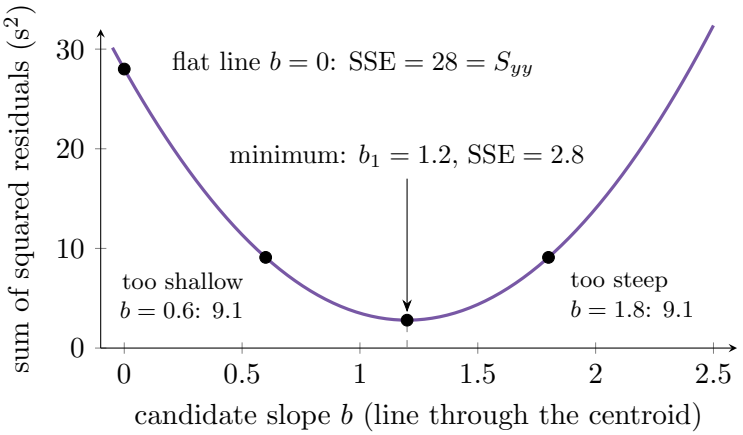


Figure 12.4: Each point on the curve is one candidate line through the centroid, scored by its sum of squared residuals, $\text{SSE}(b) = 28 - 42b + 17.5b^2$. The flat line ($b = 0$) is the left panel of Figure 12.3; the bottom of the parabola ($b = 1.2$) is the right panel. Tilting too little or too much costs the same.

12.4 A worked example by hand

Walking question

Fit the least squares line to the six uploads.
Step 1: means. $\bar{x} = 21/6 = 3.5$ MB and $\bar{y} = 36/6 = 6$ s.
Step 2: deviations and products.

x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})^2$	$(x_i - \bar{x})(y_i - \bar{y})$	$(y_i - \bar{y})^2$
1	3	−2.5	−3	6.25	7.5	9
2	5	−1.5	−1	2.25	1.5	1
3	4	−0.5	−2	0.25	1.0	4
4	7	0.5	1	0.25	0.5	1
5	8	1.5	2	2.25	3.0	4
6	9	2.5	3	6.25	7.5	9
		0	0	$S_{xx} = 17.5$	$S_{xy} = 21$	$S_{yy} = 28$

Step 3: slope and intercept.

$$b_1 = \frac{S_{xy}}{S_{xx}} = \frac{21}{17.5} = 1.2, \qquad b_0 = \bar{y} - b_1\bar{x} = 6 - 1.2 \times 3.5 = 1.8.$$

The fitted line is $\hat{y} = 1.8 + 1.2x$.
Step 4: check with r. $s_x = \sqrt{17.5/5} = 1.871$, $s_y = \sqrt{28/5} = 2.366$, $r = 21/\sqrt{17.5 \times 28} = 21/\sqrt{490} = 0.9487$. Then $r\, s_y/s_x = 0.9487 \times 2.366/1.871 = 1.2$. Same slope.
Step 5: fitted values and residuals.

x_i	y_i	$\hat{y}_i = 1.8 + 1.2x_i$	$e_i = y_i - \hat{y}_i$	e_i^2
1	3	3.0	0.0	0.00
2	5	4.2	0.8	0.64
3	4	5.4	-1.4	1.96
4	7	6.6	0.4	0.16
5	8	7.8	0.2	0.04
6	9	9.0	0.0	0.00
			$\sum e_i = 0$	SSE $= 2.80$

The residuals sum to zero, as the normal equations promise. So does the second normal equation:

$$\sum x_i e_i = 2(0.8) + 3(-1.4) + 4(0.4) + 5(0.2) = 0.$$

In practice one function call does all of this.

```
import numpy as np
from scipy import stats
x = np.array([1, 2, 3, 4, 5, 6]); y = np.array([3, 5, 4, 7, 8, 9])
fit = stats.linregress(x, y)
print(fit.slope, fit.intercept)   # 1.2  1.8
print(fit.rvalue**2)              # 0.9  (R^2, Section 12.6)
print(fit.stderr, fit.pvalue)     # 0.2  0.0039 (Section 12.9)
```

12.5 Interpreting slope and intercept

Slope. $b_1 = 1.2$ s/MB. Each extra megabyte is associated with 1.2 more seconds of upload time *on average*. The units of the slope are always (units of y) per (unit of x). Change x from MB to KB and the slope becomes 0.0012 s/KB. The fit is identical; only the units moved.

Intercept. $b_0 = 1.8$ s is the predicted time at $x = 0$: a zero-byte upload. Here that reading makes sense: it is fixed overhead (connection setup, handshake). Often it does not. Regress adult weight on height and the intercept is the predicted weight of a person 0 cm tall, which is meaningless. The intercept is still needed to place the line; just do not interpret it outside the range of the data.

Common trap

"Associated with" is not "causes." A regression slope summarises how y and x move together in *these data*. If the data come from an observational study, a confounder could drive both (Chapter 5). Only a randomised experiment, where x is set by the experimenter, licenses reading b_1 as "if I increase x by one, y goes up by b_1." File size is a reasonable causal story here, but the regression alone does not prove it.

12.6 How good is the fit? R^2

Before seeing x, the best constant prediction of y is $\bar{y}$, with total squared error S_{yy}. After fitting the line, the squared error drops to SSE. The fraction of the original squared error that the line removed measures how useful x is.

Definition 12.4 (Sums of squares and R^2)**.**

$$\begin{aligned} \text{SST} &= \sum (y_i - \bar{y})^2 = S_{yy} && \text{(total)},\\ \text{SSR} &= \sum (\hat{y}_i - \bar{y})^2 && \text{(regression, “explained”)},\\ \text{SSE} &= \sum (y_i - \hat{y}_i)^2 && \text{(error, “unexplained”)}. \end{aligned}$$

The **coefficient of determination** is $R^2 = \text{SSR}/\text{SST} = 1 - \text{SSE}/\text{SST}$.

Theorem 12.2 (Sum of squares decomposition)**.** *For a least squares fit with an intercept,* $\text{SST} = \text{SSR} + \text{SSE}$*, and* $R^2 = r^2$*.*

Proof. Split each deviation: $y_i - \bar{y} = (\hat{y}_i - \bar{y}) + e_i$. Square and sum:

$$\text{SST} = \text{SSR} + \text{SSE} + 2\sum_{i=1}^{n} (\hat{y}_i - \bar{y}) e_i.$$

Since $\hat{y}_i - \bar{y} = b_1(x_i - \bar{x})$, the cross term is $2b_1\big(\sum x_i e_i - \bar{x}\sum e_i\big)$, and both sums vanish by the normal equations. For the second claim, $\text{SSR} = b_1^2 S_{xx} = S_{xy}^2/S_{xx}$, so

$$R^2 = \frac{S_{xy}^2}{S_{xx} S_{yy}} = r^2. \qquad \square$$

Figure 12.5 shows the split for a single point. The cross term vanishing is a Pythagorean theorem: the vector of residuals is perpendicular to the vector of fitted deviations.

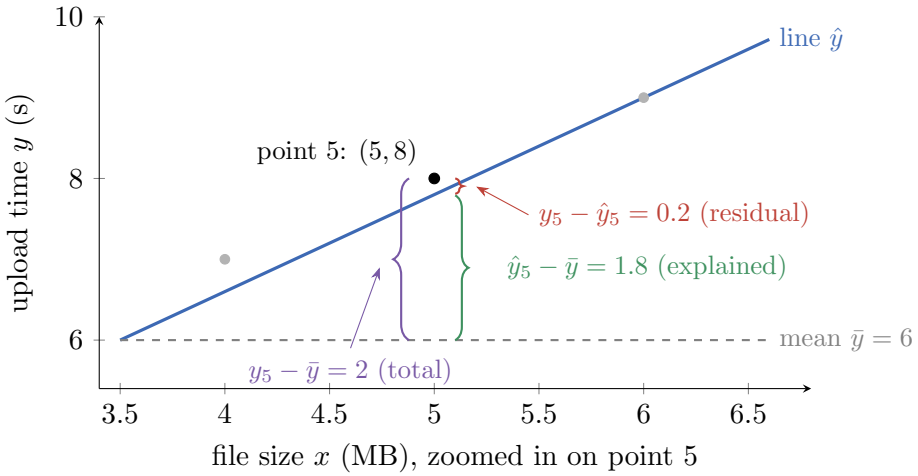


Figure 12.5: Zoomed in on the upload at $(5, 8)$. Its total distance above the mean (purple, 2) splits into the part the line explains (green, 1.8) plus the residual the line misses (red, 0.2). Square these pieces and add over all six points to get $\text{SST} = \text{SSR} + \text{SSE}$.

For the uploads: SST = 28, SSE = 2.8, so SSR = 25.2 and

$$R^2 = \frac{25.2}{28} = 0.90 = (0.9487)^2 = r^2.$$

File size explains 90% of the variation in upload time. The other 10% is network noise and anything else we did not record.

Pause

Pause and think: what is R^2 if the fitted slope is exactly 0? And if every point lies on the line? (Answers: 0, because then $\hat{y}_i = \bar{y}$ and SSR = 0; and 1, because SSE = 0.)

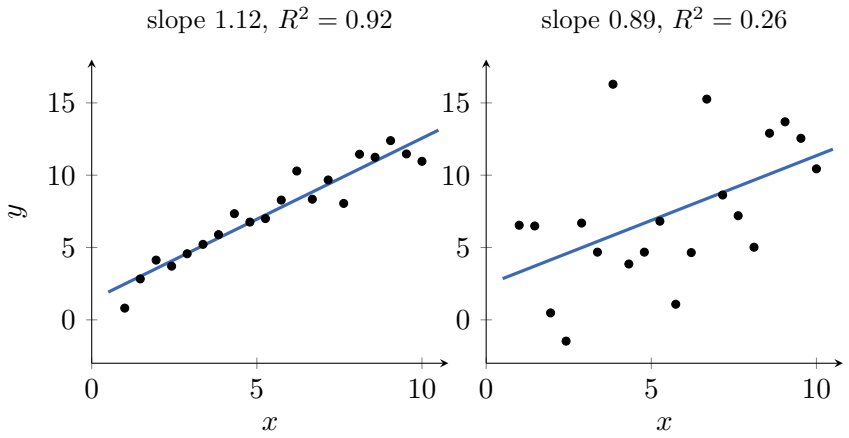


Figure 12.6: Similar slopes (titles), very different R^2. Both panels use the same axes. On the left the points hug the line; on the right the trend is real but most of the variation in y comes from noise that x does not explain.

Common trap

R^2 is not a grade for the model. A low R^2 can come with a precisely estimated, important slope (a drug that lowers blood pressure by 5 mmHg on average, in a population where blood pressure varies by 20). A high R^2 can come with a wrong model (a straight line through a curve, Section 12.8). R^2 also says nothing about causation, and it depends on the range of x: sample a wider range of x and R^2 goes up even though σ is unchanged.

12.7 Regression to the mean

Write the fitted line in standard units. Let $z_x = (x - \bar{x})/s_x$ and $z_y = (y - \bar{y})/s_y$. Using $b_1 = r\, s_y/s_x$ and the fact that the line passes through the centroid,

$$\hat{z}_y = r\, z_x.$$

Since $|r| \le 1$, the prediction is always *closer to the mean* (in SD units) than the input. A point 2 SDs above average in x is predicted to be only $2r$ SDs above average in y.

Example 12.1 (Parents and children)**.** Fifty parent–adult child pairs (simulated, heights in cm) have $\bar{x} = 170.0$, $s_x = 6.16$ for parents, $\bar{y} = 170.6$, $s_y = 6.09$ for children, and $r = 0.34$. The slope is $b_1 = r\, s_y/s_x = 0.339$, so the fitted line is $\hat{y} = 113.0 + 0.339x$. A parent at 182 cm is $(182 - 170.0)/6.16 = 1.95$ SDs above average. Predicted child height: $170.6 + 0.339 \times (182 - 170.0) \approx 174.6$ cm, only $0.34 \times 1.95 = 0.67$ SDs above the children's average.

History

In 1805 Adrien-Marie Legendre was trying to pin down the orbits of comets from more observations than unknowns: the equations could not all hold at once because every measurement had error. He proposed choosing the answer that makes the sum of squared mismatches smallest, and called it the "method of least squares." Carl Friedrich Gauss published it in 1809, claimed he had used it since 1795, and showed it is the best choice when errors follow the bell curve that now carries his name (Section 12.13). The word "regression" came eighty years later. Francis Galton measured the heights of 928 adult children and their parents and found in 1886 that children of unusually tall or short parents were, on average, closer to the middle. He called it "regression towards mediocrity" ("mediocrity" meaning the average). The name stuck to the whole method of fitting lines, even though the effect it describes is a property of imperfect correlation, not of lines.

Nothing biological pulls children toward the average. Tall parents are tall partly because of genes they pass on and partly because of luck that they do not. The luck does not repeat, so on average the children are less extreme.

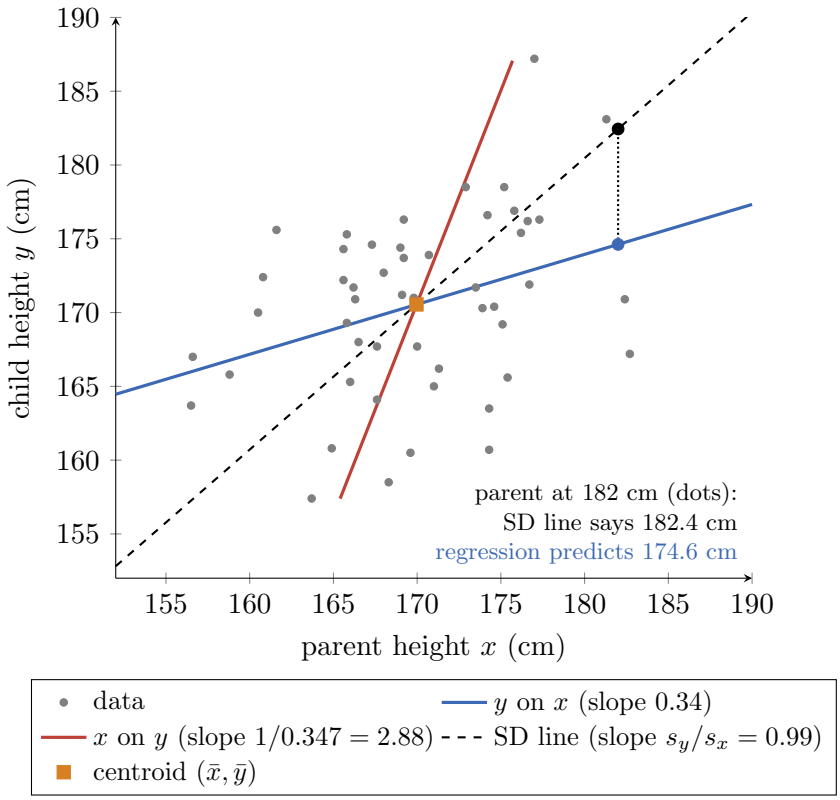


Figure 12.7: Two regression lines for the same data. Blue predicts child from parent; red predicts parent from child (redrawn in these axes). Both pass through the centroid (orange) and both are flatter, *in their own direction*, than the dashed SD line. The dots at a parent height of 182 cm show the effect: one SD-for-one-SD would give a child of 182.4 cm, but the regression line predicts only 174.6 cm. That pull toward the average is regression to the mean.

Why y on x differs from x on y. Regressing x on y minimises *horizontal* residuals and gives $x = c_0 + c_1 y$ with $c_1 = r\, s_x/s_y$. If you solve this for y, its slope in the (x, y) plane is $1/c_1 = s_y/(r\, s_x)$. The y-on-x slope is $r\, s_y/s_x$. These agree only when $r = \pm 1$. For $|r| < 1$ the two lines sit on either side of the SD line (Figure 12.7). For the uploads, x on y gives $\hat{x} = -1 + 0.75y$, i.e. $y = 1.33 + 1.33x$ when rearranged, steeper than $1.8 + 1.2x$.

Common trap

Do not invert a regression line to predict x from y. If you want to predict parent height from child height, fit the regression of x on y. Each line is the best predictor in one direction only.

What is really happening

Chapter 11 showed the practical cost: whenever units are selected for being extreme (the worst-performing servers this week, the lowest pretest scores), they drift back toward average with no intervention at all, and a slope of $r < 1$ is the reason. The fix is a control group selected the same way (Chapter 5).

12.8 Residual plots and diagnostics

b_0, b_1 and R^2 can be computed for any cloud of points. Whether a straight line is a sensible summary is a separate question, and the residuals answer it. The basic tool is the **residual plot**: residuals e_i on the vertical axis against fitted values $\hat{y}_i$ (or against x_i; for one predictor $\hat{y}_i$ is a linear function of x_i, so the two plots have the same pattern).

If the model is right, the residuals are just noise: a horizontal band centred on 0, with no pattern and constant width.

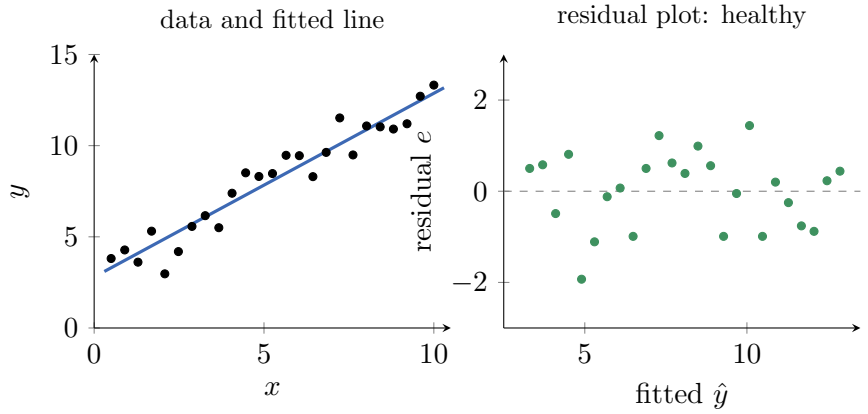


Figure 12.8: A model that fits. The residual plot is a patternless horizontal band around zero with roughly constant width.

Three patterns signal trouble.

Curvature. A U or arch in the residuals means the true relationship is not a line. The line is too high in the middle and too low at the ends (or the reverse). Figure 12.9 shows a quadratic relationship fit by a line. The fit has $R^2 = 0.92$, which looks excellent, yet the residuals reveal the model is wrong. Fix: transform x or y, or add an x^2 term (Chapter 13).

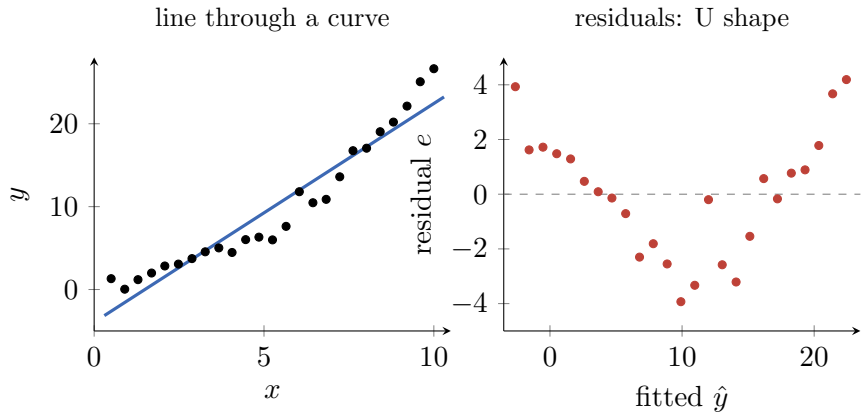


Figure 12.9: Curvature. The scatter plot hides it (the line "looks fine" and $R^2 = 0.92$), but the residual plot shows a clear U: positive, then negative, then positive.

Fan shape (heteroscedasticity). Residuals spread out as $\hat{y}$ grows. The noise is not constant: big files have more variable upload times than small ones. The line itself may be fine, but $\mathrm{SE}(b_1)$ and every interval in Section 12.9 assume constant σ and will be wrong. Fix: model $\log y$, use weighted least squares, or use robust ("sandwich") standard errors.

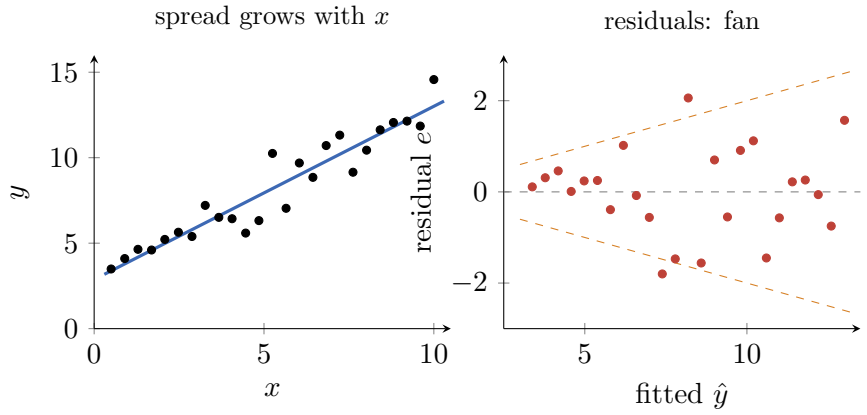


Figure 12.10: Fan shape. Residuals are tight for small fitted values and wide for large ones (orange guides). The constant-variance assumption fails.

Outliers, leverage, and influence. These are three different things.

Definition 12.5 (Outlier, leverage, influence)**.** An **outlier** is a point with a large residual: its y is far from the line. A point has high **leverage** if its x is far from $\bar{x}$; the leverage of point i is

$$h_i = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{S_{xx}}.$$

A point is **influential** if deleting it changes the fitted line a lot. Influence needs both: high leverage *and* a y value off the line the other points suggest.

Figure 12.11 adds one extra point to ten well-behaved points (base fit $\hat{y} = 1.49 + 0.741x$) in three ways.

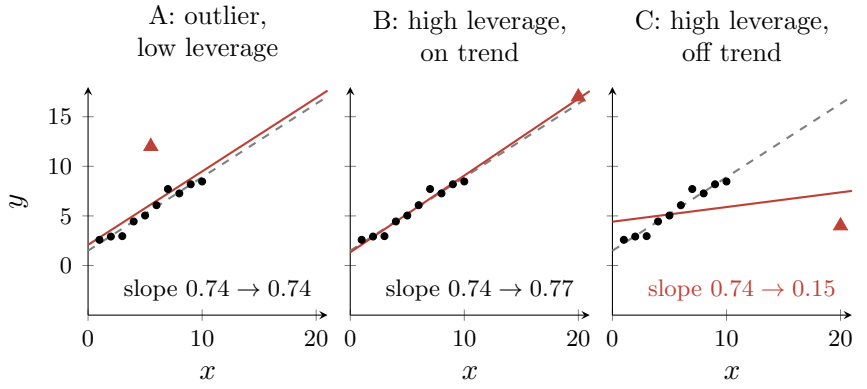


Figure 12.11: Dashed grey: fit to the ten black points. Red: fit after adding the red triangle. A (outlier near $\bar{x}$) only lifts the line; slope stays 0.741. B (far in x but on trend) barely moves it; slope 0.774. C (far in x and off trend) drags the slope from 0.741 to 0.148. Only C is influential.

Why it works

Why does a far-out x give a point so much pull? The slope is a weighted average of the points' individual slopes from the centroid:

$$b_1 = \sum_{i=1}^{n} \frac{(x_i - \bar{x})^2}{S_{xx}} \cdot \frac{y_i - \bar{y}}{x_i - \bar{x}},$$

with weight proportional to $(x_i - \bar{x})^2$ (a point with $x_i = \bar{x}$ gets weight 0 and is left out of the sum). A point ten units from $\bar{x}$ gets a hundred times the weight of a point one unit away. It acts like a long lever arm on a seesaw pivoting at the centroid.

A standard numerical summary of influence is **Cook's distance**, which measures how much all fitted values move when point i is deleted; values near or above 1 deserve a look. Never delete a point just because it is influential. Find out why it is different: a data-entry error, a different population, or the most informative observation you have.

12.9 Inference for the slope

The slope $b_1 = 1.2$ came from six uploads. Six different uploads would give a different slope. Just as $\bar{x}$ has a sampling distribution (Chapter 6), so does b_1. Figure 12.12 simulates it: twenty samples of six uploads at the same file sizes, from a true line $y = 2 + 1.2x$ with $\sigma = 0.8$.

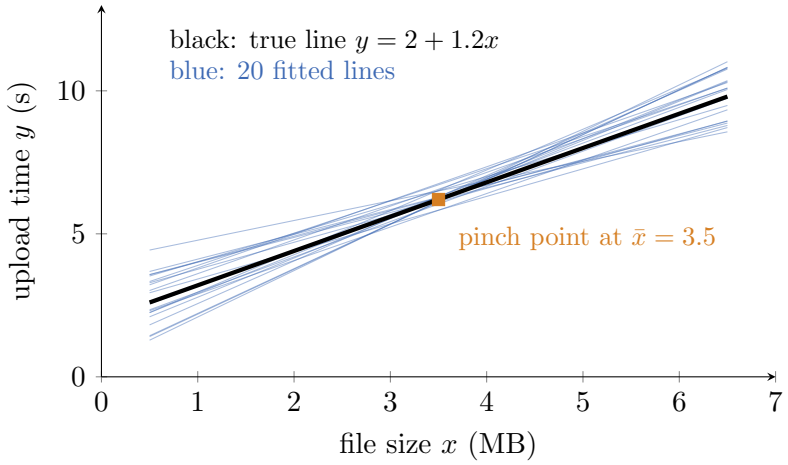


Figure 12.12: Twenty least squares lines, each fitted to a fresh sample of six points from the same model. The lines wobble around the truth. They pinch together near $\bar{x} = 3.5$ (orange) and fan out at the ends, because a slope error matters more far from the pivot.

Theorem 12.3 (Sampling distribution of b_1)**.** *Treat the x_i as fixed and assume the model of Section 12.1. Then*

$$\mathbb{E}[b_1] = \beta_1, \qquad \mathrm{SD}(b_1) = \frac{\sigma}{\sqrt{S_{xx}}}.$$

If the errors are Normal, b_1 is exactly Normal.

Proof. Because $\sum(x_i - \bar{x})\bar{y} = 0$, we can write $b_1 = \sum_i w_i y_i$ with weights $w_i = (x_i - \bar{x})/S_{xx}$. These weights satisfy $\sum w_i = 0$, $\sum w_i x_i = 1$, and $\sum w_i^2 = 1/S_{xx}$. Substituting $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$,

$$b_1 = \beta_0 \sum w_i + \beta_1 \sum w_i x_i + \sum w_i \varepsilon_i = \beta_1 + \sum w_i \varepsilon_i.$$

The noise term has mean 0, so $\mathbb{E}[b_1] = \beta_1$. Independent errors with variance σ^2 give $\mathrm{Var}(b_1) = \sigma^2 \sum w_i^2 = \sigma^2/S_{xx}$. A linear combination of independent Normals is Normal. □

The formula $\sigma/\sqrt{S_{xx}}$ tells you how to design an experiment. The slope is pinned down better when the noise σ is small, when there are more points (each adds to S_{xx}), and when the x values are spread out. Testing upload time only on files between 3.4 and 3.6 MB would tell you almost nothing about the slope. Note $S_{xx} = (n-1)s_x^2$, so $\mathrm{SD}(b_1) = \sigma/(s_x\sqrt{n-1})$: the familiar $1/\sqrt{n}$ again. Figure 12.13 shows the difference spread makes.

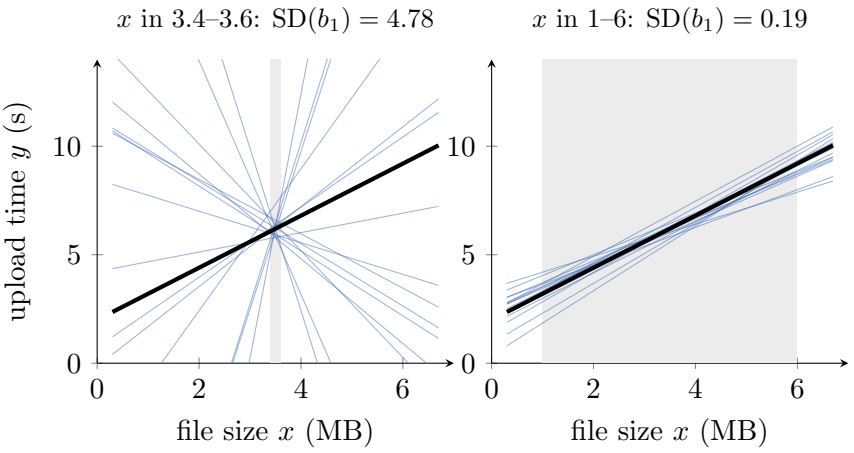


Figure 12.13: Same true line (black), same noise $\sigma = 0.8$, six uploads per sample, fifteen samples; only the choice of file sizes (grey band) differs. Squeezed into 3.4–3.6 MB (left), the fitted slopes (blue) point almost anywhere, from −7.5 to 12.2. Spread over 1–6 MB (right), every fitted line stays close to the truth.

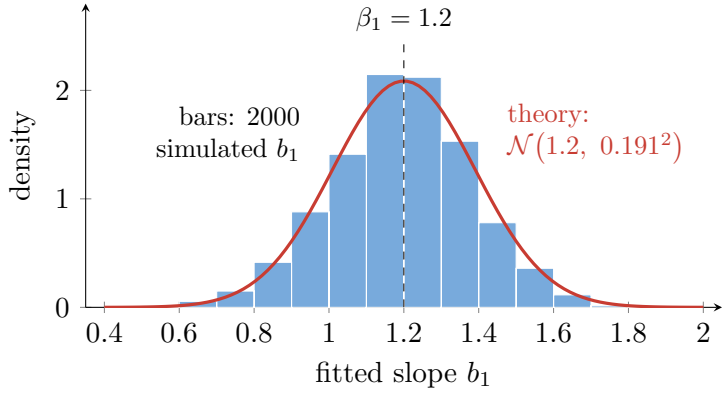


Figure 12.14: Histogram of b_1 over 2000 simulated samples of six uploads. It matches the Normal curve with mean $\beta_1 = 1.2$ and SD $\sigma/\sqrt{S_{xx}} = 0.8/\sqrt{17.5} = 0.191$ predicted by Theorem 12.3.

12.9.1 Estimating σ and the standard error

We do not know σ. Estimate it from the residuals:

$$s = \sqrt{\frac{\text{SSE}}{n-2}} = \sqrt{\frac{\sum e_i^2}{n-2}}.$$

This is called the **residual standard error**. We divide by $n-2$, not n, for the same reason the sample variance divides by $n-1$ (Chapter 2): the residuals are measured from a line fitted to the same data, so they are too small on average. Fitting used up two degrees of freedom (one each for b_0 and b_1); the residuals satisfy two linear constraints (the normal equations), leaving $n-2$ free. With this divisor, $\mathbb{E}[s^2] = \sigma^2$.

Plugging in gives the standard error:

$$\text{SE}(b_1) = \frac{s}{\sqrt{S_{xx}}}.$$

Theorem 12.4 (t statistic for the slope)**.** *Under the Normal-error model,*

$$T = \frac{b_1 - \beta_1}{\text{SE}(b_1)} \sim t_{n-2}.$$

This is the same move as the one-sample t test of Chapter 9: replace the unknown σ by s and pay for it with a t distribution, here with $n-2$ degrees of freedom. A $100(1-\alpha)\%$ confidence interval for the slope is

$$b_1 \pm t^*_{n-2}\,\text{SE}(b_1),$$

where t^*_{n-2} is the t_{n-2} critical value ($t_{\alpha/2,\,n-2}$ in the notation of Chapter 7), and the test of $H_0\colon \beta_1 = 0$ ("x has no linear association with y") uses $t = b_1/\,\text{SE}(b_1)$.

Walking question

Is upload time really related to file size, and how precisely do we know the slope?
Residual SE. $s = \sqrt{2.8/4} = \sqrt{0.7} = 0.837$ s.
SE of slope. $\text{SE}(b_1) = 0.837/\sqrt{17.5} = 0.837/4.183 = 0.200$ s/MB.
Test. $t = 1.2/0.2 = 6.0$ on $n-2 = 4$ degrees of freedom. The two-sided p-value is $2\,\mathbb{P}(t_4 > 6.0) = 0.0039$. Reject $H_0\colon \beta_1 = 0$ at any conventional level.
95% CI. $t^*_4 = 2.776$. The interval is $1.2 \pm 2.776 \times 0.2 = 1.2 \pm 0.555 = [0.645,\ 1.755]$ s/MB.
Reading. Each extra MB adds between about 0.65 and 1.76 seconds on average. With only six points the interval is wide, but it excludes 0.

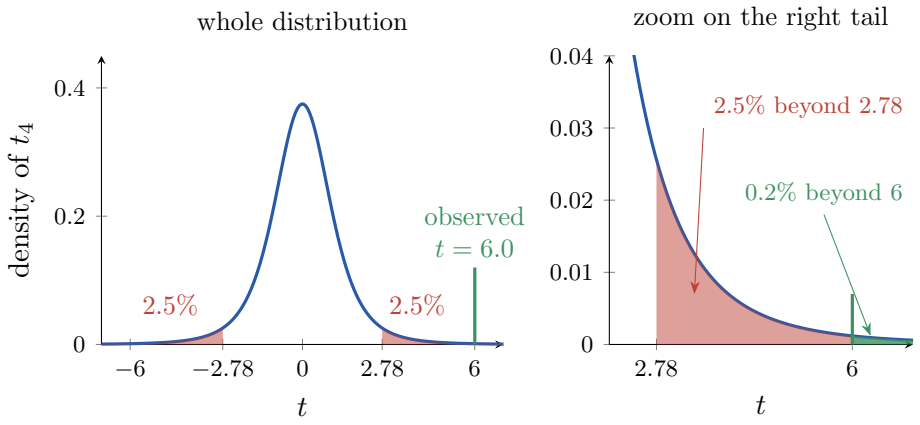


Figure 12.15: The t_4 distribution if the true slope were 0. The red tails (beyond ± 2.776) hold 5% of the probability, so any $|t| > 2.776$ is significant at the 5% level. The observed $t = 6$ lies far out; the right panel magnifies the right tail, where only 0.2% lies beyond 6 (green). Both tails together give $p = 0.0039$.

Common trap

Testing $\beta_1 = 0$ is the same as testing $\rho = 0$ (zero population correlation): $t = b_1 / \mathrm{SE}(b_1) = r\sqrt{n-2}/\sqrt{1-r^2}$. Check: $0.9487 \times 2/\sqrt{0.1} = 6.0$. A tiny p-value says the slope is not zero. It does not say the slope is large, that the line fits well, or that x causes y.

12.10 Predicting: mean response vs. a new observation

Suppose a 4 MB file is about to be uploaded. The point prediction is $\hat{y}_0 = 1.8 + 1.2 \times 4 = 6.6$ s. Two different questions have this same point estimate but different error bars.

1. **Mean response.** What is the *average* upload time of all 4 MB files, $\beta_0 + \beta_1 \cdot 4$? Uncertainty comes only from not knowing the true line.

2. **Prediction.** How long will *this one* 4 MB upload take? Now add the noise ε of the single new upload on top.

Write $\hat{y}_0 = \bar{y} + b_1(x_0 - \bar{x})$. The two pieces $\bar{y}$ and b_1 are uncorrelated (because $\sum w_i = 0$), so

$$\mathrm{Var}(\hat{y}_0) = \frac{\sigma^2}{n} + (x_0 - \bar{x})^2 \frac{\sigma^2}{S_{xx}}.$$

A new observation is $y_0 = \beta_0 + \beta_1 x_0 + \varepsilon_0$ with fresh noise independent of the data, so $\mathrm{Var}(y_0 - \hat{y}_0) = \sigma^2 + \mathrm{Var}(\hat{y}_0)$. Replacing σ by s:

Definition 12.6 (Confidence and prediction intervals)**.** At a new x_0, with $\hat{y}_0 = b_0 + b_1 x_0$ and

$t^* = t^*_{n-2}$:

$$\text{CI for the mean response:} \quad \hat{y}_0 \pm t^* \, s \sqrt{\frac{1}{n} + \frac{(x_0 - \bar{x})^2}{S_{xx}}},$$

$$\text{Prediction interval for a new } y\text{:} \quad \hat{y}_0 \pm t^* \, s \sqrt{1 + \frac{1}{n} + \frac{(x_0 - \bar{x})^2}{S_{xx}}}.$$

For $x_0 = 4$: $(x_0 - \bar{x})^2 / S_{xx} = 0.25/17.5 = 0.0143$, and $1/n = 0.1667$.

$$\text{mean:} \quad 6.6 \pm 2.776 \times 0.837\sqrt{0.1810} = 6.6 \pm 0.99 = [5.61,\ 7.59],$$

$$\text{new upload:} \quad 6.6 \pm 2.776 \times 0.837\sqrt{1.1810} = 6.6 \pm 2.52 = [4.08,\ 9.12].$$

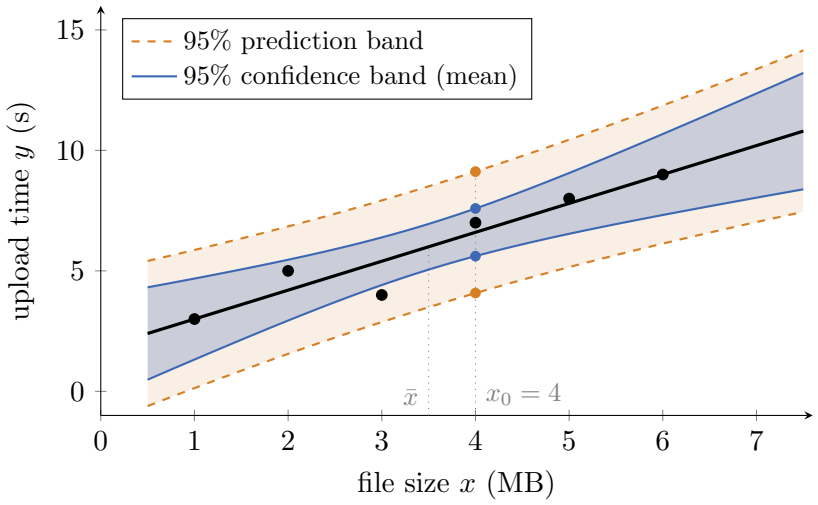


Figure 12.16: Confidence band for the mean response (blue) and prediction band for a single new upload (orange). The dots at $x_0 = 4$ MB mark the two intervals computed above: the mean time lies in $[5.61,\ 7.59]$ s (blue), one upload in $[4.08,\ 9.12]$ s (orange). Both bands are narrowest at $\bar{x} = 3.5$ and widen with distance from it. The prediction band is much wider because it includes the noise of one upload.

Intuition

As $n \to \infty$ the confidence band shrinks to the true line: with enough data you know the average exactly. The prediction band does not shrink below $\pm t^* \sigma$: no amount of data predicts the jitter of one specific upload. Dashboards that show a "confidence band" around a forecast are usually showing the narrow one. If you want to know where tomorrow's single value will land, you need the wide one.

```
n, x0 = len(x), 4.0
yhat = fit.intercept + fit.slope * x
s = np.sqrt(np.sum((y - yhat)**2) / (n - 2))      # 0.837
Sxx = np.sum((x - x.mean())**2)                   # 17.5
y0 = fit.intercept + fit.slope * x0               # 6.6
tc = stats.t.ppf(0.975, n - 2)                    # 2.776
```

```
se_mean = s * np.sqrt(1/n + (x0 - x.mean())**2 / Sxx)
se_pred = s * np.sqrt(1 + 1/n + (x0 - x.mean())**2 / Sxx)
print(y0 - tc*se_mean, y0 + tc*se_mean)          # 5.61 7.59
print(y0 - tc*se_pred, y0 + tc*se_pred)          # 4.08 9.12
```

12.11 The assumptions: LINE

The least squares line can always be computed. Its standard errors, p-values and intervals rest on four assumptions, remembered as LINE. Figure 12.17 lists each one, what breaks without it, and how to check it.

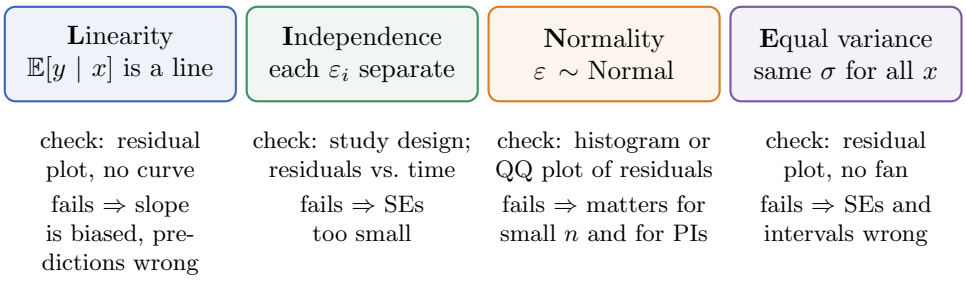


Figure 12.17: The four LINE assumptions, how to check each, and the cost of violating it. Linearity is about the fit; the other three are about the error bars.

The assumptions are not equally important.

- **Linearity** matters most. If the mean is not a line, b_1 estimates nothing meaningful.
- **Independence** is the one you cannot check from a scatter plot. Repeated measurements on the same user, or a time series where today resembles yesterday, make the effective sample size smaller than n and the reported standard errors too small.
- **Normality** matters least for the slope. By the Central Limit Theorem $b_1 = \beta_1 + \sum w_i \varepsilon_i$ is close to Normal for moderate n even when ε is not. It matters more for prediction intervals, which are about a single ε.
- **Equal variance** affects every standard error. A fan-shaped residual plot means the intervals are too narrow in some places and too wide in others.

12.12 Transformations and extrapolation

Many relationships are straight only on a log scale. Figure 12.18 shows thirty animals' body mass and resting metabolic rate (simulated from Kleiber's law). On the raw scale the points crowd into the corner and bend. On log-log axes they fall on a line.

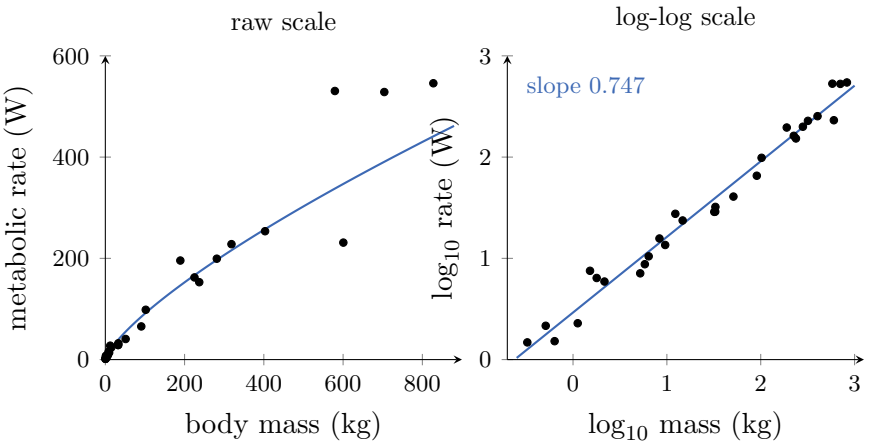


Figure 12.18: Left: a power law on raw axes is curved, and its noise fans out. Right: on log-log axes it is a straight line with even scatter. The fitted log-log slope 0.747 is the exponent; the blue curve on the left is $\hat{y} = 2.92\,x^{0.747}$.

If $y = ax^b$ times multiplicative noise, then

$$\log y = \log a + b \log x + \text{noise},$$

which is the simple linear regression model in the variables $(\log x, \log y)$. The fit gives $\log_{10} \hat{y} = 0.465 + 0.747 \log_{10} x$, so $\hat{y} = 10^{0.465} x^{0.747} = 2.92\,x^{0.747}$.

Interpreting log-log slopes. The slope is an *elasticity*: a 1% increase in x goes with about a b% increase in y. Doubling body mass multiplies metabolic rate by $2^{0.747} \approx 1.68$. Other common forms:

- $\log y$ on x (log-linear): one unit of x multiplies y by e^{b_1}; for small b_1, that is about a $100b_1$% change. Natural for growth and for latencies, which are right-skewed.
- y on $\log x$ (linear-log): multiplying x by 10 adds b_1 to y (for $\log_{10}$). Natural for diminishing returns.

A log transform often fixes curvature and a fan shape at the same time, as here. Use it when y is positive and its spread grows with its level.

Common trap

A line fitted to $\log y$ predicts the mean of $\log y$. Back-transforming gives $10^{\widehat{\log y}}$, which estimates the *median* of y (if the log-scale noise is symmetric), not the mean. The mean of y is larger.

Extrapolation. A regression line describes the data inside the range of x you observed. Outside that range you have no evidence the line continues. Figure 12.19 shows eight measurements for $x \in [0.5, 4]$ that a line fits beautifully ($R^2 = 0.96$). The true curve levels off. At $x = 12$ the line predicts 21.3; the truth is 9.9.

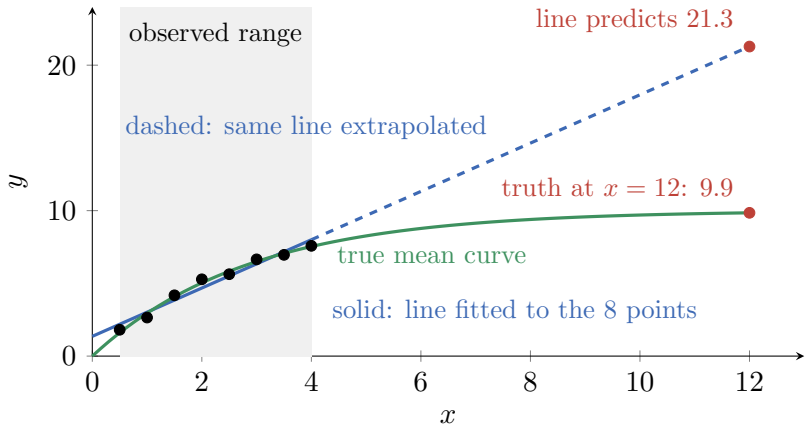


Figure 12.19: Extrapolation. Inside the shaded data range the line is an excellent approximation. Beyond it the line and the truth diverge, and nothing in the data could have warned you.

The prediction interval formula grows with $(x_0 - \bar{x})^2$, so it does widen outside the data. But it only accounts for uncertainty in the slope and intercept *assuming the line is right*. It cannot account for the line being wrong, which is the bigger risk far from the data. Capacity planning ("latency grows linearly with load, so at 3× load…") is the classic victim: queues are linear at low utilisation and explode near saturation.

12.13 Least squares is maximum likelihood

Section 12.2 chose least squares because it is convenient. There is a better reason. Assume the Normal-error model, $y_i \sim \mathcal{N}(\beta_0 + \beta_1 x_i,\ \sigma^2)$ independently. The probability density of the observed data, viewed as a function of the parameters, is the **likelihood**:

$$L(\beta_0, \beta_1, \sigma) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{(y_i - \beta_0 - \beta_1 x_i)^2}{2\sigma^2} \right).$$

Take logs:

$$\log L = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \beta_0 - \beta_1 x_i)^2.$$

For any fixed σ, the only term involving β_0, β_1 is $-\mathrm{SSE}/(2\sigma^2)$. Maximising $\log L$ is therefore the same as minimising SSE. The maximum likelihood estimates of slope and intercept are exactly the least squares estimates.

Maximising over σ as well gives $\hat{\sigma}^2_{\mathrm{MLE}} = \mathrm{SSE}/n$. That is biased low; the unbiased version divides by $n - 2$, the s^2 of Section 12.9. Same story as n versus $n - 1$ for the sample variance.

Connection

This equivalence is why "mean squared error loss" is the default in machine learning regression. Training a neural network with MSE loss is maximum likelihood under the assumption that targets are the network's output plus Gaussian noise. Swap the noise model and the loss changes: Laplace noise gives absolute error (L1) loss; Bernoulli outcomes give cross-entropy,

which is logistic regression (Chapter 13). Chapter 14 develops maximum likelihood in general, and Chapter 15 returns to the loss-function view.

Upload time surely depends on more than file size: network type, server region, time of day. Chapter 13 puts several predictors into one regression and asks what each slope means when the others are held fixed.

12.14 Summary

Model	$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$, errors independent, mean 0, variance σ^2
Residual	$e_i = y_i - \hat{y}_i$, the vertical distance to the line
Least squares slope	$b_1 = S_{xy}/S_{xx} = r\, s_y/s_x$
Least squares intercept	$b_0 = \bar{y} - b_1\bar{x}$; line passes through $(\bar{x}, \bar{y})$
Normal equations	$\sum e_i = 0$, $\sum x_i e_i = 0$
Decomposition	$\text{SST} = \text{SSR} + \text{SSE}$; $R^2 = \text{SSR}/\text{SST} = r^2$
Regression to the mean	$\hat{z}_y = r\, z_x$; y-on-x and x-on-y lines differ unless $\lvert r\rvert = 1$
Residual SE	$s = \sqrt{\text{SSE}/(n-2)}$
Slope inference	$\text{SE}(b_1) = s/\sqrt{S_{xx}}$; $(b_1 - \beta_1)/\,\text{SE}(b_1) \sim t_{n-2}$
Mean response CI	$\hat{y}_0 \pm t^* s\sqrt{1/n + (x_0 - \bar{x})^2/S_{xx}}$
Prediction interval	$\hat{y}_0 \pm t^* s\sqrt{1 + 1/n + (x_0 - \bar{x})^2/S_{xx}}$
Diagnostics	residual plot: curve $\Rightarrow$ nonlinearity; fan $\Rightarrow$ unequal variance; high leverage + off trend $\Rightarrow$ influential
Assumptions	LINE: Linearity, Independence, Normality, Equal variance
Log-log slope	elasticity: 1% more $x \approx b_1$% more y
Likelihood view	Gaussian noise $\Rightarrow$ least squares = maximum likelihood

12.15 Exercises

Exercise 12.1. Five points: $x = 0, 1, 2, 3, 4$ and $y = 1, 3, 2, 5, 4$. Compute $\bar{x}$, $\bar{y}$, S_{xx}, S_{xy}, S_{yy}, the least squares line, r and R^2.

Exercise 12.2. For the data in Exercise 1, compute SSE, s, $\text{SE}(b_1)$ and the t statistic for $H_0\colon \beta_1 = 0$. With $t^*_3 = 3.182$, is the slope significant at the 5% level? What is the 95% CI for β_1?

Exercise 12.3. A study reports $\bar{x} = 50$, $s_x = 10$, $\bar{y} = 20$, $s_y = 5$, $r = 0.6$. Find the regression line of y on x and the regression line of x on y. Predict y when $x = 70$.

Exercise 12.4. Midterm and final exam scores have correlation $r = 0.5$. A student scores 2 SDs above the mean on the midterm. Predict how many SDs above the mean she will score on the final.

A student who scored 2 SDs above the mean on the final: predict her midterm in SD units. Why is neither answer 2?

Exercise 12.5. Prove from the normal equations that the residuals of a least squares fit with an intercept satisfy $\sum_i \hat{y}_i e_i = 0$.

Exercise 12.6. In the upload example, x is converted from MB to KB (1 MB = 1000 KB) and y from seconds to milliseconds. What happens to b_1, b_0, R^2 and the t statistic for the slope?

Exercise 12.7. For the upload data, compute the 95% confidence interval for the mean response and the 95% prediction interval at $x_0 = 7$ MB. Why are both wider than at $x_0 = 4$? What extra caveat applies at $x_0 = 7$?

Exercise 12.8. A log-log regression of house price on floor area gives slope 0.8. By what factor does the predicted price change when floor area doubles? When it grows by 10%?

Exercise 12.9. You want to estimate how request latency depends on payload size, and you can choose 20 payload sizes in the range 1 to 100 KB. Using $\mathrm{SD}(b_1) = \sigma/\sqrt{S_{xx}}$, which choice of sizes gives the most precise slope? Why might you not use that choice in practice?

Exercise 12.10. Show that under Gaussian errors, maximising the log-likelihood over σ^2 (with β_0, β_1 at their least squares values) gives $\hat{\sigma}^2 = \mathrm{SSE}/n$.

Answers

1. $\bar{x} = 2$, $\bar{y} = 3$, $S_{xx} = 10$, $S_{xy} = 8$, $S_{yy} = 10$. $b_1 = 0.8$, $b_0 = 3 - 0.8 \times 2 = 1.4$, so $\hat{y} = 1.4 + 0.8x$. $r = 8/\sqrt{100} = 0.8$, $R^2 = 0.64$.

2. $\mathrm{SSE} = S_{yy} - b_1 S_{xy} = 10 - 6.4 = 3.6$; $s = \sqrt{3.6/3} = 1.095$; $\mathrm{SE}(b_1) = 1.095/\sqrt{10} = 0.346$; $t = 0.8/0.346 = 2.31 < 3.182$. Not significant ($p = 0.10$). CI: $0.8 \pm 3.182 \times 0.346 = [-0.30,\ 1.90]$, which contains 0.

3. y on x: $b_1 = 0.6 \times 5/10 = 0.3$, $b_0 = 20 - 0.3 \times 50 = 5$, so $\hat{y} = 5 + 0.3x$. x on y: $c_1 = 0.6 \times 10/5 = 1.2$, $c_0 = 50 - 1.2 \times 20 = 26$, so $\hat{x} = 26 + 1.2y$. At $x = 70$: $\hat{y} = 26$.

4. Final: $0.5 \times 2 = 1$ SD above. Midterm given final: also 1 SD above. Each prediction regresses toward the mean because $|r| < 1$; the extreme score is partly luck, which does not carry over.

5. $\hat{y}_i = b_0 + b_1 x_i$, so $\sum \hat{y}_i e_i = b_0 \sum e_i + b_1 \sum x_i e_i = 0 + 0$.

6. b_1 becomes $1.2 \times 1000/1000 = 1.2$ ms/KB (unchanged numerically here, since both scales changed by 1000). b_0 becomes 1800 ms. R^2 and t are unchanged: they are unit-free.

7. $\hat{y}_0 = 10.2$. $(7 - 3.5)^2/17.5 = 0.7$. Mean CI: $10.2 \pm 2.776 \times 0.837\sqrt{0.8667} = [8.04,\ 12.36]$. PI: $10.2 \pm 2.776 \times 0.837\sqrt{1.8667} = [7.03,\ 13.37]$. Wider because x_0 is farther from $\bar{x}$. And $x_0 = 7$ is outside the observed range $[1, 6]$, so it is an extrapolation: the intervals assume the line still holds there.

8. Doubling: $2^{0.8} \approx 1.74$, a 74% increase. 10% more area: $1.1^{0.8} \approx 1.079$, about 7.9% more.

9. Put 10 sizes at 1 KB and 10 at 100 KB; this maximises S_{xx}. But with only two distinct x values you cannot see curvature, so you cannot check linearity. Spreading some points across the range is safer.

10. $\partial \log L/\partial\sigma^2 = -n/(2\sigma^2) + \mathrm{SSE}/(2\sigma^4) = 0$ gives $\sigma^2 = \mathrm{SSE}/n$.

Chapter 13

Multiple and Logistic Regression

Chapter 12 fitted a straight line through a cloud of points: one predictor x, one response y. Real questions rarely have one cause. A house price depends on size *and* age *and* location. A page conversion depends on time on page *and* device *and* traffic source. This chapter does two things. It lets the line take in many predictors at once, which turns it into a plane and then into a projection in high-dimensional space. Then it bends the model so it can predict a yes/no outcome. The second step leads straight to the loss function that trains most classifiers in machine learning.

13.1 From a line to a plane

Here are the first rows of a dataset of 40 houses. Price is in thousands of dollars, size in square metres, age in years.

house	size (m^2)	age (yr)	price ($k)
1	144	16	308
2	185	53	343
3	166	31	313
4	84	51	161
5	95	38	142
⋮	⋮	⋮	⋮

With one predictor we drew $\hat{y} = b_0 + b_1 x$, a line. With two predictors the natural guess is

$$\widehat{\text{price}} = b_0 + b_1 \cdot \text{size} + b_2 \cdot \text{age},$$

which is a flat plane hovering over the (size, age) floor. Least squares again picks the plane that makes the sum of squared vertical gaps as small as possible. For these 40 houses the computer returns

$$\widehat{\text{price}} = 60.08 + 1.861 \cdot \text{size} - 1.393 \cdot \text{age}.$$

Read it as: one extra square metre adds about $1,861; one extra year of age subtracts about $1,393.

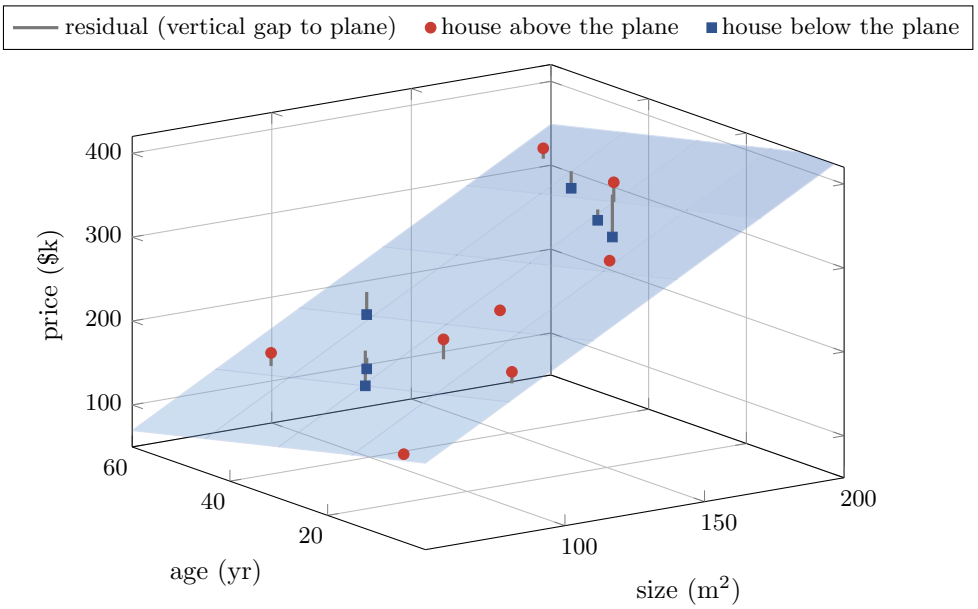


Figure 13.1: With two predictors the fitted model is a plane: the predicted price for every (size, age) pair. Red dots are houses priced above the plane, blue squares houses priced below it, and each grey stick is a residual. Least squares tilts the plane to make the sum of the squared stick lengths as small as possible, exactly as in Chapter 12. (Fourteen of the 40 houses are shown.)

With three or more predictors there is no picture of the plane any more, but the algebra does not care. We need a notation that handles any number of predictors at once.

13.2 The matrix form

13.2.1 Stacking the equations

Take a tiny dataset so we can do everything by hand: four houses, size x in units of 100 m^2, price y in units of \$100k.

$$x = (1,\ 2,\ 3,\ 4), \qquad y = (3,\ 5,\ 4,\ 8).$$

The model says each price is intercept plus slope times size plus an error:

$$\begin{aligned} 3 &= \beta_0 + \beta_1 \cdot 1 + \varepsilon_1, \\ 5 &= \beta_0 + \beta_1 \cdot 2 + \varepsilon_2, \\ 4 &= \beta_0 + \beta_1 \cdot 3 + \varepsilon_3, \\ 8 &= \beta_0 + \beta_1 \cdot 4 + \varepsilon_4. \end{aligned}$$

Four equations with the same shape. Stack them into vectors and a matrix:

$$\underbrace{\begin{bmatrix} 3 \\ 5 \\ 4 \\ 8 \end{bmatrix}}_{y} = \underbrace{\begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix}}_{X} \underbrace{\begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}}_{\beta} + \underbrace{\begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \varepsilon_3 \\ \varepsilon_4 \end{bmatrix}}_{\varepsilon}.$$

Definition 13.1 (Linear model in matrix form)**.** With n observations and p predictors,

$$y = X\beta + \varepsilon,$$

where y is the $n \times 1$ vector of responses, β is the $(p+1) \times 1$ vector of coefficients, ε is the $n \times 1$ vector of errors, and X is the $n \times (p+1)$ **design matrix**: one row per observation, one column per predictor, plus a leading column of 1s.

The design matrix is the spreadsheet of inputs. The column of 1s is the **intercept column**: it multiplies β_0, so every row gets the same constant added. Without it the fitted plane would be forced through the origin.

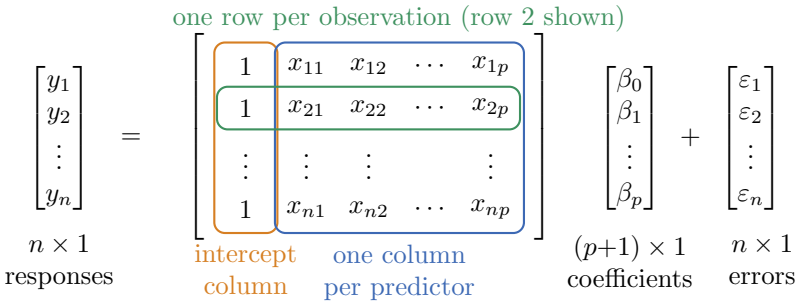


Figure 13.2: The anatomy of $y = X\beta + \varepsilon$. The design matrix X holds the inputs; its first column of 1s carries the intercept. Multiplying row i of X by β gives the model's prediction for observation i.

13.2.2 The normal equations

We want the β that minimises the sum of squared residuals

$$S(\beta) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \|y - X\beta\|^2.$$

The symbol $\|v\|^2$ is the squared length of a vector: the sum of its squared entries. So $S(\beta)$ is the squared distance between the data vector y and the prediction vector $X\beta$.

Expand it, treating vectors like numbers but keeping track of transposes ($v^\top$ turns a column into a row):

$$S(\beta) = y^\top y - 2\beta^\top X^\top y + \beta^\top X^\top X \beta.$$

This is a bowl-shaped quadratic in β. At the bottom of a bowl the slope in every direction is zero. The vector of slopes (the **gradient**, one partial derivative per coefficient) is

$$\nabla S(\beta) = -2X^\top y + 2X^\top X \beta.$$

Setting it to zero gives the central equation of this chapter.

Theorem 13.1 (Normal equations)**.** *The least squares coefficients $\hat{\beta} = (b_0, b_1, \ldots, b_p)^\top$ satisfy*

$$X^\top X \, \hat{\beta} = X^\top y.$$

If the columns of X are linearly independent, $X^\top X$ is invertible and

$$\hat{\beta} = (X^\top X)^{-1} X^\top y.$$

Example 13.1 (Four houses by hand)**.** With X and y above,

$$X^\top X = \begin{bmatrix} 4 & 10 \\ 10 & 30 \end{bmatrix}, \qquad X^\top y = \begin{bmatrix} 3+5+4+8 \\ 3+10+12+32 \end{bmatrix} = \begin{bmatrix} 20 \\ 57 \end{bmatrix}.$$

The entries of $X^\top X$ are $n = 4$, $\sum x_i = 10$ and $\sum x_i^2 = 30$. The determinant is $4 \cdot 30 - 10 \cdot 10 = 20$, so

$$\hat{\beta} = \frac{1}{20} \begin{bmatrix} 30 & -10 \\ -10 & 4 \end{bmatrix} \begin{bmatrix} 20 \\ 57 \end{bmatrix} = \frac{1}{20} \begin{bmatrix} 600 - 570 \\ -200 + 228 \end{bmatrix} = \begin{bmatrix} 1.5 \\ 1.4 \end{bmatrix}.$$

The fitted line is $\hat{y} = 1.5 + 1.4x$, the same answer the slope formula $b_1 = S_{xy}/S_{xx}$ from Chapter 12 gives. Fitted values are $2.9, 4.3, 5.7, 7.1$ and residuals are $e = (0.1,\ 0.7,\ -1.7,\ 0.9)$.

Two checks on those residuals. They sum to zero: $0.1 + 0.7 - 1.7 + 0.9 = 0$. And weighted by x they also sum to zero: $0.1 + 1.4 - 5.1 + 3.6 = 0$. Those two facts are the normal equations rewritten as $X^\top e = 0$: the residual vector has zero dot product with every column of X. That sentence has a picture.

```
import numpy as np
X = np.array([[1, 1], [1, 2], [1, 3], [1, 4]], dtype=float)
y = np.array([3, 5, 4, 8], dtype=float)
beta = np.linalg.solve(X.T @ X, X.T @ y)        # normal equations: [1.5, 1.4]
beta2, *_ = np.linalg.lstsq(X, y, rcond=None)   # same answer, numerically safer
e = y - X @ beta
print(beta, X.T @ e)                            # X^T e is (0, 0) up to rounding
```

13.3 The geometric picture: least squares is a projection

Think of the n responses as a single arrow y in n-dimensional space. Each column of X is also an arrow in that space. Every possible prediction $X\beta$ is a weighted sum of the columns, so the set of all possible predictions is a flat sheet through the origin: the **column space** of X, meaning every vector you can build by stretching and adding the columns. With two columns it is a plane.

The data arrow y almost never lies in that plane (noise pushes it out). Least squares asks: which point in the plane is closest to y? The answer is the foot of the perpendicular dropped from y onto the plane. That foot is $\hat{y}$, and the perpendicular itself is the residual vector $e = y - \hat{y}$.

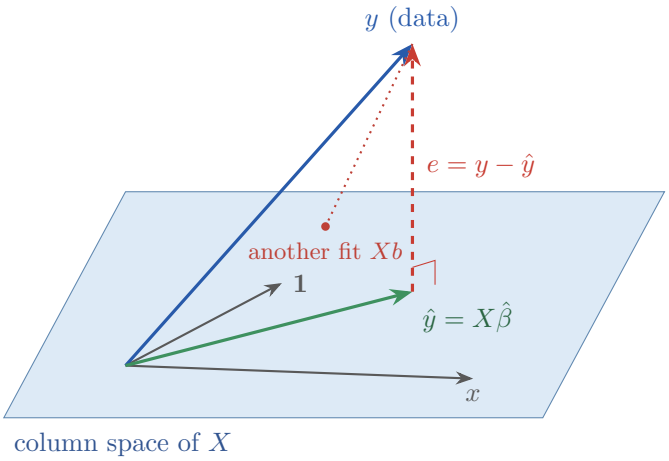


Figure 13.3: Least squares as projection. The grey arrows are the two columns of X: the column of 1s ($\mathbf{1}$) and the predictor column (x). The data vector y sticks out of the plane of possible predictions they span. The closest point in the plane is $\hat{y}$, reached by dropping a perpendicular (the small square marks the right angle). Any other fit Xb in the plane (dotted) is farther from y. The residual e is at a right angle to every column of X, which is exactly the statement $X^\top e = 0$.

Why it works

Why does the perpendicular give the closest point? Take any other point Xb in the plane. The three points y, $\hat{y}$, Xb form a right triangle with the right angle at $\hat{y}$, because $\hat{y} - Xb$ lies in the plane and e is perpendicular to the plane. Pythagoras gives

$$\|y - Xb\|^2 = \|e\|^2 + \|\hat{y} - Xb\|^2 \;\geq\; \|e\|^2.$$

So no other point beats $\hat{y}$. Writing "e is perpendicular to every column" as $X^\top(y - X\hat{\beta}) = 0$ and rearranging gives the normal equations again, with no calculus at all.

The same right triangle, with $\bar{y}\mathbf{1}$ in place of Xb, is the decomposition SST = SSR + SSE from Chapter 12. R^2 is the squared cosine of the angle between the centred y and the plane: how much of the arrow lies *in* the plane.

13.3.1 The hat matrix

Substitute $\hat{\beta}$ into $\hat{y} = X\hat{\beta}$:

$$\hat{y} = X(X^\top X)^{-1}X^\top y = Hy, \qquad H = X(X^\top X)^{-1}X^\top.$$

H is called the **hat matrix** because it puts the hat on y. It is the projection machine: feed in any vector, get back its shadow on the column space. Two facts follow from the picture. Projecting twice changes nothing, so $HH = H$. And the residual is what is left over, $e = (I - H)y$.

The diagonal entry h_{ii} is the **leverage** of observation i: how strongly y_i pulls on its own fitted value $\hat{y}_i$. For the four houses, with one predictor, $h_{ii} = \frac{1}{n} + (x_i - \bar{x})^2/S_{xx}$ with $\bar{x} = 2.5$ and $S_{xx} = 5$:

$$h_{11} = 0.25 + \tfrac{2.25}{5} = 0.7, \quad h_{22} = 0.25 + \tfrac{0.25}{5} = 0.3, \quad h_{33} = 0.3, \quad h_{44} = 0.7.$$

The end points have the most leverage, as the seesaw intuition of Chapter 12 suggested. The leverages sum to 2, the number of columns. In general they sum to $p + 1$.

Pause

Pause and think: if you add a predictor that is pure noise, can $\|e\|^2$ go up? (No. The column space only grows, so the closest point can only get closer. That is why R^2 never falls when a variable is added, and why we will need adjusted R^2.)

13.4 What a coefficient means: "holding the others fixed"

In $\widehat{\text{price}} = 60.08 + 1.861 \cdot \text{size} - 1.393 \cdot \text{age}$, compare two houses with the same age whose sizes differ by 1 m^2. Their predicted prices differ by exactly 1.861 (thousand dollars). That is the meaning of a coefficient in multiple regression: *the predicted change in y for a one-unit change in x_j with every other predictor held fixed.*

This is a different number from the simple-regression slope, and the difference can be dramatic. Here are 12 houses with size and number of bedrooms.

size (m^2)	60	70	80	90	100	110	120	130	140	150	160	170
bedrooms	1	2	1	2	3	2	3	4	3	4	5	4
price (\$k)	161	130	188	185	191	228	219	238	268	307	287	317

Regress price on bedrooms alone:

$$\widehat{\text{price}} = 122.6 + 36.7 \cdot \text{bedrooms}.$$

Each extra bedroom is worth \$36,700. Now add size:

$$\widehat{\text{price}} = 31.5 + 2.127 \cdot \text{size} - 17.5 \cdot \text{bedrooms}.$$

The bedroom coefficient flipped sign. Holding size fixed, an extra bedroom *lowers* the price by \$17,500.

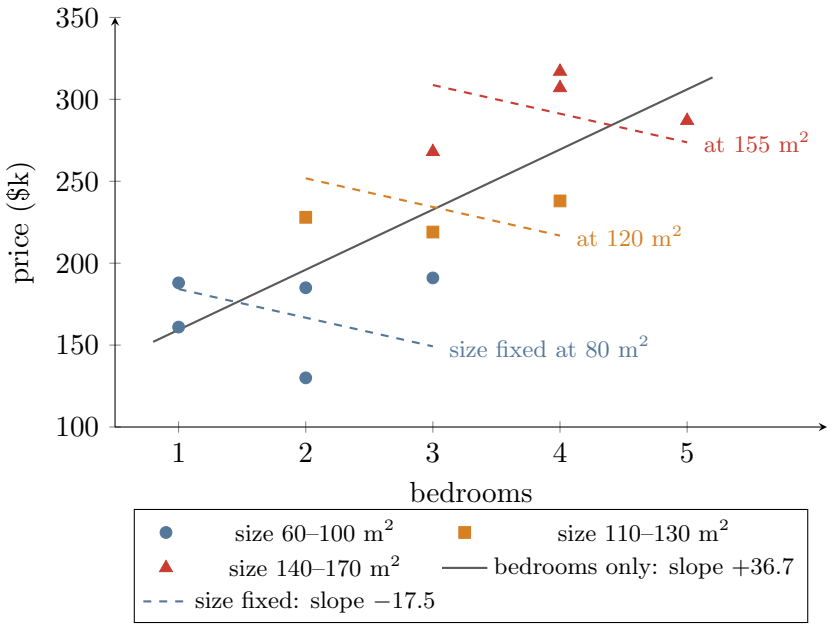


Figure 13.4: Confounding in a regression. Across all houses, more bedrooms go with higher prices (solid grey line, slope +36.7), because big houses have more bedrooms. Now hold size fixed: each dashed line is the multiple-regression prediction at one size (80, 120 or 155 m^2), and each slopes *down* (−17.5 per bedroom), because at a given size more bedrooms means smaller rooms. Marker shape shows each house's size group.

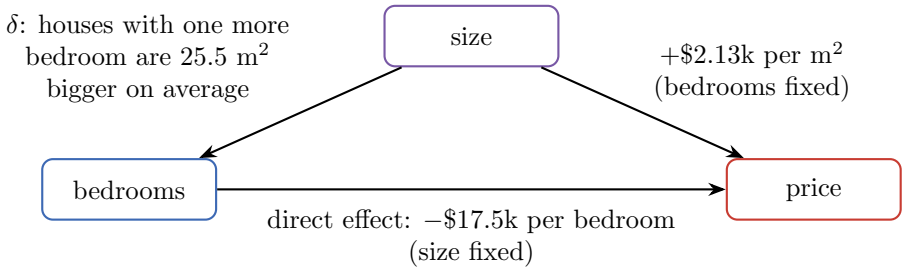


Figure 13.5: Why the coefficient flips. Size drives both bedroom count and price. The bedrooms-only slope adds two routes from bedrooms to price: the direct arrow (−17.5) plus the detour through size ($25.5 \times 2.13 \approx 54.2$), giving $-17.5 + 54.2 = +36.7$.

The arithmetic in Figure 13.5 is exact, not a heuristic. Regress size on bedrooms and you get slope $\delta = 25.47$: each extra bedroom comes with 25.47 m^2 more house on average. Then

$$\underbrace{36.69}_{\text{simple slope}} = \underbrace{-17.48}_{\text{bedrooms, size fixed}} + \underbrace{2.1267}_{\text{size coef}} \times \underbrace{25.47}_{\delta}.$$

This identity is called the **omitted variable bias** formula: leaving out a variable that is correlated with an included one shifts the included coefficient by (the omitted variable's coefficient) × (δ, the slope from regressing the omitted variable on the included one). It is the regression version of the confounding story from Chapter 5.

Common trap

"Holding others fixed" is a statement about the fitted model, not about the world. Regression adjusts only for variables you put in. If an unmeasured variable (school district, say) drives both bedrooms and price, the bedroom coefficient still carries its shadow. And if no house in your data has 5 bedrooms in 60 m^2, the model's prediction for one is extrapolation, not evidence.

13.5 Building richer models with the same machinery

The word "linear" in linear regression means linear in the coefficients β. The columns of X can be anything you compute from the data. That freedom gives categorical predictors, interactions and curves at no extra cost.

13.5.1 Categorical predictors and dummy variables

Request latency (ms) from three regions, four requests each:

region	latencies				mean
US	42	45	40	47	43.5
EU	51	55	49	53	52.0
Asia	78	72	75	79	76.0

Region is a label, not a number. Coding US= 1, EU= 2, Asia= 3 would force the EU to sit exactly halfway between the others. Instead create **dummy variables**: columns of 0s and 1s that switch on for one category. Pick US as the **baseline** (reference category) and add two columns:

$$d_{\text{EU}} = \begin{cases} 1 & \text{EU} \\ 0 & \text{otherwise} \end{cases}, \qquad d_{\text{Asia}} = \begin{cases} 1 & \text{Asia} \\ 0 & \text{otherwise} \end{cases}.$$

The model $\hat{y} = b_0 + b_1 d_{\text{EU}} + b_2 d_{\text{Asia}}$ predicts b_0 for US, $b_0 + b_1$ for EU and $b_0 + b_2$ for Asia. Least squares makes each prediction the group mean, so

$$b_0 = 43.5, \qquad b_1 = 52.0 - 43.5 = 8.5, \qquad b_2 = 76.0 - 43.5 = 32.5.$$

Each dummy coefficient is a difference from the baseline. Its t-test asks whether that region differs from US. That is the pooled two-sample t-test of Chapter 9 in regression clothing, except that the spread is pooled across all three regions.

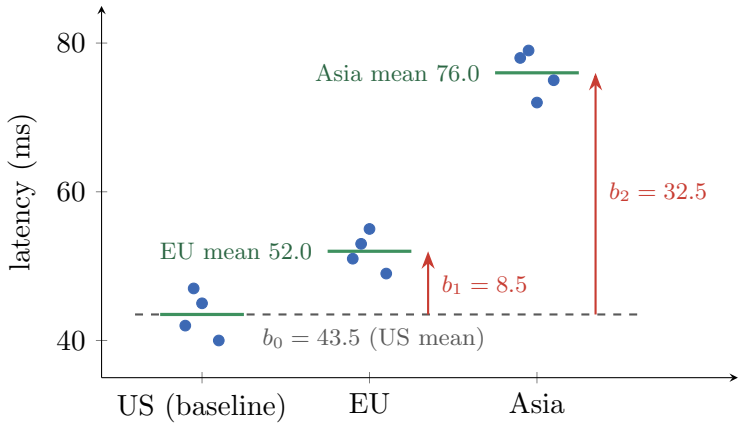


Figure 13.6: Dummy coding. Blue dots are the 12 requests; green bars are the three group means. The intercept b_0 is the baseline (US) mean, extended as the dashed line, and each dummy coefficient is the red jump from that line up to its own group's mean.

Common trap

The **dummy variable trap**: with an intercept, use $k-1$ dummies for k categories. If you add d_{US} too, then $d_{\text{US}} + d_{\text{EU}} + d_{\text{Asia}} = 1$ in every row, which equals the intercept column. The columns are linearly dependent, $X^\top X$ has no inverse, and the coefficients are not defined. Software either errors or silently drops a column.

13.5.2 Interactions

Latency also depends on payload size. Suppose an old and a new server version both get slower with payload. Two very different stories are possible, and Figure 13.7 shows both. Let x be payload (KB) and $d = 1$ for the new version.

- *Parallel lines*: $\hat{y} = 20 + 0.5x - 8d$. The new version is 8 ms faster at every payload.

- *Non-parallel lines*: $\hat{y} = 20 + 0.5x - 8d - 0.2\,xd$. The new version is 8 ms faster at $x = 0$ and its slope is $0.5 - 0.2 = 0.3$ ms per KB, so the gap grows with payload.

The extra column $x \cdot d$ is an **interaction term**: a product of two predictors, which lets the effect of one depend on the value of the other.

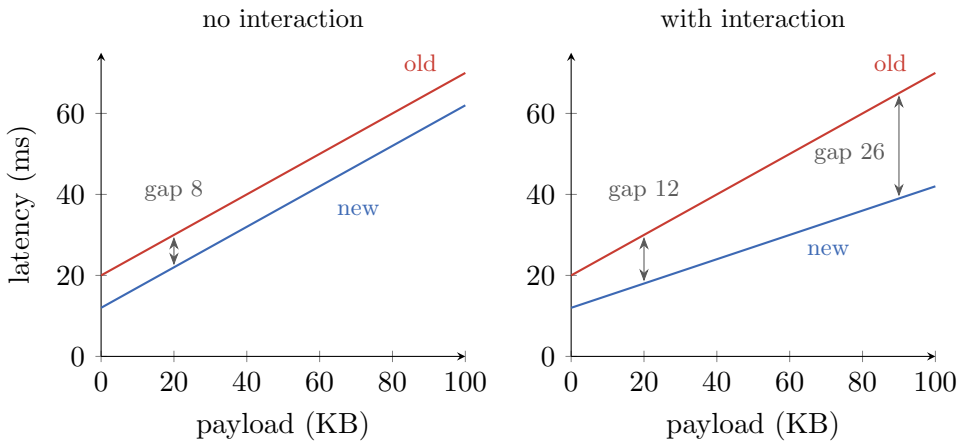


Figure 13.7: Predicted latency for the old (red) and new (blue) server versions. Left: without an interaction term the model can only shift the line, so the gap is the same 8 ms at every payload. Right: with an $x \cdot d$ term the slopes may differ, and the gap grows from 8 ms at 0 KB (12 ms at 20 KB, 26 ms at 90 KB) to 28 ms at 100 KB.

Common trap

Once an interaction is in the model, the coefficient on d alone (-8) is the effect of the new version *at* $x = 0$ only. Reporting it as "the effect of the new version" is wrong whenever $x = 0$ is far from typical payloads. Centring x (subtracting its mean) before forming the product makes that coefficient the effect at an average payload.

13.5.3 Polynomial features and overfitting

A curve is a line in disguise. To fit $\hat{y} = b_0 + b_1 x + b_2 x^2 + b_3 x^3$, put the columns $1, x, x^2, x^3$ into X and run the same normal equations. The model is nonlinear in x but linear in the b's, which is all least squares needs.

How high should the degree go? Take 10 noisy points from a wave and fit degrees 1, 3 and 9.

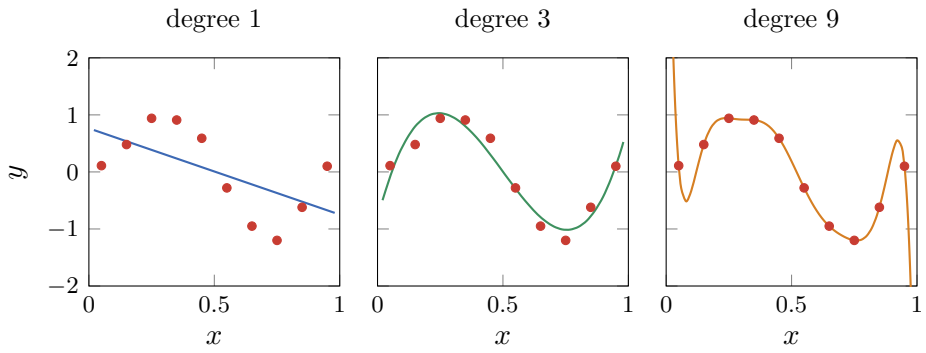


Figure 13.8: The same 10 points fitted with polynomials of degree 1, 3 and 9. Degree 1 misses the wave (underfitting). Degree 3 follows it. Degree 9 has 10 coefficients for 10 points, so it passes through every point exactly. It wiggles to do so, and just outside the outermost points it shoots off the chart: at $x = 0.98$ it predicts -2.8 (overfitting).

The degree-9 fit has zero training error, yet it predicts new data worse than any degree from 3 to 8. To see this, draw 200 fresh points from the same process and measure mean squared error (MSE) on them, the **test error**:

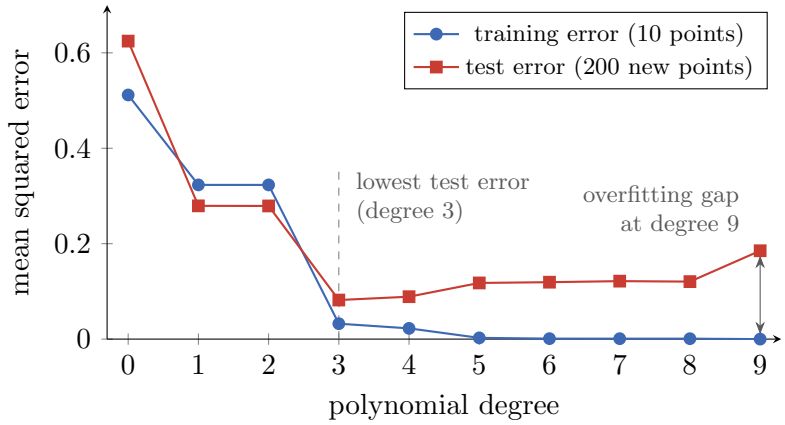


Figure 13.9: Training error never rises as the model grows; at degree 9 it is exactly zero. Test error falls, bottoms out at degree 3 (0.082), then rises to 0.185 at degree 9. The gap between the curves is overfitting: the model memorises the noise in its own sample.

Connection

This U-shaped test curve is the bias–variance trade-off. Simple models are biased (they cannot bend enough); flexible models have high variance (they bend to every quirk of the sample). Chapter 15 develops it with train/validation/test splits, cross-validation and regularisation (ridge and lasso), which add a penalty to $\|y - X\beta\|^2$ to keep coefficients small.

13.6 Multicollinearity

Suppose we add house size measured in square feet to a model that already has size in square metres. The two columns carry the same information: their correlation in our 40 houses is 0.9996 (the square-foot figure was rounded with a little recording noise). Here is what happens to the size coefficient:

model	size (m^2) coef	its SE
size + age	1.861	0.081
size + size_ft^2 + age	1.393	2.918

The standard error grew about 36 times. The fit itself barely changed (R^2 is 0.938 either way), and the combined effect $1.393 + 10.764 \times 0.0432 = 1.858$ per m^2 is still right. The model just cannot tell how to split the credit between two near-identical columns, so any split is roughly as good as any other.

Multicollinearity means some predictor is nearly a linear combination of the others. In the projection picture, two column arrows point in almost the same direction. The plane they span is well defined, so $\hat{y}$ is stable, but the coordinates of $\hat{y}$ along two nearly parallel arrows are not: a tiny wobble in y sends them flying in opposite directions (Figure 13.10).

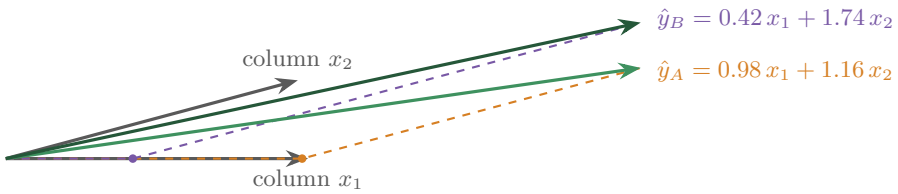


Figure 13.10: Why near-duplicate columns make coefficients unstable. The grey arrows are two predictor columns pointing almost the same way (15° apart). To reach a fitted vector you walk some distance along x_1 (to the dot), then parallel to x_2 (dashed path); those two distances are the coefficients. Two samples give green arrows $\hat{y}_A$ and $\hat{y}_B$ that differ only slightly, yet the x_1 coefficient drops from 0.98 to 0.42 while the x_2 coefficient rises from 1.16 to 1.74.

Definition 13.2 (Variance inflation factor)**.** Regress predictor x_j on all the other predictors and call the resulting R^2 value R_j^2. The **variance inflation factor** is

$$\mathrm{VIF}_j = \frac{1}{1 - R_j^2}.$$

The name is literal. One can show

$$\mathrm{Var}(\hat{\beta}_j) = \frac{\sigma^2}{\sum_i (x_{ij} - \bar{x}_j)^2} \cdot \mathrm{VIF}_j,$$

so the first factor is the simple-regression variance from Chapter 12, and VIF is how much the overlap with other predictors multiplies it. The standard error is multiplied by $\sqrt{\mathrm{VIF}_j}$. In the square-feet example VIF = 1274 and $\sqrt{1274} \approx 35.7$, matching the 36-fold jump.

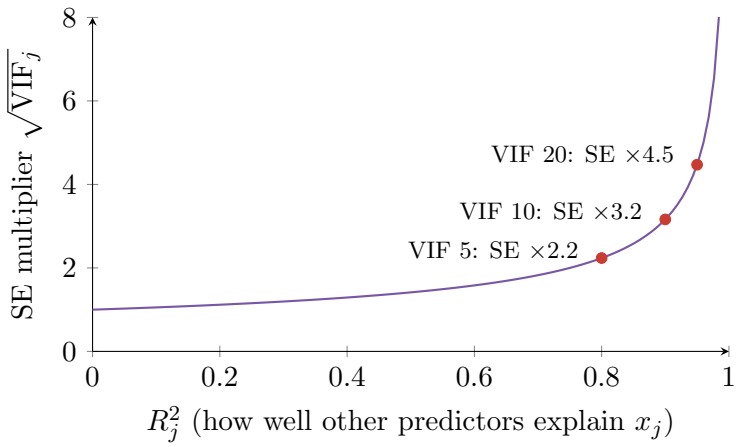


Figure 13.11: Standard errors stay modest until a predictor is largely explained by the others, then blow up. Common rules of thumb flag VIF above 5 or 10.

Intuition

Multicollinearity hurts interpretation, not prediction. If you only want $\hat{y}$, a model with two copies of size predicts fine. If you want to say "size matters, and by this much", drop one of the duplicates, combine them, or collect data where they vary separately. Perfect collinearity (VIF $= \infty$), such as the dummy trap, makes $X^\top X$ non-invertible.

13.7 Adjusted R^2

$R^2 = 1 - \text{SSE}/\text{SST}$ never decreases when you add a column. So it cannot tell you whether the new column helps. **Adjusted** R^2 charges for each coefficient:

$$R^2_{\text{adj}} = 1 - (1 - R^2)\,\frac{n-1}{n-p-1}.$$

Rearranged, $1 - R^2_{\text{adj}} = \dfrac{\text{SSE}/(n-p-1)}{\text{SST}/(n-1)}$: adjusted R^2 compares the residual variance to the total variance, each divided by its degrees of freedom instead of by n. It is the same $n-1$ logic as the sample variance in Chapter 2: each fitted coefficient uses up one degree of freedom. A new column lowers SSE but also lowers $n-p-1$, so adjusted R^2 rises only if SSE drops by enough to pay for it.

For the 40 houses with size, age and distance to the city centre, $R^2 = 0.9388$ and $p = 3$:

$$R^2_{\text{adj}} = 1 - 0.0612 \times \frac{39}{36} = 0.9337.$$

Drop distance and refit: R^2 falls slightly to 0.9377, but with $p = 2$,

$$R^2_{\text{adj}} = 1 - 0.0623 \times \frac{39}{37} = 0.9343,$$

which is higher. Distance added less than the price of one coefficient.

13.8 Uncertainty in the coefficients

Each $\hat\beta_j$ is computed from a random sample, so it has a sampling distribution (Chapter 6). Its standard deviation is the standard error we need for confidence intervals and tests.

Theorem 13.2 (Covariance of the least squares estimator)**.** *If $y = X\beta + \varepsilon$ with X fixed, $\mathbb{E}[\varepsilon] = 0$ and $\mathrm{Cov}(\varepsilon) = \sigma^2 I$ (errors uncorrelated with equal variance σ^2), then $\mathbb{E}[\hat\beta] = \beta$ and*

$$\mathrm{Cov}(\hat\beta) = \sigma^2 (X^\top X)^{-1}.$$

Why it works

Write $A = (X^\top X)^{-1} X^\top$, so $\hat\beta = Ay$. Substitute $y = X\beta + \varepsilon$:

$$\hat\beta = AX\beta + A\varepsilon = \beta + A\varepsilon,$$

since $AX = (X^\top X)^{-1} X^\top X = I$. The error term has mean zero, so $\hat\beta$ is unbiased. For a constant matrix A, $\mathrm{Cov}(A\varepsilon) = A\,\mathrm{Cov}(\varepsilon)\,A^\top$ (the matrix version of $\mathrm{Var}(aX) = a^2\,\mathrm{Var}(X)$). So

$$\mathrm{Cov}(\hat\beta) = A(\sigma^2 I)A^\top = \sigma^2 (X^\top X)^{-1} X^\top X (X^\top X)^{-1} = \sigma^2 (X^\top X)^{-1}.$$

The diagonal of $\mathrm{Cov}(\hat\beta)$ holds the variances of the individual coefficients; the off-diagonal entries say how their errors move together. We do not know σ^2, so we estimate it from the residuals, dividing by the residual degrees of freedom:

$$s^2 = \frac{\mathrm{SSE}}{n - p - 1}, \qquad \mathrm{SE}(\hat\beta_j) = s\sqrt{\left[(X^\top X)^{-1}\right]_{jj}}.$$

Example 13.2 (Standard errors for the four houses)**.** Residuals were $(0.1, 0.7, -1.7, 0.9)$, so $\mathrm{SSE} = 0.01 + 0.49 + 2.89 + 0.81 = 4.2$ and $s^2 = 4.2/(4-2) = 2.1$. The inverse was $(X^\top X)^{-1} = \frac{1}{20}\begin{bmatrix} 30 & -10 \\ -10 & 4 \end{bmatrix}$. Hence

$$\mathrm{SE}(\hat\beta_1) = \sqrt{2.1 \times \tfrac{4}{20}} = \sqrt{0.42} = 0.648, \qquad \mathrm{SE}(\hat\beta_0) = \sqrt{2.1 \times \tfrac{30}{20}} = \sqrt{3.15} = 1.775.$$

The slope SE matches Chapter 12's $s/\sqrt{S_{xx}} = \sqrt{2.1/5} = 0.648$. The negative off-diagonal entry says that if the slope comes out too high, the intercept tends to come out too low: the line pivots around $(\bar x, \bar y)$.

With normal errors, $(\hat\beta_j - \beta_j)/\,\mathrm{SE}(\hat\beta_j)$ follows a t distribution with $n - p - 1$ degrees of freedom. That gives the usual recipes from Chapters 7 and 9:

$$\hat\beta_j \pm t^*_{n-p-1}\,\mathrm{SE}(\hat\beta_j), \qquad t = \frac{\hat\beta_j}{\mathrm{SE}(\hat\beta_j)} \quad \text{for } H_0\colon \beta_j = 0.$$

13.9 Reading a regression output table

Here is the full fit of price on size, age and distance (km to the centre) for the 40 houses, in the layout printed by Python's `statsmodels`.

```python
import statsmodels.formula.api as smf
fit = smf.ols("price ~ size + age + dist", data=houses).fit()
print(fit.summary())
# categorical and interaction terms use the same formula language:
# smf.ols("latency ~ C(region) + payload * new_version", data=df)
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  price   R-squared:                       0.939
Model:                            OLS   Adj. R-squared:                  0.934
Method:                 Least Squares   F-statistic:                     183.9
No. Observations:                  40   Prob (F-statistic):           6.98e-22
Df Residuals:                      36   Log-Likelihood:                -179.55
Df Model:                           3   AIC:                             367.1
                                        BIC:                             373.9
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     68.2558     15.856      4.305      0.000      36.099     100.413
size           1.8437      0.084     21.885      0.000       1.673       2.015
age           -1.4069      0.215     -6.547      0.000      -1.843      -0.971
dist          -0.4276      0.536     -0.797      0.431      -1.515       0.660
==============================================================================
```

Line by line, top block:

- **No. Observations** $n = 40$. **Df Model** $p = 3$ predictors. **Df Residuals** $n - p - 1 = 36$: the divisor in s^2 and the degrees of freedom for every t below.
- **R-squared** 0.939: the model explains 93.9% of the variance in price. **Adj. R-squared** 0.934 is the penalised version from Section 13.7.
- **F-statistic** 183.9 and **Prob (F-statistic)** 7×10^{-22}: a test of H_0: *all three slopes are zero.* It compares explained variance per predictor to residual variance per degree of freedom, $F = \frac{(\mathrm{SST}-\mathrm{SSE})/p}{\mathrm{SSE}/(n-p-1)}$. A tiny p-value says the model as a whole beats predicting $\bar{y}$ for every house.
- **Log-Likelihood**, **AIC**, **BIC**: measures of fit based on the likelihood, with AIC and BIC adding a penalty per parameter. Lower AIC/BIC is better when comparing models on the same data. Chapter 14 explains likelihood.

Coefficient block, one row per column of X:

- **coef**: $\hat{\beta}_j$. Holding age and distance fixed, each extra m^2 adds \$1,844; holding size and distance fixed, each year of age subtracts \$1,407. The intercept (68.3) is the predicted price of a 0 m^2, new house at the city centre: a mathematical anchor, not a real house.
- **std err**: $s\sqrt{[(X^\top X)^{-1}]_{jj}}$.
- **t**: coef divided by std err. For size, $1.8437/0.0842 = 21.9$: the estimate sits 22 standard errors from zero.
- **P> |t|**: two-sided p-value for H_0: $\beta_j = 0$, from the t_{36} distribution. For distance it is 0.431: the data are consistent with distance having no effect once size and age are known.
- **[0.025 0.975]**: the 95% confidence interval $\hat{\beta}_j \pm 2.028$ SE, since $t^*_{36} = 2.028$. For size: $1.8437 \pm 2.028 \times 0.0842 = [1.673,\ 2.015]$. The distance interval $[-1.515,\ 0.660]$ contains 0,

matching its large p-value.

The residual standard error, $s = \sqrt{\text{SSE}/36} = 22.7$, is not printed in the summary. In `statsmodels` it is the square root of `fit.scale`. It says typical prediction misses are around \$23k.

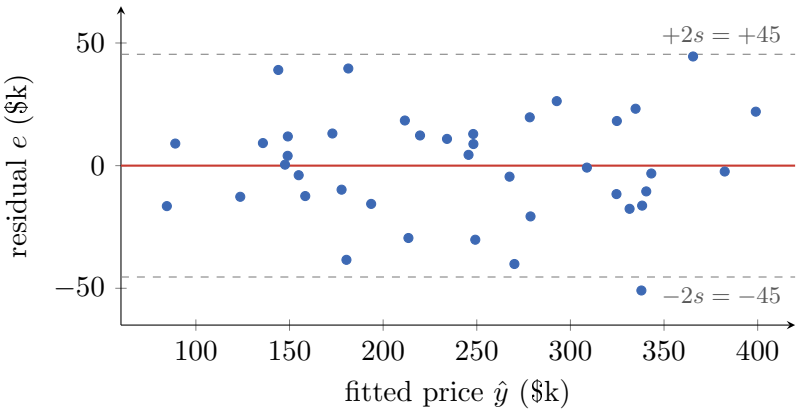


Figure 13.12: Before trusting the table, check the residuals against fitted values. Here they form a patternless band around zero with roughly constant spread, and nearly all lie within the dashed lines at $\pm 2s = \pm 45$ (one sits just on the upper line). A funnel or a curve here would mean the standard errors in the table are not to be trusted.

Common trap

A small p-value says a coefficient is distinguishable from zero, not that it is large or causal. A large p-value (distance, 0.43) does not prove the effect is zero: the interval runs from -1.5 to $+0.7$ thousand dollars per km, which includes effects someone might care about. And the p-values are computed one at a time; scanning twenty of them for stars runs into the multiple-testing problem of Chapter 11.

13.10 Logistic regression: predicting yes or no

13.10.1 Why a straight line fails

Twenty visitors to a product page: minutes spent on the page and whether they bought (1) or not (0).

minutes	1	2	2	3	4	4	5	5	6	6
bought	0	0	0	0	0	1	0	0	1	0
minutes	7	7	8	8	9	10	11	12	13	14
bought	1	0	1	1	0	1	1	1	1	1

We want $\mathbb{P}(\text{buy} \mid \text{minutes})$. Fitting an ordinary line to the 0/1 column gives $\hat{y} = -0.120 + 0.0906 \cdot$ minutes. At 0 minutes it predicts a probability of -0.12; at 16 minutes, 1.33. Neither is a probability.

The errors are not normal either (each y is 0 or 1) and their variance $p(1-p)$ changes with x, so the standard errors from Section 13.8 are wrong too.

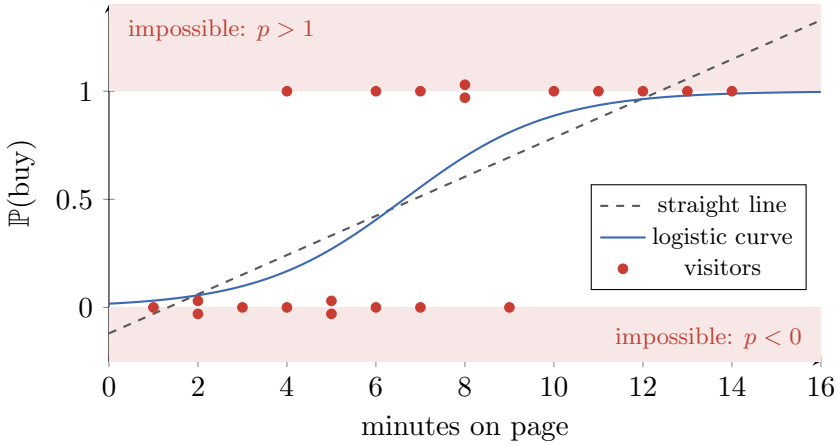


Figure 13.13: The 20 visitors: dots at 1 bought, dots at 0 did not (two visitors with the same minutes are nudged slightly apart). The straight line leaves $[0, 1]$ into the shaded impossible regions below 1.3 minutes and above 12.4 minutes. The logistic curve bends to stay inside, approaching 0 and 1 without crossing.

13.10.2 Odds, log-odds and the sigmoid

The fix is to model a quantity that can take any real value, then squash it back into $[0, 1]$. Two steps get there.

The **odds** of an event with probability p are $p/(1-p)$: how many times more likely it is to happen than not. Probability 0.8 is odds of 4 ("4 to 1"); probability 0.5 is odds 1. Odds range over $(0, \infty)$.

The **log-odds**, or **logit**, is $\log \frac{p}{1-p}$ (natural log). It ranges over all real numbers: $p = 0.5$ gives 0, $p = 0.8$ gives $\log 4 = 1.386$, $p = 0.2$ gives -1.386. A line can live on that scale without hitting any wall.

Definition 13.3 (Logistic regression)**.** For a 0/1 response with predictors $x_1, \ldots, x_p$, logistic regression models

$$\log \frac{p}{1-p} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p = x^\top \beta,$$

equivalently

$$p = \sigma(x^\top \beta), \qquad \sigma(z) = \frac{1}{1+e^{-z}}.$$

The function σ is the **sigmoid** (or logistic function): it maps any real number z to a probability.

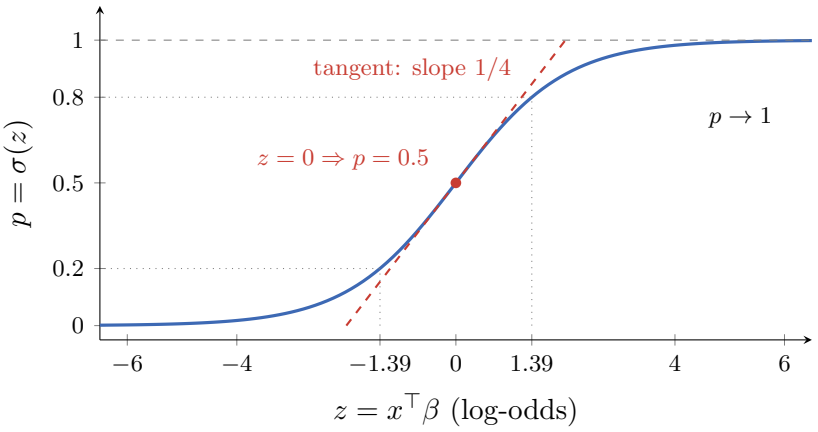


Figure 13.14: The sigmoid turns log-odds into probability. It is symmetric about $(0, 0.5)$, steepest there (dashed tangent, slope $1/4$), and flattens toward 0 and 1 without reaching them. The dotted guides read off two values: log-odds $+1.39$ is probability 0.8 and log-odds -1.39 is probability 0.2.

13.10.3 Interpreting coefficients as odds ratios

Fitting the page data (method in the next subsection) gives

$$\log \frac{\hat{p}}{1-\hat{p}} = -4.045 + 0.610 \cdot \text{minutes}.$$

On the log-odds scale this is a line: each extra minute adds 0.610 to the log-odds. Exponentiate and it becomes multiplication: each extra minute multiplies the odds by

$$e^{0.610} = 1.84.$$

That number is the **odds ratio**: the factor by which the odds change for a one-unit increase in the predictor, holding other predictors fixed.

minutes	log-odds $-4.045 + 0.610x$	odds	$\hat{p}$
2	-2.825	0.059	0.056
5	-0.994	0.370	0.270
8	0.837	2.31	0.698
12	3.278	26.5	0.964

From 5 to 8 minutes the odds go from 0.370 to 2.31, a factor of $2.31/0.370 = 6.24 = 1.84^3$. The probability rises by 0.43. From 8 to 11 minutes the odds again grow 6.24-fold, but the probability can only rise from 0.70 to about 0.94. Constant effect on the odds; diminishing effect on the probability (Figure 13.15).

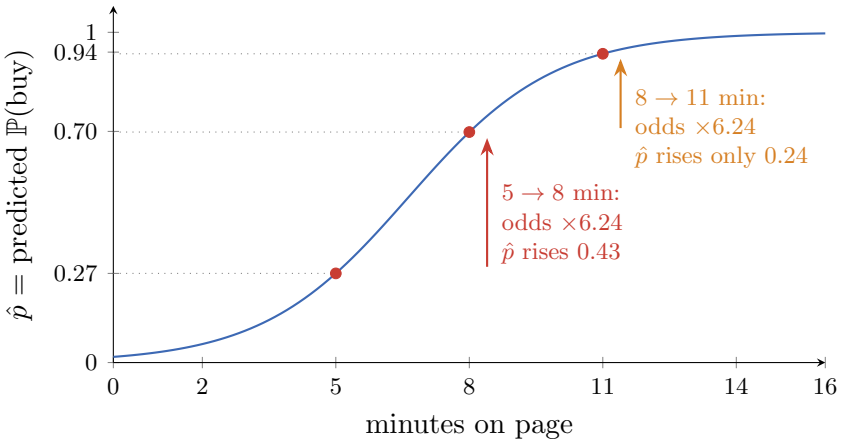


Figure 13.15: The fitted curve for the page data. Each three-minute step multiplies the odds of buying by the same $1.84^3 = 6.24$, but the probability gained shrinks as $\hat{p}$ nears 1: $+0.43$ from 5 to 8 minutes, only $+0.24$ from 8 to 11.

Common trap

An odds ratio of 1.84 does not mean "84% more likely to buy". It means the odds, not the probability, grow by 84%. When p is small the two are close (odds $\approx p$), which is why the confusion survives; when p is near 0.5 or above they diverge sharply.

Pause

Pause and think: a coefficient of $\beta = 0$ gives an odds ratio of $e^0 = 1$. What does a negative coefficient do to the odds? (It gives an odds ratio below 1: $\beta = -0.69$ halves the odds.)

13.10.4 Fitting by maximum likelihood

There are no residuals to square in the usual sense, so we fit differently. For each candidate β, the model assigns visitor i a probability $p_i = \sigma(x_i^\top \beta)$ of buying. The probability the model assigns to what *actually happened* is p_i if they bought and $1 - p_i$ if not, which can be written in one expression as $p_i^{y_i}(1 - p_i)^{1-y_i}$. Assuming visitors are independent, multiply over all of them:

$$L(\beta) = \prod_{i=1}^{n} p_i^{y_i}(1 - p_i)^{1-y_i}.$$

L is the **likelihood**: how probable the observed data are under a given β. **Maximum likelihood** picks the β that makes the data most probable. Products of many small numbers are awkward, so take logs and flip the sign to get a loss to minimise:

$$\ell(\beta) = -\sum_{i=1}^{n} \Big[y_i \log p_i + (1 - y_i) \log(1 - p_i) \Big].$$

Divided by n, this is the **log loss**, also called binary cross-entropy. Each visitor contributes $-\log$(probability given to the true outcome).

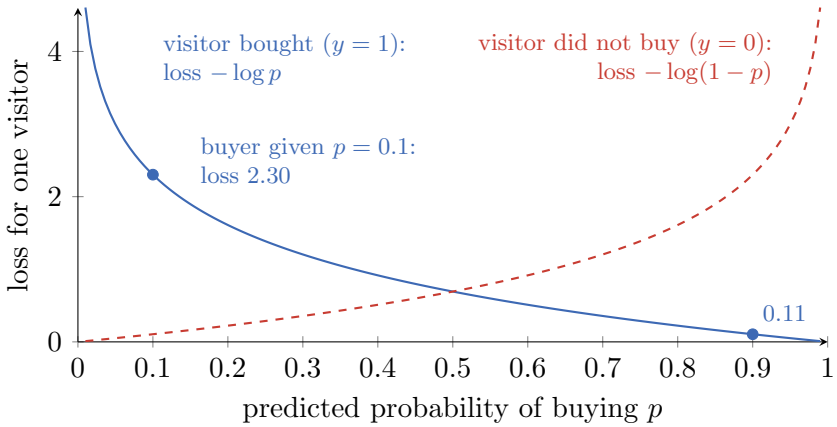


Figure 13.16: Log loss punishes confident mistakes hardest. For a buyer (solid curve), predicting $p = 0.9$ costs only 0.11, but predicting $p = 0.1$ costs 2.30 (marked dots). Each curve shoots to infinity as the prediction becomes certain and wrong.

There is no closed-form solution like the normal equations. But ℓ is convex (a single bowl with no false bottoms), so iterative methods find the minimum reliably. Software uses Newton's method, which re-solves a weighted least squares problem at each step; machine learning libraries use gradient descent. Both use the same gradient,

$$\nabla \ell(\beta) = X^\top (p - y),$$

which has the same shape as the least squares gradient $X^\top(X\beta - y)$, with predicted probabilities in place of predicted values. At the optimum the residuals $y - \hat{p}$ are again orthogonal to every column of X.

For the page data the minimum log loss is 0.415 per visitor. Standard errors come from the curvature of ℓ at its minimum: the analogue of $\sigma^2(X^\top X)^{-1}$ is $(X^\top W X)^{-1}$, where W is diagonal with entries $\hat{p}_i(1 - \hat{p}_i)$. Here $\mathrm{SE}(\hat{\beta}_1) = 0.268$, so $z = 0.610/0.268 = 2.27$ and $p = 0.023$. The 95% interval for β_1 is $0.610 \pm 1.96 \times 0.268$; exponentiating its endpoints gives an odds-ratio interval of $[1.09,\ 3.12]$.

```
import numpy as np, statsmodels.formula.api as smf
fit = smf.logit("bought ~ minutes", data=visits).fit()
print(fit.params)                 # Intercept -4.045, minutes 0.610
print(np.exp(fit.params))         # odds ratio per minute: 1.84
print(np.exp(fit.conf_int()))     # odds-ratio CI: [1.09, 3.12]
# sklearn.linear_model.LogisticRegression adds a ridge penalty by default
```

Common trap

If some combination of predictors separates the 0s from the 1s perfectly, the likelihood keeps increasing as the coefficients grow without bound, and the fit "converges" to huge numbers with huge standard errors. This is called **complete separation**. The fix is more data, fewer predictors, or a penalty on coefficient size.

13.10.5 From probabilities to decisions

A classifier needs a yes/no answer. The usual rule is: predict 1 when $\hat{p} \geq 0.5$. Since $\sigma(z) \geq 0.5$ exactly when $z \geq 0$, this is the same as $x^\top \hat{\beta} \geq 0$. For the page data, $-4.045 + 0.610x \geq 0$ when $x \geq 6.63$ minutes.

With two predictors, $\hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 = 0$ is a straight line in the (x_1, x_2) plane: the **decision boundary**. With p predictors it is a flat hyperplane. Figure 13.17 shows 30 users described by two engagement scores, with the fitted model $\log \frac{\hat{p}}{1-\hat{p}} = -12.55 + 1.94x_1 + 1.98x_2$.

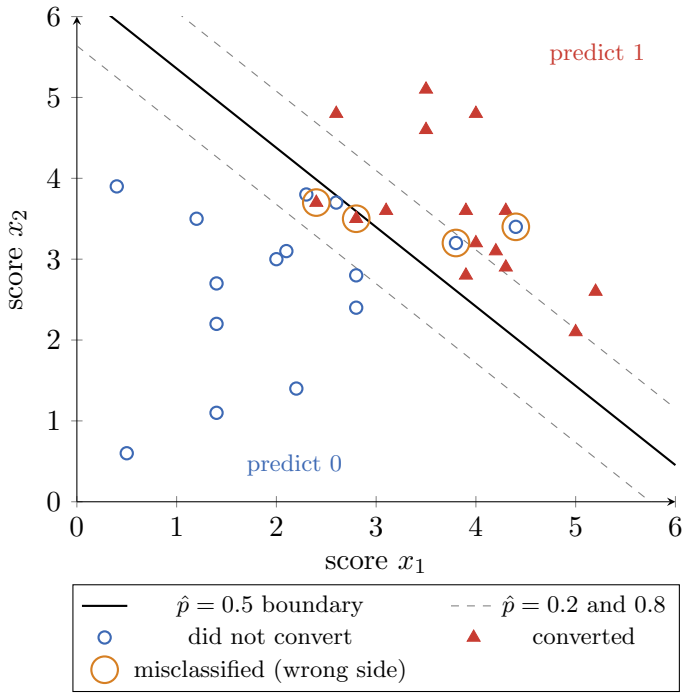


Figure 13.17: The decision boundary of logistic regression is a straight line where the log-odds equal zero: above-right the model predicts “convert”, below-left “not”. The dashed lines are where $\hat{p} = 0.8$ (upper) and $\hat{p} = 0.2$ (lower). The four ringed users fall on the wrong side, three of them inside that uncertain band, so training accuracy is 26/30.

Connection

A logistic regression is a neural network with no hidden layer: a weighted sum of inputs followed by a sigmoid, trained by minimising cross-entropy with gradient descent. Replace the sigmoid by a softmax over K classes and you get multinomial logistic regression, the output layer of nearly every classifier. Curved boundaries come from feeding in nonlinear features, exactly as polynomial columns bent the regression line.

13.11 Generalised linear models: the umbrella

Linear and logistic regression share a three-part structure:

1. a **linear predictor** $\eta = x^\top \beta$, the weighted sum of inputs;
2. a **link function** g that connects the mean of y to η through $g(\mathbb{E}[y \mid x]) = \eta$;
3. a **distribution** for y around its mean.

Linear regression uses the identity link and a normal distribution. Logistic regression uses the logit link and a Bernoulli distribution (a single 0/1 trial). Counts, such as errors per hour or clicks per page view, use a log link and a Poisson distribution, so $\log \mathbb{E}[y] = x^\top \beta$ and each coefficient multiplies the expected count by e^{β_j}. Any model of this form is a **generalised linear model** (GLM), and all of them are fitted by maximum likelihood with the same algorithm.

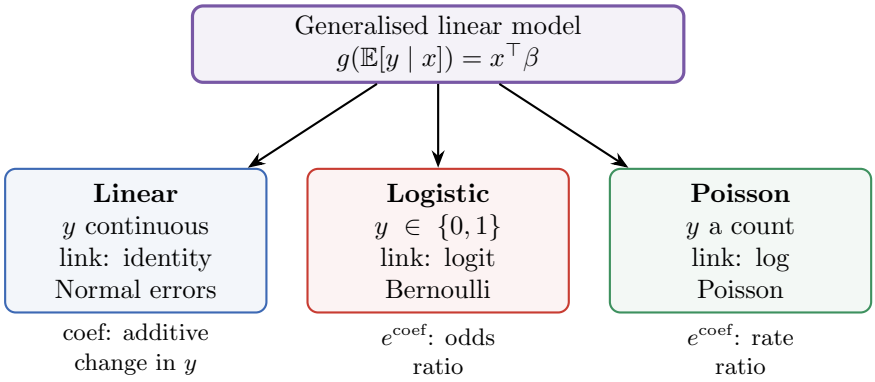


Figure 13.18: Generalised linear models share one linear predictor and differ in the link function and the distribution of y. The link determines how to read a coefficient: additively, as an odds ratio, or as a rate ratio.

History

In 1944 the American statistician Joseph Berkson, working on dose–response data (what fraction of animals respond at each dose of a drug), proposed the log-odds transform and named it the "logit". The competing approach of the day, the "probit", used the normal distribution's cumulative curve and needed tables of the normal to fit; the logit had a simple closed form. For decades, regression for continuous data, for proportions and for counts were taught as separate subjects with separate fitting tricks. In 1972 John Nelder and Robert Wedderburn showed they were one model with different link functions and distributions, all fittable by a single iterative weighted least squares algorithm. That paper created the term "generalised linear model" and the software (GLIM, and later R's `glm`) that made logistic and Poisson regression routine.

13.12 Summary

- **Matrix form**: $y = X\beta + \varepsilon$, with design matrix X (rows = observations, columns = intercept plus predictors).
- **Normal equations**: $X^\top X \hat{\beta} = X^\top y$, so $\hat{\beta} = (X^\top X)^{-1} X^\top y$ when the columns are independent.

- **Geometry**: $\hat{y} = Hy$ is the orthogonal projection of y onto the column space of X; the residual satisfies $X^\top e = 0$. Leverages h_{ii} sum to $p + 1$.
- **Interpretation**: $\hat{\beta}_j$ is the change in $\hat{y}$ per unit of x_j with the other predictors fixed. Adding or dropping a correlated variable can change it, even flip its sign (omitted variable bias).
- **Features**: dummies ($k - 1$ for k categories), interactions (products, non-parallel lines), polynomials (curves). More flexibility lowers training error and eventually raises test error.
- **Multicollinearity**: $\text{VIF}_j = 1/(1 - R_j^2)$ multiplies $\text{Var}(\hat{\beta}_j)$; it hurts interpretation, not prediction.
- **Adjusted** $R^2 = 1 - (1 - R^2)\frac{n-1}{n-p-1}$ can fall when a useless variable is added.
- **Uncertainty**: $\text{Cov}(\hat{\beta}) = \sigma^2(X^\top X)^{-1}$, estimated with $s^2 = \text{SSE}/(n - p - 1)$; t-tests and CIs use $n - p - 1$ degrees of freedom.
- **Logistic regression**: $\log \frac{p}{1-p} = x^\top \beta$, $p = \sigma(x^\top \beta)$; e^{β_j} is an odds ratio; fit by maximum likelihood (minimum log loss); decision boundary $x^\top \hat{\beta} = 0$ is linear.
- **GLMs**: linear predictor + link + distribution; linear, logistic and Poisson regression are special cases.

Chapter 14 takes the idea that fitted logistic regression, maximum likelihood, and develops it as a general method for estimation, together with its Bayesian counterpart.

13.13 Exercises

1. For $x = (0, 1, 2, 3, 4)$ and $y = (1, 3, 2, 5, 4)$, write the design matrix, compute $X^\top X$ and $X^\top y$, and solve the normal equations.
2. A model gives $\widehat{\text{price}} = 50 + 2.0 \cdot \text{size} - 1.5 \cdot \text{age}$ (price in \$k, size in m^2, age in years). Predict the price of a 120 m^2, 10-year-old house, and state in words what -1.5 means.
3. A regression of salary on years of experience gives a slope of 3 (\$k per year). Adding education changes it to 2. Using the omitted variable bias formula, what does this tell you about the relationship between experience and education, given that education has a positive coefficient?
4. A dataset has a "day of week" column with 7 values. How many dummy columns should a model with an intercept use, and what goes wrong if you use 7?
5. In $\hat{y} = 10 + 2x + 5d + 3xd$ with $d \in \{0, 1\}$, give the intercept and slope of the line for each group. Are the lines parallel?
6. With $n = 30$, $p = 5$ and $R^2 = 0.40$, compute adjusted R^2.
7. A predictor has $R_j^2 = 0.8$ when regressed on the other predictors. By what factor is its standard error inflated?
8. Show that the hat matrix satisfies $HH = H$, and explain geometrically why this must be true.
9. In a logistic regression for churn, the coefficient on "received discount" is 0.405. Compute the odds ratio. If a customer's churn probability is 0.20 without the discount, what is it with the discount?

10. A logistic model predicts $\hat{p} = 0.8$ for one positive example and $\hat{p} = 0.3$ for one negative example. Compute the average log loss over the two.

Answers

1. X has rows $(1, x_i)$. $X^\top X = \begin{bmatrix} 5 & 10 \\ 10 & 30 \end{bmatrix}$, $X^\top y = (15,\ 38)$. Determinant 50, so $\hat{\beta} = \frac{1}{50}(30 \cdot 15 - 10 \cdot 38,\ -10 \cdot 15 + 5 \cdot 38) = (1.4,\ 0.8)$.
2. $50 + 240 - 15 = 275$, so \$275k. Among houses of the same size, each extra year of age lowers the predicted price by \$1,500.
3. Simple slope = direct slope + (education coef)$\times\delta$, so $3 = 2 + (\text{education coef}) \times \delta$. The product is $1 > 0$; with a positive education coefficient, $\delta > 0$: more experienced people in this data also have more education, and part of the simple slope was education's effect.
4. Six, with one day as baseline. With seven, the dummies sum to the intercept column, $X^\top X$ is singular and the coefficients are not identifiable.
5. $d = 0$: $\hat{y} = 10 + 2x$. $d = 1$: $\hat{y} = 15 + 5x$. Not parallel; the interaction coefficient 3 is the difference in slopes.
6. $1 - 0.60 \times 29/24 = 1 - 0.725 = 0.275$.
7. VIF $= 1/0.2 = 5$; the SE is multiplied by $\sqrt{5} \approx 2.24$.
8. $HH = X(X^\top X)^{-1}X^\top X(X^\top X)^{-1}X^\top = X(X^\top X)^{-1}X^\top = H$. Projecting a vector that already lies in the column space leaves it where it is.
9. Odds ratio $e^{0.405} \approx 1.50$. Odds $0.2/0.8 = 0.25$ become 0.375, so $p = 0.375/1.375 \approx 0.273$.
10. Losses $-\log 0.8 = 0.223$ and $-\log(1 - 0.3) = -\log 0.7 = 0.357$; average 0.290.

Chapter 14

Likelihood, Estimation and Bayes

Chapters 12 and 13 fitted lines and logistic curves by minimising a loss: squared error for regression, cross-entropy for classification. Those losses were handed to you. This chapter explains where they come from. The answer is one idea, *likelihood*, and it also tells us how precise a fitted parameter is and how to fold prior knowledge into a fit. By the end, ridge and lasso regression will turn out to be the same idea again, with a prior attached.

14.1 Estimators and how they miss

14.1.1 An estimator is a recipe

Five API calls to a new service took these times in milliseconds:

$$4, \quad 7, \quad 5, \quad 9, \quad 5.$$

We want the true mean latency μ and the true variance σ^2 of this service. Two natural guesses are the sample mean and the sample variance:

$$\bar{x} = \frac{4+7+5+9+5}{5} = 6, \qquad \sum (x_i - \bar{x})^2 = 4 + 1 + 1 + 9 + 1 = 16.$$

Dividing 16 by $n-1 = 4$ gives $s^2 = 4$ (Chapter 2). Dividing by $n = 5$ gives 3.2. Both are reasonable recipes. Which is better? To answer that we need to say what "better" means for a recipe, not for one number.

Definition 14.1. An **estimator** $\hat{\theta}$ is a function of the data that is used to guess a parameter θ. Picture a machine: data go in, one number comes out. The number it produces on the data you actually have is the **estimate**.

Because the data are random, the estimator is a random variable. If we reran the experiment, we would get five different latencies and a different $\bar{x}$. The spread of these hypothetical outputs is the **sampling distribution** of the estimator (Chapter 6): the histogram you would see if you repeated the whole study many times.

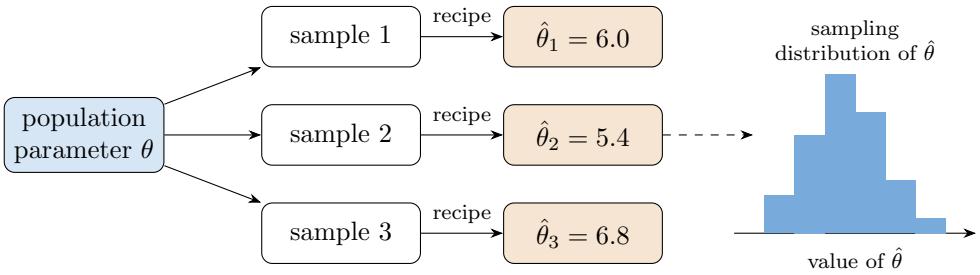


Figure 14.1: The same recipe applied to three different samples gives three different estimates (6.0, 5.4, 6.8). Repeat the study many times and the estimates pile up into the histogram on the right. Judging an estimator means judging that whole histogram, not one number.

14.1.2 Bias, variance and mean squared error

Two things can go wrong with a recipe. Its outputs can be centred in the wrong place, or they can be scattered widely.

Definition 14.2. The **bias** of $\hat{\theta}$ is $\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta$: how far the centre of the sampling distribution sits from the truth. The **variance** $\text{Var}(\hat{\theta})$ measures how scattered the outputs are around their own centre. The **mean squared error** is $\text{MSE}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2]$: the average squared distance from the truth.

A dartboard makes the two failure modes visible. The bullseye is θ; each dart is the estimate from one repeated study.

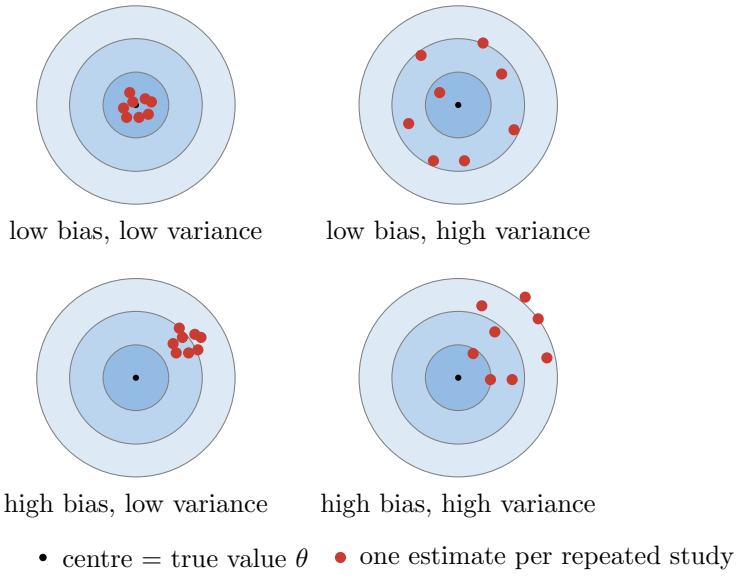


Figure 14.2: Bias is where the cloud of darts is centred; variance is how wide the cloud is. The top-left estimator is the goal. The bottom-left one is precise but consistently wrong, and no amount of repetition reveals that from the darts alone.

The mean squared error charges for both failures, and it splits cleanly into the two.

Theorem 14.1 (Bias–variance decomposition)**.** *For any estimator,* $\mathrm{MSE}(\hat\theta) = \mathrm{Bias}(\hat\theta)^2 + \mathrm{Var}(\hat\theta)$.

Why it works

Write $m = \mathbb{E}[\hat\theta]$ for the centre of the darts. Add and subtract it:

$$\hat\theta - \theta = \underbrace{(\hat\theta - m)}_{\text{scatter}} + \underbrace{(m - \theta)}_{\text{bias}}.$$

Square and take expectations:

$$\mathbb{E}[(\hat\theta - \theta)^2] = \mathbb{E}[(\hat\theta - m)^2] + 2(m-\theta)\,\mathbb{E}[\hat\theta - m] + (m-\theta)^2.$$

The middle term vanishes because $\mathbb{E}[\hat\theta - m] = m - m = 0$: scatter around the centre averages to zero. The first term is $\mathrm{Var}(\hat\theta)$ and the last is Bias^2. It is Pythagoras: the error vector has a "random" leg and a "systematic" leg at right angles.

Pause

Pause and think: an estimator always outputs the number 5, whatever the data. What are its variance and its MSE? (Variance 0; MSE $= (5-\theta)^2$. Superb if $\theta = 5$, useless otherwise. Low variance alone is not a virtue.)

14.1.3 Unbiased is not the same as best

Return to the variance of the latencies. For Normal data with n points, the sum of squared deviations $SS = \sum(x_i - \bar x)^2$ satisfies $\mathbb{E}[SS] = (n-1)\sigma^2$ and $\mathrm{Var}(SS) = 2(n-1)\sigma^4$ (see Probability From the Ground Up, Ch. 30). So dividing SS by some number d gives

$$\mathrm{Bias} = \frac{n-1}{d}\sigma^2 - \sigma^2, \qquad \mathrm{Var} = \frac{2(n-1)}{d^2}\sigma^4.$$

With $n = 5$ and $\sigma^2 = 1$:

divisor d	bias^2	variance	MSE
$n-1 = 4$ (Bessel, Chapter 2)	0	0.500	0.500
$n = 5$	0.040	0.320	0.360
$n+1 = 6$	0.111	0.222	0.333

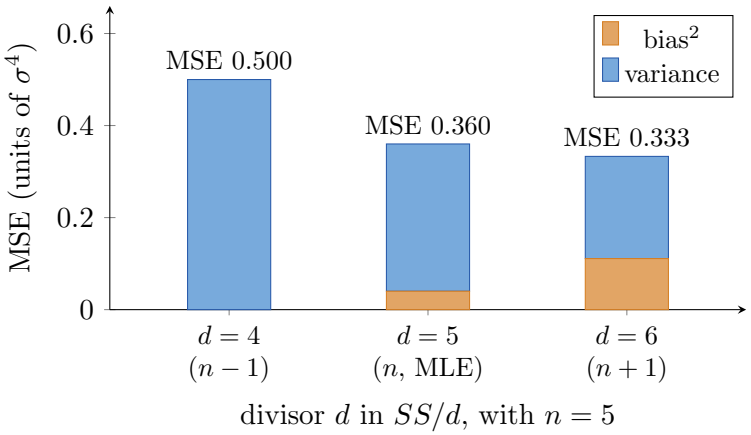


Figure 14.3: Each bar is the total MSE of one divisor, split into bias^2 (orange) and variance (blue). Dividing by a larger number adds a little bias but removes more variance, so the total falls: the unbiased choice $d = n - 1$ has the largest MSE of the three.

Unbiasedness is a property of the centre of the darts. It says nothing about how close any single dart lands. Machine learning leans on exactly this trade: regularisation (Section 14.6) deliberately adds bias to cut variance.

14.1.4 Consistency

An estimator is **consistent** if it closes in on the truth as the sample grows: for any tolerance $\varepsilon > 0$, $\mathbb{P}(|\hat{\theta}_n - \theta| > \varepsilon) \to 0$ as $n \to \infty$. In pictures, the sampling distribution piles up into a spike at θ. A simple sufficient condition is that both bias and variance go to zero, because then MSE goes to zero.

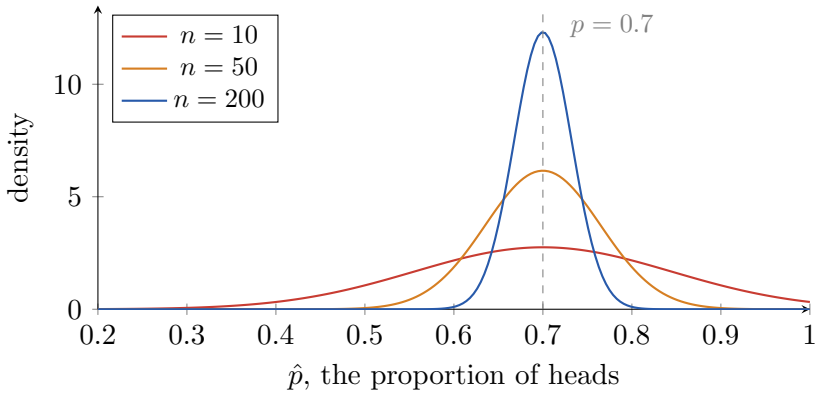


Figure 14.4: Sampling distribution of the proportion $\hat{p}$ when the true $p = 0.7$ (Normal approximation, SD $\sqrt{0.21/n}$). The curve tightens around the truth as n grows: $\hat{p}$ is consistent.

Both variance recipes above are consistent: their bias (0 or $-\sigma^2/n$) and their variance both vanish as n grows. At large n the choice of divisor stops mattering.

14.1.5 Method of moments, quickly

The oldest recipe for building estimators: match sample averages to their theoretical formulas and solve. A **moment** is an average of a power, such as $\mathbb{E}[X]$ or $\mathbb{E}[X^2]$.

Example 14.1. A job's runtime in seconds is uniform between 0 and an unknown maximum θ. Five runs took 3.1, 0.4, 5.2, 1.9 and 4.4, so $\bar{x} = 3.0$. Theory says $\mathbb{E}[X] = \theta/2$. Setting $\theta/2 = 3.0$ gives $\hat{\theta}_{\text{MoM}} = 6.0$.

Method of moments is quick and usually consistent, but it can be wasteful or even silly: if one run had taken 6.5 seconds while $\bar{x}$ stayed at 3.0, it would still say $\theta = 6.0$, a value the data have already ruled out. The method we turn to next never makes that mistake.

14.2 Likelihood

14.2.1 Turning the probability around

A coin is flipped 10 times and lands heads 7 times. Which values of p, the chance of heads, are plausible?

For any candidate p we can compute the probability of what we saw. For one specific sequence with 7 heads and 3 tails, that probability is $p^7(1-p)^3$:

$$p = 0.5: \quad 0.5^7\, 0.5^3 = 0.000977,$$
$$p = 0.7: \quad 0.7^7\, 0.3^3 = 0.002224.$$

The data are about 2.28 times more probable under $p = 0.7$ than under $p = 0.5$. Do this for every p and plot.

Definition 14.3. The **likelihood function** is $L(\theta) = p(\text{data} \mid \theta)$, read as a function of θ with the data held fixed. It scores each candidate parameter by how well it predicts the data you already have.

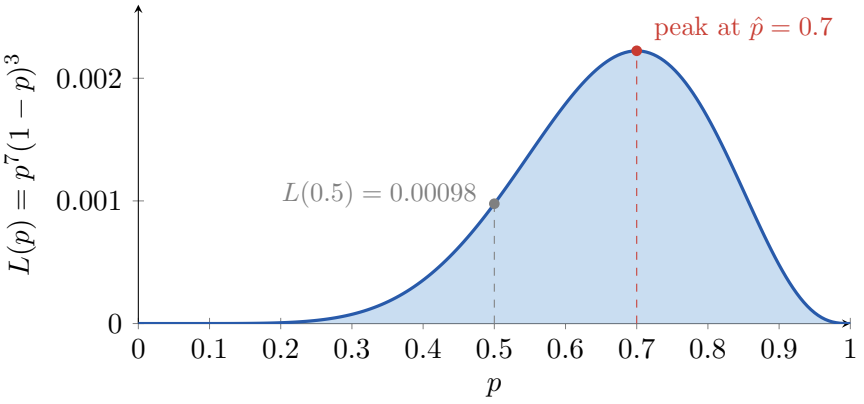


Figure 14.5: Likelihood of p after 7 heads in 10 flips. It peaks at 0.7, is still sizeable at 0.5, and is essentially zero below 0.2. The width of the hump shows how uncertain p still is.

Common trap

The likelihood is not a probability distribution over θ. It does not integrate to 1 (the area under Figure 14.5 is about 0.00076), and "$L(0.7) = 0.0022$" does not mean "$p = 0.7$ has probability 0.0022". Only ratios of likelihoods carry meaning. Turning likelihood into a distribution over θ requires a prior, which is the Bayesian step in Section 14.5.

The **maximum likelihood estimator** (MLE) is the parameter that makes the data most probable:

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} L(\theta).$$

14.2.2 Logs make it easy

For independent observations the likelihood is a product, $L(\theta) = \prod_i p(x_i \mid \theta)$. Products of many small numbers underflow on a computer and are awkward to differentiate. Take logs: the **log-likelihood** $\ell(\theta) = \log L(\theta) = \sum_i \log p(x_i \mid \theta)$ turns the product into a sum. The logarithm is increasing, so it peaks at the same θ.

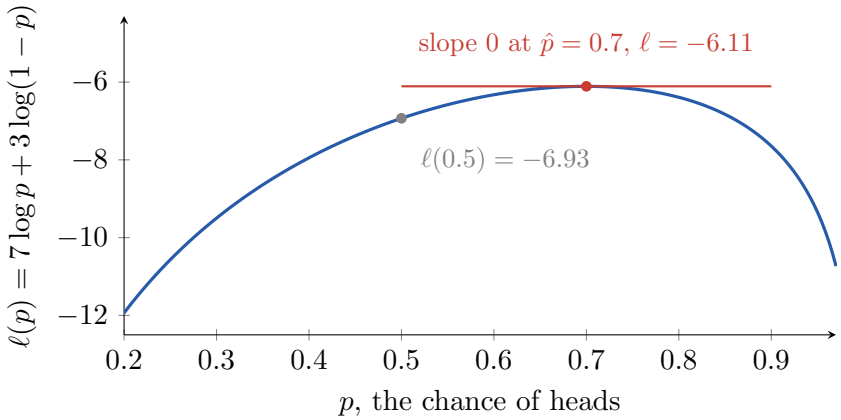


Figure 14.6: The log of the curve in Figure 14.5. It peaks at the same $p = 0.7$, but is a smooth hill that is easy to differentiate: the MLE is where the tangent (red line) is flat.

14.2.3 MLE for a Bernoulli proportion

With k successes in n trials, $\ell(p) = k \log p + (n-k) \log(1-p)$. Set the derivative to zero:

$$\ell'(p) = \frac{k}{p} - \frac{n-k}{1-p} = 0 \implies k(1-p) = (n-k)p \implies \hat{p} = \frac{k}{n}.$$

The MLE is the observed proportion, 7/10. Here the method only confirms common sense; the next two examples show it producing answers that are less obvious.

14.2.4 MLE for a Normal: why it divides by n

Take $x_1, \dots, x_n \overset{\text{iid}}{\sim} \mathcal{N}(\mu, \sigma^2)$. Each point contributes $\log p(x_i) = -\frac{1}{2}\log(2\pi\sigma^2) - (x_i-\mu)^2/(2\sigma^2)$, so

$$\ell(\mu, \sigma^2) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i-\mu)^2.$$

Maximising over μ only involves the sum of squares, which is smallest at $\hat{\mu} = \bar{x}$. Plug that in and differentiate in $v = \sigma^2$:

$$\frac{\partial \ell}{\partial v} = -\frac{n}{2v} + \frac{SS}{2v^2} = 0 \implies \hat{\sigma}^2_{\text{MLE}} = \frac{1}{n}\sum_{i=1}^{n}(x_i-\bar{x})^2.$$

For the latencies, $\hat{\mu} = 6$ and $\hat{\sigma}^2 = 16/5 = 3.2$, not $s^2 = 4$.

Why it works

The MLE divides by n because it plugs in $\bar{x}$ as if it were the true μ. But $\bar{x}$ was chosen to be as close as possible to these particular points, so deviations from $\bar{x}$ are systematically smaller than deviations from μ. Chapter 2 fixed this by dividing by $n-1$. The MLE does not correct, so it is biased low by the factor $(n-1)/n$. As Figure 14.3 showed, that bias is not a disaster: the MLE has lower MSE than s^2. For large n the two agree.

14.2.5 MLE for a Poisson rate

A service logged these error counts in six consecutive hours: $2, 0, 3, 1, 4, 2$. A Poisson model with rate λ (the standard model for counts of rare, independent events) gives $p(k \mid \lambda) = e^{-\lambda}\lambda^k/k!$. Then

$$\ell(\lambda) = \sum_i \left(k_i \log\lambda - \lambda - \log k_i!\right) = 12\log\lambda - 6\lambda - \text{const},$$

and $\ell'(\lambda) = 12/\lambda - 6 = 0$ gives $\hat{\lambda} = 2 = \bar{x}$.

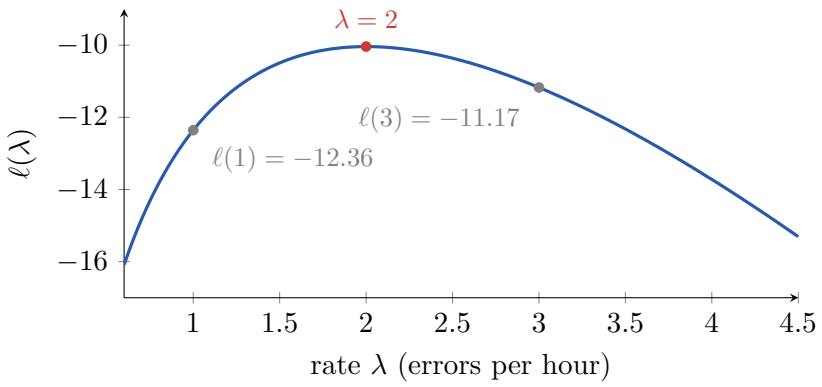


Figure 14.7: Poisson log-likelihood for the counts $2, 0, 3, 1, 4, 2$. It peaks at the mean count. The hill is asymmetric: rates below the peak lose plausibility faster than rates above it.

For the Uniform runtime example, the likelihood is θ^{-n} when θ is at least the largest observation and 0 otherwise, so the MLE is the maximum, 5.2. It can never propose a θ the data have ruled out. (It is biased low; multiplying by $(n+1)/n$ fixes that, giving 6.24.)

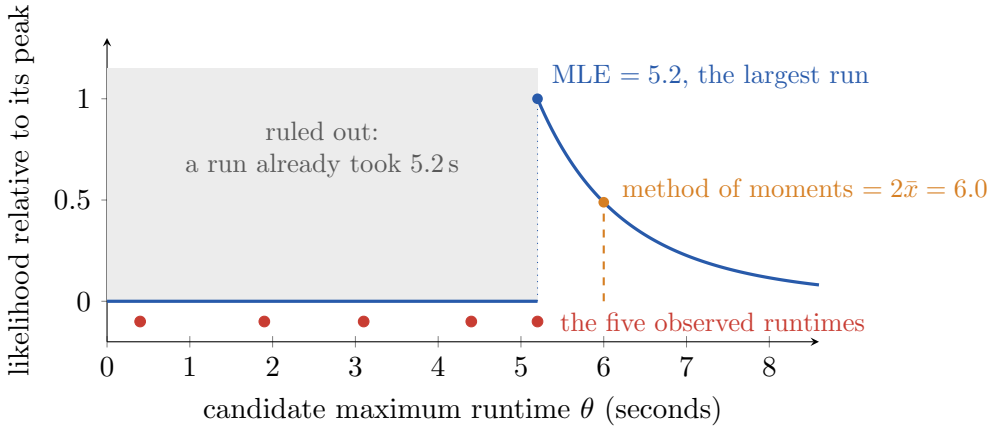


Figure 14.8: Likelihood θ^{-5} for the five Uniform runtimes. Any θ below the largest run (5.2 s) has likelihood zero, because it could not have produced that run, so the peak sits right at 5.2. Method of moments looks only at $\bar{x}$ and ignores this hard edge, which is how it can propose an impossible value.

```python
import numpy as np
from scipy import stats, optimize
x = np.array([4, 7, 5, 9, 5])
print(stats.norm.fit(x))        # (6.0, 1.789): sigma_hat^2 = 3.2, divides by n
k = np.array([2, 0, 3, 1, 4, 2])
nll = lambda lam: -stats.poisson.logpmf(k, lam).sum()
print(optimize.minimize_scalar(nll, bounds=(0.01, 10), method="bounded").x)  # 2.0
```

14.3 Every loss function is a likelihood

14.3.1 Least squares is Gaussian MLE

Regression assumes $y_i = f(x_i) + \varepsilon_i$ with noise $\varepsilon_i \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2)$. The likelihood of the observed y's is the Normal density evaluated at each residual, so the negative log-likelihood is

$$-\ell = \frac{n}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\sum_i \big(y_i - f(x_i)\big)^2.$$

For any fixed σ, the only part that depends on f is the sum of squared residuals. Maximising the likelihood is *exactly* minimising squared error. The least-squares line of Chapter 12 is the MLE under Gaussian noise, and $\hat{\sigma}^2 = \text{SSE}/n$ falls out as a bonus.

14.3.2 Cross-entropy is Bernoulli MLE

A classifier outputs $\hat{p}_i = \mathbb{P}(y_i = 1 \mid x_i)$. Each label is a coin flip with that probability, so $p(y_i) = \hat{p}_i^{\,y_i}(1-\hat{p}_i)^{1-y_i}$ and

$$-\log p(y_i) = -\big[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\big].$$

That is the binary cross-entropy loss from Chapter 13. A confident correct prediction ($\hat{p} = 0.9$, $y = 1$) costs $-\log 0.9 = 0.105$. A confident wrong one ($\hat{p} = 0.2$, $y = 1$) costs $-\log 0.2 = 1.609$. For K classes the same reasoning with a categorical distribution gives $-\log \hat{p}_{\text{true class}}$: softmax cross-entropy.

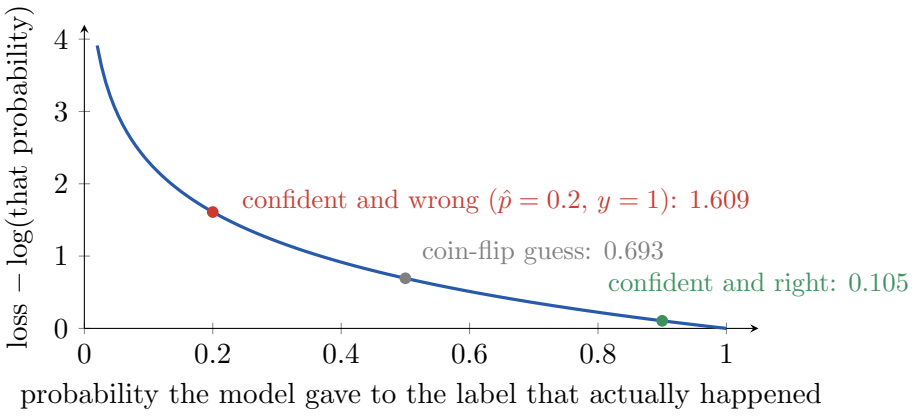


Figure 14.9: Cross-entropy is minus the log of the probability given to what really happened. Near 1 it costs almost nothing; as that probability heads to 0 the cost climbs without limit. That is why a confidently wrong classifier is punished so hard.

14.3.3 Absolute error is Laplace MLE

The **Laplace distribution** has density $\frac{1}{2b}e^{-|x-\mu|/b}$: a two-sided exponential, a sharp peak with heavier tails than the Normal. Its negative log density is $|x-\mu|/b + \log 2b$, so maximum likelihood minimises the sum of absolute errors.

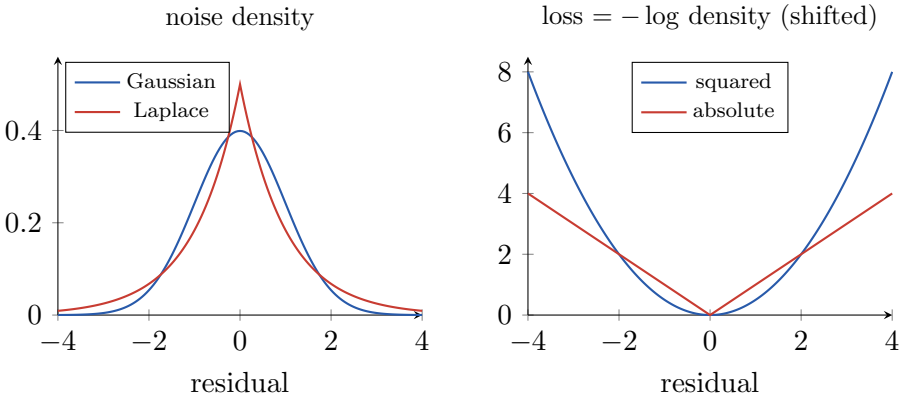


Figure 14.10: Each noise model on the left produces the loss on the right. Gaussian tails die fast, so a large residual is "impossible" and the squared loss punishes it heavily. Laplace tails are fatter, so big residuals cost only linearly.

Minimising $\sum(x_i - c)^2$ over a constant c gives the mean; minimising $\sum|x_i - c|$ gives the median (Chapter 2). Five response times $1, 2, 3, 4, 20$ ms make the difference concrete: the mean is 6, the median is 3.

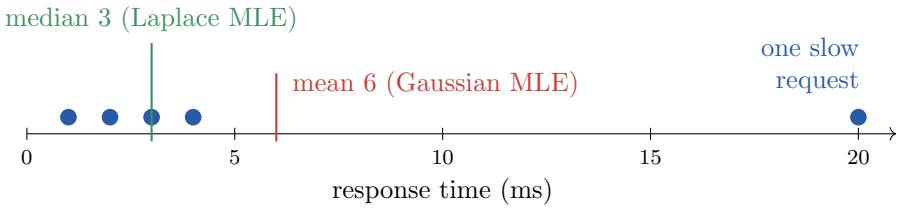


Figure 14.11: Five response times, four fast and one slow. The slow request drags the Gaussian MLE (mean) to 6, above four of the five points; the Laplace MLE (median) stays with the bulk. Picking a loss is picking a belief about how often big errors happen.

14.3.4 The dictionary

loss	noise model	best constant	used in
squared error	Gaussian	mean	linear regression
absolute error	Laplace	median	robust regression
pinball loss at level τ	asymmetric Laplace	τ-quantile	p99 forecasting
binary cross-entropy	Bernoulli	proportion	logistic regression
categorical cross-entropy	categorical	class frequencies	softmax classifiers
$\lambda - y \log \lambda$	Poisson	mean count	count regression

The **pinball loss** (Chapter 3) is a lopsided absolute error. For a residual $r = y - c$ it charges τr when $r \geq 0$ (you predicted too low) and $(1-\tau)|r|$ when $r < 0$ (too high). At $\tau = 0.99$, predicting too low costs 99 times more per millisecond than predicting too high, so the best constant c is pushed up until only 1% of the data lie above it: the 99th percentile. At $\tau = 0.5$ it is half the absolute error, and the best constant is the median.

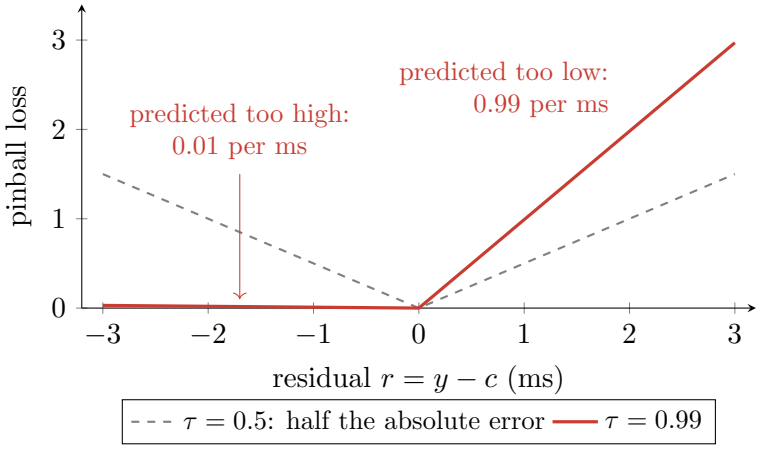


Figure 14.12: The pinball loss is a V with unequal arms. At $\tau = 0.99$ (red) undershooting costs 99 times more than overshooting, so the best constant is pushed up until only 1% of the data lie above it. At $\tau = 0.5$ (dashed) the arms are equal and the best constant is the median.

Connection

In ML papers the training objective is often written $\min_\theta \mathbb{E}_{(x,y)\sim\text{data}}[-\log p_\theta(y \mid x)]$. Read it as "average negative log-likelihood over the training set". Every row of the table is this formula with a different noise model plugged in. When a paper says "we model the output as a Gaussian with learned variance", it means the loss has an extra $\log\sigma$ term and the squared error is divided by σ^2. Chapter 15 returns to this.

Pause

Pause and think: you want a model that predicts p99 latency, not mean latency. Should you train it with squared error? (No. Squared error targets the mean. Use the pinball loss at $\tau = 0.99$, whose best constant is the 99th percentile.)

14.4 How sure is the MLE?

14.4.1 Curvature is information

Look again at the log-likelihood of Figure 14.6. If the hill is sharp, moving p a little away from the peak costs a lot of log-likelihood, so only a narrow range of p is plausible. If the hill is flat, many values fit almost equally well. Sharpness at the top is measured by the second derivative.

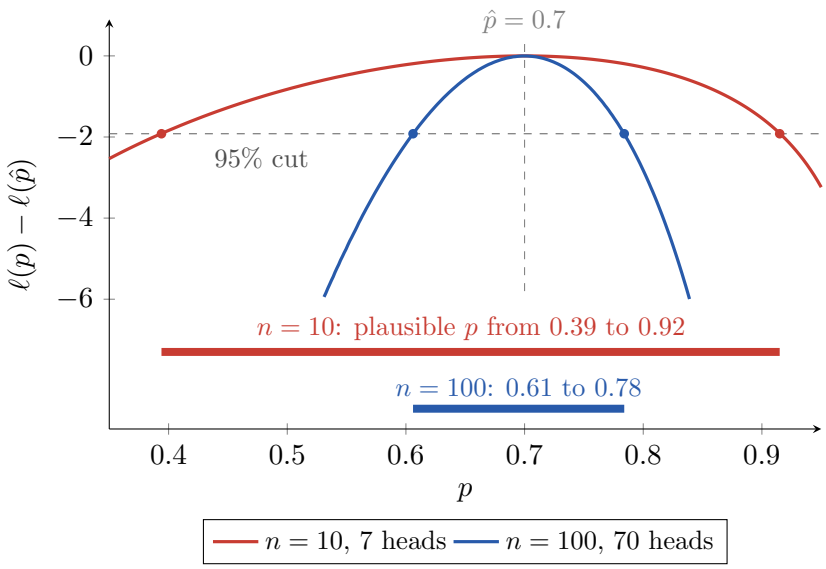


Figure 14.13: Same MLE, different evidence. With ten times the data the log-likelihood is ten times as curved, so it drops below the 95% cut (dashed) much sooner: the bars underneath show the plausible range shrinking about $\sqrt{10} \approx 3$ times.

Definition 14.4. The **observed information** is $J(\hat\theta) = -\ell''(\hat\theta)$, the curvature of the log-likelihood at its peak. Its expected value over datasets, $I(\theta) = -\mathbb{E}[\ell''(\theta)]$, is the **Fisher information**. Both measure how sharply the data pin down θ.

Theorem 14.2 (Large-sample behaviour of the MLE)**.** *Under regularity conditions (smooth model,*

θ not at the edge of its range), for large n

$$\hat{\theta}_{MLE} \overset{approx}{\sim} \mathcal{N}\Big(\theta,\ \frac{1}{I(\theta)}\Big), \qquad \mathrm{SE}(\hat{\theta}) \approx \frac{1}{\sqrt{I(\hat{\theta})}}.$$

Why it works

Near its peak, any smooth hill looks like a parabola: $\ell(\theta) \approx \ell(\hat{\theta}) - \frac{1}{2} J\,(\theta - \hat{\theta})^2$. That is the log of a Normal density in θ with variance $1/J$. Curvature and variance are reciprocals. The full proof (a Taylor expansion of ℓ' plus the CLT) is in Probability From the Ground Up, Ch. 30.

Example 14.2. For the coin, $\ell''(p) = -k/p^2 - (n-k)/(1-p)^2$. At $\hat{p} = k/n$ this simplifies to $-n/(\hat{p}(1-\hat{p}))$. With 7 heads in 10: $J = 10/0.21 = 47.6$ and

$$\mathrm{SE}(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}} = \sqrt{0.021} = 0.145.$$

This is the familiar standard error of a proportion from Chapter 7, now derived from curvature. With 70 heads in 100 it is 0.046. For a Normal mean, $I = n/\sigma^2$ gives $\mathrm{SE} = \sigma/\sqrt{n}$. For the Poisson rate, $I = n/\lambda$ gives $\mathrm{SE}(\hat{\lambda}) = \sqrt{2/6} = 0.58$ errors per hour.

What is really happening

This is how software reports standard errors for logistic regression and most fitted models. It finds the peak of ℓ, computes the matrix of second derivatives there (the Hessian), inverts it, and takes square roots of the diagonal. Every "std err" column in a regression table is a curvature reading.

14.4.2 Likelihood ratios

To compare two parameter values, compare how well they predict the data. The **likelihood-ratio statistic** for testing $H_0\colon \theta = \theta_0$ is

$$\Lambda = 2\big[\ell(\hat{\theta}) - \ell(\theta_0)\big],$$

twice the drop in log-likelihood from the peak down to θ_0. When H_0 is true and n is large, Λ approximately follows a χ^2_1 distribution (Chapter 10), whose 95th percentile is 3.84.

For the coin and $H_0\colon p = 0.5$: $\Lambda = 2(-6.109 + 6.931) = 1.65$, below 3.84, with p-value 0.20. Seven heads in ten is not strong evidence against a fair coin. Inverting the test gives a **likelihood interval**: all p whose log-likelihood is within $3.84/2 = 1.92$ of the peak.

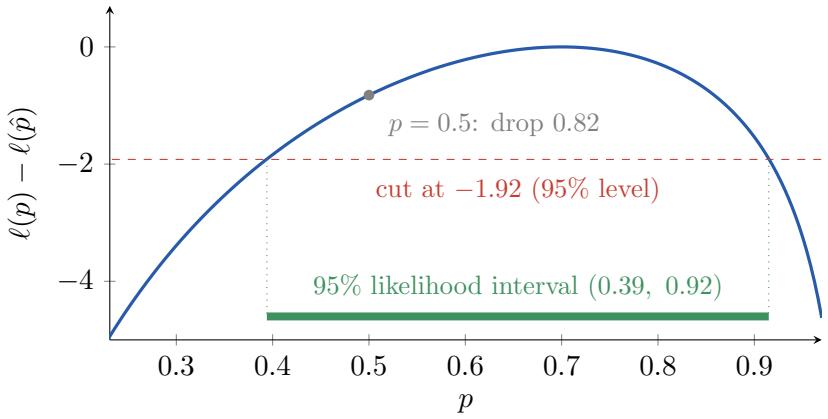


Figure 14.14: The likelihood-ratio test of $p = 0.5$ asks how far ℓ falls from its peak. It falls only 0.82 (so $\Lambda = 1.65$), well inside the cut. The set of p above the cut is a 95% interval; unlike $\hat{p} \pm 1.96$ SE it is lopsided, following the shape of the hill.

The Wald interval $0.7 \pm 1.96(0.145) = (0.42,\ 0.98)$ forces symmetry. With small n and p near an edge, the likelihood interval is the more trustworthy of the two. The "deviance" printed by logistic-regression software is the same kind of statistic: twice a log-likelihood gap, here between a model and a smaller model nested inside it.

History

In 1922 the British statistician and geneticist Ronald Fisher published "On the mathematical foundations of theoretical statistics". He was stuck on a basic question: given data and a model with unknown constants, which recipe gives the best estimate? The competing methods of his day, such as the method of moments, gave different answers with no principled way to choose. Fisher proposed choosing the value that maximises what he named the *likelihood*, and in the same paper defined consistency (closing in on the truth with more data) and efficiency (having the smallest possible variance). His later notion of *information*, the curvature above, told scientists how many observations an experiment needed. Nearly every model fitted today, from logistic regression to a neural network, is trained by Fisher's recipe.

14.5 The Bayesian view

14.5.1 From likelihood to a distribution over θ

Likelihood ranks parameter values but cannot say "there is a 90% chance p exceeds 0.5". Bayes' rule can, if we are willing to state what we believed before the data.

Definition 14.5. The **prior** $\pi(\theta)$ is a probability distribution over the parameter that encodes belief before seeing the data. The **posterior** $\pi(\theta \mid \text{data})$ is the belief afterwards. Bayes' rule links them:

$$\pi(\theta \mid \text{data}) = \frac{L(\theta)\,\pi(\theta)}{\int L(t)\,\pi(t)\,\mathrm{d}t} \ \propto\ \text{likelihood} \times \text{prior}.$$

The denominator, the **evidence**, is only a normalising constant that makes the posterior integrate to 1.

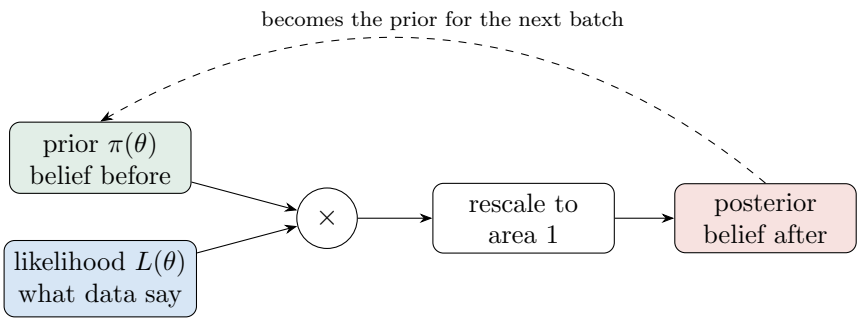


Figure 14.15: The Bayesian update. Multiply belief by evidence, rescale, and the result can serve as the prior when more data arrive.

14.5.2 Beta–Binomial: the worked case

For a proportion p the convenient prior is the **Beta distribution** $\text{Beta}(a, b)$, with density proportional to $p^{a-1}(1-p)^{b-1}$ on $[0, 1]$. Think of a and b as **pseudo-counts**: imaginary heads and tails you have "seen" before any real data. Its mean is $a/(a+b)$, the pseudo-head fraction. $\text{Beta}(1, 1)$ is flat; $\text{Beta}(2, 2)$ is a gentle hump at 0.5; $\text{Beta}(20, 20)$ is a firm belief that p is near 0.5.

Multiply the prior by the likelihood $p^k(1-p)^{n-k}$:

$$p^{a-1}(1-p)^{b-1} \cdot p^k(1-p)^{n-k} = p^{a+k-1}(1-p)^{b+n-k-1}.$$

That is another Beta. The update is just counting:

$$\text{Beta}(a, b) \xrightarrow{k \text{ heads, } n-k \text{ tails}} \text{Beta}(a+k,\ b+n-k).$$

A prior that stays in the same family after updating is called **conjugate**.

Example 14.3. Prior $\text{Beta}(2, 2)$, data 7 heads in 10. The posterior is $\text{Beta}(9, 5)$, with

- posterior mean $9/14 = 0.643$, pulled from 0.7 toward the prior's 0.5;
- posterior mode $(9-1)/(9+5-2) = 8/12 = 0.667$;
- posterior SD 0.124, close to the frequentist SE of 0.145;
- 95% **credible interval** $(0.386,\ 0.861)$: the central 95% of the posterior.

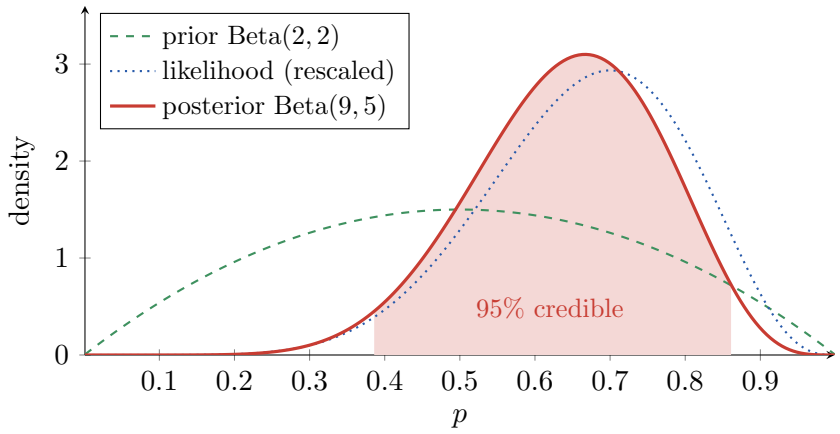


Figure 14.16: The posterior sits between the prior and the likelihood, closer to the likelihood because ten flips outweigh the prior's two pseudo-flips each way. The shaded region holds 95% of posterior probability.

14.5.3 How much does the prior matter?

The posterior mean $(a+k)/(a+b+n)$ adds the pseudo-counts to the real counts as if they were real flips. So a prior is worth $a+b$ observations: Beta$(2,2)$ is worth 4 flips, Beta$(20,20)$ is worth 40.

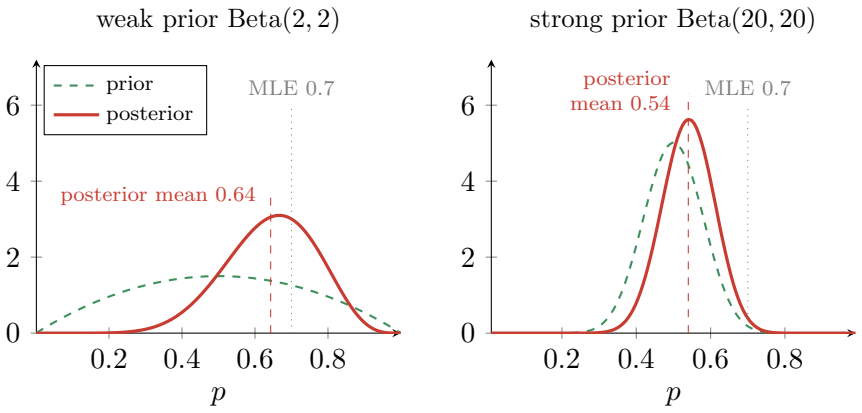


Figure 14.17: Same data (7 of 10), two priors (green dashed). Compare where the posterior mean (red dashed line) lands relative to the MLE. The weak prior lets the data speak: posterior mean 0.64, close to 0.7. The strong prior, worth 40 flips, overrules ten flips: posterior Beta$(27,23)$ with mean 0.54.

As data accumulate, any reasonable prior gets swamped. Keep the observed rate at 70% and grow n:

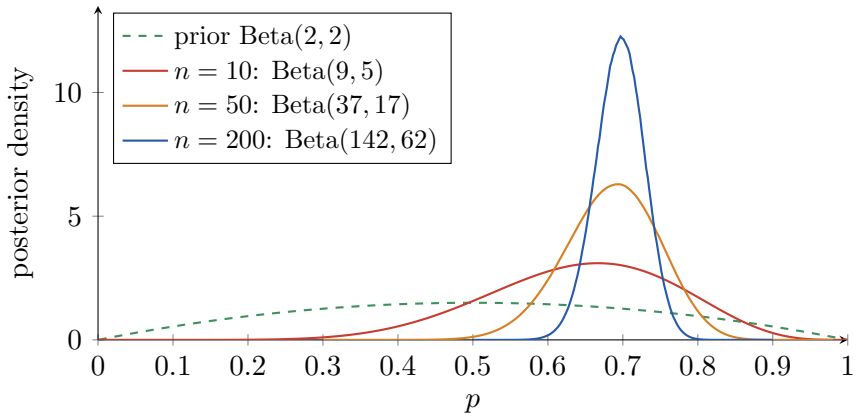


Figure 14.18: Sequential updating with 70% heads throughout. The posterior narrows (SD 0.124, 0.063, 0.032) and its mean approaches 0.7 (0.643, 0.685, 0.696). With enough data, Bayesian and maximum-likelihood answers coincide.

Pause

Pause and think: you start from Beta(9, 5) and flip ten more times, getting 6 heads. What is the new posterior? (Beta(15, 9). Updating in two batches gives the same answer as updating once with 13 heads in 20 from Beta(2, 2).)

14.5.4 Credible intervals versus confidence intervals

The two kinds of interval answer different questions.

- A 95% **confidence interval** (Chapter 7) is produced by a recipe that, over many repeated studies, captures the fixed true p 95% of the time. The randomness is in the interval. For one realised interval, "95% probability p is inside" is not a statement the method makes.
- A 95% **credible interval** contains 95% of the posterior probability. Given the prior and the model, "p is in here with probability 0.95" is exactly what it means. The randomness is in your knowledge of p.

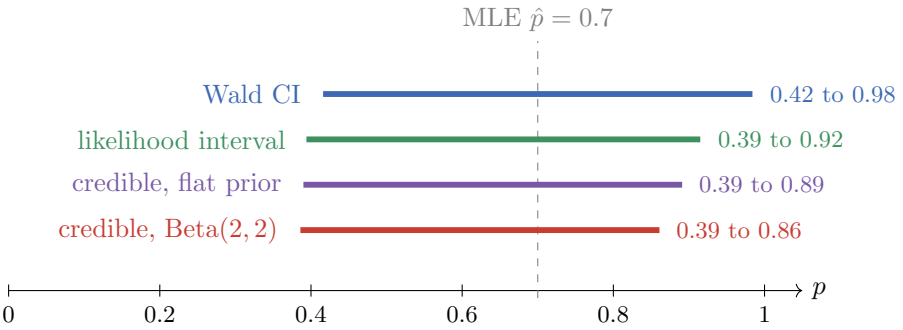


Figure 14.19: Four 95% intervals for p after 7 of 10 heads. With this little data the method matters: the symmetric Wald interval stretches almost to 1, while the likelihood and Bayesian intervals reach further below 0.7 than above it, following the lopsided shape of the likelihood.

With a flat prior $\mathrm{Beta}(1,1)$ the posterior is $\mathrm{Beta}(8,4)$, interval $(0.390,\ 0.891)$. For large n and a weak prior, credible and confidence intervals nearly coincide numerically; the difference is mostly in how you are allowed to read them.

```
from scipy import stats
a, b, k, n = 2, 2, 7, 10
post = stats.beta(a + k, b + n - k)          # Beta(9, 5)
print(post.mean(), post.interval(0.95))       # 0.643, (0.386, 0.861)
print(1 - post.cdf(0.5))                      # P(p > 0.5 | data) = 0.867
```

The last line is something only the Bayesian view can give: a direct probability that the coin favours heads.

14.6 MAP estimation is regularisation

14.6.1 The posterior mode as a penalised MLE

Often we want one number rather than a whole posterior. The **maximum a posteriori** (MAP) estimate is the posterior's peak. Take logs of Bayes' rule; the evidence is a constant and drops out:

$$\hat{\theta}_{\mathrm{MAP}} = \arg\max_{\theta} \big[\, \ell(\theta) + \log \pi(\theta) \,\big] = \arg\min_{\theta} \big[\, \underbrace{-\ell(\theta)}_{\text{loss}} + \underbrace{(-\log \pi(\theta))}_{\text{penalty}} \,\big].$$

MAP is the MLE plus a penalty, and the penalty is the negative log prior. A flat prior adds nothing, so MAP equals MLE. For the coin with $\mathrm{Beta}(2,2)$, the MAP is 0.667, the posterior mode found above.

14.6.2 Gaussian prior gives ridge; Laplace prior gives lasso

Put a Normal prior $\mathcal{N}(0, \tau^2)$ on each regression weight β_j. Then $-\log \pi(\beta) = \sum_j \beta_j^2/(2\tau^2) + \text{const}$. With Gaussian noise of variance σ^2, multiply the whole objective by $2\sigma^2$:

$$\hat{\beta}_{\mathrm{MAP}} = \arg\min_{\beta} \Big[\sum_i (y_i - x_i^\top \beta)^2 + \lambda \sum_j \beta_j^2 \Big], \qquad \lambda = \frac{\sigma^2}{\tau^2}.$$

That is **ridge regression** (L2 regularisation, or "weight decay" in neural networks). A tight prior (small τ) means a big λ and strong shrinkage toward zero.

A Laplace prior with scale b gives $-\log \pi(\beta) = \sum_j |\beta_j|/b$ and so

$$\hat{\beta}_{\mathrm{MAP}} = \arg\min_{\beta} \Big[\sum_i (y_i - x_i^\top \beta)^2 + \lambda \sum_j |\beta_j| \Big], \qquad \lambda = \frac{2\sigma^2}{b}.$$

That is the **lasso** (L1 regularisation).

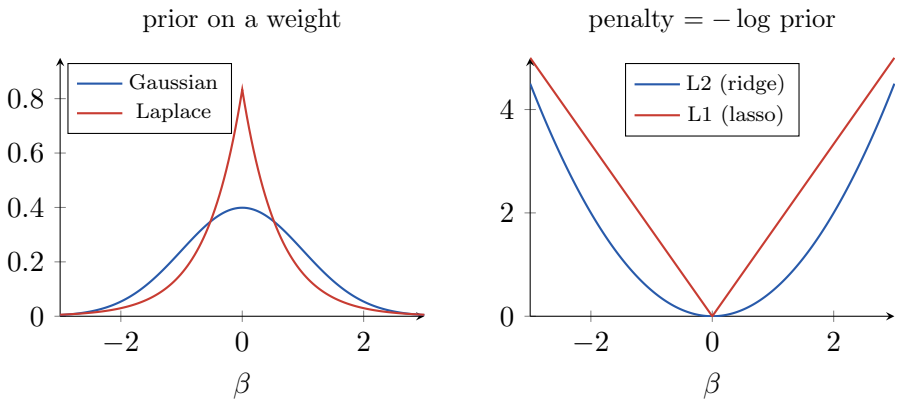


Figure 14.20: The Laplace prior has a sharp spike at zero; its penalty is a V with a corner there. The Gaussian prior is smooth at zero; its penalty is a bowl. The corner is what lets lasso set weights exactly to zero.

Example 14.4. Estimate a single mean with $n = 4$ observations, $\bar{x} = 6$, noise $\sigma = 2$, and prior $\mu \sim \mathcal{N}(0, 1)$. Then $\lambda = \sigma^2/\tau^2 = 4$ and the ridge objective $\sum(x_i - \mu)^2 + 4\mu^2$ is minimised at

$$\hat{\mu}_{\text{MAP}} = \frac{n\bar{x}}{n + \lambda} = \frac{24}{8} = 3.$$

The data say 6, the prior says 0, and they carry equal weight here (data precision $n/\sigma^2 = 1$ equals prior precision $1/\tau^2 = 1$), so the answer is halfway. With a Laplace prior of scale $b = 1$, the MAP is the "soft-threshold" of $\bar{x}$: move it toward zero by $\sigma^2/(nb) = 1$, giving 5, and stop at zero if you would cross it.

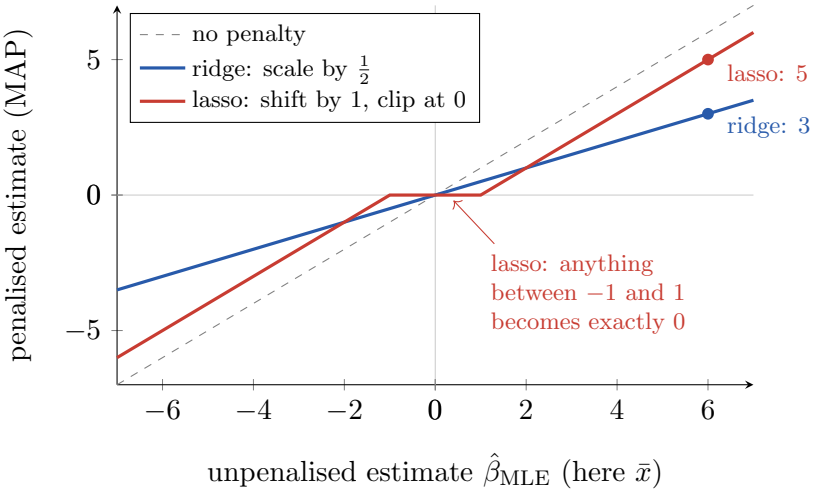


Figure 14.21: How each penalty transforms an estimate, using the numbers of the example above (one weight, or several whose predictors are uncorrelated): the dots show $\bar{x} = 6$ becoming 3 under ridge and 5 under lasso. Ridge shrinks everything proportionally and never reaches zero. Lasso subtracts a fixed amount and sets small estimates (between -1 and 1) exactly to zero, which performs variable selection.

Intuition

Chapter 13 mentioned ridge and lasso as penalties added to the loss to keep coefficients small. On its own that looks like a hack: add a term because it helps on test data. Through the Bayesian lens it is honest modelling. You believe most weights are small (Gaussian prior) or that most are exactly zero and a few matter (Laplace prior), and the penalty is that belief written as a loss. Returning to Figure 14.3: the prior adds bias toward zero and buys a larger cut in variance.

Common trap

MAP is a point estimate, not a full Bayesian answer. It throws away the posterior's width, and it depends on how you parametrise: the mode of a density changes if you switch from p to $\log p$, while posterior probabilities of events do not. If you need uncertainty, report a credible interval, not just the MAP.

14.7 When frequentist versus Bayesian matters in practice

With plenty of data and a weak prior, both schools return the same estimate and nearly the same interval, as Figure 14.18 showed. The choice matters at the edges.

situation	frequentist habit	Bayesian habit
large n, simple model	MLE ± 1.96 SE	essentially identical
small n or zero events (0 of 20 failures)	MLE $= 0$, Wald SE $= 0$: nonsense	prior keeps estimate above 0
many related groups (per-store rates)	estimate each separately	shrink toward shared mean (hierarchical prior)
A/B test readout	p-value, fixed sample size	$\mathbb{P}$(B beats A $\mid$ data)
regularised ML models	penalty chosen by cross-validation	penalty is a prior; same fit
regulated settings (drug trials)	required error-rate guarantees	used with care, prior must be justified

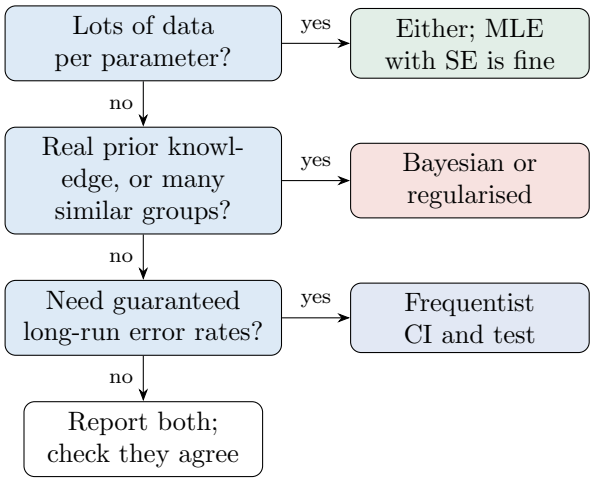


Figure 14.22: A rough guide. Most of the time the answers agree; the decision is about the cases where data are thin.

Example 14.5. A new checkout flow has 0 failures in 20 trials. The MLE failure rate is 0, with a Wald SE of 0, an interval that claims certainty. A flat Beta$(1,1)$ prior gives a Beta$(1,21)$ posterior with mean $1/22 = 0.045$ and 95% upper bound $1 - 0.05^{1/21} = 0.133$. The frequentist "rule of three" gives a similar upper bound, $3/20 = 0.15$. Both say the same thing: zero failures in twenty does not mean the rate is zero.

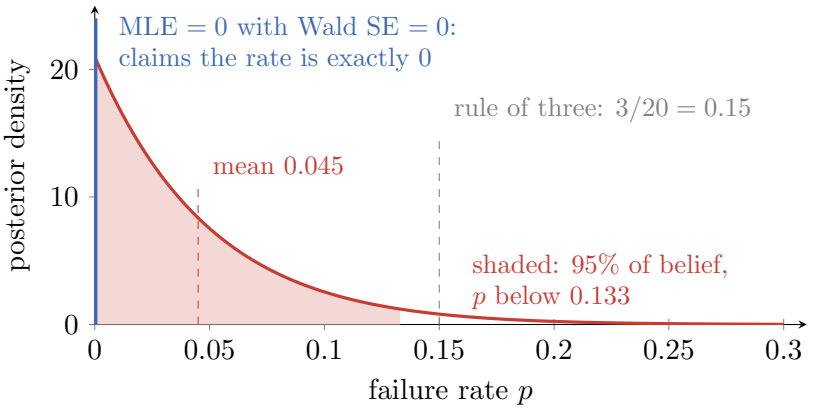


Figure 14.23: Zero failures in 20 trials. The MLE (blue, at 0) puts all its weight on a rate of exactly zero. The Beta$(1,21)$ posterior (red) is highest at zero but keeps real belief in rates up to about 0.13, close to the frequentist rule-of-three bound.

For depth, Probability From the Ground Up treats estimation theory in Ch. 30 and Bayesian inference in Ch. 31. Chapter 15 uses this chapter's dictionary to read the loss functions, regularisers and $\mathbb{E}_{x\sim p}[\cdot]$ notation of machine learning.

14.8 Summary

- An **estimator** is a recipe from data to a number; it has a sampling distribution. MSE $=$ Bias$^2 +$ Var, so a little bias can buy a big cut in variance.

- **Consistency**: the sampling distribution collapses onto θ as $n \to \infty$.
- The **likelihood** $L(\theta) = p(\text{data} \mid \theta)$ scores parameters by how well they predict the data. Work with $\ell = \log L$.
- MLEs: Bernoulli $\hat{p} = k/n$; Normal $\hat{\mu} = \bar{x}$, $\hat{\sigma}^2 = SS/n$ (biased, unlike s^2); Poisson $\hat{\lambda} = \bar{x}$.
- Losses are negative log-likelihoods: squared error $\leftrightarrow$ Gaussian, absolute error $\leftrightarrow$ Laplace, cross-entropy $\leftrightarrow$ Bernoulli or categorical.
- Curvature of ℓ is information: $\mathrm{SE}(\hat{\theta}) \approx 1/\sqrt{I(\hat{\theta})}$.
- Likelihood ratio $\Lambda = 2[\ell(\hat{\theta}) - \ell(\theta_0)] \approx \chi^2_1$; cut at 3.84.
- Bayes: posterior $\propto$ likelihood $\times$ prior. Beta–Binomial: $\mathrm{Beta}(a, b) \to \mathrm{Beta}(a + k,\, b + n - k)$.
- Credible intervals are probability statements about θ; confidence intervals are statements about the recipe.
- MAP = MLE + penalty $-\log \pi$. Gaussian prior = ridge (L2), Laplace prior = lasso (L1).

14.9 Exercises

1. An estimator has bias 0.5 and variance 2. What is its MSE? A second estimator is unbiased with variance 2.4. Which has lower MSE?
2. A feature was used in 3 of 12 sessions. Find the MLE of the usage rate and its approximate standard error from the Fisher information.
3. Daily incident counts over five days are $5, 3, 4, 6, 2$. Assuming a Poisson model, find $\hat{\lambda}$ and $\mathrm{SE}(\hat{\lambda})$.
4. For the data $2, 4, 6, 8$ compute the Normal MLE $\hat{\sigma}^2$ and the sample variance s^2. Explain the difference in one sentence.
5. For Uniform$(0, \theta)$ data, show that $\hat{\theta} = 2\bar{x}$ is unbiased with variance $\theta^2/(3n)$. (Use $\mathrm{Var}(X) = \theta^2/12$.)
6. With a flat Beta$(1, 1)$ prior and the data of Exercise 2, give the posterior, its mean and its mode. Which of these equals the MLE?
7. A coin shows 60 heads in 100 flips. Compute the likelihood-ratio statistic for $H_0\colon p = 0.5$ and decide at the 5% level.
8. Ten observations have $\bar{x} = 2$. Find the ridge estimate of the mean, minimising $\sum (x_i - \mu)^2 + \lambda \mu^2$ with $\lambda = 10$. What prior variance τ^2 does this correspond to if $\sigma^2 = 1$?
9. You are training a model to predict the 99th percentile of request latency. Which loss from the dictionary should you use, and why not squared error?
10. Two analysts use priors Beta$(1, 1)$ and Beta$(30, 30)$ for the same conversion rate. After 400 conversions in 1000 visits, compute both posterior means. Why are they close?

Answers

1. $0.5^2 + 2 = 2.25$. The unbiased one has MSE 2.4, so the biased estimator is better.
2. $\hat{p} = 0.25$; SE $= \sqrt{0.25 \cdot 0.75/12} = 0.125$.
3. $\hat{\lambda} = 20/5 = 4$; $I = n/\lambda$ so SE $= \sqrt{4/5} = 0.894$.
4. $\bar{x} = 5$, $SS = 20$; $\hat{\sigma}^2 = 20/4 = 5$, $s^2 = 20/3 = 6.67$. The MLE measures spread around $\bar{x}$, which hugs the data, and does not correct for it.
5. $\mathbb{E}[2\bar{x}] = 2 \cdot \theta/2 = \theta$; $\mathrm{Var}(2\bar{x}) = 4 \cdot \theta^2/(12n) = \theta^2/(3n)$.
6. Beta$(4, 10)$; mean $4/14 = 0.286$; mode $3/12 = 0.25$. The mode (MAP) equals the MLE, because a flat prior adds no penalty.
7. $\Lambda = 2[60 \log(0.6/0.5) + 40 \log(0.4/0.5)] = 4.03 > 3.84$; reject at 5% ($p \approx 0.045$).
8. $\hat{\mu} = 10 \cdot 2/(10 + 10) = 1$. $\lambda = \sigma^2/\tau^2$ gives $\tau^2 = 0.1$.
9. The pinball loss at $\tau = 0.99$; its minimiser is the 99th percentile. Squared error is minimised by the mean.
10. $401/1002 = 0.400$ and $430/1060 = 0.406$. A prior worth 60 pseudo-visits is small next to 1000 real ones.

Chapter 15

Statistics for Machine Learning

Chapter 14 turned data into estimates: maximise a likelihood, or combine it with a prior. Machine learning does the same thing at a larger scale, with its own notation and its own habits. Open a paper and you meet $\mathbb{E}_{x\sim p}[\ell(f_\theta(x), y)]$, $\hat{\theta}$, arg min, "validation AUC 0.91". Every one of those symbols is a statistical idea you already own. This chapter is the decoder ring, plus the handful of statistical tools that ML practice leans on daily: honest error estimates, the bias–variance tradeoff, regularisation, classifier evaluation, and error bars on the metrics themselves.

15.1 A notation decoder

Here is a sentence lifted from the style of a typical paper:

$$\hat{\theta} \;=\; \arg\min_{\theta} \; \frac{1}{n}\sum_{i=1}^{n} \ell\big(f_\theta(x_i),\, y_i\big), \qquad (x_i, y_i) \overset{\text{iid}}{\sim} p.$$

Read aloud: "theta-hat is the parameter value that makes the average loss on the n training examples as small as possible, where the examples were drawn independently from the same distribution p." Each piece deserves a picture.

- A **loss** $\ell(\hat{y}, y)$ is a number that says how bad a prediction $\hat{y}$ is when the truth is y. Squared loss $(\hat{y} - y)^2$ is the one from regression (Chapter 12).
- f_θ is a **model**: a formula with adjustable knobs θ (the **parameters**) that turns an input x into a prediction.
- The hat on $\hat{\theta}$ means "computed from data". It is an estimate, and it would come out differently on a different sample. Same convention as $\hat{p}$ and $\hat{\beta}$ in earlier chapters.
- $\arg\min_\theta g(\theta)$ is the *location* of the minimum, not the minimum value. If $g(\theta) = (\theta - 3)^2 + 5$ then $\min g = 5$ but $\arg\min g = 3$.

Table 15.1 collects the symbols you will meet most often.

Symbol	Read it as	What it means
$x \sim p$	"x drawn from p"	x is a random draw from the distribution p.
$(x_i, y_i) \overset{\text{iid}}{\sim} p$	"i.i.d. from p"	Independent draws, all from the same p. Each example tells you nothing about the next.
$\mathbb{E}_{x\sim p}[f(x)]$	"expected f under p"	The long-run average of $f(x)$ if you kept drawing x from p.
$\frac{1}{n}\sum_i f(x_i)$	"sample average"	The data's estimate of $\mathbb{E}_{x\sim p}[f(x)]$.
$\hat{p}$ or $\hat{p}_n$	"empirical distribution"	Put weight $1/n$ on each observed point. Averages under $\hat{p}$ are sample averages.
$p(y \mid x; \theta)$	"prob. of y given x, at θ"	A model's predicted distribution for the label. The semicolon separates data from parameters.
$\hat{\theta}$, $\hat{y}$	"theta-hat", "y-hat"	Quantities estimated or predicted from data.
$\theta^\star$	"theta-star"	The best possible parameter value (usually unknowable).
$\arg\min_\theta L(\theta)$	"the θ minimising L"	Where the minimum occurs, not its value.
$\nabla_\theta L$	"gradient of L"	Vector of slopes of L in each parameter direction.
Σ	"covariance matrix"	Variances on the diagonal, covariances off it (Section 15.9).

Table 15.1: Decoder for common ML notation. Nearly every entry is an expectation, a sample average, or an estimate of one.

15.1.1 True distribution versus empirical distribution

The distinction that matters most in the table is p versus $\hat{p}$. The **true distribution** p is the process that generates data: every email that will ever arrive, every future house sale. You never see it. The **empirical distribution** $\hat{p}_n$ is the data you actually hold, viewed as a distribution that puts probability $1/n$ on each point.

Take $n = 12$ draws from a standard Normal:

$$-1.95,\ -1.69,\ -1.41,\ -0.30,\ -0.08,\ -0.04,\ 0.06,\ 0.21,\ 0.39,\ 0.50,\ 0.59,\ 1.87.$$

The empirical CDF climbs by $1/12$ at each point. Figure 15.1 lays it over the true CDF. The staircase tracks the smooth curve, imperfectly. With more data the steps get finer and the fit tightens. This is the Glivenko–Cantelli theorem that justified the bootstrap in Chapter 8; Probability From the Ground Up has the proof.

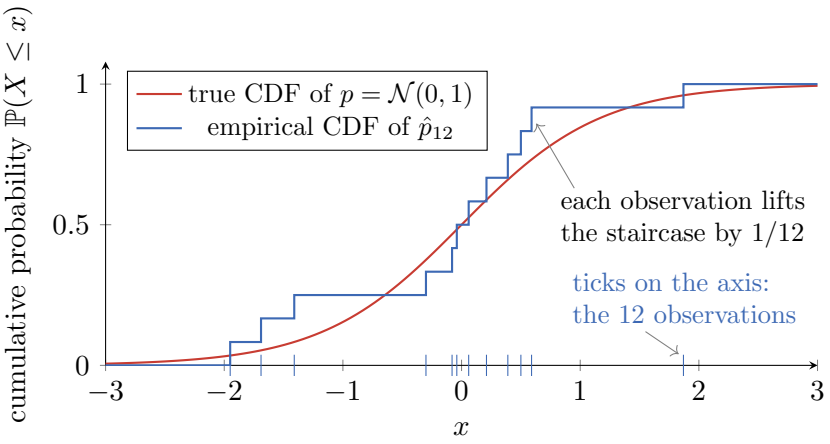


Figure 15.1: The empirical distribution $\hat{p}_{12}$ (blue staircase, one step of 1/12 at each observed value) is the data's stand-in for the unknown true distribution p (smooth red curve). The two agree roughly but not exactly. Training a model on data means optimising under $\hat{p}$ while hoping to do well under p.

Intuition

Any expectation under $\hat{p}_n$ is just a sample average: $\mathbb{E}_{x\sim\hat{p}_n}[f(x)] = \frac{1}{n}\sum_i f(x_i)$. So when a paper switches between $\mathbb{E}_{x\sim p}$ and $\mathbb{E}_{x\sim\hat{p}}$, it is switching between "the average over the whole world" and "the average over my dataset". The gap between the two is what the rest of this chapter is about.

15.2 Expected risk, empirical risk, and the generalisation gap

15.2.1 A tiny regression, by hand

Five houses: floor area x (hundreds of m^2) and price y (hundreds of thousands).

$$x = (1, 2, 3, 4, 5), \qquad y = (2.5,\ 3.5,\ 6.5,\ 8.0,\ 9.5).$$

Try the model $f_\theta(x) = \theta x$ with $\theta = 2$. Predictions are $2, 4, 6, 8, 10$; residuals $y - \hat{y}$ are 0.5, -0.5, 0.5, 0, -0.5; squared losses are 0.25, 0.25, 0.25, 0, 0.25. Their average is

$$\hat{R}(2) = \frac{0.25 + 0.25 + 0.25 + 0 + 0.25}{5} = 0.20.$$

That number is the **empirical risk**: the average loss on the data you have (Figure 15.2).

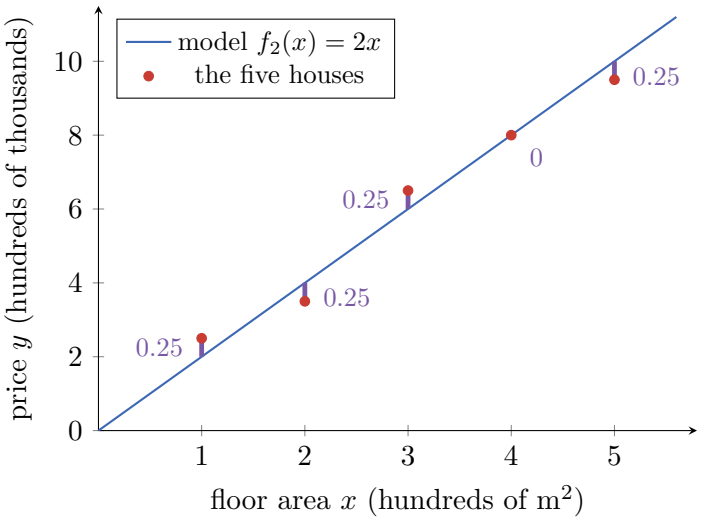


Figure 15.2: Empirical risk as a picture. Each purple segment is a residual $y - \hat{y}$ of size 0.5 (house 4 sits exactly on the line), and the number beside it is its squared loss $0.5^2 = 0.25$. The average of the five numbers, $1.0/5 = 0.20$, is $\hat{R}(2)$. A different θ tilts the line and changes all five numbers.

Definition 15.1 (Expected and empirical risk)**.** For a model f and loss ℓ, the **expected risk** (also called true risk or population risk) is the average loss on a fresh example from the true distribution:

$$R(f) = \mathbb{E}_{(x,y)\sim p}\left[\ell(f(x), y)\right].$$

The **empirical risk** is the same average over the n training examples:

$$\hat{R}_n(f) = \frac{1}{n}\sum_{i=1}^{n} \ell(f(x_i), y_i) = \mathbb{E}_{(x,y)\sim \hat{p}_n}\left[\ell(f(x), y)\right].$$

For any *fixed* model chosen before seeing the data, $\hat{R}_n(f)$ is a sample mean of i.i.d. losses. So it is unbiased for $R(f)$, and its standard error is $\mathrm{SD}(\ell)/\sqrt{n}$ (Chapter 6). The trouble starts when the model is *chosen using* the same data.

15.2.2 Empirical risk minimisation

Empirical risk minimisation (ERM) picks the model with the smallest training loss:

$$\hat{f} = \arg\min_{f \in \mathcal{F}} \hat{R}_n(f),$$

where $\mathcal{F}$ is the family of models you allow (all lines, all depth-5 trees, all networks of a given shape). For our houses and $f_\theta(x) = \theta x$, setting the derivative of $\sum (y_i - \theta x_i)^2$ to zero gives

$$\hat{\theta} = \frac{\sum x_i y_i}{\sum x_i^2} = \frac{108.5}{55} \approx 1.973, \qquad \hat{R}_5(\hat{\theta}) \approx 0.192.$$

ERM found a slightly better θ than our guess of 2, and its training loss dropped from 0.200 to 0.192. Least squares (Chapter 12) and maximum likelihood (Chapter 14) are both ERM: least squares with squared loss, maximum likelihood with loss $-\log p(y \mid x; \theta)$.

Why it works

Why is the training loss of $\hat{f}$ optimistic? Because $\hat{f}$ was picked to make $\hat{R}_n$ small. Some of the small value is real skill and some is luck: the chosen model happens to fit the particular noise in this sample. A fresh sample has different noise, so on average over samples $\mathbb{E}[\hat{R}_n(\hat{f})] \leq \mathbb{E}[R(\hat{f})]$. The same thing happened with s^2 in Chapter 2: measuring spread around $\bar{x}$, a point fitted to the data, underestimates spread around μ, and the $n-1$ corrects for it.

Definition 15.2 (Generalisation gap)**.** The **generalisation gap** of a fitted model is $R(\hat{f}) - \hat{R}_n(\hat{f})$: how much worse it does on new data than on the data it was trained on.

A flexible model can drive the training loss to zero by memorising. A polynomial of degree $n-1$ passes through all n points exactly. Its training loss is 0 and its gap is everything. Figure 15.3 shows the typical pattern as the model gets more flexible.

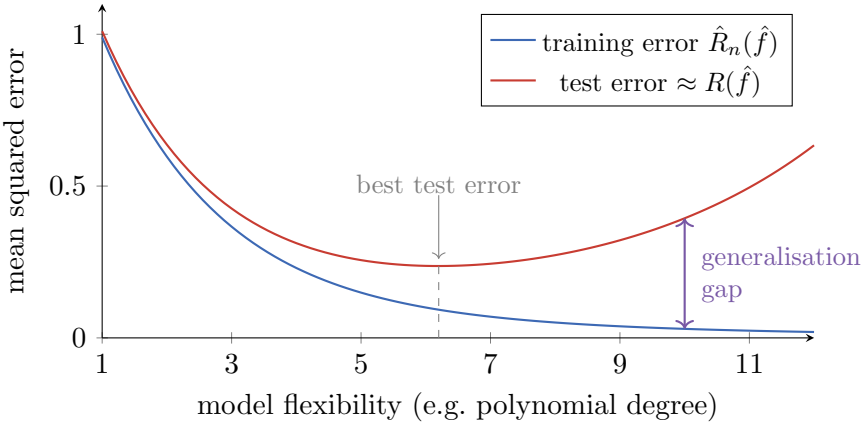


Figure 15.3: The typical shape (schematic curves). Training error keeps falling as flexibility grows; test error falls, bottoms out, then climbs. The vertical distance between them is the generalisation gap, and it widens as the model gets more flexible. Training error alone always says "more flexible is better".

Pause

Pause and think: a model scores 0.0 training error on 1000 examples. What, if anything, does that tell you about its test error? (Almost nothing. It tells you the model is flexible enough to memorise 1000 examples.)

15.3 Honest error estimates: splits and cross-validation

15.3.1 Train, validation, test

The fix for optimistic training error is the same idea as an experiment's control group: score the model on data it has never touched. Standard practice cuts the data into three parts (Figure 15.4).

- **Training set**: used to fit the parameters θ.

- **Validation set**: used to choose among models and **hyperparameters** (settings you pick rather than fit, such as polynomial degree or regularisation strength).
- **Test set**: touched once, at the end, to report performance.

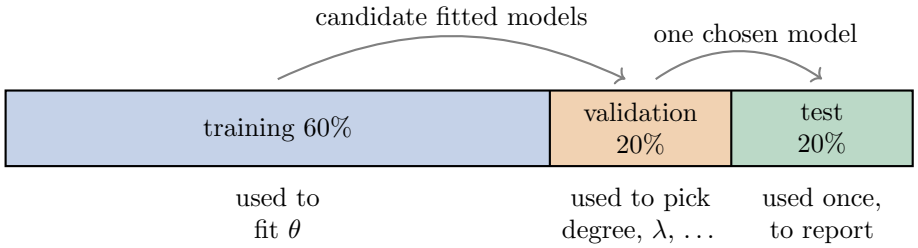


Figure 15.4: A three-way split. Each part answers one question. The test score is honest only if nothing about the model was chosen by looking at the test set.

Why does the validation set not suffice for reporting? Because you used it to choose. If you try 50 hyperparameter settings and report the best validation score, you have run ERM over those 50 settings on the validation set, and the winner's score is optimistic for the same reason training error is. This is the multiple-comparisons problem of Chapter 11 again.

15.3.2 k-fold cross-validation

With 500 examples, holding out 20% for validation wastes data and gives a noisy score. **k-fold cross-validation** reuses every example for both jobs. Cut the data into k equal **folds**. For each fold in turn, train on the other $k-1$ folds and score on the held-out fold. Average the k scores (Figure 15.5).

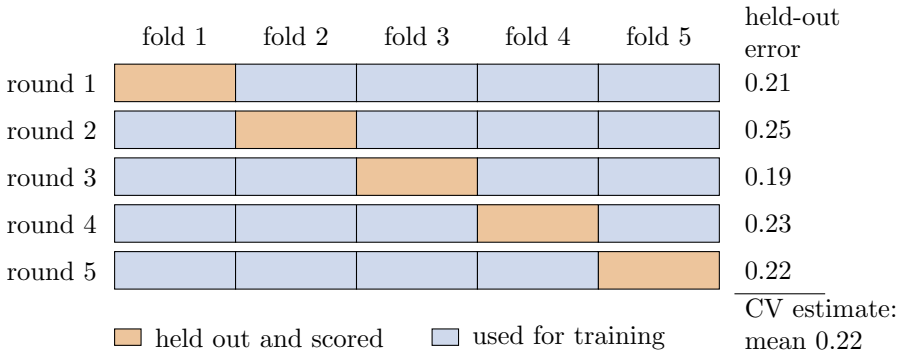


Figure 15.5: Five-fold cross-validation. In each round one fold (orange) is held out and scored while the model is trained on the other four (blue). Each example is held out exactly once, and the five held-out errors are averaged into one CV estimate.

With the five fold errors $0.21, 0.25, 0.19, 0.23, 0.22$, the CV estimate is their mean, 0.22. Their standard deviation is 0.022. The naive standard error $0.022/\sqrt{5} = 0.010$ is too small: the five training sets overlap by 75%, so the fold scores are positively correlated, and the averaging buys less

than independence would. Treat the fold-to-fold spread as a rough guide to stability, not as a valid confidence interval.

Common choices are $k = 5$ or $k = 10$. $k = n$ is **leave-one-out** CV: nearly unbiased but slow and high-variance. In all cases the final model is refit on all the data after CV has chosen its settings.

15.3.3 Data leakage

Data leakage is any route by which information from the held-out data reaches the model during training. It makes held-out scores optimistic, sometimes wildly so. Classic leaks:

- Standardising features (subtracting the mean, dividing by the SD) using *all* the data, then splitting. The training set has now seen the test set's mean.
- Selecting the "20 most predictive features" on all the data, then cross-validating a model on those features.
- Random splits of time-series data, so the model trains on Thursday and is tested on Wednesday.
- Duplicate or near-duplicate rows (the same patient twice, the same image resized) landing on both sides of the split.
- A feature that is a consequence of the label, such as "refund issued" when predicting fraud.

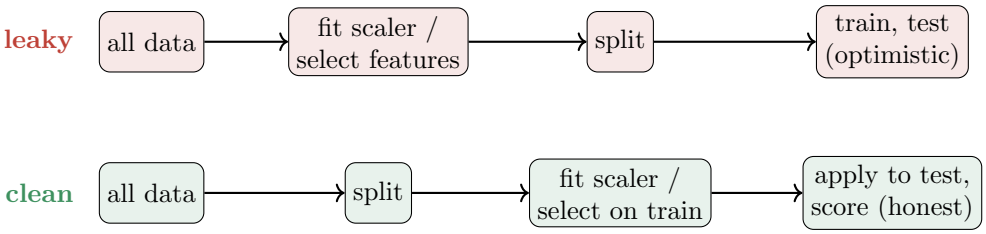


Figure 15.6: The order of operations decides whether the test score is honest. Anything that learns from data (scaling, imputation, feature selection) belongs inside the training side of the split, and inside each CV fold.

Common trap

Feature selection before cross-validation is the most damaging leak because it looks innocent. With 5000 pure-noise features and 50 examples, picking the 20 features most correlated with the label on all the data and then cross-validating can report accuracy far above chance on labels that are coin flips. The selection already saw every fold.

15.4 The bias–variance tradeoff for models

Chapter 14 split the error of an estimator into bias and variance: $\text{MSE} = \text{Bias}^2 + \text{Var}$. Bias is how far the estimator's average lands from the target; variance is how much it scatters from sample to sample. A fitted model is an estimator too. It estimates the true regression function $f(x) = \mathbb{E}[y \mid x]$, the average y at input x, and the same split applies at every x.

15.4.1 Many fits, one picture

Take the true curve $f(x) = \sin(2\pi x)$ on $[0, 1]$. At 15 evenly spaced inputs $x = \frac{1}{30}, \frac{3}{30}, \ldots, \frac{29}{30}$, draw noisy values $y = f(x) + \varepsilon$ with noise SD $\sigma = 0.3$, fit a model, and repeat on fresh datasets. Each dataset gives a different fitted curve. Figure 15.7 fits a straight line and a degree-9 polynomial to the same six datasets.

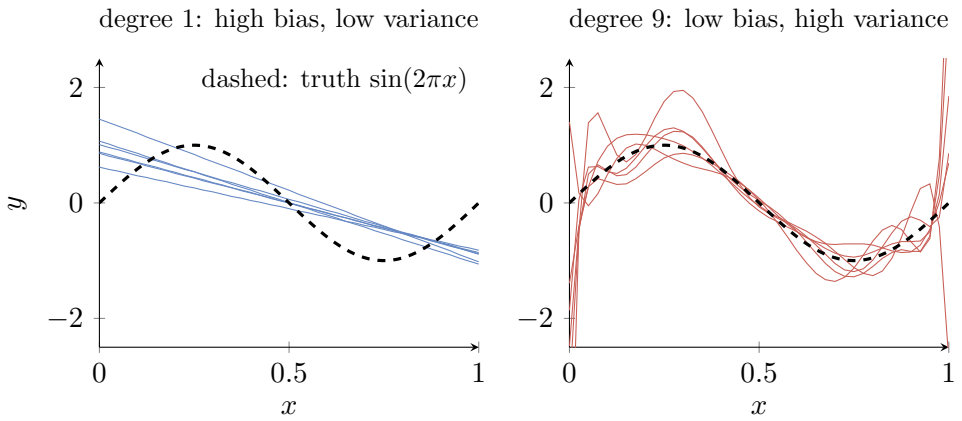


Figure 15.7: The same six datasets of 15 noisy points, fitted two ways; dashed black is the truth. Left: the lines agree with each other but all miss the wave (bias). Right: the polynomials follow the wave on average but disagree wildly with each other, especially near the edges, where some shoot off the plot (variance).

Repeat this on 20,000 simulated datasets. At each x, the average of the fitted curves gives the bias (its distance from the truth) and their spread gives the variance. Averaging both over $x \in [0, 1]$, with $\sigma^2 = 0.09$:

model	bias^2	variance	bias^2 + variance + σ^2
degree 1	0.200	0.012	0.302
degree 3	0.005	0.025	0.120
degree 9	0.000	0.209	0.299

Degree 3 wins. It has enough flexibility to bend once up and once down, and not enough to chase noise. Degrees 1 and 9 score about the same, for opposite reasons.

Theorem 15.1 (Bias–variance decomposition)**.** *Let $y = f(x) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\mathrm{Var}(\varepsilon) = \sigma^2$, and let $\hat{f}$ be fitted on a random training set independent of the new noise ε. At a fixed input x,*

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \underbrace{\sigma^2}_{\text{noise}} + \underbrace{\left(\mathbb{E}[\hat{f}(x)] - f(x)\right)^2}_{\text{bias}^2} + \underbrace{\mathrm{Var}\left(\hat{f}(x)\right)}_{\text{variance}},$$

where the expectation is over both the training set and the new noise.

Why it works

Write $y - \hat{f} = \varepsilon + (f - \mathbb{E}\,\hat{f}) + (\mathbb{E}\,\hat{f} - \hat{f})$. The three pieces are: fresh noise, a constant offset, and the fit's wobble around its own average. Square and take expectations. The cross terms vanish: ε is independent of the fit and has mean zero, and $\mathbb{E}\,\hat{f} - \hat{f}$ has mean zero by construction. What remains is the sum of the three squared pieces. No model can beat the noise term σ^2; it is the floor.

Plotted against flexibility, bias^2 falls, variance rises, and their sum with the noise floor traces the U of Figure 15.8.

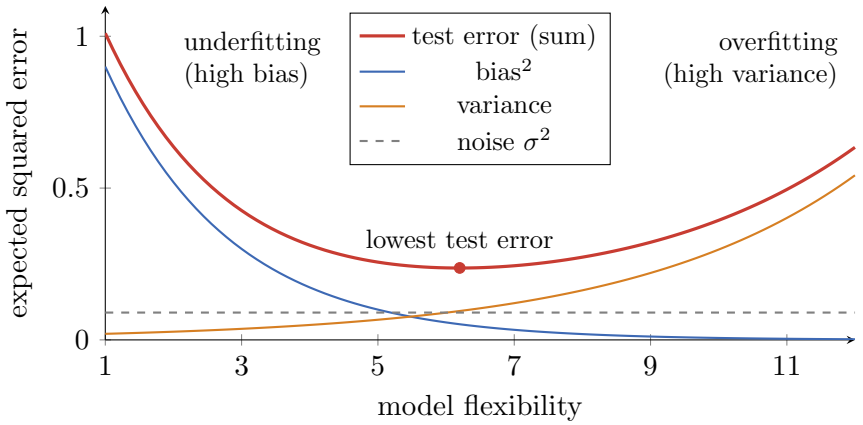


Figure 15.8: The U-shaped test error is the sum of a falling bias^2 curve, a rising variance curve, and a constant noise floor. Cross-validation is a way of locating the bottom of the U from data.

Connection

Very large neural networks sometimes show *double descent*: test error rises near the point where the model can just barely interpolate the data, then falls again as the model grows further. It does not contradict the decomposition above, which always holds. It says the variance curve need not keep rising forever once training finds the smoothest of the many interpolating solutions. Goodfellow et al. Ch. 5 discusses capacity in the classical picture.

15.5 Regularisation: ridge and lasso

If variance is the problem, add bias on purpose. **Regularisation** adds a penalty to the training loss that discourages large coefficients:

$$\text{ridge:}\quad \hat{\beta}^{\text{ridge}} = \arg\min_{\beta}\ \textstyle\sum_i (y_i - x_i^\top \beta)^2 + \lambda \sum_j \beta_j^2,$$

$$\text{lasso:}\quad \hat{\beta}^{\text{lasso}} = \arg\min_{\beta}\ \textstyle\sum_i (y_i - x_i^\top \beta)^2 + \lambda \sum_j |\beta_j|.$$

Here $\lambda \geq 0$ is a hyperparameter: the price per unit of coefficient size. $\lambda = 0$ gives ordinary least squares. Large λ pushes every coefficient toward 0. The features should be standardised first so that "size" means the same thing for each.

15.5.1 Shrinkage by hand

In the simplest case, where the features are uncorrelated and scaled to unit length (an orthonormal design), both penalties act on each least-squares coefficient $\hat{\beta}_j$ separately:

$$\hat{\beta}_j^{\text{ridge}} = \frac{\hat{\beta}_j}{1+\lambda}, \qquad \hat{\beta}_j^{\text{lasso}} = \operatorname{sign}(\hat{\beta}_j)\,\max\bigl(|\hat{\beta}_j| - \tfrac{\lambda}{2},\, 0\bigr).$$

Ridge shrinks every coefficient by the same factor. Lasso subtracts a fixed amount and stops at zero (**soft thresholding**). Take least-squares coefficients $(3.0,\ 1.0,\ -0.4)$ and $\lambda = 1$:

	β_1	β_2	β_3
least squares	3.0	1.0	-0.4
ridge, $\lambda = 1$ (divide by 2)	1.5	0.5	-0.2
lasso, $\lambda = 1$ (subtract 0.5, floor at 0)	2.5	0.5	0

Lasso removed the small coefficient entirely. That is **variable selection** for free. Figure 15.9 draws both rules as functions.

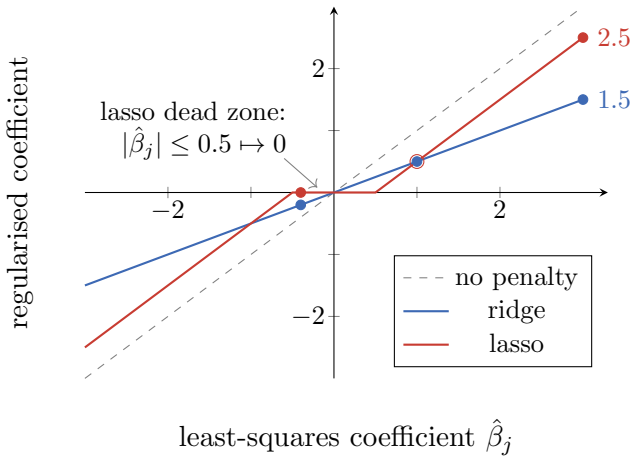
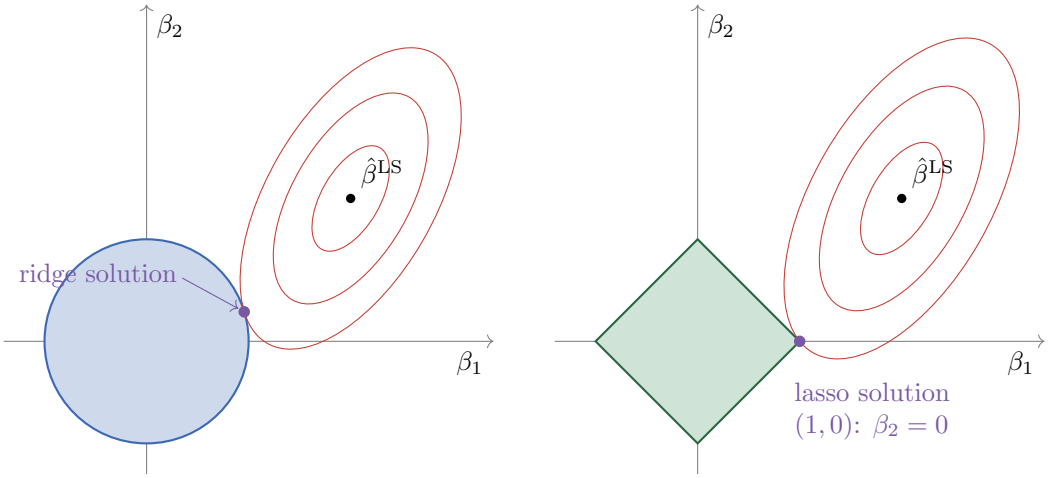


Figure 15.9: Shrinkage rules for $\lambda = 1$ with an orthonormal design; dots mark the worked example $\hat{\beta} = (3.0,\ 1.0,\ -0.4)$. Ridge (blue) halves every coefficient, so it never reaches zero. Lasso (red) moves every coefficient 0.5 toward zero and sets anything in $[-0.5, 0.5]$ exactly to zero, which is why -0.4 becomes 0.

15.5.2 Why lasso gives exact zeros: the geometry

Each penalised problem is equivalent to a constrained one: minimise the squared error subject to $\sum \beta_j^2 \le t$ (ridge) or $\sum |\beta_j| \le t$ (lasso), for some budget t that shrinks as λ grows. In two dimensions the ridge budget is a disc and the lasso budget is a diamond. The squared-error loss has elliptical contours centred on the least-squares solution. The answer is the first point where a growing ellipse touches the budget region.



ridge budget: disc $\beta_1^2 + \beta_2^2 \leq 1$
both coefficients shrink, neither is zero

lasso budget: diamond $|\beta_1| + |\beta_2| \leq 1$
the contour hits a corner on the β_1 axis

Figure 15.10: Constraint geometry. Red ellipses are contours of the squared error around the least-squares fit $\hat{\beta}^{\mathrm{LS}} = (2, 1.4)$. Left: the smallest contour touching the disc meets it at $(0.96, 0.29)$. Right: it meets the diamond at the corner $(1, 0)$. Corners sit on the axes, so lasso solutions often have exact zeros.

Intuition

A disc has no corners, so the touching point can be anywhere on its rim and will almost never land exactly on an axis. A diamond is all corners and flat edges, and in high dimensions the corners and low-dimensional edges stick out. Expanding ellipses tend to hit them first. Each corner is a point where some coefficients are zero.

Connection

Regularisation is a prior in disguise (Chapter 14). With Normal errors, ridge is the MAP estimate under independent Normal priors $\beta_j \sim \mathcal{N}(0, \tau^2)$, and λ is the ratio σ^2/τ^2. Lasso is the MAP estimate under Laplace priors, with density proportional to $e^{-|\beta_j|/b}$: two exponential tails glued back to back, with a sharp peak at zero. That peak plays the role of the diamond's corner. Weight decay in neural networks is ridge applied to every weight.

15.6 Evaluating classifiers

15.6.1 The confusion matrix

A spam filter scores 1000 emails. 100 are spam (the **positive** class) and 900 are not. It flags 110 emails: 80 spam and 30 legitimate. Every prediction falls in one of four cells (Figure 15.11).

	actually spam	actually not	
flagged	TP = 80 true positive	FP = 30 false positive	110 flagged
not flagged	FN = 20 false negative	TN = 870 true negative	890 passed
	100 spam	900 not spam	

Figure 15.11: Confusion matrix for the spam filter. Green cells are correct, red cells are errors. Every classification metric is a ratio of these four counts.

From the four counts:

$$\begin{aligned}
\text{accuracy} &= \frac{TP + TN}{\text{all}} = \frac{80 + 870}{1000} = 0.95, \\
\text{precision} &= \frac{TP}{TP + FP} = \frac{80}{110} \approx 0.727 \quad \text{(of the flagged, how many are spam)}, \\
\text{recall} = \text{TPR} &= \frac{TP}{TP + FN} = \frac{80}{100} = 0.80 \quad \text{(of the spam, how much was caught)}, \\
\text{FPR} &= \frac{FP}{FP + TN} = \frac{30}{900} \approx 0.033 \quad \text{(of the good mail, how much was flagged)}.
\end{aligned}$$

Recall is also called **sensitivity** or **true positive rate** (TPR). $1 - \text{FPR} = 0.967$ is the **specificity**. The **F1 score** combines precision P and recall R through their harmonic mean:

$$F_1 = \frac{2PR}{P + R} = \frac{2(0.727)(0.80)}{0.727 + 0.80} \approx 0.762.$$

The harmonic mean is dragged toward the smaller of the two, so F_1 is high only when both are.

Common trap

Accuracy 0.95 sounds good until you notice that a filter that flags nothing scores $900/1000 = 0.90$. When classes are imbalanced, always compare against the "predict the majority class" baseline, and prefer precision and recall.

15.6.2 Scores, thresholds, and the ROC curve

Most classifiers output a **score** (often a predicted probability) and flag an example when the score exceeds a **threshold**. Moving the threshold trades one error for the other (Figure 15.12).

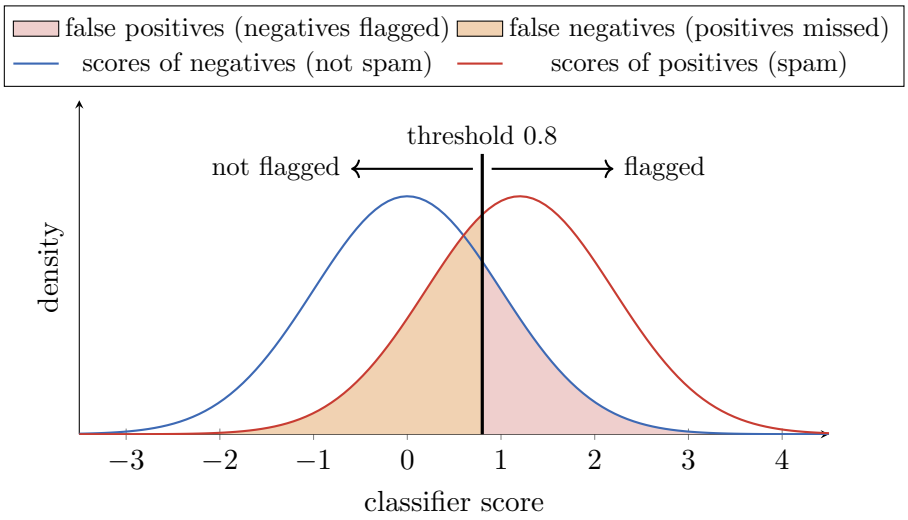


Figure 15.12: Score distributions for negatives (blue) and positives (red) overlap. Everything right of the threshold is flagged. Sliding it right shrinks the false positives (red shading) and grows the false negatives (orange shading). No threshold removes both.

The **ROC curve** (receiver operating characteristic) plots TPR against FPR for every possible threshold. Here are 10 test examples sorted by score, with true labels (+ = positive):

score	0.95	0.90	0.80	0.70	0.60	0.55	0.45	0.40	0.30	0.20
label	+	+	−	+	+	−	−	+	−	−

There are 5 positives and 5 negatives. Start with the threshold above every score: nothing flagged, $(\text{FPR}, \text{TPR}) = (0, 0)$. Lower the threshold past one example at a time. Each positive moves the point up by $1/5$; each negative moves it right by $1/5$. The path is

$$(0,0) \to (0,0.2) \to (0,0.4) \to (0.2,0.4) \to (0.2,0.6) \to (0.2,0.8)$$
$$\to (0.4,0.8) \to (0.6,0.8) \to (0.6,1) \to (0.8,1) \to (1,1).$$

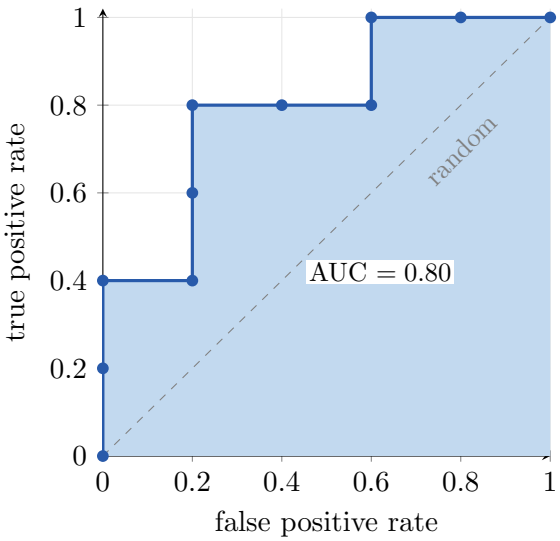


Figure 15.13: ROC curve for the 10 scored examples. Each up-step is a positive, each right-step a negative, taken in score order. The shaded area is 0.80. The dashed diagonal is a classifier that guesses at random.

Definition 15.3 (AUC)**.** The **AUC** is the area under the ROC curve. Equivalently, it is the probability that a randomly chosen positive gets a higher score than a randomly chosen negative (ties count one half).

Check the equivalence on our data. There are $5 \times 5 = 25$ positive–negative pairs. Count the pairs in which the positive outscores the negative: the positive at 0.95 beats all 5 negatives, 0.90 beats 5, 0.70 beats 4, 0.60 beats 4, and 0.40 beats 2. Total 20, and $20/25 = 0.80$, matching the area. The area interpretation is a picture; the pairs interpretation is the one to remember. It is the Mann–Whitney U statistic of Chapter 10 divided by the number of pairs.

Pause

Pause and think: what AUC does a classifier get if it outputs the same score for every example? (One half. Every pair is a tie.) What if you flip the sign of every score of a model with AUC 0.80? (0.20.)

History

ROC curves come from radar. During the Second World War, operators watched noisy screens and had to call each blip “aircraft” or “noise”. A jumpy operator caught every plane and raised many false alarms; a cautious one missed planes. Engineers wanted to compare the *receivers* (the radar sets) without mixing in each operator’s caution. Their answer was to plot hit rate against false-alarm rate across all levels of caution. In 1954 Peterson, Birdsall and Fox turned this into signal detection theory, and psychologists Green and Swets used it in the 1960s to separate what a listener can hear from how willing they are to say “yes”. Hanley and McNeil (1982) showed medical researchers that the area under the curve equals the probability of ranking a random sick patient above a random healthy one. That is the AUC every ML dashboard now reports.

15.6.3 Precision–recall curves and base rates

ROC curves ignore how common positives are: TPR is computed within positives, FPR within negatives. Precision mixes the two, so it depends on the **base rate** (the fraction of examples that are positive). The **precision–recall (PR) curve** plots precision against recall across thresholds. For our 10 examples, flagging the top k for $k = 1, \dots, 10$ gives recall = (positives flagged)/5 and precision = (positives flagged)/k:

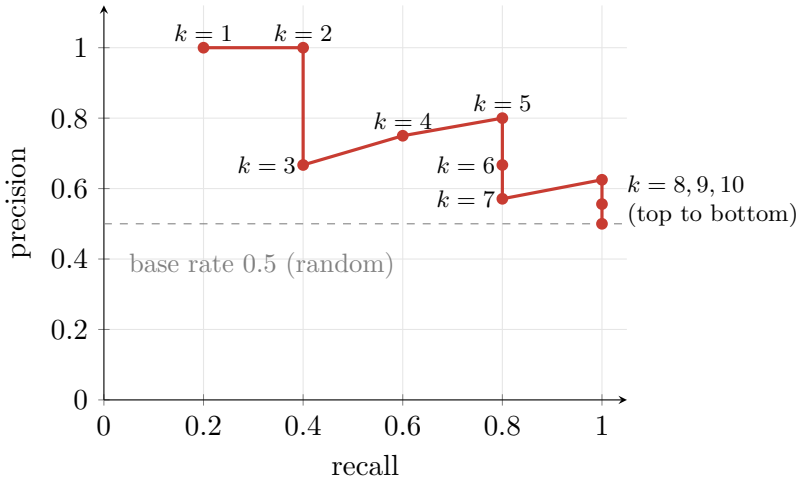


Figure 15.14: Precision–recall curve for the same 10 examples; dot k means "flag the top k scores". It zigzags: each positive flagged moves right and up, each negative flagged drops straight down. A random classifier's precision equals the base rate (dashed), so the floor of this plot moves with how rare positives are.

Base rates can turn a good classifier into a disappointing one. Keep the spam filter's TPR = 0.8 and FPR = 1/30, and deploy it where only 1% of 100,000 emails are spam. It catches $0.8 \times 1000 = 800$ spam and flags $99{,}000/30 = 3300$ good emails. Precision is

$$\frac{800}{800 + 3300} \approx 0.195.$$

Four out of five flags are now wrong, with the same ROC point. In general

$$\text{precision} = \frac{\text{TPR} \cdot \pi}{\text{TPR} \cdot \pi + \text{FPR} \cdot (1 - \pi)},$$

where π is the base rate. This is Bayes' rule (Chapter 14) with the classifier as the test.

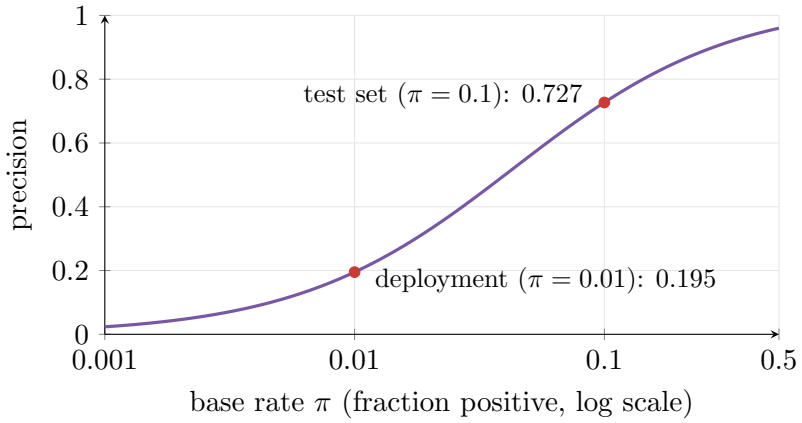


Figure 15.15: Precision of a fixed classifier (TPR = 0.8, FPR = 1/30) as the base rate changes. Same model, same ROC point, wildly different precision. Report the base rate with every precision number.

Use ROC/AUC when you care about ranking quality independent of class balance. Use PR curves when positives are rare and the cost of a false alarm matters, as in fraud, search, and medical screening.

15.6.4 Calibration

A weather forecaster who says "70% chance of rain" is **calibrated** if it rains on about 70% of such days. A classifier is calibrated if, among examples it scores near 0.7, about 70% are positive. AUC cannot see calibration: squaring every score leaves the ranking, and hence the AUC, unchanged, while wrecking the probabilities.

To check, sort 1000 test predictions into ten bins by predicted probability and compare each bin's mean prediction with the fraction of positives in it. The result is a **reliability diagram** (Figure 15.16).

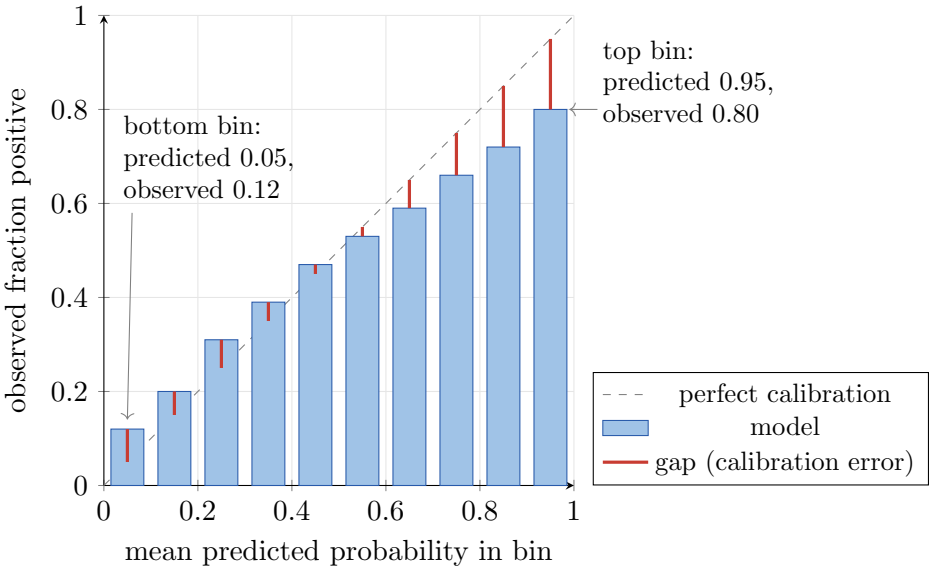


Figure 15.16: Reliability diagram for an overconfident model. Each red segment is one bin's gap between the bar (what happened) and the diagonal (what was predicted). Predictions of 0.95 come true only 80% of the time, and predictions of 0.05 come true 12% of the time: the model's probabilities are too extreme at both ends.

A one-number summary is the **expected calibration error** (ECE): the bin-size-weighted average of |mean prediction − observed fraction|. With bin sizes $300, 120, 80, 60, 50, 50, 60, 80, 100, 100$ for the ten bins above, the gaps are $0.07, 0.05, 0.06, 0.04, 0.02, 0.02, 0.06, 0.09, 0.13, 0.15$, and

$$\text{ECE} = \frac{300(0.07) + 120(0.05) + \cdots + 100(0.15)}{1000} = \frac{75}{1000} = 0.075.$$

Overconfidence of this shape is common in large neural networks. A cheap fix is to refit a one-parameter rescaling of the scores on the validation set (temperature scaling, a close cousin of logistic regression from Chapter 13).

15.7 Uncertainty in ML metrics

A test-set metric is a statistic computed from a sample. It has a standard error. A leaderboard that reports "87.0% vs 86.4%" without one is reporting noise as often as signal.

15.7.1 A confidence interval for accuracy

Each test example is a Bernoulli trial: right (1) or wrong (0). If the test examples are i.i.d., the number correct is Binomial, and accuracy is a sample proportion. Chapter 7's interval applies directly. With 435 correct out of $n = 500$, $\hat{a} = 0.87$:

$$\text{SE} = \sqrt{\frac{\hat{a}(1-\hat{a})}{n}} = \sqrt{\frac{0.87 \times 0.13}{500}} \approx 0.0150, \qquad 0.87 \pm 1.96 \times 0.0150 = [0.841,\ 0.899].$$

The true accuracy could plausibly be anywhere from 84% to 90%. To pin accuracy near 0.9 to ± 1 point at 95% confidence you need $n \approx 1.96^2(0.9)(0.1)/0.01^2 \approx 3460$ test examples. For accuracies near 0 or 1 or small n, use the Wilson interval instead (second line of output below).

```
import numpy as np
from scipy import stats
k, n = 435, 500
a = k / n; se = np.sqrt(a * (1 - a) / n)
print(a - 1.96 * se, a + 1.96 * se)          # Wald: 0.841, 0.899
print(stats.binomtest(k, n).proportion_ci(method="wilson"))  # 0.838, 0.897
```

15.7.2 A bootstrap interval for AUC

AUC has no simple standard-error formula worth memorising. The bootstrap (Chapter 8) handles it: resample the test set with replacement, recompute AUC, repeat, and read off percentiles. Resample positives and negatives separately so every replicate keeps the same class balance.

On a test set of 60 positives and 60 negatives, a model scores AUC = 0.769. Two thousand stratified bootstrap replicates give the histogram in Figure 15.17. The middle 95% runs from 0.680 to 0.852, and the bootstrap SE is 0.043.

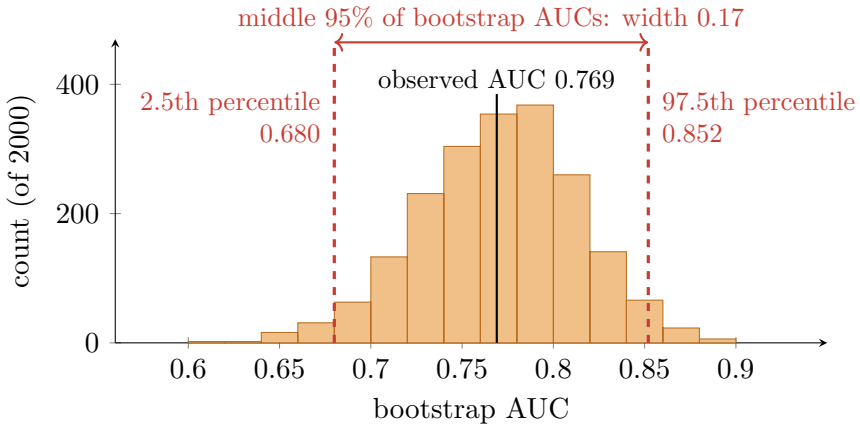


Figure 15.17: Bootstrap distribution of AUC on a 120-example test set. The 95% percentile interval is nearly 0.17 wide. Small test sets give AUCs that are much less precise than their three decimal places suggest.

```
import numpy as np
rng = np.random.default_rng(0)
def auc(p, n):                          # P(pos score > neg score), ties = 1/2
    return np.mean(p[:, None] > n) + 0.5 * np.mean(p[:, None] == n)
def boot_auc(pos, neg, B=2000):
    reps = [auc(rng.choice(pos, len(pos)), rng.choice(neg, len(neg)))
            for _ in range(B)]
    return np.percentile(reps, [2.5, 97.5])
```

15.7.3 Comparing two models on the same test set

Model A gets 435/500 right and model B gets 420/500. Is A better, or did it get lucky? Two separate confidence intervals overlap heavily, but that comparison throws away information: both models were scored on the *same* examples. This is paired data, like the before/after measurements of Chapter 9. Only the examples where the models disagree carry evidence about which is better.

	B right	B wrong
A right	395	$b = 40$
A wrong	$c = 25$	40

Check the margins: A is right on $395 + 40 = 435$, B on $395 + 25 = 420$. If the models were equally good, each of the $b + c = 65$ disagreements would be a fair coin flip between "A wins" and "B wins". **McNemar's test** asks whether 40 versus 25 is a surprising split for 65 fair flips:

$$\chi^2 = \frac{(b-c)^2}{b+c} = \frac{(40-25)^2}{65} = \frac{225}{65} \approx 3.46, \qquad p \approx 0.063 \;\; (\chi^2 \text{ with 1 df}).$$

The exact binomial version (count ≤ 25 or ≥ 40 out of 65 under $p = 1/2$) gives $p \approx 0.082$. The three-point accuracy gap is suggestive but not convincing on 500 examples (Figure 15.18).

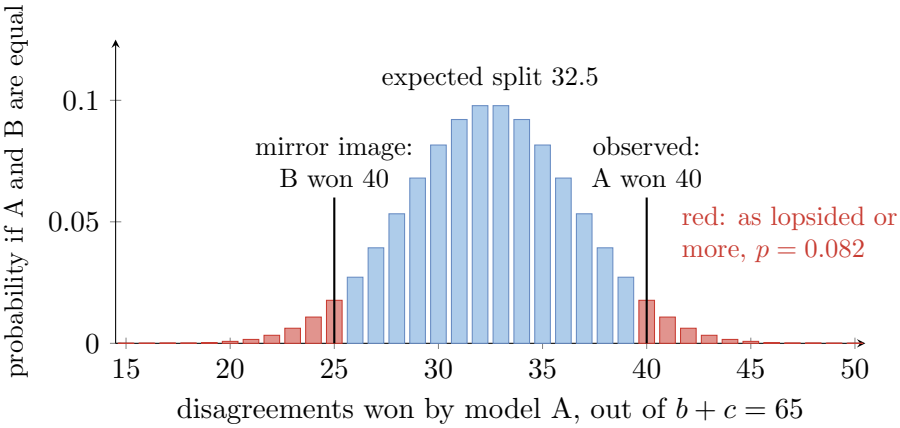


Figure 15.18: McNemar's test as a coin-flip picture. If the models were equally good, the number of the 65 disagreements won by A would follow this Binomial(65, $\frac{1}{2}$) distribution, centred on 32.5. The red bars are splits at least as lopsided as the observed 40–25; together they have probability 0.082, the exact p-value. Such a split is not rare enough to be convincing.

```
from scipy import stats
b, c = 40, 25                          # discordant pairs
chi2 = (b - c)**2 / (b + c)
print(chi2, stats.chi2.sf(chi2, 1))    # 3.46, 0.063
print(stats.binomtest(c, b + c, 0.5).pvalue)   # exact: 0.082
```

For metrics that are not per-example averages (AUC, F1), use a **paired bootstrap**: resample test-example indices with replacement, compute the metric for *both* models on the same resample, record

the difference, and look at the percentile interval of the differences. If it excludes zero, the gap is unlikely to be resampling noise.

Common trap

Comparing a new model against a baseline across 20 random seeds and 10 benchmarks, then reporting the one where it wins, is the garden of forking paths from Chapter 11. Decide the comparison in advance, and correct for the number of comparisons you actually ran.

15.8 Stochastic gradients are Monte Carlo estimates

Gradient descent minimises the training loss by repeatedly stepping downhill: $\theta \leftarrow \theta - \eta \, \nabla_\theta \hat{R}_n(\theta)$, where the **learning rate** η is the step size. The gradient of an average is the average of the gradients:

$$\nabla_\theta \hat{R}_n(\theta) = \frac{1}{n} \sum_{i=1}^{n} \nabla_\theta \, \ell_i(\theta), \qquad \ell_i(\theta) = \ell\big(f_\theta(x_i), y_i\big).$$

With n in the millions, computing all n terms per step is wasteful. **Stochastic gradient descent** (SGD) instead averages over a random **minibatch** of B examples. That is exactly a sample mean estimating a population mean, where the "population" is the n per-example gradients. An average of random draws used to estimate an expectation is called a **Monte Carlo estimate**; the bootstrap of Chapter 8 was one too.

15.8.1 By hand: four points

Fit $f_\theta(x) = \theta x$ with squared loss to $x = (1, 2, 3, 4)$, $y = (2, 5, 5, 9)$, at the current value $\theta = 1$. The per-example gradient is $-2x_i(y_i - \theta x_i)$:

$$g = \big(-2(1)(1),\ -2(2)(3),\ -2(3)(2),\ -2(4)(5)\big) = (-2,\ -12,\ -12,\ -40), \qquad \bar{g} = -16.5.$$

The six possible minibatches of size $B = 2$ give gradient estimates

$$\tfrac{-2-12}{2} = -7, \quad \tfrac{-2-12}{2} = -7, \quad \tfrac{-2-40}{2} = -21, \quad \tfrac{-12-12}{2} = -12, \quad \tfrac{-12-40}{2} = -26, \quad \tfrac{-12-40}{2} = -26.$$

Their average is $(-7-7-21-12-26-26)/6 = -99/6 = -16.5$, the full gradient. So the minibatch gradient is **unbiased**. It is noisy: single examples ($B = 1$) give estimates with variance 200.75; pairs give variance 66.9. In general the variance falls like $1/B$, so the standard error falls like $1/\sqrt{B}$, the same $\sigma/\sqrt{n}$ law as Chapter 6. (Here the drop is a bit faster than $1/B$ because we sample without replacement from only four points.) Figure 15.19 lays out every possible estimate.

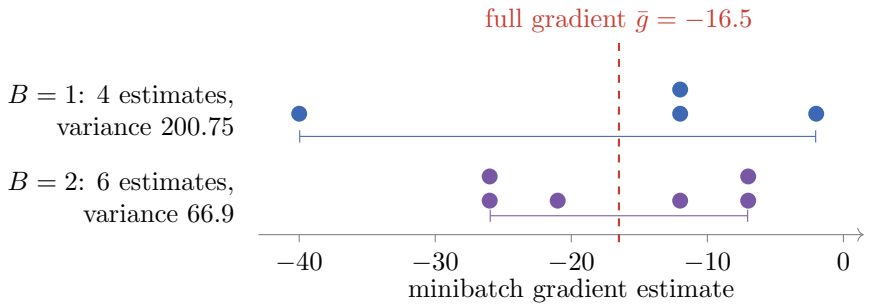


Figure 15.19: Every minibatch gradient for the four-point example; stacked dots are equal estimates. Both rows balance on the full gradient -16.5 (dashed), which is what unbiased means. Doubling the batch size pulls the estimates in toward it: the range shrinks from $[-40, -2]$ to $[-26, -7]$.

What is really happening

Read the true objective as $R(\theta) = \mathbb{E}_{(x,y)\sim p}[\ell]$. Then each fresh minibatch is a Monte Carlo estimate of $\nabla R(\theta)$ itself, not just of the training-set gradient. In one pass over fresh data, SGD is directly descending the expected risk. That is why papers write the gradient as $\mathbb{E}_{(x,y)\sim p}[\nabla_\theta \ell]$ and then approximate it with a batch average in the next line.

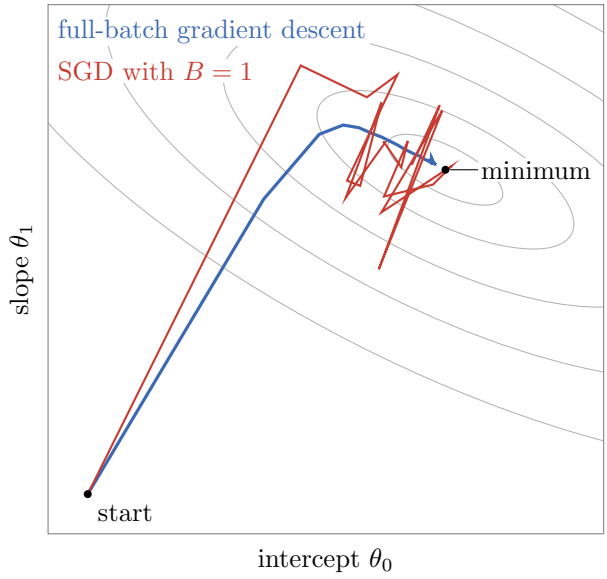


Figure 15.20: Gradient descent on a two-parameter linear regression (200 points), plotted every second step. Grey ellipses are contours of the training loss. Full-batch steps (blue) glide smoothly to the minimum. Single-example steps (red) point downhill only on average, so the path jitters around the minimum instead of settling. Larger batches or a shrinking learning rate reduce the jitter.

15.9 Covariance matrices, the multivariate Normal, and PCA

ML inputs are vectors. A house is (area, age, distance to centre); an image is a million pixels. The spread of a vector needs more than one number.

15.9.1 The covariance matrix

Six points in two dimensions:

$$(2,1),\ (3,3),\ (4,3),\ (5,5),\ (6,4),\ (4,2).$$

The mean is $(4,3)$. The deviations from the mean are $(-2,-2), (-1,0), (0,0), (1,2), (2,1), (0,-1)$. Using the $n-1=5$ divisor from Chapter 2:

$$s_{11} = \tfrac{4+1+0+1+4+0}{5} = 2, \qquad s_{22} = \tfrac{4+0+0+4+1+1}{5} = 2, \qquad s_{12} = \tfrac{4+0+0+2+2+0}{5} = 1.6.$$

Stack them into the **sample covariance matrix**

$$S = \begin{pmatrix} 2 & 1.6 \\ 1.6 & 2 \end{pmatrix}.$$

The diagonal holds each coordinate's variance. The off-diagonal holds the covariance of Chapter 4; dividing by the SDs gives the correlation $1.6/\sqrt{2 \cdot 2} = 0.8$. In general, for data vectors $x_i \in \mathbb{R}^d$ with mean $\bar{x}$,

$$S = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})(x_i - \bar{x})^\top,$$

a $d \times d$ symmetric matrix. Its population version is $\Sigma = \mathbb{E}[(x-\mu)(x-\mu)^\top]$.

15.9.2 The multivariate Normal

The **multivariate Normal** $\mathcal{N}(\mu, \Sigma)$ is the bell curve in d dimensions. Its density is

$$p(x) = \frac{1}{(2\pi)^{d/2}\,|\Sigma|^{1/2}} \exp\Big(-\tfrac{1}{2}(x-\mu)^\top \Sigma^{-1} (x-\mu) \Big).$$

The exponent is a squared distance that measures each direction in units of that direction's spread. Its contours are ellipses centred at μ, with axes along the eigenvectors of Σ and half-lengths proportional to the square roots of the eigenvalues. When $d = 1$ it reduces to the familiar $\mathcal{N}(\mu, \sigma^2)$ density. Bishop Ch. 2 is the standard reference.

15.9.3 PCA: the directions of greatest spread

An **eigenvector** of S is a direction v that S only stretches, never turns: $Sv = \lambda v$, and the stretch factor λ (no relation to the ridge penalty) is its **eigenvalue**. For a covariance matrix, the eigenvalue is the variance of the data measured along v, so the eigenvectors are the directions along which the data are most and least spread out. For our S, the eigenvalues are $2 + 1.6 = 3.6$ and $2 - 1.6 = 0.4$, with eigenvectors $(1,1)/\sqrt{2}$ and $(1,-1)/\sqrt{2}$. Check one: $S(1,1)^\top = (3.6, 3.6)^\top = 3.6\,(1,1)^\top$.

Principal component analysis (PCA) uses these directions as new axes. The first principal component points along $(1,1)/\sqrt{2}$, and the variance of the data projected onto it is 3.6. The total variance is $2 + 2 = 4$, so the first component captures $3.6/4 = 90\%$ of it. Dropping the second component compresses each point from two numbers to one while losing only 10% of the variance.

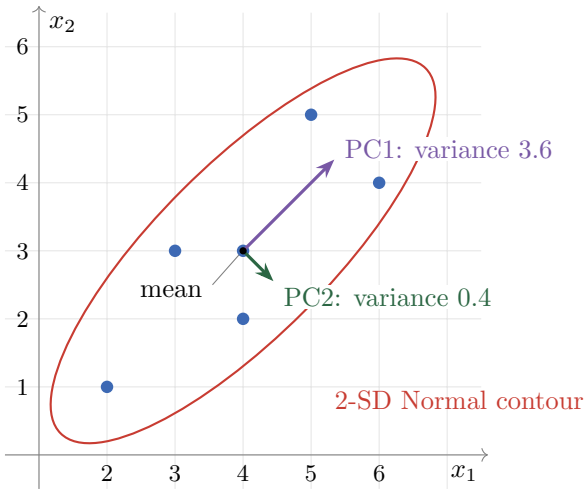


Figure 15.21: Six points with covariance matrix S. The arrows are the eigenvectors of S, each drawn from the mean $(4, 3)$ with length equal to its standard deviation ($\sqrt{3.6} \approx 1.90$ and $\sqrt{0.4} \approx 0.63$). The red ellipse is the 2-SD contour of a Normal with this covariance; its long axis is PC1. PCA keeps that axis and drops the short one.

Connection

PCA, Gaussian mixture models, Gaussian processes, and the Kalman filter are all built on covariance matrices. The initialisation schemes and normalisation layers in deep learning are attempts to keep the covariance of activations close to the identity matrix. ISLR Ch. 12 and Bishop Ch. 12 cover PCA in full.

15.10 What you can now read

The statistical half of the standard ML texts is now familiar ground. Table 15.2 maps their early chapters to this book.

Text and chapter	Topic	Where it is here
ISLR Ch. 2	bias–variance, train vs test MSE	Ch. 14, Sec. 15.4
ISLR Ch. 3–4	linear and logistic regression	Ch. 12–13
ISLR Ch. 5	cross-validation, bootstrap	Ch. 8, Sec. 15.3
ISLR Ch. 6	ridge, lasso, model selection	Sec. 15.5
ISLR Ch. 12	PCA	Sec. 15.9
Bishop Ch. 1	curve fitting, overfitting, Bayes, decision theory	Ch. 14, Secs. 15.2–15.4
Bishop Ch. 2	Bernoulli, Binomial, Gaussian, MLE	Ch. 6, 14, Sec. 15.9
Bishop Ch. 3	linear basis models, regularisation, bias–variance	Ch. 12, Secs. 15.4–15.5
Goodfellow Ch. 3	probability, expectation, covariance, Gaussians	Sec. 15.1, 15.9
Goodfellow Ch. 5	capacity, validation, estimators, bias/variance, MLE, SGD	this whole chapter, Ch. 14

Table 15.2: Reading map. ISLR is James, Witten, Hastie and Tibshirani, *An Introduction to Statistical Learning* (2nd ed.). Bishop is *Pattern Recognition and Machine Learning*. Goodfellow is Goodfellow, Bengio and Courville, *Deep Learning*.

When a page gets dense, apply the decoder: find each expectation and ask "average of what, over which distribution?"; find each hat and ask "estimated from which data?"; find each reported number and ask "what is its standard error?"

15.11 Summary

- $\mathbb{E}_{x\sim p}$ is an average over the world; $\frac{1}{n}\sum_i$ is the same average over your data ($\mathbb{E}_{x\sim\hat{p}}$). Hats mark estimates; arg min is a location.
- ERM minimises empirical risk. Its training loss is optimistic; the generalisation gap $R(\hat{f}) - \hat{R}_n(\hat{f})$ grows with flexibility.
- Train / validation / test: fit, choose, report. k-fold CV reuses data for choosing. Every data-dependent preprocessing step goes inside the split.
- Expected test error $= \sigma^2 + \text{bias}^2 + \text{variance}$. Flexibility trades bias for variance and the sum is U-shaped.
- Ridge divides coefficients toward zero; lasso subtracts toward zero and produces exact zeros (corners of the diamond).
- From the confusion matrix: precision $= TP/(TP+FP)$, recall $= TP/(TP+FN)$, $F_1 =$ harmonic mean. AUC $= \mathbb{P}(\text{score}_+ > \text{score}_-)$. Precision depends on the base rate; ROC does not. Calibration is a separate property, read off a reliability diagram.

- Metrics have standard errors: binomial CI for accuracy, bootstrap for AUC, McNemar or paired bootstrap to compare two models on one test set.
- A minibatch gradient is an unbiased Monte Carlo estimate of the full gradient with SE $\propto 1/\sqrt{B}$.
- The covariance matrix holds variances and covariances; its eigenvectors are the principal components and the axes of multivariate Normal contours.

Chapter 16 collects the formulas from the whole book in one place, together with a decision guide for choosing an interval or a test.

15.12 Exercises

1. A fraud model on 1000 transactions has $TP = 45$, $FP = 15$, $FN = 5$, $TN = 935$. Compute accuracy, precision, recall and F_1. What accuracy does "never flag" get?
2. A classifier gets 368 of 400 test examples right. Give a 95% confidence interval for its accuracy.
3. Models A and B are scored on the same test set. A is right and B wrong on 30 examples; B is right and A wrong on 12. Compute McNemar's χ^2 and its p-value.
4. Two positives score 0.9 and 0.4; two negatives score 0.6 and 0.2. Compute the AUC by counting pairs, and draw the ROC curve.
5. With an orthonormal design, least squares gives $\hat{\beta} = (2.0, -0.3, 0.8)$. Find the ridge and lasso estimates for $\lambda = 1$ using the formulas of Section 15.5.
6. A screening test has TPR $= 0.9$ and FPR $= 0.05$. The condition affects 2% of people tested. What fraction of positive results are true positives?
7. A dataset has covariance matrix $\begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}$. What are the principal components, and what fraction of the variance does the first one capture?
8. Show that a minibatch gradient computed from B examples drawn uniformly at random with replacement is unbiased for the full-batch gradient, and that its variance is σ^2/B where σ^2 is the variance of the per-example gradients.
9. A team standardises all features using the full dataset's mean and SD, then runs 5-fold CV. Another team picks the 10 features most correlated with the label on the full dataset, then runs 5-fold CV. Which leak is worse, and why?
10. How many test examples do you need for a 95% CI of half-width 0.02 on an accuracy near 0.8?

Answers

1. Accuracy is $980/1000 = 0.98$, precision is $45/60 = 0.75$, and recall is $45/50 = 0.90$. So $F_1 = 1.35/1.65 \approx 0.818$. Never flagging scores 0.95.
2. $\hat{a} = 0.92$, SE $= \sqrt{0.92 \times 0.08/400} \approx 0.0136$, interval $0.92 \pm 0.027 = [0.893,\ 0.947]$.
3. $\chi^2 = (30 - 12)^2/42 = 324/42 \approx 7.71$, $p \approx 0.005$. Strong evidence that A is better on this distribution.

4. Pairs: $0.9 > 0.6$, $0.9 > 0.2$, $0.4 > 0.2$, and $0.4 < 0.6$. AUC $= 3/4 = 0.75$. ROC path: $(0,0) \to (0,0.5) \to (0.5,0.5) \to (0.5,1) \to (1,1)$.

5. Ridge: (1.0, −0.15, 0.4). Lasso: (1.5, 0, 0.3).

6. $\dfrac{0.9(0.02)}{0.9(0.02) + 0.05(0.98)} = \dfrac{0.018}{0.067} \approx 0.27$. About three positives in four are false.

7. The coordinate axes: PC1 along x_1 (variance 4), PC2 along x_2 (variance 1). PC1 captures $4/5 = 80\%$.

8. Each sampled index I_j is uniform on $\{1, \ldots, n\}$, so $\mathbb{E}[g_{I_j}] = \frac{1}{n}\sum_i g_i = \bar{g}$, and the batch mean has expectation $\bar{g}$ by linearity. The B draws are independent, each with variance σ^2, so the variance of their mean is σ^2/B (Chapter 6).

9. Feature selection is far worse. Standardising leaks only a mean and SD per feature, which barely changes. Selecting features by their correlation with the label lets every fold's labels influence which features the model sees, which can produce high CV accuracy on pure noise.

10. $n = 1.96^2(0.8)(0.2)/0.02^2 \approx 1536.6$, so about 1540 examples.

Part V

Reference

Chapter 16

Appendix: Formula Sheet and Decision Guide

Chapters 1–15 built each tool from a small dataset up. This appendix puts the results on as few pages as possible: formulas, critical values and decision rules, each pointing back to the chapter that explains it. Every critical value below was computed with `scipy.stats` and rounded, not copied from an older table.

16.1 Notation

Symbol	Meaning
$x_1, \dots, x_n$	observed data values (lower case = numbers you have)
$X_1, \dots, X_n$	the same values viewed as random variables before sampling
$x_{(1)} \le \cdots \le x_{(n)}$	order statistics: the data sorted from smallest to largest
n	sample size; n_1, n_2 for two groups
$\mu,\ \sigma^2,\ \sigma$	population mean, variance, standard deviation
$\bar{x},\ s^2,\ s$	sample mean, sample variance (divide by $n-1$), sample SD
$p,\ \hat{p}$	population proportion and sample proportion $\hat{p} = k/n$
$\theta,\ \hat{\theta}$	a generic parameter and its estimate (a hat means "estimated from data")
$\mathbb{E}[X]$, $\mathrm{Var}(X)$, $\mathrm{Cov}(X,Y)$	expectation, variance and covariance of random variables
$\mathbb{E}_{x\sim p}[f(x)]$	average of $f(x)$ when x is drawn from distribution p (ML notation)
$\mathrm{SE}(\hat{\theta})$	standard error: the SD of the sampling distribution of $\hat{\theta}$
$X \sim \mathcal{N}(\mu, \sigma^2)$	X is normal with mean μ and variance σ^2 (second slot is variance)
$X_i \overset{\text{iid}}{\sim} F$	independent draws, all from the same distribution F
$z_q,\ t_{q,\nu}$	the point with upper-tail area q under $\mathcal{N}(0,1)$ and under the t distribution with ν df (so $z_{0.025} = 1.96$)
$z^*,\ t^*$	critical value for a stated confidence level: $z^* = z_{\alpha/2}$ and $t^* = t_{\alpha/2,\nu}$, e.g. $z^* = 1.96$ at 95%
$\alpha,\ \beta,\ 1-\beta$	type I error rate, type II error rate, power
$H_0,\ H_1$	null and alternative hypotheses
$\hat{\beta}_0,\ \hat{\beta}_1,\ \hat{y}_i,\ e_i$	fitted intercept, slope, fitted value, residual $e_i = y_i - \hat{y}_i$
$S_{xx},\ S_{xy}$	$\sum (x_i - \bar{x})^2$ and $\sum (x_i - \bar{x})(y_i - \bar{y})$

16.2 Descriptive statistics

Every formula in the table is evaluated on the same five latencies (ms): $2, 4, 4, 5, 10$. The data are already sorted. The mean is $25/5 = 5$; the deviations are $-3, -1, -1, 0, 5$; their squares sum to $9 + 1 + 1 + 0 + 25 = 36$.

Statistic	Formula	On $2, 4, 4, 5, 10$
Mean	$\bar{x} = \frac{1}{n}\sum_i x_i$	$25/5 = 5$
Median	middle of sorted data; average of the two middle values if n even	4
Sample variance	$s^2 = \frac{1}{n-1}\sum_i (x_i - \bar{x})^2$	$36/4 = 9$
Population variance	$\sigma^2 = \frac{1}{N}\sum_i (x_i - \mu)^2$	$36/5 = 7.2$
Sample SD	$s = \sqrt{s^2}$	3
Range	$x_{(n)} - x_{(1)}$	$10 - 2 = 8$
Quartiles	Q_1, Q_3 = 25th, 75th percentiles (linear interpolation at position $(n-1)q$)	$Q_1 = 4$, $Q_3 = 5$
IQR	$Q_3 - Q_1$	1
MAD	$\text{median}_i \lvert x_i - \text{median}(x) \rvert$; $\times 1.4826$ estimates σ for normal data	$\text{median}(2, 0, 0, 1, 6) = 1$
z-score	$z_i = (x_i - \bar{x})/s$	for 10: $5/3 \approx 1.67$
Coefficient of variation	$s/\bar{x}$	$3/5 = 0.6$
Tukey fences	$Q_1 - 1.5$ IQR, $Q_3 + 1.5$ IQR	2.5 and 6.5: so 2, 10 flagged

Common trap

`np.std(x)` divides by n, not $n - 1$. On the data above it returns $\sqrt{7.2} \approx 2.68$, not 3. Use `np.std(x, ddof=1)`. Pandas `Series.std()` already uses $n - 1$. Quantile conventions also differ between tools (there are at least nine); for small n the quartiles can change by a whole data point (Chapter 3).

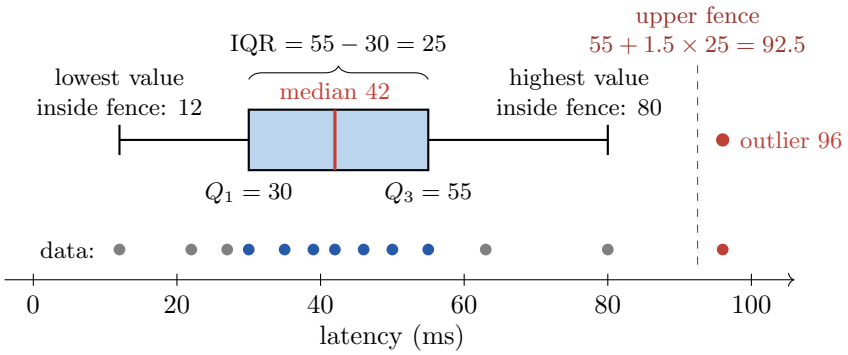


Figure 16.1: Box plot anatomy, drawn over the 13 latencies it summarises (dots). The box spans $Q_1 = 30$ to $Q_3 = 55$ and contains the middle seven values (blue); the red line is the median, 42. Whiskers stop at the most extreme values inside the fences (the lower fence, $30 - 37.5 = -7.5$, is off the scale), and 96, beyond the dashed upper fence, is drawn on its own as an outlier.

16.2.1 Two variables

Take three points $(x, y) = (1, 2), (2, 4), (3, 9)$. Then $\bar{x} = 2$, $\bar{y} = 5$, deviations $dx = (-1, 0, 1)$, $dy = (-3, -1, 4)$, so $S_{xy} = 3 + 0 + 4 = 7$, $S_{xx} = 2$, $S_{yy} = 26$.

Quantity	Formula	On the three points
Sample covariance	$s_{xy} = \frac{S_{xy}}{n-1}$	$7/2 = 3.5$
Correlation	$r = \frac{S_{xy}}{\sqrt{S_{xx} S_{yy}}}$	$7/\sqrt{52} \approx 0.971$
OLS slope	$\hat{\beta}_1 = \frac{S_{xy}}{S_{xx}} = r \, \frac{s_y}{s_x}$	$7/2 = 3.5$
OLS intercept	$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$	$5 - 7 = -2$
R^2 (simple regression)	$r^2 = 1 - \frac{\sum e_i^2}{S_{yy}}$	$0.971^2 \approx 0.942$

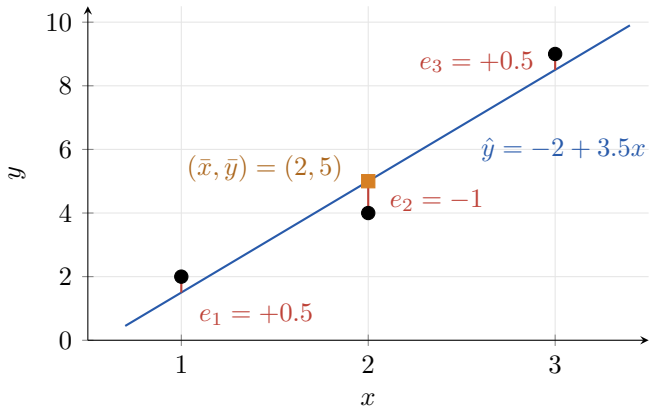


Figure 16.2: The three points from the table with their least-squares line. The red segments are the residuals; their squares sum to $0.25 + 1 + 0.25 = 1.5$, so $R^2 = 1 - 1.5/26 \approx 0.942$. The line always passes through the mean point $(\bar{x}, \bar{y})$ (orange square).

16.3 Expectation and variance rules

Proofs are in Probability From the Ground Up; Chapter 6 of this book uses them to derive standard errors. Here a, b are constants and X, Y random variables.

Rule	Formula
Linearity (always)	$\mathbb{E}[aX + bY + c] = a\,\mathbb{E}[X] + b\,\mathbb{E}[Y] + c$
Shift and scale	$\mathrm{Var}(aX + b) = a^2\,\mathrm{Var}(X), \quad \mathrm{SD}(aX + b) = \lvert a\rvert\,\mathrm{SD}(X)$
Shortcut	$\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$
Sum	$\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\,\mathrm{Cov}(X, Y)$
Difference	$\mathrm{Var}(X - Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) - 2\,\mathrm{Cov}(X, Y)$
Independent	$\mathrm{Cov}(X, Y) = 0$, so variances add for sums *and* differences
Covariance	$\mathrm{Cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}[X]\,\mathbb{E}[Y], \quad \mathrm{Corr}(X, Y) = \frac{\mathrm{Cov}(X,Y)}{\mathrm{SD}(X)\,\mathrm{SD}(Y)}$
Mean of iid sample	$\mathbb{E}[\bar{X}] = \mu, \quad \mathrm{Var}(\bar{X}) = \sigma^2/n, \quad \mathrm{SD}(\bar{X}) = \sigma/\sqrt{n}$
Sum of iid sample	$\mathbb{E}[\sum X_i] = n\mu, \quad \mathrm{SD}(\sum X_i) = \sigma\sqrt{n}$
Function of X (LOTUS)	$\mathbb{E}[g(X)] = \sum_x g(x)\,p(x)$ or $\int g(x) f(x)\,\mathrm{d}x$
Tower rule	$\mathbb{E}[X] = \mathbb{E}\left[\mathbb{E}[X \mid Y]\right]$
Total variance	$\mathrm{Var}(X) = \mathbb{E}[\mathrm{Var}(X \mid Y)] + \mathrm{Var}(\mathbb{E}[X \mid Y])$
Bias–variance	$\mathrm{MSE}(\hat{\theta}) = \mathrm{Bias}(\hat{\theta})^2 + \mathrm{Var}(\hat{\theta})$

Common trap

$\mathbb{E}[g(X)] \neq g(\mathbb{E}[X])$ in general. The mean of squared latencies is not the square of the mean latency, and $\mathbb{E}[1/X] \neq 1/\mathbb{E}[X]$. Linearity holds only for linear g. Likewise $\mathrm{SD}(X + Y) \neq \mathrm{SD}(X) + \mathrm{SD}(Y)$: variances add, standard deviations do not.

16.3.1 Common distributions

Distribution	Pmf / pdf	Mean	Variance
Bernoulli(p)	$p^x(1-p)^{1-x}$, $x \in \{0, 1\}$	p	$p(1-p)$
Binomial(n, p)	$\binom{n}{x} p^x (1-p)^{n-x}$	np	$np(1-p)$
Poisson(λ)	$e^{-\lambda}\lambda^x/x!$	λ	λ
Geometric(p) (trials)	$(1-p)^{x-1}p$, $x \geq 1$	$1/p$	$(1-p)/p^2$
Uniform(a, b)	$1/(b-a)$ on $[a, b]$	$(a+b)/2$	$(b-a)^2/12$
Exponential(λ)	$\lambda e^{-\lambda x}$, $x \geq 0$	$1/\lambda$	$1/\lambda^2$
$\mathcal{N}(\mu, \sigma^2)$	$\frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)}$	μ	σ^2
t_ν	symmetric, heavier tails than normal	0 ($\nu > 1$)	$\nu/(\nu-2)$ ($\nu > 2$)
χ^2_k	sum of k squared $\mathcal{N}(0, 1)$	k	$2k$

16.4 Standard errors

A standard error is the SD of an estimate across hypothetical repeated samples (Chapter 6). In practice we plug sample quantities into the population formula.

Estimate	Standard error	Notes
Mean $\bar{x}$	$\frac{\sigma}{\sqrt{n}} \approx \frac{s}{\sqrt{n}}$	needs independent observations
Proportion $\hat{p}$	$\sqrt{\frac{p(1-p)}{n}} \approx \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$	a mean of 0/1 values
Paired difference $\bar{d}$	$\frac{s_d}{\sqrt{n}}$	s_d = SD of within-pair differences
Difference of means	$\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}$	independent groups (Welch)
Difference of proportions	$\sqrt{\frac{\hat{p}_1(1-\hat{p}_1)}{n_1} + \frac{\hat{p}_2(1-\hat{p}_2)}{n_2}}$	A/B tests; for the test use pooled $\hat{p}$
Median	$\frac{1}{2f(m)\sqrt{n}}$	$f(m)$ = density at the median; normal data: $1.2533\,\sigma/\sqrt{n}$; bootstrap in practice
Regression slope $\hat{\beta}_1$	$\frac{s_e}{\sqrt{S_{xx}}}$, $s_e = \sqrt{\frac{\sum e_i^2}{n-2}}$	shrinks with spread of x
Correlation (Fisher)	$\frac{1}{\sqrt{n-3}}$ on $z = \tanh^{-1}(r)$ scale	back-transform CI ends with tanh
Any statistic	SD of the statistic over bootstrap resamples	Chapter 8

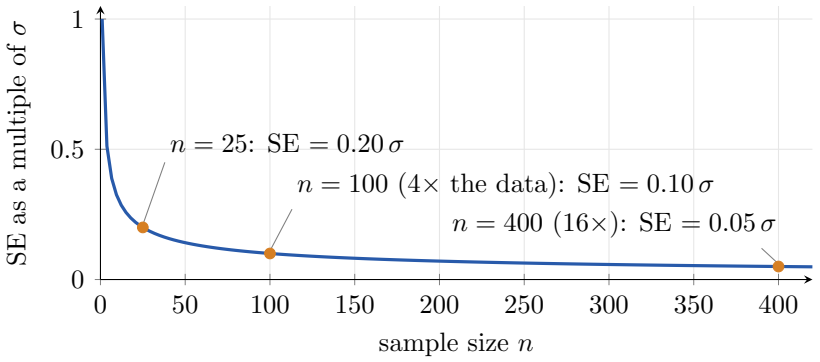


Figure 16.3: The $1/\sqrt{n}$ law: the SE of a mean is $\sigma/\sqrt{n}$. Follow the orange points: each halving of the standard error costs four times the data, so going from 25 to 100 to 400 observations halves the SE twice. The curve flattens, which is why extra data buys less and less precision.

Pause

A dashboard shows a conversion rate of 5% from 2,000 users with SE $\approx$ 0.49 percentage points. What SE do you expect at 8,000 users with the same rate? (Four times the data, so half the SE: about 0.24 points.)

16.5 Confidence interval recipes

Almost every interval in the book has the shape *estimate* ± *critical value* × *SE* (Chapter 7). The exceptions are Wilson, order-statistic and percentile-bootstrap intervals, which need not be symmetric.

Target	Interval	Use when
Mean, z	$\bar{x} \pm z^* \, \sigma/\sqrt{n}$	σ truly known (rare), or n in the hundreds
Mean, t	$\bar{x} \pm t^*_{n-1} \, s/\sqrt{n}$	default for a mean; fine for mild skew once $n \gtrsim 30$
Paired mean	$\bar{d} \pm t^*_{n-1} \, s_d/\sqrt{n}$	before/after on the same units
Two means	$(\bar{x}_1 - \bar{x}_2) \pm t^*_\nu \sqrt{s_1^2/n_1 + s_2^2/n_2}$	independent groups; Welch ν from software
Proportion, Wald	$\hat{p} \pm z^* \sqrt{\hat{p}(1-\hat{p})/n}$	only if $n\hat{p} \geq 10$ and $n(1-\hat{p}) \geq 10$
Proportion, Wilson	$\dfrac{\hat{p} + \frac{z^2}{2n}}{1 + \frac{z^2}{n}} \pm \dfrac{z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$	default for a proportion; good even for small n or $\hat{p}$ near 0 or 1
Two proportions	$(\hat{p}_1 - \hat{p}_2) \pm z^*$ SE (unpooled)	A/B conversion rates with healthy counts
Median (exact)	$[x_{(j)}, \, x_{(n-j+1)}]$, j from Binomial$(n, \frac{1}{2})$	any continuous distribution; no shape assumption
Any statistic	percentile bootstrap: 2.5th and 97.5th percentiles of $\hat{\theta}^*$	medians, p99, ratios, R^2; n not tiny
Regression slope	$\hat{\beta}_1 \pm t^*_{n-2}$ SE$(\hat{\beta}_1)$	linear trend, roughly constant residual spread

Example: 8 successes in 20 trials, $z = 1.96$. Wilson gives [0.219, 0.613], centred at 0.416: pulled from $\hat{p} = 0.4$ toward $\frac{1}{2}$. Wald gives $0.4 \pm 0.215 = [0.185, \, 0.615]$, symmetric about $\hat{p}$. For small n or $\hat{p}$ near 0 or 1, Wald's actual coverage falls well below the nominal 95%; Wilson's stays close.

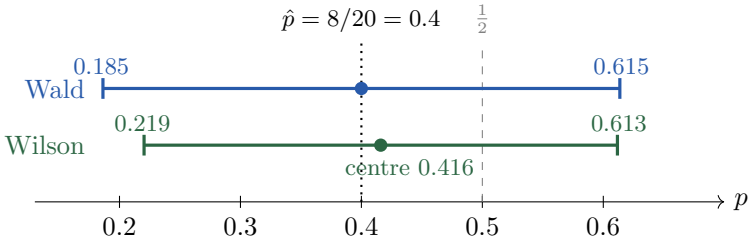


Figure 16.4: The two 95% intervals for 8 successes in 20 trials (dots mark each interval's centre). Wald is centred on $\hat{p} = 0.4$; Wilson's centre is pulled toward $\frac{1}{2}$, so its lower end moves in from 0.185 to 0.219 while its upper end barely changes.

16.5.1 Order-statistic CI for the median

The number of data points below the true median is Binomial$(n, \frac{1}{2})$. Pick the largest j with $\mathbb{P}(\text{Binomial}(n, \frac{1}{2}) \le j - 1) \le 0.025$. Then $[x_{(j)}, x_{(n-j+1)}]$ covers the median with probability at least 95%. Because the binomial is discrete, the actual coverage is a little above 95%.

n	10	15	20	25	30	50	100
ranks $(j,\ n-j+1)$	(2, 9)	(4, 12)	(6, 15)	(8, 18)	(10, 21)	(18, 33)	(40, 61)
exact coverage	0.979	0.965	0.959	0.957	0.957	0.967	0.965

For large n the ranks are approximately $\frac{n}{2} \pm 0.98\sqrt{n}$ (that is, $\frac{n}{2} \pm z^*\sqrt{n}/2$).

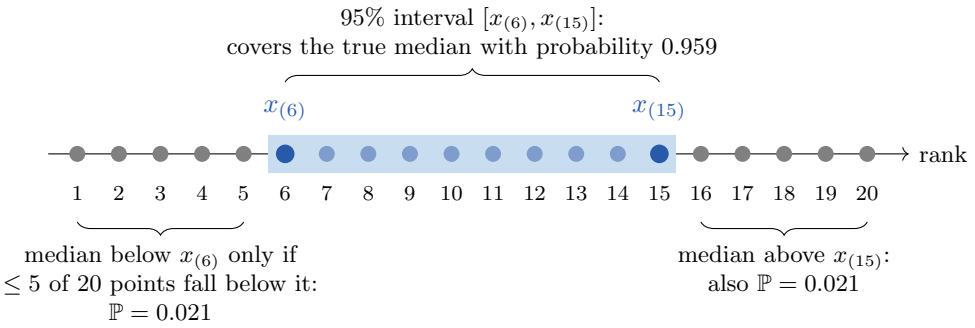


Figure 16.5: The order-statistic interval for $n = 20$, drawn on the ranks of the sorted data. The interval runs from the 6th to the 15th smallest value; the true median escapes on either side only if 5 or fewer points land on that side, which happens with probability 0.021 each, leaving coverage $1 - 2(0.021) = 0.959$.

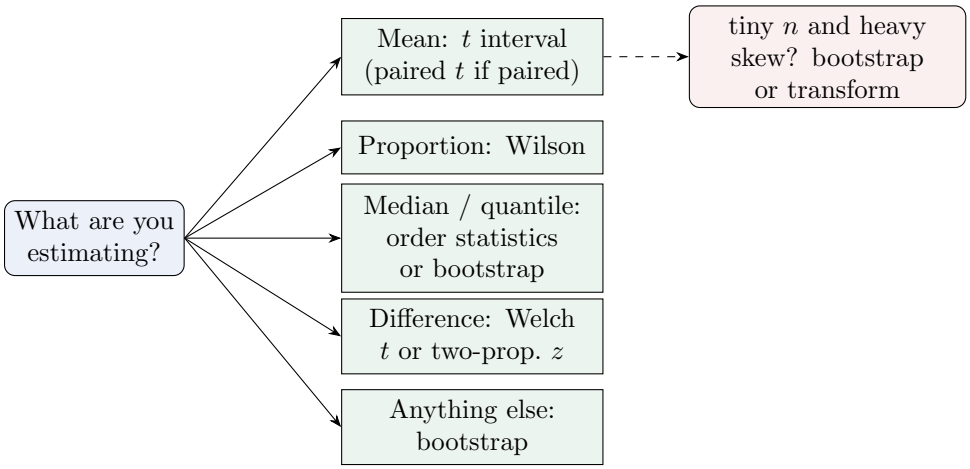


Figure 16.6: Choosing a confidence interval. Start from the parameter, not from the formula; the bootstrap is the fallback whenever no clean SE formula exists.

16.6 Choosing a test

Figure 16.7 is the decision tree from Chapters 9 and 10. Read it left to right. Green edges lead to the test for roughly normal data (or large n, where the CLT makes the mean normal anyway). Red edges lead to the rank-based or resampling alternative for skewed data, outliers, ordinal scores or small samples.

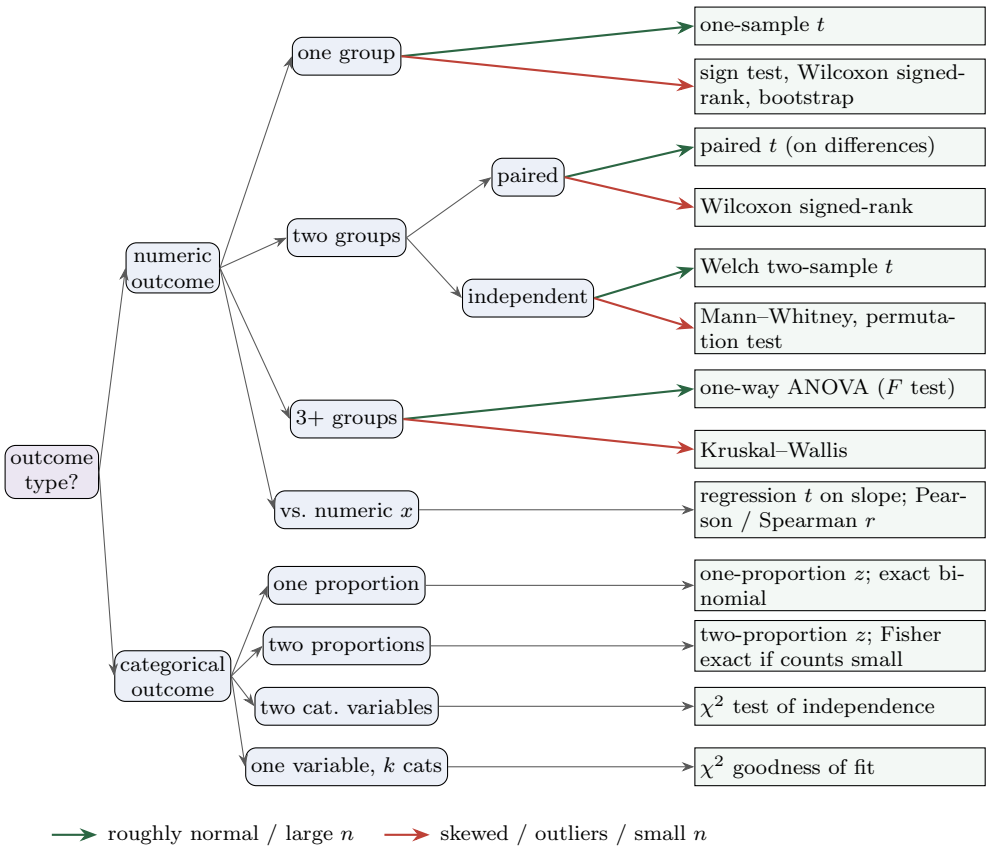


Figure 16.7: Test chooser. Grey edges are questions about the design; at the right, a green edge gives the test for roughly normal data and a red edge its robust alternative. Decide the outcome type, then the number of groups, then pairing, then whether a normal approximation is safe. Every test here has a matching confidence interval, which is usually more informative than the p-value.

Common trap

"Paired" is a property of the design, not of the data. Two columns of the same length are not paired unless each row is the same unit measured twice (same user, same server, same patient). Running an independent-samples test on paired data throws away the pairing and loses power; running a paired test on independent data is simply wrong.

Pause

Page load times (heavily right-skewed) for the same 40 pages, before and after a CDN change. Which leaf of Figure 16.7? (Numeric, two groups, paired. With $n = 40$ the mean difference is

close to normal, so a paired t is defensible; Wilcoxon signed-rank is the robust check.)

16.6.1 Test statistics at a glance

Test	Statistic	Reference distribution
One-sample t	$t = \dfrac{\bar{x} - \mu_0}{s/\sqrt{n}}$	t_{n-1}
Welch t	$t = \dfrac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_1^2/n_1 + s_2^2/n_2}}$	t_ν, Welch–Satterthwaite ν
One proportion	$z = \dfrac{\hat{p} - p_0}{\sqrt{p_0(1-p_0)/n}}$	$\mathcal{N}(0,1)$
Two proportions	$z = \dfrac{\hat{p}_1 - \hat{p}_2}{\sqrt{\hat{p}(1-\hat{p})(1/n_1 + 1/n_2)}}$, pooled $\hat{p}$	$\mathcal{N}(0,1)$
χ^2 (GOF or table)	$\sum \dfrac{(O-E)^2}{E}$, table: $E = \dfrac{\text{row} \times \text{col}}{\text{total}}$	χ^2_{k-1} or $\chi^2_{(r-1)(c-1)}$
Regression slope	$t = \hat{\beta}_1 / \operatorname{SE}(\hat{\beta}_1)$	t_{n-2}
ANOVA	$F = \dfrac{\text{between-group MS}}{\text{within-group MS}}$	$F_{k-1,\,n-k}$ (k groups, n total)

16.7 Critical values

16.7.1 Standard normal

Confidence level (two-sided)	80%	90%	95%	99%
$z^* = z_{\alpha/2}$	1.2816	1.6449	1.9600	2.5758
One-sided z_α at the same α	0.8416	1.2816	1.6449	2.3263

The one-sided row doubles as the power table: $z_{0.20} = 0.8416$ and $z_{0.10} = 1.2816$ are the values of z_β for 80% and 90% power.

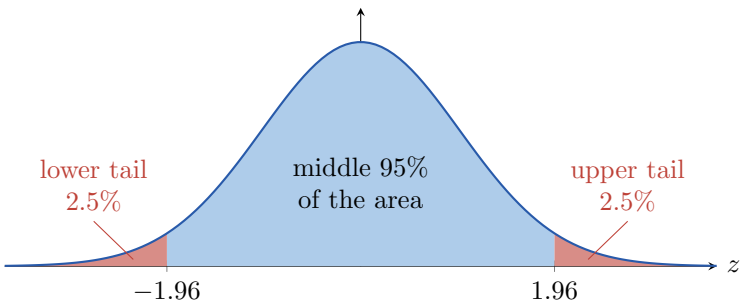


Figure 16.8: Where $z^* = 1.96$ comes from: the central 95% of $\mathcal{N}(0,1)$ lies between ± 1.96, leaving 2.5% in each tail.

16.7.2 Student's t (two-sided)

Rows are degrees of freedom ν; for a one-sample mean $\nu = n - 1$, for a regression slope $\nu = n - 2$. The ∞ row is the normal.

ν	90%	95%	99%	ν	90%	95%	99%
1	6.314	12.706	63.657	18	1.734	2.101	2.878
2	2.920	4.303	9.925	19	1.729	2.093	2.861
3	2.353	3.182	5.841	20	1.725	2.086	2.845
4	2.132	2.776	4.604	21	1.721	2.080	2.831
5	2.015	2.571	4.032	22	1.717	2.074	2.819
6	1.943	2.447	3.707	23	1.714	2.069	2.807
7	1.895	2.365	3.499	24	1.711	2.064	2.797
8	1.860	2.306	3.355	25	1.708	2.060	2.787
9	1.833	2.262	3.250	26	1.706	2.056	2.779
10	1.812	2.228	3.169	27	1.703	2.052	2.771
11	1.796	2.201	3.106	28	1.701	2.048	2.763
12	1.782	2.179	3.055	29	1.699	2.045	2.756
13	1.771	2.160	3.012	30	1.697	2.042	2.750
14	1.761	2.145	2.977	40	1.684	2.021	2.704
15	1.753	2.131	2.947	60	1.671	2.000	2.660
16	1.746	2.120	2.921	120	1.658	1.980	2.617
17	1.740	2.110	2.898	∞	1.645	1.960	2.576

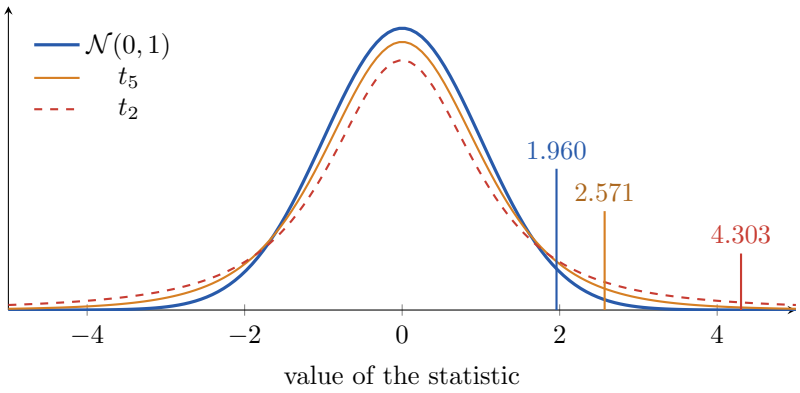


Figure 16.9: The t density has heavier tails than the normal when ν is small. Each vertical line is that curve's 95% critical value, leaving 2.5% of its own area to the right: the heavier the tails, the further out it sits (1.960 for the normal, 2.571 for t_5, 4.303 for t_2).

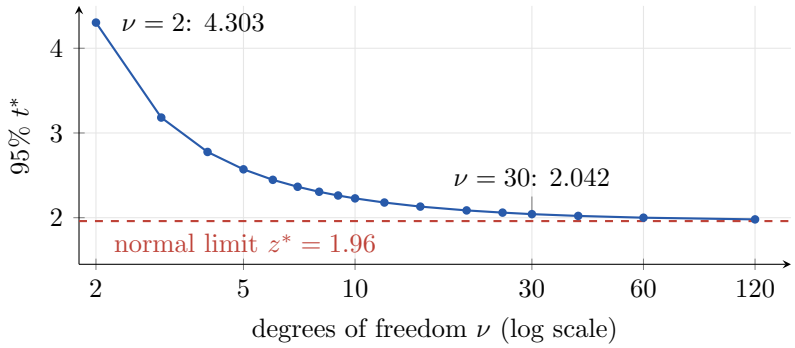


Figure 16.10: The 95% t critical value drops fast and then crawls toward 1.96. Past $\nu \approx 30$ the difference from z is under 5%.

16.7.3 Chi-square (upper tail)

Critical value c with $\mathbb{P}(\chi^2_k > c) = \alpha$. For goodness of fit with k categories use $k - 1$ df; for an $r \times c$ table use $(r - 1)(c - 1)$ df.

df	$\alpha = 0.10$	0.05	0.01	df	$\alpha = 0.10$	0.05	0.01
1	2.706	3.841	6.635	7	12.017	14.067	18.475
2	4.605	5.991	9.210	8	13.362	15.507	20.090
3	6.251	7.815	11.345	9	14.684	16.919	21.666
4	7.779	9.488	13.277	10	15.987	18.307	23.209
5	9.236	11.070	15.086	15	22.307	24.996	30.578
6	10.645	12.592	16.812	20	28.412	31.410	37.566

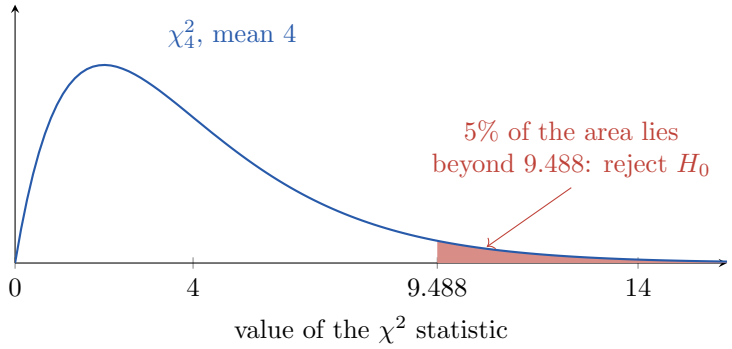


Figure 16.11: The χ^2_4 density. Its 95th percentile is 9.488: a statistic beyond that rejects at $\alpha = 0.05$. Chi-square tests are always upper-tail because large $(O - E)^2$ means poor fit.

16.8 Sample size and power

All formulas round *up*. Here E is the desired margin of error (half-width), δ the smallest difference worth detecting, $z_{\alpha/2} = 1.96$ for 95% (two-sided $\alpha = 0.05$), and $z_\beta = 0.8416$ for 80% power. In the two-sample rows n is the size of *each* group.

Goal	Formula	Example
Mean to $\pm E$	$n = \left(\frac{z_{\alpha/2}\,\sigma}{E}\right)^2$	$\sigma = 10$, $E = 2$: $96.04 \to 97$
Proportion to $\pm E$	$n = \frac{z_{\alpha/2}^2\,p(1-p)}{E^2}$ (worst case $p = \frac{1}{2}$)	$E = 0.03$: $1067.11 \to 1068$
Two means, power $1-\beta$	$n = \frac{2(z_{\alpha/2} + z_\beta)^2\sigma^2}{\delta^2}$	$\sigma = 10$, $\delta = 5$: $62.79 \to 63$ (t-exact: 64)
Two proportions ($q = 1-p$)	$n = \frac{(z_{\alpha/2} + z_\beta)^2\,[p_1q_1 + p_2q_2]}{\delta^2}$	$10\% \to 12\%$: $3838.1 \to 3839$

With $(1.96+0.8416)^2 = 7.849$, the two-proportion example is $7.849 \times (0.09+0.1056)/0.02^2 = 3838.1$. Detecting a lift from 10% to 12% takes almost four thousand users per arm. Because δ sits squared in the denominator, halving δ quadruples n.

Why it works

The two-means formula comes from requiring two things at once. Under H_0 the difference $\bar{X}_1 - \bar{X}_2$ has SE $\sigma\sqrt{2/n}$, and we reject when it exceeds $z_{\alpha/2}\,\sigma\sqrt{2/n}$. Under the alternative it is centred at δ with the same SE, and we want it above that threshold with probability $1-\beta$. That means the threshold must sit at least z_β standard errors below δ: $\delta - z_{\alpha/2}\sigma\sqrt{2/n} \geq z_\beta\,\sigma\sqrt{2/n}$. Divide by $\sigma\sqrt{2/n}$ and square: $n \geq 2(z_{\alpha/2} + z_\beta)^2\sigma^2/\delta^2$. (The chance of rejecting in the wrong tail is negligible and ignored.)

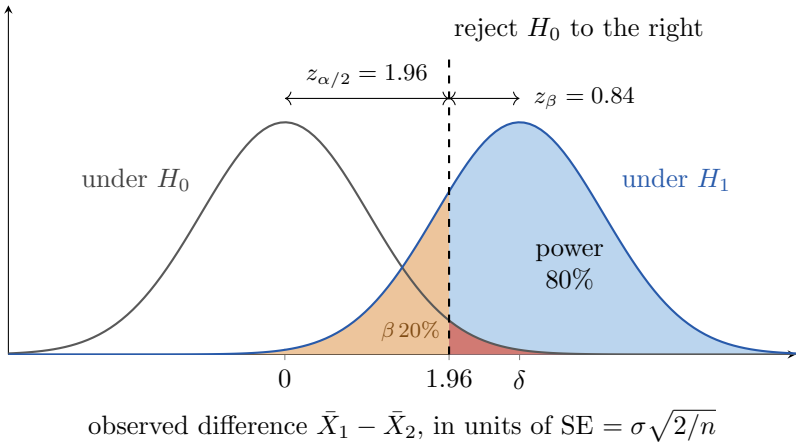


Figure 16.12: Where the sample-size formula comes from. The grey curve is the difference under H_0, the blue curve under the alternative centred at δ. Choosing n so that δ sits $1.96 + 0.84 = 2.80$ standard errors from zero puts 80% of the blue curve beyond the rejection line (power, blue) and 20% before it (missed effects, orange). The thin red sliver is the 2.5% of the grey curve beyond the line: false alarms in that tail.

16.9 Python one-liners

Imports assumed: `import numpy as np`, `from scipy import stats`, `import statsmodels.api as sm`. Statsmodels helpers live in `statsmodels.stats.proportion`, `.power` and `.multitest`.

Task	Code
mean, SD ($n-1$)	`x.mean(), x.std(ddof=1)`
quartiles, p99	`np.percentile(x, [25, 50, 75, 99])`
SE of mean	`stats.sem(x)`
z^*, t^*	`stats.norm.ppf(0.975), stats.t.ppf(0.975, df)`
t CI for mean	`stats.t.interval(0.95, len(x)-1, loc=x.mean(), scale=stats.sem(x))`
Wilson CI	`proportion_confint(k, n, method="wilson")`
bootstrap CI	`stats.bootstrap((x,), np.median).confidence_interval`
one-sample t	`stats.ttest_1samp(x, mu0)`
Welch t	`stats.ttest_ind(a, b, equal_var=False)`
paired t	`stats.ttest_rel(before, after)`
Mann–Whitney	`stats.mannwhitneyu(a, b)`
Wilcoxon signed-rank	`stats.wilcoxon(before, after)`
binomial test	`stats.binomtest(k, n, p=0.5)`
two-proportion z	`proportions_ztest([k1, k2], [n1, n2])`
χ^2 on a table	`stats.chi2_contingency(table)`
Fisher exact	`stats.fisher_exact(table)`
ANOVA, Kruskal	`stats.f_oneway(a, b, c), stats.kruskal(a, b, c)`
correlation	`stats.pearsonr(x, y), stats.spearmanr(x, y)`
linear regression	`sm.OLS(y, sm.add_constant(X)).fit().summary()`
logistic regression	`sm.Logit(y, sm.add_constant(X)).fit().summary()`
sample size ($d = 0.5$)	`TTestIndPower().solve_power(effect_size=0.5, alpha=0.05, power=0.8)`
FDR control	`multipletests(pvals, method="fdr_bh")`

The power call returns 63.77, so 64 per group, matching the t-exact entry in the sample-size table. All scipy tests return an object with `.statistic` and `.pvalue`.

16.10 Glossary

Term	Definition
Alternative hypothesis	The claim H_1 that the data are tested for evidence of, e.g. $\mu_B \neq \mu_A$.
ANOVA	A test of whether three or more group means are all equal, via a ratio of variances.
Bias	$\mathbb{E}[\hat{\theta}] - \theta$: how far an estimator is off on average.
Bootstrap	Estimating a sampling distribution by resampling the data with replacement.
Box plot	A plot of median, quartiles, whiskers and outliers.
Central limit theorem	Sums and means of many independent values are approximately normal.
Confidence interval	A range produced by a procedure that captures the true parameter in a stated fraction of repeated samples.
Confidence level	That long-run capture fraction, e.g. 95%.

Term	Definition
Confounder	A variable that affects both treatment and outcome, creating spurious association.
Correlation	A unit-free measure in $[-1, 1]$ of linear association between two variables.
Covariance	Average product of deviations from the means; sign gives direction of association.
Critical value	The cutoff from a reference distribution, e.g. 1.96, used for intervals and tests.
Degrees of freedom	The number of independent pieces of information left after estimating parameters.
Effect size	The size of a difference in meaningful units, or standardised (Cohen's d).
Estimator	A rule that turns data into a guess of a parameter; the guess is the estimate.
Expectation	The probability-weighted average of a random variable.
False discovery rate	Expected fraction of rejected nulls that are actually true.
Heteroscedasticity	Residual spread that changes with x; breaks the usual regression SEs.
Histogram	Bar chart of counts of values falling into bins.
Independence	Knowing one variable's value tells you nothing about another's.
Interquartile range	$Q_3 - Q_1$: the width of the middle 50% of the data.
Law of large numbers	The sample mean converges to the population mean as n grows.
Least squares	Choosing parameters that minimise the sum of squared residuals.
Likelihood	The probability (or density) of the observed data, viewed as a function of the parameters.
Margin of error	Half the width of a symmetric confidence interval.
Maximum likelihood	Estimating parameters by the values that maximise the likelihood.
Median	The middle value: half the data below, half above.
Null hypothesis	The default claim H_0, usually "no effect", assumed when computing a p-value.
Outlier	A point far from the bulk of the data, e.g. beyond the Tukey fences.
p-value	Probability, assuming H_0, of a statistic at least as extreme as the one observed.
Parameter	A fixed, unknown number describing the population, such as μ or p.
Percentile	The value below which a given percentage of data falls; p99 is the 99th.
Power	Probability that a test rejects H_0 when a specified alternative is true.
Regression	Modelling the mean of an outcome as a function of predictors.
Residual	Observed minus fitted value, $e_i = y_i - \hat{y}_i$.
Sampling distribution	The distribution of a statistic over all possible samples of size n.
Standard deviation	Square root of the variance; typical distance from the mean.
Standard error	Standard deviation of an estimator's sampling distribution.
Statistic	Any number computed from the sample, such as $\bar{x}$ or s.
Type I / II error	Rejecting a true H_0 / failing to reject a false H_0.